Operating Systems Theory

Processes, Memory, Concurrency, and the Abstractions That Run the World

Achraf SOLTANI

2026

Contents

Preface **1**
What This Book Covers . . . 1
The Programmer's Perspective . . . 2
Callout Boxes Used in This Book . . . 2
Prerequisites . . . 2
Acknowledgements . . . 3

Chapter 1: What Is an Operating System? **4**
The Three Roles of an Operating System . . . 4
Role 1: Resource Manager . . . 4
Role 2: Abstraction Provider . . . 6
Role 3: Protection Boundary . . . 7
The Interplay of Roles . . . 8
A Formal View: The OS as a State Machine . . . 8
Historical Evolution . . . 9
Era 1: No Operating System (1940s–1950s) . . . 9
Era 2: Batch Systems (Late 1950s–1960s) . . . 10
Era 3: Multiprogramming (1960s–1970s) . . . 11
Era 4: Time-Sharing (1970s–1980s) . . . 12
Era 5: Unix and the Modern Era (1970s–Present) . . . 13
Timeline Summary . . . 14
Kernel Mode vs User Mode . . . 14
Kernel Mode (Supervisor Mode) . . . 15
User Mode . . . 15
Mode Transitions . . . 15
The System Call as the OS API . . . 17
OS Structure Overview . . . 19
The Process Subsystem . . . 19
The Memory Subsystem . . . 20
The File Subsystem . . . 21
The I/O Subsystem . . . 22
Subsystem Interactions . . . 23
POSIX and the System Call Interface . . . 24
What is POSIX? . . . 24
Key POSIX System Calls . . . 24
The fork-exec Model . . . 26
File Descriptors: The Universal Handle . . . 28
Beyond POSIX: Linux-Specific Extensions . . . 29
The System Call Interface in Practice . . . 30
Tracing System Calls with strace . . . 30
System Call Overhead . . . 32

The Cost-Benefit Calculus . . . 33
The Kernel's Execution Model . . . 34
Design Goals and Trade-offs . . . 36
Correctness vs Performance . . . 36
Generality vs Specialisation . . . 36
Security vs Usability . . . 36
Compatibility vs Innovation . . . 37
Summary . . . 37
Exercises . . . 38
Exercise 1.1: Role Classification . . . 38
Exercise 1.2: State Machine Model . . . 38
Exercise 1.3: System Call Cost Analysis . . . 39
Exercise 1.4: fork-exec Implementation . . . 39
Exercise 1.5: Historical Analysis . . . 39
Exercise 1.6: POSIX Portability . . . 40
Exercise 1.7: Protection Boundary Analysis . . . 40

Chapter 2: Architectural Patterns **42**
The Fundamental Question: What Belongs in the Kernel? . . . 42
Monolithic Kernels . . . 43
Architecture . . . 43
Linux: The Canonical Monolithic Kernel . . . 44
Loadable Kernel Modules . . . 46
Advantages of Monolithic Kernels . . . 47
Disadvantages of Monolithic Kernels . . . 48
Microkernels . . . 49
Architecture . . . 49
The IPC Problem . . . 50
First-Generation Microkernels: Mach . . . 51
L4: High-Performance Microkernels . . . 52
seL4: The Formally Verified Microkernel . . . 52
Minix 3: Reliability Through Isolation . . . 54
Advantages of Microkernels . . . 54
Disadvantages of Microkernels . . . 55
Hybrid Kernels . . . 55
Architecture . . . 55
Windows NT . . . 56
macOS / XNU . . . 58
Exokernels . . . 59
Architecture . . . 59
Secure Bindings . . . 60
Library Operating Systems . . . 61
Performance Results and Legacy . . . 61
Unikernels . . . 62
Architecture . . . 62
How Unikernels Work . . . 63
MirageOS . . . 64
IncludeOS . . . 64

Use Cases . . . 65
Advantages and Disadvantages . . . 65
Comparative Analysis . . . 66
Performance Comparison . . . 66
Security Comparison . . . 67
Complexity and Development Effort . . . 67
A Formal Comparison: TCB and Failure Probability . . . 68
Case Study: The Tanenbaum-Torvalds Debate . . . 69
Emerging Patterns . . . 70
Multi-kernel (Barrelfish) . . . 70
OS as a Library (Dune, gVisor) . . . 71
eBPF: Programmable Kernel Extensions . . . 71
Summary . . . 73
Exercises . . . 73
Exercise 2.1: Architecture Classification . . . 73
Exercise 2.2: IPC Overhead Analysis . . . 74
Exercise 2.3: TCB Size and Verification . . . 74
Exercise 2.4: Kernel Module Implementation . . . 75
Exercise 2.5: Exokernel Design . . . 75
Exercise 2.6: Unikernel Security Analysis . . . 76
Exercise 2.7: Architecture Trade-off Matrix . . . 76

Chapter 3: Hardware-Software Interface **77**
CPU Privilege Rings . . . 77
The Need for Hardware-Enforced Privilege . . . 77
x86 Protection Rings . . . 78
The Global Descriptor Table (GDT) . . . 80
ARM Exception Levels . . . 80
RISC-V Privilege Modes . . . 82
Interrupts . . . 83
What is an Interrupt? . . . 83
Polling vs Interrupts . . . 84
Hardware Interrupt Mechanism . . . 85
The Interrupt Descriptor Table (IDT) . . . 86
Interrupt Handling Flow . . . 87
Message Signalled Interrupts (MSI/MSI-X) . . . 88
Software Interrupts (Traps) . . . 89
Top-Half and Bottom-Half Processing . . . 90
Exceptions: Faults, Traps, and Aborts . . . 92
Exception Classification . . . 92
x86 Exception Vectors . . . 93
The Page Fault Handler in Detail . . . 94
Double Faults and Triple Faults . . . 97
Direct Memory Access (DMA) . . . 98
The Problem: CPU-Mediated I/O . . . 98
How DMA Works . . . 99
DMA and Cache Coherence . . . 100
Scatter-Gather DMA . . . 101

IOMMUs: DMA with Protection . . . 102
I/O Buses and Device Communication . . . 103
The Modern Bus Hierarchy . . . 103
PCI Express (PCIe) . . . 104
NVMe: A Modern Storage Interface . . . 105
USB Architecture . . . 106
Memory-Mapped I/O vs Port-Mapped I/O . . . 107
Two Approaches to Device Communication . . . 107
Comparison . . . 108
The `volatile` Keyword and Memory Ordering . . . 109
Timer Hardware and Preemptive Scheduling . . . 110
The Role of the Timer . . . 110
x86 Timer Hardware . . . 110
The Tick Rate and Tickless Kernels . . . 111
Implementing a Timer Interrupt Handler . . . 112
Preemption Latency . . . 114
Real-Time Scheduling Constraints . . . 115
Summary . . . 116
Exercises . . . 116
Exercise 3.1: Privilege Level Analysis . . . 116
Exercise 3.2: Interrupt Latency Measurement . . . 117
Exercise 3.3: DMA Buffer Management . . . 117
Exercise 3.4: Page Fault Handler Design . . . 118
Exercise 3.5: NVMe vs AHCI Quantitative Comparison . . . 118
Exercise 3.6: Timer Analysis . . . 119
Exercise 3.7: Writing an Interrupt Handler . . . 119

Chapter 4: Processes and Threads **120**
4.1 Program vs Process . . . 120
4.2 Process States . . . 121
4.2.1 Zombie and Orphan Processes . . . 123
4.3 Process Memory Layout . . . 124
4.4 The Process Control Block . . . 126
4.4 Process Creation . . . 128
4.4.1 The Unix Model: fork() and exec() . . . 128
4.4.2 The exec() Family . . . 129
4.4.3 The Windows Model: CreateProcess() . . . 130
4.4.4 Process Hierarchies . . . 131
4.5.5 Process Groups and Sessions . . . 131
4.5 Process Termination . . . 132
4.5.1 posix_spawn(): A Modern Alternative . . . 133
4.5.2 Process Resource Limits . . . 134
4.6 Context Switching . . . 135
4.6.1 The Context Switch Mechanism . . . 135
4.6.2 The Cost of Context Switching . . . 136
4.6.3 Hardware Support for Context Switching . . . 137
4.7 Threads . . . 139
4.7.1 Why Threads? . . . 139

4.7.2 Thread State . . . 140
4.8 Thread Models . . . 140
4.8.1 Many-to-One (M:1) — User-Level Threads . . . 140
4.8.2 One-to-One (1:1) — Kernel Threads . . . 141
4.8.3 Many-to-Many (M:N) — Hybrid Model . . . 141
4.9 Green Threads and Goroutines . . . 142
4.9.1 What Makes Goroutines Different . . . 142
4.9.2 The GMP Model . . . 144
4.9.3 Goroutine Preemption . . . 145
4.10 Linux clone() and Thread Creation . . . 146
4.11 POSIX Threads (pthreads) . . . 148
4.11.1 Thread Lifecycle . . . 148
4.11.2 Thread Attributes . . . 150
4.12 Thread-Local Storage . . . 150
4.12.1 TLS in C . . . 151
4.12.2 TLS Implementation . . . 152
4.12.3 TLS in Go . . . 152
4.13 Comparing Threading Models in Practice . . . 153
4.14 Kernel Threads . . . 155
4.15 Parallelism and Amdahl's Law . . . 155
4.15.1 Data Parallelism vs Task Parallelism . . . 157
4.15.2 False Sharing . . . 158
4.15.3 Thread Safety Levels . . . 160
4.16 Process and Thread Lifecycle Summary . . . 161
Exercises . . . 162

Chapter 5: CPU Scheduling **164**
5.1 Scheduling Criteria . . . 164
5.1.1 CPU Bursts and I/O Bursts . . . 165
5.2 Preemptive vs Non-Preemptive Scheduling . . . 165
5.2.1 The Dispatcher . . . 166
5.3 First-Come, First-Served (FCFS) . . . 166
5.3.1 FCFS with Staggered Arrivals . . . 167
5.3.2 Implementation . . . 168
5.4 Shortest Job First (SJF) . . . 169
5.4.1 The Prediction Problem . . . 170
5.4.2 Starvation in SJF . . . 171
5.5 Shortest Remaining Time First (SRTF) . . . 172
5.6 Round-Robin (RR) Scheduling . . . 173
5.6.1 Quantum Analysis . . . 174
5.6.2 Turnaround Time Analysis . . . 175
5.6.3 Implementation . . . 176
5.7 Priority Scheduling . . . 178
5.7.1 Priority Inversion . . . 178
5.7.2 Priority Inheritance and Priority Ceiling . . . 179
5.8 Lottery and Stride Scheduling . . . 180
5.8.1 Lottery Scheduling . . . 180
5.8.2 Stride Scheduling . . . 181

5.9 Multi-Level Feedback Queue (MLFQ) . . . 182
5.8.1 Problems with Basic MLFQ . . . 182
5.8.2 MLFQ Refinements . . . 183
5.9.3 Detailed MLFQ Trace . . . 183
5.9.4 MLFQ in Real Systems . . . 184
5.10 Linux Scheduler History . . . 185
5.10.1 The O(n) Scheduler (Linux 2.4) . . . 185
5.10.2 The O(1) Scheduler (Linux 2.6.0–2.6.22) . . . 185
5.10.3 The Completely Fair Scheduler (Linux 2.6.23+) . . . 185
5.10.4 Virtual Runtime . . . 186
5.10.5 Nice Values and Weights . . . 186
5.10.6 The Red-Black Tree . . . 187
5.10.7 Time Slice Computation . . . 187
5.10.8 Sleeper Fairness . . . 188
5.10.9 Group Scheduling (CFS Bandwidth Control) . . . 189
5.10.10 EEVDF: CFS's Successor . . . 189
5.11 Real-Time Scheduling . . . 190
5.11.1 Real-Time Task Model . . . 190
5.11.2 Rate Monotonic Scheduling (RMS) . . . 190
5.11.3 Earliest Deadline First (EDF) . . . 192
5.11.4 Linux SCHED_DEADLINE . . . 193
5.11.5 Windows Thread Scheduling . . . 194
5.12 Multiprocessor Scheduling . . . 195
5.12.1 Processor Affinity . . . 195
5.12.2 Load Balancing . . . 196
5.12.3 NUMA-Aware Scheduling . . . 197
5.12.4 Gang Scheduling . . . 197
5.12.5 Heterogeneous Multiprocessor Scheduling . . . 198
5.13 Scheduling in Practice: A Comparison . . . 198
5.14 Scheduling Algorithm Evaluation . . . 200
5.14.1 Deterministic Modelling . . . 200
5.14.2 Queueing Models . . . 201
5.14.3 Simulation . . . 202
5.14.4 Implementation and Measurement . . . 203
Exercises . . . 204

Chapter 6: Inter-Process Communication 206
6.1 Taxonomy of IPC Mechanisms . . . 206
6.2 Shared Memory . . . 207
6.2.1 POSIX Shared Memory . . . 207
6.2.2 mmap: Memory-Mapped Files . . . 210
6.2.3 Race Conditions in Shared Memory . . . 211
6.3 Pipes . . . 211
6.3.1 Anonymous Pipes . . . 212
6.3.2 Pipe Semantics . . . 213
6.3.3 Pipe Buffer Sizes . . . 213
6.3.4 Named Pipes (FIFOs) . . . 214
6.4 Signals . . . 215

6.4.1 Signal Categories . . . 215
6.4.2 Signal Handling . . . 216
6.4.3 Signal Masks . . . 218
6.4.4 Real-Time Signals . . . 219
6.5 Message Passing . . . 220
6.5.1 Synchronous vs Asynchronous Message Passing . . . 220
6.5.2 POSIX Message Queues . . . 221
6.5.3 Mailboxes and Ports . . . 223
6.6 Unix Domain Sockets . . . 224
6.6.1 Stream Example . . . 224
6.6.2 Unix Domain Sockets vs TCP for Local IPC . . . 226
6.6.3 Passing File Descriptors . . . 226
6.7 D-Bus and Higher-Level IPC Frameworks . . . 227
6.7.1 D-Bus . . . 227
6.7.2 Alternatives to D-Bus . . . 229
6.8 Remote Procedure Call (RPC) . . . 229
6.8.1 RPC Architecture . . . 229
6.8.2 Marshalling and Wire Formats . . . 230
6.8.3 gRPC . . . 230
6.8.4 RPC Semantics . . . 233
6.9 Comparing IPC Mechanisms . . . 234
6.10 io_uring: Asynchronous I/O for Linux . . . 237
6.10.1 io_uring Performance . . . 238
6.11 IPC Design Patterns . . . 239
6.11.1 Producer-Consumer . . . 239
6.11.2 Client-Server . . . 240
6.11.3 Publish-Subscribe . . . 240
6.12 Security Considerations . . . 240
Exercises . . . 241

Chapter 7: Synchronisation Primitives 243
7.1 The Critical Section Problem . . . 243
7.1.1 Motivation . . . 243
7.1.2 Correctness Requirements . . . 244
7.1.3 The Difficulty of the Problem . . . 245
7.1.4 Naive Attempts . . . 245
7.1.5 The N-Process Generalisation . . . 246
7.2 Peterson’s Algorithm . . . 247
7.2.1 The Algorithm . . . 247
7.2.2 Correctness Proof . . . 248
7.2.3 Limitations . . . 249
7.3 Hardware Support for Synchronisation . . . 249
7.3.1 Test-and-Set . . . 250
7.3.1b Test-and-Test-and-Set (TTAS) . . . 250
7.3.2 Compare-and-Swap (CAS) . . . 252
7.3.2b The Consensus Hierarchy . . . 253
7.3.3 Fetch-and-Add . . . 253
7.3.4 LL/SC: Load-Linked and Store-Conditional . . . 255

7.4 Mutexes . . . 256
7.4.1 Definition and Semantics . . . 256
7.4.2 Implementation . . . 256
7.4.3 Recursive Mutexes . . . 258
7.4.4 The Futex Mechanism in Detail . . . 258
7.5 Semaphores . . . 260
7.5.1 Definition . . . 260
7.5.1b Semaphore Implementation . . . 260
7.5.2 Producer-Consumer Problem . . . 261
7.5.3 Readers-Writers Problem . . . 263
7.5.4 Dining Philosophers Problem . . . 266
7.6 Monitors and Condition Variables . . . 268
7.6.1 The Problem with Semaphores . . . 268
7.6.2 Monitor Definition . . . 268
7.6.3 Condition Variables . . . 268
7.6.4 Mesa vs Hoare Semantics . . . 270
7.7 Spinlocks vs Sleeping Locks . . . 272
7.7.1 When to Use Which . . . 272
7.7.2 Adaptive Spinning . . . 273
7.7.3 Exponential Backoff . . . 273
7.7.4 Cost Analysis . . . 274
7.8 Read-Write Locks . . . 274
7.8.1 Definition . . . 274
7.8.2 Implementation . . . 275
7.8.3 Sequence Locks (Seqlocks) . . . 276
7.8.4 Choosing Between Lock Types . . . 278
7.8.5 Barriers . . . 278
7.9 Programmer's Perspective: Go's Synchronisation Primitives . . . 281
7.10 Summary . . . 282
Exercises . . . 283

Chapter 8: Deadlock **285**
8.1 System Model . . . 285
8.1.1 Resources and Processes . . . 285
8.1.2 Formal State Description . . . 286
8.1.3 State Transitions . . . 287
8.2 Deadlock Characterisation . . . 288
8.2.1 Coffman's Four Conditions . . . 288
8.2.2 Resource Allocation Graph . . . 290
8.2.3 Graph Reduction . . . 291
8.3 Deadlock Prevention . . . 291
8.3.1 Breaking Mutual Exclusion . . . 292
8.3.2 Breaking Hold and Wait . . . 292
8.3.3 Breaking No Preemption . . . 293
8.3.4 Breaking Circular Wait . . . 294
8.3.5 Comparison of Prevention Strategies . . . 295
8.4 Deadlock Avoidance: The Banker's Algorithm . . . 299
8.4.1 Safe and Unsafe States . . . 299

8.4.2 The Banker's Algorithm . . . 300
8.4.3 Worked Example 1 . . . 303
8.4.4 Worked Example 2: Unsafe State . . . 304
8.4.5 Proof of Safety . . . 305
8.4.6 Complexity and Limitations . . . 305
8.4.7 The Single-Resource Banker's Algorithm . . . 306
8.5 Deadlock Detection . . . 307
8.5.1 Wait-For Graph . . . 307
8.5.2 Detection Algorithm for Multiple-Instance Resources . . . 309
8.5.3 When to Run Detection . . . 312
8.6 Deadlock Recovery . . . 313
8.6.1 Process Termination . . . 313
8.6.2 Resource Preemption . . . 313
8.6.3 Checkpointing for Rollback . . . 314
8.7 Livelock and Starvation . . . 316
8.7.1 Livelock . . . 316
8.7.2 Starvation . . . 319
8.7.3 Formal Relationships . . . 320
8.8 Deadlock in Distributed Systems . . . 320
8.8.1 The Distributed Deadlock Problem . . . 320
8.8.2 Distributed Detection Algorithms . . . 321
8.8.3 The Phantom Deadlock Problem . . . 322
8.9 Practical Strategies in Real Systems . . . 323
8.9.1 The Ostrich Algorithm . . . 323
8.9.2 Combined Approaches . . . 324
8.10 Summary . . . 327
Exercises . . . 328

Chapter 9: Concurrent Data Structures 331
9.1 Lock-Based Data Structures . . . 331
9.1.1 Coarse-Grained Locking . . . 331
9.1.2 Fine-Grained Locking . . . 333
9.1.3 Hand-Over-Hand Locking (Lock Coupling) . . . 334
9.1.4 Optimistic Locking . . . 337
9.2 Lock-Free Programming . . . 338
9.2.1 Progress Guarantees . . . 338
9.2.2 CAS Loops . . . 338
9.2.2b Correctness of the Lock-Free Stack . . . 340
9.2.3 The ABA Problem . . . 340
9.3 Wait-Free Data Structures . . . 342
9.3.1 Herlihy's Universal Construction . . . 342
9.3.2 Practical Considerations . . . 344
9.3.3 The Price of Wait-Freedom . . . 345
9.4 The Michael-Scott Lock-Free Queue . . . 345
9.4.1 Structure . . . 345
9.4.2 Enqueue . . . 346
9.4.3 Dequeue . . . 347
9.5 Memory Reclamation . . . 348

9.5.1 The Problem . . . 348
9.5.2 Hazard Pointers . . . 349
9.5.3 Epoch-Based Reclamation . . . 351
9.5.4 Choosing a Reclamation Strategy . . . 354
9.6 Memory Ordering . . . 355
9.6.1 The Need for Memory Ordering . . . 355
9.6.2 Sequential Consistency . . . 355
9.6.3 Acquire-Release Semantics . . . 356
9.6.3b Hardware Memory Models . . . 357
9.6.4 Relaxed Ordering . . . 358
9.6.5 Memory Order Hierarchy . . . 358
9.7 The C11/C++11 Memory Model . . . 361
9.7.1 Formalisation . . . 361
9.7.2 Modification Order and Coherence . . . 361
9.7.3 Practical Implications . . . 362
9.8 Linearisability . . . 363
9.8.1 Definition . . . 363
9.8.2 Linearisation Points . . . 364
9.8.3 Composability . . . 364
9.8.4 Verifying Linearisability . . . 365
9.8.5 Alternatives to Linearisability . . . 365
9.8.6 Practical Correctness Testing . . . 366
9.9 Programmer's Perspective: Go's Lock-Free Channels . . . 369
9.10 Summary . . . 370
Exercises . . . 371

Chapter 10: Memory Management Fundamentals 373
10.1 Address Spaces . . . 373
10.1.1 Logical and Physical Addresses . . . 373
10.1.2 Address Space Size . . . 374
10.1.3 The Address Space as an Abstraction . . . 375
10.2 Address Binding . . . 375
10.2.1 Compile-Time Binding . . . 376
10.2.2 Load-Time Binding . . . 376
10.2.3 Execution-Time Binding . . . 377
10.2.4 Dynamic Loading and Linking . . . 378
10.3 Base and Limit Registers . . . 379
10.3.1 Mechanism . . . 379
10.3.2 Context Switching . . . 380
10.3.3 Worked Examples . . . 380
10.3.4 Relocation with Base-Limit . . . 381
10.3.5 Limitations of Base-Limit . . . 381
10.4 Contiguous Memory Allocation . . . 382
10.4.1 Fixed Partitions . . . 382
10.4.2 Variable Partitions . . . 383
10.4.3 Free List Data Structures . . . 384
10.4.4 Coalescing Policies . . . 385
10.5 Memory Allocation Strategies . . . 386

10.5.1 First-Fit . . . 386
10.5.2 Next-Fit . . . 386
10.5.3 Best-Fit . . . 387
10.5.4 Worst-Fit . . . 387
10.5.5 Comparative Analysis . . . 387
10.5.6 The Fifty-Percent Rule . . . 388
10.5.7 Formal Analysis of Allocation Strategies . . . 390
10.6 Fragmentation . . . 390
10.6.1 Internal Fragmentation . . . 390
10.6.2 External Fragmentation . . . 391
10.6.3 Quantifying External Fragmentation . . . 391
10.6.4 Compaction . . . 392
10.7 Swapping . . . 394
10.7.1 Basic Swapping . . . 394
10.7.2 Backing Store . . . 395
10.7.3 Cost of Swapping . . . 395
10.7.4 Swapping Policy . . . 396
10.7.5 Partial Swapping and Paging . . . 397
10.8 The Memory Hierarchy Perspective . . . 397
10.9 Memory Layout of a Process . . . 398
10.10 Protection and Sharing . . . 402
10.10.1 Protection Mechanisms . . . 402
10.10.2 Sharing . . . 402
10.10.3 Memory Protection Keys (MPK) . . . 403
10.11 Segmentation . . . 403
10.12 Paged Segmentation . . . 405
10.13 Historical Context . . . 406
10.14 Summary . . . 407

Chapter 11: Virtual Memory 409
11.1 Demand Paging . . . 409
11.1.1 The Concept . . . 409
11.1.2 Page Faults . . . 410
11.1.3 Types of Page Faults . . . 411
11.1.4 Performance of Demand Paging . . . 412
11.1.5 Pure Demand Paging vs Prepaging . . . 413
11.2 Page Tables . . . 414
11.2.1 Single-Level Page Table . . . 415
11.2.2 The Size Problem . . . 416
11.2.3 Multi-Level Page Tables . . . 417
11.2.4 Four-Level Page Table (x86-64) . . . 418
11.2.5 Five-Level Page Tables . . . 420
11.2.6 Inverted Page Tables . . . 420
11.3 Translation Lookaside Buffer (TLB) . . . 422
11.3.1 The Performance Problem . . . 422
11.3.2 TLB Operation . . . 422
11.3.2a TLB Reach . . . 423
11.3.3 TLB Hit Ratio and EAT . . . 424

11.3.4 TLB Miss Handling . . . 425
11.3.5 Address Space Identifiers (ASID) . . . 426
11.4 Page Table Entry Format . . . 427
11.4.1 x86-64 PTE Format . . . 427
11.4.2 The Present Bit . . . 428
11.4.3 The Dirty Bit . . . 428
11.4.4 The Accessed Bit . . . 429
11.4.5 The NX (No-Execute) Bit . . . 429
11.5 Shared Pages . . . 429
11.5.1 Shared Libraries . . . 429
11.5.2 Copy-on-Write (CoW) . . . 430
11.6 Page Size Trade-offs . . . 434
11.6.1 Arguments for Small Pages . . . 434
11.6.2 Arguments for Large Pages . . . 434
11.6.3 Quantitative Analysis . . . 434
11.6.4 Common Page Sizes in Practice . . . 435
11.7 Huge Pages and TLB Pressure . . . 436
11.8 Virtual Memory in Kernel Space . . . 437
11.9 Summary . . . 437

Chapter 12: Page Replacement and Thrashing 440
12.1 The Page Replacement Problem . . . 440
12.1.1 Formulation . . . 440
12.1.2 Evaluation Metric . . . 441
12.2 Belady's Optimal Algorithm (OPT) . . . 441
12.2.1 Definition . . . 441
12.2.2 Proof of Optimality . . . 442
12.2.3 Why OPT is Impractical . . . 443
12.2.4 OPT as an Offline Algorithm . . . 443
12.3 FIFO (First-In, First-Out) . . . 444
12.3.1 Analysis of FIFO . . . 445
12.4 Belady's Anomaly . . . 447
12.4.1 The Counter-Example . . . 447
12.4.2 Stack Algorithms and Immunity . . . 449
12.5 LRU (Least Recently Used) . . . 450
12.5.1 Definition . . . 450
12.5.2 Implementation Challenges . . . 451
12.5.3 LRU and the Stack Distance . . . 452
12.6 LRU Approximations . . . 453
12.6.1 The Clock Algorithm (Second-Chance) . . . 453
12.6.2 Enhanced Second-Chance (NRU) . . . 454
12.6.3 Linux's Two-List LRU . . . 455
12.6.4 Multi-Generation LRU (MGLRU) . . . 456
12.7 Counting-Based Algorithms . . . 457
12.7.1 LFU (Least Frequently Used) . . . 457
12.7.2 MFU (Most Frequently Used) . . . 457
12.8 Frame Allocation . . . 457
12.8.1 Minimum Number of Frames . . . 457

12.8.2 Equal Allocation . . . 458
12.8.3 Proportional Allocation . . . 458
12.8.4 Global vs Local Replacement . . . 458
12.9 Working Set Model . . . 460
12.9.1 Locality and the Working Set . . . 460
12.9.2 The Working Set Model . . . 461
12.9.3 Choosing Δ . . . 461
12.9.4 Implementing the Working Set Model . . . 461
12.9.5 Locality and Phase Behaviour . . . 462
12.10 Thrashing . . . 463
12.10.1 Definition and Cause . . . 463
12.10.2 Detecting Thrashing . . . 464
12.10.3 Preventing Thrashing . . . 464
12.10a Quantifying Thrashing . . . 467
12.10a.1 Page Fault Rate Model . . . 467
12.10a.2 The Lifetime Curve . . . 467
12.10b Practical Thrashing Prevention on Linux . . . 468
12.10b.1 Watermarks . . . 468
12.10b.2 Memory Cgroups . . . 469
12.10b.3 Swap Limits . . . 469
12.11 Comparative Summary of Algorithms . . . 470
12.12 Summary . . . 470

Chapter 13: Advanced Memory Topics 473
13.1 Copy-on-Write . . . 473
13.1.1 The fork() Problem . . . 473
13.1.2 Reference Counting . . . 474
13.1.3 CoW Fault Handling . . . 474
13.1.4 Cost Analysis . . . 475
13.1.5 CoW for Zero Pages . . . 475
13.2 Memory-Mapped Files . . . 476
13.2.1 Concept . . . 476
13.2.2 mmap() Semantics . . . 476
13.2.3 Private vs Shared Mappings . . . 477
13.2.4 msync() . . . 477
13.2.5 Performance Advantages . . . 478
13.2.6 Memory-Mapped IPC . . . 479
13.3 Kernel Memory Allocation . . . 482
13.3.1 The Buddy System . . . 482
13.3.1a Buddy System: Deallocation and Coalescing . . . 483
13.3.2 The Slab Allocator . . . 485
13.3.3 SLUB: The Simplified Slab . . . 486
13.3.4 kmalloc and the Size-Class Hierarchy . . . 486
13.4 NUMA: Non-Uniform Memory Access . . . 487
13.4.1 NUMA Architecture . . . 487
13.4.2 NUMA Ratio . . . 487
13.4.3 NUMA-Aware Allocation . . . 488
13.4.4 NUMA and Page Migration . . . 488

13.4.5 NUMA Topology Discovery . . . 489
13.5 Huge Pages . . . 491
13.5.1 TLB Pressure Revisited . . . 491
13.5.2 Explicit Huge Pages (hugetlbfs) . . . 492
13.5.3 Transparent Huge Pages (THP) . . . 492
13.5.4 THP Drawbacks . . . 493
13.6 Memory Compression . . . 493
13.6.1 Motivation . . . 493
13.6.2 zswap . . . 494
13.6.3 zram . . . 494
13.6.4 Compression Algorithms . . . 495
13.6.5 When to Use Memory Compression . . . 496
13.7 Address Space Layout Randomisation (ASLR) . . . 496
13.7.1 The Security Problem . . . 496
13.7.2 ASLR Mechanism . . . 496
13.7.3 Entropy . . . 497
13.7.4 Checking ASLR . . . 498
13.7.5 ASLR Limitations . . . 499
13.8 Go's Memory Allocator . . . 499
13.9 Summary . . . 502

Chapter 14: File Systems 504
14.1 The File Abstraction . . . 504
14.1.1 File Attributes . . . 504
14.1.2 File Operations . . . 506
14.1.3 File Access Methods . . . 506
14.2 Directory Structure . . . 507
14.2.1 Single-Level Directory . . . 508
14.2.2 Two-Level Directory . . . 508
14.2.3 Tree-Structured Directory . . . 509
14.2.4 Acyclic-Graph Directory . . . 509
14.2.5 General Graph Directory . . . 510
14.3 File System Mounting and VFS . . . 511
14.3.1 Mounting . . . 511
14.3.2 The Virtual File System (VFS) Layer . . . 512
14.4 Allocation Methods . . . 515
14.4.1 Contiguous Allocation . . . 515
14.4.2 Linked Allocation . . . 516
14.4.3 Indexed Allocation (The Inode) . . . 517
14.5 Free-Space Management . . . 519
14.5.1 Bitmap (Bit Vector) . . . 519
14.5.2 Linked List . . . 520
14.5.3 Grouping . . . 521
14.5.4 Counting . . . 521
14.6 Journaling File Systems . . . 521
14.6.1 The Consistency Problem . . . 521
14.6.2 Write-Ahead Logging . . . 522
14.6.3 Journaling Modes . . . 523

14.6.4 ext4 Journaling Implementation 524
14.6.5 XFS Journaling 524
14.7 Log-Structured File Systems 525
14.7.1 The LFS Concept 525
14.7.2 Garbage Collection 525
14.7.3 F2FS: Flash-Friendly File System 526
14.8 Copy-on-Write File Systems 527
14.8.1 The CoW Principle 527
14.8.2 Btrfs 527
14.8.3 ZFS 528
14.9 Crash Recovery Analysis 531
14.9.1 The Crash Consistency Model 531
14.9.2 fsck: The Pre-Journaling Approach 533
14.10 Performance and Caching 534
14.9.1 The Buffer Cache (Page Cache) 534
14.9.2 Memory-Mapped File I/O 535
14.9.3 The Dentry Cache 537
14.9.3 Read-Ahead 537
14.9.4 Write Ordering and Barriers 537
14.10 File System Comparison 537
14.11 File Locking 540
14.11.1 Advisory vs Mandatory Locking 540
14.11.2 POSIX File Locking 541
14.11.3 Open File Description Locks (OFD Locks) 542
14.12 The ext4 On-Disk Format 543
14.12.1 Block Groups 543
14.12.2 The ext4 Inode Structure 543
14.12.3 Directory Entries 544
14.13 Special File Systems 545
14.13.1 procfs (`/proc`) 545
14.13.2 sysfs (`/sys`) 545
14.13.3 tmpfs 546
14.13.4 devtmpfs (`/dev`) 546
14.14 File System Performance Tuning 546
14.14.1 Block Size Selection 546
14.14.2 Inode Density 547
14.14.3 Mount Options for Performance 548
Exercises 548

Chapter 15: I/O Systems **550**
15.1 I/O Hardware Foundations 550
15.1.1 Ports, Buses, and Controllers 550
15.1.2 Register Access: Port-Mapped vs Memory-Mapped I/O 551
15.2 I/O Communication Mechanisms 553
15.2.1 Polling (Programmed I/O) 553
15.2.2 Interrupt-Driven I/O 554
15.2.3 Direct Memory Access (DMA) 556
15.2.4 The IOMMU: Protecting DMA 558

15.3 Kernel I/O Subsystem . . . 559
15.3.1 I/O Scheduling . . . 559
15.3.2 Buffering . . . 560
15.3.3 Caching . . . 561
15.3.4 Spooling . . . 561
15.4 Block Devices and Character Devices . . . 561
15.5 Disk Scheduling Algorithms . . . 564
15.5.1 FCFS (First-Come, First-Served) . . . 564
15.5.2 SSTF (Shortest Seek Time First) . . . 564
15.5.3 SCAN (Elevator Algorithm) . . . 565
15.5.4 C-SCAN (Circular SCAN) . . . 566
15.5.5 LOOK and C-LOOK . . . 566
15.5.6 Comparison Summary . . . 566
15.5.7 Seek Time Modelling . . . 567
15.6 NVMe and Modern Storage . . . 568
15.6.1 NVMe Architecture . . . 568
15.6.2 Command Queues and Parallelism . . . 569
15.6.3 Interrupt Coalescing . . . 570
15.6.4 No Mechanical Seek . . . 570
15.7 Asynchronous I/O . . . 571
15.7.1 The Scalability Problem . . . 571
15.7.2 POSIX AIO . . . 571
15.7.3 Event-Driven I/O: select, poll, epoll . . . 572
15.7.3 Linux io_uring . . . 574
15.8 Device Drivers . . . 577
15.8.1 Driver Architecture . . . 577
15.8.2 Top-Half and Bottom-Half Processing . . . 580
15.9 Performance Considerations . . . 582
15.9.1 System Call Overhead . . . 582
15.9.2 Zero-Copy I/O . . . 583
15.9.3 I/O Latency Hierarchy . . . 583
15.10 The Linux I/O Scheduler in Detail . . . 586
15.10.1 Multi-Queue Block Layer (blk-mq) . . . 586
15.10.2 Available I/O Schedulers . . . 586
15.10.3 I/O Priorities and cgroups . . . 587
15.11 Power Management and I/O . . . 588
15.11.1 Device Power States . . . 588
15.11.2 I/O Impact on System Power . . . 588
15.12 Error Handling in I/O . . . 589
15.12.1 Error Classification . . . 589
15.12.2 Error Propagation . . . 590
15.12.3 Timeout Handling . . . 590
Exercises . . . 591

Chapter 16: Storage Reliability **593**
16.1 Disk Structure and Geometry . . . 593
16.1.1 Hard Disk Drive (HDD) Anatomy . . . 593
16.1.2 Logical Block Addressing (LBA) . . . 594

16.1.3 Solid-State Drive (SSD) Structure . . . 595
16.1.4 NAND Cell Types and Reliability . . . 596
16.2 RAID: Redundant Arrays of Independent Disks . . . 597
16.2.1 RAID 0: Striping . . . 597
16.2.2 RAID 1: Mirroring . . . 598
16.2.3 RAID 5: Distributed Parity . . . 599
16.2.4 RAID 6: Double Parity . . . 600
16.2.5 RAID 10: Mirrored Stripes . . . 601
16.2.6 RAID Level Comparison . . . 601
16.3 Reliability Analysis . . . 604
16.3.1 The Bathtub Curve . . . 604
16.3.2 Mean Time To Failure (MTTF) . . . 605
16.3.3 Mean Time To Repair (MTTR) . . . 605
16.3.4 RAID Reliability Calculation . . . 606
16.3.5 Unrecoverable Read Errors (URE) During Rebuild . . . 608
16.4 Error Detection and Correction . . . 609
16.4.1 Cyclic Redundancy Check (CRC) . . . 609
16.4.2 Error-Correcting Codes (ECC) . . . 610
16.4.3 Checksums in ZFS and Btrfs . . . 611
16.5 The Flash Translation Layer (FTL) . . . 611
16.5.1 FTL Address Mapping . . . 611
16.5.2 Over-Provisioning . . . 612
16.6 Write Amplification and Wear Levelling . . . 613
16.6.1 The Write Amplification Factor . . . 613
16.6.2 Wear Levelling . . . 614
16.7 TRIM and Garbage Collection . . . 615
16.7.1 The TRIM Command . . . 615
16.7.2 Garbage Collection in SSDs . . . 616
16.8 Backup Strategies . . . 616
16.8.1 Backup Types . . . 616
16.8.2 The 3-2-1 Rule . . . 617
16.8.3 Snapshot-Based Backups . . . 618
16.9 Data Integrity and Silent Corruption . . . 618
16.9.1 The Problem of Silent Corruption . . . 618
16.9.2 Scrubbing . . . 619
16.9.3 End-to-End Checksums . . . 620
16.10 Putting It All Together: A Reliability Stack . . . 624
16.11 Erasure Coding Beyond RAID . . . 626
16.11.1 Limitations of Traditional RAID . . . 626
16.11.2 Reed-Solomon Erasure Codes . . . 626
16.11.3 Locally Repairable Codes (LRC) . . . 627
16.12 Storage Tiering and Caching . . . 628
16.12.1 The Storage Hierarchy . . . 628
16.12.2 SSD Caching (bcache, dm-cache, ZFS L2ARC) . . . 628
16.13 Storage Protocols and Interfaces . . . 630
16.13.1 Protocol Evolution . . . 630
16.13.2 Network-Attached Storage Protocols . . . 630
16.14 Practical Drive Selection . . . 631

16.14.1 HDD vs SSD Decision Matrix . . . 631
16.14.2 Enterprise vs Consumer Drives . . . 632
16.14.3 Drive Qualification and Testing . . . 633
16.15 Long-Term Data Preservation . . . 633
16.15.1 Media Longevity . . . 633
16.15.2 Format Longevity . . . 634
Exercises . . . 635

Chapter 17: Security and Protection 637
17.1 Protection Domains and the Access Matrix . . . 637
17.1.1 The Access Matrix Model . . . 638
17.1.2 Access Control Lists (ACLs) . . . 639
17.1.3 Capability Lists . . . 640
17.1.4 Comparing ACLs and Capabilities . . . 641
17.2 Discretionary Access Control (DAC) . . . 642
17.2.1 Unix Permission Bits . . . 642
17.2.2 Linux Capabilities . . . 643
17.2.3 POSIX ACLs . . . 645
17.3 Mandatory Access Control (MAC) . . . 646
17.3.1 The Bell-LaPadula Model (Confidentiality) . . . 647
17.3.2 The Biba Model (Integrity) . . . 648
17.3.3 Combining BLP and Biba . . . 649
17.4 Role-Based Access Control (RBAC) . . . 650
17.5 Linux Security Modules: SELinux and AppArmor . . . 653
17.5.1 SELinux . . . 654
17.5.2 AppArmor . . . 655
17.6 Sandboxing . . . 656
17.6.1 seccomp-bpf . . . 656
17.6.2 pledge and unveil (OpenBSD) . . . 658
17.6.3 Containers as Isolation Boundaries . . . 659
17.7 Hardware Security: Spectre and Meltdown . . . 661
17.7.1 Speculative Execution Background . . . 661
17.7.2 Meltdown (CVE-2017-5754) . . . 661
17.7.3 Spectre (CVE-2017-5753, CVE-2017-5715) . . . 662
17.7.4 KPTI: Kernel Page Table Isolation . . . 663
17.7.5 Beyond Spectre and Meltdown . . . 664
17.8 Trusted Computing . . . 665
17.8.1 Trusted Platform Module (TPM) . . . 665
17.8.2 Secure Boot and Measured Boot . . . 666
17.9 Buffer Overflow Attacks and Mitigations . . . 667
17.9.1 Stack Smashing . . . 667
17.9.2 Heap Overflow and Use-After-Free . . . 668
17.9.3 Return-Oriented Programming (ROP) . . . 668
17.9.4 Defence: ASLR, Stack Canaries, NX Bit . . . 669
17.10 Putting It All Together: Defence in Depth . . . 672

Chapter 18: Virtualisation 676
18.1 Why Virtualise? . . . 676

18.2 Full Virtualisation vs Paravirtualisation . . . 677
18.3 The Popek-Goldberg Theorem . . . 678
18.4 Type-1 Hypervisors . . . 680
18.4.1 Xen . . . 680
18.4.2 KVM . . . 682
18.4.3 VMware ESXi . . . 683
18.4.4 Trap-and-Emulate with Hardware Assistance . . . 683
18.5 Type-2 Hypervisors . . . 687
18.5.1 VirtualBox . . . 687
18.5.2 QEMU . . . 688
18.6 Binary Translation vs Hardware-Assisted Virtualisation . . . 688
18.7 Memory Virtualisation . . . 690
18.7.1 Shadow Page Tables . . . 690
18.7.2 Extended Page Tables (EPT / NPT) . . . 691
18.8 I/O Virtualisation . . . 692
18.8.1 Emulated Devices . . . 692
18.8.2 Virtio . . . 693
18.8.3 SR-IOV and VFIO . . . 694
18.9 Containers . . . 695
18.9.1 Linux Namespaces . . . 695
18.9.2 Cgroups v2 . . . 697
18.9.3 OCI Runtime Specification and runc . . . 698
18.9.4 Podman vs Docker . . . 699
18.10 The Virtualisation Spectrum . . . 703

Chapter 19: Distributed Operating System Concepts 706
19.1 Motivation and Design Goals . . . 706
19.1.1 Why Distribute? . . . 706
19.1.2 Transparency . . . 707
19.1.3 System Models . . . 707
19.2 Distributed Clocks . . . 708
19.2.1 Happened-Before and Lamport Timestamps . . . 708
19.2.2 Vector Clocks . . . 710
19.2.3 Physical Clock Synchronisation: NTP and PTP . . . 712
19.3 Distributed Mutual Exclusion . . . 715
19.3.1 Centralised Algorithm . . . 715
19.3.2 Ricart-Agrawala Algorithm . . . 716
19.3.3 Token Ring . . . 717
19.4 Consensus . . . 717
19.4.1 The FLP Impossibility Result . . . 718
19.4.2 Paxos . . . 719
19.4.3 Raft: A Detailed Walkthrough . . . 720
19.5 Distributed File Systems . . . 723
19.5.1 NFS (Network File System) . . . 723
19.5.2 GFS and HDFS . . . 724
19.5.3 Ceph . . . 725
19.6 Distributed Shared Memory . . . 725
19.6.1 Consistency Models . . . 725

19.6.2 Trade-offs . . . 726
19.7 The CAP Theorem . . . 727
19.8 Failure Detection . . . 730
19.8.1 Heartbeats . . . 730
19.8.2 The Phi Accrual Failure Detector . . . 731
19.8.3 SWIM (Scalable Weakly-consistent Infection-style Membership) 731
19.9 Theoretical Boundaries . . . 734

Chapter 20: The Future of Operating Systems 737
20.1 Unikernels Revisited . . . 737
20.1.1 Compile-Time Specialisation . . . 738
20.1.2 MirageOS . . . 739
20.1.3 Unikraft . . . 739
20.2 eBPF: Safe Kernel Extension . . . 740
20.2.1 The eBPF Architecture . . . 741
20.2.2 The eBPF Verifier . . . 742
20.2.3 Applications of eBPF . . . 742
20.3 io_uring: Kernel-Bypass I/O . . . 745
20.3.1 Submission Modes . . . 745
20.3.2 Performance Impact . . . 746
20.3.3 Advanced Features . . . 747
20.4 Capability-Based Hardware: CHERI . . . 747
20.5 Persistent Memory . . . 748
20.5.1 DAX File Systems . . . 749
20.5.2 The Crash Consistency Challenge . . . 750
20.6 Microkernels Redux: seL4 and Fuchsia . . . 752
20.6.1 seL4: The Verified Kernel . . . 752
20.6.2 Fuchsia and Zircon . . . 753
20.7 Rust in the Kernel . . . 753
20.7.1 Rust's Safety Model . . . 753
20.7.2 Linux Rust Modules . . . 754
20.7.3 The Safe/Unsafe Boundary . . . 756
20.8 WebAssembly as a Portable OS Interface . . . 758
20.8.1 WASI: The WebAssembly System Interface . . . 758
20.8.2 Wasm as a Container Alternative . . . 759
20.9 The Convergence of OS and Language Runtime . . . 760
20.9.1 Go and the GMP Scheduler . . . 760
20.9.2 The Erlang BEAM . . . 760
20.9.3 Green Threads Everywhere . . . 760
20.10 Summary: The OS in the Next Decade . . . 762

Preface

An operating system is the most intimate piece of software your programs will ever encounter. Every process you launch, every byte you allocate, every file you open, every network packet you send — all of it passes through the operating system. Yet most programmers treat it as a black box, invoking system calls without understanding the machinery behind them. This book opens the box.

Operating Systems Theory is the twenty-first volume in the Dr.H series. It sits at the intersection of hardware and software, drawing on the foundations established in *Computer Architecture* (Book 19) and *Algorithms and Data Structures* (Book 11). Where those books gave you the machine and the abstractions, this book gives you the layer that connects them: the operating system.

What This Book Covers

The book is organised into six parts spanning twenty chapters:

- **Part I: Foundations** (Chapters 1–3) establishes what an operating system is, surveys the major architectural patterns (monolithic, microkernel, hybrid, exokernel, unikernel), and examines the hardware–software interface that makes kernel-mode operation possible.
- **Part II: Processes and Scheduling** (Chapters 4–6) covers the process and thread abstractions, CPU scheduling theory (with proofs of optimality for key algorithms), and inter-process communication mechanisms from pipes to RPC.
- **Part III: Concurrency** (Chapters 7–9) tackles the hardest problems in systems programming: synchronisation primitives, deadlock theory (including the Banker's Algorithm), and modern lock-free and wait-free data structures.
- **Part IV: Memory** (Chapters 10–13) progresses from basic memory management through virtual memory and page replacement to advanced topics including NUMA, huge pages, slab allocators, and the Go runtime's memory allocator.
- **Part V: Storage and I/O** (Chapters 14–16) covers file system design (from ext4 to ZFS), I/O subsystem architecture (including io_uring), and storage reliability (RAID, checksums, wear levelling).

- **Part VI: Protection and Modern Topics** (Chapters 17–20) addresses OS security (access control, Spectre/Meltdown, sandboxing), virtualisation (hypervisors, containers, Podman), distributed OS concepts (consensus, CAP theorem), and the future of operating systems (eBPF, unikernels, capability hardware, Rust in the kernel).

The Programmer's Perspective

Throughout the book, you will find **Programmer's Perspective** callout boxes. These bridge the gap between theory and practice, showing how the concepts materialise in real systems — primarily Linux, Go, and C. When we discuss virtual memory, you will see how Go's runtime manages its heap. When we cover scheduling, you will trace through the GMP model that powers goroutines. When we examine containers, you will see the Linux namespace and cgroup primitives that make them work.

The goal is not to teach you to use an OS, but to understand one deeply enough that you could build one.

Callout Boxes Used in This Book

Definitions introduce formal terminology and precise statements that you should internalise.

Theorems state and prove important results. Proofs are included where they illuminate the underlying ideas.

Examples work through concrete scenarios, calculations, or code to ground the theory in practice.

Programmer's Perspective boxes connect OS theory to real-world systems programming in Go, C, and Linux.

Exercises appear at the end of every chapter — seven per chapter, progressing from comprehension to analysis to design.

Prerequisites

This book assumes familiarity with:

- **C programming** — you should be comfortable reading and writing C, including pointers, structs, and basic systems programming.

- **Computer Architecture** (Book 19) — CPU pipelines, caches, virtual addressing, privilege levels.
- **Algorithms** (Book 11) — asymptotic analysis, trees, graphs, hash tables.
- **Discrete Mathematics** (Book 7) — sets, relations, proof techniques.

No prior operating systems coursework is required. We build from first principles.

Acknowledgements

This book was written as part of the Dr.H project — a self-directed journey through theoretical computer science. The debt to Silberschatz, Galvin, and Gagne (*Operating System Concepts*), Tanenbaum and Bos (*Modern Operating Systems*), and Love (*Linux Kernel Development*) is evident throughout. Any errors that remain are mine alone.

Achraf SOLTANI
April 2026

Chapter 1: What Is an Operating System?

Every piece of software you have ever used – from a command-line compiler to a web browser rendering millions of pixels – runs on top of an operating system. The OS is the single most consequential piece of software on any general-purpose computer. It decides which programs run, when they run, how much memory they receive, and whether they are allowed to touch a particular file or device. Yet most programmers interact with it only indirectly, through high-level libraries that hide the underlying machinery. This chapter strips away that abstraction layer and examines what an operating system actually is, why it exists, and how it presents itself to the programs that depend on it.

We begin with the three fundamental roles that every operating system must fulfil, then trace the historical evolution from bare-metal programming to the sophisticated multi-subsystem kernels of today. We examine the hardware mechanism – dual-mode CPU operation – that makes protection possible, and the software mechanism – system calls – that mediates every interaction between application code and the kernel. Finally, we survey POSIX, the standard that codifies this interface, and explore the practical tools that reveal the OS boundary at work.

The Three Roles of an Operating System

An operating system serves three distinct but intertwined roles. Every design decision in OS theory can be traced back to tensions between these roles.

Role 1: Resource Manager

A computer contains a finite set of physical resources: CPU cores, bytes of RAM, disk sectors, network bandwidth, GPU compute units. Multiple programs compete for these resources simultaneously. The OS acts as an *arbiter*, deciding how to allocate resources fairly, efficiently, and without conflict.

Resource management encompasses:

- **CPU scheduling** – deciding which thread runs on which core and for how long.

- **Memory allocation** – partitioning physical RAM among processes and the kernel itself.
- **I/O multiplexing** – ensuring that two processes do not simultaneously write to the same disk sector or network port.
- **Energy management** – throttling CPU frequency and powering down idle devices on battery-constrained systems.
- **Bandwidth allocation** – dividing network and disk bandwidth among competing applications using mechanisms like traffic shaping and I/O schedulers.

The fundamental constraint is scarcity. If every program could have unlimited CPU time and unlimited memory, there would be no need for an operating system at all. The existence of the OS is a direct consequence of resource contention.

Note

Resource multiplexing refers to the technique of sharing a single physical resource among multiple logical consumers. *Time multiplexing* shares a resource over time (e.g., CPU scheduling assigns each process a time slice), while *space multiplexing* divides a resource into partitions (e.g., memory pages assigned to different processes). Most OS resources use a combination of both: the CPU is time-multiplexed, memory is space-multiplexed, and the disk is both (space-multiplexed into blocks owned by different files, and time-multiplexed when multiple processes issue I/O requests).

To illustrate the consequences of poor resource management, consider what happens without it. Suppose two programs attempt to write to the same disk block simultaneously. Without OS mediation, one program's data could be partially overwritten by the other, producing a corrupted block that neither intended. The OS prevents this by serialising disk writes through a block I/O layer that queues requests, resolves conflicts, and ensures atomicity.

Similarly, without CPU scheduling, a single compute-bound program would consume 100% of a CPU core indefinitely. Other programs – including the user's shell and desktop environment – would be starved of CPU time and become unresponsive. The scheduler prevents this by preempting long-running programs and distributing CPU time according to a fairness policy.

Role 2: Abstraction Provider

Raw hardware is extraordinarily difficult to program directly. Consider writing data to a spinning hard disk: you would need to issue commands to the disk controller specifying cylinder, head, and sector numbers; handle rotational latency; manage the DMA transfer; poll or handle an interrupt when the transfer completes; and retry on error. Every program that touches the disk would need to duplicate this logic.

The OS provides *abstractions* – simplified, uniform interfaces that hide hardware complexity:

Hardware Reality	OS Abstraction
Physical RAM addresses	Virtual address spaces
Disk sectors and cylinders	Files and directories
CPU cores and registers	Processes and threads
Network interface cards	Sockets and connections
GPU command buffers	Graphics contexts
Timer hardware counters	Clock and sleep APIs

These abstractions serve two purposes. First, they make programming tractable: a `write()` system call is vastly simpler than programming a disk controller. Second, they provide *portability*: the same program binary can run on machines with different disk controllers, as long as the OS exposes the same `write()` interface.

The quality of an abstraction is measured by how well it balances simplicity with power. A too-simple abstraction forces programmers to work around its limitations (e.g., early Unix's lack of file locking forced applications to use ad-hoc locking schemes). A too-complex abstraction is difficult to learn and implement correctly (e.g., the Windows registry, with its hierarchical key-value store, complex permission model, and transaction support).

Info

The concept of abstraction in operating systems mirrors the layered model in computer architecture. Just as the ISA provides an abstraction boundary between hardware and software, the system call interface provides an abstraction boundary between the kernel and user programs. Each layer hides complexity and exposes a cleaner interface to the layer above. The key insight, attributed to David Wheeler, is that "all problems in computer science can be solved by another level of indirection" – but each level of indirection adds overhead, so the art lies in choosing the right levels.

Role 3: Protection Boundary

Without an OS, any program could read or write any byte of memory, access any device, and corrupt any other program's data. The OS enforces *isolation*: each process operates within a confined sandbox, unable to interfere with other processes or with the kernel itself.

Protection operates at multiple levels:

- **Memory isolation.** Each process has its own virtual address space. Accessing memory outside that space triggers a hardware fault that the OS intercepts.
- **Privilege separation.** The CPU distinguishes between kernel mode (where the OS runs) and user mode (where applications run). Certain instructions – such as those that modify page tables or disable interrupts – are only available in kernel mode.
- **Access control.** The file system enforces permissions: user A cannot read user B's files unless explicitly authorised. On Unix systems, each file has an owner, a group, and permission bits (read, write, execute) for owner, group, and others.
- **Resource limits.** The OS can cap the amount of CPU time, memory, or disk space a process may consume. On Linux, this is enforced via cgroups (control groups), which are the foundation of container isolation.
- **Namespace isolation.** Modern Linux kernels can give each process (or group of processes) its own view of the file system, network stack, process table, and other OS resources. This is the mechanism behind containers: each container sees its own root file system, its own network interfaces, and its own PID 1, even though it shares the host kernel.

Tip

Protection is not merely a security feature – it is a reliability feature. A bug in one user-space program should not crash the entire system. The OS's protection boundary confines the blast radius of faults to the offending process. This is why a segmentation fault in your web browser does not crash your text editor or your operating system: the hardware MMU and the kernel's signal delivery mechanism collaborate to terminate only the faulting process.

The Interplay of Roles

The three roles are not independent – they interact in subtle ways. Resource management often requires protection: the scheduler must prevent a process from consuming more CPU time than its allocation, which requires the timer interrupt (a protection mechanism) to enforce preemption. Abstraction often requires resource management: the virtual memory abstraction allocates physical pages on demand, which is a resource management function. Protection often requires abstraction: the file permission model is an abstraction over raw disk sectors, and the process isolation model is an abstraction over raw memory addresses.

Understanding this interplay is essential for understanding why OS design is difficult. Optimising one role often compromises another. Making the system call interface simpler (better abstraction) may require more kernel code (harder to protect). Making resource allocation more efficient may require exposing hardware details (weaker abstraction). Making protection stronger may require more mode transitions (lower performance).

A Formal View: The OS as a State Machine

We can model an operating system as a state machine $\mathcal{S} = (Q, q_0, \Sigma, \delta)$ where:

- Q is the set of all possible system states, including the contents of every register, memory cell, and device register.
- $q_0 \in Q$ is the initial state at boot time.
- Σ is the set of events: hardware interrupts, system calls, timer ticks, and I/O completions.
- $\delta : Q \times \Sigma \to Q$ is the transition function – the kernel code that responds to each event and produces a new system state.

Every system call is a transition. When a user program invokes `read(fd, buf, n)`, the event $\sigma_{\text{read}} \in \Sigma$ triggers a transition that may block the calling process, initiate a DMA transfer, and schedule another process to run. The new state $q' = \delta(q, \sigma_{\text{read}})$ reflects all of these changes.

This model is useful for reasoning about correctness. A *safety property* asserts that the system never enters a bad state (e.g., two processes never hold the same lock simultaneously). A *liveness property* asserts that the system eventually reaches a good state (e.g., every I/O request eventually completes). Formal verification of OS kernels, such as the seL4 project, proves these properties rigorously over the state machine model.

$$\forall q \in Q, \forall \sigma \in \Sigma : \text{invariant}(q) \implies \text{invariant}(\delta(q, \sigma))$$

The equation above expresses the key inductive argument: if the system invariant holds in state q, it must also hold after any transition. This is the foundation of every kernel correctness proof.

The state space Q of a real operating system is astronomically large – for a system with 16 GB of RAM, $|Q| \geq 2^{128 \times 10^9}$, making exhaustive exploration impossible. Formal verification works by proving properties about the *transition function* δ rather than enumerating states. If we can prove that δ preserves the invariant for every possible input, we know the invariant holds in every reachable state, regardless of how many states exist.

Historical Evolution

The history of operating systems is a history of increasing abstraction, driven by the need to share ever-more-powerful hardware among ever-more-demanding users.

Era 1: No Operating System (1940s–1950s)

The earliest computers – ENIAC, EDSAC, UNIVAC – had no operating system at all. A single programmer had exclusive access to the machine for a scheduled block of time. Programs were loaded via punched cards or paper tape, and the programmer interacted directly with the hardware through front-panel switches and lights.

The workflow was:

1. Sign up for a time slot on the machine (perhaps 2 hours).
2. Arrive at the machine room with a deck of punched cards.
3. Load the cards into the card reader.

4. Set switches on the front panel to configure the initial state.
5. Press the start button.
6. Wait. If the program crashed, examine the console lights and registers to diagnose the problem.
7. Fix the error, modify the card deck, and repeat.

This approach was simple but extraordinarily wasteful. The computer sat idle while the programmer debugged their program, loaded cards, or examined output. CPU utilisation rates were often below 10%. A machine that cost millions of dollars spent most of its time waiting for humans.

Era 2: Batch Systems (Late 1950s–1960s)

To improve utilisation, operators began grouping jobs into *batches*. A resident monitor – the first operating system – loaded jobs sequentially from a card reader, executed each one, and printed the output. The programmer never touched the machine directly; instead, they submitted a deck of cards and collected printout hours or days later.

```
BATCH PROCESSING TIMELINE
──────────────────────────────────────────────────────────────

  Time ──────────────────────────────────────────────────────►

  CPU:  │ Job A │ Job B │ Job C │ Job D │ Job E │
        ├───────┼───────┼───────┼───────┼───────┤
  I/O:  │ idle  │ idle  │ idle  │ idle  │ idle  │

  Problem: CPU waits for I/O; I/O waits for CPU.
  Utilisation remains low because jobs are sequential.
──────────────────────────────────────────────────────────────
```

The batch monitor introduced the first system calls – primitives for reading cards, writing to the printer, and signalling job completion. It also introduced the first protection problem: a buggy job could overwrite the monitor itself. Early solutions used hardware *memory bounds registers* to confine each job to its allocated region.

Job Control Language (JCL) emerged as the first “shell” – a way for programmers to specify job parameters (which compiler to use, which input files to read, which output files to write) in the card deck itself. IBM’s JCL for OS/360 was notoriously cryptic:

```
//MYJOB  JOB  (ACCT),'PROGRAMMER NAME',CLASS=A
//STEP1  EXEC PGM=FORTRAN
//SYSIN  DD   *
```

```
      PROGRAM HELLO
      PRINT *, 'HELLO WORLD'
      END
/*
//SYSPRINT DD  SYSOUT=A
```

Era 3: Multiprogramming (1960s–1970s)

The key insight of multiprogramming was that while one job waits for I/O (a disk read, a tape rewind), the CPU could execute another job. By keeping multiple jobs in memory simultaneously and switching between them, the CPU could be kept busy almost continuously.

```
MULTIPROGRAMMING TIMELINE
────────────────────────────────────────────────────────────────────────

  Time ──────────────────────────────────────────────────────────────►

  CPU:  │ Job A │ Job B │ Job A │ Job C │ Job B │ Job C │
        ├───────┼───────┼───────┼───────┼───────┼───────┤
  I/O:  │       │ A:disk│       │ A:net │ C:dsk │       │

  While Job A waits for disk I/O, the CPU runs Job B.
  CPU utilisation increases dramatically.
────────────────────────────────────────────────────────────────────────
```

Multiprogramming required significant OS infrastructure:

- **Memory management** – multiple jobs in memory simultaneously, each needing protection from the others. Base and limit registers were used initially; later systems introduced segmentation and paging.
- **CPU scheduling** – deciding which job to run when the current one blocks on I/O. Early schedulers used simple priority schemes; later ones introduced round-robin and multi-level feedback queues.
- **I/O device management** – handling concurrent I/O requests from multiple jobs without corruption. Device queues, interrupt handlers, and DMA transfers all originated in this era.
- **Spooling** – Simultaneous Peripheral Operation On-Line. Output was written to disk rather than directly to the printer, allowing multiple jobs' output to be printed later without tying up the CPU.

IBM's OS/360 (1964) was the landmark multiprogramming system. It supported mul-

tiple job classes, spooled I/O, and a rudimentary file system. It was also notoriously complex – Fred Brooks' *The Mythical Man-Month* drew its lessons from the OS/360 project, concluding that "adding manpower to a late software project makes it later."

We can quantify the benefit of multiprogramming. If a single job spends a fraction p of its time waiting for I/O, then the CPU utilisation with n jobs in memory (assuming they all have independent I/O patterns) is:

$$U = 1 - p^n$$

For $p = 0.8$ (a job that spends 80% of its time waiting for I/O):

n (jobs in memory)	U (CPU utilisation)
1	0.20
2	0.36
4	0.59
8	0.83
16	0.97

This formula, while simplified (it assumes statistical independence of I/O patterns), demonstrates why multiprogramming was transformative: even with heavily I/O-bound jobs, keeping enough of them in memory ensures high CPU utilisation.

Era 4: Time-Sharing (1970s–1980s)

Multiprogramming optimised throughput – total work done per unit time. But interactive users care about *response time* – how quickly the system responds to a keystroke. Time-sharing systems gave each user a short *quantum* of CPU time (typically 10–100 milliseconds) and rapidly rotated among users, creating the illusion that each had a dedicated machine.

Note

The distinction between multiprogramming and time-sharing is one of emphasis, not mechanism. Both keep multiple programs in memory and switch between them. Multiprogramming switches on I/O events to maximise throughput; time-sharing switches on timer interrupts to minimise response time. Modern systems do both simultaneously.

MIT's Compatible Time-Sharing System (CTSS, 1961) and Multics (1969) pioneered these ideas. CTSS was one of the first systems to provide interactive computing to multiple simultaneous users, each with their own terminal. Its success demonstrated that time-sharing was practical, contradicting sceptics who believed the overhead of context switching would make it uneconomical.

Multics, a joint project between MIT, Bell Labs, and General Electric, was far more ambitious. It introduced concepts that remain central to modern operating systems:

- Hierarchical file system with directory trees and path names
- Per-process virtual memory with demand paging (segments and pages)
- Dynamic linking of shared libraries at load time
- Ring-based protection (eight privilege levels, more than any successor has used)
- Access control lists on files (beyond simple permission bits)
- Hot-pluggable hardware and online reconfiguration
- A shell implemented as a user-level program (not built into the kernel)

Multics was commercially unsuccessful – too complex, too slow, too expensive. But its influence was enormous: virtually every OS concept in use today can be traced to Multics.

Era 5: Unix and the Modern Era (1970s–Present)

Ken Thompson and Dennis Ritchie, frustrated by Multics' complexity, created Unix at Bell Labs in 1969. Where Multics was ambitious and baroque, Unix was minimalist and elegant. Its design philosophy – small programs that do one thing well, connected by pipes – proved extraordinarily influential.

Unix introduced or popularised:

- The process model (fork/exec/wait) – a clean separation between creating a process and specifying what it should run
- The unified file abstraction ("everything is a file") – devices, pipes, and sockets are all accessed through file descriptors
- The shell as a user-level program – not part of the kernel, replaceable by the user
- Pipes for inter-process communication – connecting the output of one program to the input of another
- The C programming language as a systems implementation language
- Portable OS code (Unix was rewritten in C in 1973, making it the first portable operating system)
- Regular expressions for text processing (through tools like `grep`, `sed`, `awk`)

From Unix descended two major lineages: the BSD family (FreeBSD, OpenBSD, NetBSD, macOS) and the System V family (Solaris, AIX, HP-UX). Linux, created by Linus Torvalds in 1991, borrowed Unix's design but was written from scratch, avoiding AT&T licensing issues.

The Unix philosophy can be summarised in three principles, articulated by Doug McIlroy:

1. Make each program do one thing well.
2. Expect the output of every program to become the input to another.
3. Design and build software to be tried early, ideally within weeks.

These principles shaped not just operating system design but the entire culture of software engineering.

Timeline Summary

Era	Period	Key Systems	Key Innovation
No OS	1940s–1950s	ENIAC, EDSAC	Direct hardware access
Batch	Late 1950s–1960s	FMS, IBSYS	Resident monitor, job queues
Multiprogramming	1960s–1970s	OS/360, THE	Multiple jobs in memory
Time-sharing	1970s–1980s	CTSS, Multics	Interactive terminals, quanta
Modern	1970s–present	Unix, Linux, Windows NT	Portability, networking, virtualisation

Kernel Mode vs User Mode

The most fundamental architectural feature that enables an operating system is the CPU's *dual-mode operation*. The processor maintains a status bit – often called the *mode bit* – that indicates whether the currently executing code is trusted (kernel mode) or untrusted (user mode).

Kernel Mode (Supervisor Mode)

In kernel mode, the CPU can execute *any* instruction, including privileged instructions that:

- Modify the page table base register (changing virtual-to-physical address mappings)
- Enable or disable hardware interrupts
- Access I/O device registers directly
- Execute the `HLT` instruction to halt the processor
- Write to model-specific registers (MSRs) for CPU configuration
- Load the interrupt descriptor table register (IDTR)
- Load the global descriptor table register (GDTR)
- Access debug registers for hardware breakpoints

Only the operating system kernel runs in kernel mode. When the kernel executes, it has unrestricted access to all hardware and all memory. This is an immense amount of power, which is why kernel code must be written with extreme care: a bug in the kernel can compromise the entire system.

User Mode

In user mode, the CPU restricts the instruction set. Any attempt to execute a privileged instruction triggers a *general protection fault* – a hardware exception that transfers control to the kernel. The offending process is typically terminated with a signal (e.g., `SIGSEGV` or `SIGILL` on Unix).

User-mode code can only access its own virtual address space. It cannot directly touch hardware devices, other processes' memory, or kernel data structures. This restriction is enforced by the hardware MMU (Memory Management Unit), which consults the page table on every memory access and checks the access permissions.

The restrictions of user mode are comprehensive:

- Cannot modify page tables (so it cannot access other processes' memory)
- Cannot disable interrupts (so the kernel can always preempt it)
- Cannot access I/O ports (so it cannot directly control hardware)
- Cannot change the privilege level (so it cannot promote itself to kernel mode)
- Cannot access kernel memory (even though kernel pages exist in the same virtual address space, they are marked supervisor-only)

Mode Transitions

There are exactly three ways to transition from user mode to kernel mode:

1. **System call (trap).** The user program executes a special instruction (`SYSCALL` on x86-64, `SVC` on ARM) that transfers control to the kernel at a predetermined entry point. This is a *voluntary* transition – the user program explicitly requests kernel services.
2. **Hardware interrupt.** An external device (disk controller, network card, timer) signals the CPU. The CPU suspends the current user program and transfers control to the kernel's interrupt handler. This is an *involuntary* transition – the user program did not request it and may not even be aware of it.
3. **Exception (fault).** The user program triggers an error condition – a page fault, a division by zero, an illegal instruction. The CPU transfers control to the kernel's exception handler. This is also involuntary, but unlike interrupts, it is caused by the program's own instruction, not by an external event.

```
MODE TRANSITIONS
──────────────────────────────────────────────────────────────


         System Call         Interrupt          Exception
User ──────────────────► Kernel ◄──────────── Hardware
Mode   (voluntary)   Mode   (involuntary)       Mode
         ◄──────────                            ▲
         return from                            │
         system call                            │
                                        Page fault,
                                        div by zero,
                                        illegal insn
──────────────────────────────────────────────────────────────
```

The transition from kernel mode back to user mode occurs when the kernel executes a special return instruction (`SYSRET` on x86-64, `ERET` on ARM) that restores the saved user-mode context and clears the kernel-mode bit.

Importantly, there is no way to transition from user mode to kernel mode at an arbitrary kernel address. The `SYSCALL` instruction always jumps to the address stored in a specific MSR (Model-Specific Register), which is set by the kernel during boot and cannot be modified from user mode. Similarly, interrupts and exceptions always vector through the Interrupt Descriptor Table (IDT), which is also set by the kernel. This *controlled entry* property is essential for security: without it, a malicious program could jump to the middle of a kernel function, bypassing security checks.

Programmer's Perspective

Understanding mode transitions explains why system calls are expensive relative to ordinary function calls. A regular function call on x86-64 involves pushing a return address and jumping to the target – roughly 1–2 nanoseconds. A system call requires saving all user-mode registers, switching the stack pointer to the kernel stack, executing the kernel code, restoring registers, and switching back. On a modern x86-64 processor, the `SYSCALL/SYSRET` pair alone costs approximately 50–100 nanoseconds, plus the actual kernel work. On systems with Meltdown mitigations (KPTI – Kernel Page Table Isolation), each mode switch requires flushing page table entries, adding another 200–500 nanoseconds. This is why high-performance systems use techniques like `io_uring` (Linux 5.1+) to batch system calls and reduce mode transitions. In Go, the runtime's `syscall.RawSyscall` bypasses some runtime bookkeeping for performance, but even so, the hardware cost of the mode switch remains.

The System Call as the OS API

System calls are the *only* mechanism by which user-space programs can request services from the kernel. They form the Application Programming Interface of the operating system.

A system call is conceptually similar to a function call, but with three critical differences:

1. **Privilege elevation.** The system call transitions the CPU to kernel mode, granting access to privileged resources.
2. **Controlled entry point.** The user program cannot jump to an arbitrary kernel address. The `SYSCALL` instruction always vectors to a single entry point (the system call dispatcher), which uses the system call number to index into a jump table.
3. **Argument validation.** The kernel must validate every argument passed from user space. A user could pass a malicious pointer or an out-of-range value. Trusting user-supplied data without validation is the root cause of many kernel security vulnerabilities.

The typical system call flow on x86-64 Linux proceeds as follows:

```
SYSTEM CALL FLOW (x86-64 Linux)
────────────────────────────────────────────────────────────
```

```
User Space:
  1. Place system call number in RAX
  2. Place arguments in RDI, RSI, RDX, R10, R8, R9
  3. Execute SYSCALL instruction

─── mode switch ─── (hardware saves RIP, RFLAGS to RCX, R11)

Kernel Space:
  4. entry_SYSCALL_64 saves user registers on kernel stack
  5. Index into sys_call_table[RAX]
  6. Execute the handler (e.g., sys_read, sys_write)
  7. Place return value in RAX
  8. Execute SYSRET instruction

─── mode switch ─── (hardware restores RIP, RFLAGS from RCX, R11)

User Space:
  9. Check RAX for error (negative = -errno)
```

A concrete example in C – reading bytes from a file:

```c
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void) {
    char buf[1024];
    int fd = open("/etc/hostname", O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    ssize_t n = read(fd, buf, sizeof(buf) - 1);
    if (n < 0) {
        perror("read");
        close(fd);
        return 1;
    }
```

```
    buf[n] = '\0';
    printf("hostname: %s", buf);
    close(fd);
    return 0;
}
```

In this program, `open()`, `read()`, `write()` (called by `printf` internally), and `close()` are all system calls. Each one triggers a user-to-kernel-to-user mode transition. The C library (`glibc`, `musl`) provides thin wrapper functions that set up registers and execute the `SYSCALL` instruction.

The number of system calls varies by OS. Linux 6.x defines approximately 450 system calls (the exact number depends on the architecture). macOS defines approximately 500. Windows defines roughly 2000 (though many are undocumented). The trend is toward more system calls over time, as new kernel features are added.

OS Structure Overview

A modern operating system is organised into several subsystems, each managing a different class of resource. These subsystems interact extensively but can be understood independently.

The Process Subsystem

A *process* is an instance of a running program. It consists of:

- A virtual address space (code, data, heap, stack segments)
- One or more threads of execution, each with its own program counter, stack pointer, and register state
- Open file descriptors, signal handlers, and other kernel-managed metadata
- A process ID (PID), parent process ID (PPID), user ID (UID), and group ID (GID)
- Scheduling priority and CPU affinity
- Resource usage counters (CPU time consumed, memory pages allocated, I/O bytes transferred)

The process subsystem handles:

- **Process creation** (`fork()` on Unix, `CreateProcess()` on Windows) – creating new processes by cloning existing ones or loading new programs.
- **Process termination** (`exit()`, signal delivery) – cleaning up resources when a process finishes or is killed.

- **CPU scheduling** (selecting which process/thread runs next on each core) – the topic of Chapters 5 and 6 of this book.
- **Inter-process communication** (pipes, shared memory, message queues, sockets) – allowing processes to exchange data and coordinate.
- **Synchronisation** (mutexes, semaphores, condition variables) – preventing race conditions when multiple threads access shared data.

Info

The process is arguably the most important abstraction in operating systems. Every other subsystem exists to serve processes: memory management provides each process with a virtual address space, the file system provides persistent storage for process data, and the I/O subsystem provides communication channels. The process concept is so fundamental that it appears in every OS ever built, from the simplest embedded RTOS to the most complex mainframe OS.

The Memory Subsystem

The memory subsystem manages the mapping between *virtual addresses* (what processes see) and *physical addresses* (actual RAM locations). Key responsibilities include:

- **Virtual memory** – giving each process the illusion of a large, contiguous address space, even when physical RAM is limited. On a 64-bit system, each process can address up to 2^{48} bytes (256 TB) of virtual memory, even though the machine may have only 16 GB of physical RAM.
- **Demand paging** – loading pages from disk into RAM only when accessed, and evicting infrequently used pages back to disk (the swap partition or swap file).
- **Page table management** – maintaining the multi-level data structures that the hardware MMU uses to translate virtual addresses to physical addresses.
- **Memory protection** – ensuring that each process can only access its own pages, and that read-only pages (such as code) cannot be written.
- **Shared memory** – allowing multiple processes to share physical pages, used for inter-process communication and shared libraries (e.g., `libc.so` is mapped into every process's address space but exists only once in physical RAM).

The memory subsystem interacts tightly with the hardware Memory Management Unit (MMU). When a process accesses a virtual address, the MMU translates it to a

physical address using the page table. If the page is not in RAM (a *page fault*), the MMU raises an exception, and the kernel's page fault handler loads the page from disk.

$$\text{virtual address} \xrightarrow{\text{MMU + page table}} \text{physical address}$$

On a system with 4 KB pages (the standard on x86-64), a 256 TB virtual address space contains $2^{48}/2^{12} = 2^{36} \approx 68$ billion virtual pages. Storing a page table entry for each would require 512 GB of RAM – larger than the address space itself. This is why page tables use a multi-level tree structure (4 levels on x86-64), where only the branches corresponding to mapped pages actually exist.

The File Subsystem

The file system provides a persistent, hierarchical namespace for data. It abstracts away the details of disk geometry (sectors, tracks, platters) or flash translation layers (wear levelling, garbage collection) and presents a simple model: named files organised in directories.

Key responsibilities:

- **Namespace management** – creating, deleting, and renaming files and directories. The namespace is a tree (or DAG, in the case of hard links) rooted at `/`.
- **Storage allocation** – mapping file data to disk blocks and managing free space. Different file systems use different allocation strategies: ext4 uses extents (contiguous runs of blocks), btrfs uses B-trees, and ZFS uses a copy-on-write block allocation tree.
- **Metadata management** – maintaining file attributes (size, permissions, timestamps, ownership) in data structures called *inodes* (on Unix) or *MFT entries* (on NTFS).
- **Caching** – buffering frequently accessed file data in RAM (the *page cache* on Linux) to reduce disk I/O. On a system with ample RAM, the page cache can absorb the vast majority of read operations, serving data from memory at nanosecond latency rather than from disk at millisecond latency.
- **Journaling/logging** – ensuring file system consistency after crashes. A journaling file system (ext4, NTFS, XFS) writes pending metadata changes to a log before applying them. If the system crashes mid-operation, the log can be replayed at boot time to restore consistency.

- **Virtual File System (VFS)** – a kernel-internal abstraction layer that presents a uniform interface to user space, regardless of the underlying file system type. The VFS allows a single `read()` system call to work whether the file resides on ext4, NFS, procfs, or a FUSE-based user-space file system.

The I/O Subsystem

The I/O subsystem manages communication with hardware devices: disks, network interfaces, keyboards, displays, USB devices, and more. It provides a layered architecture:

```
I/O SUBSYSTEM LAYERS
──────────────────────────────────────────────────────────────

User program
    │
    ▼
System call interface    (read, write, ioctl)
    │
    ▼
Device-independent layer (buffering, caching, error handling)
    │
    ▼
Device driver            (hardware-specific code)
    │
    ▼
Device controller        (hardware registers, DMA)
    │
    ▼
Physical device           (disk, NIC, keyboard)
──────────────────────────────────────────────────────────────
```

The *device driver* is the critical abstraction boundary. Each driver understands the specific hardware it manages and presents a uniform interface to the device-independent layer above. This is why adding a new hardware device to an OS requires only writing a new driver, not modifying the kernel's core I/O logic.

On Linux, the device driver model distinguishes between:

- **Character devices** (serial ports, keyboards, mice) – accessed as a stream of bytes, one at a time.
- **Block devices** (disks, SSDs) – accessed in fixed-size blocks, with a request queue and I/O scheduler.

- **Network devices** (Ethernet, Wi-Fi) – accessed through the socket API, with a packet-based interface.

Subsystem Interactions

These four subsystems do not operate in isolation. Consider what happens when a process calls `read(fd, buf, 4096)`:

1. The **process subsystem** identifies the calling process and validates the file descriptor `fd` against the process's file descriptor table.
2. The **file subsystem** looks up the VFS inode associated with `fd`, determines the file system type, and calls the appropriate file system's `read` method. The method maps the file offset to a disk block number, checking the page cache first.
3. If the data is cached in the page cache, the **memory subsystem** copies it directly from the cache page to the user-space buffer at address `buf`. The `read()` returns immediately, without any disk I/O.
4. If the data is not cached, the **I/O subsystem** issues a read request to the disk driver, which programs the disk controller via DMA.
5. The **process subsystem** blocks the calling thread and schedules another runnable thread to use the CPU.
6. When the disk DMA transfer completes, the disk controller raises a hardware interrupt. The **I/O subsystem** invokes the driver's interrupt handler, which marks the I/O request as complete and wakes the page cache waiter.
7. The **memory subsystem** copies the data from the newly populated page cache page into the process's user-space buffer at address `buf`, checking that the destination page is mapped and writable.
8. The **process subsystem** unblocks the calling thread and eventually schedules it to resume. The `read()` system call returns the number of bytes read.

A single `read()` call thus involves all four subsystems cooperating through well-defined internal interfaces.

POSIX and the System Call Interface

What is POSIX?

POSIX (Portable Operating System Interface) is a family of standards specified by IEEE (formally, IEEE 1003) that define the API, shell, and utility interfaces for Unix-like operating systems. POSIX compliance means that a program written using POSIX APIs can be compiled and run on any conforming system without modification.

Note

POSIX is a *source-level* portability standard, not a binary compatibility standard. A POSIX program must be recompiled for each target platform because different systems have different instruction sets, calling conventions, and binary formats. Binary compatibility is handled separately by standards like the ELF specification and platform-specific ABIs (Application Binary Interfaces).

POSIX was created in the 1980s to resolve the growing incompatibilities between different Unix variants. As Unix fragmented into BSD, System V, and various vendor-specific versions, programs written for one variant often failed to compile on another. POSIX defined a common subset that all Unix-like systems could agree on, restoring portability.

The standard is organised into several parts:

- **POSIX.1 (IEEE 1003.1)** – the core API: system calls, C library functions, header files, and error codes.
- **POSIX.2 (IEEE 1003.2)** – the shell and utility interface: the behaviour of `sh`, `awk`, `grep`, `make`, and other standard commands.
- **POSIX threads (pthreads)** – the multi-threading API: thread creation, mutexes, condition variables, read-write locks.
- **POSIX real-time extensions** – priority scheduling, timers, asynchronous I/O, message queues.

Key POSIX System Calls

The POSIX system call interface encompasses several hundred calls. The most fundamental ones fall into categories:

Process management:

System Call	Purpose
`fork()`	Create a child process (clone of the parent)
`exec()`	Replace the current process image with a new program
`wait()` / `waitpid()`	Wait for a child process to terminate
`exit()`	Terminate the calling process
`getpid()`	Return the process ID
`kill()`	Send a signal to a process

File operations:

System Call	Purpose
`open()`	Open a file and return a file descriptor
`close()`	Close a file descriptor
`read()`	Read bytes from a file descriptor
`write()`	Write bytes to a file descriptor
`lseek()`	Reposition the file offset
`stat()`	Retrieve file metadata
`unlink()`	Remove a directory entry (delete a file)

Directory operations:

System Call	Purpose
`mkdir()`	Create a directory
`rmdir()`	Remove an empty directory
`opendir()` / `readdir()`	Iterate over directory entries
`chdir()`	Change the working directory

Inter-process communication:

System Call	Purpose
`pipe()`	Create a unidirectional data channel
`mmap()`	Map a file or device into memory
`shmget()` / `shmat()`	System V shared memory
`socket()` / `bind()` / `listen()` / `accept()`	Network communication

The fork-exec Model

The Unix process creation model is distinctive: creating a new process is separated into two steps.

1. `fork()` creates an exact copy of the calling process. Both parent and child resume execution at the instruction following the `fork()` call. The only difference is the return value: `fork()` returns 0 in the child and the child's PID in the parent.
2. `exec()` replaces the calling process's memory image with a new program loaded from disk. The process ID does not change – the same process is now running a different program.

This two-step design is elegant because it allows the parent to modify the child's environment between `fork()` and `exec()` – for example, redirecting file descriptors to implement shell I/O redirection, changing the working directory, setting environment variables, or modifying signal handlers.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
```

```
        /* Child process */
        printf("Child: my PID is %d, parent PID is %d\n",
               getpid(), getppid());

        /* Replace this process with /bin/ls */
        execlp("ls", "ls", "-l", "/tmp", NULL);

        /* If exec returns, it failed */
        perror("execlp");
        exit(1);
    }

    /* Parent process */
    printf("Parent: created child with PID %d\n", pid);

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Parent: child exited with status %d\n",
               WEXITSTATUS(status));
    }

    return 0;
}
```

Modern implementations of `fork()` use *copy-on-write* (COW) semantics: the parent and child initially share the same physical memory pages, marked read-only. Only when either process writes to a page is a private copy made. This makes `fork()` fast even for processes with large address spaces, because the copying is deferred until (and unless) it is needed.

Programmer's Perspective

Go deliberately avoids exposing `fork()` directly because goroutines and the Go runtime's thread model make `fork()` semantics dangerous. When you `fork()` a multi-threaded process, only the calling thread survives in the child – all other threads vanish, potentially leaving mutexes locked and data structures in inconsistent states. The Go runtime is inherently multi-threaded (the garbage collector, the network poller, and the timer goroutine all run on separate OS threads), so `fork()` would leave the child in an unrecoverable state. Instead, Go provides `os/exec.Command()` which internally uses a combination of `clone()` (on Linux) or `posix_spawn()` to safely create child processes. If you need low-level control, the `syscall` package exposes `syscall.ForkExec()`, which atomically forks and execs in a single operation, avoiding the hazardous window between `fork()` and `exec()`.

File Descriptors: The Universal Handle

In Unix, a *file descriptor* is a small non-negative integer that serves as a handle to an open file, pipe, socket, or device. File descriptors are per-process: each process maintains a table mapping integers to kernel file objects.

By convention, every process starts with three open file descriptors:

FD	Name	C Constant	Purpose
0	Standard input	`STDIN_FILENO`	Input stream (keyboard by default)
1	Standard output	`STDOUT_FILENO`	Output stream (terminal by default)
2	Standard error	`STDERR_FILENO`	Error stream (terminal by default)

The beauty of this design is *uniformity*. The `read()` and `write()` system calls work identically regardless of whether the file descriptor refers to a regular file, a pipe, a network socket, or a hardware device. This is the Unix philosophy of "everything is a file" – or more precisely, "everything is a file descriptor."

This uniformity has profound consequences for software composition. The shell command `sort < input.txt | uniq > output.txt` works because:

1. `sort` reads from FD 0 (redirected to `input.txt` by the shell).
2. `sort` writes to FD 1 (connected to `uniq`'s FD 0 by a pipe).
3. `uniq` reads from FD 0 (the pipe) and writes to FD 1 (redirected to `output.txt`).

Neither `sort` nor `uniq` needs to know anything about files, pipes, or I/O redirection. They simply read from FD 0 and write to FD 1. The shell arranges the plumbing using `fork()`, `exec()`, `pipe()`, and `dup2()`.

```c
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

int main(void) {
    const char *msg = "Hello from file descriptor 1\n";

    /* Write to stdout using the raw system call */
    write(STDOUT_FILENO, msg, strlen(msg));

    /* Open a file -- returns the lowest available FD */
    int fd = open("/tmp/test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd >= 0) {
        const char *data = "Written via file descriptor\n";
        write(fd, data, strlen(data));
        close(fd);
    }

    return 0;
}
```

Beyond POSIX: Linux-Specific Extensions

While POSIX provides a portable baseline, Linux extends it significantly with system calls that have no POSIX equivalent:

- `epoll` – scalable I/O event notification (replaces `select`/`poll` for high-connection-count servers). Can monitor millions of file descriptors efficiently using a kernel-internal red-black tree.
- `io_uring` – asynchronous I/O with shared ring buffers between kernel and user space. Allows submitting and completing I/O operations with zero system calls

in the steady state (the kernel and user space communicate through shared memory queues).

- `clone()` – fine-grained process/thread creation (more flexible than `fork()`). The caller specifies exactly which resources to share with the child: memory, file descriptors, signal handlers, PID namespace, network namespace, etc.
- `namespaces` and `cgroups` – the building blocks of containers. Namespaces provide isolated views of system resources (PID, network, filesystem, user IDs). Cgroups limit, account for, and isolate resource usage (CPU, memory, I/O).
- `seccomp` – system call filtering for sandboxing. A process can install a BPF filter that restricts which system calls it (and its children) can invoke, reducing the attack surface.
- `bpf()` – programmable in-kernel virtual machine for tracing, networking, and security. eBPF programs are verified for safety before execution and run at near-native speed.

These extensions are not portable to other Unix systems, but they enable performance and functionality that POSIX alone cannot provide.

The System Call Interface in Practice

Tracing System Calls with strace

The `strace` utility intercepts and records every system call made by a process. It is the single most useful tool for understanding the interaction between a program and the OS kernel.

```c
/* trace_demo.c -- a minimal program to demonstrate strace */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

int main(void) {
    /* This single line generates multiple system calls:
     * write() to stdout, and internally fstat() to check
     * if stdout is a terminal (for buffering decisions). */
    printf("Hello, OS Theory!\n");
```

```
    /* Explicit file I/O */
    int fd = open("/tmp/os_demo.txt",
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    const char *data = "Operating Systems Theory\n";
    write(fd, data, strlen(data));
    close(fd);

    return 0;
}
```

Compiling and tracing this program:

```
$ gcc -o trace_demo trace_demo.c
$ strace -e trace=open,openat,read,write,close ./trace_demo

openat(AT_FDCWD, "/tmp/os_demo.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "Hello, OS Theory!\n", 18)     = 18
write(3, "Operating Systems Theory\n", 25) = 25
close(3)                                = 0
+++ exited with 0 +++
```

Each line shows: the system call name, its arguments, and its return value. This reveals exactly how the program interacts with the kernel – information that is invisible from source code alone.

Programmer's Perspective

When you use `strace` to trace a Go program, you will see Linux-specific system calls that the Go runtime uses internally. For example, `futex()` for goroutine synchronisation, `clone()` for creating OS threads, `epoll_wait()` for the network poller, and `mmap()`/`madvise()` for memory management. Running `strace -c ./myprogram` gives you a summary of all system calls and their frequency – an invaluable tool for understanding where your program spends its time in the kernel. Try it on a simple Go HTTP server and you will see that `epoll_wait` dominates, as the runtime's netpoller waits for incoming connections. The `-f` flag traces all child threads, which is essential for Go programs since the runtime spawns multiple OS threads. The `ltrace` tool is the user-space counterpart, tracing library calls rather than system calls. Together, `strace` and `ltrace` give you a complete picture of the boundary between your code and the OS.

System Call Overhead

System calls are not free. Each invocation incurs overhead from:

1. **Mode switch cost.** The `SYSCALL`/`SYSRET` instruction pair on x86-64 takes approximately 50–150 cycles.
2. **TLB and cache effects.** Entering the kernel may pollute the L1 instruction cache and TLB with kernel code and data, displacing user-space entries.
3. **Security mitigations.** On systems with Meltdown/Spectre mitigations enabled (KPTI – Kernel Page Table Isolation), each mode switch requires flushing and reloading page tables, adding 200–500 cycles.
4. **Argument validation.** The kernel must check every pointer and length argument for validity before using it.
5. **Auditing and security hooks.** If SELinux, AppArmor, or other Linux Security Modules (LSMs) are active, each system call triggers security policy checks.

The following table shows approximate costs for common system calls on a modern x86-64 Linux system (single-core, no contention):

System Call	Approximate Latency
`getpid()`	50–100 ns (cached, vDSO-optimised)
`gettimeofday()`	15–30 ns (vDSO, no kernel entry)

System Call	Approximate Latency
`read()` (cached data)	200–500 ns
`write()` (buffered)	200–500 ns
`open()`	1–5 μs (path resolution, permission checks)
`fork()`	50–200 μs (depends on address space size)
`mmap()`	1–10 μs

Note

Linux's vDSO (virtual Dynamic Shared Object) is a clever optimisation that maps a small kernel-provided shared library into every process's address space. Functions like `gettimeofday()` can read the current time from a memory-mapped kernel page without ever entering kernel mode, reducing their cost from hundreds of nanoseconds to tens. The vDSO contains read-only data that the kernel updates on every timer tick, and a handful of functions that can operate on this data entirely in user space. The `clock_gettime()`, `getcpu()`, and `gettimeofday()` functions are all vDSO-accelerated on Linux.

The Cost-Benefit Calculus

Understanding system call overhead informs practical design decisions. Consider a program that reads a file byte by byte:

```c
/* Slow: one system call per byte */
char c;
while (read(fd, &c, 1) == 1) {
    process(c);
}
```

If the file contains n bytes, this issues n system calls, each costing approximately 300 ns. For a 1 MB file ($n = 10^6$), the overhead alone is 300 milliseconds – entirely dominated by mode-switch costs.

The solution is *buffered I/O*: read large blocks and process them in user space.

```c
/* Fast: one system call per 4096 bytes */
char buf[4096];
ssize_t n;
while ((n = read(fd, buf, sizeof(buf))) > 0) {
```

```
    for (ssize_t i = 0; i < n; i++) {
        process(buf[i]);
    }
}
```

Now the same 1 MB file requires only 256 system calls ($10^6/4096 \approx 256$), reducing overhead to approximately 77 microseconds – a 4000x improvement. This is precisely why the C standard library's `stdio.h` (`fread`, `fgets`, `fprintf`) performs user-space buffering automatically.

$$\text{overhead} = n_{\text{syscalls}} \times t_{\text{syscall}} = \left\lceil \frac{\text{file size}}{\text{buffer size}} \right\rceil \times t_{\text{syscall}}$$

As the buffer size increases, the overhead decreases hyperbolically. But beyond a point (typically 4–16 KB, matching the OS page size and disk sector size), the benefit of larger buffers diminishes because the per-byte transfer cost dominates and cache locality degrades.

The Kernel's Execution Model

A common misconception is that the kernel is a separate program running alongside user processes. In reality, the kernel has no dedicated process or thread of its own (with minor exceptions for kernel worker threads). Instead, the kernel executes *on behalf of* user processes.

When process P makes a system call, P's thread transitions to kernel mode and executes kernel code. From the scheduler's perspective, the thread is still "P's thread" – it simply has elevated privileges. The kernel's code runs in P's context, using P's kernel stack.

Similarly, when a hardware interrupt arrives while process P is running, P's thread is commandeered to execute the interrupt handler. After the handler completes, control may return to P or switch to a different process if the scheduler so decides.

This model has important implications:

- The kernel does not consume CPU time independently; its time is charged to the processes that invoke it. In `top` or `htop`, the "system" CPU percentage reflects time spent in kernel mode on behalf of user processes.
- Kernel code must be reentrant: multiple threads may be executing kernel code simultaneously on different cores. Shared kernel data structures must be protected by locks or other synchronisation mechanisms.

- Kernel code must never block indefinitely while holding a spinlock, because doing so would prevent other threads from making progress on that core (and potentially cause deadlock if another core needs the same lock).
- The kernel stack is a limited resource – typically 8 KB or 16 KB per thread on Linux. Kernel functions must be conservative with stack usage; deep recursion or large stack-allocated buffers can overflow the kernel stack, causing a kernel panic.

```
KERNEL EXECUTION MODEL
──────────────────────────────────────────────────────────────

 Core 0                    Core 1
 ────────                  ────────

 Process A (user mode)     Process B (user mode)
     │                         │
     │ system call             │ interrupt
     ▼                         ▼
 Process A (kernel mode)   Process B (kernel mode)
     │ executing sys_read      │ executing IRQ handler
     │                         │
     ▼                         ▼
 Process A (user mode)     Process C (user mode)
                           [scheduler chose C]
──────────────────────────────────────────────────────────────
```

There is one important exception to the "kernel runs on behalf of processes" model: **kernel threads** (kthreads). These are threads created by the kernel itself, with no associated user-space process. They perform background tasks like:

- `kswapd` – the swap daemon, which proactively moves pages to swap when memory pressure is high.
- `ksoftirqd` – processes deferred interrupt work (softirqs) when the softirq load is too high to handle in interrupt context.
- `kworker` – worker threads that execute asynchronous work items queued by drivers and other kernel subsystems.
- `migration` – handles CPU migration of threads for load balancing.

You can see kernel threads in `ps aux` – they appear with names in square brackets (e.g., `[kswapd0]`, `[ksoftirqd/0]`).

Design Goals and Trade-offs

Operating system designers face fundamental trade-offs that have no universally correct resolution. Every OS reflects a particular set of priorities.

Correctness vs Performance

A correct but slow OS is useless in practice; a fast but buggy OS is dangerous. The tension manifests in decisions like:

- **Locking granularity.** A single global lock (the "Big Kernel Lock" that Linux used until version 2.6.39) is simple and correct but serialises all kernel operations. Fine-grained locking allows concurrency but introduces the risk of deadlocks and race conditions. Linux's evolution from the Big Kernel Lock to per-subsystem locks to per-object locks is a case study in this trade-off.
- **Bounds checking.** Validating every array index and pointer dereference prevents buffer overflows but adds overhead. Most production kernels check only at system call boundaries and trust internal kernel code.
- **Defensive programming.** Adding assertions, sanity checks, and redundant validation catches bugs early but costs CPU cycles. Linux's `CONFIG_DEBUG_*` options enable extensive checking in development builds but are disabled in production.

Generality vs Specialisation

A general-purpose OS (Linux, Windows) must handle everything from embedded devices to supercomputers. A specialised OS (RTOS, unikernel) can be optimised for a single workload but is useless outside that niche.

Linux is the supreme example of generality: the same kernel runs on smartphones (Android), supercomputers (100% of the Top500), cloud servers, routers, televisions, and automobiles. This generality comes at a cost: the kernel must support thousands of device drivers, dozens of file systems, and multiple scheduling policies, all of which add complexity and code size.

Security vs Usability

Strict security policies (mandatory access control, capability-based addressing) make systems harder to use and administer. Permissive defaults (Windows XP running everything as Administrator) make systems easy to use but vulnerable. The challenge

is finding the right default: secure enough to prevent common attacks, permissive enough to not frustrate users.

Compatibility vs Innovation

Maintaining backward compatibility constrains design: x86-64 still supports real-mode instructions from the 8086 era, and Linux still supports the 32-bit `INT 0x80` system call path. Breaking compatibility enables cleaner designs but strands existing software. Linus Torvalds has famously declared "we do not break user space" as Linux's cardinal rule, even when the existing interface is acknowledged to be poorly designed.

Programmer's Perspective

These trade-offs are not abstract philosophical questions – they affect your daily work. When you choose between Go's `os.ReadFile()` (simple but loads the entire file into memory) and `bufio.Scanner` (more complex but memory-efficient), you are making the same correctness-vs-performance trade-off that OS designers face. When you choose between `sync.Mutex` (simple, correct, potentially slow under contention) and `sync/atomic` operations (fast but error-prone), you are navigating the same locking granularity trade-off. When you decide whether to use cgo for performance-critical code (breaking Go's cross-compilation guarantee) or stay in pure Go (portable but potentially slower), you are confronting the compatibility-vs-innovation trade-off. OS design principles are software engineering principles at the most demanding scale.

Summary

An operating system is simultaneously a resource manager, an abstraction provider, and a protection boundary. It evolved from non-existent (bare hardware) through batch monitors, multiprogramming systems, and time-sharing systems to the complex, multi-subsystem kernels we use today. The CPU's dual-mode operation – kernel mode and user mode – provides the hardware foundation for protection, and system calls provide the controlled gateway between the two modes. The POSIX standard codifies the system call interface, enabling portable software across Unix-like systems. Understanding the OS at this level – not just as a black box that runs programs, but as a carefully engineered piece of software that mediates every interaction between your code and the hardware – is the foundation for everything that follows in this book.

Exercises

Exercise 1.1: Role Classification

Classify each of the following OS activities as primarily resource management, abstraction provision, or protection enforcement. Some may involve more than one role; justify your reasoning for each.

a) The scheduler selects a new process to run after a time quantum expires.

b) The virtual memory system translates virtual address 0x7FFE4000 to physical address 0x1A3C000.

c) The file system returns `EACCES` when a process tries to open a file it does not have permission to read.

d) The network stack fragments a 5000-byte message into multiple Ethernet frames.

e) The OOM (Out of Memory) killer terminates a process to free memory when RAM is exhausted.

f) The VFS layer allows the same `read()` system call to work on ext4 files, NFS mounts, and `/proc` virtual files.

g) Linux cgroups limit a container to using at most 2 CPU cores and 4 GB of RAM.

Exercise 1.2: State Machine Model

Consider an OS managing two processes, P1 and P2, on a single-core CPU. The system state includes: which process is running, the PC (program counter) value for each process, and the contents of a single shared variable x (initially 0).

a) Define the state space Q formally, specifying the type and range of each component.

b) List the possible events Σ (assume: P1 can increment x, P2 can decrement x, and a timer interrupt can cause a context switch).

c) Write the transition function δ for the timer interrupt event. Be precise about what state changes and what is preserved.

d) Prove that the invariant "exactly one process is in the running state at any time" is preserved by all transitions.

e) Is the property "x is always non-negative" a safety property, a liveness property, or neither? Justify your answer and determine whether this property

holds for all possible executions.

Exercise 1.3: System Call Cost Analysis

A program reads a 10 MB file using `read()` system calls. Each system call has a fixed overhead of 400 ns (mode switch + validation) plus a variable cost of 10 ns per byte transferred (memory copy).

a) Calculate the total time to read the file with a buffer size of 1 byte.

b) Calculate the total time with a buffer size of 4096 bytes.

c) Calculate the total time with a buffer size of 1 MB.

d) Derive a formula for total time T as a function of buffer size B, and find the buffer size that minimises T. Explain why the optimal buffer size in practice is often close to the page size (4096 bytes) despite the formula suggesting a larger value.

e) How does the analysis change if the file data is not in the page cache and must be read from an NVMe SSD with a 10 μs latency per I/O request?

Exercise 1.4: fork-exec Implementation

Write a C program that implements a simple shell command: `cat file1 file2 > output`. Your program must:

a) Use `fork()` to create a child process.

b) In the child, redirect standard output to the file `output` (using `open()` and `dup2()`).

c) In the child, use `execlp()` to run `cat` with the given arguments.

d) In the parent, wait for the child to finish and print its exit status.

e) Explain why the file descriptor manipulation must happen between `fork()` and `exec()`, and what would go wrong if it happened before `fork()`.

f) Explain why `dup2(fd, STDOUT_FILENO)` is used instead of simply closing FD 1 and opening the output file (relying on `open()` to return the lowest available FD). What race condition could the simpler approach introduce in a multi-threaded program?

Exercise 1.5: Historical Analysis

Compare the process creation models of Multics and Unix.

a) Multics used a single `create_process()` call that specified the program to run, the initial arguments, and the execution environment. Unix uses the two-step `fork()`/`exec()` model. What are the advantages and disadvantages of each approach?

b) The Plan 9 operating system (also from Bell Labs) replaced `fork()` with `rfork()`, which allows fine-grained control over which resources are shared between parent and child (memory, file descriptors, namespace). Linux's `clone()` serves a similar purpose. Explain why this finer granularity is useful, giving at least two concrete examples.

c) Modern systems like Windows use `CreateProcess()`, which is closer to the Multics model. Why might a modern OS designer prefer this over `fork()`/`exec()`? Consider the implications for multi-threaded programs and the performance of `fork()` on systems with large address spaces.

Exercise 1.6: POSIX Portability

A developer writes a Linux application that uses `epoll` for network event notification, `io_uring` for asynchronous file I/O, and `clone()` with `CLONE_NEWNET` for network namespace isolation.

a) Which of these APIs are POSIX-compliant? Which are Linux-specific?

b) For each Linux-specific API, identify the closest POSIX-compliant alternative and explain what functionality would be lost.

c) The developer wants to port the application to FreeBSD. For each Linux-specific API, describe the FreeBSD equivalent (if one exists) and the porting effort required.

d) Design an abstraction layer (a set of C function signatures) that would allow the application to use the optimal platform-specific API on each OS while presenting a uniform interface to the application code.

Exercise 1.7: Protection Boundary Analysis

Consider a system without hardware-enforced kernel/user mode separation (i.e., all code runs at the same privilege level, as on some early microcontrollers).

a) Can such a system still implement an operating system? What properties would it lack compared to a system with hardware protection?

b) Describe a software-only mechanism that could provide *some* isolation between processes on such a system. What are its limitations? (Hint: consider

language-based protection, as used in Singularity OS or Java's JVM.)

c) The WebAssembly (Wasm) runtime provides memory isolation without hardware privilege rings. Explain how Wasm achieves this and compare its isolation guarantees to those provided by hardware-enforced virtual memory.

d) Some real-time operating systems (e.g., FreeRTOS on Cortex-M0, which lacks an MPU) run all tasks in the same address space. Analyse the security implications and explain what mitigations are available.

Chapter 2: Architectural Patterns

The previous chapter established what an operating system does. This chapter examines how it is structured internally – the architectural decisions that determine where kernel code lives, how components communicate, and what belongs inside the trusted computing base. These decisions have profound consequences for performance, security, reliability, and the ability to formally verify correctness. No single architecture dominates; each pattern represents a different point in a vast design space, optimised for different constraints.

We begin with the fundamental question – what code should run in kernel mode? – and then systematically examine five architectural patterns: monolithic kernels, microkernels, hybrid kernels, exokernels, and unikernels. For each, we describe the structure, analyse the trade-offs, and examine real systems that embody the pattern. We conclude with a rigorous comparative analysis that equips you to reason about architectural choices for any given deployment scenario.

The Fundamental Question: What Belongs in the Kernel?

Every OS architecture must answer one question: which code runs in kernel mode (ring 0, EL1) with full hardware access, and which code runs in user mode with restricted privileges?

Code that runs in kernel mode has two properties:

1. **Power.** It can execute privileged instructions, access any memory address, and manipulate hardware directly.
2. **Risk.** A bug in kernel-mode code can crash the entire system, corrupt arbitrary memory, or create security vulnerabilities exploitable by any process.

The tension between power and risk drives all architectural decisions. More code in the kernel means more power and potentially more performance (fewer mode transitions), but also more risk. Less code in the kernel means less risk but more overhead from crossing the user/kernel boundary.

We define the *Trusted Computing Base* (TCB) as the set of all code that must be correct for the system's security properties to hold.

Note

Trusted Computing Base (TCB). The TCB of a system is the totality of hardware, firmware, and software components that are critical to its security. A vulnerability in any TCB component can compromise the entire system. A smaller TCB is easier to audit, test, and formally verify, and presents a smaller attack surface. The TCB includes not only the kernel but also the bootloader, firmware, hypervisor (if present), and any user-space code that runs with elevated privileges (e.g., setuid binaries).

The size of the TCB varies dramatically across architectures:

Architecture	Approximate TCB Size	Example
Monolithic kernel	15–30 million LoC	Linux 6.x
Hybrid kernel	5–10 million LoC	Windows NT kernel
Microkernel	10,000–50,000 LoC	seL4, L4
Exokernel	5,000–20,000 LoC	MIT Exokernel
Unikernel	Application-specific	MirageOS, IncludeOS

To appreciate the significance of these numbers, consider that the probability of a security-critical bug is roughly proportional to code size. A system with 30 million lines of privileged code has approximately 3000x the attack surface of a system with 10,000 lines – a qualitative, not merely quantitative, difference.

Monolithic Kernels

Architecture

In a monolithic kernel, the entire operating system – process scheduler, memory manager, file systems, device drivers, network stack, and security modules – runs in a single address space in kernel mode. All components can call each other directly through function calls, share data structures without serialisation, and access hardware without any indirection.

```
MONOLITHIC KERNEL ARCHITECTURE
──────────────────────────────────────────────────────────────


User Space
```

```
┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
│ Process A│ │ Process B│ │ Process C│ │ Process D│
└──────────┘ └──────────┘ └──────────┘ └──────────┘
     │            │            │            │
═══ │═══════════ │═══════════ │═══════════ │═══════════
     │     System Call Interface (single entry point)
     ▼            ▼            ▼            ▼
┌──────────────────────────────────────────────────┐
│                  KERNEL SPACE                    │
│                                                  │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐         │
│  │ Scheduler│ │ Memory Mgr│ │ VFS Layer│         │
│  └──────────┘ └──────────┘ └──────────┘         │
│                                   │              │
│                                   │              │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐         │
│  │ Net Stack│ │  IPC     │ │ File Sys │         │
│  │ (TCP/IP) │ │          │ │ ext4/btrfs│         │
│  └──────────┘ └──────────┘ └──────────┘         │
│                                                  │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐         │
│  │ Disk Driver│ │ NIC Driver│ │ USB Driver│         │
│  └──────────┘ └──────────┘ └──────────┘         │
└──────────────────────────────────────────────────┘
                     Hardware
```

The term “monolithic” is somewhat misleading. It does not mean the kernel is an undifferentiated mass of code. Internally, the kernel is highly modular, with well-defined interfaces between subsystems. The term means that all modules share the same address space and the same privilege level – a bug in any module can corrupt any other module’s data.

Linux: The Canonical Monolithic Kernel

Linux is the most prominent monolithic kernel in production use. As of version 6.x, the Linux kernel comprises approximately 28 million lines of code (including all drivers and architectures). The kernel source tree is organised into major subsystems:

Directory	Subsystem	Approximate Size
`kernel/`	Core (scheduler, signals, timers)	150,000 LoC
`mm/`	Memory management	120,000 LoC
`fs/`	File systems (ext4, btrfs, XFS, NFS, ...)	1,200,000 LoC
`net/`	Networking (TCP/IP, netfilter, ...)	900,000 LoC
`drivers/`	Device drivers	18,000,000+ LoC
`arch/`	Architecture-specific code (x86, ARM, ...)	1,500,000 LoC
`security/`	Security modules (SELinux, AppArmor, ...)	200,000 LoC
`crypto/`	Cryptographic algorithms	100,000 LoC
`sound/`	Audio subsystem (ALSA)	600,000 LoC

The `drivers/` directory alone accounts for over 60% of the kernel's total size. This is characteristic of monolithic kernels: the vast majority of code is device drivers, and all of it runs with full kernel privileges.

The internal structure of Linux demonstrates that "monolithic" does not mean "unstructured." The kernel enforces a layered architecture through conventions and code review:

- The **VFS layer** provides a uniform file system interface, insulating upper layers from file system implementation details.
- The **block layer** provides a uniform block device interface, insulating file systems from device driver details.
- The **netdev layer** provides a uniform network interface, insulating protocol stacks from NIC driver details.

However, these layers are enforced by convention, not by hardware. Any kernel code can bypass any layer and access any data structure directly, because all code shares the same address space.

Loadable Kernel Modules

Pure monolithic design would require recompiling the entire kernel to add support for a new device. Linux solves this with *loadable kernel modules* (LKMs) – object files that can be inserted into and removed from the running kernel without rebooting.

A kernel module is compiled against the kernel headers and loaded with the `insmod` or `modprobe` command. Once loaded, the module's code runs in kernel space with full privileges, indistinguishable from statically compiled kernel code. The module infrastructure provides:

- **Symbol export** – modules can export symbols (functions and variables) for other modules to use.
- **Dependency resolution** – `modprobe` automatically loads prerequisite modules.
- **Reference counting** – prevents unloading a module that is in use.
- **Version checking** – prevents loading a module compiled against a different kernel version (vermagic).

```c
/* hello_module.c -- a minimal Linux kernel module */
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("OS Theory");
MODULE_DESCRIPTION("A minimal kernel module");
MODULE_VERSION("1.0");

static int __init hello_init(void) {
    pr_info("hello: module loaded (kernel version %s)\n",
            UTS_RELEASE);
    return 0;   /* 0 = success, negative = error code */
}

static void __exit hello_exit(void) {
    pr_info("hello: module unloaded\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

The Makefile for building this module:

```
obj-m += hello_module.o

KDIR := /lib/modules/$(shell uname -r)/build

all:
    make -C $(KDIR) M=$(PWD) modules

clean:
    make -C $(KDIR) M=$(PWD) clean
```

Building and loading:

```
$ make
$ sudo insmod hello_module.ko
$ dmesg | tail -1
[12345.678] hello: module loaded (kernel version 6.5.0-generic)
$ sudo rmmod hello_module
$ dmesg | tail -1
[12345.789] hello: module unloaded
```

Programmer's Perspective

Writing kernel modules is one of the few programming activities where a bug can instantly crash the entire machine. There is no memory protection between your module and the rest of the kernel – a null pointer dereference in a module triggers a kernel panic, not a segfault. A buffer overflow in a module can corrupt the scheduler's run queue, the page cache, or the file system's inode table. This is why kernel development requires extreme discipline: rigorous code review (every patch to the Linux kernel is reviewed by at least two maintainers), static analysis tools (`sparse`, `smatch`, `coccinelle`), and thorough testing in virtual machines before deployment. The `pr_info()` function is the kernel equivalent of `printf` – its output goes to the kernel log buffer, readable via `dmesg`. Note that kernel code cannot use floating-point arithmetic: the kernel does not save/restore FPU state on every kernel entry (for performance reasons), so using floating-point instructions would corrupt user-space FPU registers. The `printk` format specifier `%pK` prints kernel pointers with hashing to prevent address leaks to unprivileged users.

Advantages of Monolithic Kernels

Performance. Internal function calls within the kernel are simple C function calls – no IPC overhead, no serialisation, no context switches. A file system can call the mem-

ory manager directly; the network stack can call the scheduler directly. On a hot path like packet reception, this directness matters enormously. A Linux networking stack can process millions of packets per second, partly because each packet traverses the stack through direct function calls rather than IPC messages.

Simplicity of internal interfaces. Kernel subsystems share data structures directly. The page cache, for example, is accessible to both the file system and the memory manager without any marshalling. The `struct page` data structure, which represents a physical page frame, is used by the memory allocator, the page cache, the swap subsystem, and the DMA subsystem – all directly, with no copying or serialisation.

Mature ecosystem. Linux has drivers for virtually every hardware device in existence, built over three decades of development by thousands of contributors. The `drivers/` directory contains support for devices ranging from IBM mainframe channel adapters to Raspberry Pi GPIO pins.

Debugging and tracing infrastructure. Because all kernel code runs in the same address space, tools like ftrace, perf, and eBPF can trace any kernel function, measure any code path, and correlate events across subsystems – capabilities that are much harder to achieve in a microkernel where subsystems run in separate address spaces.

Disadvantages of Monolithic Kernels

Large TCB. Every line of kernel code is in the TCB. A buffer overflow in an obscure USB driver can compromise the entire system. A 2011 study by Chou et al. found that Linux device drivers had an error rate 3–7x higher than the rest of the kernel. Since drivers comprise the majority of kernel code and run with full privileges, they are the dominant source of kernel crashes.

Fault propagation. A bug in any kernel component can corrupt shared data structures, causing failures that manifest in unrelated subsystems. A memory corruption bug in a network driver might cause a page cache corruption that crashes the file system hours later. Debugging such failures is notoriously difficult because the symptom is far removed from the cause.

Difficult to verify. Formal verification of 28 million lines of C code is computationally and intellectually infeasible with current techniques. Even partial verification of individual subsystems is extremely challenging because of the unrestricted interactions between kernel components.

Kernel-mode attack surface. Every system call, every device driver, and every kernel module is a potential entry point for an attacker. A single exploitable bug anywhere in the kernel gives the attacker full system access.

Info

The Linux kernel's quality is maintained not by formal verification but by an extraordinarily rigorous development process: every patch is reviewed by subsystem maintainers, tested by automated CI systems (the kernel test robot), run through static analysers, and deployed incrementally through the stable/longterm release cycle. This process catches most bugs but cannot guarantee their absence – a fundamental limitation of testing-based quality assurance.

Microkernels

Architecture

A microkernel takes the opposite approach: *minimise* the code running in kernel mode. The kernel provides only the most fundamental services:

1. **Address space management** – creating and manipulating page tables.
2. **Thread management** – creating, scheduling, and destroying threads.
3. **Inter-process communication (IPC)** – sending messages between processes.
4. **Basic hardware access** – interrupt forwarding and device capability management.

Everything else – file systems, device drivers, the network stack, even parts of the memory manager – runs as user-space servers that communicate through IPC messages.

```
MICROKERNEL ARCHITECTURE
────────────────────────────────────────────────────────────


User Space
┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│ Process A│  │ Process B│  │ File Sys │  │ Net Stack│
└──────────┘  └──────────┘  └──────────┘  └──────────┘
     │            │            │            │
     │     ┌──────┴────────────┴────────────┘
     │     │     IPC Messages
     │     │
┌────┴─────┴──────────────────────────────────────────┐
│ Disk     │ NIC      │ USB      │ Display  │ Timer    │
│ Driver   │ Driver   │ Driver   │ Driver   │ Driver   │
│ (user)   │ (user)   │ (user)   │ (user)   │ (user)   │
```

```
└──────┴──────┴──────┴──────┴──────┴──────┘
   │      │      │      │      │
═══╪══════╪══════╪══════╪══════╪═══════════
   ▼      ▼      ▼      ▼      ▼

┌──────────────────────────────────────────┐
│            MICROKERNEL (ring 0)          │
│                                          │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐  │
│  │ Address  │ │ Thread   │ │   IPC    │  │
│  │ Spaces   │ │ Scheduling│ │ Mechanism│  │
│  └──────────┘ └──────────┘ └──────────┘  │
└──────────────────────────────────────────┘

                  Hardware
──────────────────────────────────────────────
```

The microkernel philosophy is summarised by the *minimality principle*: a service should be moved to user space unless moving it would prevent the implementation of the system's required functionality. This principle was articulated by Jochen Liedtke, the architect of the L4 microkernel family.

The IPC Problem

The critical design challenge in microkernels is IPC performance. In a monolithic kernel, a file `read()` is a direct function call to the file system code, costing perhaps 50 nanoseconds of overhead. In a microkernel, the same operation requires multiple IPC messages:

1. The application sends a "read" message to the file system server (user $\rightarrow$ kernel $\rightarrow$ user).
2. The file system server sends a "read block" message to the disk driver (user $\rightarrow$ kernel $\rightarrow$ user).
3. The disk driver programs the hardware and waits for a completion interrupt.
4. The disk driver sends the data back to the file system server (user $\rightarrow$ kernel $\rightarrow$ user).
5. The file system server sends the data back to the application (user $\rightarrow$ kernel $\rightarrow$ user).

Each arrow represents a mode transition and a potential context switch. If each IPC costs 1 microsecond, a single file read requires 4 IPC round trips = 8 mode transitions, adding 4–8 microseconds of overhead that does not exist in a monolithic design.

Let T_{mono} be the cost of a file read in a monolithic kernel, and T_{micro} be the cost in a

microkernel:

$$T_{\text{mono}} = T_{\text{syscall}} + T_{\text{fs}} + T_{\text{io}}$$

$$T_{\text{micro}} = T_{\text{syscall}} + T_{\text{fs}} + T_{\text{io}} + k \times T_{\text{ipc}}$$

where k is the number of IPC messages required and T_{ipc} is the per-message IPC cost. The overhead ratio is:

$$\text{overhead} = \frac{T_{\text{micro}} - T_{\text{mono}}}{T_{\text{mono}}} = \frac{k \times T_{\text{ipc}}}{T_{\text{syscall}} + T_{\text{fs}} + T_{\text{io}}}$$

For a cached file read (no disk I/O), $T_{\text{io}} = 0$ and the denominator is small, making the relative overhead large. For a read that actually hits disk ($T_{\text{io}} \gg k \times T_{\text{ipc}}$), the IPC overhead is negligible.

This observation leads to an important insight: **microkernel overhead is significant for fast-path operations but negligible for slow-path operations**. Since disk and network I/O dominate real workloads, the practical impact of IPC overhead is often smaller than micro-benchmarks suggest.

First-Generation Microkernels: Mach

Mach, developed at Carnegie Mellon University in the 1980s, was the first widely deployed microkernel. It provided virtual memory management, IPC, and thread scheduling in the kernel, with file systems, device drivers, and network stacks running as user-space servers.

Mach's IPC was designed for generality: messages could contain arbitrary data, port rights (capabilities), and out-of-line memory regions. This flexibility came at a cost – each IPC operation required:

- Marshalling the message into a kernel-managed buffer
- Copying data from sender to kernel to receiver (double copy)
- Scheduling the receiver to run
- Returning the result through the same path

The result was approximately 100x overhead compared to a monolithic kernel function call. This performance gap was the central criticism of microkernels in the early 1990s and fuelled the famous Tanenbaum-Torvalds debate.

L4: High-Performance Microkernels

Jochen Liedtke's L4 microkernel (1993) demonstrated that the IPC overhead problem was not inherent to the microkernel concept but rather a consequence of poor implementation in Mach. Liedtke redesigned IPC from first principles:

- **Synchronous IPC.** The sender blocks until the receiver accepts the message, eliminating the need for kernel-managed message buffers. The message exists only in CPU registers and the kernel stack during the transfer.
- **Direct process switch.** When process A sends a message to process B, the kernel switches directly from A to B without going through the scheduler. This halves the number of context switches and eliminates scheduling overhead.
- **Register-based short messages.** Small messages (up to a few machine words) are passed in CPU registers, avoiding memory copies entirely. On x86, the `EAX`, `EBX`, `ECX`, etc. registers carry the message payload.
- **Typed transfers for large data.** Large transfers use *map* and *grant* operations that modify page table entries rather than copying data. The sender can map a page into the receiver's address space with a single TLB operation, achieving zero-copy semantics.
- **Minimal kernel.** The L4 microkernel is approximately 10,000 lines of code, compared to Mach's hundreds of thousands. Less code means fewer cache misses in the IPC path.

These optimisations reduced IPC latency from approximately 100 microseconds (Mach on i486) to approximately 5 microseconds (L4 on i486) – a 20x improvement that brought microkernel IPC within an order of magnitude of a function call.

Note

Modern L4 descendants (seL4, Fiasco.OC, NOVA) achieve IPC latencies of 200–400 nanoseconds on current x86-64 hardware. While still 2–4x more expensive than a monolithic kernel function call on the same hardware, this overhead is acceptable for most workloads, especially when weighed against the security and reliability benefits. The key insight from L4 is that IPC performance is an engineering challenge, not a fundamental limitation of the microkernel architecture.

seL4: The Formally Verified Microkernel

seL4, developed at NICTA (now Data61/CSIRO) in Australia, is the world's first operating system kernel with a complete, machine-checked proof of functional correctness.

The proof establishes that:

1. The C implementation of seL4 correctly implements its abstract specification (refinement proof).
2. The binary code produced by the compiler correctly implements the C source (translation validation via a separate tool).
3. The kernel enforces the security properties defined by its access control model (integrity and confidentiality proofs).

The proof covers approximately 10,000 lines of C code and required roughly 200,000 lines of Isabelle/HOL proof script. The verification effort took approximately 20 person-years.

Info

seL4 Functional Correctness Theorem. Let S_{abs} be the abstract specification of seL4 (a state machine in Haskell), let S_C be the C implementation, and let S_{bin} be the compiled binary. Then for all possible executions: $\text{behaviour}(S_{\text{bin}}) \subseteq \text{behaviour}(S_C) \subseteq \text{behaviour}(S_{\text{abs}})$. That is, the binary can only exhibit behaviours that are permitted by the C source, which in turn can only exhibit behaviours permitted by the abstract specification. This is a *refinement* relation: each layer refines (implements faithfully) the layer above it.

This result means that seL4 is provably free of:

- Buffer overflows
- Null pointer dereferences
- Arithmetic overflows (in security-critical paths)
- Use-after-free bugs
- Memory leaks (in kernel objects)
- Deadlocks (in the kernel itself)
- Information leaks between isolated domains (integrity and confidentiality)

No monolithic kernel can make these guarantees because formal verification of millions of lines of code remains intractable. The seL4 team estimates that verifying the Linux kernel, if the same proof-to-code ratio applied, would require approximately 560,000 person-years.

seL4 uses a *capability-based* access control model. Each kernel object (thread, address space, endpoint, page frame) is referenced through a *capability* – a kernel-managed, unforgeable token that grants specific access rights. A thread can only access objects for which it holds a capability, and capabilities can only be created through explicit kernel operations (not by guessing addresses).

Minix 3: Reliability Through Isolation

Minix 3, developed by Andrew Tanenbaum (who also created the original Minix that inspired Linux), is a microkernel-based system designed for extreme reliability. Its key innovation is *automatic driver recovery*:

1. Each device driver runs as an isolated user-space process with minimal privileges. The driver process has access only to the I/O ports and memory regions required for its specific device, enforced by the kernel.
2. A *reincarnation server* monitors all driver processes via heartbeat messages. If a driver fails to respond to a heartbeat within a timeout, the reincarnation server considers it crashed.
3. If a driver crashes (segfault, infinite loop detected by watchdog, etc.), the reincarnation server automatically restarts it by spawning a new instance and re-initialising the device.
4. The restarted driver re-initialises its device and resumes operation, often without the user noticing any interruption beyond a brief pause.

This self-healing property is impossible in a monolithic kernel where a driver crash corrupts kernel data structures. In Minix 3, the crashed driver's memory is simply reclaimed by the kernel, leaving all other system components – including other drivers, the file system, and the network stack – unaffected.

Minix 3 enforces the *principle of least authority* (POLA): each system component has only the privileges it needs to perform its function, and no more. A printer driver has access to the printer's I/O ports but cannot touch the network card's registers or the disk controller's DMA buffers. This containment ensures that even a malicious driver (not just a buggy one) cannot compromise other system components.

Advantages of Microkernels

Small TCB. Only 10,000–50,000 lines of code run in kernel mode, dramatically reducing the attack surface and making formal verification feasible.

Fault isolation. A bug in a user-space driver or server cannot corrupt kernel data structures. The worst case is that a single server crashes and can be restarted.

Security. The principle of least privilege is enforced architecturally: drivers and servers have only the capabilities they need, not full kernel access. Even a compromised driver cannot read other processes' memory or escalate privileges.

Modularity. System components can be independently developed, tested, and replaced. You can swap the file system server without rebooting. You can run two

different file system implementations simultaneously and compare their behaviour.

Verifiability. A kernel of 10,000 lines can be formally verified, as demonstrated by seL4. This provides mathematical guarantees of correctness that no amount of testing can achieve.

Disadvantages of Microkernels

IPC overhead. Even with optimised IPC, the cost of crossing the user/kernel boundary multiple times per operation adds latency and reduces throughput for fast-path operations.

Complexity of user-space servers. Moving functionality out of the kernel does not eliminate complexity – it redistributes it. User-space servers must handle their own error recovery, resource management, and concurrency. A user-space file system server is not simpler than a kernel-space file system – it is merely isolated.

Ecosystem maturity. Microkernel-based systems have fewer drivers, applications, and developers than Linux. This is a self-reinforcing problem: fewer users means less driver support, which means fewer users.

Performance tuning difficulty. Optimising a multi-server system requires understanding IPC patterns, server scheduling, and cache behaviour across multiple address spaces – more complex than optimising a single kernel address space.

Hybrid Kernels

Architecture

A hybrid kernel attempts to combine the performance of a monolithic kernel with the modularity of a microkernel. The strategy is to run most OS services in kernel mode (like a monolithic kernel) but to structure them as separate modules with well-defined interfaces (like a microkernel). Some services – particularly drivers that are unreliable or infrequently used – may run in user space.

```
HYBRID KERNEL ARCHITECTURE
──────────────────────────────────────────────────────────────


User Space
┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
│ Process A│ │ Process B│ │ User-mode│ │ Subsystem│
│          │ │          │ │ Driver   │ │ Server   │
└────┬─────┘ └────┬─────┘ └────┬─────┘ └────┬─────┘
```

```
     |           |           |           |
═══  |           |           |           |  ═══
     ▼           ▼           ▼           ▼

┌──────────────────────────────────────────────────┐
│                   KERNEL SPACE                   │
│  ┌──────────────────────────────────────────────┐ │
│  │           Executive / Core Services          │ │
│  │  ┌──────────┐ ┌──────────┐ ┌──────────┐      │ │
│  │  │Scheduler │ │ VMM      │ │ IPC      │      │ │
│  │  └──────────┘ └──────────┘ └──────────┘      │ │
│  └──────────────────────────────────────────────┘ │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐          │
│  │ File Sys │ │ Net Stack│ │ Drivers  │          │
│  │ (kernel) │ │ (kernel) │ │ (kernel) │          │
│  └──────────┘ └──────────┘ └──────────┘          │
└──────────────────────────────────────────────────┘

                      Hardware
═══════════════════════════════════════════════════════
```

The label “hybrid” is somewhat contentious. Critics argue that most hybrid kernels are essentially monolithic kernels with a microkernel-inspired internal structure – the fact that services run in kernel mode means they share all the security and reliability problems of monolithic kernels. Proponents counter that the internal modularity provides engineering benefits (cleaner interfaces, easier testing) even without hardware-enforced isolation.

Windows NT

The Windows NT kernel (used in all modern Windows versions from Windows NT 3.1 through Windows 11) is the most commercially successful hybrid kernel. Its architecture was designed by Dave Cutler, who previously led the VMS operating system project at Digital Equipment Corporation. The architecture has two layers:

The NT Kernel (ntoskrnl.exe) runs in kernel mode and provides:

- Thread scheduling and synchronisation primitives (dispatcher objects)
- Interrupt and exception dispatching (IDT management, DPC – Deferred Procedure Calls)
- Low-level hardware abstraction (via the HAL – Hardware Abstraction Layer, a replaceable DLL)
- Trap handling and system call dispatching

The Executive sits above the kernel (but still in kernel mode) and provides higher-level services:

- **Object Manager** – a unified namespace for kernel objects (processes, threads, files, mutexes, events, sections). Every kernel resource is an "object" with a type, a security descriptor, and a reference count.
- **I/O Manager** – a layered driver model based on I/O Request Packets (IRPs). An I/O request is encapsulated in an IRP that passes through a stack of driver layers (file system driver, volume manager, disk driver), each processing and forwarding it.
- **Memory Manager** – virtual memory, section objects (memory-mapped files), page file management, working set trimming.
- **Configuration Manager** – the Windows Registry, a hierarchical key-value database for system and application configuration.
- **Security Reference Monitor** – access control, token management, security auditing. Every object access is checked against the caller's security token and the object's DACL (Discretionary Access Control List).
- **Process Manager** – process and thread creation, job objects (groups of processes with shared resource limits).
- **Cache Manager** – unified cache for file data, accessed by the Memory Manager and the I/O Manager.

The "hybrid" designation comes from the fact that several subsystems run in user mode as *environment subsystems*:

- The Windows subsystem (`csrss.exe`) manages the Win32 API and console windows.
- The Session Manager (`smss.exe`) manages login sessions and subsystem processes.
- User-mode drivers via the UMDF (User-Mode Driver Framework) – USB cameras, printers, and other non-performance-critical devices can be implemented as user-mode services, gaining the fault isolation benefits of a microkernel.

Windows NT was originally designed with a more microkernel-like structure (influenced by Mach), but over successive versions, more components were moved into kernel mode for performance. The most notable migration was the graphics subsystem (`win32k.sys`), moved from user space to kernel space in Windows NT 4.0. This improved GUI performance significantly but introduced a large, complex, and historically bug-prone component into the TCB. The `win32k.sys` driver has been one of the most frequent sources of Windows kernel vulnerabilities.

macOS / XNU

Apple's XNU kernel (the basis for macOS, iOS, iPadOS, watchOS, and tvOS) is a hybrid that combines three heritage components:

- **Mach microkernel** – provides IPC (Mach messages and ports), virtual memory management, and basic thread scheduling. Mach ports are the fundamental IPC mechanism: every system service, including the file system and the window server, is accessed through Mach port messages.
- **BSD layer** – provides the POSIX API: processes, file systems (HFS+, APFS), networking (TCP/IP, Unix domain sockets), and security (sandboxing, code signing). This layer runs in kernel mode alongside Mach, not as a separate user-space server.
- **I/O Kit** – an object-oriented, restricted C++ (called Embedded C++) driver framework. I/O Kit uses a publish-subscribe model for device matching: when a new device appears, the I/O Kit finds and loads the appropriate driver based on a matching dictionary in the driver's Info.plist.

```
XNU KERNEL STRUCTURE
────────────────────────────────────────────────────────────────────────

User Space     Applications, Frameworks, daemons

═══════════════════════════════════════════════════════════════════════

Kernel Space
 ┌──────────────────────────────────────────────────────────────┐
 │                     BSD Layer                                 │
 │    (POSIX API, VFS, TCP/IP, security, processes)              │
 ├──────────────────────────────────────────────────────────────┤
 │                     Mach Layer                                │
 │    (IPC, virtual memory, thread scheduling)                   │
 ├──────────────────────────────────────────────────────────────┤
 │                     I/O Kit                                   │
 │    (driver framework, device tree, power mgmt)                │
 ├──────────────────────────────────────────────────────────────┤
 │           Platform Expert / HAL                               │
 └──────────────────────────────────────────────────────────────┘
                         Hardware
────────────────────────────────────────────────────────────────────────
```

XNU is structured internally as a microkernel (Mach provides the primitive abstractions), but the BSD layer and I/O Kit run in the same address space as Mach, making it functionally monolithic from a privilege perspective. A bug in the BSD network stack can corrupt Mach data structures, and vice versa.

Apple has increasingly moved toward user-space drivers with *DriverKit* (introduced in macOS 10.15), which provides a user-space driver framework with a subset of I/O Kit's API. This is a gradual migration toward microkernel-like fault isolation for drivers, without changing the kernel architecture.

Exokernels

Architecture

The exokernel, proposed by Dawson Engler at MIT in the mid-1990s, takes a radical approach: the kernel should *not* provide abstractions at all. Instead, it should only *securely multiplex hardware* – dividing physical resources among applications – and let each application define its own abstractions through a *library OS* linked into its address space.

Traditional operating systems impose policies: a single page replacement algorithm, a single TCP congestion control algorithm, a single file cache eviction strategy. These policies represent compromises that work reasonably well for average workloads but are suboptimal for specific applications. The exokernel philosophy is that the kernel should not impose policies; it should only provide mechanisms for secure resource sharing, and let applications choose their own policies.

```
EXOKERNEL ARCHITECTURE
──────────────────────────────────────────────────────────────────


┌──────────────────────┐  ┌──────────────────────┐
│     Application A    │  │     Application B    │
│  ┌────────────────┐  │  │  ┌────────────────┐  │
│  │ Library OS     │  │  │  Library OS       │  │
│  │ (custom FS,    │  │  │ (custom net,     │  │
│  │  custom VM,    │  │  │  custom sched,   │  │
│  │  custom net)   │  │  │  standard FS)    │  │
│  └────────────────┘  │  │  └────────────────┘  │
└──────────────────────┘  └──────────────────────┘
           │                         │
═══════════╪═══════════════════════════╪═══════════
```

```
          ▼                         ▼
 ┌──────────────────────────────────────────────────┐
 │                  EXOKERNEL                       │
 │                                                  │
 │  ┌──────────┐  ┌──────────┐  ┌──────────┐       │
 │  │ Secure   │  │ Physical │  │ Network  │       │
 │  │ Bindings │  │ Memory   │  │ Multiplex│       │
 │  │          │  │ Multiplex│  │          │       │
 │  └──────────┘  └──────────┘  └──────────┘       │
 └──────────────────────────────────────────────────┘
                     Hardware
 ──────────────────────────────────────────────────────────
```

Secure Bindings

The exokernel's key mechanism is the *secure binding*: a kernel-level binding that authorises an application to use a specific physical resource (a page of RAM, a disk block, a network packet filter) without going through the kernel on every access.

Three types of secure bindings:

1. **Hardware mechanisms.** The exokernel programs the hardware (TLB entries, page tables, network packet filters) on behalf of the application. Once the binding is installed, the application accesses the resource directly through hardware, with no kernel involvement.

2. **Software caching.** The exokernel caches authorisation decisions (e.g., "this application is allowed to access disk block 1234") to avoid re-checking on every access.

3. **Downloaded code.** The application downloads a small, verified code fragment (similar to a packet filter) into the kernel, which the kernel executes on the application's behalf. This is remarkably similar to modern eBPF.

For example, to allocate a physical memory page:

1. The application requests page frame number 0x1A3C from the exokernel.
2. The exokernel checks the application's capabilities and, if authorised, grants ownership.
3. The application's library OS installs a page table entry mapping a virtual address to physical frame 0x1A3C.
4. All subsequent accesses to that page go directly through the hardware MMU – no kernel involvement.

The exokernel only intervenes when resources are allocated, revoked, or when access violations occur.

Library Operating Systems

The library OS is a user-space library that provides traditional OS abstractions (file systems, network protocols, virtual memory policies) tailored to the application's needs. Different applications can use different library OSes:

- A database server might use a library OS with a custom buffer cache optimised for B-tree access patterns, evicting pages based on the B-tree structure rather than LRU.
- A web server might use a library OS with a custom TCP stack optimised for short-lived connections, eliminating the TIME_WAIT state.
- A scientific application might use a library OS with a custom memory allocator optimised for large matrix operations, using huge pages and NUMA-aware placement.

This per-application customisation is the exokernel's greatest strength. In a traditional OS, all applications share the same policies. With an exokernel, each application can choose the policies that best match its workload.

Performance Results and Legacy

The MIT Exokernel project (Aegis/Xok, 1995–1998) demonstrated impressive performance gains:

- A custom library OS file system (XN) achieved 5–8x the throughput of a traditional Unix file system for database workloads.
- A custom library OS network stack achieved 3–5x the throughput of the BSD TCP/IP stack for web server workloads.
- Application-level virtual memory management reduced page fault handling time by 2–4x.

These gains come not from any inherent speed advantage of the exokernel design but from the ability to *specialise* policies for specific workloads – something traditional OS abstractions prevent.

No production exokernel system is in widespread use today, but the exokernel philosophy has influenced modern systems in important ways:

- **DPDK and SPDK** bypass the Linux kernel's network and storage stacks, giving applications direct access to NIC and NVMe hardware through

user-space drivers. This is essentially an exokernel approach applied to specific subsystems.

- **Linux's `io_uring`** gives applications direct access to kernel I/O queues via shared memory rings, eliminating system call overhead in the steady state – a form of secure binding.
- **XDP/eBPF** allows applications to download code into the kernel for packet processing, mimicking the exokernel's downloaded-code mechanism.

Programmer's Perspective

The exokernel philosophy manifests in modern Go systems programming in a subtle way. When you use `syscall.Mmap` to memory-map a file directly, bypassing Go's `os.File` abstraction and the standard library's buffered I/O, you are doing something exokernel-like: taking direct control of the memory-to-file mapping, choosing your own access patterns and caching strategy instead of relying on the OS's page cache and Go's `bufio` layer. DPDK-based Go networking libraries (like Google's `gVisor` netstack) bypass the kernel's TCP/IP stack entirely, implementing the protocol in user space for maximum performance and flexibility. These are exokernel ideas applied pragmatically within the Linux ecosystem. The lesson of the exokernel is not that you should replace Linux with an exokernel – it is that abstraction has a cost, and when that cost matters, you should be able to reach through the abstraction to the hardware.

Unikernels

Architecture

A unikernel is a specialised, single-address-space machine image constructed by compiling an application together with only the OS library components it needs, linked directly against the hardware abstraction layer. The result is a bootable image that runs a single application with no distinction between kernel mode and user mode, no process isolation, and no multi-user support.

```
UNIKERNEL vs TRADITIONAL OS
────────────────────────────────────────────────────────────────


Traditional OS Stack:            Unikernel:

┌──────────────────────┐         ┌──────────────────────┐
│   Application        │         │                      │
```

```
├──────────────────────┤          │   Application      │
│   Language Runtime   │          │   + OS Libraries   │
├──────────────────────┤          │   + Runtime        │
│   Std C Library      │          │   (single binary)  │
├──────────────────────┤          │                    │
│   System Calls       │          └────────┬───────────┘
├──────────────────────┤                   │
│   OS Kernel          │          ─────────┼──────────
├──────────────────────┤                   │
│   Hypervisor / HW    │          ┌────────┴───────────┐
└──────────────────────┘          │  Hypervisor / HW   │
                                  └────────────────────┘

  Multiple layers,                  Single layer,
  general purpose,                  specialised,
  large attack surface              minimal attack surface
```

How Unikernels Work

The build process for a unikernel:

1. The developer writes application code in a high-level language (OCaml for MirageOS, C/C++ for IncludeOS, Rust for Tock, Haskell for HaLVM).
2. A build tool analyses the application's dependencies and selects only the required OS library components. If the application does not use the file system, the file system library is not linked. If it does not use networking, the TCP/IP stack is omitted.
3. The application and selected libraries are compiled and linked into a single binary image, with a minimal hardware abstraction layer providing boot code, interrupt handling, and device access.
4. The image is bootable on a hypervisor (Xen, KVM, VMware, QEMU) or, in some cases, bare metal.

The resulting image is typically 1–10 MB in size, compared to hundreds of megabytes for a traditional OS installation. Boot times are measured in milliseconds rather than seconds.

MirageOS

MirageOS is an OCaml-based unikernel framework developed at the University of Cambridge. The application and all OS libraries are written in OCaml, a type-safe functional language with strong static guarantees. This means that entire classes of bugs – buffer overflows, use-after-free, null pointer dereferences, type confusion – are eliminated by the type system at compile time, without requiring a runtime privilege boundary.

A MirageOS unikernel for a simple web server includes:

- The application code (~500 lines of OCaml)
- The HTTP library (cohttp, ~3,000 lines)
- The TCP/IP stack (mirage-tcpip, ~8,000 lines)
- The Xen block/network device drivers (~2,000 lines)
- The OCaml runtime (~5,000 lines)

Total: approximately 18,000 lines of code – compared to millions of lines for a Linux VM running the same application.

MirageOS uses OCaml's module system to parameterise the application over its dependencies. A network application is written against an abstract `NETWORK` module type; at build time, the developer chooses a concrete implementation (Xen netfront, Unix socket, or a simulated network for testing). This *functorial* abstraction makes unikernels testable on a Unix host before deployment on a hypervisor.

IncludeOS

IncludeOS is a C/C++ unikernel designed for cloud services. It provides a POSIX-compatible API subset, making it possible to run existing C/C++ applications with minimal modification. Key characteristics:

- Boot time: 300 microseconds to first instruction (vs 5–30 seconds for a full Linux VM)
- Image size: 1–5 MB typical (vs 200+ MB for a minimal Linux installation)
- Memory footprint: 5–50 MB typical (vs 200+ MB for a minimal Linux VM)
- Networking: custom TCP/IP stack with zero-copy packet handling

IncludeOS demonstrates that the unikernel approach is not limited to exotic functional languages – it can be applied to mainstream C/C++ codebases with moderate engineering effort.

Use Cases

Unikernels are suited for:

- **Cloud microservices** – each service is a single-purpose VM, isolated by the hypervisor. The minimal image size enables rapid scaling: starting a new instance takes milliseconds, not minutes.
- **Network functions** – firewalls, load balancers, DNS servers that need minimal latency and maximal throughput. ClickOS (a unikernel for middlebox processing) can boot in 30 milliseconds and process 10 Gbps of network traffic.
- **IoT devices** – resource-constrained environments where a full OS is too heavy. A unikernel running on a microcontroller can provide a TCP/IP stack and application logic in under 1 MB.
- **Security-critical appliances** – the minimal attack surface is attractive for devices exposed to hostile networks. No shell, no SSH daemon, no package manager means no secondary attack vectors.

Advantages and Disadvantages

Advantages:

- Tiny attack surface: no shell, no package manager, no unused services, no login mechanism.
- Fast boot: millisecond-scale, enabling rapid scaling and live migration.
- Small image size: efficient storage, network transfer, and cache utilisation.
- Strong isolation: each unikernel is a separate VM, isolated by the hypervisor.
- Language-level safety (for type-safe unikernels like MirageOS and HaLVM).
- Deterministic behaviour: no background services, no cron jobs, no unexpected processes.

Disadvantages:

- No runtime debugging tools: no shell, no `strace`, no `gdb`, no `tcpdump`. Debugging requires serial console logging, specialised trace points, or solo5/hvt-based debugging frameworks.
- Single application per VM: no multi-process, no multi-user, no general-purpose computing.
- Limited hardware support: typically runs on hypervisors, not bare metal (though some unikernels support bare-metal deployment on specific hardware).
- Immutable deployment: changing the application requires rebuilding and redeploying the entire image. There is no `apt install` or `dnf update`.
- Ecosystem immaturity: limited library support compared to Linux.

Programmer's Perspective

Unikernels represent an extreme point in the design space, but their ideas are infiltrating mainstream systems. Go's static linking model produces self-contained binaries that include the runtime and garbage collector – conceptually similar to a unikernel that includes its own memory manager. Container images built with `FROM scratch` in a Containerfile (for Podman) contain only the application binary and its direct dependencies – no shell, no package manager, no init system. This "distroless" approach captures many unikernel benefits (small attack surface, fast startup, deterministic behaviour) while retaining the familiar container orchestration tooling (Kubernetes, Nomad). When you build a Go microservice as `CGO_ENABLED=0 go build -o /app` and package it in a scratch container, you are essentially building a unikernel that runs on a container runtime instead of a hypervisor. The mental model is identical: a single application, statically linked, with no external dependencies, isolated by the platform.

Comparative Analysis

Performance Comparison

The performance comparison between architectures depends heavily on the workload:

Operation	Monolithic	Microkernel	Hybrid	Exokernel	Unikernel
System call overhead	Baseline	2–4x baseline	~1x baseline	0.5x baseline	0x (no syscall)
IPC latency	N/A (function call)	200–500 ns	N/A (function call)	N/A	N/A
File I/O (cached)	Fast	Moderate	Fast	Very fast (custom)	Fast
Network throughput	Good	Moderate	Good	Excellent (custom)	Excellent
Boot time	5–30 s	2–10 s	10–60 s	1–5 s	0.001–0.5 s

Operation	Monolithic	Microkernel	Hybrid	Exokernel	Unikernel
Context switch	1–5 μs	2–8 μs	1–5 μs	1–3 μs	N/A (single app)

For general-purpose workloads, monolithic and hybrid kernels dominate because their internal function-call-based communication avoids IPC overhead. For specialised workloads (high-frequency trading, network functions, embedded systems), exokernels and unikernels can outperform traditional kernels by eliminating unnecessary abstraction layers.

Security Comparison

Property	Monolithic	Microkernel	Hybrid	Exokernel	Unikernel
TCB size	Very large	Very small	Large	Small	Minimal
Driver isolation	None	Full (user space)	Partial	Application-level	N/A
Formal verification	Infeasible	Demonstrated (seL4)	Infeasible	Partially feasible	Language-dependent
Attack surface	Large	Small	Large	Small	Minimal
Privilege escalation risk	High	Low	High	Medium	N/A (single priv)

Complexity and Development Effort

Property	Monolithic	Microkernel	Hybrid	Exokernel	Unikernel
Kernel complexity	High	Low	High	Low	Low
User-space complexity	Low	High	Medium	Very high	Application-level

Property	Monolithic	Microkernel	Hybrid	Exokernel	Unikernel
Driver development	In-kernel (risky)	User-space (safe)	Mixed	Library OS	Linked in
Debugging difficulty	Moderate	Easy (user-space)	Moderate	Hard	Hard
Ecosystem maturity	Excellent	Limited	Good	Minimal	Growing

A Formal Comparison: TCB and Failure Probability

Let p be the probability of a bug per line of code, and let n be the number of lines in the TCB. If we assume bugs are independently distributed (a simplification), the probability of at least one bug in the TCB is:

$$P(\text{at least one bug}) = 1 - (1-p)^n \approx 1 - e^{-pn} \quad \text{for small } p$$

For a monolithic kernel with $n = 20 \times 10^6$ LoC and an industry-average defect density of $p = 10^{-3}$ (one bug per thousand lines):

$$P_{\text{mono}} \approx 1 - e^{-10^{-3} \times 20 \times 10^6} = 1 - e^{-20000} \approx 1$$

For a microkernel with $n = 10^4$ LoC:

$$P_{\text{micro}} \approx 1 - e^{-10^{-3} \times 10^4} = 1 - e^{-10} \approx 0.99995$$

Both are near certainty – which is why simply reducing TCB size is insufficient. The real value is that a microkernel's smaller TCB is *verifiable*. seL4's formal proof drives p effectively to zero for the verified code, yielding $P_{\text{seL4}} = 0$ for the proven properties.

But the story does not end with the kernel. The total system reliability depends on the reliability of *all* components, not just the kernel. In a microkernel system, a buggy user-space file system server can still lose your data – it just cannot crash the kernel. The microkernel's contribution is *fault containment*: a bug in one component cannot propagate to unrelated components.

The expected number of *system-wide crashes* (crashes that affect all running applications) is:

$$E[\text{crashes/year}]_{\text{mono}} = \lambda_{\text{kernel}} + \lambda_{\text{drivers}}$$

$$E[\text{crashes/year}]_{\text{micro}} = \lambda_{\text{kernel}} \quad (\text{driver crashes are contained})$$

where λ_{kernel} and λ_{drivers} are the failure rates of the kernel and drivers, respectively. Since $\lambda_{\text{drivers}} \gg \lambda_{\text{kernel}}$ (drivers have 3–7x higher bug density and constitute the majority of kernel code), the microkernel's elimination of driver-induced system crashes is a dramatic reliability improvement.

Note

This probabilistic model is highly simplified. In practice, bugs cluster in complex code paths, and the assumption of independence is unrealistic. Nonetheless, the model illustrates the fundamental relationship between TCB size and reliability: all else being equal, a smaller TCB has fewer bugs, and fault containment prevents individual bugs from causing system-wide failures.

Case Study: The Tanenbaum-Torvalds Debate

In January 1992, Andrew Tanenbaum (author of Minix) and Linus Torvalds (creator of Linux) engaged in a famous Usenet debate on comp.os.minix about OS architecture. Tanenbaum argued that monolithic kernels were obsolete and that microkernels were the future. Torvalds countered that monolithic kernels were more practical and performant.

Tanenbaum's key arguments:

1. Microkernels are more modular, reliable, and maintainable.
2. Moving to microkernels is an inevitable trend, just as structured programming replaced goto-based code.
3. Linux's monolithic design is "a giant step back into the 1970s."
4. Portability requires a microkernel (Minix ran on multiple architectures; early Linux was x86-only).

Torvalds' key counter-arguments:

1. Mach-based microkernels (the only practical ones at the time) were demonstrably slower.
2. Monolithic kernels with loadable modules provide sufficient modularity.

3. Practical engineering trumps theoretical elegance – Linux works, and works well.
4. Portability is achievable in a monolithic kernel (Linux later ported to dozens of architectures).

Three decades later, the debate remains unresolved. Linux dominates servers, desktops, embedded systems, and supercomputers (100% of the Top500 as of 2024). seL4 dominates high-assurance systems where formal verification is required (military systems, autonomous vehicles, medical devices). Hybrid kernels (Windows, macOS) dominate consumer desktops and mobile devices. Each architecture thrives in its niche.

The real lesson of the debate is that the "best" architecture depends on the deployment context. There is no universal winner – only trade-offs.

Programmer's Perspective

Understanding kernel architecture is not merely academic – it affects the tools and techniques available to you as a systems programmer. On Linux (monolithic), you can write kernel modules and eBPF programs that run inside the kernel, use ftrace and perf for system-wide tracing, and leverage 30 years of driver support. On microkernels, your "kernel extensions" are regular user-space processes that you can debug with `gdb`, test with standard unit testing frameworks, and restart without rebooting. If you are building a high-assurance system (medical devices, avionics, autonomous vehicles), you should seriously consider seL4 as the foundation – its formal proof provides guarantees that no amount of Linux kernel testing can match. If you are building cloud infrastructure, Linux's maturity, driver support, and container ecosystem (cgroups, namespaces, seccomp) are unmatched. As a practical exercise, try building the Linux kernel from source (`make defconfig && make -j$(nproc)`) and booting it in QEMU – the process reveals the monolithic kernel's complexity (thousands of configuration options) and its flexibility (runs on everything from smartwatches to supercomputers).

Emerging Patterns

Multi-kernel (Barrelfish)

The Barrelfish OS, developed at ETH Zurich and Microsoft Research, treats a multi-core machine as a distributed system. Each core runs its own kernel instance (a "CPU driver"), and cores communicate through explicit message passing rather than

shared memory. This design avoids the cache coherence overhead of shared-memory synchronisation on NUMA architectures and scales better on machines with hundreds of cores.

Barrelfish maintains a *system knowledge base* – a Prolog-like constraint database that stores information about the hardware topology (NUMA distances, cache sharing, interrupt routing) and uses it to make placement decisions. This is a fundamentally different approach to hardware abstraction: instead of hiding hardware details behind uniform interfaces, Barrelfish exposes them to the OS through a queryable knowledge base.

OS as a Library (Dune, gVisor)

The "OS as a library" approach runs a library implementation of OS abstractions in a hardware-isolated environment. Intel's VT-x virtualisation extensions enable a process to run in its own hardware-isolated domain (VMX non-root mode) with its own page tables, exception handlers, and I/O access, while still being managed by the host OS.

Dune (from Stanford) uses this approach to give applications direct, safe access to hardware features normally reserved for the kernel: page tables, exceptions, and privilege rings. A Dune application can implement its own page fault handler, garbage collector, or sandboxing mechanism using hardware privilege rings, without modifying the host kernel.

Google's gVisor is a user-space kernel that intercepts and reimplements Linux system calls. It provides a subset of the Linux syscall interface, implementing it in a memory-safe language (Go), and runs as a regular process on the host OS. Container workloads run on gVisor instead of directly on the host kernel, providing an additional layer of isolation: even if an application exploits a kernel vulnerability, it only compromises the gVisor process, not the host kernel.

eBPF: Programmable Kernel Extensions

eBPF (extended Berkeley Packet Filter) allows user-space programs to load small, verified programs into the Linux kernel. The eBPF verifier statically analyses each program to ensure it terminates, does not access invalid memory, and does not violate security policies. Once verified, the program runs in kernel mode at near-native speed.

eBPF blurs the monolithic/microkernel boundary: the base kernel is monolithic, but user-defined extensions are sandboxed and verified, similar to the safety guarantees of a microkernel's user-space servers. eBPF is used for:

- **Networking** – XDP (eXpress Data Path) programs process packets at the driver level, before the kernel's network stack.
- **Tracing** – kprobes, tracepoints, and uprobes for dynamic kernel and user-space tracing.
- **Security** – LSM (Linux Security Module) hooks for custom security policies.
- **Scheduling** – sched_ext allows user-space scheduling policies loaded via eBPF.

```c
/* Simplified XDP program that drops ICMP packets */
#include <linux/bpf.h>
#include <linux/if_ether.h>
#include <linux/ip.h>

SEC("xdp")
int xdp_drop_icmp(struct xdp_md *ctx) {
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    /* Parse Ethernet header */
    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)
        return XDP_PASS;
    if (eth->h_proto != __constant_htons(ETH_P_IP))
        return XDP_PASS;

    /* Parse IP header */
    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end)
        return XDP_PASS;

    /* Drop ICMP packets */
    if (ip->protocol == IPPROTO_ICMP)
        return XDP_DROP;

    return XDP_PASS;
}

char _license[] SEC("license") = "GPL";
```

This XDP program runs in the kernel at the earliest point of packet reception, before the normal networking stack processes the packet. It can drop, redirect, or modify packets at wire speed – a capability that previously required custom kernel modules

or dedicated hardware.

Summary

The choice of OS architecture is the most consequential design decision in systems software. Monolithic kernels like Linux pack everything into a single privileged address space, maximising performance at the cost of a massive TCB. Microkernels like seL4 minimise the privileged code to a verifiable core, achieving provable correctness at the cost of IPC overhead. Hybrid kernels like Windows NT and XNU attempt a pragmatic middle ground, keeping performance-critical services in the kernel while allowing some components to run in user space. Exokernels eliminate OS-imposed abstractions, giving applications direct control over hardware at the cost of increased application complexity. Unikernels collapse the application and OS into a single bootable image, achieving minimal footprint and attack surface at the cost of generality. Each architecture represents a different set of priorities – performance, security, reliability, flexibility, verifiability – and the right choice depends entirely on the constraints of the target domain.

Exercises

Exercise 2.1: Architecture Classification

For each of the following systems, identify the kernel architecture (monolithic, microkernel, hybrid, exokernel, or unikernel) and justify your classification with at least two structural arguments:

a) A system where the file system, network stack, and all device drivers run as user-space processes, and the kernel provides only IPC, scheduling, and address space management.

b) A system where a web server application is compiled together with a TCP/IP stack and a network driver into a single bootable image that runs on a hypervisor.

c) A system where the kernel provides only hardware multiplexing, and each application links against its own library that implements file system and networking abstractions.

d) A system where all OS services run in kernel mode but are structured as separate modules that communicate through well-defined interfaces, with some infrequently used drivers running in user mode.

e) A system where the kernel contains the scheduler, memory manager, file system, network stack, and 5000 device drivers, all compiled into a single binary that runs in ring 0.

Exercise 2.2: IPC Overhead Analysis

Consider a microkernel-based system where a user-space file system server handles file `read()` requests. Each IPC message costs $T_{\text{ipc}} = 400$ ns (including two mode transitions). A `read()` request requires 4 IPC messages (request to file server, file server to disk driver, disk driver reply, file server reply).

a) Calculate the IPC overhead for a single `read()` call.

b) If the same operation in a monolithic kernel costs 200 ns of overhead (a single function call chain), what is the overhead ratio (microkernel / monolithic)?

c) Now suppose the monolithic kernel experiences one kernel crash per 1000 hours of operation due to driver bugs, and each crash costs 5 minutes of downtime. The microkernel system experiences zero kernel crashes (bugs in user-space drivers cause only server restarts, taking 100 ms). If the system handles 10^8 `read()` calls per hour, calculate the total cost (time overhead + crash downtime) for each architecture over 1000 hours. Which architecture has lower total cost?

d) At what bug rate (crashes per hour) do the two architectures break even in total cost?

e) Extend the analysis to account for the value of data integrity: assume each kernel crash has a 1% probability of corrupting the file system, requiring a 1-hour fsck. How does this change the comparison?

Exercise 2.3: TCB Size and Verification

The seL4 microkernel has approximately 10,000 lines of C code, and its formal verification required approximately 200,000 lines of proof (a 20:1 proof-to-code ratio).

a) If the Linux kernel has 28 million lines of code and the same proof-to-code ratio applied, how many lines of proof would be needed to verify Linux? If a verification engineer can produce 10 lines of proof per day, how many person-years would the verification take?

b) Explain why the 20:1 ratio would likely be *higher* for Linux, not the same. Consider: the complexity of pointer aliasing in a large address space, the number of entry points, the diversity of hardware configurations, and the use of inline assembly.

c) Propose a strategy for partially verifying a monolithic kernel. Which components would you prioritise, and why? How would you define the interface between verified and unverified components?

d) seL4's proof assumes specific hardware behaviour (e.g., the MMU correctly translates addresses). What happens if this assumption is violated? Research one real hardware bug (e.g., Meltdown, Rowhammer) and explain how it interacts with seL4's correctness guarantees.

Exercise 2.4: Kernel Module Implementation

Write a Linux kernel module that:

a) Creates a `/proc/os_theory` entry that, when read, returns a string containing the current system uptime in seconds and the number of context switches since boot.

b) Implements proper cleanup (removing the `/proc` entry) when the module is unloaded.

c) Uses `pr_info()` to log a message each time the `/proc` file is read.

Provide the complete C source code and the `Makefile` needed to build it. Explain what would happen if your module contained a null pointer dereference: how would the system behave, and how does this differ from the same bug in a user-space program?

Exercise 2.5: Exokernel Design

Design a library OS for a key-value store application running on an exokernel. Your library OS must manage:

a) **Physical memory allocation.** Describe how the application would request physical pages from the exokernel, and how it would implement its own virtual-to-physical mapping for a hash table data structure. What advantage does controlling page placement give over the kernel's default allocation?

b) **Disk I/O.** Describe how the application would request direct access to disk blocks from the exokernel, and how it would implement its own on-disk format optimised for key-value storage. Compare the expected performance with that of a key-value store running on ext4.

c) **Networking.** Describe how the application would register a packet filter with the exokernel to receive only its own network traffic, and how it would implement a custom protocol optimised for key-value operations (avoiding the overhead of TCP for short request/response pairs).

Exercise 2.6: Unikernel Security Analysis

A company deploys a web application as a unikernel image on a cloud hypervisor. An attacker discovers a buffer overflow vulnerability in the application code.

a) Compare the impact of this vulnerability in the unikernel deployment versus a traditional deployment (application running on a full Linux OS). Consider: what can the attacker access, what persistent state exists, and what other services might be compromised.

b) The unikernel has no shell, no `ssh` daemon, and no package manager. Explain how this affects the attacker's ability to escalate the compromise (install backdoors, pivot to other systems, exfiltrate data).

c) Identify two security *disadvantages* of the unikernel approach compared to a traditional OS deployment. (Hint: consider Address Space Layout Randomisation in a single-address-space design, and the implications of running all code at the same privilege level.)

d) Propose a mitigation for each disadvantage you identified in part (c).

Exercise 2.7: Architecture Trade-off Matrix

You are designing operating systems for four different deployment scenarios. For each scenario, recommend a kernel architecture and justify your choice with at least three specific technical arguments:

a) A pacemaker implanted in a patient's chest, which must operate reliably for 10+ years without maintenance or software updates.

b) A cloud platform hosting thousands of microservices from different (potentially hostile) customers, where strong isolation and rapid scaling are required.

c) A high-frequency trading system where every microsecond of latency costs money, running a single application on dedicated hardware.

d) A general-purpose desktop operating system for consumers with diverse hardware (thousands of device types) and software needs (productivity, gaming, development).

Chapter 3: Hardware-Software Interface

The operating system does not float above the hardware as an independent entity. It is welded to the machine at dozens of contact points: privilege rings, interrupt lines, exception vectors, DMA channels, memory-mapped device registers, and timer circuits. Understanding these mechanisms at the hardware level is essential because the OS kernel is, at its core, a program that responds to hardware events. Every context switch, every page fault, every packet received from the network – all begin with a hardware signal that the kernel must handle correctly, quickly, and without losing any information. This chapter traces each of these mechanisms from the electrical signal to the kernel code that processes it.

We cover six major hardware mechanisms: CPU privilege rings that enforce the kernel/user boundary, interrupts that notify the CPU of external events, exceptions that handle synchronous error conditions, DMA that offloads bulk data transfers from the CPU, I/O buses that connect devices to the system, and timer hardware that drives preemptive scheduling. For each mechanism, we examine the hardware design, the software interface, and the OS design patterns that exploit them.

CPU Privilege Rings

The Need for Hardware-Enforced Privilege

Chapter 1 introduced the distinction between kernel mode and user mode. This section examines the hardware mechanisms that enforce it. The fundamental requirement is simple: user-space code must not be able to execute certain instructions (those that modify page tables, disable interrupts, or halt the processor), and any attempt to do so must be trapped by the hardware and reported to the kernel.

Without hardware enforcement, the OS would have to rely on software checks before every privileged operation – an approach that is both slow (every instruction would need validation) and insecure (a malicious program could jump directly to kernel code, bypassing the checks). Hardware enforcement is automatic, zero-cost on the common path (instructions that are permitted execute at full speed), and inescapable (there is no software trick that can bypass it).

The hardware mechanisms for privilege enforcement vary across processor architectures, but all share the same fundamental design: a processor status register encodes the current privilege level, and the instruction execution logic checks this level before executing any privileged operation.

x86 Protection Rings

The x86 architecture defines four privilege levels, numbered 0 (most privileged) through 3 (least privileged). These are encoded in the *Current Privilege Level* (CPL) field, which occupies the low two bits of the code segment register (CS).

```
x86 PROTECTION RINGS
────────────────────────────────────────────────────────────────────────


            ┌──────────────────────────────────┐
            │         Ring 3 (CPL=3)           │
            │      User Applications           │
            │                                  │
            │   ┌────────────────────────┐     │
            │   │     Ring 2 (CPL=2)     │     │
            │   │   Device Drivers (*)   │     │
            │   │                        │     │
            │   │  ┌──────────────────┐  │     │
            │   │  │  Ring 1 (CPL=1) │  │     │
            │   │  │  OS Services (*) │  │     │
            │   │  │                  │  │     │
            │   │  │ ┌──────────────┐ │  │     │
            │   │  │ │Ring 0        │ │  │     │
            │   │  │ │Kernel        │ │  │     │
            │   │  │ └──────────────┘ │  │     │
            │   │  └──────────────────┘  │     │
            │   └────────────────────────┘     │
            └──────────────────────────────────┘

  (*) Rings 1 and 2 are defined by the hardware but
      unused by all major operating systems.
      Linux and Windows use only Ring 0 and Ring 3.
────────────────────────────────────────────────────────────────────────
```

In practice, all major x86 operating systems use only two rings:

- **Ring 0** – kernel mode. The OS kernel, including all drivers and kernel modules,

runs here.

- **Ring 3** – user mode. All user-space applications run here.

Rings 1 and 2 were intended for OS services and device drivers respectively, providing a graduated trust model. However, no mainstream OS uses them for three reasons:

1. The overhead of ring transitions (similar to system call overhead) made the fine-grained ring model impractical. Two ring transitions per driver call (ring 3 $\rightarrow$ ring 1 $\rightarrow$ ring 0 and back) are more expensive than one (ring 3 $\rightarrow$ ring 0).
2. The x86-64 long mode simplifies the privilege model: the segment-based protection mechanisms of rings 1 and 2 are less relevant when paging provides the primary memory protection.
3. Virtualisation extensions (VT-x) added a separate "ring -1" (VMX root mode) for hypervisors, further reducing the need for intermediate rings.

The CPL is checked by the hardware on every memory access and instruction execution:

- If the CPL is 3 and the instruction is privileged (e.g., `CLI` to disable interrupts, `STI` to enable them, `HLT` to halt the processor, `LGDT` to load the GDT, `MOV CR3` to change page tables, `WRMSR` to write model-specific registers), the CPU raises a general protection fault (#GP, vector 13).
- If the CPL is 3 and the memory access targets a page marked as supervisor-only (the U/S bit in the page table entry is 0), the CPU raises a page fault (#PF, vector 14).
- If the CPL is 0, all instructions and all memory addresses are accessible.

Note

Current Privilege Level (CPL). The CPL is a 2-bit field stored in the low two bits of the CS (Code Segment) and SS (Stack Segment) registers. It indicates the privilege level of the currently executing code. On x86-64, CPL is 0 for kernel mode and 3 for user mode. The CPL is set by the hardware during ring transitions (system calls, interrupts, exception returns) and cannot be modified directly by software. The hardware also checks the Descriptor Privilege Level (DPL) of segment descriptors and the Requested Privilege Level (RPL) of segment selectors, but these are less relevant in long mode where segmentation is largely disabled.

The Global Descriptor Table (GDT)

On x86-64, the GDT defines the memory segments used by the OS. Although x86-64 uses a flat memory model (all segments have base 0 and cover the entire address space), the GDT is still required for:

- Defining the kernel code segment (ring 0) and user code segment (ring 3)
- Defining the kernel data segment and user data segment
- Holding the Task State Segment (TSS) descriptor, which points to the kernel stack for each CPU

A minimal GDT for a 64-bit kernel:

```
/* Minimal x86-64 GDT structure */
struct gdt_entry {
    uint16_t limit_low;
    uint16_t base_low;
    uint8_t  base_mid;
    uint8_t  access;      /* Type, DPL, present bit */
    uint8_t  flags_limit;  /* Flags + limit high nibble */
    uint8_t  base_high;
} __attribute__((packed));

/* 64-bit GDT entries (simplified) */
struct gdt_entry gdt[] = {
    {0},                          /* 0x00: Null descriptor (required) */
    {0, 0, 0, 0x9A, 0x20, 0},   /* 0x08: Kernel code (ring 0, 64-bit) */
    {0, 0, 0, 0x92, 0x00, 0},   /* 0x10: Kernel data (ring 0) */
    {0, 0, 0, 0xFA, 0x20, 0},   /* 0x18: User code (ring 3, 64-bit) */
    {0, 0, 0, 0xF2, 0x00, 0},   /* 0x20: User data (ring 3) */
    /* TSS descriptor follows (16 bytes in 64-bit mode) */
};
```

The access byte encodes the DPL: `0x9A` has DPL=0 (kernel), `0xFA` has DPL=3 (user). When the CPU executes a `SYSCALL` instruction, it loads the kernel CS selector (0x08), setting CPL to 0. When it executes `SYSRET`, it loads the user CS selector (0x18), setting CPL to 3.

ARM Exception Levels

ARM processors (ARMv8-A and later) use a four-level hierarchy called *Exception Levels* (EL0 through EL3), where higher numbers indicate greater privilege – the opposite convention from x86.

```
ARM EXCEPTION LEVELS
────────────────────────────────────────────────────────────────

 EL3: Secure Monitor
      (ARM Trusted Firmware, secure world transitions)
      ┌──────────────────────────────────────────────────┐
      │  Manages transitions between Secure and          │
      │  Non-secure worlds. Runs secure monitor code.    │
      │  Controls which physical addresses are visible   │
      │  to the Normal world.                            │
      └──────────────────────────────────────────────────┘

 EL2: Hypervisor
      (KVM, Xen, or bare-metal hypervisor)
      ┌──────────────────────────────────────────────────┐
      │  Manages virtual machines. Controls stage-2      │
      │  address translation (IPA to PA). Traps          │
      │  sensitive guest operations. Manages virtual     │
      │  interrupts for guests.                          │
      └──────────────────────────────────────────────────┘

 EL1: OS Kernel
      (Linux, Windows, or guest OS)
      ┌──────────────────────────────────────────────────┐
      │  Full OS kernel. Manages processes, memory,      │
      │  devices, file systems. Controls stage-1         │
      │  page tables (VA to IPA or PA).                  │
      └──────────────────────────────────────────────────┘

 EL0: User Applications
      (all unprivileged code)
      ┌──────────────────────────────────────────────────┐
      │  Cannot access system registers. Cannot          │
      │  execute privileged instructions. Restricted     │
      │  to its own virtual address space.               │
      └──────────────────────────────────────────────────┘

 Privilege increases ──────────────────────────────────────►
 EL0         EL1          EL2          EL3
────────────────────────────────────────────────────────────────
```

ARM's exception level model is cleaner than x86's ring model for several reasons:

1. **Dedicated hypervisor level (EL2).** Virtualisation is a first-class concept in the architecture, not retrofitted. The hypervisor has its own system registers, its own page table (stage 2 translation), and its own exception handlers, all architecturally distinct from the OS kernel at EL1.
2. **Dedicated secure monitor level (EL3).** The ARM TrustZone architecture provides hardware-enforced isolation between a "secure world" (running trusted firmware, cryptographic operations, DRM) and a "normal world" (running the OS and applications), with EL3 controlling transitions between them. This is used on billions of smartphones for secure boot, key storage, and payment processing.
3. **Clean exception routing.** Each exception level has a dedicated exception vector table (accessed via the VBAR_ELn register), and exceptions are routed to the appropriate level based on their type and the current execution level.

Transitions between exception levels occur through exceptions (interrupts, system calls, faults) and exception returns. The `SVC` instruction (Supervisor Call) transitions from EL0 to EL1 – the ARM equivalent of x86's `SYSCALL`. The `HVC` instruction (Hypervisor Call) transitions from EL1 to EL2. The `SMC` instruction (Secure Monitor Call) transitions from EL1 or EL2 to EL3. The `ERET` instruction (Exception Return) transitions from a higher EL to a lower one.

Each exception level has its own dedicated system registers:

- `SCTLR_ELn` – system control register (enables/disables caches, MMU, alignment checks)
- `TTBR0_ELn`, `TTBR1_ELn` – translation table base registers (page table pointers)
- `VBAR_ELn` – vector base address register (exception vector table location)
- `SP_ELn` – stack pointer for exception level n

This per-level register bank means that transitioning between exception levels does not require saving and restoring registers to memory – the hardware provides separate storage at each level.

RISC-V Privilege Modes

RISC-V, the open-source instruction set architecture, defines three privilege modes:

Mode	Abbreviation	Typical Use
Machine mode (M-mode)	M	Firmware, bootloader, most trusted code

Mode	Abbreviation	Typical Use
Supervisor mode (S-mode)	S	OS kernel
User mode (U-mode)	U	Applications

M-mode is always present and has unrestricted access to the hardware. S-mode and U-mode are optional (embedded systems may implement only M-mode). The key architectural features are:

- **Control and Status Registers (CSRs).** Each privilege mode has its own set of CSRs. Attempts to access a CSR belonging to a higher privilege mode trap to that mode. CSRs are accessed via dedicated instructions (`csrrw`, `csrrs`, `csrrc`).
- **Physical Memory Protection (PMP).** M-mode can configure PMP entries that restrict which physical address ranges are accessible in S-mode and U-mode. This provides a coarse-grained protection mechanism that is simpler than paging.
- **Sv39/Sv48 page tables.** S-mode uses a page table format (Sv39 with 39-bit virtual addresses, or Sv48 with 48-bit virtual addresses) that the MMU walks on every memory access, providing fine-grained virtual memory and protection.
- **Trap delegation.** M-mode can delegate specific exception types to S-mode using the `medeleg` and `mideleg` CSRs. For example, page faults can be delegated to S-mode (the OS kernel) while machine timer interrupts are handled in M-mode (firmware).

RISC-V's clean, minimal privilege model makes it attractive for research and for building formally verified systems. Several verified RISC-V processors and OS kernels are under active development.

Interrupts

What is an Interrupt?

An interrupt is a hardware signal that asynchronously notifies the CPU that an external event requires attention. When an interrupt arrives, the CPU suspends its current activity, saves minimal state, and transfers control to a predefined handler function.

Interrupts are the heartbeat of an operating system. Without them, the kernel would have to continuously poll every device to check whether it has data ready – wasting

enormous amounts of CPU time. With interrupts, the CPU can execute user programs and be notified only when a device actually needs attention.

Note

Interrupt. An asynchronous signal from a hardware device to the CPU, indicating that the device requires service. The CPU responds by suspending the current execution context, saving its state, and transferring control to an interrupt handler (also called an Interrupt Service Routine, or ISR). After the handler completes, the CPU restores the saved state and resumes the interrupted execution. The key property is *asynchrony*: interrupts can arrive between any two instructions, at any time, regardless of what the CPU is currently doing.

Polling vs Interrupts

Before examining interrupt hardware, it is instructive to understand the alternative: *polling*. In a polled I/O system, the CPU periodically checks each device's status register to see if data is available:

```
/* Polling loop -- wasteful but simple */
while (1) {
    if (serial_status_reg & DATA_READY) {
        char c = serial_data_reg;
        process_char(c);
    }
    if (disk_status_reg & TRANSFER_COMPLETE) {
        handle_disk_completion();
    }
    if (nic_status_reg & PACKET_RECEIVED) {
        handle_packet();
    }
    /* ... check every device ... */
}
```

Polling has two fundamental problems:

1. **CPU waste.** The CPU spends most of its time checking devices that have nothing to report. On a system with 50 devices where each is active 1% of the time, 98% of polling cycles are wasted.

2. **Latency.** If the polling loop takes 1 ms to cycle through all devices, the worst-case response time to any event is 1 ms. Reducing this by polling more

frequently wastes even more CPU time.

Interrupts solve both problems: the CPU does productive work until a device actively signals that it needs attention, and the response latency is bounded by the interrupt delivery time (typically 1–5 μs), not by a polling cycle.

However, interrupts have their own cost: each interrupt requires saving and restoring CPU state, transitioning to kernel mode, and executing the handler. For very high-rate events (millions of network packets per second), the interrupt overhead can overwhelm the CPU. This is why high-performance systems often use a *hybrid* approach: interrupts for low-rate events, polling for high-rate events. Linux's NAPI (New API) for network drivers uses this strategy: it starts with interrupt-driven reception and switches to polling when the packet rate exceeds a threshold.

Hardware Interrupt Mechanism

The hardware interrupt delivery path involves multiple components:

```
INTERRUPT DELIVERY PATH
────────────────────────────────────────────────────────────────


┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│  Disk    │  │ Network  │  │ Keyboard │  │  Timer   │
│Controller│  │  Card    │  │Controller│  │  Chip    │
└────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘
     │ IRQ         │ IRQ         │ IRQ         │ IRQ
     │             │             │             │
┌────▼─────────────▼─────────────▼─────────────▼────┐
│           INTERRUPT CONTROLLER                    │
│     (APIC on x86, GIC on ARM, PLIC on RISC-V)     │
│                                                   │
│  Prioritises interrupts, masks/unmasks lines,     │
│  routes interrupts to specific CPU cores          │
└──────────────────────┬────────────────────────────┘
                       │ INT signal
                       ▼
┌──────────────────────────────────────────────────┐
│                    CPU CORE                      │
│                                                  │
│  1. Finish current instruction                   │
│  2. Check IF (Interrupt Flag) in RFLAGS          │
│  3. If IF=1: save RIP, RFLAGS, CS, SS on stack   │
```

```
│  4. Clear IF to mask further interrupts              │
│  5. Read vector number from interrupt controller     │
│  6. Look up handler address in IDT[vector]           │
│  7. Jump to handler (ring 0)                         │
└──────────────────────────────────────────────────────┘
```

On x86 systems, the *Advanced Programmable Interrupt Controller* (APIC) manages interrupt delivery. The APIC architecture has two components:

- **Local APIC (LAPIC).** Each CPU core has its own LAPIC, which receives interrupts from the I/O APIC, from other CPUs (inter-processor interrupts, IPIs), and from its own timer. The LAPIC is accessed through memory-mapped registers at a fixed physical address (typically 0xFEE00000).
- **I/O APIC.** One or more I/O APICs aggregate interrupt lines from external devices. Each I/O APIC has a *redirection table* that maps device interrupt lines to specific vectors and target CPU cores. The OS configures the redirection table to balance interrupt load across cores.

Each interrupt is assigned a *vector number* (0–255 on x86). Vectors 0–31 are reserved for CPU exceptions (page faults, division by zero, etc.). Vectors 32–255 are available for hardware interrupts. The vector number indexes into the *Interrupt Descriptor Table* (IDT), which maps each vector to a handler address.

The Interrupt Descriptor Table (IDT)

The IDT is an array of 256 *gate descriptors*, each specifying:

- The address of the handler function (64 bits in long mode, split across three fields)
- The code segment selector (which determines the privilege level for the handler)
- The gate type: interrupt gate (clears IF, preventing nested interrupts) or trap gate (leaves IF unchanged)
- The IST (Interrupt Stack Table) index: if non-zero, the CPU switches to a dedicated stack before invoking the handler, providing a known-good stack even when the interrupted task's stack is corrupted

The IDT base address and limit are stored in the IDTR register, which is loaded by the `LIDT` instruction (a privileged instruction, executable only in ring 0).

```
/* x86-64 IDT entry structure */
struct idt_entry {
```

```
    uint16_t offset_low;      /* Handler address bits 0-15 */
    uint16_t segment_sel;     /* Code segment selector (0x08 for kernel) */
    uint8_t  ist;             /* Interrupt Stack Table index (0-7) */
    uint8_t  type_attr;       /* Type (0xE=interrupt gate, 0xF=trap gate) */
                              /* + DPL (0=kernel, 3=user) + Present bit */
    uint16_t offset_mid;      /* Handler address bits 16-31 */
    uint32_t offset_high;     /* Handler address bits 32-63 */
    uint32_t reserved;        /* Must be zero */
} __attribute__((packed));

/* Set up an IDT entry */
void set_idt_entry(struct idt_entry *idt, int vector,
                   void (*handler)(void), uint8_t ist,
                   uint8_t type, uint8_t dpl) {
    uint64_t addr = (uint64_t)handler;
    idt[vector].offset_low  = addr & 0xFFFF;
    idt[vector].segment_sel = 0x08;  /* Kernel code segment */
    idt[vector].ist         = ist;
    idt[vector].type_attr   = type | (dpl << 5) | 0x80; /* Present=1 */
    idt[vector].offset_mid  = (addr >> 16) & 0xFFFF;
    idt[vector].offset_high = (addr >> 32) & 0xFFFFFFFF;
    idt[vector].reserved    = 0;
}

/* Install IDT */
struct {
    uint16_t limit;
    uint64_t base;
} __attribute__((packed)) idtr;

void install_idt(struct idt_entry *idt) {
    idtr.limit = sizeof(struct idt_entry) * 256 - 1;
    idtr.base  = (uint64_t)idt;
    asm volatile("lidt %0" : : "m"(idtr));
}
```

Interrupt Handling Flow

When a hardware interrupt arrives at vector n, the CPU performs the following sequence atomically (i.e., it cannot be interrupted mid-sequence):

1. **Save context.** Push the current values of SS, RSP, RFLAGS, CS, and RIP onto the *kernel stack* (pointed to by the TSS – Task State Segment, or an IST stack if specified).
2. **Disable interrupts.** If the IDT entry is an interrupt gate, clear the IF (Interrupt Flag) in RFLAGS, preventing nested interrupts.
3. **Load handler.** Read `IDT[n]` to obtain the handler address and segment selector.
4. **Transfer control.** Set CS and RIP from the IDT entry, switching to ring 0 if not already there. If the CPL changed (from 3 to 0), the CPU also switches to the kernel stack.

The handler then executes. When it finishes, it executes `IRETQ` (Interrupt Return), which atomically restores SS, RSP, RFLAGS, CS, and RIP from the stack, returning to the interrupted code (and ring 3 if appropriate).

The total number of CPU cycles consumed by an interrupt entry and return (without the handler itself) is approximately 200–500 cycles on a modern x86-64 processor, depending on whether a privilege level change occurs and whether the TLB must be flushed (KPTI).

Info

The distinction between *interrupt gates* and *trap gates* in the IDT concerns whether interrupts are automatically disabled upon entry. An interrupt gate clears the IF flag, preventing nested interrupts. A trap gate leaves IF unchanged, allowing other interrupts to preempt the handler. Hardware interrupt handlers typically use interrupt gates (to prevent re-entrant interrupt handling that could corrupt shared data structures), while system call handlers and breakpoint handlers may use trap gates (to allow interrupts during long-running operations). The choice affects both correctness (preventing re-entrancy bugs) and latency (interrupt masking duration).

Message Signalled Interrupts (MSI/MSI-X)

Traditional interrupts use dedicated physical interrupt lines (wires) between devices and the interrupt controller. MSI (Message Signalled Interrupts), introduced with PCI 2.2 and extended by MSI-X, replaces physical lines with *memory writes*: to signal an interrupt, the device writes a specific value to a specific address (both configured by the OS). The memory write is routed by the system's memory address decoder to the LAPIC, which interprets it as an interrupt.

MSI-X advantages:

- **More vectors.** MSI-X supports up to 2048 interrupt vectors per device, vs 4 for MSI and 24 for legacy I/O APIC lines. This allows a modern NVMe SSD with 128 queues to have a dedicated interrupt vector per queue, one per CPU core.
- **No sharing.** Each MSI-X vector is unique to a device, eliminating the need for interrupt sharing (where the handler must poll all devices sharing the same line to find which one interrupted).
- **Better performance.** MSI-X interrupts can target specific CPU cores directly, without routing through the I/O APIC, reducing delivery latency.

Software Interrupts (Traps)

In addition to hardware interrupts generated by devices, the CPU can generate *software interrupts* through explicit instructions:

- `INT n` – invoke interrupt handler at vector n. Used historically for system calls on 32-bit Linux (`INT 0x80`).
- `SYSCALL` / `SYSENTER` – optimised system call instructions that bypass the IDT entirely, using dedicated MSRs for the handler address. `SYSCALL` stores the return address in RCX and RFLAGS in R11, avoiding memory accesses.
- `SVC` (ARM) – Supervisor Call, the ARM equivalent of x86's `SYSCALL`.
- `ECALL` (RISC-V) – Environment Call, transitions from U-mode to S-mode (or from S-mode to M-mode).

Software interrupts have the same effect as hardware interrupts: they save state and transfer control to a handler in kernel mode. The difference is that software interrupts are *synchronous* – they occur at a specific point in the instruction stream, predictably – while hardware interrupts are *asynchronous*.

Programmer's Perspective

On Linux, you can examine the interrupt counts for every vector on every CPU core by reading `/proc/interrupts`. Run `watch -n 1 cat /proc/interrupts` on a Linux system and you will see the counters incrementing in real time. The `LOC` (Local timer) row shows the scheduler timer interrupts – typically 250–1000 per second per core, depending on the kernel's `CONFIG_HZ` setting and whether the tickless mode is active. The `RES` (Rescheduling) row shows inter-processor interrupts (IPIs) used by the scheduler to wake up idle cores when new work arrives. The `NMI` (Non-Maskable Interrupt) row shows interrupts that cannot be disabled by software – used for watchdog timers and performance monitoring. Monitoring `/proc/interrupts` over time reveals the interrupt load distribution across cores and can diagnose performance issues like "interrupt storms" (one device flooding the CPU with interrupts) or unbalanced interrupt routing (one core handling all network interrupts while others are idle). On a server handling heavy network traffic, you can use `irqbalance` to distribute interrupt load, or manually configure IRQ affinity via `/proc/irq/N/smp_affinity`.

Top-Half and Bottom-Half Processing

Hardware interrupt handlers face a fundamental tension: they must complete quickly (because interrupts are disabled during the handler, blocking other devices from being serviced) but some interrupt processing requires significant work (e.g., reassembling a fragmented network packet, updating file system metadata after a disk write, processing USB enumeration events).

Linux resolves this with a *split interrupt handling* model:

Top half (hardirq). The interrupt handler runs in *interrupt context* with interrupts disabled on the current CPU. It does the absolute minimum: acknowledge the device (clear its interrupt status register), copy data from device registers or DMA buffers to kernel memory, and schedule deferred work. The top half must not sleep, must not allocate memory with `GFP_KERNEL`, and must not take any sleeping lock (mutex). It may only take spinlocks.

Bottom half (softirq / tasklet / workqueue). Deferred processing runs later, with interrupts re-enabled. It performs the expensive work: network protocol processing (TCP reassembly, checksum verification), buffer management, and waking up waiting processes.

```
SPLIT INTERRUPT HANDLING
```

```
Hardware interrupt arrives
      │
      ▼
┌──────────────────────────┐
│     TOP HALF (hardirq)   │  Interrupts DISABLED
│                          │  Must be fast (<100 us)
│  - Acknowledge device    │  Cannot sleep
│  - Read device registers │  Cannot allocate (GFP_KERNEL)
│  - Copy DMA data         │  Can only take spinlocks
│  - Schedule bottom half  │
└──────────────────────────┘
              │
              │
      IRETQ (interrupts re-enabled)
              │
              ▼
┌──────────────────────────┐
│   BOTTOM HALF (softirq)  │  Interrupts ENABLED
│                          │  Can do substantial work
│  - Protocol processing   │  Can allocate memory
│  - Reassemble packets    │  Runs in softirq context
│  - Wake waiting processes │  (still cannot sleep)
└──────────────────────────┘
```

Linux provides three bottom-half mechanisms, in order of increasing flexibility:

Mechanism	Context	Can Sleep?	Serialisation	Typical Use
Softirq	Interrupt (softirq)	No	Per-CPU (same softirq can run on different CPUs)	Network RX/TX, block I/O, timer callbacks
Tasklet	Interrupt (softirq)	No	Per-tasklet (same tasklet never runs on two CPUs)	Simple deferred work for a specific device

Mechanism	Context	Can Sleep?	Serialisation	Typical Use
Workqueue	Process (kworker)	Yes	Configurable	Complex work requiring memory allocation or sleeping

Softirqs are the most performance-critical: Linux defines exactly 10 softirq types (including `NET_RX_SOFTIRQ`, `NET_TX_SOFTIRQ`, `BLOCK_SOFTIRQ`, `TIMER_SOFTIRQ`), each processing work for all devices of that type on the current CPU. Softirqs are checked and executed at strategic points: after interrupt handlers return, after system calls, and by the `ksoftirqd` kernel thread when the softirq load is heavy.

The `kworker` threads that execute workqueue items are visible in `ps` output. Each CPU has its own `kworker/N:M` threads, plus system-wide `kworker/u:N` threads for unbound work items. When you see `kworker` threads consuming CPU in `top`, they are executing deferred interrupt processing or other asynchronous kernel work.

Exceptions: Faults, Traps, and Aborts

Exception Classification

Exceptions are synchronous events generated by the CPU during instruction execution. Unlike interrupts (which are asynchronous and external), exceptions are caused by the currently executing instruction and are deterministic: the same instruction in the same state always produces the same exception.

The x86 architecture classifies exceptions into three categories:

Faults. The exception is reported *before* the faulting instruction completes (or after partial execution that can be rolled back). The saved instruction pointer (RIP) points to the faulting instruction, so the handler can fix the problem and re-execute the instruction. The canonical example is a page fault: the MMU detects that a page is not present, the page fault handler loads the page from disk, and the faulting instruction is re-executed – this time successfully.

Traps. The exception is reported *after* the trapping instruction completes. The saved RIP points to the *next* instruction. Traps are used for debugging (the `INT3` breakpoint instruction, which is a single-byte opcode `0xCC`) and system calls (the `SYSCALL` instruction on x86-64).

Aborts. The exception indicates an unrecoverable error (hardware failure, inconsistent internal state). The faulting instruction cannot be identified or restarted. Aborts typically result in system termination. The canonical example is a machine check exception (#MC, vector 18), which indicates a hardware error such as an uncorrectable memory ECC failure or a CPU microcode bug.

Note

Page Fault. A page fault (vector 14 on x86) occurs when the CPU attempts to access a virtual address that is either not mapped in the page table, mapped with insufficient permissions, or marked as not present. The page fault handler examines the faulting address (stored in the CR2 register on x86) and the error code (pushed onto the stack by the CPU) to determine the cause and take appropriate action: load the page from disk, allocate a new page, send SIGSEGV to the process, or trigger a kernel panic.

x86 Exception Vectors

The first 32 IDT vectors (0–31) are reserved for CPU exceptions:

Vector	Mnemonic	Type	Cause
0	#DE	Fault	Division by zero or division overflow
1	#DB	Fault/Trap	Debug (hardware breakpoint, single-step)
2	NMI	Interrupt	Non-Maskable Interrupt (hardware failure, watchdog)
3	#BP	Trap	Breakpoint (`INT3`, single-byte `0xCC`)
6	#UD	Fault	Undefined/invalid opcode

Vector	Mnemonic	Type	Cause
8	#DF	Abort	Double fault (exception during exception handling)
10	#TS	Fault	Invalid TSS
11	#NP	Fault	Segment not present
12	#SS	Fault	Stack-segment fault
13	#GP	Fault	General protection (privilege violation, bad selector)
14	#PF	Fault	Page fault
18	#MC	Abort	Machine check (uncorrectable hardware error)
19	#XM	Fault	SIMD floating-point exception

The Page Fault Handler in Detail

The page fault is the most important exception in a modern OS because it is the mechanism that implements demand paging, copy-on-write, memory-mapped files, and lazy allocation. When a page fault occurs, the CPU provides two pieces of information:

1. **The faulting address** – stored in the CR2 register (x86) or the FAR_ELn register (ARM). This is the virtual address that the instruction tried to access.
2. **The error code** – pushed onto the stack (x86), with bit flags:
   - Bit 0 (P): 0 = page not present, 1 = protection violation
   - Bit 1 (W): 0 = read access, 1 = write access
   - Bit 2 (U): 0 = kernel mode, 1 = user mode
   - Bit 3 (RSVD): 1 = reserved bit violation in page table
   - Bit 4 (I): 1 = instruction fetch (NX violation)

The kernel's page fault handler then classifies the fault:

```
PAGE FAULT CLASSIFICATION
────────────────────────────────────────────────────────────────────────


  Page fault at address A, error code E
        │
        ├── Is A in the process's valid VMA (vm_area_struct)?
        │   │
        │   ├── NO
        │   │   ├── Is A just below the stack VMA?
        │   │   │   ├── YES ──► Stack growth
        │   │   │   │          (expand VMA, allocate page)
        │   │   │   └── NO ──► SEGFAULT (send SIGSEGV)
        │   │   │
        │   └── YES
        │       │
        │       ├── P=0 (page not present)
        │       │   ├── Anonymous mapping ──► Allocate zero page
        │       │   ├── File mapping ──► Read page from file
        │       │   └── Swapped out ──► Read page from swap
        │       │
        │       ├── P=1, W=1 (write to read-only page)
        │       │   ├── COW page ──► Copy page, mark writable
        │       │   └── Truly read-only ──► SEGFAULT
        │       │
        │       └── P=1, I=1 (exec on NX page) ──► SEGFAULT
────────────────────────────────────────────────────────────────────────
```

A simplified page fault handler:

```c
/* Simplified page fault handler (conceptual) */
void page_fault_handler(struct interrupt_frame *frame,
                        uint64_t error_code) {
    uint64_t fault_addr;
    asm volatile("mov %%cr2, %0" : "=r"(fault_addr));

    struct process *proc = current_process();
    struct vm_area *vma = find_vma(proc->mm, fault_addr);

    /* Check if the address is in a valid VMA */
```

```
if (vma == NULL) {
    /* Maybe stack growth? */
    if (fault_addr >= proc->mm->stack_vma->start - PAGE_SIZE
        && fault_addr < proc->mm->stack_vma->start) {
        expand_stack(proc->mm, fault_addr);
        return;  /* Re-execute faulting instruction */
    }
    send_signal(proc, SIGSEGV);
    return;
}

/* Permission check */
if (error_code & PF_WRITE) {
    if (!(vma->prot & PROT_WRITE)) {
        send_signal(proc, SIGSEGV);
        return;
    }
}

/* Page not present -- determine type */
struct page *new_page;

if (vma->flags & VM_COW && (error_code & PF_WRITE)) {
    /* Copy-on-write fault */
    struct page *old_page = get_page_at(proc->mm, fault_addr);
    new_page = alloc_page(GFP_USER);
    memcpy(page_to_virt(new_page),
           page_to_virt(old_page), PAGE_SIZE);
    map_page(proc->mm->pgd, fault_addr,
             new_page, vma->prot);
    put_page(old_page);  /* Decrement reference count */
} else if (vma->flags & VM_FILE) {
    /* Memory-mapped file */
    new_page = alloc_page(GFP_USER);
    read_page_from_file(vma->file,
                        fault_addr - vma->start,
                        new_page);
    map_page(proc->mm->pgd, fault_addr,
             new_page, vma->prot);
} else {
```

```
        /* Anonymous mapping -- zero-fill */
        new_page = alloc_page(GFP_USER | __GFP_ZERO);
        map_page(proc->mm->pgd, fault_addr,
                 new_page, vma->prot);
    }
    /* Return from handler; CPU re-executes faulting instruction */
}
```

Programmer's Perspective

Page faults are not errors – they are a normal part of program execution. When you start a Go program, the binary is memory-mapped into the process's address space via `mmap()`, but no pages are actually loaded into RAM. As the program executes, each new code page triggers a page fault, and the kernel loads it from disk on demand. You can observe this with `perf stat -e page-faults ./myprogram` – even a simple "Hello, World" program generates hundreds of page faults. The Go runtime's memory allocator (`runtime.mallocgc`) allocates virtual address space liberally using `mmap` with `MAP_ANONYMOUS`, but physical pages are not committed until the program actually writes to them. This means a Go program's virtual memory size (VIRT in `top`) can be much larger than its resident set size (RES) – the difference is pages that are mapped but have not yet been faulted in. Understanding this distinction is crucial for capacity planning: a Go program showing 1 GB of VIRT but 50 MB of RES is using 50 MB of physical RAM, not 1 GB. The `MADV_DONTNEED` advice via `madvise()` tells the kernel to drop specific pages from RAM, and the Go runtime uses this aggressively to return unused heap memory to the OS.

Double Faults and Triple Faults

A *double fault* (#DF, vector 8) occurs when the CPU encounters an exception while trying to invoke the handler for a prior exception. Specifically, a double fault is raised when:

- A page fault occurs while trying to deliver a page fault, general protection fault, or invalid TSS exception.
- A general protection fault occurs while trying to deliver a page fault, general protection fault, or invalid TSS exception.

For example, if a page fault occurs, the CPU tries to invoke the page fault handler at the address in `IDT[14]`. If that handler's code page is itself not present (another page fault), or if the kernel stack is corrupted (a stack segment fault), the CPU raises

a double fault instead.

A *triple fault* occurs when an exception occurs during the double fault handler itself. The CPU's response to a triple fault is architectural: the processor resets, effectively rebooting the machine. Triple faults are the x86's mechanism of last resort when the exception handling infrastructure itself is broken.

To prevent double faults from causing triple faults, the x86-64 architecture provides the *Interrupt Stack Table* (IST). The IST is a set of up to 7 dedicated stack pointers stored in the TSS. Each IDT entry can specify an IST index; when the corresponding exception occurs, the CPU switches to the IST-specified stack before pushing the exception frame. The double fault handler is always configured to use an IST stack, ensuring it has a valid stack even when the normal kernel stack is corrupted or exhausted.

Linux configures IST stacks for:

- Double fault (#DF) – IST 1
- NMI – IST 2 (NMIs can arrive at any time, including during other exception handlers)
- Machine check (#MC) – IST 3
- Debug (#DB) – IST 4

Direct Memory Access (DMA)

The Problem: CPU-Mediated I/O

Without DMA, every byte transferred between a device and memory must pass through the CPU. The CPU executes a loop: read a byte from the device's data register (a memory-mapped I/O address), write it to memory, increment the pointer, check whether the transfer is complete. This is called *Programmed I/O* (PIO).

For a simple keyboard, PIO is adequate – keystrokes arrive at most a few hundred times per second, and each transfer is a single byte. But for a disk transferring megabytes per second or a network card processing millions of packets, PIO wastes enormous amounts of CPU time on mindless data copying.

```
PROGRAMMED I/O (PIO) vs DMA
────────────────────────────────────────────────────────────────


PIO: CPU copies every byte

 Device ──byte──▶ CPU ──byte──▶ Memory
```

```
  Device ──byte──▶ CPU ──byte──▶ Memory
  Device ──byte──▶ CPU ──byte──▶ Memory
  ... (thousands of iterations for one disk block)

  CPU utilisation: 100% during transfer.
  Cannot execute user programs.
  For a 1 GB/s SSD: CPU would spend entire time copying.

 DMA: Device writes directly to memory

  CPU: "Transfer 4096 bytes from disk to address 0x1A3C000"
       │ (one MMIO write to program DMA controller)
       │
       │ CPU is FREE to run user programs
       │
       ▼
  DMA Controller ──────────────────────▶ Memory
  (autonomous transfer over memory bus)
       │
       ▼
  Interrupt: "Transfer complete"
       │
       ▼
  CPU: Process transferred data
```

The performance difference is dramatic. Consider copying 4 KB from a device to memory:

- **PIO**: 4096 `inb/outb` operations at ~100 ns each = 410 μs, CPU busy the entire time.
- **DMA**: 1 MMIO write to start transfer (~200 ns) + transfer time over bus (~1 μs at PCIe speed) + 1 interrupt (~2 μs) = ~3 μs total, CPU free during the transfer.

DMA reduces the CPU overhead of a 4 KB transfer by approximately 100x.

How DMA Works

DMA allows hardware devices to read from and write to main memory *without CPU intervention*. The CPU initiates the transfer by programming the DMA controller with:

1. **Source address** – the device register (for device-to-memory reads) or memory address (for memory-to-device writes).
2. **Destination address** – the memory address (for reads) or device register (for writes).
3. **Transfer length** – the number of bytes to transfer.
4. **Direction** – device-to-memory (read) or memory-to-device (write).
5. **Transfer mode** – single-cycle (one word per bus request), burst (multiple words in a continuous burst), or block (entire transfer without releasing the bus).

Once programmed, the DMA controller takes ownership of the memory bus and performs the transfer autonomously. The CPU can execute other instructions concurrently, as long as they do not require the memory bus. When the transfer completes, the controller raises an interrupt to notify the CPU.

DMA and Cache Coherence

DMA introduces a subtle but critical problem: the DMA controller writes directly to physical memory, bypassing the CPU's cache hierarchy. If the CPU has cached a copy of the memory region being written by DMA, the cached copy becomes stale – a *cache coherence* problem.

Consider the sequence:

1. CPU reads address A – value 42 is loaded into the L1 cache.
2. DMA writes value 99 to address A in physical memory.
3. CPU reads address A again – gets 42 from cache, not 99 from memory.

$$\text{If cache}[A] \neq \text{memory}[A] \text{ after DMA write, the CPU reads stale data.}$$

Two solutions exist:

Cache-coherent DMA. On systems with hardware cache coherence for DMA (most modern x86 systems and ARM systems with coherent interconnects like ACE/CCI), the DMA controller participates in the cache coherence protocol. DMA writes snoop the CPU caches and invalidate corresponding cache lines, ensuring the CPU always sees fresh data. This is transparent to software but requires hardware support in the interconnect fabric.

Non-coherent DMA. On systems without hardware DMA coherence (some ARM SoCs, many embedded systems), the OS must manually manage coherence through cache maintenance operations:

- Before a device-to-memory DMA read: *invalidate* the CPU cache lines covering the target memory region, forcing the CPU to re-read from memory after the transfer.
- Before a memory-to-device DMA write: *flush* (clean) the CPU cache lines covering the source region, writing modified data from cache back to memory so the device reads current data.

The Linux DMA API abstracts these operations:

```
/* Allocate DMA-coherent memory (always consistent, no manual sync) */
void *buf = dma_alloc_coherent(dev, 4096, &dma_handle, GFP_KERNEL);
/* buf:        kernel virtual address
 * dma_handle: physical address for programming the device */

/* Alternatively, map existing memory for DMA (streaming) */
dma_addr_t dma_addr = dma_map_single(dev, kbuf, len,
                                     DMA_FROM_DEVICE);
/* ... device performs DMA ... */
dma_unmap_single(dev, dma_addr, len, DMA_FROM_DEVICE);
/* After unmap, cache is synchronised and CPU can read the data */
```

Tip

The `dma-buf` subsystem in Linux allows DMA buffers to be shared between devices – for example, a camera can capture a frame into a DMA buffer that is then handed directly to a GPU for processing, then to a display controller for rendering, all without the CPU ever copying the pixel data. This *zero-copy* pipeline is essential for real-time video processing and is used extensively on Android devices.

Scatter-Gather DMA

Simple DMA transfers a contiguous block of physical memory. But real data is often scattered across non-contiguous physical pages (because the OS uses paging, and virtual memory is contiguous but physical memory is not).

Scatter-gather DMA allows the DMA controller to process a list of (address, length) pairs – called a *scatter-gather list* (SGL) – in a single operation:

```
SCATTER-GATHER DMA
──────────────────────────────────────────────────────────────


 SGL (array of {phys_addr, length} entries):
```

```
[0] addr=0x1A000  len=4096  ──▶ Physical page at frame 0x1A
[1] addr=0x2B000  len=4096  ──▶ Physical page at frame 0x2B
[2] addr=0x0F000  len=2048  ──▶ Physical page at frame 0x0F

DMA controller processes all entries sequentially,
assembling contiguous device data from scattered pages.

Total transfer: 10,240 bytes across 3 non-contiguous pages.
Without scatter-gather: 3 separate DMA operations required.
```

Scatter-gather is critical for performance: without it, the OS would need to either allocate physically contiguous memory for every DMA buffer (expensive, fragmentation-prone, and sometimes impossible on systems with fragmented physical memory) or perform multiple DMA transfers (one per page), each requiring a separate interrupt.

IOMMUs: DMA with Protection

On a system without an IOMMU (I/O Memory Management Unit), a device performing DMA can read from or write to *any* physical memory address. A malicious or buggy device (or driver) could corrupt kernel data, read other processes' memory, or inject code into the kernel.

The IOMMU (Intel VT-d, AMD-Vi, ARM SMMU) provides address translation and access control for DMA, analogous to how the CPU's MMU provides address translation for CPU memory accesses:

```
IOMMU OPERATION


Without IOMMU:
Device ──DMA addr──▶ Physical Memory (any address!)

With IOMMU:
Device ──DMA addr──▶ IOMMU ──translated──▶ Physical Memory
                      │
                 I/O Page Table
                 (configured by OS,
                  restricts device access
                  to authorised regions)
```

IOMMUs are essential for:

- **Device isolation in virtualisation.** Each virtual machine is assigned specific devices via PCI passthrough (VFIO), and the IOMMU ensures the device can only access that VM's memory, not the hypervisor's or other VMs' memory.
- **Driver bug containment.** A buggy driver that programs incorrect DMA addresses is confined to its IOMMU domain. The DMA transaction faults at the IOMMU, generating an IOMMU fault report, rather than corrupting arbitrary memory.
- **User-space device drivers.** Frameworks like DPDK and SPDK allow user-space programs to directly control PCIe devices. The IOMMU ensures these programs cannot corrupt kernel or other processes' memory via DMA – only the pages explicitly mapped into the IOMMU domain are accessible.

I/O Buses and Device Communication

The Modern Bus Hierarchy

Modern computer systems organise I/O devices into a hierarchy of buses, each with different bandwidth, latency, and protocol characteristics:

```
MODERN BUS HIERARCHY (x86 Desktop/Server)
────────────────────────────────────────────────────────────────


┌──────────┐                    ┌──────────┐
│   CPU 0  │────────────────────│   CPU 1  │
└──────────┘   UPI / Infinity   └──────────┘
      │        Fabric (CPU link)       │
      │                                │
┌─────┴────────────────────────────────┴──────┐
│            PCIe Root Complex                 │
│    (integrated in CPU die)                   │
└──────┬──────┬──────┬──────┬──────┬──────────┘
       │      │      │      │      │
┌──────┐ ┌────┐ ┌────┐ ┌────┐ ┌──────────┐
│ GPU  │ │NVMe│ │ NIC│ │NVMe│ │  Chipset │
│PCIe  │ │ SSD│ │10GbE│ │ SSD│ │  (PCH)   │
│ x16  │ │ x4 │ │ x4 │ │ x4 │ └──┬──┬──┬──┘
└──────┘ └────┘ └────┘ └────┘    │  │  │
                                 │  │  └── SATA (HDD/SSD)
```

```
                                        │   │
                                        │   └── USB 3.x Hub
                                        │         ├── Keyboard
                                        │         ├── Mouse
                                        │         └── External Drive
                                        │
                                        └── Low-speed buses
                                              (SPI, LPC, SMBus)
                                              ├── BIOS Flash
                                              ├── TPM
                                              └── Fan/Temp sensors
```

PCI Express (PCIe)

PCIe is the primary high-speed bus in modern computers. Unlike older parallel buses (PCI, ISA), PCIe uses *serial point-to-point links*, where each device has a dedicated connection to a PCIe switch or the root complex.

Key characteristics:

- **Lanes.** Each PCIe link consists of one or more lanes (x1, x2, x4, x8, x16). Each lane is a pair of differential signal pairs (one for transmit, one for receive), providing full-duplex communication.
- **Bandwidth per lane by generation:**

Generation	Per-lane bandwidth (each direction)	x16 total
PCIe 3.0	1 GB/s	16 GB/s
PCIe 4.0	2 GB/s	32 GB/s
PCIe 5.0	4 GB/s	64 GB/s
PCIe 6.0	8 GB/s	128 GB/s

- **Transaction layer.** PCIe uses a packet-based protocol with three layers: Transaction Layer (TLP – Transaction Layer Packets for memory reads/writes, I/O, configuration), Data Link Layer (CRC error detection, flow control, retry), and Physical Layer (electrical signalling, encoding).
- **Enumeration.** At boot time, the firmware and/or OS scans the PCIe bus tree by reading configuration space registers at each possible device address

(bus:device:function). The `lspci` command on Linux displays this tree, showing every discovered device with its vendor ID, device ID, and resource allocations.

NVMe: A Modern Storage Interface

NVMe (Non-Volatile Memory Express) is a storage protocol designed specifically for PCIe-attached SSDs, replacing the legacy AHCI protocol designed for spinning disks.

NVMe achieves high performance through:

- **Massive parallelism.** NVMe supports up to 65,535 I/O queues, each with up to 65,536 entries. This allows different CPU cores to submit I/O independently, without any locking or synchronisation.
- **Submission/completion model.** The OS writes I/O commands to submission queues and reads completions from completion queues. Both are ring buffers in host memory, accessed by the device via DMA. The only synchronisation points are doorbell writes (host to device, via MMIO) to notify the device of new commands, and completion entries (device to host, via DMA) to report results.

```
NVMe COMMAND SUBMISSION FLOW
──────────────────────────────────────────────────────────────────


 Host Memory:
 ┌──────────────────────────────────────────────┐
 │ Submission Queue 0 (Admin)                   │
 │ [cmd][cmd][   ][   ][   ][   ]               │
 └──────────────────────────────────────────────┘


 ┌──────────────────────────────────────────────┐
 │ Submission Queue 1 (I/O, CPU core 0)         │
 │ [cmd][cmd][cmd][   ][   ][   ]               │──► DMA read
 └──────────────────────────────────────────────┘    by NVMe SSD
                                                          │
                                                          │
 ┌──────────────────────────────────────────────┐         │
 │ Completion Queue 1                           │         │
 │ [cpl][   ][   ][   ][   ][   ]               │◄── DMA write
 └──────────────────────────────────────────────┘    by NVMe SSD

 1. Host writes command to SQ tail
 2. Host writes SQ tail doorbell (MMIO)
 3. Device reads command via DMA
```

```
4. Device processes I/O
5. Device writes completion to CQ via DMA
6. Device raises MSI-X interrupt
7. Host reads completion, processes result
8. Host writes CQ head doorbell (MMIO)
```

- **Minimal CPU involvement.** Each I/O command requires exactly: one memory write (the command, 64 bytes), one MMIO write (the doorbell, 4 bytes), and one memory read (the completion, 16 bytes). Compare this to AHCI, which requires reading and writing multiple device registers per command.

The result: a modern NVMe SSD can sustain millions of IOPS (I/O operations per second) with queue depths of hundreds, while AHCI is limited to 32 outstanding commands per port and achieves at most a few hundred thousand IOPS even on fast SSDs.

USB Architecture

USB (Universal Serial Bus) uses a tiered-star topology where the host controller manages all communication. USB is relevant to OS design because of its complexity:

- **Enumeration and configuration.** When a device is connected, the host controller detects the voltage change on the data lines, resets the device, assigns it a unique address (1–127), reads its descriptors (device, configuration, interface, endpoint), and notifies the OS to load the appropriate driver.
- **Transfer types.** USB defines four transfer types optimised for different workloads:

Type	Bandwidth Guarantee	Error Recovery	Use Case
Control	Low, guaranteed	Yes (retry)	Configuration, device setup
Bulk	High, best-effort	Yes (retry)	Storage, printers
Interrupt	Low, guaranteed latency	Yes (retry)	Keyboard, mouse, gamepad
Isochronous	Guaranteed bandwidth	No retry	Audio, video streaming

- **Hot-plugging.** USB devices can be connected and disconnected at any time. The OS must handle sudden device removal gracefully: cancel pending I/O

operations, release resources, notify user-space applications. A poorly written driver that does not handle surprise removal can cause kernel crashes.

Memory-Mapped I/O vs Port-Mapped I/O

Two Approaches to Device Communication

The CPU communicates with hardware devices by reading and writing device *registers* – small memory-like locations on the device controller. There are two mechanisms for accessing these registers:

Port-Mapped I/O (PMIO). The CPU has a separate I/O address space, accessed via special instructions (`IN` and `OUT` on x86). The I/O address space is 16 bits wide (65,536 ports). Legacy x86 devices use specific port addresses by convention.

```
/* Port-mapped I/O: reading from a serial port (x86 legacy) */
static inline uint8_t inb(uint16_t port) {
    uint8_t value;
    asm volatile("inb %1, %0" : "=a"(value) : "Nd"(port));
    return value;
}

static inline void outb(uint16_t port, uint8_t value) {
    asm volatile("outb %0, %1" : : "a"(value), "Nd"(port));
}

/* Read from COM1 serial port */
uint8_t data   = inb(0x3F8);  /* Data register */
uint8_t status = inb(0x3FD);  /* Line status register */
```

Memory-Mapped I/O (MMIO). Device registers are mapped into the CPU's physical address space. The CPU accesses them using ordinary load and store instructions. The memory controller routes accesses to specific physical address ranges to the appropriate device instead of to RAM.

```
/* Memory-mapped I/O: accessing a modern device (e.g., NVMe, LAPIC) */

/* The device's BAR (Base Address Register) specifies the MMIO region.
 * On Linux, the kernel maps this into virtual address space. */
volatile uint32_t *regs = ioremap(pci_resource_start(pdev, 0),
                                  pci_resource_len(pdev, 0));
```

```c
/* Read the device's capabilities register (offset 0x00) */
uint32_t caps = readl(&regs[0]);

/* Write to the device's command register (offset 0x04) */
writel(0x01, &regs[1]);

/* NVMe: write submission queue tail doorbell */
writel(new_tail, &regs[0x1000 / 4 + queue_id * 2]);
```

Comparison

Property	Port-Mapped I/O	Memory-Mapped I/O
Instruction type	Special (`IN`/`OUT`)	Normal load/store (`MOV`)
Address space	Separate 16-bit I/O space	Shared with memory (64-bit)
Address range	65,536 ports max	Full physical address space
Architecture support	x86 only	All architectures
Protection	Ring 0 only (or IOPL bits)	Standard page table permissions
Compiler visibility	Always volatile (special insn)	Requires `volatile` keyword
Cacheability	Always uncacheable	Configurable (must be UC for devices)
Modern usage	Legacy devices only	All modern devices (PCIe, USB, etc.)

Note

MMIO has essentially won the architectural competition. All modern device interfaces (PCIe BARs, USB controller registers, GPU command channels) use memory-mapped I/O exclusively. Port-mapped I/O survives only for legacy x86 devices: the 8042 keyboard controller (ports 0x60, 0x64), the PIC (ports 0x20, 0xA0), the serial ports (0x3F8–0x3FF), and the PIT timer (port 0x40). New device specifications universally mandate MMIO.

The `volatile` Keyword and Memory Ordering

In MMIO code, the C compiler must not optimise away or reorder device register accesses. Two issues arise:

1. Optimisation. Without `volatile`, the compiler may cache a device register read in a CPU register and not re-read it:

```
/* Without volatile -- DANGEROUS for MMIO */
uint32_t *status = (uint32_t *)0xFE000000;
while (*status & BUSY_BIT) { /* spin */ }
/* Compiler may load *status once and loop forever */
```

The `volatile` keyword forces the compiler to emit a load instruction for every read:

```
/* With volatile -- compiler will re-read on each iteration */
volatile uint32_t *status = (volatile uint32_t *)0xFE000000;
while (*status & BUSY_BIT) { /* spin */ }
```

Info

Volatile Semantics in C. The C standard (C11, section 6.7.3) specifies that accesses to `volatile`-qualified objects are "side effects" that the compiler must not optimise away, reorder with respect to other volatile accesses, or coalesce. Every read of a `volatile` object must result in an actual load instruction, and every write must result in an actual store instruction, in the order specified by the abstract machine. However, `volatile` only controls the *compiler* – it does not emit hardware memory barriers. On weakly-ordered architectures (ARM, RISC-V, POWER), additional barriers may be required to prevent the *CPU* from reordering accesses at the hardware level.

2. Memory ordering. Even with `volatile`, the CPU hardware may reorder memory accesses for performance. On x86, stores to device registers are not reordered with respect to other stores (x86 has a strong memory model), but on ARM and RISC-V, explicit memory barriers are needed:

```
/* ARM: write command then write doorbell -- must be in order */
writel(command, &regs->cmd_reg);
wmb();  /* Write memory barrier -- ensures cmd_reg is visible
           to the device before the doorbell write */
writel(1, &regs->doorbell);
```

The Linux kernel provides architecture-abstracted I/O accessor functions (`readl`, `writel`, `readq`, `writeq`) that include the appropriate memory barriers for the target architecture.

Timer Hardware and Preemptive Scheduling

The Role of the Timer

Preemptive scheduling – the ability of the OS to forcibly take the CPU away from a running process – requires a hardware mechanism that generates periodic interrupts. Without a timer, a process that enters an infinite loop would monopolise the CPU forever, because nothing would trigger a kernel entry to invoke the scheduler.

The timer interrupt is the OS scheduler's heartbeat. On each tick, the kernel's timer interrupt handler:

1. Updates the system clock and per-process CPU time accounting.
2. Decrements the running process's remaining quantum.
3. Checks if the quantum has expired.
4. If so, sets a "need reschedule" flag.
5. On return from the interrupt handler, the kernel checks the flag and, if set, invokes the scheduler to select the next process.

x86 Timer Hardware

x86 systems have multiple timer devices, reflecting decades of backward compatibility:

PIT (Programmable Interval Timer, Intel 8253/8254). The original PC timer, generating interrupts at a programmable frequency (the base oscillator runs at 1.193182 MHz). Accessed via I/O ports 0x40–0x43. The PIT can generate interrupts at rates from approximately 18.2 Hz to 1.193 MHz. Still present for legacy compatibility but rarely used as the primary timer on modern systems.

LAPIC Timer. Each CPU core has a Local APIC with a built-in timer that can operate in one-shot or periodic mode. This is the primary timer used by Linux on modern hardware because: - Each core has its own independent timer (essential for per-core scheduling on SMP systems). - The timer frequency is derived from the CPU bus clock, providing high resolution. - The LAPIC is accessed via MMIO (no slow I/O port instructions).

HPET (High Precision Event Timer). A memory-mapped timer with at least 10 MHz resolution and multiple comparators. Used as a fallback on systems where the TSC is unreliable. The HPET can generate interrupts on up to 32 independent comparators, each programmable to fire at a specific time.

TSC (Time Stamp Counter). A 64-bit counter that increments at the CPU's reference clock frequency (typically the base clock, e.g., 100 MHz or the crystal oscillator

frequency). The TSC is read via the `RDTSC` or `RDTSCP` instruction in approximately 10–25 cycles. It is not a timer (it does not generate interrupts), but it is the highest-resolution time source available, used for timestamps, performance measurement, and fine-grained delays.

The TSC frequency is constant on modern CPUs (`constant_tsc` flag in `/proc/cpuinfo`), meaning it does not vary with CPU frequency scaling. On multi-socket systems, the `nonstop_tsc` flag indicates the TSC continues counting during deep sleep states.

The Tick Rate and Tickless Kernels

Traditionally, the timer interrupt fires at a fixed rate – the *tick rate*. Linux historically used `CONFIG_HZ=100` (10 ms per tick), later increasing to `CONFIG_HZ=250` (4 ms) or `CONFIG_HZ=1000` (1 ms) for lower-latency scheduling.

The problem with a fixed tick rate is that timer interrupts fire even when the system is idle – waking the CPU from power-saving states (C-states), consuming energy, and generating unnecessary overhead. On a modern laptop, each unnecessary wakeup costs approximately 1 mW of power; at 1000 Hz, that is 1 W of wasted power just for timer interrupts on idle cores.

Linux's *tickless* (or *dynamic tick*, `CONFIG_NO_HZ_IDLE` and `CONFIG_NO_HZ_FULL`) mode addresses this:

- **`NO_HZ_IDLE`**: When a CPU core has no runnable processes, it disables the periodic timer interrupt entirely. The core enters a deep sleep state and is woken only by device interrupts or IPIs from other cores that have work to schedule. When the core has runnable processes, the periodic tick resumes.
- **`NO_HZ_FULL`**: Goes further – even when a core has a single runnable process, the tick is suppressed. The process runs without any timer interrupts until another process becomes runnable or a timer event occurs. This is useful for latency-sensitive workloads (HPC, real-time) that need to avoid all kernel interference.

The energy savings are significant:

$$E_{\text{fixed}} = f_{\text{tick}} \times t_{\text{idle}} \times E_{\text{wakeup}}$$

$$E_{\text{tickless}} = n_{\text{events}} \times E_{\text{wakeup}} \quad \text{where } n_{\text{events}} \ll f_{\text{tick}} \times t_{\text{idle}}$$

On a server with 64 cores, each idle 80% of the time, at `HZ=1000`, the fixed tick gen-

erates $64 \times 0.8 \times 1000 = 51,200$ unnecessary wakeups per second. Tickless mode eliminates all of them.

Implementing a Timer Interrupt Handler

The following C code shows a simplified timer interrupt handler that implements round-robin scheduling:

```c
/* Simplified timer interrupt handler (conceptual) */
#include <stdint.h>

struct task_context {
    uint64_t rax, rbx, rcx, rdx;
    uint64_t rsi, rdi, rbp, rsp;
    uint64_t r8, r9, r10, r11;
    uint64_t r12, r13, r14, r15;
    uint64_t rip, rflags;
    uint64_t cr3;  /* page table base */
};

struct task {
    struct task_context ctx;
    int    pid;
    int    quantum_remaining;  /* ticks left in current quantum */
    enum { RUNNING, READY, BLOCKED } state;
    struct task *next;         /* run queue linked list */
};

#define QUANTUM_TICKS 10  /* 10 ms quantum at HZ=1000 */

static struct task *current_task;
static struct task *run_queue_head;
static uint64_t    system_ticks;

/* Called by low-level assembly stub after saving registers */
void timer_interrupt_handler(struct task_context *saved) {
    /* 1. Save interrupted task's register state */
    current_task->ctx = *saved;

    /* 2. Acknowledge timer interrupt (LAPIC End-Of-Interrupt) */
    *((volatile uint32_t *)0xFEE000B0) = 0;  /* LAPIC EOI */
```

```
    /* 3. Update system time */
    system_ticks++;

    /* 4. Decrement quantum */
    current_task->quantum_remaining--;

    if (current_task->quantum_remaining > 0) {
        /* Quantum not expired -- resume current task */
        return;
    }

    /* 5. Quantum expired -- round-robin reschedule */
    current_task->quantum_remaining = QUANTUM_TICKS;
    current_task->state = READY;

    /* Move current task to end of run queue */
    append_to_run_queue(current_task);

    /* Select next task from head of run queue */
    struct task *next = dequeue_from_run_queue();
    if (next == current_task) {
        current_task->state = RUNNING;
        return;  /* Only one runnable task */
    }

    /* 6. Context switch */
    next->state = RUNNING;
    current_task = next;

    /* Switch page tables if necessary */
    if (saved->cr3 != next->ctx.cr3) {
        asm volatile("mov %0, %%cr3" : : "r"(next->ctx.cr3)
                     : "memory");
        /* CR3 write flushes TLB */
    }

    /* Restore next task's registers (via assembly stub) */
    *saved = next->ctx;
}
```

Programmer's Perspective

In Go, you almost never interact with timer hardware directly, but the timer profoundly affects your program's behaviour. Go's goroutine scheduler uses a combination of OS timer signals and cooperative preemption. Before Go 1.14, goroutines were only preempted at function call boundaries – a tight loop with no function calls (`for { i++ }`) could not be preempted, starving other goroutines. Go 1.14 introduced *asynchronous preemption*: the runtime sends `SIGURG` to goroutines that have been running too long, forcing them to yield. This is a user-space analogue of the kernel's timer-based preemptive scheduling. You can observe it: `strace -e signal ./myprogram` shows the preemption signals. Understanding that Go's scheduler sits *on top of* the OS scheduler helps explain subtle behaviour: goroutines are multiplexed onto OS threads (`M`s in Go runtime terminology), which are scheduled by the kernel. A goroutine blocked on a system call causes its OS thread to block, potentially triggering the Go runtime to create a new OS thread – which is why `runtime.NumGoroutine()` can be vastly larger than the `GOMAXPROCS` setting, and why `strace` shows `clone()` calls during heavy I/O.

Preemption Latency

The time between a preemption-triggering event (e.g., a higher-priority process becoming runnable) and the actual context switch to that process is called *preemption latency*:

$$T_{\text{preempt}} = T_{\text{interrupt}} + T_{\text{handler}} + T_{\text{scheduler}} + T_{\text{switch}}$$

Where:

- $T_{\text{interrupt}}$: interrupt delivery latency (hardware), typically 1–5 μs.
- T_{handler}: time to execute the interrupt handler, typically 1–10 μs.
- $T_{\text{scheduler}}$: time to select the next task, O(1) for Linux CFS in the common case.
- T_{switch}: time to perform the context switch (save/restore registers, switch page tables, flush TLB), typically 2–10 μs.

For general-purpose Linux (`CONFIG_PREEMPT_VOLUNTARY`), the total preemption latency is typically 10–100 μs. For fully preemptible Linux (`CONFIG_PREEMPT`), the target is under 50 μs. For real-time Linux (PREEMPT_RT patchset), the target is under 10–50 μs worst-case, achieved by:

- Converting all spinlocks to sleeping (priority-inheriting) mutexes

- Threading all interrupt handlers (running them as kernel threads that can be preempted)
- Minimising the time that interrupts are disabled

Real-Time Scheduling Constraints

A *hard real-time* system must guarantee that tasks meet their deadlines. The timer resolution and interrupt latency set fundamental bounds on schedulability.

Info

Rate Monotonic Scheduling (RMS) Bound. For a set of n independent periodic tasks with periods T_i and worst-case execution times C_i, the tasks are schedulable under the Rate Monotonic (fixed priority, shorter period = higher priority) algorithm if:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$$

The bound $n(2^{1/n} - 1)$ converges to $\ln 2 \approx 0.693$ as $n \to \infty$. For small n:

n	$n(2^{1/n} - 1)$
1	1.000
2	0.828
3	0.780
5	0.743
10	0.718
∞	0.693

This means a CPU utilisation above 69.3% cannot be guaranteed schedulable for arbitrary task sets under RMS. For specific *harmonic* task sets (where periods are exact multiples of each other), utilisation up to 100% is achievable.

Summary

The hardware-software interface is the foundation upon which all OS functionality is built. CPU privilege rings (x86 rings 0–3, ARM exception levels EL0–EL3, RISC-V M/S/U modes) enforce the boundary between trusted and untrusted code. Interrupts provide the asynchronous notification mechanism that lets the OS respond to hardware events without wasting CPU cycles on polling. Exceptions (faults, traps, aborts) provide synchronous notification of error conditions and are the mechanism behind demand paging, copy-on-write, and debugging. DMA offloads bulk data transfers from the CPU, with IOMMUs providing protection against rogue DMA accesses. I/O buses (PCIe, USB, NVMe) connect devices to the system and determine the available bandwidth and command interfaces. Memory-mapped I/O has supplanted port-mapped I/O as the universal device communication mechanism. Timer hardware provides the periodic interrupts that drive preemptive scheduling, with tickless kernels eliminating unnecessary overhead on idle and single-task cores. Every kernel operation – from scheduling a process to reading a file to receiving a network packet – ultimately reduces to these hardware mechanisms.

Exercises

Exercise 3.1: Privilege Level Analysis

Consider the following sequence of events on an x86-64 system running Linux:

a) A user program calls `getpid()`. Trace the CPL changes through the entire sequence, from the C library wrapper through the `SYSCALL` instruction, through the kernel handler, and back through `SYSRET`. At each step, state the CPL value and which instruction causes it to change.

b) While the `getpid()` handler is executing in the kernel, a network card raises an MSI-X interrupt. Trace the CPL changes as the CPU transitions from the system call handler to the interrupt handler and back. Does the CPL change?

c) The `SYSCALL` instruction on x86-64 does not automatically switch stacks – the kernel must switch to the kernel stack in software (reading the stack pointer from the per-CPU TSS). Explain why this is a security concern and how Linux addresses it. What would happen if a user program set RSP to point to kernel memory before executing `SYSCALL`?

d) Explain why the x86-64 `SYSCALL` instruction is faster than the older `INT 0x80` mechanism. Specifically, identify which hardware operations `SYSCALL` skips that `INT 0x80` performs.

Exercise 3.2: Interrupt Latency Measurement

Design an experiment to measure the interrupt latency of a Linux system.

a) Describe the hardware setup: which timer would you use, how would you programme it, and how would you measure the time between the expected interrupt arrival and the actual start of the handler?

b) Write pseudocode for the timer setup function and the interrupt handler, using `RDTSCP` for timestamp capture.

c) Identify at least four sources of variability in your measurements (factors that cause the measured latency to vary from one interrupt to the next).

d) Explain how each of the following Linux configurations affects your measurement: `CONFIG_HZ`, `CONFIG_PREEMPT`, `CONFIG_NO_HZ_FULL`, `isolcpus` kernel parameter.

e) How would you distinguish between interrupt *delivery* latency (hardware) and interrupt *handling* latency (software)? Can you separate these two components with TSC measurements alone?

Exercise 3.3: DMA Buffer Management

A network card receives packets via DMA into a ring buffer of 256 entries, each 2048 bytes. The driver pre-allocates all 256 buffers at initialisation time.

a) Calculate the total physical memory consumed by the ring buffer. Should the buffers be allocated with `dma_alloc_coherent()` (always cache-coherent) or `dma_map_single()` (streaming, with explicit sync points)? Justify your choice considering the access pattern (device writes, CPU reads, then device reuses).

b) Explain why the buffers must be DMA-addressable. What constraint does a 32-bit DMA-capable device impose on the physical addresses? How does the `GFP_DMA32` flag address this?

c) The driver must refill empty ring buffer entries after the kernel has processed the received packets. Design a refill strategy that minimises the window during which the ring buffer is full (and packets are dropped). Should the driver refill in the interrupt handler (top half) or in a NAPI poll function (bottom half)? Justify.

d) On a non-cache-coherent ARM platform, specify the exact cache maintenance operations the driver must perform: (i) after DMA writes a received packet to the buffer, and (ii) before returning the buffer to the device for the next receive.

Exercise 3.4: Page Fault Handler Design

Implement a page fault handler (in pseudocode or C) that correctly handles all of the following cases:

a) **Demand paging for anonymous memory**: a process accesses a page that was allocated with `mmap(MAP_ANONYMOUS)` but never written to.

b) **Demand paging for memory-mapped files**: a process accesses a page in a region created by `mmap(fd, ...)` that has not yet been loaded from disk.

c) **Copy-on-write**: a child process (created by `fork()`) writes to a page that is shared with the parent.

d) **Stack growth**: a process accesses an address just below the current stack boundary.

e) **Segmentation fault**: a process accesses an address that is not in any valid mapping.

For each case, specify the exact condition (error code bits, VMA lookup result) that distinguishes it from the others. Prove that your handler is *complete* – that every possible page fault is classified into exactly one of these categories.

Exercise 3.5: NVMe vs AHCI Quantitative Comparison

Compare the NVMe and AHCI storage interfaces:

a) AHCI supports 1 command queue per port with 32 entries maximum. NVMe supports up to 65,535 I/O queues with up to 65,536 entries each. Calculate the maximum number of outstanding I/O commands for each interface. On a 64-core system where each core submits I/O independently, which interface is the bottleneck?

b) For a single random 4 KB read: count the exact number of MMIO register writes required to submit the command on each interface. For AHCI, include: writing the command FIS, setting the command issue bit, and reading the status register. For NVMe, include: writing the SQE and the doorbell.

c) AHCI uses a single interrupt (or MSI vector) per port; NVMe supports MSI-X with up to 2048 vectors. Explain the performance implications on a multi-core system. What happens when all 64 cores must synchronise on a single interrupt vector?

Exercise 3.6: Timer Analysis

A system uses a LAPIC timer with a base frequency of 100 MHz programmed to fire every 1 ms (100,000 cycles).

a) Calculate the scheduling overhead (fraction of CPU time consumed by timer processing) if the timer interrupt handler takes 5 μs and the context switch takes 3 μs.

b) Repeat the calculation for a 10 ms quantum. Express the overhead ratio ($\text{overhead}_{1\text{ms}}/\text{overhead}_{10\text{ms}}$).

c) A tickless kernel on an idle core avoids all timer interrupts. If the system has 32 cores, each idle 70% of the time, and `HZ=1000`, how many unnecessary timer interrupts are eliminated per second?

d) Derive a formula for the minimum quantum length Q_{min} such that scheduling overhead is less than a fraction α of CPU time, given handler time T_h and context switch time T_s:

$$Q_{\text{min}} > \frac{T_h + T_s}{\alpha}$$

For $T_h = 5$ μs, $T_s = 3$ μs, $\alpha = 0.01$ (1% overhead), calculate Q_{min}.

Exercise 3.7: Writing an Interrupt Handler

Write a complete, compilable Linux kernel module that:

a) Creates a workqueue and schedules periodic work items that simulate interrupt bottom-half processing.

b) Each work item records the current timestamp (using `ktime_get_ns()`) into a circular buffer of 64 entries.

c) Exposes the recorded timestamps through a `/proc/irq_timestamps` file. When the file is read, it should display all timestamps in the buffer with nanosecond precision.

d) Properly cleans up all resources (cancel pending work, destroy workqueue, remove `/proc` entry) when the module is unloaded.

Provide the complete C source code and `Makefile`. Explain the difference between running deferred work in hardirq context (softirq/tasklet) versus process context (workqueue), and justify why a workqueue is appropriate for this exercise.

Chapter 4: Processes and Threads

A running program is not the same thing as the file sitting on disk. The moment the operating system loads an executable into memory, allocates resources, and begins executing its instructions, it creates something fundamentally new: a **process**. This chapter explores the process abstraction — how operating systems create, manage, and destroy the units of execution that make modern computing possible. We then turn to threads, the lighter-weight units of execution that share a process's address space, and examine the models that map application-level concurrency onto kernel resources.

4.1 Program vs Process

A **program** is a passive entity: a file containing machine instructions, data initialisers, and metadata stored on disk. A **process** is the active entity that arises when the operating system loads that program into memory and begins executing it. The distinction is subtle but essential.

Definition 4.1 (Program). A program is a static sequence of instructions and associated data, stored as an executable file in a filesystem. It is a passive entity that occupies no CPU time and holds no runtime state.

Definition 4.2 (Process). A process is an instance of a program in execution, together with all state necessary for its continued execution: the current values of the program counter, registers, stack, heap, open file descriptors, and associated kernel data structures.

The same program can give rise to multiple processes. Consider a web server: the executable `/usr/bin/nginx` is a single program, but the operating system may spawn dozens of worker processes from that same binary. Each process has its own address space, its own set of open file descriptors, and its own position in the instruction stream.

Example 4.1 (Multiple Instances). Running the command `python3 script.py` twice in separate terminals creates two distinct processes. Each has its own PID, its own copy of the Python interpreter's heap, and its own program counter. Modifications to a variable in one process have no effect on the other — they occupy entirely separate address spaces.

The key resources that define a process include:

- **Address space**: The virtual memory layout (text, data, heap, stack segments)
- **Program counter (PC)**: The address of the next instruction to execute
- **Processor registers**: General-purpose registers, stack pointer, frame pointer
- **Open files**: File descriptor table mapping integers to kernel file structures
- **Credentials**: User ID (UID), group ID (GID), effective permissions
- **Signal state**: Pending signals, signal handlers, signal masks
- **Scheduling attributes**: Priority, scheduling class, CPU time consumed

4.2 Process States

A process does not simply run from start to finish without interruption. Over its lifetime, it transitions through a series of well-defined states as it competes for CPU time, waits for I/O, and interacts with the scheduler.

Definition 4.3 (Five-State Process Model). The five-state process model defines the following states:

1. **New**: The process is being created; the kernel is allocating data structures and loading the executable image.
2. **Ready**: The process is loaded in memory and waiting to be assigned to a CPU by the scheduler.
3. **Running**: The process's instructions are being executed on a CPU core.
4. **Waiting** (Blocked): The process cannot proceed until some external event occurs (I/O completion, signal arrival, lock acquisition).
5. **Terminated**: The process has finished execution; the kernel retains its exit status until collected by the parent.

The transitions between these states follow strict rules:

```
                    ┌──────────────┐
                    │     New      │
                    └──────────────┘
                           │ admitted
                           ▼
          ┌──────────────────────────────┐
timeout   │                              │ scheduled
┌─────────│            Ready             │◄──────────┐
│         │                              │           │
│         └──────────────────────────────┘           │
│                        │ dispatch                  │
```

```
│                        ▼                          │
│                                                   │
└──────►┌──────────────────────────────┐────────────┘
        │          Running             │
        │                              │ I/O or event wait
        └──────────────────────────────┘
                   │          │
                   │          │
            exit   │          ▼
                   │   ┌──────────────────┐
                   │   │     Waiting      │
                   │   │    (Blocked)     │
                   │   └──────────────────┘
                   │
                   ▼
        ┌──────────────────────────────┐
        │         Terminated           │
        └──────────────────────────────┘
```

The critical transitions are:

Transition	Trigger	Direction
New → Ready	Process admitted by the kernel	Irreversible
Ready → Running	Scheduler dispatches process to CPU	May recur
Running → Ready	Timer interrupt (preemption)	May recur
Running → Waiting	Process initiates I/O or blocks on synchronisation	May recur
Waiting → Ready	I/O completes or event occurs	May recur
Running → Terminated	Process calls `exit()` or receives fatal signal	Irreversible

Example 4.2 (Process State Transitions). Consider a process that reads a file. Initially in the **Running** state, it issues a `read()` system call. The kernel places the process in the **Waiting** state while the disk controller fetches the requested block. When the DMA transfer completes and the disk raises an interrupt, the interrupt handler moves the process to the **Ready** queue. The scheduler eventually dispatches

it back to **Running**, where the `read()` call returns with the data.

4.2.1 Zombie and Orphan Processes

Two additional states arise from the parent-child relationship in Unix-like systems.

A **zombie process** has terminated but its parent has not yet called `wait()` to collect its exit status. The kernel must retain the process's entry in the process table (PID, exit code, resource usage statistics) until the parent retrieves it. Zombies consume no CPU or memory beyond their process table entry, but they do consume a PID — and PID space is finite.

An **orphan process** is one whose parent has terminated before it. On Linux, orphans are re-parented to the `init` process (PID 1) or the nearest subreaper (set via `prctl(PR_SET_CHILD_SUBREAPER)`). The adopting process is responsible for calling `wait()` on the orphan's behalf.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid == 0) {
        /* Child process */
        printf("Child PID: %d, Parent PID: %d\n", getpid(), getppid());
        sleep(2);
        printf("Child exiting (will become zombie if parent doesn't wait)\n");
        exit(0);
    } else {
        /* Parent process */
        printf("Parent PID: %d, Child PID: %d\n", getpid(), pid);

        /* Not calling wait() immediately --- child becomes zombie */
        sleep(5);

        /* Reap the zombie */
        int status;
        waitpid(pid, &status, 0);
        printf("Reaped child, exit status: %d\n", WEXITSTATUS(status));
```

```
    }

    return 0;
}
```

4.3 Process Memory Layout

Before examining the Process Control Block, it is essential to understand the memory layout of a process. Each process has a virtual address space divided into distinct segments:

Definition 4.4 (Process Address Space). The virtual address space of a process on a typical Unix system consists of the following segments, listed from low to high virtual addresses:

1. **Text segment**: The executable machine code, loaded from the ELF binary. Read-only and shared among all processes running the same program.
2. **Data segment**: Initialised global and static variables. Read-write.
3. **BSS segment** (Block Started by Symbol): Uninitialised global and static variables. Zero-filled at program start.
4. **Heap**: Dynamically allocated memory (via `malloc()`, `mmap()`). Grows upward from low addresses.
5. **Memory-mapped region**: Shared libraries, memory-mapped files, anonymous mappings. Located between heap and stack.
6. **Stack**: Function call frames, local variables, return addresses. Grows downward from high addresses.

On a 64-bit Linux system with the default address space layout:

```
High addresses
┌──────────────────────────────────┐ 0x7FFF_FFFF_FFFF
│            Kernel Space          │ (not accessible from user mode)
├──────────────────────────────────┤ 0x7FFF_FFFF_F000
│              Stack               │ grows downward
│              │                   │
│              ▼                   │
│                                  │
│         (unmapped gap)           │
│                                  │
│              ▲                   │
│              │                   │
│      Memory-mapped region        │ shared libraries, mmap
```

```
│                                   │
│              ▲                    │
│              │                    │
│             Heap                  │ grows upward
├───────────────────────────────────┤
│       BSS (uninitialised)         │
├───────────────────────────────────┤
│       Data (initialised)          │
├───────────────────────────────────┤
│       Text (code, read-only)      │
└───────────────────────────────────┘ 0x0000_0040_0000
Low addresses
```

The exact layout can be examined via `/proc/<pid>/maps`:

```
$ cat /proc/self/maps
00400000-00452000 r-xp 00000000 08:01 131074  /bin/cat       (text)
00651000-00652000 r--p 00051000 08:01 131074  /bin/cat       (rodata)
00652000-00653000 rw-p 00052000 08:01 131074  /bin/cat       (data)
01a3a000-01a5b000 rw-p 00000000 00:00 0       [heap]
7f2a1b000000-7f2a1b021000 rw-p 00000000 00:00 0              (anon mmap)
7f2a1b6e2000-7f2a1b8a7000 r-xp 00000000 08:01 393219 /lib/libc.so.6
7ffe7e5c0000-7ffe7e5e1000 rw-p 00000000 00:00 0       [stack]
```

The kernel uses **Address Space Layout Randomisation** (ASLR) to randomise the base addresses of the stack, heap, and memory-mapped regions. This makes it harder for attackers to predict the location of code or data, mitigating buffer overflow and return-oriented programming (ROP) attacks.

Example 4.3 (Inspecting Process Memory in C). The following program prints the addresses of its own memory segments:

```c
#include <stdio.h>
#include <stdlib.h>

int global_init = 42;        /* Data segment */
int global_uninit;            /* BSS segment */

int main(void) {
    int local_var = 10;       /* Stack */
    int *heap_var = malloc(sizeof(int));  /* Heap */
    *heap_var = 20;
```

```
    printf("Text  (main):     %p\n", (void *)main);
    printf("Data  (global):   %p\n", (void *)&global_init);
    printf("BSS   (uninit):   %p\n", (void *)&global_uninit);
    printf("Heap  (malloc):   %p\n", (void *)heap_var);
    printf("Stack (local):    %p\n", (void *)&local_var);
    printf("Lib   (printf):   %p\n", (void *)printf);

    free(heap_var);
    return 0;
}
```

Typical output on x86-64 Linux (with ASLR):

```
Text  (main):     0x55a3e4401169
Data  (global):   0x55a3e4404010
BSS   (uninit):   0x55a3e4404014
Heap  (malloc):   0x55a3e53a62a0
Stack (local):    0x7ffd2a3b1e4c
Lib   (printf):   0x7f8c2b4560f0
```

Note the large gaps between segments: text/data near `0x55...`, heap slightly above, libraries and stack near `0x7f...`. ASLR randomises these base addresses on each execution.

4.4 The Process Control Block

Every process in the system is represented within the kernel by a data structure called the **Process Control Block** (PCB). This structure contains all the information the operating system needs to manage the process.

Definition 4.5 (Process Control Block). The Process Control Block (PCB) is a kernel data structure that stores the complete execution context of a process. It serves as the repository for all information needed to suspend a process, switch to another, and later resume the original process exactly where it left off.

The PCB typically contains:

Identification

- Process ID (PID): A unique integer identifying the process
- Parent process ID (PPID): The PID of the process that created this one
- User ID (UID) and Group ID (GID): Credentials for access control

CPU State (Register Context)

- Program counter (PC / RIP on x86-64): Address of next instruction
- Stack pointer (SP / RSP): Top of the process's kernel or user stack
- General-purpose registers: RAX, RBX, RCX, … (on x86-64)
- Floating-point / SIMD registers: XMM, YMM, ZMM registers
- Status register (RFLAGS): Condition codes, interrupt enable flag

Memory Management

- Page table base register (CR3 on x86-64): Points to the process's page table hierarchy
- Virtual memory area (VMA) list: Describes each mapped region (text, data, heap, stack, mmap'd files)
- Memory limits: Maximum heap size, stack size, total virtual memory

Scheduling Information

- Process state: Running, Ready, Waiting, etc.
- Priority: Static and dynamic priority values
- Scheduling class: SCHED_NORMAL, SCHED_FIFO, SCHED_RR, etc.
- CPU time consumed: User time, system time, cumulative children times
- Processor affinity mask: Which CPUs this process may run on

I/O and File State

- File descriptor table: Array of pointers to kernel file structures
- Current working directory: Reference to the directory inode
- Root directory: For chroot environments

Signal State

- Pending signals: Bitmask of signals delivered but not yet handled
- Signal handlers: Function pointers for each signal number
- Signal mask: Which signals are currently blocked

On Linux, the PCB is the `task_struct` structure, defined in `include/linux/sched.h`. It is one of the largest structures in the kernel, containing over 200 fields and weighing approximately 6 KB per process.

Example 4.4 (Examining Linux task_struct). Key fields of the Linux `task_struct` include:

```
struct task_struct {
    volatile long           state;          /* process state */
    void                   *stack;          /* kernel stack pointer */
    unsigned int            flags;          /* process flags (PF_*) */
    int                     prio;           /* dynamic priority */
```

```
    int                     static_prio;     /* static priority */
    struct sched_entity     se;              /* CFS scheduling entity */
    struct mm_struct       *mm;              /* memory descriptor */
    struct files_struct    *files;           /* open file table */
    pid_t                   pid;             /* process ID */
    pid_t                   tgid;            /* thread group ID */
    struct task_struct     *parent;          /* parent process */
    struct list_head        children;        /* list of child processes */
    struct signal_struct   *signal;          /* signal handling */
    /* ... 200+ more fields ... */
};
```

The `mm` field points to the memory descriptor (`mm_struct`) which holds the page table pointer (`pgd`), the list of VMAs, and memory usage counters. When two threads belong to the same process, they share the same `mm_struct`.

4.4 Process Creation

The mechanism by which new processes come into existence differs fundamentally between Unix-like systems and Windows.

4.4.1 The Unix Model: fork() and exec()

Unix process creation follows a two-step pattern: **fork** creates a copy of the calling process, and **exec** replaces that copy's address space with a new program. This separation of concerns is one of Unix's most influential design decisions.

Definition 4.6 (fork). The `fork()` system call creates a new process (the child) that is an almost exact copy of the calling process (the parent). The child receives a copy of the parent's address space, file descriptor table, signal handlers, and other attributes. The child gets a new PID. `fork()` returns the child's PID to the parent and 0 to the child.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int x = 42;

    pid_t pid = fork();
```

```
    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        /* Child process: has a COPY of x */
        x = 99;
        printf("Child: x = %d (PID %d)\n", x, getpid());
    } else {
        /* Parent process: x is unchanged */
        wait(NULL);
        printf("Parent: x = %d (PID %d)\n", x, getpid());
    }

    return 0;
}
```

Output:

```
Child: x = 99 (PID 12346)
Parent: x = 42 (PID 12345)
```

The parent and child each have their own copy of `x`. Modifying it in one process has no effect on the other. In practice, modern kernels implement `fork()` using **copy-on-write** (COW): the parent and child initially share the same physical pages, marked read-only. A page is duplicated only when either process attempts to write to it, dramatically reducing the cost of `fork()` for processes that immediately call `exec()`.

Theorem 4.1 (Copy-on-Write Efficiency). Let P be a process with n pages of virtual memory. Under copy-on-write semantics, `fork()` requires $O(n)$ page table entries to be duplicated and marked read-only, but only $O(1)$ physical pages are allocated initially. If the child immediately calls `exec()`, only a small constant number of pages (stack, kernel structures) are ever physically copied, giving an effective cost of $O(1)$ memory for the fork+exec pattern.

4.4.2 The exec() Family

After `fork()`, the child typically calls one of the `exec()` family of functions to replace its address space with a new program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```c
int main(void) {
    pid_t pid = fork();

    if (pid == 0) {
        /* Replace this process's image with /bin/ls */
        char *args[] = {"ls", "-la", "/tmp", NULL};
        execvp("ls", args);

        /* If exec returns, it failed */
        perror("exec failed");
        _exit(1);
    } else {
        int status;
        waitpid(pid, &status, 0);
        printf("Child exited with status %d\n", WEXITSTATUS(status));
    }

    return 0;
}
```

The `exec()` call does not create a new process. It replaces the calling process's text, data, heap, and stack with those of the new program. The PID remains the same. Open file descriptors are preserved unless marked with `FD_CLOEXEC` (close-on-exec).

The `exec()` family includes six variants:

Function	Path	Arguments	Environment
`execl`	Full path	Variadic list	Inherited
`execv`	Full path	Array	Inherited
`execlp`	PATH search	Variadic list	Inherited
`execvp`	PATH search	Array	Inherited
`execle`	Full path	Variadic list	Explicit
`execvpe`	PATH search	Array	Explicit

4.4.3 The Windows Model: CreateProcess()

Windows takes a fundamentally different approach. Rather than separating creation and program loading, `CreateProcess()` performs both in a single call:

```
BOOL CreateProcessW(
    LPCWSTR               lpApplicationName,    /* executable path */
    LPWSTR                lpCommandLine,         /* command line string */
    LPSECURITY_ATTRIBUTES lpProcessAttributes,   /* security for process handle */
    LPSECURITY_ATTRIBUTES lpThreadAttributes,    /* security for thread handle */
    BOOL                  bInheritHandles,       /* handle inheritance flag */
    DWORD                 dwCreationFlags,       /* creation flags */
    LPVOID                lpEnvironment,         /* environment block */
    LPCWSTR               lpCurrentDirectory,    /* working directory */
    LPSTARTUPINFOW        lpStartupInfo,         /* startup info */
    LPPROCESS_INFORMATION lpProcessInformation   /* output: handles and IDs */
);
```

The single-call design means Windows never creates a process that is an exact copy of the parent. The child process always begins executing a specified program. This avoids the overhead of copying the parent's address space (even with COW) and the complexity of the fork+exec pattern, but at the cost of flexibility.

4.4.4 Process Hierarchies

In Unix-like systems, processes form a tree rooted at `init` (PID 1). Every process (except `init`) has exactly one parent, and may have zero or more children. This hierarchy is visible through the `pstree` command:

```
systemd(1)──┬──sshd(892)──sshd(1234)──bash(1235)──vim(1240)
            ├──nginx(900)──┬──nginx(901)
            │              ├──nginx(902)
            │              └──nginx(903)
            └──cron(850)
```

The process tree determines signal propagation (a `SIGHUP` to a session leader reaches all members), zombie reaping (the parent must call `wait()`), and resource accounting (CPU time can be summed across a subtree).

4.5.5 Process Groups and Sessions

Unix systems organise processes into **process groups** and **sessions** to support job control in interactive shells.

Definition 4.6a (Process Group and Session). A **process group** is a collection of related processes (typically a pipeline) identified by a Process Group ID (PGID), which equals the PID of the group leader. A **session** is a collection of process groups associated with a controlling terminal, identified by a Session ID (SID) equal to the

PID of the session leader. Each session has at most one **foreground process group** (which receives terminal input and signals like SIGINT from Ctrl+C) and zero or more **background process groups**.

When you type `ls | grep foo | wc -l` in a shell, the three processes form a single process group. Pressing Ctrl+C sends SIGINT to the entire foreground process group, not just one process.

```
Session (SID = bash PID)
├── Foreground process group (PGID = ls PID)
│   ├── ls    (PID=1001, PGID=1001)
│   ├── grep  (PID=1002, PGID=1001)
│   └── wc    (PID=1003, PGID=1001)
└── Background process groups
    └── ./server &  (PID=1004, PGID=1004)
```

The `setpgid()` system call changes a process's group, and `setsid()` creates a new session. Daemons typically call `setsid()` to detach from the controlling terminal, preventing terminal signals from reaching them.

Programmer's Perspective

Programmer's Perspective: fork() in Go and Its Perils. Go deliberately does not expose `fork()` in its standard library. The reason is subtle but important: `fork()` creates a child with a single thread of execution (the one that called `fork()`), but Go programs are inherently multi-threaded due to the runtime's garbage collector, network poller, and scheduler threads. After `fork()`, the child process has copies of all the mutexes from the parent, but only one of the threads that might have held those mutexes — a recipe for deadlocks. The Go standard library's `os/exec` package uses a carefully orchestrated `fork()/exec()` combination (via `posix_spawn()` on some platforms) that minimises the window between forking and exec-ing. If you need to call `fork()` without `exec()` in Go, you must use `syscall.ForkExec()` or `syscall.RawSyscall(SYS_CLONE, ...)`, and you must not call any Go runtime functions in the child before `exec()`.

4.5 Process Termination

A process terminates either voluntarily or involuntarily:

Voluntary termination:

- Normal exit: The process calls `exit(status)` with a zero status code

- Error exit: The process calls `exit(status)` with a non-zero status code
- Return from `main()`: Equivalent to calling `exit()` with the return value

Involuntary termination:

- Fatal signal: The process receives a signal whose default action is termination (e.g., `SIGSEGV`, `SIGKILL`)
- Killed by another process: Another process sends `SIGKILL` or `SIGTERM`
- Resource limit exceeded: The kernel kills the process for exceeding a resource limit (e.g., the OOM killer)

Upon termination, the kernel performs cleanup:

1. Close all open file descriptors
2. Release all memory mappings (unmap pages, free page table entries)
3. Remove the process from the scheduler's queues
4. Notify the parent via `SIGCHLD`
5. Re-parent any children to `init` or the designated subreaper
6. Retain the `task_struct` with exit status (zombie state) until the parent calls `wait()`

4.5.1 posix_spawn(): A Modern Alternative

The fork-then-exec pattern has a longstanding problem on systems without virtual memory (embedded systems) or with very large processes (where even COW page table duplication is expensive). POSIX provides `posix_spawn()` as a combined creation-and-exec operation:

```c
#include <spawn.h>
#include <stdio.h>
#include <sys/wait.h>

extern char **environ;

int main(void) {
    pid_t pid;
    char *argv[] = {"ls", "-la", "/tmp", NULL};

    int status = posix_spawn(&pid, "/bin/ls", NULL, NULL, argv, environ);
    if (status != 0) {
        fprintf(stderr, "posix_spawn failed: %d\n", status);
        return 1;
    }
```

```
    printf("Spawned child PID: %d\n", pid);
    waitpid(pid, &status, 0);
    printf("Child exited with status %d\n", WEXITSTATUS(status));

    return 0;
}
```

On Linux, `posix_spawn()` is implemented in glibc using `clone()` with `CLONE_VM | CLONE_VFORK`, which avoids duplicating the parent's page tables entirely. The child shares the parent's address space (as a vfork child) and immediately calls `exec()`. This makes `posix_spawn()` faster than `fork() + exec()` for large processes.

4.5.2 Process Resource Limits

Each process has a set of resource limits that constrain its resource consumption. These limits prevent runaway processes from consuming all available resources:

```
#include <sys/resource.h>
#include <stdio.h>

int main(void) {
    struct rlimit rl;

    /* Query the maximum stack size */
    getrlimit(RLIMIT_STACK, &rl);
    printf("Stack: soft=%lu KB, hard=%lu KB\n",
           rl.rlim_cur / 1024, rl.rlim_max / 1024);

    /* Query the maximum number of open files */
    getrlimit(RLIMIT_NOFILE, &rl);
    printf("Open files: soft=%lu, hard=%lu\n",
           rl.rlim_cur, rl.rlim_max);

    /* Query the maximum virtual memory */
    getrlimit(RLIMIT_AS, &rl);
    if (rl.rlim_cur == RLIM_INFINITY)
        printf("Virtual memory: unlimited\n");
    else
        printf("Virtual memory: soft=%lu MB\n", rl.rlim_cur / (1024*1024));
```

```
    /* Query the CPU time limit */
    getrlimit(RLIMIT_CPU, &rl);
    if (rl.rlim_cur == RLIM_INFINITY)
        printf("CPU time: unlimited\n");
    else
        printf("CPU time: soft=%lu sec\n", rl.rlim_cur);

    return 0;
}
```

Key resource limits on Linux include:

Limit	Resource	Default (soft)	Effect of exceeding
`RLIMIT_STACK`	Stack size	8 MB	SIGSEGV
`RLIMIT_NOFILE`	Open files	1024	EMFILE error
`RLIMIT_AS`	Virtual memory	Unlimited	ENOMEM error
`RLIMIT_CPU`	CPU time	Unlimited	SIGXCPU, then SIGKILL
`RLIMIT_NPROC`	Processes per user	Varies	EAGAIN on fork()
`RLIMIT_CORE`	Core dump size	0 (disabled)	Truncated core dump

The soft limit is the effective limit; it can be raised up to the hard limit by the process itself. Only root can raise the hard limit.

4.6 Context Switching

When the scheduler decides to run a different process, it must save the current process's execution state and load the saved state of the next process. This operation is called a **context switch**.

Definition 4.7 (Context Switch). A context switch is the mechanism by which the kernel saves the CPU state (registers, program counter, stack pointer, floating-point state) of the currently running process into its PCB, and loads the corresponding state from the PCB of the next process to run. The CPU then resumes executing the new process from where it previously left off.

4.6.1 The Context Switch Mechanism

On x86-64 Linux, a context switch involves these steps:

1. **Enter kernel mode**: A timer interrupt, system call, or I/O interrupt transfers control from user mode to the kernel.
2. **Save user-space registers**: The interrupt/syscall entry code pushes the user-space register state onto the kernel stack.
3. **Scheduler decision**: The kernel's scheduler selects the next process to run based on priorities, fairness, and scheduling policy.
4. **Switch kernel stacks**: The kernel saves the current process's kernel stack pointer in its `task_struct` and loads the next process's kernel stack pointer.
5. **Switch address space**: The kernel loads the next process's page table base register (CR3 on x86-64). This invalidates the TLB, though PCID (Process Context IDentifier) can avoid full flushes.
6. **Switch CPU state**: Floating-point registers (FPU/SSE/AVX state) are saved and restored lazily or eagerly, depending on kernel configuration.
7. **Return to user space**: The kernel pops the new process's user-space registers from its kernel stack and executes `iretq` (or `sysret`) to return to user mode.

4.6.2 The Cost of Context Switching

Context switches are not free. The direct costs include:

- **Register save/restore**: Saving and loading 16+ general-purpose registers, plus floating-point state (up to 2 KB for AVX-512)
- **TLB flush**: Changing the page table base register invalidates TLB entries. Without PCID, the entire TLB is flushed, causing a cascade of page table walks for subsequent memory accesses.
- **Cache pollution**: The new process's working set will evict the previous process's cache lines. The L1 cache (32–64 KB) is typically polluted within microseconds; the L2 and L3 caches are shared and experience gradual displacement.
- **Pipeline flush**: The CPU's instruction pipeline must be drained when switching to a different instruction stream. Branch predictor state (often per-address, not per-process) may also suffer.

Theorem 4.2 (Context Switch Overhead Bound). Let T_{direct} be the time to save and restore register state, T_{TLB} be the amortised cost of TLB refills after a flush, and T_{cache} be the amortised cost of cache misses as the new process warms its working set. The total cost of a context switch is:

$$T_{\text{switch}} = T_{\text{direct}} + T_{\text{TLB}} + T_{\text{cache}}$$

On modern x86-64 hardware, $T_{\text{direct}} \approx 1$–$3\ \mu$s, but $T_{\text{TLB}} + T_{\text{cache}}$ can dominate, pushing the effective cost to 5–$30\ \mu$s depending on working set size and TLB pressure.

4.6.3 Hardware Support for Context Switching

Modern processors provide features to reduce context switch costs:

- **PCID (Process Context Identifiers)**: x86-64 CR3 carries a 12-bit PCID that tags TLB entries with the process that created them. On a context switch, the kernel sets the new process's PCID without flushing the TLB. This eliminates the TLB refill cost for processes that have been recently scheduled.
- **XSAVE/XRSTOR**: These instructions save and restore extended processor state (SSE, AVX, AVX-512, MPX) in a single operation, using a state-component bitmap to skip unused register banks.
- **Lazy FPU switching**: The kernel can mark the FPU as "not available" when switching to a new process. If the new process uses floating-point instructions, a `#NM` (Device Not Available) exception triggers the kernel to load the FPU state on demand. This avoids the cost of saving/restoring FPU state for processes that do not use it. Modern kernels (Linux 4.6+) have moved to eager FPU restore due to Spectre-class vulnerabilities.

Example 4.5 (Measuring Context Switch Time). The classic way to measure context switch time is to create two processes connected by a pipe. Each process writes a byte to the pipe and reads a byte from the other end, forcing a context switch each time:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

#define ITERATIONS 100000

int main(void) {
    int pipe1[2], pipe2[2];
    pipe(pipe1);
    pipe(pipe2);
```

```
    char byte = 'x';

    if (fork() == 0) {
        /* Child: read from pipe1, write to pipe2 */
        close(pipe1[1]);
        close(pipe2[0]);
        for (int i = 0; i < ITERATIONS; i++) {
            read(pipe1[0], &byte, 1);
            write(pipe2[1], &byte, 1);
        }
        _exit(0);
    }

    /* Parent: write to pipe1, read from pipe2 */
    close(pipe1[0]);
    close(pipe2[1]);

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < ITERATIONS; i++) {
        write(pipe1[1], &byte, 1);
        read(pipe2[0], &byte, 1);
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed = (end.tv_sec - start.tv_sec) * 1e9 +
                     (end.tv_nsec - start.tv_nsec);
    printf("Average round-trip: %.1f ns\n", elapsed / ITERATIONS);
    printf("Estimated context switch: %.1f ns\n", elapsed / (2 * ITERATIONS));

    return 0;
}
```

On a modern Intel processor, this typically yields context switch times of 2–5 microseconds, though the indirect costs of cache and TLB pollution are not captured by this measurement.

4.7 Threads

A thread is an independent flow of control within a process. While processes provide isolation (separate address spaces), threads provide concurrency within a shared address space. Multiple threads within the same process share the same code, data, and heap, but each has its own stack, program counter, and register state.

Definition 4.8 (Thread). A thread (also called a lightweight process) is the smallest unit of CPU scheduling. A thread consists of a thread ID, a program counter, a register set, and a stack. It shares the code section, data section, heap, open files, and signal handlers with other threads belonging to the same process.

4.7.1 Why Threads?

The motivation for threads comes from several directions:

Responsiveness. A multi-threaded GUI application can handle user input in one thread while performing computation in another. Without threads, the entire application freezes during a long computation.

Resource sharing. Threads within the same process share memory naturally. Inter-thread communication is as simple as reading and writing shared variables (with proper synchronisation). Inter-process communication requires explicit mechanisms (pipes, shared memory, sockets).

Economy. Creating a thread is 10–100x cheaper than creating a process. A `fork()` on Linux requires duplicating the page table, file descriptor table, signal handlers, and other PCB fields. A thread creation (via `clone()` with `CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND`) shares all of these, requiring only a new stack and kernel task structure.

Scalability. On a multiprocessor system, threads in the same process can execute simultaneously on different cores, providing true parallelism. A single-threaded process is limited to one core regardless of how many are available.

Example 4.6 (Threads vs Processes for a Web Server). Consider a web server handling 10,000 concurrent connections:

- **Process-per-connection** (Apache prefork): 10,000 processes, each with its own address space. At 6 KB per `task_struct` plus several MB of virtual memory mappings, this consumes significant kernel resources and suffers from context switch overhead.
- **Thread-per-connection** (Apache worker): 10,000 threads sharing a single address space. Kernel overhead is reduced (shared page tables, file tables), but

10,000 kernel threads still strain the scheduler.

- **Event-driven with thread pool** (Nginx, Go net/http): A small number of threads (typically matching the CPU count) handle all connections using non-blocking I/O and event multiplexing (`epoll`, `kqueue`). This minimises both memory usage and context switch overhead.

4.7.2 Thread State

Each thread has its own:

- **Program counter**: Where in the code the thread is currently executing
- **Register set**: The thread's own copy of CPU registers
- **Stack**: A separate stack for local variables and function call frames
- **Thread-local storage**: Per-thread global variables
- **Scheduling state**: Ready, running, blocked, etc.

Threads within the same process share:

- **Address space**: Text, data, heap, memory-mapped regions
- **Open file descriptors**: The file descriptor table
- **Signal handlers**: The signal disposition table (but signal masks are per-thread)
- **Process ID**: All threads share the same PID (on Linux, the TGID)
- **Credentials**: UID, GID, supplementary groups

4.8 Thread Models

The relationship between application-level threads and kernel-level threads is characterised by three models.

4.8.1 Many-to-One (M:1) — User-Level Threads

In the M:1 model, multiple user-level threads are mapped onto a single kernel thread. The thread library manages scheduling, context switching, and synchronisation entirely in user space, without kernel involvement.

Advantages:

- Thread operations (create, switch, synchronise) are extremely fast — no system calls required
- Portable across operating systems that lack kernel thread support
- Scheduling can be customised (e.g., application-specific priority schemes)

Disadvantages:

- If one thread makes a blocking system call, all threads in the process block
- Cannot exploit multiple processors — the kernel sees only one schedulable entity
- A page fault in one thread blocks all threads

Historical examples include GNU Pth (POSIX user-level threads) and early Java green threads (before JDK 1.2).

4.8.2 One-to-One (1:1) — Kernel Threads

In the 1:1 model, each user-level thread maps directly to a kernel thread. The kernel handles all scheduling, context switching, and synchronisation.

Advantages:

- Threads can run in parallel on multiple CPUs
- A blocking system call in one thread does not block others
- Kernel can preempt individual threads for fair scheduling

Disadvantages:

- Thread creation requires a system call (overhead of ~2–10 microseconds)
- Each thread consumes kernel resources (kernel stack, `task_struct`)
- Scalability limited by kernel thread capacity (typically tens of thousands, not millions)

Linux (via NPTL), modern Windows, and macOS all use the 1:1 model.

4.8.3 Many-to-Many (M:N) — Hybrid Model

The M:N model maps M user-level threads onto N kernel threads, where typically $M \gg N$. A user-space scheduler (sometimes called a **scheduler activations** layer) multiplexes user threads onto the available kernel threads.

Advantages:

- Combines the scalability of user-level threads with the parallelism of kernel threads
- A blocking user thread can be parked while the kernel thread serves another
- The application can create millions of user-level threads without exhausting kernel resources

Disadvantages:

- Complex implementation: requires cooperation between user-space scheduler and kernel
- Difficult to handle blocking system calls transparently
- Debugging is harder when the mapping between user and kernel threads is indirect

Definition 4.9 (M:N Threading Model). In the M:N threading model, M application-level threads are multiplexed onto N kernel threads, where $1 \leq N \leq M$. The user-space runtime scheduler assigns user threads to available kernel threads, migrating user threads between kernel threads as needed to balance load and avoid blocking.

The M:N model is used by Go's goroutine scheduler, Erlang's BEAM VM, and historically by Solaris's lightweight process (LWP) system.

4.9 Green Threads and Goroutines

The concept of **green threads** — threads managed entirely by a runtime rather than the operating system kernel — has a long history. The term originated from the Green Team at Sun Microsystems, who implemented user-level threads for Java before native thread support was widely available.

Modern green thread implementations have evolved far beyond simple user-level threading. The most prominent example is Go's **goroutines**, which implement a sophisticated M:N scheduling model.

4.9.1 What Makes Goroutines Different

A goroutine is a function executing concurrently with other goroutines in the same address space. Goroutines differ from OS threads in several critical ways:

Property	OS Thread	Goroutine
Stack size (initial)	1–8 MB (fixed)	2–8 KB (growable)
Creation cost	~2–10 microseconds	~0.3 microseconds
Context switch cost	~1–5 microseconds	~0.2 microseconds
Maximum practical count	~10,000	~1,000,000+
Scheduling	Kernel preemptive	Runtime cooperative + preemptive (since Go 1.14)

Property	OS Thread	Goroutine
Memory overhead per unit	~8 KB kernel stack + user stack	~2–8 KB total

The key innovation is the **segmented stack** (originally) and **copyable stack** (since Go 1.4). Goroutine stacks start small (a few kilobytes) and grow dynamically by copying to a larger allocation when needed. This allows a Go program to run millions of goroutines without exhausting memory.

```go
package main

import (
    "fmt"
    "sync"
)

func worker(id int, wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Printf("Worker %d starting\n", id)
    // Simulate work
    sum := 0
    for i := 0; i < 1000; i++ {
        sum += i
    }
    fmt.Printf("Worker %d done, sum = %d\n", id, sum)
}

func main() {
    var wg sync.WaitGroup

    // Launch 100,000 goroutines --- try this with OS threads
    for i := 0; i < 100000; i++ {
        wg.Add(1)
        go worker(i, &wg)
    }

    wg.Wait()
    fmt.Println("All workers complete")
```

```
}
```

4.9.2 The GMP Model

Go's goroutine scheduler uses the **GMP model**, named after its three core abstractions:

- **G (Goroutine)**: A goroutine structure containing the stack, program counter, and scheduling metadata. Each G represents a unit of concurrent work.
- **M (Machine)**: An OS thread. Each M executes Go code by running goroutines. Ms are created on demand and cached for reuse. The number of Ms can exceed GOMAXPROCS when some are blocked in system calls.
- **P (Processor)**: A logical processor that provides the context needed to execute Go code. Each P holds a local run queue of goroutines. The number of Ps equals GOMAXPROCS (default: number of CPU cores).

The scheduling invariant is: **a goroutine (G) can only run on an OS thread (M) that is attached to a processor (P)**. This three-level structure enables efficient scheduling:

```
┌──────────────────────────────────────────────────────────┐
│                    Global Run Queue                      │
│               [G7] [G8] [G9] [G10] ...                   │
└──────────────┬──────────────────────┬────────────────────┘
               │                      │
┌──────────────▼──────────┐  ┌──────────────▼──────────┐
│    P0                   │ │    P1                   │
│  Local Queue:           │ │  Local Queue:           │
│  [G1] [G2] [G3]  │ │  [G4] [G5] [G6]   │
│                         │ │                         │
│  Running: ──────────────│ │  Running: ──────────────│
│     ▼                   │ │     ▼                   │
│  ┌─────┐                │ │  ┌─────┐                │
│  │ M0  │                │ │  │ M1  │                │
│  └─────┘                │ │  └─────┘                │
└─────────────────────────┘  └─────────────────────────┘
```

When a goroutine makes a blocking system call, the M executing it detaches from its P. The P is then handed to another M (or a new M is created) so that the remaining goroutines in P's run queue can continue executing. When the system call completes, the M attempts to reacquire a P; if none is available, the goroutine is placed on the global run queue and the M parks itself.

Programmer's Perspective

Programmer's Perspective: Go's GMP Scheduler in Detail. The Go scheduler implements **work-stealing** to balance load across Ps. When a P's local run queue is empty, it attempts (in order): (1) check the global run queue (every 61st scheduling tick, to prevent starvation), (2) check the network poller for ready goroutines, (3) steal half the goroutines from another P's local queue. The `runtime.GOMAXPROCS(n)` function sets the number of Ps. Setting it to 1 gives cooperative concurrency on a single core; setting it to `runtime.NumCPU()` (the default since Go 1.5) enables full parallelism. The scheduler became truly preemptive in Go 1.14 with the introduction of asynchronous preemption via `SIGURG` signals: a background thread (`sysmon`) detects goroutines that have been running for more than 10 ms and sends a signal to preempt them, even if they contain no function calls (tight loops without preemption points were a known issue before Go 1.14).

4.9.3 Goroutine Preemption

Early versions of Go's scheduler were **cooperatively scheduled**: goroutines yielded control only at specific preemption points (function calls, channel operations, system calls). A goroutine executing a tight computational loop without function calls could monopolise its P indefinitely, starving other goroutines.

Go 1.14 introduced **asynchronous preemption**. The `sysmon` goroutine (a special system monitor) runs on its own M without a P and periodically checks whether any goroutine has been running for too long (default threshold: 10 ms). If so, it sends a `SIGURG` signal to the offending M, which triggers a signal handler that saves the goroutine's state and schedules another goroutine from the run queue.

```go
package main

import (
    "fmt"
    "runtime"
    "time"
)

func busyLoop() {
    // Before Go 1.14, this would block the P forever
    // After Go 1.14, the scheduler preempts this goroutine
    for {
```

```
        // Pure computation, no function calls, no preemption points
        _ = 1 + 1
    }
}

func main() {
    runtime.GOMAXPROCS(1) // Single P

    go busyLoop()

    // This goroutine can still run thanks to async preemption
    time.Sleep(100 * time.Millisecond)
    fmt.Println("Main goroutine is not starved")
}
```

4.10 Linux clone() and Thread Creation

On Linux, threads and processes are both created by the `clone()` system call. The difference lies in the flags passed to `clone()`, which determine how much state is shared between the parent and child.

Definition 4.10 (clone System Call). The Linux `clone()` system call creates a new execution context (task). Its behaviour is controlled by a bitmask of flags that specify which resources are shared between the calling task and the new task, and which are copied. Both `fork()` and `pthread_create()` are implemented in terms of `clone()` with different flag combinations.

The key `clone()` flags are:

Flag	Effect
`CLONE_VM`	Share the virtual address space (memory mappings, heap)
`CLONE_FS`	Share filesystem information (root, cwd, umask)
`CLONE_FILES`	Share the file descriptor table
`CLONE_SIGHAND`	Share signal handlers
`CLONE_THREAD`	Place the new task in the same thread group (share PID/TGID)

Flag	Effect
CLONE_PARENT	New task has the same parent as the calling task
CLONE_NEWNS	New mount namespace (for containers)
CLONE_NEWPID	New PID namespace (for containers)

A `fork()` is equivalent to `clone()` with no sharing flags. A `pthread_create()` is equivalent to `clone()` with `CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD`.

Example 4.7 (Creating a Thread with clone()). The following C program uses `clone()` directly to create a thread that shares the parent's address space:

```
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#define STACK_SIZE (1024 * 1024)

static int shared_var = 0;

static int thread_func(void *arg) {
    shared_var = 42;
    printf("Child: shared_var = %d (PID=%d, TID=%d)\n",
           shared_var, getpid(), gettid());
    return 0;
}

int main(void) {
    char *stack = malloc(STACK_SIZE);
    if (!stack) {
        perror("malloc");
        return 1;
    }
```

```
    /* Stack grows downward on x86-64 */
    char *stack_top = stack + STACK_SIZE;

    int flags = CLONE_VM | CLONE_FS | CLONE_FILES |
                CLONE_SIGHAND | CLONE_THREAD | SIGCHLD;

    int tid = clone(thread_func, stack_top, flags, NULL);
    if (tid == -1) {
        perror("clone");
        return 1;
    }

    /* Wait for the child thread */
    sleep(1);
    printf("Parent: shared_var = %d (modified by child via shared VM)\n",
           shared_var);

    free(stack);
    return 0;
}
```

Because `CLONE_VM` is set, both parent and child operate on the same virtual address space. The child's modification to `shared_var` is visible to the parent. Without `CLONE_VM`, each would have its own copy (as with `fork()`).

4.11 POSIX Threads (pthreads)

The POSIX Threads standard (IEEE 1003.1c) provides a portable API for multi-threaded programming on Unix-like systems. On Linux, the pthreads library is implemented by NPTL (Native POSIX Threads Library), which uses the 1:1 threading model.

4.11.1 Thread Lifecycle

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int thread_id;
```

```
    int iterations;
} thread_arg_t;

void *compute(void *arg) {
    thread_arg_t *params = (thread_arg_t *)arg;
    long sum = 0;

    for (int i = 0; i < params->iterations; i++) {
        sum += i;
    }

    printf("Thread %d: sum = %ld\n", params->thread_id, sum);

    /* Return value accessible via pthread_join */
    long *result = malloc(sizeof(long));
    *result = sum;
    return result;
}

int main(void) {
    pthread_t threads[4];
    thread_arg_t args[4];

    for (int i = 0; i < 4; i++) {
        args[i].thread_id = i;
        args[i].iterations = 1000000 * (i + 1);

        int rc = pthread_create(&threads[i], NULL, compute, &args[i]);
        if (rc != 0) {
            fprintf(stderr, "pthread_create failed: %d\n", rc);
            return 1;
        }
    }

    /* Wait for all threads to complete */
    for (int i = 0; i < 4; i++) {
        long *result;
        pthread_join(threads[i], (void **)&result);
        printf("Thread %d returned: %ld\n", i, *result);
        free(result);
```

```
    }

    return 0;
}
```

Compile with: `gcc -pthread -o threads threads.c`

4.11.2 Thread Attributes

Thread creation can be customised via `pthread_attr_t`:

```
pthread_attr_t attr;
pthread_attr_init(&attr);

/* Set stack size to 4 MB (default is typically 8 MB on Linux) */
pthread_attr_setstacksize(&attr, 4 * 1024 * 1024);

/* Set scheduling policy */
pthread_attr_setschedpolicy(&attr, SCHED_FIFO);

/* Create a detached thread (no need to join) */
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);

pthread_t thread;
pthread_create(&thread, &attr, worker_func, NULL);

pthread_attr_destroy(&attr);
```

A **detached thread** releases its resources automatically upon termination, without requiring another thread to call `pthread_join()`. A **joinable thread** (the default) retains its exit status until joined.

4.12 Thread-Local Storage

Thread-local storage (TLS) provides a mechanism for each thread to have its own instance of a variable, even though the variable has the same name and is accessed through the same identifier in all threads.

Definition 4.11 (Thread-Local Storage). Thread-local storage (TLS) is a memory allocation mechanism where each thread in a process has its own private instance of a variable. TLS variables are global in scope (visible throughout the program) but

local in storage (each thread's copy is independent). Writes to a TLS variable in one thread do not affect the value seen by other threads.

4.12.1 TLS in C

The C11 standard introduced the `_Thread_local` storage class specifier (also available as `thread_local` via `<threads.h>`):

```c
#include <stdio.h>
#include <pthread.h>

/* Each thread gets its own copy of errno_local */
_Thread_local int errno_local = 0;

/* GCC/Clang extension (pre-C11) */
__thread int counter = 0;

void *worker(void *arg) {
    int id = *(int *)arg;
    counter = id * 100;
    errno_local = id;

    printf("Thread %d: counter=%d, errno_local=%d\n",
           id, counter, errno_local);
    return NULL;
}

int main(void) {
    pthread_t threads[4];
    int ids[4] = {0, 1, 2, 3};

    for (int i = 0; i < 4; i++) {
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }
    for (int i = 0; i < 4; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Main: counter=%d (main thread's copy)\n", counter);
    return 0;
}
```

4.12.2 TLS Implementation

On x86-64 Linux, TLS is implemented using the FS segment register. Each thread has a TLS block allocated at thread creation time. The FS base register (`IA32_FS_BASE` MSR, set via `arch_prctl(ARCH_SET_FS, addr)`) points to the thread's TLS area. Accessing a TLS variable compiles to a memory reference with an `%fs:` segment prefix:

```
; Access to a __thread variable compiles to:
movl %fs:counter@TPOFF, %eax    ; Load thread-local 'counter'
```

This single-instruction access makes TLS extremely efficient — no function calls, no hash table lookups. The cost is identical to accessing a regular global variable plus the segment prefix overhead (typically zero on modern x86-64 processors, which cache the FS base in the CPU).

4.12.3 TLS in Go

Go does not provide thread-local storage as a language feature. This is a deliberate design decision: because goroutines can migrate between OS threads (Ms in the GMP model), a TLS value that is logically "per-goroutine" cannot be stored in OS-level TLS. Instead, Go provides alternatives:

- **Context passing**: The `context.Context` type carries request-scoped values through function arguments
- **sync.Pool**: Per-processor caches that reduce contention for frequently allocated objects
- **runtime.LockOSThread()**: Pins a goroutine to its current OS thread, making OS-level TLS safe to use (used by CGo and graphics libraries that require thread affinity)

```go
package main

import (
    "context"
    "fmt"
)

type contextKey string

const requestIDKey contextKey = "requestID"

func handleRequest(ctx context.Context) {
    // Retrieve the per-request value from context
```

```go
    reqID := ctx.Value(requestIDKey).(string)
    fmt.Printf("Handling request: %s\n", reqID)
}

func main() {
    // Context replaces TLS for goroutine-scoped values
    ctx := context.WithValue(context.Background(), requestIDKey, "req-42")
    go handleRequest(ctx)

    ctx2 := context.WithValue(context.Background(), requestIDKey, "req-99")
    go handleRequest(ctx2)

    // Wait for goroutines (simplified; use sync.WaitGroup in production)
    fmt.Scanln()
}
```

Programmer's Perspective

Programmer's Perspective: Linux clone() Flags and Containers. The `clone()` system call is not just for creating threads — it is the foundation of Linux containers. By combining flags like `CLONE_NEWPID`, `CLONE_NEWNS`, `CLONE_NEWNET`, `CLONE_NEWUTS`, and `CLONE_NEWUSER`, `clone()` creates a new process that lives in isolated namespaces: it sees its own PID tree (starting from PID 1), its own filesystem mount hierarchy, its own network stack, its own hostname, and its own user/group mapping. Tools like Podman, Docker, and LXC use these namespace flags to create containers. The `unshare()` system call provides the same namespace isolation for an existing process, and `setns()` allows a process to join an existing namespace. Understanding `clone()` flags is the key to understanding how Linux containers provide process-level isolation without the overhead of full virtualisation.

4.13 Comparing Threading Models in Practice

The following table summarises the practical characteristics of the three threading models as implemented in real systems:

Aspect	Pthreads (1:1)	Go Goroutines (M:N)	Green Threads (M:1)
Implementation	Linux NPTL	Go runtime	User-space library

Aspect	Pthreads (1:1)	Go Goroutines (M:N)	Green Threads (M:1)
Kernel visibility	Full	Partial (sees Ms, not Gs)	None
Blocking syscall impact	Only calling thread blocks	M detaches from P; P continues	All threads block
CPU parallelism	Yes	Yes	No
Stack size	Fixed (default 8 MB)	Dynamic (initial 2–8 KB)	Varies
Max practical threads	~10,000	~1,000,000+	~1,000,000+ (single core)
Context switch cost	~1–5 microseconds	~0.2 microseconds	~0.05 microseconds
Preemption	Kernel-level	Runtime + signal-based	Cooperative only
Debugging	Standard tools (gdb, strace)	Specialised (delve)	Difficult

Example 4.8 (Goroutine vs Thread Memory Overhead). Consider running one million concurrent units of execution. With pthreads (default 8 MB stack per thread), the stack memory alone would require:

$$1{,}000{,}000 \times 8 \text{ MB} = 8 \text{ TB}$$

This exceeds the physical memory of virtually any system. With goroutines (initial 2 KB stack):

$$1{,}000{,}000 \times 2 \text{ KB} = 2 \text{ GB}$$

This fits comfortably in a modern server's memory. The goroutine stacks grow on demand, so most goroutines that are waiting (e.g., for channel operations) never grow beyond their initial allocation.

Theorem 4.3 (Scalability of M:N Threading). In an M:N threading model with M user-level threads, N kernel threads, and a work-stealing scheduler, the overhead

of scheduling M threads is $O(M/N)$ amortised per kernel thread. With N equal to the number of CPU cores C, the scheduler achieves $O(M/C)$ amortised overhead per core, compared to $O(M)$ total overhead for a centralised scheduler with a single global queue.

4.14 Kernel Threads

Modern operating systems use kernel threads not only to support user-level multithreading but also for internal kernel work. These kernel threads have no user-space address space (their `mm` pointer is `NULL` in the Linux `task_struct`) and execute entirely in kernel mode.

Common kernel threads on Linux include:

- **kthreadd** (PID 2): The parent of all kernel threads
- **ksoftirqd/N**: Processes software interrupts (one per CPU)
- **kworker/N:M**: Kernel worker threads for deferred work
- **migration/N**: Handles thread migration between CPUs (one per CPU)
- **rcu_preempt**, **rcu_sched**: Read-Copy-Update (RCU) processing
- **kswapd**: Page reclamation (swapping)
- **jbd2**: Journalling for ext4 filesystems
- **kcompactd**: Memory compaction

These threads are visible in `ps` output with square-bracket names:

```
$ ps aux | grep '\[' | head -10
root         2  0.0  0.0      0     0 ?  S    Apr10   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?  I<   Apr10   0:00 [rcu_gp]
root         4  0.0  0.0      0     0 ?  I<   Apr10   0:00 [rcu_par_gp]
root         5  0.0  0.0      0     0 ?  I<   Apr10   0:00 [slub_flushwq]
root         9  0.0  0.0      0     0 ?  I<   Apr10   0:00 [mm_percpu_wq]
root        10  0.0  0.0      0     0 ?  I    Apr10   0:12 [rcu_preempt]
root        11  0.0  0.0      0     0 ?  S    Apr10   0:01 [migration/0]
root        12  0.0  0.0      0     0 ?  S    Apr10   0:00 [idle_inject/0]
root        13  0.0  0.0      0     0 ?  S    Apr10   0:00 [cpuhp/0]
root        14  0.0  0.0      0     0 ?  S    Apr10   0:00 [cpuhp/1]
```

4.15 Parallelism and Amdahl's Law

The primary motivation for multithreading is to exploit parallelism on multicore processors. But how much speedup can we actually achieve? Amdahl's Law provides the

fundamental answer.

Theorem 4.4 (Amdahl's Law). If a fraction f of a program's execution time is inherently sequential (cannot be parallelised), and the remaining fraction $1 - f$ can be perfectly parallelised across N processors, then the maximum speedup is:

$$S(N) = \frac{1}{f + \frac{1-f}{N}}$$

As $N \to \infty$:

$$S(\infty) = \frac{1}{f}$$

Even with infinitely many processors, the speedup is limited by the serial fraction.

Example 4.9 (Amdahl's Law in Practice). A web server spends 10% of its request-handling time in serial operations (accepting connections, logging) and 90% in parallelisable work (request parsing, database queries, response generation).

Cores (N)	**Speedup $S(N)$**	**Efficiency $S(N)/N$**
1	1.00	100%
2	1.82	91%
4	3.08	77%
8	4.71	59%
16	6.40	40%
64	8.77	14%
∞	10.00	0%

With 10% serial work, the maximum speedup is 10x regardless of how many cores are available. Adding cores beyond 16 provides diminishing returns.

Definition 4.12 (Gustafson's Law). While Amdahl's Law assumes a fixed problem size, Gustafson's Law assumes that the problem size scales with the number of processors. If a parallel program running on N processors spends fraction s of its time in serial code and fraction $1 - s$ in parallel code, the scaled speedup is:

$$S_G(N) = N - s \cdot (N - 1) = s + N \cdot (1 - s)$$

This gives a much more optimistic view of parallelism: as we add more processors, we can solve proportionally larger problems, and the serial fraction becomes less significant.

4.15.1 Data Parallelism vs Task Parallelism

Two fundamental patterns of parallel computation arise from the thread model:

Data parallelism distributes data across threads, with each thread performing the same operation on its subset. This is the pattern used in parallel array processing, map-reduce computations, and GPU programming:

```go
package main

import (
    "fmt"
    "runtime"
    "sync"
)

func parallelSum(data []int) int64 {
    n := runtime.NumCPU()
    var total int64
    var mu sync.Mutex
    var wg sync.WaitGroup

    chunkSize := len(data) / n

    for i := 0; i < n; i++ {
        start := i * chunkSize
        end := start + chunkSize
        if i == n-1 {
            end = len(data)
        }

        wg.Add(1)
        go func(s, e int) {
            defer wg.Done()
            var localSum int64
            for j := s; j < e; j++ {
                localSum += int64(data[j])
```

```
            }
            mu.Lock()
            total += localSum
            mu.Unlock()
        }(start, end)
    }

    wg.Wait()
    return total
}

func main() {
    data := make([]int, 100_000_000)
    for i := range data {
        data[i] = i % 100
    }

    sum := parallelSum(data)
    fmt.Printf("Sum: %d\n", sum)
}
```

Task parallelism distributes different tasks across threads, with each thread performing a different operation. This is the pattern used in pipeline processing, producer-consumer systems, and service architectures.

4.15.2 False Sharing

A subtle performance problem arises when multiple threads access different variables that happen to reside on the same cache line. This is called **false sharing**: the hardware cache coherence protocol treats the entire cache line as a single unit of sharing, forcing unnecessary invalidations and transfers between cores.

Definition 4.13 (False Sharing). False sharing occurs when two or more threads on different cores modify independent variables that share the same cache line (typically 64 bytes on x86-64). Each modification invalidates the cache line on all other cores, even though no actual data sharing occurs. The result is a dramatic slowdown due to cache coherence traffic — often orders of magnitude worse than expected.

```
#include <pthread.h>
#include <stdio.h>
#include <time.h>
```

```c
#define ITERATIONS 100000000
#define NUM_THREADS 4

/* BAD: counters share cache lines (false sharing) */
struct BadCounters {
    long c0;
    long c1;
    long c2;
    long c3;
} bad;

/* GOOD: each counter on its own cache line */
struct GoodCounters {
    long c0; char pad0[56];
    long c1; char pad1[56];
    long c2; char pad2[56];
    long c3; char pad3[56];
} good;

void *increment_bad(void *arg) {
    int id = *(int *)arg;
    long *counter = &bad.c0 + id;
    for (long i = 0; i < ITERATIONS; i++) {
        (*counter)++;
    }
    return NULL;
}

void *increment_good(void *arg) {
    int id = *(int *)arg;
    long *counters[] = {&good.c0, &good.c1, &good.c2, &good.c3};
    long *counter = counters[id];
    for (long i = 0; i < ITERATIONS; i++) {
        (*counter)++;
    }
    return NULL;
}

int main(void) {
    pthread_t threads[NUM_THREADS];
```

```
    int ids[NUM_THREADS] = {0, 1, 2, 3};
    struct timespec start, end;

    /* Benchmark with false sharing */
    clock_gettime(CLOCK_MONOTONIC, &start);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, increment_bad, &ids[i]);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double bad_time = (end.tv_sec - start.tv_sec) +
                      (end.tv_nsec - start.tv_nsec) / 1e9;

    /* Benchmark without false sharing */
    clock_gettime(CLOCK_MONOTONIC, &start);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, increment_good, &ids[i]);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double good_time = (end.tv_sec - start.tv_sec) +
                       (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("With false sharing:    %.3f sec\n", bad_time);
    printf("Without false sharing: %.3f sec\n", good_time);
    printf("Slowdown factor:       %.1fx\n", bad_time / good_time);

    return 0;
}
```

On a 4-core system, the false sharing version typically runs 3–10x slower than the padded version, despite both versions performing exactly the same amount of work.

4.15.3 Thread Safety Levels

When designing multithreaded code, it is useful to classify functions and data structures by their thread safety guarantees:

Level	Description	Example
Thread-safe (reentrant)	Can be called concurrently from multiple threads without external synchronisation	`strtok_r()`, `rand_r()`, Go's `sync.Map`
Conditionally safe	Safe if different threads access different instances	C++ `std::vector` (different vectors)
Unsafe	Must not be called concurrently without external locking	`strtok()`, `asctime()`, Go's `map`

The POSIX standard marks each function as either thread-safe or not. Functions ending in `_r` (reentrant) are thread-safe versions of traditionally unsafe functions. In Go, the `sync` package provides thread-safe primitives (`Mutex`, `RWMutex`, `Map`, `Pool`), and the race detector (`go run -race`) automatically detects data races at runtime.

4.16 Process and Thread Lifecycle Summary

The following table provides a complete lifecycle comparison:

Event	Process (fork)	Thread (pthread_create)	Goroutine (go func)
Creation	`fork()` + COW pages	`clone(CLONE_VM|..`	Runtime allocation (~0.3 us)
Address space	New (COW copy)	Shared	Shared
Stack	Copied (COW)	New allocation (default 8 MB)	New allocation (initial 2-8 KB)
File descriptors	Copied	Shared	Shared (via runtime)
Scheduling	Kernel scheduler	Kernel scheduler	Go runtime scheduler
Synchronisation	IPC mechanisms (pipes, shm)	Mutexes, condition variables	Channels, mutexes

Event	Process (fork)	Thread (pthread_create)	Goroutine (go func)
Termination	`exit()`, `_exit()`	`pthread_exit()`, return	Function return
Cleanup	`wait()` by parent	`pthread_join()` or detach	Automatic (GC)

Exercises

1. **Exercise 4.1.** Write a C program that creates a child process using `fork()`. The parent process should store the value 100 in a variable `x` before forking. The child should modify `x` to 200 and print it. The parent should then print its own value of `x`. Explain why the parent and child see different values, referencing copy-on-write semantics. What physical memory operations occur if the child modifies `x` on a system using COW?

2. **Exercise 4.2.** Using the five-state process model, trace the complete sequence of state transitions for a process that: (a) is created, (b) runs for a time quantum, (c) is preempted, (d) runs again, (e) issues a disk read, (f) is woken up when the read completes, and (g) terminates normally. Draw the state diagram with labelled transitions and identify which transitions are triggered by hardware interrupts versus software actions.

3. **Exercise 4.3.** Write a C program using pthreads that creates n threads (where n is a command-line argument). Each thread should compute the sum $\sum_{i=\text{start}}^{\text{end}} i$ for its assigned range, where the full range $[1, 10^8]$ is divided equally among threads. The main thread should collect all partial sums and compute the total. Measure the wall-clock time for $n = 1, 2, 4, 8$. Plot the speedup relative to $n = 1$ and explain any deviation from linear speedup.

4. **Exercise 4.4.** Explain why Go does not expose `fork()` in its standard library. Describe a specific deadlock scenario that can occur when `fork()` is called in a multi-threaded program where one thread holds a mutex at the time of the fork. What happens to the mutex in the child process, and why can the child never acquire it?

5. **Exercise 4.5.** Compare the memory overhead of running 10,000 concurrent tasks using: (a) processes created with `fork()`, (b) POSIX threads with default stack size, and (c) Go goroutines. For each model, compute the total memory consumed by stacks alone. Then compute the total memory including kernel

data structures (assume 6 KB per `task_struct` for processes/threads, and 360 bytes per goroutine struct). Which model is feasible on a system with 8 GB of RAM?

6. **Exercise 4.6.** Write a Go program that launches 100,000 goroutines, each of which increments a shared counter using `sync/atomic.AddInt64()`. Measure the total time. Then rewrite the program using channels: each goroutine sends its increment to a single aggregator goroutine via a channel. Compare the performance of the two approaches and explain the difference in terms of the GMP scheduler's behaviour and cache contention.

7. **Exercise 4.7.** The Linux `clone()` system call creates both processes and threads depending on its flags. Write a C program that uses `clone()` with `CLONE_VM | CLONE_FS | CLONE_FILES` (but without `CLONE_THREAD`) to create a child that shares the parent's address space but has a different PID. Verify that the child can modify a variable visible to the parent. What happens if you add `CLONE_NEWPID` to the flags? What PID does the child see for itself, and what PID does the parent see for the child?

Chapter 5: CPU Scheduling

The CPU is the most contested resource in any computer system. Dozens or hundreds of processes compete for time on a handful of processor cores, and the operating system must decide, moment by moment, which process gets to run next. This decision — **CPU scheduling** — has profound implications for system responsiveness, throughput, and fairness. A poorly chosen scheduling algorithm can make a powerful machine feel sluggish, while a well-designed one can extract remarkable performance from modest hardware.

This chapter develops the theory of CPU scheduling from first principles. We begin with the criteria by which scheduling algorithms are judged, proceed through the classical algorithms (FCFS, SJF, Round-Robin), and arrive at the sophisticated multi-level feedback queues and fair schedulers used in modern operating systems. We conclude with real-time scheduling theory and multiprocessor considerations.

5.1 Scheduling Criteria

Before we can compare scheduling algorithms, we need to define what "good" scheduling means. Different workloads and different users care about different metrics.

Definition 5.1 (Scheduling Criteria). The primary metrics for evaluating CPU scheduling algorithms are:

1. **CPU utilisation**: The fraction of time the CPU is busy executing processes (not idle). Target: 40–90% on a well-loaded system.
2. **Throughput**: The number of processes that complete per unit time.
3. **Turnaround time**: The total elapsed time from process submission to completion: $T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$.
4. **Waiting time**: The total time a process spends in the ready queue, not executing: $T_{\text{waiting}} = T_{\text{turnaround}} - T_{\text{burst}}$.
5. **Response time**: The time from process submission to the first response (first time the process is scheduled): $T_{\text{response}} = T_{\text{first_run}} - T_{\text{arrival}}$.

These criteria often conflict. Maximising throughput may increase average response time (batching favours throughput but hurts interactivity). Minimising response time for interactive processes may reduce throughput (frequent context switches waste

CPU cycles on overhead). The art of scheduling lies in balancing these trade-offs for the target workload.

5.1.1 CPU Bursts and I/O Bursts

Process execution alternates between **CPU bursts** (periods of computation) and **I/O bursts** (periods of waiting for I/O operations to complete). The distribution of CPU burst lengths is a key input to scheduling algorithm design.

Empirical measurements show that CPU burst lengths follow an approximately exponential distribution: most bursts are short (a few milliseconds), with a long tail of occasional long bursts. This observation motivates algorithms like Shortest Job First that favour short bursts.

Definition 5.2 (CPU-Bound and I/O-Bound Processes). A **CPU-bound process** spends most of its time executing instructions, with long CPU bursts and infrequent I/O. A **I/O-bound process** spends most of its time waiting for I/O, with short CPU bursts and frequent I/O requests. Most interactive programs (editors, browsers, shells) are I/O-bound; most scientific computations and compilers are CPU-bound.

5.2 Preemptive vs Non-Preemptive Scheduling

A fundamental distinction among scheduling algorithms is whether the currently running process can be involuntarily removed from the CPU.

Definition 5.3 (Preemptive and Non-Preemptive Scheduling).

- **Non-preemptive (cooperative) scheduling**: Once a process is allocated the CPU, it retains the CPU until it voluntarily relinquishes it (by terminating, blocking on I/O, or explicitly yielding).
- **Preemptive scheduling**: The scheduler can forcibly remove a running process from the CPU, typically when a timer interrupt fires or when a higher-priority process becomes ready.

Non-preemptive scheduling is simpler to implement and avoids some concurrency issues (a process in a critical section cannot be preempted), but it cannot guarantee responsiveness: a CPU-bound process can monopolise the CPU indefinitely. All modern general-purpose operating systems use preemptive scheduling.

Scheduling decisions are made at four points:

1. A running process switches from running to waiting (e.g., I/O request)
2. A running process switches from running to ready (e.g., timer interrupt)
3. A process switches from waiting to ready (e.g., I/O completion)

4. A process terminates

Non-preemptive scheduling makes decisions only at points 1 and 4. Preemptive scheduling can act at all four points.

5.2.1 The Dispatcher

The **dispatcher** is the kernel component that actually performs the context switch selected by the scheduler. It is distinct from the scheduler (which makes the decision) but closely coupled to it.

Definition 5.4 (Dispatcher). The dispatcher is the kernel module that gives control of the CPU to the process selected by the scheduler. Its responsibilities include:

1. Switching context (saving the current process's registers, loading the next process's registers)
2. Switching to user mode (from kernel mode, via `iretq` or `sysret` on x86-64)
3. Jumping to the proper location in the user program to resume execution

The **dispatch latency** is the time it takes the dispatcher to stop one process and start another. It includes the time for context switching and TLB invalidation, typically 1–10 microseconds on modern hardware.

Example 5.1 (Dispatch Latency Measurement). On a Linux system, dispatch latency can be measured using `cyclictest` from the `rt-tests` package:

```
$ sudo cyclictest --mlockall --priority=80 --interval=1000 --loops=10000
T: 0 ( 1234) P:80 I:1000 C:  10000 Min:      1 Act:    3 Avg:    2 Max:   15
```

This shows minimum dispatch latency of 1 microsecond, average of 2 microseconds, and worst-case of 15 microseconds. The worst case occurs when the dispatcher must flush the TLB and the new process's working set is not in cache.

5.3 First-Come, First-Served (FCFS)

The simplest scheduling algorithm: processes are served in the order they arrive in the ready queue.

Definition 5.5 (FCFS). First-Come, First-Served (FCFS) scheduling dispatches processes in arrival order. The ready queue is a FIFO queue. FCFS is non-preemptive: once a process begins executing, it runs until it completes its CPU burst or blocks on I/O.

Example 5.2 (FCFS Scheduling). Consider three processes arriving at time 0 with the following CPU burst lengths:

Process	Arrival Time	Burst Time
P_1	0	24
P_2	0	3
P_3	0	3

Execution order P_1, P_2, P_3 (arrival order):

```
|---- P1 (24) ----|-- P2 (3) --|-- P3 (3) --|
0                 24           27           30
```

- Waiting times: $P_1 = 0$, $P_2 = 24$, $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17.0$

If instead the order were P_2, P_3, P_1:

```
|P2(3)|P3(3)|---- P1 (24) ----|
0     3     6                 30
```

- Waiting times: $P_2 = 0$, $P_3 = 3$, $P_1 = 6$
- Average waiting time: $(0 + 3 + 6)/3 = 3.0$

This example illustrates the **convoy effect**: a long CPU-bound process arriving before short I/O-bound processes forces all short processes to wait, dramatically inflating average waiting time. FCFS is simple but poorly suited to mixed workloads.

Definition 5.6 (Convoy Effect). The convoy effect occurs in FCFS scheduling when a single long CPU-bound process monopolises the CPU, causing all shorter processes to queue behind it. The average waiting time becomes dominated by the long process's burst time, regardless of how short the other processes are. Formally, if one process has burst time B and $n - 1$ processes have burst time $\epsilon \ll B$, the average waiting time under FCFS (with the long process first) is approximately $\frac{(n-1)\cdot B}{n} \approx B$ for large n.

5.3.1 FCFS with Staggered Arrivals

When processes arrive at different times, FCFS becomes more reasonable because the convoy effect is mitigated by natural interleaving:

Example 5.3 (FCFS with Staggered Arrivals). Consider:

Process	Arrival Time	Burst Time
P_1	0	5
P_2	2	3
P_3	4	1
P_4	5	4

```
|-- P1 (5) --|-- P2 (3) --|P3(1)|-- P4 (4) --|
0            5            8     9            13
```

- Waiting times: $P_1 = 0$, $P_2 = 5 - 2 = 3$, $P_3 = 8 - 4 = 4$, $P_4 = 9 - 5 = 4$
- Average waiting time: $(0 + 3 + 4 + 4)/4 = 2.75$
- Average turnaround time: $(5 + 6 + 5 + 8)/4 = 6.0$

5.3.2 Implementation

FCFS is trivially implemented with a FIFO queue:

```c
#include <stdio.h>

typedef struct {
    int pid;
    int arrival;
    int burst;
    int start;
    int finish;
    int waiting;
    int turnaround;
} Process;

void fcfs_schedule(Process procs[], int n) {
    int current_time = 0;

    /* Assume processes are sorted by arrival time */
    for (int i = 0; i < n; i++) {
        if (current_time < procs[i].arrival) {
            current_time = procs[i].arrival;  /* CPU idle */
        }
```

```
        procs[i].start = current_time;
        procs[i].finish = current_time + procs[i].burst;
        procs[i].waiting = procs[i].start - procs[i].arrival;
        procs[i].turnaround = procs[i].finish - procs[i].arrival;

        current_time = procs[i].finish;

        printf("P%d: start=%d, finish=%d, waiting=%d, turnaround=%d\n",
               procs[i].pid, procs[i].start, procs[i].finish,
               procs[i].waiting, procs[i].turnaround);
    }
}

int main(void) {
    Process procs[] = {
        {1, 0, 5, 0, 0, 0, 0},
        {2, 2, 3, 0, 0, 0, 0},
        {3, 4, 1, 0, 0, 0, 0},
        {4, 5, 4, 0, 0, 0, 0}
    };

    fcfs_schedule(procs, 4);
    return 0;
}
```

5.4 Shortest Job First (SJF)

Shortest Job First scheduling always selects the process with the smallest next CPU burst.

Definition 5.7 (Shortest Job First). Shortest Job First (SJF) scheduling selects the ready process whose next CPU burst is the shortest. When two processes have equal burst lengths, FCFS order is used as a tiebreaker. Non-preemptive SJF runs the selected process to completion of its burst.

Theorem 5.1 (Optimality of SJF). Among all non-preemptive scheduling algorithms for a set of processes with known burst times that arrive simultaneously, SJF minimises the average waiting time.

Proof. Let n processes have burst times $b_1, b_2, \dots, b_n$. Under any schedule σ (a permutation of $\{1, \dots, n\}$), the waiting time of the k-th process in the schedule is:

$$W_{\sigma(k)} = \sum_{j=1}^{k-1} b_{\sigma(j)}$$

The total waiting time is:

$$W_{\text{total}} = \sum_{k=1}^{n} W_{\sigma(k)} = \sum_{k=1}^{n} \sum_{j=1}^{k-1} b_{\sigma(j)} = \sum_{k=1}^{n} (n-k) \cdot b_{\sigma(k)}$$

The last equality follows because process $\sigma(k)$ contributes its burst time to the waiting time of all $n-k$ processes that follow it in the schedule. To minimise this weighted sum $\sum_{k=1}^{n}(n-k) \cdot b_{\sigma(k)}$, we must assign the largest weights $(n-1, n-2, \dots, 0)$ to the smallest burst times. This is achieved by sorting burst times in non-decreasing order: $b_{\sigma(1)} \leq b_{\sigma(2)} \leq \cdots \leq b_{\sigma(n)}$. This is precisely the SJF order. □

Example 5.4 (SJF Scheduling). Consider:

Process	Arrival Time	Burst Time
P_1	0.0	6
P_2	0.0	8
P_3	0.0	7
P_4	0.0	3

SJF order: P_4, P_1, P_3, P_2:

```
|P4(3)|-- P1 (6) --|-- P3 (7) --|--- P2 (8) ---|
0     3            9            16             24
```

- Waiting times: $P_4 = 0$, $P_1 = 3$, $P_3 = 9$, $P_2 = 16$
- Average waiting time: $(0+3+9+16)/4 = 7.0$

Compare with FCFS order P_1, P_2, P_3, P_4: average waiting time = $(0+6+14+21)/4 = 10.25$.

5.4.1 The Prediction Problem

SJF is optimal but impractical in its pure form: the scheduler cannot know the length of the next CPU burst in advance. The standard approximation uses **exponential averaging** to predict the next burst from past behaviour:

Definition 5.8 (Exponential Average Prediction). Let τ_{n+1} be the predicted length of the next CPU burst, t_n be the measured length of the n-th burst, and $\alpha \in [0, 1]$ be a smoothing parameter. The prediction is:

$$\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n$$

Expanding the recurrence:

$$\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \alpha \cdot t_{n-1} + (1 - \alpha)^2 \cdot \alpha \cdot t_{n-2} + \cdots + (1 - \alpha)^n \cdot \tau_0$$

Each past burst t_k is weighted by $\alpha(1 - \alpha)^{n-k}$, giving exponentially decreasing influence to older observations.

Typical values of α range from 0.5 (equal weight to recent and predicted) to 0.8 (strong bias toward recent measurements). With $\alpha = 0$, the prediction never changes; with $\alpha = 1$, the prediction equals the last observed burst.

Example 5.5 (Exponential Average Trace). A process exhibits the following CPU burst lengths: $t_1 = 6$, $t_2 = 4$, $t_3 = 6$, $t_4 = 4$, $t_5 = 13$, $t_6 = 13$, $t_7 = 13$. With $\alpha = 0.5$ and $\tau_0 = 10$:

n	**Actual** t_n	**Predicted** τ_n	**Error** $\lvert t_n - \tau_n \rvert$
1	6	10.0	4.0
2	4	8.0	4.0
3	6	6.0	0.0
4	4	6.0	2.0
5	13	5.0	8.0
6	13	9.0	4.0
7	13	11.0	2.0

The predictor adapts to the process's behaviour, though it lags behind sudden changes (the jump from burst lengths of 4–6 to 13 takes several bursts to track).

5.4.2 Starvation in SJF

A major drawback of SJF is **starvation**: a process with a long burst can wait indefinitely if shorter processes keep arriving. In a system where short jobs arrive fre-

quently, a long job may never be scheduled.

Definition 5.9 (Starvation). Starvation occurs when a process waits indefinitely in the ready queue because other processes are always selected by the scheduler. Starvation is a liveness failure: the starved process makes no progress despite being ready to run.

The standard remedy for starvation is **aging**: gradually increasing the priority of processes that have been waiting in the ready queue. After sufficient waiting time, even a long-burst process will be promoted to the highest priority and scheduled. Formally, if a process has waited w time units, its effective priority can be defined as:

$$p_{\text{effective}} = p_{\text{base}} - \lfloor w/\Delta \rfloor$$

where Δ is the aging interval. This ensures that any waiting process eventually reaches the highest priority.

5.5 Shortest Remaining Time First (SRTF)

The preemptive version of SJF is called Shortest Remaining Time First (SRTF), also known as Preemptive SJF.

Definition 5.10 (Shortest Remaining Time First). SRTF scheduling always runs the process whose remaining CPU burst time is the shortest. When a new process arrives with a burst time shorter than the remaining time of the currently running process, the running process is preempted and the new process takes the CPU.

Example 5.6 (SRTF Scheduling). Consider:

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Execution trace:

```
|P1|--- P2 (4) ---|-- P4 (5) --|---- P1 (7) ----|------ P3 (9) ------|
0  1              5            10               17                   26
```

- At $t = 0$: Only P_1 available, runs.
- At $t = 1$: P_2 arrives with burst 4; P_1 has remaining 7. Since $4 < 7$, preempt P_1, run P_2.
- At $t = 2$: P_3 arrives with burst 9; P_2 has remaining 3. Since $9 > 3$, P_2 continues.
- At $t = 3$: P_4 arrives with burst 5; P_2 has remaining 2. Since $5 > 2$, P_2 continues.
- At $t = 5$: P_2 completes. Ready: P_1 (remaining 7), P_3 (9), P_4 (5). Run P_4.
- At $t = 10$: P_4 completes. Ready: P_1 (7), P_3 (9). Run P_1.
- At $t = 17$: P_1 completes. Run P_3.
- At $t = 26$: P_3 completes.

Waiting times: $P_1 = (0-0)+(10-1) = 9$, $P_2 = 0$, $P_3 = 17-2 = 15$, $P_4 = 5-3 = 2$.

Average waiting time: $(9 + 0 + 15 + 2)/4 = 6.5$.

Theorem 5.2 (Optimality of SRTF). Among all scheduling algorithms (preemptive and non-preemptive) for a set of processes with known burst times, SRTF minimises the average waiting time.

Proof sketch. Consider any schedule S that is not SRTF. There exists a time t at which S runs process P_i while process P_j is ready with a shorter remaining time. We can construct a new schedule S' that swaps an infinitesimal interval ϵ of execution between P_i and P_j at time t. In S', P_j's waiting time decreases by ϵ and P_i's increases by ϵ. Since P_j's remaining time is shorter, it finishes sooner, reducing the waiting time of all processes waiting behind it. By induction over all such swap points, we transform any non-SRTF schedule into the SRTF schedule without increasing total waiting time. □

5.6 Round-Robin (RR) Scheduling

Round-Robin scheduling is designed for time-sharing systems. Each process gets a small unit of CPU time, called a **time quantum** (or time slice), after which it is preempted and placed at the back of the ready queue.

Definition 5.11 (Round-Robin Scheduling). Round-Robin (RR) scheduling maintains a FIFO ready queue and allocates each process a fixed time quantum q. If a process's CPU burst exceeds q, it is preempted after q time units and placed at the tail of the ready queue. If the burst is less than or equal to q, the process runs to completion and releases the CPU voluntarily.

Example 5.7 (Round-Robin with $q = 4$). Consider:

Process	Arrival Time	Burst Time
P_1	0	24
P_2	0	3
P_3	0	3

With quantum $q = 4$:

```
|P1(4)|P2(3)|P3(3)|P1(4)|P1(4)|P1(4)|P1(4)|P1(4)|
0     4     7    10    14    18    22    26    30
```

- P_1 gets 4 units, preempted, goes to back of queue
- P_2 gets 3 units, completes (burst $\leq q$)
- P_3 gets 3 units, completes
- P_1 runs remaining 20 units in 5 quanta of 4 each

Waiting times: $P_1 = (0) + (10 - 4) = 6$, $P_2 = 4$, $P_3 = 7$.

Average waiting time: $(6 + 4 + 7)/3 = 5.67$.

5.6.1 Quantum Analysis

The choice of quantum q is critical:

- **If q is very large** (larger than all burst times), RR degenerates to FCFS. No preemption ever occurs.
- **If q is very small**, the system spends most of its time context-switching rather than executing useful work.

Theorem 5.3 (Context Switch Overhead in Round-Robin). Let q be the time quantum, T_{cs} be the context switch time, and n be the number of ready processes. The fraction of CPU time spent on context switching (CPU overhead) is:

$$\text{Overhead} = \frac{T_{\text{cs}}}{q + T_{\text{cs}}}$$

For the system to spend at least fraction f of CPU time on useful work, we need:

$$q \geq \frac{T_{\text{cs}} \cdot f}{1 - f}$$

For example, with $T_{\text{cs}} = 5\ \mu\text{s}$ and $f = 0.99$ (99% useful work):

$$q \geq \frac{5 \times 0.99}{0.01} = 495\ \mu\text{s} \approx 0.5\ \text{ms}$$

In practice, typical quantum values are:

- **Linux CFS**: Dynamic, based on the number of runnable tasks and their niceness (typically 1–20 ms)
- **Windows**: 20 ms for desktop (foreground bias), 120 ms for server
- **macOS**: Varies by scheduling band; typically 10 ms for interactive, longer for background

5.6.2 Turnaround Time Analysis

Round-Robin does not optimise turnaround time — in fact, it can be worse than FCFS for homogeneous workloads:

Example 5.8 (RR Turnaround Time Penalty). Three processes, all arriving at time 0, each with burst time 10. With quantum $q = 1$:

Under FCFS: Turnaround times are 10, 20, 30. Average = 20.

Under RR ($q = 1$): Each process finishes at time 28, 29, 30 respectively (they interleave in round-robin fashion, each getting 1 unit every 3 units).

- P_1 finishes at $t = 28$ (last unit at $t = 28$)
- P_2 finishes at $t = 29$
- P_3 finishes at $t = 30$
- Average turnaround = $(28 + 29 + 30)/3 = 29.0$

The average turnaround under RR is significantly worse than FCFS (29 vs 20). This is the fundamental trade-off: RR improves response time at the cost of turnaround time. RR is designed for interactive systems where response time matters more than total completion time.

Theorem 5.4 (RR Turnaround for Equal Bursts). For n processes each with burst time B and quantum q, the average turnaround time under Round-Robin is:

$$\bar{T}^{\text{RR}}_{\text{turnaround}} = B + \frac{(n-1)(B-q)}{2} \cdot \frac{q}{B}$$

when q divides B evenly. As $q \to 0$, the turnaround time for all processes approaches $n \cdot B$ (each process finishes near the end because all are interleaved), compared to $\frac{(n+1) \cdot B}{2}$ under FCFS.

5.6.3 Implementation

```
#include <stdio.h>
#include <stdbool.h>

typedef struct {
    int pid;
    int arrival;
    int burst;
    int remaining;
    int finish;
    int waiting;
    int turnaround;
    int first_run;
    bool started;
} Process;

void rr_schedule(Process procs[], int n, int quantum) {
    int time = 0;
    int completed = 0;
    int queue[1000];
    int front = 0, rear = 0;
    bool in_queue[100] = {false};

    /* Add initially arrived processes */
    for (int i = 0; i < n; i++) {
        procs[i].remaining = procs[i].burst;
        procs[i].started = false;
        if (procs[i].arrival <= time) {
            queue[rear++] = i;
            in_queue[i] = true;
        }
    }

    while (completed < n) {
        if (front == rear) {
            /* Queue empty: advance time to next arrival */
            time++;
            for (int i = 0; i < n; i++) {
                if (!in_queue[i] && procs[i].arrival <= time &&
```

```
                procs[i].remaining > 0) {
                queue[rear++] = i;
                in_queue[i] = true;
            }
        }
        continue;
    }

    int idx = queue[front++];

    if (!procs[idx].started) {
        procs[idx].first_run = time;
        procs[idx].started = true;
    }

    int run_time = procs[idx].remaining < quantum ?
                   procs[idx].remaining : quantum;

    time += run_time;
    procs[idx].remaining -= run_time;

    /* Check for new arrivals during this quantum */
    for (int i = 0; i < n; i++) {
        if (!in_queue[i] && procs[i].arrival <= time &&
            procs[i].remaining > 0) {
            queue[rear++] = i;
            in_queue[i] = true;
        }
    }

    if (procs[idx].remaining > 0) {
        queue[rear++] = idx;  /* Re-enqueue */
    } else {
        procs[idx].finish = time;
        procs[idx].turnaround = time - procs[idx].arrival;
        procs[idx].waiting = procs[idx].turnaround - procs[idx].burst;
        completed++;
    }
}
```

```
    float avg_wait = 0, avg_turnaround = 0, avg_response = 0;
    for (int i = 0; i < n; i++) {
        avg_wait += procs[i].waiting;
        avg_turnaround += procs[i].turnaround;
        avg_response += (procs[i].first_run - procs[i].arrival);
        printf("P%d: finish=%d, wait=%d, turnaround=%d, response=%d\n",
               procs[i].pid, procs[i].finish, procs[i].waiting,
               procs[i].turnaround, procs[i].first_run - procs[i].arrival);
    }
    printf("Averages: wait=%.2f, turnaround=%.2f, response=%.2f\n",
           avg_wait / n, avg_turnaround / n, avg_response / n);
}
```

Theorem 5.5 (Response Time Bound for Round-Robin). In a Round-Robin schedule with n processes and quantum q, the worst-case response time for any process is:

$$T_{\text{response}}^{\max} = (n-1) \cdot q$$

This occurs when a process arrives just after its slot and must wait for all other $n-1$ processes to receive their quanta before being scheduled.

5.7 Priority Scheduling

In priority scheduling, each process is assigned a priority, and the scheduler always runs the highest-priority ready process. Priorities may be assigned externally (by the user or administrator) or computed internally (based on resource usage, aging, or other factors).

Definition 5.12 (Priority Scheduling). In priority scheduling, each process P_i is assigned an integer priority p_i. The scheduler selects the ready process with the highest priority (lowest numerical value in some conventions, highest in others). Priority scheduling can be either preemptive (a newly arrived higher-priority process preempts the running process) or non-preemptive (the running process completes its burst before a priority comparison is made).

5.7.1 Priority Inversion

Priority scheduling introduces a subtle problem known as **priority inversion**: a high-priority process can be blocked indefinitely by a low-priority process.

Definition 5.13 (Priority Inversion). Priority inversion occurs when a high-priority process H is blocked waiting for a resource held by a low-priority process L, while a medium-priority process M preempts L. Because M runs instead of L, the resource held by L is not released, and H remains blocked. The effective priority ordering becomes $M > H > L$, inverting the intended priority of H and M.

Example 5.9 (The Mars Pathfinder Bug). In 1997, NASA's Mars Pathfinder experienced repeated system resets caused by priority inversion. The scenario involved three tasks:

1. **Bus management task** (low priority): Held a mutex on a shared data bus
2. **Communication task** (medium priority): Ran frequently for data collection
3. **Information bus task** (high priority): Needed the bus mutex

When the low-priority task held the mutex and the medium-priority task preempted it, the high-priority task starved. A watchdog timer detected the stall and reset the system. The fix was to enable **priority inheritance** on the mutex, which the VxWorks RTOS supported but had left disabled.

5.7.2 Priority Inheritance and Priority Ceiling

Two protocols solve priority inversion:

Definition 5.14 (Priority Inheritance Protocol). When a low-priority process L holds a resource needed by a higher-priority process H, L temporarily inherits H's priority for the duration of the critical section. Once L releases the resource, its priority reverts to its original value. This prevents medium-priority processes from preempting L while it holds the resource.

Definition 5.15 (Priority Ceiling Protocol). Each resource R is assigned a **priority ceiling** $\Pi(R)$ equal to the highest priority of any process that may lock R. A process can acquire R only if its priority is strictly higher than the priority ceiling of all resources currently locked by other processes. This prevents deadlocks and bounds the duration of priority inversion.

Theorem 5.6 (Priority Inversion Bound). Under the Priority Ceiling Protocol, a high-priority process can be blocked by at most one lower-priority critical section. Specifically, if process P_i has the i-th highest priority, then P_i can be blocked for at most $\max_{j>i}\{C_j^k\}$ time units, where C_j^k is the worst-case execution time of the longest critical section of any lower-priority process P_j that accesses a resource shared with P_i or a higher-priority process.

5.8 Lottery and Stride Scheduling

Before examining MLFQ and CFS, we consider two elegant proportional-share schedulers that provide provable fairness guarantees.

5.8.1 Lottery Scheduling

Definition 5.16 (Lottery Scheduling). In lottery scheduling, each process holds some number of **lottery tickets**. At each scheduling decision, the scheduler draws a random ticket uniformly from the total pool. The process holding the winning ticket gets to run for one time quantum. A process with t_i tickets out of a total $T = \sum_j t_j$ tickets has a probability t_i/T of winning each lottery, and thus receives approximately t_i/T of the CPU over time.

Lottery scheduling has several appealing properties:

- **Proportional share**: CPU time is distributed in proportion to ticket holdings
- **Responsive**: A new process that arrives with tickets immediately participates in the next lottery
- **Simple**: Implementation requires only a random number generator and a list of ticket holders
- **Composable**: Ticket transfers allow processes to temporarily donate their CPU share to other processes (useful for priority donation)

Example 5.10 (Lottery Scheduling). Process A holds 75 tickets, process B holds 25 tickets. Total: 100 tickets.

At each scheduling event, a random number in $[0, 99]$ is drawn. If the number falls in $[0, 74]$, A runs; if it falls in $[75, 99]$, B runs.

Over 1000 quanta, the expected number of quanta for A is 750, and for B is 250. The standard deviation is:

$$\sigma = \sqrt{n \cdot p \cdot (1-p)} = \sqrt{1000 \times 0.75 \times 0.25} \approx 13.7$$

So A will receive 750 ± 27 quanta (within 2 standard deviations) with 95% probability. The fairness improves with the number of scheduling events.

Theorem 5.7 (Lottery Fairness Convergence). Let process P_i hold fraction $f_i = t_i/T$ of the total tickets. After n scheduling events, the fraction of CPU time received by P_i converges to f_i with standard deviation $O(1/\sqrt{n})$. Specifically, by the Central Limit Theorem:

$$\Pr\left[\left|\frac{X_i}{n} - f_i\right| > \epsilon\right] \leq 2\exp\left(-\frac{2n\epsilon^2}{1}\right)$$

where X_i is the number of quanta received by P_i (Hoeffding's inequality). For $\epsilon = 0.01$ and $n = 10000$, the probability of more than 1% deviation is less than $2e^{-200} \approx 0$.

5.8.2 Stride Scheduling

Lottery scheduling's randomness means it can be unfair over short timescales. **Stride scheduling** is the deterministic counterpart.

Definition 5.17 (Stride Scheduling). In stride scheduling, each process P_i with ticket count t_i is assigned a **stride** $s_i = L/t_i$, where L is a large constant (e.g., 10000). Each process maintains a **pass value**, initially 0. At each scheduling event:

1. Select the process with the smallest pass value
2. Run that process for one quantum
3. Increment that process's pass value by its stride: $\text{pass}_i \mathrel{+}= s_i$

Processes with more tickets have smaller strides and thus accumulate pass value more slowly, running more frequently.

Example 5.11 (Stride Scheduling Trace). Three processes with strides $A : 100$, $B : 200$, $C : 40$:

Step	Pass A	Pass B	Pass C	Run
0	0	0	0	A (tiebreak)
1	100	0	0	B (tiebreak)
2	100	200	0	C
3	100	200	40	C
4	100	200	80	C
5	100	200	120	A
6	200	200	120	C
7	200	200	160	C
8	200	200	200	A (tiebreak)

After 9 steps: A ran 3 times (33%), B ran 1 time (11%), C ran 5 times (56%). With

$L = 200$, tickets are $A : 2$, $B : 1$, $C : 5$, total 8. Expected: $A : 25\%$, $B : 12.5\%$, $C : 62.5\%$. The proportions converge as more steps are executed.

Stride scheduling provides **deterministic proportional-share scheduling**: unlike lottery scheduling, it never makes random choices, so fairness over short timescales is guaranteed. CFS uses a similar idea with its virtual runtime mechanism.

5.9 Multi-Level Feedback Queue (MLFQ)

The Multi-Level Feedback Queue is one of the most important scheduling algorithms in practice. It aims to optimise both turnaround time (by favouring short jobs) and response time (by rapidly serving interactive processes), without requiring prior knowledge of burst lengths.

Definition 5.18 (Multi-Level Feedback Queue). An MLFQ scheduler maintains multiple priority queues $Q_0, Q_1, \dots, Q_{k-1}$, where Q_0 has the highest priority and Q_{k-1} the lowest. Each queue may use a different scheduling algorithm (typically Round-Robin with increasing quanta). Processes move between queues based on their observed behaviour:

1. **Rule 1**: If Priority(A) > Priority(B), A runs (and B does not).
2. **Rule 2**: If Priority(A) = Priority(B), A and B run in Round-Robin within their queue.
3. **Rule 3**: When a process enters the system (or after a priority boost), it is placed in Q_0 (highest priority).
4. **Rule 4a**: If a process uses up its time quantum in queue Q_i, it is demoted to Q_{i+1}.
5. **Rule 4b**: If a process gives up the CPU before its quantum expires (e.g., for I/O), it stays in Q_i.

The intuition is that MLFQ **learns** the nature of each process. Interactive processes (short CPU bursts, frequent I/O) stay in high-priority queues. CPU-bound processes (long bursts) are gradually demoted to lower-priority queues with longer quanta.

5.8.1 Problems with Basic MLFQ

The basic rules above have several problems:

Starvation. If many interactive processes keep arriving, CPU-bound processes in low-priority queues may never run.

Gaming. A process can game the scheduler by issuing a spurious I/O request just before its quantum expires, staying in a high-priority queue while consuming nearly

as much CPU as a CPU-bound process.

Behaviour change. A CPU-bound process that becomes interactive (e.g., a compiler that finishes compiling and starts accepting user input) is stuck in a low-priority queue.

5.8.2 MLFQ Refinements

Modern MLFQ implementations address these problems:

Definition 5.19 (MLFQ with Anti-Gaming and Boosting).

- **Priority boost** (Rule 5): After a fixed time period S (the boost interval), all processes are moved to Q_0. This prevents starvation and allows CPU-bound processes that become interactive to be reclassified.
- **Accounting-based demotion** (Rule 4, revised): Instead of demoting a process when it uses a single quantum, track the **total CPU time** a process has used at its current priority level. Once it exceeds the allotment for that level, demote it, regardless of how many quanta it has used. This prevents gaming by breaking CPU time into many small intervals separated by I/O.

Example 5.12 (MLFQ with Three Queues). Consider an MLFQ with three queues:

Queue	Priority	Quantum	Algorithm
Q_0	Highest	8 ms	Round-Robin
Q_1	Medium	16 ms	Round-Robin
Q_2	Lowest	∞	FCFS

Process A (interactive): Arrives, placed in Q_0. Uses 3 ms CPU, then does I/O. Returns to Q_0. This pattern repeats — A always stays in Q_0, getting excellent response time.

Process B (CPU-bound): Arrives, placed in Q_0. Uses full 8 ms quantum, demoted to Q_1. Uses full 16 ms quantum, demoted to Q_2. Runs in FCFS order with other CPU-bound processes.

After the boost interval S, both A and B are moved back to Q_0, and the classification process begins again.

5.9.3 Detailed MLFQ Trace

Example 5.12a (Complete MLFQ Trace). Consider an MLFQ with three queues: Q_0 (quantum 2 ms), Q_1 (quantum 4 ms), Q_2 (FCFS). Boost interval $S = 20$ ms. Two

processes arrive at $t = 0$:

- P_A: Interactive (pattern: 1 ms CPU, I/O, 1 ms CPU, I/O, ...)
- P_B: CPU-bound (needs 30 ms total CPU)

Trace:

```
t=0:  P_A enters Q_0, P_B enters Q_0
      Schedule P_A from Q_0 (quantum=2)
t=0-1:  P_A runs 1 ms, issues I/O. Stays in Q_0 (didn't use full quantum)
t=1:  Schedule P_B from Q_0 (quantum=2)
t=1-3:  P_B runs 2 ms (full quantum). Demoted to Q_1.
t=3:  P_A returns from I/O, enters Q_0.
      Schedule P_A from Q_0 (higher priority than Q_1)
t=3-4:  P_A runs 1 ms, issues I/O. Stays in Q_0.
t=4:  Schedule P_B from Q_1 (quantum=4)
t=4-8:  P_B runs 4 ms (full quantum). Demoted to Q_2.
t=5:  P_A returns from I/O, enters Q_0.
      P_B is preempted! Schedule P_A from Q_0.
t=5-6:  P_A runs 1 ms, issues I/O.
t=6:  Resume P_B from Q_2 (remaining from preemption).
...
```

The key observation: P_A always stays in Q_0 because it never uses its full quantum. P_B is progressively demoted to Q_2. The interactive process receives excellent response time, while the CPU-bound process still makes progress in the background.

5.9.4 MLFQ in Real Systems

Real operating systems use MLFQ variants:

- **FreeBSD ULE scheduler**: 256 priority levels grouped into three classes (interrupt, real-time, timeshare). Interactive detection uses a "score" based on voluntary context switches vs CPU time.
- **Windows**: Uses a simplified MLFQ with 32 priority levels. Variable-priority threads (1–15) receive dynamic boosts for I/O completion, foreground status, and starvation prevention.
- **Solaris** (historically): Used a table-driven MLFQ with configurable quantum, priority boost, and demotion rules per priority level. The table could be replaced at runtime to tune for different workloads.

5.10 Linux Scheduler History

Before examining CFS, it is instructive to understand the evolution of Linux schedulers, as each generation addressed specific limitations of its predecessor.

5.10.1 The O(n) Scheduler (Linux 2.4)

The original Linux scheduler scanned the entire run queue at each scheduling decision, examining every process to find the one with the highest "goodness" score. With n runnable processes, each scheduling decision took $O(n)$ time. This was acceptable for workstations with a few dozen processes but became a bottleneck on servers with hundreds or thousands of threads.

5.10.2 The O(1) Scheduler (Linux 2.6.0–2.6.22)

Ingo Molnar's O(1) scheduler (2003) maintained two priority arrays per CPU: an **active** array and an **expired** array. Each array had 140 priority levels (0–99 for real-time, 100–139 for normal), with a bitmap indicating which levels had runnable processes. Finding the highest-priority process required only a `__ffs()` (find-first-set-bit) operation — $O(1)$ time.

The O(1) scheduler used heuristics to classify processes as "interactive" or "batch" and boosted the priority of interactive processes. However, these heuristics were fragile: they worked well for some workloads but performed poorly for others. Desktop users reported stuttering audio and video, and the heuristics were notoriously difficult to tune.

5.10.3 The Completely Fair Scheduler (Linux 2.6.23+)

The Linux Completely Fair Scheduler, introduced in kernel 2.6.23 (2007) by Ingo Molnar, replaced the O(1) scheduler. CFS takes a radically different approach: instead of using multiple queues and heuristic rules, it attempts to give each process an exactly fair share of the CPU. The insight behind CFS is simple: in an ideal multitasking CPU, every process would run simultaneously, each receiving $1/n$ of the CPU power (where n is the number of runnable processes). Since this is physically impossible on a real CPU, CFS approximates it by tracking how much CPU time each process "deserves" versus how much it has received, and always running the process that is furthest behind its fair share.

5.10.4 Virtual Runtime

The central concept in CFS is **virtual runtime** (vruntime): the amount of CPU time a process has received, weighted by its priority.

Definition 5.20 (Virtual Runtime). The virtual runtime of a process P is the total CPU time it has consumed, scaled by its weight relative to the default weight:

$$\text{vruntime}(P) = \sum \frac{\delta_t \cdot w_0}{w_P}$$

where δ_t is a physical time interval during which P was running, w_P is P's weight (determined by its nice value), and w_0 is the weight of a process with nice value 0 (the default).

A higher-priority process (lower nice value, higher weight) accumulates vruntime more slowly. A lower-priority process (higher nice value, lower weight) accumulates vruntime more quickly. The scheduler always runs the process with the **smallest vruntime**, ensuring that all processes make progress proportional to their weight.

5.10.5 Nice Values and Weights

Linux nice values range from -20 (highest priority) to $+19$ (lowest priority), with 0 as the default. CFS maps nice values to weights using a carefully chosen table:

Definition 5.21 (Nice-to-Weight Mapping). The weight w of a process with nice value n is approximately:

$$w(n) = \frac{1024}{1.25^n}$$

The ratio of weights between adjacent nice levels is approximately 1.25 (or 10% CPU share difference per nice level). The exact weights used in the Linux kernel are:

Nice	Weight	Nice	Weight
-20	88761	0	1024
-10	9548	5	335
-5	3121	10	110
-1	1277	15	36
0	1024	19	15

Example 5.13 (CFS Fair Share Computation). Two processes A and B compete for a single CPU. Process A has nice value 0 (weight 1024), process B has nice value 5 (weight 335).

Total weight: $W = 1024 + 335 = 1359$.

CPU share of A: $1024/1359 \approx 75.3\%$.

CPU share of B: $335/1359 \approx 24.7\%$.

Over a 100 ms scheduling period, A receives approximately 75.3 ms and B receives approximately 24.7 ms.

Vruntime accumulation per millisecond of physical time:

- A: Δvruntime $= 1 \times (1024/1024) = 1.0$ ms/ms
- B: Δvruntime $= 1 \times (1024/335) \approx 3.06$ ms/ms

So B's vruntime grows 3x faster, meaning B is selected less often, which is exactly the intended behaviour.

5.10.6 The Red-Black Tree

CFS uses a **red-black tree** (a self-balancing binary search tree) to maintain all runnable processes, keyed by vruntime. The process with the smallest vruntime is always the leftmost node.

Definition 5.22 (CFS Red-Black Tree). The CFS run queue is a red-black tree T where each node represents a runnable process, keyed by its vruntime. The tree provides:

- $O(\log n)$ insertion when a process becomes runnable
- $O(\log n)$ removal when a process is selected to run or blocks
- $O(1)$ access to the minimum-vruntime process (the leftmost node is cached)

where n is the number of runnable processes.

The scheduler's decision at each scheduling point is: **run the leftmost node** (minimum vruntime). After running for a time slice, the process's vruntime is updated and the node is reinserted into the tree at its new position. If another process now has a smaller vruntime, a context switch occurs.

5.10.7 Time Slice Computation

CFS does not use a fixed quantum. Instead, the **scheduling period** (also called the target latency) is divided among all runnable processes proportionally to their weights:

$$\text{timeslice}(P) = \text{period} \times \frac{w_P}{\sum_i w_i}$$

The default scheduling period is:

$$\text{period} = \max\left(\text{sched_latency}, n \times \text{sched_min_granularity}\right)$$

where `sched_latency` defaults to 6 ms, `sched_min_granularity` to 0.75 ms, and n is the number of runnable processes. With many runnable processes, the period grows to ensure each process gets at least `sched_min_granularity` of CPU time.

Programmer's Perspective

Programmer's Perspective: Tuning CFS in Linux. CFS parameters are exposed via `/proc/sys/kernel/`:

- *`sched_latency_ns` (default 6,000,000 ns = 6 ms): The target scheduling period. Lower values improve interactivity but increase context switch overhead.*
- *`sched_min_granularity_ns` (default 750,000 ns = 0.75 ms): Minimum time slice per process. Prevents excessive context switching with many runnable processes.*
- *`sched_wakeup_granularity_ns` (default 1,000,000 ns = 1 ms): Minimum vruntime advantage a newly woken process must have to preempt the current process. Higher values reduce "ping-pong" switching between processes.*

The nice value is set per-process with `nice(2)` or `setpriority(2)`. The `renice` command adjusts it at runtime. Only root can set negative nice values (higher priority). In Go, you can set the niceness of the current process via `syscall.Setpriority()`, though this affects the entire OS thread, not a single goroutine.

5.10.8 Sleeper Fairness

A subtle problem arises when a process sleeps (blocks on I/O or waits) and then wakes up. While sleeping, the process's vruntime does not advance. Meanwhile, all running processes accumulate vruntime. When the sleeper wakes up, its vruntime may be far behind the other processes, giving it a temporary CPU monopoly as the scheduler tries to "make up" for the time it was asleep.

CFS handles this by capping the vruntime advantage of newly woken processes. When a process wakes up, its vruntime is set to:

$$\text{vruntime}_{\text{wakeup}} = \max\left(\text{vruntime}_{\text{old}},\ \text{min_vruntime} - \text{sched_latency}\right)$$

where `min_vruntime` is the smallest vruntime among all currently runnable processes. This gives the waker a bounded advantage (at most one scheduling period of catch-up) without allowing it to monopolise the CPU.

5.10.9 Group Scheduling (CFS Bandwidth Control)

CFS supports hierarchical scheduling via **control groups** (cgroups). Processes in the same cgroup share a collective CPU allocation. This prevents scenarios where one user running 100 processes gets 100x the CPU of another user running 1 process.

Definition 5.23 (CFS Group Scheduling). CFS group scheduling organises processes into hierarchical groups via cgroups. Each group has a weight (derived from `cpu.shares`) that determines its share of CPU time relative to sibling groups. Within a group, individual processes are scheduled fairly according to their own weights. The two-level hierarchy ensures fairness both between groups and within groups.

Example 5.14 (Group Scheduling Fairness). Two users share a system. User A runs 10 CPU-bound processes, User B runs 1 CPU-bound process.

Without cgroups: All 11 processes have equal weight. User A gets $10/11 \approx 91\%$ of the CPU, User B gets $1/11 \approx 9\%$.

With cgroups (each user in their own cgroup with equal `cpu.shares`): User A's group gets 50% of the CPU (shared among 10 processes, 5% each). User B's group gets 50% of the CPU (all for the single process). Fair.

5.10.10 EEVDF: CFS's Successor

Linux 6.6 (October 2023) introduced **EEVDF** (Earliest Eligible Virtual Deadline First) as the default scheduler, replacing classic CFS. EEVDF extends the virtual runtime concept with per-task virtual deadlines:

Definition 5.24 (EEVDF Scheduler). In EEVDF, each task has a **virtual deadline** computed as:

$$\text{vdeadline}_i = \text{vruntime}_i + \frac{\text{request_size}}{w_i/W}$$

where request_size is the task's requested time slice and w_i/W is its weight fraction. The scheduler selects the eligible task (one whose vruntime $\leq$ the current virtual time) with the earliest virtual deadline. This provides better latency guarantees than classic CFS, particularly for latency-sensitive workloads like audio and interactive applications.

EEVDF preserves CFS's fairness properties while providing tighter bounds on how long a task can wait before being scheduled. The `sched_latency` and `sched_min_granularity` tunables are replaced by a unified model based on virtual deadlines and time slices.

5.11 Real-Time Scheduling

Real-time systems must meet **deadlines**: a correct result delivered too late is a failure. Real-time scheduling theory provides algorithms with provable guarantees about deadline satisfaction.

5.11.1 Real-Time Task Model

Definition 5.25 (Periodic Real-Time Task). A periodic real-time task τ_i is characterised by three parameters:

- C_i: Worst-case execution time (WCET) per invocation
- T_i: Period (time between successive invocations)
- D_i: Relative deadline (time by which each invocation must complete, typically $D_i = T_i$)

The utilisation of task τ_i is $U_i = C_i/T_i$, representing the fraction of CPU time the task requires.

Definition 5.26 (Feasibility and Schedulability). A set of real-time tasks is **feasible** if there exists a schedule that meets all deadlines. A scheduling algorithm is said to provide a **schedulability guarantee** if it can schedule all task sets whose total utilisation satisfies a given bound.

5.11.2 Rate Monotonic Scheduling (RMS)

Definition 5.27 (Rate Monotonic Scheduling). Rate Monotonic (RM) scheduling assigns static priorities based on period: the task with the shortest period gets the highest priority. RM is a fixed-priority, preemptive algorithm.

Theorem 5.8 (RM Optimality). Among all fixed-priority scheduling algorithms for periodic tasks with $D_i = T_i$, Rate Monotonic scheduling is optimal. That is, if any

fixed-priority assignment can schedule a task set, then RM can also schedule it.

Proof sketch. Consider any feasible fixed-priority assignment where task τ_i (period T_i) has higher priority than task τ_j (period T_j) but $T_i > T_j$ (i.e., the shorter-period task has lower priority). We can swap their priorities. The shorter-period task τ_j now has higher priority and generates less interference on τ_i per unit time (since C_j arrives more frequently but in smaller chunks relative to T_i). A careful analysis shows that the swapped assignment is also feasible. Repeated swaps converge to the RM assignment. □

Theorem 5.9 (RM Schedulability Bound). A set of n periodic tasks with $D_i = T_i$ is schedulable under Rate Monotonic scheduling if the total utilisation satisfies:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right)$$

The bound $n(2^{1/n} - 1)$ decreases with n:

n	**Bound**
1	1.000
2	0.828
3	0.780
4	0.757
5	0.743
$\to \infty$	$\ln 2 \approx 0.693$

The bound is sufficient but not necessary: many task sets with $U > n(2^{1/n} - 1)$ are still schedulable under RM, as long as $U \leq 1$.

Example 5.15 (RM Schedulability Test). Consider three tasks:

Task	C_i	T_i	U_i
τ_1	1	4	0.250
τ_2	2	6	0.333
τ_3	3	12	0.250

Total utilisation: $U = 0.250 + 0.333 + 0.250 = 0.833$.

RM bound for $n = 3$: $3(2^{1/3} - 1) = 3(1.2599 - 1) = 0.780$.

Since $U = 0.833 > 0.780$, the sufficient condition is not met. However, $U < 1$, so the task set might still be schedulable. We must perform an exact analysis (response time analysis or time-demand analysis) to determine feasibility.

Time-demand analysis for τ_3 (lowest priority): The time demand at time t is:

$$W_3(t) = C_3 + \left\lceil \frac{t}{T_1} \right\rceil C_1 + \left\lceil \frac{t}{T_2} \right\rceil C_2 = 3 + \left\lceil \frac{t}{4} \right\rceil \cdot 1 + \left\lceil \frac{t}{6} \right\rceil \cdot 2$$

We need to find the smallest $t \leq T_3 = 12$ such that $W_3(t) \leq t$:

- $t = 12$: $W_3(12) = 3 + 3 \cdot 1 + 2 \cdot 2 = 10 \leq 12$. Schedulable.

5.11.3 Earliest Deadline First (EDF)

Definition 5.28 (Earliest Deadline First). EDF scheduling is a dynamic-priority algorithm that assigns priorities based on absolute deadlines: the task whose deadline is nearest has the highest priority. At each scheduling point, EDF runs the ready task with the earliest absolute deadline.

Theorem 5.10 (EDF Optimality). EDF is optimal among all uniprocessor scheduling algorithms for preemptive scheduling of periodic and aperiodic tasks. If a set of tasks is feasible (i.e., there exists any schedule that meets all deadlines), then EDF will also meet all deadlines.

Theorem 5.11 (EDF Schedulability). A set of n periodic tasks with $D_i = T_i$ is schedulable under EDF if and only if:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1$$

This is both a necessary and sufficient condition, meaning EDF can utilise the processor up to 100% while still meeting all deadlines — a significant improvement over the RM bound of $\ln 2 \approx 69.3\%$.

Proof. The necessity is clear: if $U > 1$, the tasks require more CPU time than is available, so no schedule can meet all deadlines. For sufficiency, assume $U \leq 1$ and suppose for contradiction that EDF misses a deadline at time t^* for some task instance. Consider the busy interval $[0, t^*]$. The total demand in this interval is $\sum_i \lfloor t^*/T_i \rfloor \cdot C_i \leq t^* \cdot U \leq t^*$. Since the total demand does not exceed the available time, EDF

(which always works on the most urgent task) must have completed all work by t^*, contradicting the assumption. □

Example 5.16 (EDF vs RM). Using the same tasks from Example 5.15 ($U = 0.833$):

Under EDF, the task set is schedulable because $U = 0.833 \leq 1$. Under RM, the sufficient condition ($U \leq 0.780$) fails, though the exact analysis showed it was still schedulable. For task sets with $0.693 < U \leq 1.0$, EDF guarantees schedulability while RM may or may not succeed depending on the specific task parameters.

5.11.4 Linux SCHED_DEADLINE

Linux 3.14 (2014) introduced `SCHED_DEADLINE`, the first mainstream implementation of EDF with CBS (Constant Bandwidth Server) in a general-purpose operating system. A process scheduled with `SCHED_DEADLINE` specifies three parameters:

- **Runtime** (C): Maximum execution time per period
- **Deadline** (D): Relative deadline from the start of each period
- **Period** (T): The period of activation

```c
#define _GNU_SOURCE
#include <sched.h>
#include <linux/sched.h>
#include <stdio.h>
#include <string.h>
#include <sys/syscall.h>
#include <unistd.h>

struct sched_attr {
    unsigned int size;
    unsigned int sched_policy;
    unsigned long long sched_flags;
    int sched_nice;
    unsigned int sched_priority;
    unsigned long long sched_runtime;
    unsigned long long sched_deadline;
    unsigned long long sched_period;
};

int main(void) {
    struct sched_attr attr;
    memset(&attr, 0, sizeof(attr));
```

```
    attr.size = sizeof(attr);
    attr.sched_policy = 6;  /* SCHED_DEADLINE */
    attr.sched_runtime  =  5000000;  /* 5 ms */
    attr.sched_deadline = 10000000;  /* 10 ms */
    attr.sched_period   = 10000000;  /* 10 ms */

    int ret = syscall(SYS_sched_setattr, 0, &attr, 0);
    if (ret < 0) {
        perror("sched_setattr");
        return 1;
    }

    printf("Running with SCHED_DEADLINE: C=5ms, D=10ms, T=10ms\n");

    /* Real-time periodic task */
    for (int i = 0; i < 100; i++) {
        /* Do computation (must complete within 5 ms) */
        volatile long sum = 0;
        for (long j = 0; j < 1000000; j++) sum += j;

        /* Yield until next period */
        sched_yield();
    }

    return 0;
}
```

`SCHED_DEADLINE` has the highest priority among all Linux scheduling classes, even above `SCHED_FIFO`. The kernel performs admission control: a new `SCHED_DEADLINE` task is rejected if its acceptance would cause the total utilisation to exceed the CPU capacity. This prevents deadline misses due to overcommitment.

5.11.5 Windows Thread Scheduling

Windows uses a priority-based, preemptive scheduler with 32 priority levels:

- **Real-time priorities** (16–31): For time-critical threads. Not true hard real-time, but highest scheduling priority.
- **Variable priorities** (1–15): For normal threads. The scheduler dynamically adjusts priorities based on behaviour.
- **Idle priority** (0): Only runs when no other thread is ready.

Windows dynamically boosts thread priority in several situations:

- **Foreground boost**: Threads in the foreground window receive a priority boost (typically +2)
- **I/O completion boost**: A thread completing an I/O operation receives a temporary boost (varies by device: +1 for disk, +2 for serial, +6 for keyboard, +8 for sound)
- **Starvation prevention**: The "balance set manager" periodically boosts starved threads to priority 15 for one quantum

The Windows quantum defaults to 2 clock ticks (approximately 30 ms on client systems) for foreground threads and 12 clock ticks (approximately 180 ms) for server workloads, controllable via the "Processor scheduling" setting in System Properties.

5.12 Multiprocessor Scheduling

When a system has multiple CPUs, scheduling becomes more complex. The scheduler must decide not only **when** to run each process but also **where** (on which CPU).

5.12.1 Processor Affinity

Definition 5.29 (Processor Affinity). Processor affinity is the tendency to keep a process running on the same CPU across successive scheduling events. **Soft affinity** means the scheduler prefers to keep a process on the same CPU but may migrate it for load balancing. **Hard affinity** means the process is restricted to a specific set of CPUs and cannot be migrated.

Processor affinity matters because of caches: when a process runs on a CPU, its working set is loaded into that CPU's private L1 and L2 caches. Migrating the process to a different CPU invalidates these cached lines, causing a burst of cache misses on the new CPU (a **cold start**). The cost of migration can be tens of microseconds for L2 and hundreds of microseconds for L3.

On Linux, hard affinity is set via `sched_setaffinity()`:

```
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>

int main(void) {
    cpu_set_t mask;
    CPU_ZERO(&mask);
    CPU_SET(0, &mask);  /* Pin to CPU 0 */
```

```
    CPU_SET(1, &mask);  /* Also allow CPU 1 */

    if (sched_setaffinity(0, sizeof(mask), &mask) == -1) {
        perror("sched_setaffinity");
        return 1;
    }

    printf("Process pinned to CPUs 0 and 1\n");

    /* Verify */
    CPU_ZERO(&mask);
    sched_getaffinity(0, sizeof(mask), &mask);
    for (int i = 0; i < CPU_SETSIZE; i++) {
        if (CPU_ISSET(i, &mask)) {
            printf("  CPU %d: allowed\n", i);
        }
    }

    return 0;
}
```

5.12.2 Load Balancing

The scheduler must distribute work across CPUs to maximise throughput and minimise response time. Two approaches:

Push migration: A kernel thread periodically checks the load on each CPU and migrates processes from overloaded CPUs to underloaded ones. Linux's load balancer runs at regular intervals (controlled by `sched_migration_cost_ns`, default 500 microseconds).

Pull migration (work stealing): An idle CPU's scheduler "steals" a process from a busy CPU's run queue. This is reactive and low-overhead: no work is done unless a CPU is idle.

Definition 5.30 (Work Stealing). In a work-stealing scheduler, each CPU maintains a local run queue (often a deque). When a CPU's local queue is empty, it selects a victim CPU (typically at random) and steals half of the victim's queued work. Work stealing achieves $O(P \cdot T_\infty + T_1/P)$ expected time for a computation with work T_1 and span T_∞ on P processors.

5.12.3 NUMA-Aware Scheduling

On Non-Uniform Memory Access (NUMA) systems, memory access latency depends on which CPU accesses which memory bank. Each CPU has "local" memory (fast, ~80 ns) and "remote" memory (slow, ~150 ns). NUMA-aware scheduling keeps processes near their data:

- The scheduler prefers to run a process on the NUMA node where most of its memory resides
- Linux uses automatic NUMA balancing (`numa_balancing` sysctl): the kernel periodically unmaps pages, observes where page faults occur, and migrates pages to the NUMA node of the accessing CPU
- Explicit control via `numactl`, `mbind()`, and `set_mempolicy()`

5.12.4 Gang Scheduling

Definition 5.31 (Gang Scheduling). Gang scheduling (also called co-scheduling) schedules all threads of a parallel application simultaneously across different processors. If the application has k threads and the system has $P \geq k$ processors, all k threads run at the same time. If $P < k$, the threads are divided into gangs that are context-switched together.

Gang scheduling is particularly important for parallel applications where threads communicate frequently (e.g., via shared memory barriers). If threads are scheduled independently, a thread waiting for a synchronisation partner that is not currently running wastes CPU cycles spinning or blocking. Gang scheduling eliminates this **synchronisation waste**.

Programmer's Perspective

Programmer's Perspective: Linux sched_setscheduler() and Go's GOMAXPROCS. Linux exposes three scheduling policies through `sched_setscheduler()`:

- *`SCHED_OTHER` (default, CFS): Fair scheduling for normal processes. Priority is controlled by nice value (-20 to $+19$).*
- *`SCHED_FIFO`: Real-time FIFO. The highest-priority SCHED_FIFO process runs until it blocks or yields. Dangerous if misused — a runaway SCHED_FIFO process can lock out all normal processes.*
- *`SCHED_RR`: Real-time Round-Robin. Like SCHED_FIFO but with a time quantum (default 100 ms). Processes at the same priority level are Round-Robin scheduled.*

Real-time priorities range from 1 (lowest) to 99 (highest), and all real-time processes have priority over all SCHED_OTHER processes. Setting real-time scheduling requires `CAP_SYS_NICE` capability or root.

In Go, `runtime.GOMAXPROCS(n)` sets the number of OS threads (Ps) that can execute goroutines simultaneously. This is not a traditional scheduler setting — it controls the degree of parallelism in the Go runtime's M:N scheduler. Since Go 1.5, the default is `runtime.NumCPU()`. Setting `GOMAXPROCS=1` serialises all goroutine execution (useful for debugging race conditions). Go's work-stealing scheduler balances goroutines across Ps automatically, but you can use `runtime.LockOSThread()` to pin a goroutine to a specific OS thread for latency-sensitive work or CGo calls.

5.12.5 Heterogeneous Multiprocessor Scheduling

Modern mobile and laptop processors use **heterogeneous architectures** (e.g., ARM big.LITTLE, Intel's hybrid P-core/E-core designs) where cores have different performance and power characteristics.

Definition 5.32 (Heterogeneous Multiprocessor Scheduling). In a heterogeneous multiprocessor system, cores differ in performance, power consumption, or both. The scheduler must decide not only when to run a task but also on which type of core. High-performance cores ("big" or "P-cores") provide fast execution at higher power consumption, while efficient cores ("LITTLE" or "E-cores") provide lower performance at reduced power. The goal is to match task requirements to core capabilities: interactive and latency-sensitive tasks run on big cores; background and throughput-oriented tasks run on little cores.

Linux's **Energy Aware Scheduling** (EAS), introduced in kernel 5.0, integrates with CFS to make energy-efficient placement decisions. EAS uses an energy model (describing the power consumption of each CPU at each frequency) to find the most energy-efficient core for each newly woken task, while respecting performance constraints.

On Intel hybrid processors (Alder Lake and later), the **Thread Director** provides hardware hints to the scheduler about which core type is best for the current workload. The Linux kernel's `intel_thread_director` driver communicates these hints to the scheduler via the `sched_asym_prefer` mechanism.

5.13 Scheduling in Practice: A Comparison

Algorithm	Type	Optimal?	Starvation?	Overhead	Use Case
FCFS	Non-preemptive	No	No	$O(1)$	Batch systems
SJF	Non-preemptive	Yes (avg wait)	Yes	$O(n \log n)$	Short-job systems
SRTF	Preemptive	Yes (avg wait)	Yes	$O(n \log n)$	Theoretical
Round-Robin	Preemptive	No	No	$O(1)$	Time-sharing
Priority	Either	N/A	Yes	$O(n)$ or $O(\log n)$	Mixed systems
MLFQ	Preemptive	Heuristic	With boosting, no	$O(1)$ per level	General-purpose
CFS	Preemptive	Fair	No	$O(\log n)$	Linux default
RM	Preemptive	Yes (fixed-pri)	N/A	$O(n)$	Hard real-time
EDF	Preemptive	Yes (all)	N/A	$O(n \log n)$	Hard real-time

Theorem 5.12 (No Universally Optimal Scheduler). There is no single scheduling algorithm that simultaneously optimises all scheduling criteria (throughput, turnaround time, waiting time, response time, fairness) for all possible workloads. For any algorithm A and any set of criteria weights, there exists a workload W for which some other algorithm B outperforms A on the weighted objective.

This result is a consequence of the fundamental trade-off between throughput (favouring long, batch-oriented quanta) and responsiveness (favouring short, interactive quanta). Practical schedulers like MLFQ and CFS achieve a good compromise by adapting their behaviour to the observed workload.

Programmer's Perspective

Programmer's Perspective: Go's Work-Stealing Scheduler. Go's runtime scheduler implements a variant of work-stealing that is tailored to the goroutine model. Each P (processor) has a local run queue (a lock-free, bounded deque of 256 entries) and there is a single global run queue (mutex-protected). The scheduling algorithm for `findrunnable()` proceeds as follows:

1. *Check the local run queue (lock-free pop from head).*
2. *Every 61st scheduling tick, check the global queue to prevent starvation.*
3. *Check the network poller for goroutines unblocked by I/O readiness.*
4. *Attempt to steal from another random P's local queue (steal half, rounded up).*
5. *If all else fails, park the M (OS thread) and wait for work.*

The "steal half" strategy ensures that stolen work is substantial enough to amortise the cost of the steal. The 61-tick global queue check is a prime number to avoid phase-locked behaviour with periodic goroutine patterns. This design gives $O(1)$ amortised scheduling for the common case (local queue non-empty) and $O(P)$ worst case for stealing (must probe all Ps), where P is GOMAXPROCS.

5.14 Scheduling Algorithm Evaluation

How do we choose the right scheduling algorithm for a given system? Several evaluation methods exist, each with different trade-offs between accuracy and cost.

5.14.1 Deterministic Modelling

Deterministic modelling (also called **analytic evaluation**) uses a predetermined workload and computes the exact schedule for each algorithm. This is the method used in all our examples above: given specific arrival times and burst lengths, we compute waiting times, turnaround times, and response times exactly.

Example 5.17 (Comprehensive Algorithm Comparison). Consider the following workload:

Process	Arrival	Burst	Priority
P_1	0	8	3
P_2	1	4	1
P_3	2	2	4
P_4	3	1	2
P_5	4	5	5

Results summary (lower is better):

Algorithm	Avg Wait	Avg Turnaround	Avg Response
FCFS	7.2	11.2	7.2
SJF (non-preemptive)	4.6	8.6	4.6
SRTF	3.2	7.2	1.8
RR ($q = 2$)	6.6	10.6	2.2
Priority (preemptive)	5.0	9.0	3.0

SRTF provides the best average waiting time (as expected from Theorem 5.2), but RR provides competitive response time with much simpler implementation and no starvation risk.

5.14.2 Queueing Models

For analytical results, we can model the system as a queueing network. The simplest model treats the CPU as a single server with Poisson arrivals (rate λ) and exponentially distributed service times (rate μ):

Theorem 5.13 (Little's Law). For any stable queueing system, the long-run average number of customers in the system L, the long-run average arrival rate λ, and the long-run average time a customer spends in the system W are related by:

$$L = \lambda \cdot W$$

This holds regardless of the arrival distribution, service distribution, or scheduling discipline. Applied to CPU scheduling: if the average number of processes in the

ready queue is L_q and the average arrival rate is λ, then the average waiting time is $W_q = L_q/\lambda$.

For an M/M/1 queue (Poisson arrivals, exponential service, single server), the utilisation $\rho = \lambda/\mu$, and the average waiting time is:

$$W_q = \frac{\rho}{\mu(1-\rho)}$$

This diverges as $\rho \to 1$, reflecting the reality that a fully loaded CPU has infinite waiting times.

5.14.3 Simulation

The most flexible evaluation method is discrete-event simulation. A simulator maintains a clock, a set of events (process arrivals, burst completions, I/O completions), and the state of the scheduling algorithm. It processes events in chronological order, updating the schedule and collecting statistics.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* Simple FCFS simulator */
typedef struct {
    int arrival;
    int burst;
} Job;

int compare_arrival(const void *a, const void *b) {
    return ((Job *)a)->arrival - ((Job *)b)->arrival;
}

void simulate_fcfs(Job jobs[], int n) {
    qsort(jobs, n, sizeof(Job), compare_arrival);

    int time = 0;
    double total_wait = 0, total_turnaround = 0;

    for (int i = 0; i < n; i++) {
        if (time < jobs[i].arrival) {
            time = jobs[i].arrival;
```

```
        }
        int wait = time - jobs[i].arrival;
        int turnaround = wait + jobs[i].burst;
        total_wait += wait;
        total_turnaround += turnaround;
        time += jobs[i].burst;
    }

    printf("FCFS: avg_wait=%.2f, avg_turnaround=%.2f\n",
           total_wait / n, total_turnaround / n);
}

int main(void) {
    srand(time(NULL));

    int n = 1000;
    Job *jobs = malloc(n * sizeof(Job));

    /* Generate random workload: Poisson arrivals, exponential bursts */
    int arrival = 0;
    for (int i = 0; i < n; i++) {
        arrival += rand() % 10 + 1;  /* Inter-arrival: 1-10 */
        jobs[i].arrival = arrival;
        jobs[i].burst = rand() % 20 + 1;  /* Burst: 1-20 */
    }

    simulate_fcfs(jobs, n);

    free(jobs);
    return 0;
}
```

Simulation allows evaluation with realistic workload distributions (e.g., heavy-tailed burst distributions, bursty arrivals) that are difficult to analyse mathematically.

5.14.4 Implementation and Measurement

The ultimate evaluation is to implement the algorithm in a real operating system and measure its performance with actual workloads. Linux provides several tools for this:

- **`perf sched`**: Records and analyses scheduling events (context switches,

migration, latency)

- **`schedstat`**: Per-CPU scheduling statistics in `/proc/schedstat`
- **`/proc/<pid>/sched`**: Per-process scheduling statistics (wait time, run time, switches)
- **`trace-cmd`**: Frontend for `ftrace` kernel tracing, including the `sched` tracer

```
$ perf sched record -- sleep 10
$ perf sched latency
-------------------------------------------------
  Task              |   Runtime   |  Switches  | Avg delay |
-------------------------------------------------
  firefox       :  4218.432 ms  |     3847   | 0.312 ms  |
  Xorg          :  1847.221 ms  |     5821   | 0.087 ms  |
  pulseaudio    :   127.894 ms  |      842   | 0.041 ms  |
  bash          :    34.102 ms  |       12   | 2.341 ms  |
-------------------------------------------------
```

Exercises

1. **Exercise 5.1.** Five processes arrive at the following times with the given burst lengths:

Process	Arrival	Burst
P_1	0	10
P_2	1	5
P_3	3	2
P_4	5	8
P_5	6	4

   Compute the average waiting time and average turnaround time under: (a) FCFS, (b) non-preemptive SJF, (c) SRTF, and (d) Round-Robin with $q = 3$. Draw the Gantt chart for each. Which algorithm gives the best average waiting time, and why?

2. **Exercise 5.2.** Prove that for n periodic real-time tasks, the Rate Monotonic utilisation bound $n(2^{1/n} - 1)$ converges to $\ln 2$ as $n \to \infty$. *Hint*: Use the Taylor expansion of $2^{1/n} = e^{(\ln 2)/n}$ and the limit $\lim_{n\to\infty} n(e^{x/n} - 1) = x$.

3. **Exercise 5.3.** Consider two processes running under CFS on a single-core system. Process A has nice value 0 (weight 1024) and process B has nice value 10 (weight 110). Compute: (a) the CPU share of each process, (b) the time slice of each process assuming a scheduling period of 6 ms, (c) how much physical time must elapse for each process to accumulate 1 ms of vruntime. If both processes are CPU-bound, describe the scheduling pattern over a 60 ms window.

4. **Exercise 5.4.** An MLFQ has three queues with quanta 4 ms, 8 ms, and 16 ms. A process P arrives and exhibits the following CPU burst pattern: 3 ms, I/O, 6 ms, I/O, 20 ms. Trace P's movement through the queues under: (a) basic MLFQ rules (Rule 4a: demote on full quantum), (b) accounting-based MLFQ (demote when total CPU time at a level exceeds the quantum). In which case does P receive better response time, and why?

5. **Exercise 5.5.** Three periodic tasks have parameters $\tau_1 = (C_1 = 1, T_1 = 5)$, $\tau_2 = (C_2 = 2, T_2 = 8)$, $\tau_3 = (C_3 = 4, T_3 = 20)$. Determine whether this task set is schedulable under: (a) Rate Monotonic scheduling (using both the utilisation bound and exact response time analysis), (b) EDF scheduling. Draw the schedule for both algorithms over the hyperperiod $\text{lcm}(5, 8, 20) = 40$ time units.

6. **Exercise 5.6.** The priority inversion problem can lead to unbounded blocking under basic priority scheduling. Describe a concrete scenario with three processes H (high priority), M (medium), and L (low) where H is blocked for an arbitrarily long time due to priority inversion. Then show how the Priority Inheritance Protocol bounds H's blocking time to the length of L's critical section. Finally, explain why Priority Ceiling Protocol provides an even stronger guarantee (at most one blocking per resource).

7. **Exercise 5.7.** Write a C program that creates two processes: one CPU-bound (infinite loop incrementing a counter) and one I/O-bound (repeatedly writing to `/dev/null` with small writes). Use `sched_setscheduler()` to set the CPU-bound process to `SCHED_RR` with priority 50 and the I/O-bound process to `SCHED_FIFO` with priority 60. Observe the behaviour using `top` or `htop`. Then swap the priorities and observe again. Explain the difference in terms of Linux's real-time scheduling semantics. *Note*: this requires root or `CAP_SYS_NICE`.

Chapter 6: Inter-Process Communication

Processes, by design, are isolated from one another. Each lives in its own address space, unable to read or modify another process's memory. This isolation is a cornerstone of system security and stability — a buggy process cannot corrupt another's data. But isolation creates a problem: processes that must cooperate need a way to exchange data and coordinate their actions. The mechanisms that enable this cooperation are collectively called **Inter-Process Communication** (IPC).

This chapter surveys the IPC landscape from low-level shared memory and pipes to high-level frameworks like gRPC and D-Bus. We examine the trade-offs between each mechanism — latency, throughput, ease of programming, and safety — and connect the theory to real system implementations on Linux.

6.1 Taxonomy of IPC Mechanisms

IPC mechanisms fall into two broad categories:

Definition 6.1 (Shared Memory IPC). In shared memory IPC, two or more processes map the same region of physical memory into their respective virtual address spaces. Communication occurs by reading and writing to this shared region. The operating system is involved only in establishing the mapping; subsequent data transfer occurs without kernel intervention, making shared memory the fastest IPC mechanism.

Definition 6.2 (Message Passing IPC). In message passing IPC, processes exchange data by sending and receiving messages through a kernel-mediated channel (pipe, socket, message queue). The kernel copies data from the sender's address space to the receiver's (or to an intermediate buffer). Message passing is safer (no shared state to corrupt) but slower (each transfer requires at least one system call and one data copy).

The following table summarises the primary IPC mechanisms on Unix-like systems:

Mechanism	Type	Scope	Persistence	Typical Latency
Shared memory	Shared	Same machine	Kernel-managed	~10-100 ns
Anonymous pipe	Message	Parent-child	Process lifetime	~1-5 us
Named pipe (FIFO)	Message	Same machine	Filesystem	~1-5 us
Unix domain socket	Message	Same machine	Filesystem	~1-3 us
TCP/UDP socket	Message	Network-wide	Connection lifetime	~10-100 us (local)
Signal	Notification	Same machine	Instantaneous	~1-10 us
Message queue	Message	Same machine	Kernel-managed	~2-5 us
Memory-mapped file	Shared	Same machine	Filesystem	~10-100 ns

6.2 Shared Memory

Shared memory is the highest-performance IPC mechanism because, after the initial setup, data transfer between processes involves no system calls and no kernel mediation. Processes simply read and write to ordinary memory addresses.

6.2.1 POSIX Shared Memory

The POSIX shared memory API provides a portable interface for creating and mapping shared memory regions:

```
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```c
#define SHM_NAME "/my_shared_mem"
#define SHM_SIZE 4096

/* Producer process */
int main(void) {
    /* Create shared memory object */
    int fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (fd == -1) {
        perror("shm_open");
        return 1;
    }

    /* Set size */
    if (ftruncate(fd, SHM_SIZE) == -1) {
        perror("ftruncate");
        return 1;
    }

    /* Map into address space */
    void *ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    /* Write data */
    const char *message = "Hello from the producer process!";
    memcpy(ptr, message, strlen(message) + 1);
    printf("Producer wrote: %s\n", message);

    /* Cleanup */
    munmap(ptr, SHM_SIZE);
    close(fd);

    /* Consumer will read; producer keeps the object alive */
    printf("Waiting for consumer... (press Enter)\n");
    getchar();
```

```
    /* Remove shared memory object */
    shm_unlink(SHM_NAME);

    return 0;
}
```

The consumer process:

```
#include <fcntl.h>
#include <sys/mman.h>
#include <stdio.h>
#include <unistd.h>

#define SHM_NAME "/my_shared_mem"
#define SHM_SIZE 4096

/* Consumer process */
int main(void) {
    /* Open existing shared memory object */
    int fd = shm_open(SHM_NAME, O_RDONLY, 0);
    if (fd == -1) {
        perror("shm_open");
        return 1;
    }

    /* Map read-only */
    void *ptr = mmap(NULL, SHM_SIZE, PROT_READ,
                     MAP_SHARED, fd, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    /* Read data */
    printf("Consumer read: %s\n", (char *)ptr);

    munmap(ptr, SHM_SIZE);
    close(fd);

    return 0;
}
```

Compile both with: `gcc -o producer producer.c -lrt` and `gcc -o consumer consumer.c -lrt`

6.2.2 mmap: Memory-Mapped Files

The `mmap()` system call maps a file (or anonymous memory) into a process's address space. When used with `MAP_SHARED`, modifications are visible to all processes mapping the same file, providing a form of shared memory backed by the filesystem.

Definition 6.3 (Memory-Mapped I/O). Memory-mapped I/O via `mmap()` establishes a correspondence between a region of a process's virtual address space and a file (or device). Reads and writes to the mapped region translate to reads and writes of the file. The kernel's page cache handles the actual I/O, providing transparent caching and write-back.

```c
#include <fcntl.h>
#include <sys/mman.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    int fd = open("/tmp/shared_data.bin", O_CREAT | O_RDWR, 0666);
    ftruncate(fd, 4096);

    /* MAP_SHARED: changes visible to other processes and written to file */
    int *data = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    close(fd);

    /* Write structured data */
    data[0] = 42;      /* Sequence number */
    data[1] = 12345;   /* Payload */

    /* Force write to backing file (optional; happens lazily otherwise) */
    msync(data, 4096, MS_SYNC);

    printf("Wrote: seq=%d, payload=%d\n", data[0], data[1]);

    munmap(data, 4096);
    return 0;
```

```
}
```

6.2.3 Race Conditions in Shared Memory

The fundamental problem with shared memory is that concurrent access requires explicit synchronisation. Without it, processes can observe inconsistent data.

Definition 6.4 (Race Condition). A race condition occurs when the behaviour of a program depends on the relative timing of events (e.g., the order in which two processes access shared memory). Race conditions lead to non-deterministic bugs that are difficult to reproduce and diagnose.

Example 6.1 (Shared Memory Race Condition). Two processes share a counter in shared memory. Each process increments the counter 1,000,000 times. The increment operation `counter++` compiles to three machine instructions:

1. Load `counter` from memory into a register
2. Add 1 to the register
3. Store the register back to `counter`

If both processes execute these instructions concurrently, the following interleaving can occur:

```
Process A                   Process B
──────────────────────────────────────────

LOAD counter (= 100)
                            LOAD counter (= 100)
ADD 1 (register = 101)
                            ADD 1 (register = 101)
STORE counter (= 101)
                            STORE counter (= 101)
```

Both processes read 100, increment to 101, and store 101. One increment is lost. After 2,000,000 increments, the counter may hold any value between 1,000,000 and 2,000,000, depending on how many increments are lost to races.

Shared memory IPC must be paired with synchronisation primitives (mutexes, semaphores, atomic operations) to prevent race conditions. These are covered in detail in subsequent chapters on synchronisation.

6.3 Pipes

Pipes are the simplest and oldest Unix IPC mechanism. A pipe provides a unidirectional byte stream between two processes.

6.3.1 Anonymous Pipes

Definition 6.5 (Anonymous Pipe). An anonymous pipe is a kernel buffer (typically 64 KB on Linux) that connects the standard output of one process to the standard input of another. It is created by the `pipe()` system call, which returns two file descriptors: one for the read end and one for the write end. Anonymous pipes can only be shared between related processes (parent-child or siblings), because the file descriptors are inherited through `fork()`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int pipefd[2];

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();

    if (pid == 0) {
        /* Child: reads from pipe */
        close(pipefd[1]);  /* Close write end */

        char buffer[256];
        ssize_t n = read(pipefd[0], buffer, sizeof(buffer) - 1);
        if (n > 0) {
            buffer[n] = '\0';
            printf("Child received: %s\n", buffer);
        }

        close(pipefd[0]);
        _exit(0);
    } else {
        /* Parent: writes to pipe */
        close(pipefd[0]);  /* Close read end */
```

```
        const char *msg = "Hello from parent!";
        write(pipefd[1], msg, strlen(msg));

        close(pipefd[1]);  /* Signal EOF to child */
        wait(NULL);
    }

    return 0;
}
```

The shell pipe operator `|` creates anonymous pipes between commands:

```
$ cat /var/log/syslog | grep error | wc -l
```

This creates two pipes: one connecting `cat`'s stdout to `grep`'s stdin, and another connecting `grep`'s stdout to `wc`'s stdin.

6.3.2 Pipe Semantics

Pipe operations have well-defined semantics that depend on the state of both ends:

Operation	Both ends open	Write end closed	Read end closed
`read()`	Blocks if empty; returns data if available	Returns 0 (EOF) when buffer drained	N/A
`write()`	Blocks if full; writes data if space	N/A	`SIGPIPE` signal; `write()` returns -1 with `errno = EPIPE`

Theorem 6.1 (Pipe Atomicity). On POSIX-compliant systems, writes to a pipe of `PIPE_BUF` bytes or fewer are guaranteed to be atomic: if two processes write to the same pipe simultaneously, and both writes are $\leq$ `PIPE_BUF` bytes, the data from the two writes will not be interleaved. On Linux, `PIPE_BUF = 4096` bytes. Writes larger than `PIPE_BUF` are not atomic and may be interleaved with writes from other processes.

6.3.3 Pipe Buffer Sizes

The kernel pipe buffer size has evolved over Linux kernel versions:

- **Linux 2.6.11 and earlier**: Single page (4 KB) buffer
- **Linux 2.6.11 to 2.6.35**: 64 KB buffer (16 pages in a circular buffer)
- **Linux 2.6.35+**: Configurable per-pipe via `fcntl(fd, F_SETPIPE_SZ, size)`, up to `/proc/sys/fs/pipe-max-size` (default 1 MB)

```c
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(void) {
    int pipefd[2];
    pipe(pipefd);

    /* Query current pipe buffer size */
    int size = fcntl(pipefd[0], F_GETPIPE_SZ);
    printf("Default pipe buffer: %d bytes\n", size);

    /* Increase to 1 MB */
    int new_size = fcntl(pipefd[0], F_SETPIPE_SZ, 1024 * 1024);
    printf("New pipe buffer: %d bytes\n", new_size);

    close(pipefd[0]);
    close(pipefd[1]);
    return 0;
}
```

6.3.4 Named Pipes (FIFOs)

Definition 6.6 (Named Pipe / FIFO). A named pipe (FIFO) is a pipe with a name in the filesystem. Unlike anonymous pipes, named pipes can be used between unrelated processes. A FIFO is created with `mkfifo()` and appears as a special file in the filesystem. Opening a FIFO for reading blocks until another process opens it for writing, and vice versa (unless `O_NONBLOCK` is used).

```c
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <unistd.h>

/* Writer process */
```

```c
int main(void) {
    const char *fifo_path = "/tmp/my_fifo";

    /* Create FIFO (if it doesn't exist) */
    mkfifo(fifo_path, 0666);

    /* Open for writing (blocks until a reader opens the other end) */
    printf("Waiting for reader...\n");
    int fd = open(fifo_path, O_WRONLY);

    const char *msg = "Hello via named pipe!";
    write(fd, msg, strlen(msg));
    printf("Wrote: %s\n", msg);

    close(fd);
    return 0;
}
```

Named pipes have the same semantics as anonymous pipes (unidirectional, byte stream, `PIPE_BUF` atomicity) but with filesystem persistence: the FIFO special file persists until explicitly removed with `unlink()`.

6.4 Signals

Signals are the oldest form of IPC in Unix. A signal is a software interrupt delivered to a process, notifying it of an asynchronous event.

Definition 6.7 (Signal). A signal is an asynchronous notification sent to a process (or a specific thread within a process) to indicate that an event has occurred. Each signal has a number (1–64 on Linux), a name (e.g., `SIGTERM`, `SIGSEGV`), and a default disposition (terminate, ignore, stop, or core dump). A process can change the disposition of most signals by installing a signal handler.

6.4.1 Signal Categories

The standard POSIX signals include:

Signal	Number	Default Action	Cause
`SIGHUP`	1	Terminate	Terminal hangup

Signal	Number	Default Action	Cause
`SIGINT`	2	Terminate	Ctrl+C
`SIGQUIT`	3	Core dump	Ctrl+\
`SIGILL`	4	Core dump	Illegal instruction
`SIGABRT`	6	Core dump	`abort()` call
`SIGFPE`	8	Core dump	Arithmetic error (e.g., division by zero)
`SIGKILL`	9	Terminate	Uncatchable kill
`SIGSEGV`	11	Core dump	Segmentation fault
`SIGPIPE`	13	Terminate	Write to pipe with no reader
`SIGALRM`	14	Terminate	Timer expired
`SIGTERM`	15	Terminate	Polite termination request
`SIGCHLD`	17	Ignore	Child process terminated
`SIGSTOP`	19	Stop	Uncatchable stop
`SIGTSTP`	20	Stop	Ctrl+Z
`SIGURG`	23	Ignore	Used by Go runtime for goroutine preemption

`SIGKILL` and `SIGSTOP` cannot be caught, blocked, or ignored. All other signals can have custom handlers installed.

6.4.2 Signal Handling

```c
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

volatile sig_atomic_t got_signal = 0;

void handler(int signum) {
    /* Only async-signal-safe functions here */
    got_signal = signum;
}
```

```c
int main(void) {
    struct sigaction sa;
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;  /* Restart interrupted system calls */

    /* Install handler for SIGINT (Ctrl+C) */
    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    /* Install handler for SIGTERM */
    if (sigaction(SIGTERM, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("PID %d waiting for signals (Ctrl+C or kill %d)...\n",
           getpid(), getpid());

    while (!got_signal) {
        pause();  /* Sleep until a signal is delivered */
    }

    printf("\nReceived signal %d, shutting down gracefully.\n", got_signal);

    return 0;
}
```

Programmer's Perspective

Programmer's Perspective: Signal Safety and Go. Signal handling in C is notoriously tricky. Inside a signal handler, only **async-signal-safe** functions may be called (about 100 functions listed in POSIX, including `write()`, `_exit()`, and `signal()` itself, but NOT `printf()`, `malloc()`, or `pthread_mutex_lock()`). The reason is that a signal can interrupt a non-reentrant function mid-execution; calling the same function from the handler would corrupt its internal state. The canonical pattern is to set a `volatile sig_atomic_t` flag in the handler and check it in the main loop.

Go abstracts signals through the `os/signal` package, which delivers signals to channels:

```go
package main

import (
    "fmt"
    "os"
    "os/signal"
    "syscall"
)

func main() {
    sigChan := make(chan os.Signal, 1)
    signal.Notify(sigChan, syscall.SIGINT, syscall.SIGTERM)

    fmt.Println("Waiting for signal...")
    sig := <-sigChan
    fmt.Printf("Received %v, shutting down.\n", sig)
}
```

The Go runtime intercepts signals at the OS level and translates them into channel sends, which are safe to handle with arbitrary Go code (no async-signal-safety restrictions). The runtime reserves certain signals for internal use: `SIGURG` for goroutine preemption (since Go 1.14), `SIGPROF` for CPU profiling, and several others. User code should not catch these.

6.4.3 Signal Masks

Each thread has a **signal mask**: a bitmask specifying which signals are currently blocked (deferred). A blocked signal is held pending until it is unblocked.

```c
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

int main(void) {
    sigset_t mask, oldmask, pending;

    /* Block SIGINT */
    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);
```

```
    sigprocmask(SIG_BLOCK, &mask, &oldmask);

    printf("SIGINT blocked. Press Ctrl+C (it will be held pending)...\n");
    sleep(5);

    /* Check for pending signals */
    sigpending(&pending);
    if (sigismember(&pending, SIGINT)) {
        printf("SIGINT is pending!\n");
    }

    /* Unblock SIGINT --- pending signal will be delivered immediately */
    printf("Unblocking SIGINT...\n");
    sigprocmask(SIG_SETMASK, &oldmask, NULL);

    /* If SIGINT was pending, default handler runs here (terminate) */
    printf("This line may not be reached if Ctrl+C was pressed.\n");

    return 0;
}
```

6.4.4 Real-Time Signals

Standard signals (1–31) have a significant limitation: they are not queued. If the same signal is sent multiple times while blocked, only one instance is delivered when the signal is unblocked. Real-time signals (32–64, accessed via `SIGRTMIN` to `SIGRTMAX`) address this:

Definition 6.8 (Real-Time Signals). Real-time signals (numbers 32–64 on Linux) provide guaranteed delivery semantics:

1. **Queuing**: Multiple instances of the same real-time signal are queued and delivered individually
2. **Ordering**: Real-time signals are delivered in signal-number order (lower numbers first); multiple instances of the same signal are delivered in the order sent
3. **Data payload**: Real-time signals can carry an integer or pointer value via `sigqueue()`, enabling simple data transfer alongside the notification

```
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <unistd.h>

void rt_handler(int sig, siginfo_t *info, void *ucontext) {
    printf("Received RT signal %d with value %d from PID %d\n",
           sig, info->si_value.sival_int, info->si_pid);
}

int main(void) {
    struct sigaction sa;
    sa.sa_sigaction = rt_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_SIGINFO;  /* Use sa_sigaction instead of sa_handler */

    sigaction(SIGRTMIN, &sa, NULL);

    /* Send a real-time signal to ourselves with a data payload */
    union sigval value;
    value.sival_int = 12345;
    sigqueue(getpid(), SIGRTMIN, value);

    /* Allow signal delivery */
    sleep(1);

    return 0;
}
```

6.5 Message Passing

Message passing provides structured communication between processes through explicit send and receive operations. Unlike shared memory, message passing does not require processes to share any memory; all data transfer is mediated by the kernel.

6.5.1 Synchronous vs Asynchronous Message Passing

Definition 6.9 (Synchronous Message Passing). In synchronous (blocking) message passing:

- **Blocking send**: The sender blocks until the receiver has received the message
- **Blocking receive**: The receiver blocks until a message is available

Synchronous message passing provides a natural rendezvous point: when `send()`

returns, the sender knows the message has been received. This simplifies reasoning about program correctness but can reduce concurrency (the sender is idle while waiting).

Definition 6.10 (Asynchronous Message Passing). In asynchronous (non-blocking) message passing:

- **Non-blocking send**: The sender places the message in a buffer and returns immediately, without waiting for the receiver
- **Non-blocking receive**: The receiver returns immediately with a message (if available) or an indication that no message is pending

Asynchronous message passing requires a buffer (mailbox) to hold messages between send and receive. It maximises concurrency but introduces complexity: the sender does not know when (or if) the message is received, and buffer overflow must be handled.

6.5.2 POSIX Message Queues

POSIX message queues provide a kernel-managed message passing interface with priority support:

```
#include <fcntl.h>
#include <mqueue.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define QUEUE_NAME  "/my_queue"
#define MAX_MSG_SIZE 256
#define MAX_MSGS     10

/* Sender */
int main(void) {
    struct mq_attr attr;
    attr.mq_flags = 0;
    attr.mq_maxmsg = MAX_MSGS;
    attr.mq_msgsize = MAX_MSG_SIZE;
    attr.mq_curmsgs = 0;

    mqd_t mq = mq_open(QUEUE_NAME, O_CREAT | O_WRONLY, 0666, &attr);
    if (mq == (mqd_t)-1) {
```

```
        perror("mq_open");
        return 1;
    }

    const char *messages[] = {
        "First message (priority 1)",
        "Second message (priority 5)",
        "Third message (priority 3)"
    };
    unsigned int priorities[] = {1, 5, 3};

    for (int i = 0; i < 3; i++) {
        if (mq_send(mq, messages[i], strlen(messages[i]) + 1,
                    priorities[i]) == -1) {
            perror("mq_send");
        } else {
            printf("Sent: %s (priority %u)\n", messages[i], priorities[i]);
        }
    }

    mq_close(mq);
    return 0;
}
```

The receiver retrieves messages in priority order (highest priority first):

```
#include <fcntl.h>
#include <mqueue.h>
#include <stdio.h>
#include <stdlib.h>

#define QUEUE_NAME   "/my_queue"
#define MAX_MSG_SIZE 256

/* Receiver */
int main(void) {
    mqd_t mq = mq_open(QUEUE_NAME, O_RDONLY);
    if (mq == (mqd_t)-1) {
        perror("mq_open");
        return 1;
    }
```

```
    char buffer[MAX_MSG_SIZE + 1];
    unsigned int priority;

    for (int i = 0; i < 3; i++) {
        ssize_t n = mq_receive(mq, buffer, MAX_MSG_SIZE + 1, &priority);
        if (n >= 0) {
            buffer[n] = '\0';
            printf("Received (priority %u): %s\n", priority, buffer);
        }
    }

    mq_close(mq);
    mq_unlink(QUEUE_NAME);
    return 0;
}
```

Compile with: `gcc -o sender sender.c -lrt` and `gcc -o receiver receiver.c -lrt`

Output (note priority ordering):

```
Received (priority 5): Second message (priority 5)
Received (priority 3): Third message (priority 3)
Received (priority 1): First message (priority 1)
```

6.5.3 Mailboxes and Ports

Definition 6.11 (Mailbox). A mailbox (also called a port) is a kernel object that holds a queue of messages. Processes send messages to a mailbox and receive messages from a mailbox. Multiple processes can send to the same mailbox, and (depending on the implementation) multiple processes can receive from it. A mailbox decouples senders from receivers: neither needs to know the identity of the other.

Mailbox semantics vary by operating system:

- **Mach** (macOS/iOS kernel): Ports are the fundamental IPC mechanism. Each task has a set of port rights (send rights, receive rights). Messages are sent to ports, and the kernel queues them until the receiver collects them. Mach messages can carry structured data, memory regions (via out-of-line memory), and port rights.
- **POSIX message queues**: Named kernel objects (as shown above) that act as mailboxes with priority ordering.

- **SysV message queues**: Older IPC mechanism using `msgget()`, `msgsnd()`, `msgrcv()`. Messages have a type field that allows selective reception (receive only messages of a specific type).

6.6 Unix Domain Sockets

Unix domain sockets provide bidirectional communication between processes on the same machine, using the familiar socket API. They are faster than TCP sockets for local communication because they bypass the network stack entirely.

Definition 6.12 (Unix Domain Socket). A Unix domain socket is a socket that uses a filesystem path (or an abstract name in the Linux-specific abstract namespace) instead of an IP address and port. Communication occurs entirely within the kernel, without network protocol processing (no TCP/IP headers, no checksums, no routing). Unix domain sockets support both stream (SOCK_STREAM) and datagram (SOCK_DGRAM) communication.

6.6.1 Stream Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>

#define SOCKET_PATH "/tmp/my_socket"

/* Server */
int main(void) {
    int server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("socket");
        return 1;
    }

    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, SOCKET_PATH, sizeof(addr.sun_path) - 1);
```

```
    unlink(SOCKET_PATH);  /* Remove stale socket file */

    if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
        perror("bind");
        return 1;
    }

    if (listen(server_fd, 5) == -1) {
        perror("listen");
        return 1;
    }

    printf("Server listening on %s\n", SOCKET_PATH);

    int client_fd = accept(server_fd, NULL, NULL);
    if (client_fd == -1) {
        perror("accept");
        return 1;
    }

    char buffer[256];
    ssize_t n = read(client_fd, buffer, sizeof(buffer) - 1);
    if (n > 0) {
        buffer[n] = '\0';
        printf("Server received: %s\n", buffer);

        const char *reply = "Message received!";
        write(client_fd, reply, strlen(reply));
    }

    close(client_fd);
    close(server_fd);
    unlink(SOCKET_PATH);

    return 0;
}
```

6.6.2 Unix Domain Sockets vs TCP for Local IPC

Example 6.2 (Performance Comparison: Unix Socket vs TCP Loopback). On a typical Linux system, benchmarking request-response latency for small messages (64 bytes):

Transport	Latency (median)	Throughput (messages/sec)
Unix domain socket (SOCK_STREAM)	~5 us	~200,000
TCP loopback (127.0.0.1)	~12 us	~85,000
Unix domain socket (SOCK_DGRAM)	~4 us	~250,000
Shared memory + futex	~0.5 us	~2,000,000

Unix domain sockets are roughly 2–3x faster than TCP loopback because they avoid the TCP/IP protocol stack (no TCP segmentation, acknowledgment, congestion control, checksum computation). Shared memory with futex synchronisation is another order of magnitude faster but requires careful synchronisation.

Unix domain sockets have a unique capability: they can transfer **file descriptors** between processes via `SCM_RIGHTS` ancillary messages. This is used by process managers (like systemd) to pass listening sockets to child processes, and by container runtimes to pass device file descriptors.

6.6.3 Passing File Descriptors

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>

/* Send a file descriptor over a Unix domain socket */
int send_fd(int socket, int fd_to_send) {
    struct msghdr msg = {0};
    struct iovec iov;
    char buf[1] = {'F'};  /* Must send at least one byte of data */

    iov.iov_base = buf;
```

```
    iov.iov_len = 1;
    msg.msg_iov = &iov;
    msg.msg_iovlen = 1;

    /* Ancillary data carrying the file descriptor */
    char cmsg_buf[CMSG_SPACE(sizeof(int))];
    msg.msg_control = cmsg_buf;
    msg.msg_controllen = sizeof(cmsg_buf);

    struct cmsghdr *cmsg = CMSG_FIRSTHDR(&msg);
    cmsg->cmsg_level = SOL_SOCKET;
    cmsg->cmsg_type = SCM_RIGHTS;
    cmsg->cmsg_len = CMSG_LEN(sizeof(int));
    memcpy(CMSG_DATA(cmsg), &fd_to_send, sizeof(int));

    return sendmsg(socket, &msg, 0);
}
```

Theorem 6.2 (File Descriptor Transfer Semantics). When a file descriptor f is transferred from process A to process B via `SCM_RIGHTS` over a Unix domain socket, process B receives a new file descriptor number f' (chosen by the kernel) that refers to the same underlying kernel file description (struct file) as f in process A. The two file descriptors share the file offset, status flags, and access mode. Process A and B can independently close their respective file descriptors; the underlying file is closed only when all referring file descriptors are closed.

6.7 D-Bus and Higher-Level IPC Frameworks

While pipes, sockets, and shared memory provide the building blocks of IPC, real-world desktop and server applications typically use higher-level frameworks that provide structured messaging, service discovery, and security.

6.7.1 D-Bus

Definition 6.13 (D-Bus). D-Bus is a message bus system providing structured IPC for Linux desktop and system services. It defines a binary wire protocol and provides two standard bus instances:

- **System bus**: For communication between system services (one per machine)
- **Session bus**: For communication between user applications (one per login session)

D-Bus uses Unix domain sockets as its transport and provides service naming, interface discovery, signal broadcasting, and security policies.

D-Bus messages are typed and structured. Each message contains:

- **Path**: Object path (e.g., `/org/freedesktop/NetworkManager`)
- **Interface**: Interface name (e.g., `org.freedesktop.NetworkManager`)
- **Member**: Method name or signal name (e.g., `GetDevices`)
- **Arguments**: Typed arguments following D-Bus type signatures

Example 6.3 (D-Bus Architecture). A typical D-Bus interaction for retrieving the battery level:

```
Application                    D-Bus Daemon                 UPower Service
    |                              |                              |
    |  MethodCall:                 |                              |
    |  dest=org.freedesktop.UPower |                              |
    |  path=/org/freedesktop/      |                              |
    |       UPower/devices/bat0    |                              |
    |  iface=org.freedesktop.      |                              |
    |        DBus.Properties       |                              |
    |  method=Get                  |                              |
    |  args=("...Device",          |                              |
    |        "Percentage")         |                              |
    |------------------------------>|  Forward to UPower           |
    |                              |------------------------------>|
    |                              |                              |
    |                              |  MethodReturn:               |
    |                              |<------------------------------|
    |  Reply: Variant(85.0)        |                              |
    |<------------------------------|                              |
```

The D-Bus daemon acts as a message router, matching destination bus names to connected services. Services register well-known names (e.g., `org.freedesktop.UPower`) that clients use to address them.

D-Bus is used extensively in the Linux desktop ecosystem:

- **systemd**: Service management via `org.freedesktop.systemd1`
- **NetworkManager**: Network configuration
- **PulseAudio/PipeWire**: Audio routing
- **Freedesktop notifications**: Desktop notification popups
- **GNOME/KDE services**: Settings, power management, screen locking

6.7.2 Alternatives to D-Bus

While D-Bus dominates the Linux desktop, other higher-level IPC frameworks exist:

- **Varlink**: A simpler alternative to D-Bus used by systemd for some interfaces, based on JSON over Unix domain sockets
- **kdbus / Bus1**: Kernel-based message bus proposals (kdbus was rejected from mainline Linux; Bus1 is an ongoing effort)
- **Binder**: Android's IPC mechanism, based on shared memory with kernel-mediated transaction semantics
- **XPC** (macOS): Apple's lightweight IPC framework for communication between launchd-managed services

6.8 Remote Procedure Call (RPC)

Remote Procedure Call is an IPC paradigm that makes invoking a function on a remote machine (or a different process) look syntactically identical to calling a local function. The RPC framework handles the messy details of serialisation, network communication, and error handling.

6.8.1 RPC Architecture

Definition 6.14 (Remote Procedure Call). A Remote Procedure Call (RPC) is a communication protocol that allows a program to execute a procedure (function) on a remote server as if it were a local call. The RPC framework provides:

1. **Interface Definition Language (IDL)**: A language-neutral specification of the service's API
2. **Stubs**: Generated client and server code that handles marshalling/unmarshalling
3. **Marshalling** (serialisation): Converting function arguments from in-memory representation to a wire format
4. **Unmarshalling** (deserialisation): Converting wire format back to in-memory representation
5. **Transport**: The underlying communication channel (TCP, Unix socket, HTTP/2)

The RPC call flow:

```
Client Process                                    Server Process
┌──────────────────┐                            ┌──────────────────┐
│ Application Code │                            │ Application Code │
│                  │                            │                  │
│ result = Add(3,5)│                            │ func Add(a,b) {  │
```

```
│       │          │                          │   return a + b   │
│       ▼          │                          │ }                │
│ ┌─────────────┐  │                          │ ┌──────────────┐ │
│ │ Client Stub │  │                          │ │ Server Stub  │ │
│ │ Marshal args│  │                          │ │ Unmarshal    │ │
│ └─────────────┘  │                          │ └──────▲───────┘ │
│        │         │                          │        │         │
│ ┌──────▼──────┐  │    Network / Socket      │ ┌──────┴───────┐ │
│ │  Transport  │──┼──────────────────────────►│ │  Transport   │ │
│ └─────────────┘  │◄──────────────────────────┼─│              │ │
└──────────────────┘        Response           └──────────────────┘
```

6.8.2 Marshalling and Wire Formats

Marshalling must handle:

- **Byte order**: Big-endian vs little-endian (network byte order is big-endian by convention)
- **Alignment and padding**: Different architectures have different alignment requirements
- **Pointer translation**: Pointers are meaningless across address spaces; pointer-based structures must be serialised into flat representations
- **Type safety**: The receiver must be able to reconstruct the original typed data

Common wire formats:

Format	Type	Size	Speed	Human-readable
Protocol Buffers	Binary	Compact	Fast	No
FlatBuffers	Binary (zero-copy)	Compact	Fastest	No
JSON	Text	Verbose	Slow	Yes
MessagePack	Binary	Compact	Fast	No
XML	Text	Very verbose	Slow	Yes
Cap’n Proto	Binary (zero-copy)	Compact	Fastest	No

6.8.3 gRPC

gRPC (Google Remote Procedure Call) is the dominant modern RPC framework. It uses HTTP/2 as its transport and Protocol Buffers (protobuf) as its IDL and serialisa-

tion format.

Definition 6.15 (gRPC). gRPC is an open-source RPC framework that uses:

- **Protocol Buffers (protobuf)**: For defining service interfaces and message types
- **HTTP/2**: For transport, providing multiplexed streams, header compression, and bidirectional streaming
- **Code generation**: The `protoc` compiler generates client and server stubs in 12+ languages

gRPC supports four communication patterns: 1. **Unary RPC**: Single request, single response 2. **Server streaming**: Single request, stream of responses 3. **Client streaming**: Stream of requests, single response 4. **Bidirectional streaming**: Stream of requests and responses simultaneously

Example 6.4 (gRPC Service Definition and Go Implementation). A simple calculator service:

```
// calculator.proto
syntax = "proto3";
package calculator;

option go_package = "calculator/pb";

service Calculator {
    rpc Add (AddRequest) returns (AddResponse);
    rpc SumStream (stream Number) returns (SumResponse);
}

message AddRequest {
    double a = 1;
    double b = 2;
}

message AddResponse {
    double result = 1;
}

message Number {
    double value = 1;
}
```

```
message SumResponse {
    double total = 1;
    int32 count = 2;
}
```

Server implementation in Go:

```go
package main

import (
    "context"
    "fmt"
    "io"
    "log"
    "net"

    pb "calculator/pb"

    "google.golang.org/grpc"
)

type server struct {
    pb.UnimplementedCalculatorServer
}

func (s *server) Add(ctx context.Context,
    req *pb.AddRequest) (*pb.AddResponse, error) {
    result := req.A + req.B
    return &pb.AddResponse{Result: result}, nil
}

func (s *server) SumStream(
    stream pb.Calculator_SumStreamServer) error {
    var total float64
    var count int32

    for {
        num, err := stream.Recv()
        if err == io.EOF {
            return stream.SendAndClose(&pb.SumResponse{
                Total: total,
```

```
                Count: count,
            })
        }
        if err != nil {
            return err
        }
        total += num.Value
        count++
    }
}

func main() {
    lis, err := net.Listen("tcp", ":50051")
    if err != nil {
        log.Fatalf("failed to listen: %v", err)
    }

    s := grpc.NewServer()
    pb.RegisterCalculatorServer(s, &server{})

    fmt.Println("gRPC server listening on :50051")
    if err := s.Serve(lis); err != nil {
        log.Fatalf("failed to serve: %v", err)
    }
}
```

6.8.4 RPC Semantics

RPC must handle network failures, which do not exist in local function calls. The semantics of how failures are handled define three categories:

Definition 6.16 (RPC Failure Semantics).

- **At-most-once**: The server executes the RPC at most one time. If the client does not receive a response (due to network failure), it does not know whether the server executed the call. This is the default in most RPC systems.
- **At-least-once**: The client retries the RPC until it receives a response. The server may execute the call multiple times. Safe only for **idempotent** operations (operations that produce the same result regardless of how many times they are executed, e.g., `SET key=value`).
- **Exactly-once**: The server executes the RPC exactly once, regardless of

network failures. This requires persistent state on both client and server (transaction IDs, duplicate detection). Extremely difficult to implement in general; approximated by systems like Google Spanner.

Theorem 6.3 (Impossibility of Transparent RPC). It is impossible to make remote procedure calls completely transparent (indistinguishable from local calls) due to the fundamental differences between local and remote execution:

1. **Partial failure**: A remote call can fail in ways a local call cannot (network partition, server crash). The caller must handle these failures explicitly.
2. **Latency**: Remote calls are orders of magnitude slower than local calls ($~$1 ms vs $~$10 ns). Treating them as local calls leads to severe performance problems.
3. **Memory access**: Remote calls cannot pass pointers (they are meaningless in the remote address space). All arguments must be serialised.

This was formalised by Waldo et al. in "A Note on Distributed Computing" (1994), arguing that distributed objects should not be treated as local objects.

6.9 Comparing IPC Mechanisms

The choice of IPC mechanism depends on the specific requirements:

Example 6.5 (IPC Selection Guide).

High throughput, low latency, same machine: Use shared memory with lock-free data structures. Typical for high-frequency trading systems, real-time audio processing, and database engines (e.g., PostgreSQL shared buffers).

Structured communication between services: Use gRPC with Protocol Buffers. Provides type safety, code generation, streaming, and works across machines. Typical for microservice architectures.

Shell-style process pipelines: Use pipes. Simple, composable, and zero-configuration. The Unix philosophy of small tools connected by pipes.

Desktop application integration: Use D-Bus. Service discovery, security policies, and a well-established ecosystem of services on Linux desktops.

Process supervision and file descriptor passing: Use Unix domain sockets with `SCM_RIGHTS`. Used by systemd, container runtimes, and web server process managers.

Simple notifications: Use signals. No data transfer beyond the signal number (or a small payload with real-time signals). Used for process lifecycle management (SIGCHLD, SIGTERM).

The following table provides a quantitative comparison on a typical Linux system (Intel Core i7, kernel 6.x):

Mechanism	Latency (64B msg)	Throughput (64B msgs/s)	Setup Complexity	Cross-Machine
Shared memory + atomic	~50 ns	~20,000,000	High	No
Unix domain socket	~5 us	~200,000	Low	No
Pipe	~3 us	~300,000	Very low	No
TCP loopback	~12 us	~85,000	Low	Yes
POSIX message queue	~4 us	~250,000	Medium	No
Signal	~2 us	~500,000	Low	No (same machine)
gRPC (loopback)	~100 us	~10,000	Medium	Yes
D-Bus	~200 us	~5,000	High	No

Programmer's Perspective

Programmer's Perspective: Go Channels as CSP-Style IPC. Go channels implement Communicating Sequential Processes (CSP), Tony Hoare's formal model of concurrent computation. In CSP, processes communicate by sending and receiving values on named channels; there is no shared state. Go's channels provide exactly this abstraction:

```go
package main

import (
    "fmt"
    "sync"
)

func producer(ch chan<- int, id int, wg *sync.WaitGroup) {
```

```go
    defer wg.Done()
    for i := 0; i < 5; i++ {
        ch <- id*100 + i
    }
}

func consumer(ch <-chan int, done chan<- bool) {
    for val := range ch {
        fmt.Printf("Consumed: %d\n", val)
    }
    done <- true
}

func main() {
    ch := make(chan int, 10)  // Buffered channel (async)
    done := make(chan bool)
    var wg sync.WaitGroup

    // Launch 3 producers
    for i := 0; i < 3; i++ {
        wg.Add(1)
        go producer(ch, i, &wg)
    }

    // Launch 1 consumer
    go consumer(ch, done)

    // Wait for all producers, then close channel
    wg.Wait()
    close(ch)
    <-done
}
```

Unbuffered channels (`make(chan int)`) provide synchronous message passing: the sender blocks until the receiver is ready, creating a rendezvous. Buffered channels (`make(chan int, n)`) provide asynchronous message passing with a bounded buffer of size n. The `select` statement multiplexes across multiple channels, analogous to `epoll` for file descriptors. Channels are the preferred IPC mechanism within a Go program; for communication between separate OS processes, Go programs typically use gRPC (with

google.golang.org/grpc), Unix domain sockets (via net.Dial("unix", path)), or named pipes.

6.10 io_uring: Asynchronous I/O for Linux

Traditional IPC mechanisms rely on synchronous system calls: `read()`, `write()`, `send()`, `recv()`. Each call transitions from user mode to kernel mode and back, incurring overhead of ~100–200 ns per system call on modern hardware. For high-throughput I/O workloads, this overhead becomes significant.

Definition 6.17 (io_uring). `io_uring` (introduced in Linux 5.1, 2019) is an asynchronous I/O interface based on two ring buffers shared between user space and the kernel:

- **Submission Queue (SQ)**: User space writes I/O request descriptors (Submission Queue Entries, SQEs) to this ring. Each SQE describes an operation (read, write, send, recv, accept, etc.) and its parameters.
- **Completion Queue (CQ)**: The kernel writes completion results (Completion Queue Entries, CQEs) to this ring. Each CQE contains the result code and user data identifying which request completed.

Both rings are memory-mapped and accessible without system calls after initial setup. User space can submit multiple I/O requests and poll for completions without entering the kernel, achieving zero-syscall I/O in the best case.

```c
#include <liburing.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define QUEUE_DEPTH 32

int main(void) {
    struct io_uring ring;

    /* Initialise io_uring with queue depth 32 */
    if (io_uring_queue_init(QUEUE_DEPTH, &ring, 0) < 0) {
        perror("io_uring_queue_init");
        return 1;
    }
```

```
    /* Open a file */
    int fd = open("/tmp/uring_test.txt",
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);

    /* Prepare a write request */
    const char *data = "Hello from io_uring!\n";
    struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
    io_uring_prep_write(sqe, fd, data, strlen(data), 0);
    io_uring_sqe_set_data(sqe, (void *)1);  /* User data for identification */

    /* Submit the request (single syscall for potentially many operations) */
    io_uring_submit(&ring);

    /* Wait for completion */
    struct io_uring_cqe *cqe;
    io_uring_wait_cqe(&ring, &cqe);

    if (cqe->res < 0) {
        fprintf(stderr, "Write failed: %s\n", strerror(-cqe->res));
    } else {
        printf("Wrote %d bytes via io_uring\n", cqe->res);
    }

    io_uring_cqe_seen(&ring, cqe);

    close(fd);
    io_uring_queue_exit(&ring);
    return 0;
}
```

Compile with: `gcc -o uring_example uring_example.c -luring`

6.10.1 io_uring Performance

The key advantage of `io_uring` is **batching and zero-copy submission**. Rather than making one system call per I/O operation, the application fills multiple SQEs in the submission ring and submits them with a single `io_uring_enter()` call (or no system call at all, when using `IORING_SETUP_SQPOLL` mode with a kernel polling thread).

Example 6.6 (io_uring vs epoll for Network I/O). Benchmarks by the io_uring author (Jens Axboe) show:

Workload	epoll (ops/sec)	io_uring (ops/sec)	Improvement
Random 4K reads (NVMe)	~200,000	~1,600,000	8x
TCP echo (small messages)	~350,000	~500,000	1.4x
TCP accept storm	~100,000	~400,000	4x

The greatest gains come from storage I/O, where `io_uring` can drive NVMe devices to their full IOPS capacity by eliminating per-operation system call overhead. For network I/O, the gains are more modest but still significant at high connection counts.

Programmer's Perspective

Programmer's Perspective: io_uring in Go and System Design. Go's runtime uses `epoll` (Linux) or `kqueue` (macOS/BSD) for its internal network poller, which underpins all goroutine-based network I/O. As of Go 1.22, the runtime does not use `io_uring` for its built-in I/O, though proposals exist to add it. Third-party libraries like `github.com/iceber/iouring-go` and `github.com/pawelgaczynski/gain` provide Go wrappers around `io_uring` for applications that need maximum I/O throughput.

> *The design principle behind `io_uring` — **shared-memory ring buffers between user space and kernel** — is the same principle used in high-performance networking (DPDK, XDP/AF_XDP) and GPU command submission (DRM/KMS). The pattern eliminates the two most expensive operations in I/O: system call overhead (mode switch) and data copying (user-to-kernel buffer copy). When designing high-performance systems, consider whether your I/O pattern can benefit from batched, asynchronous submission via `io_uring` rather than synchronous `read()`/`write()` calls.*

6.11 IPC Design Patterns

Several recurring patterns appear in IPC-based system design:

6.11.1 Producer-Consumer

The producer-consumer pattern decouples data generation from data processing. One or more producer processes write to a shared buffer (pipe, message queue, shared memory ring buffer), and one or more consumer processes read from it.

Definition 6.18 (Producer-Consumer Pattern). In the producer-consumer pattern, producers generate data items and place them in a bounded buffer of capacity N. Consumers remove data items from the buffer and process them. The buffer provides flow control: producers block when the buffer is full, and consumers block when the buffer is empty. This pattern decouples the rate of production from the rate of consumption.

The key synchronisation requirements are:

1. **Mutual exclusion**: At most one process accesses the buffer at a time (if using shared memory)
2. **Full buffer**: Producers must wait when the buffer has N items
3. **Empty buffer**: Consumers must wait when the buffer has 0 items

6.11.2 Client-Server

The client-server pattern uses a well-known endpoint (a named pipe, a Unix socket path, or a network address) where the server listens for requests. Clients connect, send requests, and receive responses.

6.11.3 Publish-Subscribe

In publish-subscribe, publishers emit events without knowing which processes will receive them. Subscribers register interest in specific event types. A message broker (e.g., D-Bus, Redis pub/sub, NATS) routes events from publishers to matching subscribers.

6.12 Security Considerations

IPC mechanisms have security implications that system designers must consider:

Permission control:

- POSIX shared memory objects: Controlled by filesystem-like permissions (owner, group, other)
- Unix domain sockets: The socket file has filesystem permissions; additionally, `SO_PEERCRED` reveals the peer's PID, UID, and GID
- Named pipes: Filesystem permissions on the FIFO special file
- Signals: Only processes with the same UID (or root) can send signals to each other

Data confidentiality:

- Shared memory and pipes transfer data in plaintext within the kernel

- For cross-machine IPC (TCP sockets, gRPC), TLS encryption should be used
- D-Bus supports Unix credentials passing and AppArmor/SELinux integration for access control

Theorem 6.4 (IPC Isolation Boundary). In a correctly configured operating system, IPC is the only mechanism by which processes in different address spaces can exchange data. Specifically, if processes A and B have disjoint virtual address spaces and neither has access to `/proc/<pid>/mem` or `ptrace()` capabilities for the other, then all data exchange between A and B must occur through kernel-mediated IPC mechanisms. This is a consequence of the virtual memory isolation guarantee provided by the MMU and the kernel's page table management.

Exercises

1. **Exercise 6.1.** Write a C program that implements a producer-consumer system using POSIX shared memory and POSIX semaphores. The shared memory region should contain a circular buffer of 16 integer slots. The producer writes the integers 1 through 100 to the buffer; the consumer reads and prints them. Use two semaphores (`empty` and `full`) and a mutex to synchronise access. Verify that all 100 integers are received in order.

2. **Exercise 6.2.** Compare the latency of three IPC mechanisms by writing a ping-pong benchmark in C. Two processes alternate sending a single byte to each other for 100,000 round trips. Implement the benchmark using: (a) an anonymous pipe pair, (b) a Unix domain socket pair (SOCK_STREAM), and (c) TCP sockets on localhost (127.0.0.1). Measure the average round-trip time for each and explain the performance differences in terms of the kernel code path each mechanism traverses.

3. **Exercise 6.3.** The `PIPE_BUF` atomicity guarantee ensures that writes of up to 4096 bytes to a pipe are atomic. Design an experiment to demonstrate what happens when multiple processes simultaneously write messages larger than `PIPE_BUF` to the same pipe. Write a C program where 4 child processes each write a 8192-byte message (filled with a distinct character, e.g., 'A', 'B', 'C', 'D') to the same pipe. The parent reads from the pipe and checks whether the messages are interleaved. Report your findings and explain why `PIPE_BUF` atomicity matters for logging systems that use pipes.

4. **Exercise 6.4.** Implement a simple RPC system in Go without using any RPC framework. Define a calculator service with `Add(a, b float64) float64` and `Multiply(a, b float64) float64` methods. The client sends requests as JSON over a Unix domain socket; the server parses the request, dispatches to the

appropriate method, and returns the result as JSON. Handle errors (unknown method, malformed request) gracefully. Then compare the latency of your hand-written RPC with Go's `net/rpc` package and `google.golang.org/grpc` for the same operations.

5. **Exercise 6.5.** Signals are sometimes called "software interrupts." Explain the analogy by comparing: (a) how hardware interrupts are delivered to the CPU, (b) how signals are delivered to a process, (c) the mechanisms for masking/blocking both, and (d) the restrictions on what code can execute in an interrupt handler vs a signal handler. Write a C program that demonstrates signal masking: block `SIGINT`, send `SIGINT` to the process 5 times, then unblock `SIGINT` and observe how many times the signal handler is invoked. Repeat with a real-time signal and observe the difference.

6. **Exercise 6.6.** Write a Go program that demonstrates the performance difference between unbuffered and buffered channels. Create a pipeline of 4 stages (goroutines), where each stage reads an integer from its input channel, increments it, and sends it to its output channel. Run 1,000,000 values through the pipeline. Measure the throughput with: (a) unbuffered channels between all stages, (b) buffered channels with capacity 1, (c) buffered channels with capacity 100, and (d) buffered channels with capacity 10,000. Plot throughput vs buffer size and explain the results in terms of goroutine scheduling and synchronisation overhead.

7. **Exercise 6.7.** Unix domain sockets can transfer file descriptors via `SCM_RIGHTS` ancillary messages. Write a C program with two processes: a "file server" that opens a file and sends the file descriptor to a "client" process over a Unix domain socket. The client reads from the received file descriptor and prints the contents. Verify that the client can read the file even if it does not have filesystem permission to open the file directly (the server must have the permission). Explain the security implications of file descriptor passing and why it is used by container runtimes and process managers.

Chapter 7: Synchronisation Primitives

"Shared mutable state is the root of all evil." — attributed to various, but universally felt

---

When multiple threads or processes access shared resources, the absence of coordination leads to race conditions — subtle, intermittent bugs that corrupt data, crash systems, and resist debugging. Synchronisation primitives are the tools we use to impose order on concurrent execution. They range from simple hardware instructions that atomically modify a single word of memory, through software abstractions such as mutexes and semaphores, to high-level constructs like monitors and condition variables.

This chapter builds the theory from the ground up. We begin with the formal statement of the critical section problem and its correctness requirements. We then examine algorithmic solutions (Peterson's algorithm), hardware-level atomic instructions, and the rich hierarchy of synchronisation abstractions that modern operating systems and language runtimes provide.

7.1 The Critical Section Problem

7.1.1 Motivation

Consider two threads, each incrementing a shared counter:

```
/* Shared variable */
int counter = 0;

/* Thread 1 */
void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        counter++;  /* NOT atomic */
    }
```

```
    return NULL;
}

/* Thread 2 runs the same function */
```

On a typical machine, the increment `counter++` compiles to three instructions: load the value from memory into a register, add one, and store the result back. If two threads execute these instructions concurrently, their operations can interleave:

```
Thread 1: LOAD counter  (reads 42)
Thread 2: LOAD counter  (reads 42)
Thread 1: ADD 1         (register = 43)
Thread 2: ADD 1         (register = 43)
Thread 1: STORE counter (writes 43)
Thread 2: STORE counter (writes 43)
```

Both threads performed an increment, but the counter advanced by only one. This is a **race condition**: the program's correctness depends on the relative timing of thread execution.

Definition 7.1 (Race Condition). A race condition occurs when the outcome of a computation depends on the non-deterministic interleaving of operations by concurrent threads or processes, and at least one interleaving produces an incorrect result.

Definition 7.2 (Critical Section). A critical section is a segment of code that accesses a shared resource (variable, data structure, file, device) and must not be executed concurrently by more than one thread. The structure of a process participating in a critical section protocol is:

```
Entry section      -- acquire permission to enter
Critical section   -- access shared resource
Exit section       -- release permission
Remainder section  -- code that does not access shared resources
```

7.1.2 Correctness Requirements

Any solution to the critical section problem must satisfy three properties:

Definition 7.3 (Critical Section Requirements).

1. **Mutual exclusion.** If process P_i is executing in its critical section, no other process P_j ($j \neq i$) may be executing in its critical section simultaneously.
2. **Progress.** If no process is executing in its critical section and some processes wish to enter, the selection of which process enters next cannot be postponed

indefinitely. Only processes not in their remainder sections may participate in the decision, and the decision must be made in a finite number of steps.

3. **Bounded waiting.** There exists a bound B such that, after a process P_i has requested entry to its critical section, at most B other processes are allowed to enter their critical sections before P_i is granted entry. This prevents starvation.

The progress requirement is subtle. It forbids solutions where all processes are stuck waiting even though the critical section is free. It also prevents solutions that delegate the decision to processes executing in their remainder section — those processes may never execute again.

7.1.3 The Difficulty of the Problem

The critical section problem may seem trivial at first glance, but it is surprisingly hard to solve correctly without hardware support. The fundamental difficulty is that ordinary read and write operations are not atomic at the system level: between the time a thread reads a variable and acts on the value it read, another thread may have changed that variable. This observation — that checking a condition and acting on it are two separate steps, susceptible to interleaving — is the essence of the **time-of-check to time-of-use (TOCTOU)** problem.

Definition 7.3b (TOCTOU Vulnerability). A time-of-check to time-of-use (TOCTOU) vulnerability exists when a program checks a condition and then takes action based on the result of that check, but the condition may have changed between the check and the action due to concurrent modification.

Every naive approach to the critical section problem falls prey to TOCTOU. The challenge is to design protocols where the check and the action are effectively atomic — either through careful algorithmic design (Peterson's algorithm) or through hardware atomic instructions (CAS, test-and-set).

7.1.4 Naive Attempts

Attempt 1: Alternation. A simple approach uses a shared turn variable:

```
int turn = 0;  /* shared */

/* Process 0 */
while (turn != 0) { /* busy wait */ }
/* critical section */
turn = 1;
```

```
/* Process 1 */
while (turn != 1) { /* busy wait */ }
/* critical section */
turn = 0;
```

This satisfies mutual exclusion (only the process whose turn it is can enter) but violates **progress**. If process 0 executes its critical section and sets `turn = 1`, then enters a long remainder section, process 1 can enter and exit, but now `turn = 0` and process 1 cannot re-enter even though the critical section is free. Process 1 depends on process 0 to take its turn.

Attempt 2: Flag array. Each process sets a flag indicating its desire to enter:

```
int flag[2] = {0, 0};  /* shared */

/* Process i */
flag[i] = 1;
while (flag[1 - i]) { /* busy wait */ }
/* critical section */
flag[i] = 0;
```

This violates **mutual exclusion** under certain interleavings. If both processes set their flags to 1 before either checks the other's flag, both will wait forever — a livelock. Alternatively, if we check before setting, both can enter simultaneously.

These failures motivate more careful solutions.

7.1.5 The N-Process Generalisation

For $N > 2$ processes, the critical section problem becomes significantly harder. The **filter lock** (also called the **generalised Peterson's algorithm**) uses $N - 1$ levels, where each level eliminates at least one contending process:

```
#define N 8  /* number of processes */
int level[N];  /* level[i] = current level of process i */
int victim[N]; /* victim[k] = process that yields at level k */

void filter_lock(int i) {
    for (int k = 1; k < N; k++) {
        level[i] = k;
        victim[k] = i;
        /* Wait while there exists another process at level >= k
           and I am the victim at level k */
        while (victim[k] == i) {
```

```
            int found = 0;
            for (int j = 0; j < N; j++) {
                if (j != i && level[j] >= k) {
                    found = 1;
                    break;
                }
            }
            if (!found) break;
        }
    }
}

void filter_unlock(int i) {
    level[i] = 0;
}
```

At each level k, at most $N - k$ processes can pass through. At level $N - 1$, at most one process remains. The filter lock satisfies mutual exclusion and freedom from starvation, but its space complexity is $O(N)$ shared variables, and a process must traverse $O(N)$ levels — making it impractical for large N.

Theorem 7.0b (Lower Bound on Shared Variables). Any solution to the N-process mutual exclusion problem using only read-write registers requires at least N shared variables (Burns and Lynch, 1993).

This lower bound explains why practical synchronisation relies on hardware atomic instructions rather than software algorithms: hardware provides the "bigger hammer" needed to solve the problem efficiently.

7.2 Peterson's Algorithm

7.2.1 The Algorithm

Peterson's algorithm (1981) is the simplest correct software-only solution for two processes under the sequential consistency memory model.

```
int flag[2] = {0, 0};  /* desire to enter */
int turn;               /* whose turn to yield */

/* Process i (i = 0 or 1) */
void enter_critical_section(int i) {
```

```
    int j = 1 - i;
    flag[i] = 1;        /* I want to enter */
    turn = j;            /* but I yield to you */
    while (flag[j] && turn == j) {
        /* busy wait */
    }
}

void exit_critical_section(int i) {
    flag[i] = 0;
}
```

The key insight is the `turn` variable: when both processes want to enter, the one that set `turn` last (the more "polite" one) yields. This combines the flag approach with the turn approach, fixing the defects of both.

7.2.2 Correctness Proof

Theorem 7.1 (Peterson's Algorithm is Correct). Peterson's algorithm satisfies mutual exclusion, progress, and bounded waiting for two processes, assuming sequential consistency.

Proof. We prove each property separately.

Mutual exclusion. Suppose, for contradiction, that both P_0 and P_1 are in their critical sections simultaneously. Then both passed their while-loop conditions: `flag[j] && turn == j` evaluated to false for both $i = 0$ and $i = 1$.

For P_0: either $\texttt{flag}[1] = 0$ or $\texttt{turn} = 0$. For P_1: either $\texttt{flag}[0] = 0$ or $\texttt{turn} = 1$.

Since both are in the critical section, both set their flags to 1 before entering (and flags are only cleared in the exit section). So $\texttt{flag}[0] = 1$ and $\texttt{flag}[1] = 1$. Therefore:

- P_0 entered because $\texttt{turn} = 0$
- P_1 entered because $\texttt{turn} = 1$

But `turn` is a single variable that cannot simultaneously hold both values. Contradiction. □

Progress. Suppose the critical section is free and at least one process wishes to enter. If only one process (say P_i) has $\texttt{flag}[i] = 1$, then $\texttt{flag}[j] = 0$, and P_i's while condition is immediately false — it enters. If both processes set their flags, one of them wrote to `turn` last, and that process yields to the other. The other enters immediately. In either case, the decision is made without involving processes in their remainder sections. □

Bounded waiting. After P_i sets `flag`$[i] = 1$ and `turn` $= j$, if P_j is in its critical section, P_j will eventually exit and set `flag`$[j] = 0$, allowing P_i to enter. If P_j then re-enters the entry section, it sets `turn` $= i$, which allows P_i to proceed. Thus at most one entry by P_j can occur before P_i enters. The bound is $B = 1$. □

7.2.3 Limitations

Peterson's algorithm assumes **sequential consistency**: all memory operations appear in some global total order consistent with each thread's program order. Modern processors with store buffers, write-combining buffers, and relaxed memory models may reorder the writes to `flag[i]` and `turn`, breaking the algorithm.

Example 7.1 (Reordering Breaks Peterson's). On an x86 processor, stores to different addresses may become visible in different orders to different cores (the Total Store Order model permits store-load reordering). Consider:

```
Process 0:                    Process 1:
  flag[0] = 1                   flag[1] = 1
  turn = 1                      turn = 0
  read flag[1], read turn       read flag[0], read turn
```

Under TSO, the store to `flag[0]` may sit in process 0's store buffer while process 0 reads `flag[1]` from main memory (which still holds 0). Similarly for process 1. Both see the other's flag as 0 and both enter. Memory fences are required to prevent this.

Programmer's Perspective

On modern hardware, Peterson's algorithm requires explicit memory barriers to work correctly. On x86 (TSO), a `mfence` instruction between the writes and the reads suffices. On ARM or RISC-V (weaker memory models), both store-release and load-acquire barriers are needed. In practice, nobody implements Peterson's algorithm on real hardware — atomic instructions (Section 7.3) are both simpler and faster. Peterson's algorithm remains essential as a teaching tool and as the foundation for understanding why hardware support is necessary.

7.3 Hardware Support for Synchronisation

Software-only solutions like Peterson's algorithm are fragile under relaxed memory models and do not generalise efficiently to N processes. Modern processors provide

atomic read-modify-write instructions that serve as the building blocks for all practical synchronisation primitives.

7.3.1 Test-and-Set

Definition 7.4 (Test-and-Set). The test-and-set instruction atomically reads a memory location, returns its old value, and writes 1 (or true) to that location:

$$\text{TestAndSet}(\texttt{lock}) : \begin{cases} \texttt{old} \leftarrow \texttt{*lock} \\ \texttt{*lock} \leftarrow 1 \\ \text{return } \texttt{old} \end{cases}$$

The entire operation is indivisible: no other processor can observe a state where the read has completed but the write has not.

A simple spinlock using test-and-set:

```
#include <stdatomic.h>

typedef atomic_int spinlock_t;

void spin_lock(spinlock_t *lock) {
    while (atomic_exchange(lock, 1) == 1) {
        /* spin */
    }
}

void spin_unlock(spinlock_t *lock) {
    atomic_store(lock, 0);
}
```

This satisfies mutual exclusion: only the thread that observes `old = 0` from the exchange enters the critical section. However, it does not guarantee bounded waiting — a thread could spin indefinitely while other threads repeatedly acquire and release the lock.

7.3.1b Test-and-Test-and-Set (TTAS)

The simple test-and-set spinlock has a severe performance problem on multicore systems. Each `atomic_exchange` generates a write to the lock variable, which invalidates the cache line on all other cores. With N cores spinning, every iteration generates $N - 1$ cache invalidations, creating an $O(N^2)$ bus traffic explosion.

The **test-and-test-and-set (TTAS)** optimisation reduces bus traffic by reading the lock (a non-invalidating cache hit) before attempting the expensive atomic exchange:

```
void ttas_lock(spinlock_t *lock) {
    while (1) {
        /* Test: spin on cached copy (read-only, no bus traffic) */
        while (atomic_load_explicit(lock, memory_order_relaxed) == 1) {
            /* spin locally on cached copy */
        }
        /* Test-and-set: try to acquire */
        if (atomic_exchange(lock, 1) == 0) {
            return;  /* acquired */
        }
        /* Failed: another thread got it first. Back to spinning. */
    }
}
```

The key insight is that while the lock is held, all spinning threads read their local cached copy of the lock variable, generating no bus traffic. Only when the lock is released (and cache lines are invalidated) do the spinners attempt the expensive exchange. This reduces steady-state bus traffic from $O(N)$ per spin iteration to $O(1)$.

Example 7.1b (TTAS Cache Behaviour). Consider 4 cores spinning on a lock held by Core 0:

```
TAS (simple):
  Every iteration: Core 1 writes lock -> invalidates Core 2, 3 cache
                   Core 2 writes lock -> invalidates Core 1, 3 cache
                   Core 3 writes lock -> invalidates Core 1, 2 cache
  Bus traffic per iteration: O(N)

TTAS:
  While lock held: Cores 1, 2, 3 read local cache -> 0 bus transactions
  On release:      Core 0 writes lock = 0 -> invalidates all caches
                   Cores 1, 2, 3 read lock (cache miss) -> 3 bus reads
                   Cores 1, 2, 3 attempt exchange -> 3 bus writes
  Total bus traffic per acquisition: O(N) instead of O(N * hold_time)
```

TTAS with exponential backoff further reduces the “thundering herd” effect: after a failed exchange, each thread waits a random exponentially increasing delay before retrying.

7.3.2 Compare-and-Swap (CAS)

Definition 7.5 (Compare-and-Swap). The compare-and-swap (CAS) instruction atomically compares the value at a memory location with an expected value and, if they match, replaces it with a new value:

$$\text{CAS}(\texttt{addr}, \texttt{expected}, \texttt{new}) : \begin{cases} \texttt{old} \leftarrow \texttt{*addr} \\ \text{if } \texttt{old} = \texttt{expected} : \texttt{*addr} \leftarrow \texttt{new} \\ \text{return } \texttt{old} \end{cases}$$

The return value (or a boolean success indicator) tells the caller whether the swap occurred.

CAS is strictly more powerful than test-and-set. It is the foundation of lock-free programming (Chapter 9). On x86, it is implemented by the `LOCK CMPXCHG` instruction; on ARM, by the `LDXR/STXR` (load-exclusive/store-exclusive) pair.

```c
#include <stdatomic.h>

void cas_lock(atomic_int *lock) {
    int expected = 0;
    while (!atomic_compare_exchange_weak(lock, &expected, 1)) {
        expected = 0;  /* reset expected after failure */
    }
}

void cas_unlock(atomic_int *lock) {
    atomic_store(lock, 0);
}
```

Theorem 7.2 (CAS Consensus Number). The compare-and-swap instruction has an infinite consensus number: it can solve the consensus problem for any number of threads. This means CAS can implement any concurrent data structure in a wait-free manner (via Herlihy's universal construction).

Proof sketch. Given n threads, each with a proposed value, allocate a shared variable initialised to $\perp$ (null). Each thread attempts $\text{CAS}(\texttt{decision}, \perp, \texttt{my_value})$. Exactly one thread succeeds (the first to execute CAS), and all threads can read the decided value. Since no thread blocks and the operation completes in a single CAS per thread, this is wait-free. □

7.3.2b The Consensus Hierarchy

Herlihy (1991) classified synchronisation primitives by their **consensus number** — the maximum number of threads for which the primitive can solve the consensus problem. This classification reveals a strict hierarchy of synchronisation power:

Definition 7.5b (Consensus Number). The consensus number of a synchronisation primitive is the maximum number of threads n for which the primitive can solve the **consensus problem**: given n threads, each with a proposed value, all threads must agree on one of the proposed values (agreement), the decided value must have been proposed by some thread (validity), and every thread must decide in a finite number of steps (termination).

Primitive	Consensus Number	Examples
Atomic read/write registers	1	Ordinary variables
Test-and-set, swap, fetch-and-add	2	`XCHG`, `XADD`
Compare-and-swap, LL/SC	∞	`CMPXCHG`, `LDXR/STXR`

Theorem 7.2b (Consensus Hierarchy Separation). No synchronisation primitive with consensus number k can solve consensus for $k+1$ threads. In particular, read-write registers cannot solve consensus for 2 threads (the FLP impossibility result for asynchronous systems), and test-and-set cannot solve consensus for 3 threads.

This hierarchy has profound practical implications. Since CAS has an infinite consensus number, it can implement **any** concurrent object for **any** number of threads. This is why CAS is the fundamental building block of all modern lock-free data structures (Chapter 9). Test-and-set, despite being simpler and cheaper, cannot implement lock-free structures for more than 2 threads.

7.3.3 Fetch-and-Add

Definition 7.6 (Fetch-and-Add). The fetch-and-add instruction atomically reads a memory location, adds a value to it, and returns the old value:

$$\text{FetchAndAdd}(\texttt{addr}, \texttt{val}) : \begin{cases} \texttt{old} \leftarrow \texttt{*addr} \\ \texttt{*addr} \leftarrow \texttt{old} + \texttt{val} \\ \text{return } \texttt{old} \end{cases}$$

Fetch-and-add enables the construction of **ticket locks**, which provide fairness (bounded waiting):

```
#include <stdatomic.h>

typedef struct {
    atomic_int ticket;
    atomic_int serving;
} ticket_lock_t;

void ticket_lock(ticket_lock_t *lock) {
    int my_ticket = atomic_fetch_add(&lock->ticket, 1);
    while (atomic_load(&lock->serving) != my_ticket) {
        /* spin */
    }
}

void ticket_unlock(ticket_lock_t *lock) {
    atomic_fetch_add(&lock->serving, 1);
}
```

Each arriving thread takes a unique ticket number. The lock holder increments `serving` on release, granting entry to the next ticket holder in FIFO order. This is a direct analogue of the numbering system at a bakery counter.

Theorem 7.3 (Ticket Lock Bounded Waiting). The ticket lock satisfies mutual exclusion, progress, and bounded waiting with bound $B = N - 1$, where N is the number of contending threads.

Proof. Mutual exclusion: two threads hold the lock simultaneously only if `serving` equals both their ticket numbers, which is impossible since `fetch_add` assigns distinct tickets. Progress: when the lock is released, `serving` is incremented, and the thread holding the next ticket enters. Bounded waiting: a thread with ticket k waits for at most $N-1$ other threads (those with tickets $k-N+1, \ldots, k-1$ modulo wrap-around) to be served. □

Programmer's Perspective

The Linux kernel uses a variant of the ticket lock called an **MCS lock** (Mellor-Crummey and Scott, 1991) for its spinlock implementation. The problem with basic ticket locks is cache contention: all spinning threads repeatedly read the same `serving` variable, causing a cache-line ping-pong storm on multi-socket systems. MCS locks solve this by having each thread spin on its own local cache line. When you call `spin_lock()` in the Linux kernel (on x86), the implementation uses `LOCK XADD` (fetch-and-add) for the initial ticket acquisition and queued spinning for the wait phase.

7.3.4 LL/SC: Load-Linked and Store-Conditional

ARM, RISC-V, MIPS, and PowerPC architectures do not provide CAS directly. Instead, they offer a pair of instructions:

- **Load-Linked (LL) / Load-Exclusive (LDXR)**: reads a memory location and sets a hardware reservation on that cache line.
- **Store-Conditional (SC) / Store-Exclusive (STXR)**: writes to the location only if the reservation is still valid (no other core has written to the same cache line since the LL). Returns success or failure.

```
/* ARM64 CAS emulation using LDXR/STXR (pseudocode) */
int cas_arm64(int *addr, int expected, int new_val) {
    int old;
    int success;
    do {
        old = __LDXR(addr);      /* load-exclusive */
        if (old != expected) {
            __CLREX();            /* clear exclusive monitor */
            return 0;             /* CAS failed */
        }
        success = __STXR(addr, new_val);  /* store-exclusive */
    } while (!success);          /* retry if reservation lost */
    return 1;                    /* CAS succeeded */
}
```

The LL/SC pair is immune to the ABA problem (Chapter 9) because the reservation is invalidated by any write to the cache line, not just by a value change. However, spurious failures are possible (the SC may fail even without interference from other threads), which is why the retry loop is necessary.

7.4 Mutexes

7.4.1 Definition and Semantics

Definition 7.7 (Mutex). A mutex (mutual exclusion lock) is a synchronisation object with two operations:

- `lock`(m): acquires exclusive ownership of m. If m is already held by another thread, the calling thread blocks (is suspended) until m becomes available.
- `unlock`(m): releases ownership of m. If other threads are blocked waiting for m, one of them is woken and granted ownership.

A mutex has a single owner at any time. Only the owner may call `unlock`.

The fundamental difference between a mutex and a spinlock is the **blocking** behaviour. When a spinlock is contended, the waiting thread burns CPU cycles in a busy-wait loop. When a mutex is contended, the waiting thread is placed on a queue and descheduled by the operating system, allowing other threads to use the CPU.

7.4.2 Implementation

A typical mutex implementation combines an atomic state variable with a wait queue managed by the kernel:

```c
#include <stdatomic.h>
#include <linux/futex.h>  /* Linux-specific */

typedef struct {
    atomic_int state;  /* 0 = unlocked, 1 = locked (no waiters),
                          2 = locked (waiters present) */
} mutex_t;

void mutex_lock(mutex_t *m) {
    int c;
    /* Fast path: try to acquire uncontended lock */
    c = 0;
    if (atomic_compare_exchange_strong(&m->state, &c, 1))
        return;

    /* Slow path: lock is contended */
```

```
    do {
        /* If state was already 2, or we successfully set it to 2 */
        if (c == 2 || atomic_compare_exchange_strong(&m->state, &c, 2)) {
            /* Sleep until woken by unlock */
            futex_wait(&m->state, 2);
        }
        c = 0;
    } while (!atomic_compare_exchange_strong(&m->state, &c, 2));
}

void mutex_unlock(mutex_t *m) {
    if (atomic_fetch_sub(&m->state, 1) != 1) {
        /* There were waiters (state was 2) */
        atomic_store(&m->state, 0);
        futex_wake(&m->state, 1);  /* wake one waiter */
    }
}
```

This three-state design (from Ulrich Drepper's "Futexes Are Tricky") ensures that the common uncontended case requires only a single CAS in userspace — no system call. The kernel is involved only when actual contention exists.

Example 7.2 (Mutex Performance). Consider a lock protecting a counter incremented 10 million times by 4 threads on a 4-core machine. The critical section is a single increment (a few nanoseconds). Measured performance:

Strategy	Time	Notes
No synchronisation	8 ms	Incorrect result
Spinlock	350 ms	Heavy cache contention
Mutex (futex-based)	420 ms	Overhead from context switches
Atomic increment	120 ms	No lock needed; hardware atomics

For tiny critical sections, atomic operations outperform both spinlocks and mutexes. As the critical section grows longer, mutexes become superior because they free the CPU during the wait.

7.4.3 Recursive Mutexes

Definition 7.8 (Recursive Mutex). A recursive (or reentrant) mutex permits the same thread to acquire the lock multiple times without deadlocking. The mutex maintains an ownership count; each `lock` increments the count, and each `unlock` decrements it. The lock is released only when the count reaches zero.

```
typedef struct {
    mutex_t base;          /* underlying non-recursive mutex */
    atomic_int owner;      /* thread ID of owner, or 0 */
    int recursion_count;   /* number of recursive acquisitions */
} recursive_mutex_t;

void recursive_lock(recursive_mutex_t *m) {
    int self = get_thread_id();
    if (atomic_load(&m->owner) == self) {
        m->recursion_count++;
        return;
    }
    mutex_lock(&m->base);
    atomic_store(&m->owner, self);
    m->recursion_count = 1;
}

void recursive_unlock(recursive_mutex_t *m) {
    m->recursion_count--;
    if (m->recursion_count == 0) {
        atomic_store(&m->owner, 0);
        mutex_unlock(&m->base);
    }
}
```

Recursive mutexes are controversial. Their proponents argue they simplify designs where a function that holds a lock calls another function that also requires the lock. Their detractors argue that they mask poor design: if you need recursive locking, your lock granularity or call graph is wrong.

7.4.4 The Futex Mechanism in Detail

The Linux futex (Fast Userspace muTeX) mechanism, introduced in Linux 2.6, is the foundation of all userspace synchronisation in Linux. Understanding its design illuminates the boundary between userspace and kernel synchronisation.

Definition 7.8b (Futex). A futex is a 32-bit integer in userspace memory, combined with a kernel wait queue indexed by the integer's address. The kernel provides two operations:

- `futex_wait(addr, expected)`: if `*addr == expected`, put the calling thread to sleep on the wait queue for `addr`. Otherwise, return immediately (the condition changed before we could sleep).
- `futex_wake(addr, n)`: wake up to n threads sleeping on the wait queue for `addr`.

The key invariant is that `futex_wait` atomically checks the value and sleeps. This prevents the **lost wakeup** problem: if the value changes between the check and the sleep, `futex_wait` returns immediately instead of sleeping forever.

The futex design reflects a fundamental principle: the **common case** (uncontended lock acquisition) should be as fast as possible, even if the **uncommon case** (contention) is slightly slower. In the uncontended case, a mutex acquisition is a single CAS in userspace — no system call, no kernel involvement, no context switch. The cost is approximately 10–20 nanoseconds on modern hardware. Only when contention occurs does the thread enter the kernel via the `futex` system call.

Example 7.2b (Futex Performance Breakdown).

Path	Operations	Approximate cost
Uncontended lock	1 CAS	10–20 ns
Contended lock (sleep)	1 CAS (fail) + `futex_wait` syscall + context switch	1–10 μs
Unlock with waiters	1 atomic store + `futex_wake` syscall + context switch	1–10 μs
Unlock without waiters	1 atomic store	5–10 ns

In a typical server application, 95–99% of lock acquisitions are uncontended, so the amortised cost of locking is very close to the uncontended cost of 10–20 ns. This is why futex-based mutexes are vastly superior to always-kernel approaches (like System V semaphores, which require a system call on every operation).

7.5 Semaphores

7.5.1 Definition

Edsger Dijkstra introduced semaphores in 1965 as a generalisation of the mutex concept.

Definition 7.9 (Semaphore). A semaphore S is a non-negative integer variable accessed through two atomic operations:

- `wait`(S) (historically $P(S)$, from the Dutch *probeer*, "try"):

$$\texttt{wait}(S) : \text{while } S \leq 0 \text{ do block; } S \leftarrow S - 1$$

- `signal`(S) (historically $V(S)$, from *verhoog*, "increment"):

$$\texttt{signal}(S) : S \leftarrow S + 1; \text{ wake one blocked process if any}$$

A **binary semaphore** has values restricted to $\{0, 1\}$ and behaves similarly to a mutex (but without ownership semantics). A **counting semaphore** may take any non-negative integer value.

Example 7.3 (Semaphore as Mutex). A binary semaphore initialised to 1 provides mutual exclusion:

```
sem_t mutex;
sem_init(&mutex, 0, 1);  /* initial value = 1 */

/* Thread */
sem_wait(&mutex);     /* P: decrement, block if 0 */
/* critical section */
sem_post(&mutex);     /* V: increment */
```

The key difference from a true mutex is that any thread can call `sem_post`, not just the one that called `sem_wait`. This makes semaphores more flexible but also more error-prone.

7.5.1b Semaphore Implementation

A semaphore can be implemented using a mutex and a condition variable, demonstrating how higher-level primitives are built from lower-level ones:

```
#include <pthread.h>
```

```c
typedef struct {
    int value;
    pthread_mutex_t lock;
    pthread_cond_t cond;
} sem_impl_t;

void sem_impl_init(sem_impl_t *s, int initial_value) {
    s->value = initial_value;
    pthread_mutex_init(&s->lock, NULL);
    pthread_cond_init(&s->cond, NULL);
}

void sem_impl_wait(sem_impl_t *s) {
    pthread_mutex_lock(&s->lock);
    while (s->value <= 0)
        pthread_cond_wait(&s->cond, &s->lock);
    s->value--;
    pthread_mutex_unlock(&s->lock);
}

void sem_impl_signal(sem_impl_t *s) {
    pthread_mutex_lock(&s->lock);
    s->value++;
    pthread_cond_signal(&s->cond);
    pthread_mutex_unlock(&s->lock);
}
```

The `while` loop in `sem_impl_wait` is necessary because of Mesa semantics: the signalled thread may not run immediately, and another thread could decrement the semaphore before it resumes.

7.5.2 Producer-Consumer Problem

The producer-consumer (bounded-buffer) problem is a canonical synchronisation scenario:

Definition 7.10 (Producer-Consumer Problem). Given a buffer of size N shared between producer threads (which insert items) and consumer threads (which remove items):

1. Producers must block when the buffer is full.
2. Consumers must block when the buffer is empty.

3. Access to the buffer must be mutually exclusive.

The semaphore solution uses three semaphores:

```
#include <semaphore.h>
#include <pthread.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;   /* counts empty slots */
sem_t full;    /* counts filled slots */
sem_t mutex;   /* protects buffer access */

void init(void) {
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
}

void *producer(void *arg) {
    while (1) {
        int item = produce_item();
        sem_wait(&empty);       /* wait for empty slot */
        sem_wait(&mutex);       /* enter critical section */
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        sem_post(&mutex);       /* exit critical section */
        sem_post(&full);        /* signal that a slot is filled */
    }
    return NULL;
}

void *consumer(void *arg) {
    while (1) {
        sem_wait(&full);        /* wait for filled slot */
        sem_wait(&mutex);       /* enter critical section */
        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
```

```
        sem_post(&mutex);       /* exit critical section */
        sem_post(&empty);       /* signal that a slot is freed */
        consume_item(item);
    }
    return NULL;
}
```

Theorem 7.4 (Correctness of Bounded-Buffer Solution). The three-semaphore bounded-buffer solution satisfies:

1. **Mutual exclusion**: the `mutex` semaphore ensures only one thread modifies the buffer at a time.
2. **No buffer overflow**: a producer blocks when `empty = 0` (all slots are full).
3. **No buffer underflow**: a consumer blocks when `full = 0` (all slots are empty).
4. **No deadlock**: a process that holds `mutex` never waits on `empty` or `full` (the semaphore order prevents circular waiting).

Proof. The invariant `empty` $+$ `full` $= N$ is maintained: each `sem_wait(&empty)` / `sem_post(&full)` pair preserves it, as does each `sem_wait(&full)` / `sem_post(&empty)` pair. Since both semaphores are non-negative, `full` $\leq N$ and `empty` $\leq N$. The ordering constraint — always acquire `empty` or `full` before `mutex` — prevents deadlock because no thread holds `mutex` while waiting for a counting semaphore. □

7.5.3 Readers-Writers Problem

Definition 7.11 (Readers-Writers Problem). A shared database is accessed by two classes of processes:

- **Readers**: only read the database; multiple readers can access it simultaneously.
- **Writers**: modify the database; a writer requires exclusive access (no other readers or writers).

First readers-writers solution (readers preference):

```
sem_t rw_mutex;     /* controls writer access */
sem_t mutex;        /* protects read_count */
int read_count = 0;

void init(void) {
    sem_init(&rw_mutex, 0, 1);
    sem_init(&mutex, 0, 1);
}
```

```
void *reader(void *arg) {
    sem_wait(&mutex);
    read_count++;
    if (read_count == 1)
        sem_wait(&rw_mutex);  /* first reader locks out writers */
    sem_post(&mutex);

    /* read the database */

    sem_wait(&mutex);
    read_count--;
    if (read_count == 0)
        sem_post(&rw_mutex);  /* last reader lets writers in */
    sem_post(&mutex);
    return NULL;
}

void *writer(void *arg) {
    sem_wait(&rw_mutex);
    /* write to the database */
    sem_post(&rw_mutex);
    return NULL;
}
```

This solution allows readers to starve writers: as long as at least one reader is active, new readers can enter continuously, and the writer never gets access.

Second readers-writers solution (writers preference):

To prevent writer starvation, we add a mechanism that blocks new readers when a writer is waiting:

```
sem_t rw_mutex;     /* controls writer access */
sem_t mutex;        /* protects read_count and write_count */
sem_t read_try;     /* blocks readers when writer is waiting */
int read_count = 0;
int write_count = 0;

void init(void) {
    sem_init(&rw_mutex, 0, 1);
    sem_init(&mutex, 0, 1);
    sem_init(&read_try, 0, 1);
```

```
}

void *reader(void *arg) {
    sem_wait(&read_try);      /* check if readers are allowed */
    sem_wait(&mutex);
    read_count++;
    if (read_count == 1)
        sem_wait(&rw_mutex);  /* first reader locks out writers */
    sem_post(&mutex);
    sem_post(&read_try);

    /* read the database */

    sem_wait(&mutex);
    read_count--;
    if (read_count == 0)
        sem_post(&rw_mutex);  /* last reader lets writers in */
    sem_post(&mutex);
    return NULL;
}

void *writer(void *arg) {
    sem_wait(&read_try);      /* block new readers */
    sem_wait(&rw_mutex);      /* exclusive access */

    /* write to the database */

    sem_post(&rw_mutex);
    sem_post(&read_try);      /* allow readers again */
    return NULL;
}
```

When a writer calls `sem_wait(&read_try)`, it blocks subsequent readers from entering the `read_try` section. Readers already inside the critical section can finish, but no new readers can start. Once all existing readers exit and release `rw_mutex`, the writer proceeds. This gives writers preference at the cost of potentially starving readers.

Theorem 7.4b (No Fair Readers-Writers Solution with Semaphores Alone). Neither the readers-preference nor the writers-preference solution is fair: one class of processes can starve the other. A **fair** readers-writers solution requires additional mechanisms such as a FIFO queue or a turnstile that alternates between readers and

writers.

7.5.4 Dining Philosophers Problem

Definition 7.12 (Dining Philosophers). Five philosophers sit at a round table. Between each pair of adjacent philosophers lies one chopstick (five total). A philosopher alternates between thinking and eating. To eat, a philosopher must acquire both the left and right chopsticks. The problem is to design a protocol that prevents deadlock and starvation.

Naive solution (deadlock-prone):

```
sem_t chopstick[5];

void *philosopher(void *arg) {
    int i = *(int *)arg;
    while (1) {
        think();
        sem_wait(&chopstick[i]);           /* pick up left */
        sem_wait(&chopstick[(i + 1) % 5]); /* pick up right */
        eat();
        sem_post(&chopstick[(i + 1) % 5]); /* put down right */
        sem_post(&chopstick[i]);           /* put down left */
    }
    return NULL;
}
```

If all five philosophers pick up their left chopstick simultaneously, all will block waiting for the right one. This is a textbook deadlock.

Solution: Asymmetric ordering. Philosopher 4 picks up chopsticks in reverse order (right first, then left). This breaks the circular wait condition:

```
void *philosopher(void *arg) {
    int i = *(int *)arg;
    while (1) {
        think();
        if (i == 4) {
            sem_wait(&chopstick[0]);           /* right first */
            sem_wait(&chopstick[4]);           /* then left */
        } else {
            sem_wait(&chopstick[i]);           /* left first */
            sem_wait(&chopstick[(i + 1) % 5]); /* then right */
```

```
        }
        eat();
        sem_post(&chopstick[(i + 1) % 5]);
        sem_post(&chopstick[i]);
    }
    return NULL;
}
```

More generally, one can impose a total order on resource acquisition (always acquire the lower-numbered chopstick first), which is the resource ordering strategy for deadlock prevention (Chapter 8).

Solution: Limit concurrency. An alternative approach limits the number of philosophers allowed to eat simultaneously. With a semaphore initialised to $N - 1 = 4$, at most 4 philosophers can attempt to pick up chopsticks at once. Since 5 chopsticks exist and at most 4 philosophers hold one each, at least one chopstick remains free, guaranteeing that at least one philosopher can obtain both chopsticks:

```
sem_t seats;  /* limits concurrent eaters */
sem_init(&seats, 0, 4);  /* N - 1 */

void *philosopher(void *arg) {
    int i = *(int *)arg;
    while (1) {
        think();
        sem_wait(&seats);                       /* sit at table */
        sem_wait(&chopstick[i]);                /* pick up left */
        sem_wait(&chopstick[(i + 1) % 5]);      /* pick up right */
        eat();
        sem_post(&chopstick[(i + 1) % 5]);
        sem_post(&chopstick[i]);
        sem_post(&seats);                       /* leave table */
    }
    return NULL;
}
```

Theorem 7.4c (Dining Philosophers with N-1 Seats). Allowing at most $N - 1$ of N philosophers to attempt eating simultaneously guarantees deadlock freedom with N chopsticks.

Proof. With at most $N - 1$ philosophers at the table and N chopsticks, by the pigeonhole principle, at least one philosopher has both adjacent chopsticks available and can eat. That philosopher will eventually finish and release both chopsticks, al-

lowing others to proceed. Since at least one philosopher always makes progress, no deadlock occurs. □

This solution has the drawback of reduced concurrency: even when enough chopsticks are free for all 5 philosophers, only 4 are allowed to try.

7.6 Monitors and Condition Variables

7.6.1 The Problem with Semaphores

Semaphores are powerful but low-level. Common errors include:

- Swapping the order of `wait` operations (e.g., acquiring `mutex` before `empty` in the producer-consumer problem) $\rightarrow$ deadlock.
- Forgetting to call `signal` $\rightarrow$ permanent blocking.
- Calling `signal` instead of `wait` (or vice versa) $\rightarrow$ violation of mutual exclusion.

Monitors, introduced by C.A.R. Hoare (1974) and Per Brinch Hansen (1973), encapsulate synchronisation within an abstract data type, making errors structurally impossible.

7.6.2 Monitor Definition

Definition 7.13 (Monitor). A monitor is a synchronisation construct consisting of:

1. A set of programmer-defined procedures (entry points).
2. Private data variables accessible only through these procedures.
3. An implicit mutual exclusion lock: at most one thread may be executing any monitor procedure at any time.
4. **Condition variables** that allow threads to wait for specific conditions inside the monitor.

7.6.3 Condition Variables

Definition 7.14 (Condition Variable). A condition variable c associated with a monitor supports two operations:

- `wait`(c): the calling thread releases the monitor lock, blocks on c's wait queue, and re-acquires the monitor lock when signalled.
- `signal`(c): if any threads are waiting on c, one is woken. If no threads are waiting, the signal is lost (it is not queued).

Unlike semaphores, condition variables have no state. A signal with no waiters has no effect.

Example 7.4 (Bounded Buffer with Monitor). The producer-consumer problem expressed as a monitor:

```
#include <pthread.h>

#define BUFFER_SIZE 10

typedef struct {
    int buffer[BUFFER_SIZE];
    int count, in, out;
    pthread_mutex_t lock;
    pthread_cond_t not_full;
    pthread_cond_t not_empty;
} bounded_buffer_t;

void bb_init(bounded_buffer_t *bb) {
    bb->count = bb->in = bb->out = 0;
    pthread_mutex_init(&bb->lock, NULL);
    pthread_cond_init(&bb->not_full, NULL);
    pthread_cond_init(&bb->not_empty, NULL);
}

void bb_insert(bounded_buffer_t *bb, int item) {
    pthread_mutex_lock(&bb->lock);
    while (bb->count == BUFFER_SIZE)
        pthread_cond_wait(&bb->not_full, &bb->lock);
    bb->buffer[bb->in] = item;
    bb->in = (bb->in + 1) % BUFFER_SIZE;
    bb->count++;
    pthread_cond_signal(&bb->not_empty);
    pthread_mutex_unlock(&bb->lock);
}

int bb_remove(bounded_buffer_t *bb) {
    pthread_mutex_lock(&bb->lock);
    while (bb->count == 0)
        pthread_cond_wait(&bb->not_empty, &bb->lock);
    int item = bb->buffer[bb->out];
```

```
    bb->out = (bb->out + 1) % BUFFER_SIZE;
    bb->count--;
    pthread_cond_signal(&bb->not_full);
    pthread_mutex_unlock(&bb->lock);
    return item;
}
```

Note the `while` loop around `pthread_cond_wait` — this is essential under Mesa semantics (see below).

7.6.4 Mesa vs Hoare Semantics

The critical question in monitor design is: when `signal(c)` wakes a waiting thread, which thread runs next?

Definition 7.15 (Hoare Semantics). When thread A executes `signal`(c), the signalled thread B runs immediately inside the monitor. Thread A is suspended until B exits the monitor or waits again. The condition that B was waiting for is guaranteed to hold when B resumes.

Definition 7.16 (Mesa Semantics). When thread A executes `signal`(c), it merely marks thread B as runnable. Thread A continues executing in the monitor. By the time B actually runs, the condition may no longer hold, so B must recheck it (hence the `while` loop around `wait`).

Property	Hoare	Mesa
Condition guaranteed on wake	Yes	No — must recheck
Signaller suspended	Yes	No
Wait pattern	`if (!condition) wait(c)`	`while (!condition) wait(c)`
Implementation complexity	High	Low
Used in practice	Rare	Universal (POSIX, Java, Go)

Example 7.4b (Dining Philosophers with Monitor). The dining philosophers problem can be elegantly solved using a monitor that tracks the state of each philosopher:

```
#include <pthread.h>
```

```c
#define N 5
enum state_t { THINKING, HUNGRY, EATING };

typedef struct {
    enum state_t state[N];
    pthread_mutex_t lock;
    pthread_cond_t can_eat[N];  /* one condition per philosopher */
} dining_monitor_t;

void try_eat(dining_monitor_t *m, int i) {
    /* Can eat only if hungry and neither neighbour is eating */
    if (m->state[i] == HUNGRY &&
        m->state[(i + N - 1) % N] != EATING &&
        m->state[(i + 1) % N] != EATING) {
        m->state[i] = EATING;
        pthread_cond_signal(&m->can_eat[i]);
    }
}

void pickup(dining_monitor_t *m, int i) {
    pthread_mutex_lock(&m->lock);
    m->state[i] = HUNGRY;
    try_eat(m, i);
    while (m->state[i] != EATING)
        pthread_cond_wait(&m->can_eat[i], &m->lock);
    pthread_mutex_unlock(&m->lock);
}

void putdown(dining_monitor_t *m, int i) {
    pthread_mutex_lock(&m->lock);
    m->state[i] = THINKING;
    /* Check if neighbours can now eat */
    try_eat(m, (i + N - 1) % N);
    try_eat(m, (i + 1) % N);
    pthread_mutex_unlock(&m->lock);
}
```

This monitor-based solution avoids the individual chopstick locks entirely. A philosopher eats only when both neighbours are not eating, checked atomically inside the monitor. When a philosopher finishes, it signals its neighbours, who may then tran-

sition from HUNGRY to EATING. This solution is deadlock-free and starvation-free (under fair scheduling of condition variable waits).

Theorem 7.5 (Mesa Semantics Sufficiency). Any correct program under Hoare semantics remains correct under Mesa semantics if every `wait` is enclosed in a `while` loop that rechecks the condition.

Proof. Under Hoare semantics, the condition holds immediately upon waking from `wait`. Under Mesa semantics, the condition may not hold, but the `while` loop causes the thread to re-wait. The thread proceeds past the `while` only when the condition is actually true, which is exactly the Hoare guarantee. The only difference is that under Mesa semantics, a thread may wait multiple times, but since each wait releases and re-acquires the lock correctly, correctness is preserved. □

7.7 Spinlocks vs Sleeping Locks

7.7.1 When to Use Which

The choice between spinning and sleeping depends on the expected wait time relative to the cost of a context switch:

Definition 7.17 (Context Switch Cost). A context switch involves saving the current thread's register state, updating scheduler data structures, flushing and reloading TLB entries, and potentially flushing cache lines. The total cost on modern hardware is typically 1–10 microseconds.

Use spinlocks when:

- The critical section is shorter than two context switches (the waiting thread would waste more time sleeping and waking than spinning).
- You are in a context where sleeping is not permitted (e.g., interrupt handlers in the kernel).
- The number of contending threads is small relative to the number of cores.

Use sleeping locks (mutexes) when:

- The critical section is long (file I/O, network operations, complex computations).
- There are many more contending threads than available cores.
- You want to free the CPU for other work during the wait.

7.7.2 Adaptive Spinning

Modern mutex implementations combine both strategies. The Linux kernel's `mutex_lock()` and Go's `sync.Mutex` use **adaptive spinning**: they spin for a short time hoping the lock will be released quickly, then fall back to sleeping if the lock remains held.

Example 7.5 (Adaptive Spinning in Practice). The Linux kernel's `mutex_optimistic_spin()` function checks whether the lock owner is currently running on a CPU. If so, it spins — the owner is likely to release the lock soon. If the owner is not running (was preempted or is sleeping), spinning is futile, so the waiter sleeps immediately. This heuristic dramatically reduces unnecessary context switches for short critical sections while avoiding wasted spinning when the lock holder is not making progress.

7.7.3 Exponential Backoff

A key optimisation for spinlocks under contention is **exponential backoff**: after failing to acquire the lock, a thread waits for a random delay before retrying, and the delay doubles after each failure:

```c
#include <stdatomic.h>
#include <time.h>

#define MIN_DELAY 100     /* nanoseconds */
#define MAX_DELAY 100000  /* nanoseconds */

void spin_lock_backoff(spinlock_t *lock) {
    int delay = MIN_DELAY;
    while (1) {
        /* TTAS: test first, then try to acquire */
        while (atomic_load_explicit(lock, memory_order_relaxed) == 1)
            ;  /* spin on local cache */
        if (atomic_exchange(lock, 1) == 0)
            return;  /* acquired */
        /* Backoff: wait before retrying */
        struct timespec ts = {0, delay + (rand() % delay)};
        nanosleep(&ts, NULL);
        delay = (delay * 2 < MAX_DELAY) ? delay * 2 : MAX_DELAY;
    }
}
```

Exponential backoff reduces contention (fewer threads hammering the bus simultaneously) at the cost of increased latency when the lock becomes free.

7.7.4 Cost Analysis

Let T_{cs} be the critical section duration and T_{ctx} be the context switch cost. For a sleeping lock, the waiting thread incurs $2 \cdot T_{\text{ctx}}$ (sleep + wake). For a spinlock, the waiting thread burns CPU for T_{cs} on average.

Spinning is cheaper when $T_{\text{cs}} < 2 \cdot T_{\text{ctx}}$. On modern hardware with $T_{\text{ctx}} \approx 5\,\mu$s, this threshold is approximately $10\,\mu$s.

Example 7.5b (Lock Strategy Decision Tree).

```
Is the critical section < 10 us?
├── Yes: Is it in an interrupt handler?
│   ├── Yes: Use raw spinlock (cannot sleep in interrupt context)
│   └── No: Use spinlock with TTAS + backoff
└── No: Is it > 100 us?
    ├── Yes: Use sleeping mutex (futex-based)
    └── No: Use adaptive spinning mutex
            (spin briefly, then sleep)
```

This decision tree captures the practical heuristics used by kernel developers and language runtime designers. The Linux kernel uses raw spinlocks for interrupt handlers, MCS-style spinlocks for short critical sections, and sleeping mutexes for longer ones. The Go runtime's `sync.Mutex` uses adaptive spinning (spin for up to 4 iterations before sleeping).

7.8 Read-Write Locks

7.8.1 Definition

Definition 7.18 (Read-Write Lock). A read-write lock (RWLock) permits concurrent access by multiple readers or exclusive access by a single writer:

- `read_lock`(rw): acquires shared access. Blocks only if a writer holds the lock.
- `read_unlock`(rw): releases shared access.
- `write_lock`(rw): acquires exclusive access. Blocks if any reader or writer holds the lock.
- `write_unlock`(rw): releases exclusive access.

Read-write locks improve throughput for workloads dominated by reads. If r readers and w writers access a resource, and the read-to-write ratio is $r : w$, the maximum concurrency with a regular mutex is 1, while with an RWLock it is r (when no writers are active).

7.8.2 Implementation

```
#include <pthread.h>

typedef struct {
    pthread_mutex_t lock;
    pthread_cond_t readers_ok;
    pthread_cond_t writer_ok;
    int active_readers;
    int active_writers;
    int waiting_writers;
} rwlock_t;

void rwlock_init(rwlock_t *rw) {
    pthread_mutex_init(&rw->lock, NULL);
    pthread_cond_init(&rw->readers_ok, NULL);
    pthread_cond_init(&rw->writer_ok, NULL);
    rw->active_readers = 0;
    rw->active_writers = 0;
    rw->waiting_writers = 0;
}

void rwlock_read_lock(rwlock_t *rw) {
    pthread_mutex_lock(&rw->lock);
    /* Writers-preference: block if writers are waiting */
    while (rw->active_writers > 0 || rw->waiting_writers > 0)
        pthread_cond_wait(&rw->readers_ok, &rw->lock);
    rw->active_readers++;
    pthread_mutex_unlock(&rw->lock);
}

void rwlock_read_unlock(rwlock_t *rw) {
    pthread_mutex_lock(&rw->lock);
    rw->active_readers--;
    if (rw->active_readers == 0)
```

```
        pthread_cond_signal(&rw->writer_ok);
    pthread_mutex_unlock(&rw->lock);
}

void rwlock_write_lock(rwlock_t *rw) {
    pthread_mutex_lock(&rw->lock);
    rw->waiting_writers++;
    while (rw->active_readers > 0 || rw->active_writers > 0)
        pthread_cond_wait(&rw->writer_ok, &rw->lock);
    rw->waiting_writers--;
    rw->active_writers = 1;
    pthread_mutex_unlock(&rw->lock);
}

void rwlock_write_unlock(rwlock_t *rw) {
    pthread_mutex_lock(&rw->lock);
    rw->active_writers = 0;
    /* Wake one writer or all readers */
    pthread_cond_signal(&rw->writer_ok);
    pthread_cond_broadcast(&rw->readers_ok);
    pthread_mutex_unlock(&rw->lock);
}
```

This implementation gives **writers preference**: when a writer is waiting, new readers are blocked. This prevents writer starvation at the cost of potentially starving readers under heavy write contention.

7.8.3 Sequence Locks (Seqlocks)

Definition 7.19 (Sequence Lock). A sequence lock uses a sequence counter to allow readers to proceed without acquiring any lock. Writers increment the counter before and after writing. Readers check the counter before and after reading: if the counter changed or is odd (indicating a write in progress), the read is retried.

```
#include <stdatomic.h>

typedef struct {
    atomic_int sequence;
    pthread_mutex_t write_lock;
} seqlock_t;
```

```c
void seqlock_write_lock(seqlock_t *sl) {
    pthread_mutex_lock(&sl->write_lock);
    atomic_fetch_add(&sl->sequence, 1);  /* odd = write in progress */
    atomic_thread_fence(memory_order_release);
}

void seqlock_write_unlock(seqlock_t *sl) {
    atomic_thread_fence(memory_order_release);
    atomic_fetch_add(&sl->sequence, 1);  /* even = write complete */
    pthread_mutex_unlock(&sl->write_lock);
}

int seqlock_read_begin(seqlock_t *sl) {
    int seq;
    do {
        seq = atomic_load(&sl->sequence);
    } while (seq & 1);  /* wait if write in progress */
    atomic_thread_fence(memory_order_acquire);
    return seq;
}

int seqlock_read_retry(seqlock_t *sl, int old_seq) {
    atomic_thread_fence(memory_order_acquire);
    return atomic_load(&sl->sequence) != old_seq;
}
```

Usage pattern for readers:

```c
/* Reader: optimistic, lock-free read */
int seq;
struct data local_copy;
do {
    seq = seqlock_read_begin(&lock);
    local_copy = shared_data;  /* may read torn data */
} while (seqlock_read_retry(&lock, seq));
/* local_copy is now consistent */
```

Seqlocks are ideal when:

- Reads are far more frequent than writes.
- The shared data is small (fits in a few cache lines).
- Readers can tolerate occasionally reading inconsistent data (since they will

retry).

The Linux kernel uses seqlocks extensively, notably for reading the system time (`jiffies`, `xtime`), where reads happen billions of times per second but writes happen only at timer-tick frequency.

7.8.4 Choosing Between Lock Types

The following table summarises the synchronisation landscape and helps practitioners select the right primitive for each scenario:

Scenario	Best primitive	Reason
Single writer, rare writes, many readers	Seqlock	Readers never block; near-zero overhead
Many readers, occasional writers	Read-write lock (writer preference)	Concurrent reads; writers not starved
Equal reads and writes	Mutex	RWLock overhead not justified
Single shared counter	`atomic_fetch_add`	No lock needed
Complex multi-variable update	Mutex + condition variable	Atomicity across variables
Real-time, must not block	Spinlock + disable preemption	Bounded worst-case latency

Example 7.6 (Seqlock for System Time). The Linux kernel stores the current time in a `timekeeper` structure protected by a seqlock. Every timer interrupt (typically every 1–10 ms), the kernel writes the new time using `write_seqlock`. Every call to `gettimeofday()` reads the time using the seqlock's optimistic read pattern. Since timer interrupts are rare compared to time queries, the reader almost never needs to retry, achieving near-zero synchronisation overhead for the common case.

7.8.5 Barriers

Definition 7.20 (Barrier). A barrier is a synchronisation construct where N threads must all arrive at the barrier point before any of them is allowed to proceed past it. The barrier enforces a global synchronisation point in a parallel computation.

Barriers are essential in parallel algorithms that operate in phases: all threads complete phase k before any thread begins phase $k+1$. Examples include parallel matrix operations, iterative solvers, and parallel sorting.

```c
#include <pthread.h>

typedef struct {
    pthread_mutex_t lock;
    pthread_cond_t all_arrived;
    int count;       /* number of threads currently waiting */
    int threshold;   /* total number of threads */
    int generation;  /* prevents premature release on reuse */
} barrier_t;

void barrier_init(barrier_t *b, int num_threads) {
    pthread_mutex_init(&b->lock, NULL);
    pthread_cond_init(&b->all_arrived, NULL);
    b->count = 0;
    b->threshold = num_threads;
    b->generation = 0;
}

void barrier_wait(barrier_t *b) {
    pthread_mutex_lock(&b->lock);
    int gen = b->generation;
    b->count++;
    if (b->count == b->threshold) {
        /* Last thread to arrive: reset and wake all */
        b->count = 0;
        b->generation++;
        pthread_cond_broadcast(&b->all_arrived);
    } else {
        /* Wait for all threads to arrive */
        while (gen == b->generation)
            pthread_cond_wait(&b->all_arrived, &b->lock);
    }
    pthread_mutex_unlock(&b->lock);
}
```

The `generation` counter is critical for reusable barriers: without it, a fast thread that passes through the barrier and loops back could see the old `count` value and proceed

prematurely. The generation counter ensures that threads from different phases do not interfere.

Example 7.6b (Parallel Matrix Computation with Barrier). A parallel iterative solver (e.g., Jacobi iteration) updates each cell of a matrix based on its neighbours' values from the previous iteration:

```
/* Each thread handles a portion of the matrix */
void *worker(void *arg) {
    int my_id = *(int *)arg;
    for (int iter = 0; iter < MAX_ITER; iter++) {
        /* Phase 1: compute new values from old values */
        for (int i = my_start; i < my_end; i++)
            for (int j = 1; j < N - 1; j++)
                new_matrix[i][j] = 0.25 * (old_matrix[i-1][j] +
                    old_matrix[i+1][j] + old_matrix[i][j-1] +
                    old_matrix[i][j+1]);

        barrier_wait(&phase_barrier);  /* all threads finish computing */

        /* Phase 2: swap old and new matrices */
        if (my_id == 0)
            swap_pointers(&old_matrix, &new_matrix);

        barrier_wait(&swap_barrier);   /* all threads see swapped pointers */
    }
    return NULL;
}
```

Without the barriers, Thread A might start reading `old_matrix` values that Thread B has already overwritten with new values, producing incorrect results. The barrier ensures phase separation.

7.9 Programmer's Perspective: Go's Synchronisation Primitives

Programmer's Perspective

Go provides a rich set of synchronisation primitives in the `sync` and `sync/atomic` packages, designed around the principle that synchronisation should be explicit and composable. Understanding their implementation reveals how the concepts from this chapter map to a modern language runtime. **sync.Mutex** is a sleeping lock with adaptive spinning. Its internal structure uses a 32-bit state word encoding whether the lock is held, whether a goroutine is woken, whether the lock is in starvation mode, and the number of waiters. The fast path is a single `atomic.CompareAndSwapInt32`. If that fails, the goroutine spins briefly (up to 4 iterations on a multicore machine) before parking itself on a semaphore queue managed by the Go runtime. After 1 ms of waiting, the lock enters **starvation mode**, where ownership is handed directly to the longest-waiting goroutine (FIFO), preventing tail-latency problems:

```go
package main

import (
    "fmt"
    "sync"
)

func main() {
    var mu sync.Mutex
    counter := 0

    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            mu.Lock()
            counter++
            mu.Unlock()
        }()
    }

    wg.Wait()
    fmt.Println("Counter:", counter) // always 1000
}
```


sync.RWMutex implements a read-write lock. Internally, it uses a `sync.Mutex` for write serialisation plus atomic counters for reader tracking. When a writer

Programmer's Perspective

When choosing between Go's synchronisation primitives, consider these guidelines:

- Use **channels** for communication between goroutines (passing ownership of data).
- Use **sync.Mutex** when protecting a shared data structure that multiple goroutines read and write.
- Use **sync.RWMutex** when reads dominate writes (at least 10:1 ratio; otherwise the reader-tracking overhead negates the benefit).
- Use **sync/atomic** for single-variable updates where you need maximum performance and can reason about memory ordering.
- Use **sync.WaitGroup** for fork-join parallelism (launch N goroutines, wait for all to complete).
- Use **sync.Once** for lazy initialisation that must happen exactly once across all goroutines.

The `go test -race` flag enables the Go race detector, which instruments all memory accesses and reports data races at runtime. It is implemented using ThreadSanitizer (TSan) from the LLVM project. Running your test suite with `-race` is non-negotiable for any concurrent Go code.

7.10 Summary

This chapter developed the theory and practice of synchronisation primitives, from the formal requirements of the critical section problem through hardware atomic instructions to high-level abstractions.

Key results:

- The critical section problem requires three properties: mutual exclusion, progress, and bounded waiting. Any correct solution must satisfy all three.
- Peterson's algorithm is the simplest correct two-process solution under sequential consistency, but fails under relaxed memory models. The filter lock generalises to N processes but requires $O(N)$ shared variables.
- Hardware provides test-and-set, compare-and-swap (with infinite consensus number), and fetch-and-add as building blocks. The consensus hierarchy classifies primitives by their synchronisation power: CAS is universal (infinite consensus number), while test-and-set is limited to 2 threads.

- Mutexes provide sleeping mutual exclusion. The Linux futex mechanism avoids system calls in the uncontended case, achieving 10–20 ns lock acquisition. The three-state design (unlocked, locked-no-waiters, locked-with-waiters) ensures that the kernel is involved only when contention actually occurs.
- Semaphores generalise mutexes to counting and signalling. They solve the canonical problems: producer-consumer (bounded buffer), readers-writers (shared database), and dining philosophers (resource contention).
- Monitors encapsulate synchronisation within abstract data types. Mesa semantics (used universally in practice) requires `while`-loop condition checking around every `wait`. Hoare semantics is theoretically cleaner but impractical to implement.
- The spinlock vs sleeping lock decision depends on critical section duration relative to context switch cost. TTAS with exponential backoff optimises spinlocks; adaptive spinning combines spinning and sleeping.
- Read-write locks optimise for read-dominated workloads. Sequence locks provide lock-free reads at the cost of occasional retries. Barriers synchronise phases of parallel computations.

The next chapter examines what happens when synchronisation goes wrong: the theory of deadlock, its detection, prevention, avoidance, and recovery.

Exercises

Exercise 7.1. Prove that any solution to the two-process critical section problem using only shared read-write registers requires at least two registers. (*Hint*: consider an execution where one process crashes in its remainder section — the other process must still be able to enter its critical section, so the crashed process's state must be recorded somewhere.)

Exercise 7.2. The **bakery algorithm** (Lamport, 1974) solves the critical section problem for N processes. Each process P_i selects a ticket number $\texttt{number}[i] = 1 + \max(\texttt{number}[0], \dots, \texttt{number}[N-1])$ and waits until its ticket is the smallest among all interested processes (with ties broken by process ID). Prove that the bakery algorithm satisfies mutual exclusion, progress, and bounded waiting. What assumption about the atomicity of reading and writing `number[i]` is required?

Exercise 7.3. Consider a system with three processes P_0, P_1, P_2 and three resources R_0, R_1, R_2 (one instance each). The access pattern is:

- P_0: acquires R_0, then R_1
- P_1: acquires R_1, then R_2
- P_2: acquires R_2, then R_0

(a) Draw the resource allocation graph when all three processes hold their first resource and are waiting for their second. (b) Is this system deadlocked? (c) Propose a resource ordering that prevents deadlock.

Exercise 7.4. Implement a **readers-writers lock** with **fair scheduling**: neither readers nor writers can starve. Your solution should use POSIX mutexes and condition variables. Prove that your implementation satisfies the fairness property by showing that the maximum number of consecutive same-type accesses is bounded.

Exercise 7.5. A semaphore can be implemented using a mutex and a condition variable. Write the implementation in C using `pthread_mutex_t` and `pthread_cond_t`. Prove that your implementation is correct: `wait` decrements the count and blocks when zero, `signal` increments and wakes exactly one waiter.

Exercise 7.6. The **sleeping barber problem** is a classic synchronisation puzzle. A barber shop has one barber, one barber chair, and N waiting chairs. If there are no customers, the barber sleeps. When a customer arrives, they wake the barber (if sleeping) or sit in a waiting chair (if the barber is busy). If all waiting chairs are full, the customer leaves. Solve this problem using semaphores. Prove your solution is free of deadlock and starvation.

Exercise 7.7. A **seqlock** allows readers to proceed without blocking, at the cost of occasionally retrying. Analyse the expected number of reader retries as a function of the write frequency f_w (writes per second), the write duration T_w (seconds), and the read duration T_r (seconds). Under what conditions does a seqlock outperform a readers-writers lock? Assume Poisson-distributed write arrivals and derive the expected retry probability per read attempt.

Chapter 8: Deadlock

"The dining philosophers problem illustrates that even well-intentioned processes, following individually reasonable protocols, can collectively reach a state from which no progress is possible." — adapted from Dijkstra (1971)

Synchronisation primitives solve the problem of concurrent access to shared resources. But they introduce a new hazard: **deadlock**, a condition where a set of processes are each waiting for a resource held by another process in the set, forming a cycle of dependencies from which none can escape. Deadlock is not merely a theoretical curiosity — it brings production systems to a halt, and its diagnosis often requires understanding the global state of a system from local observations.

This chapter develops the theory of deadlock systematically. We formalise the system model, state the four necessary conditions for deadlock (Coffman's conditions), and then explore four strategies: prevention (making deadlock structurally impossible), avoidance (making safe decisions at runtime), detection (identifying deadlock after it occurs), and recovery (breaking the deadlock). We also examine the related phenomena of livelock and starvation.

8.1 System Model

8.1.1 Resources and Processes

Definition 8.1 (Resource). A resource is any entity that a process requires to make progress: CPU time, memory pages, disk blocks, file locks, semaphores, mutexes, database records, network sockets, or physical devices such as printers and tape drives.

Resources are classified as:

- **Reusable**: can be used repeatedly without being consumed (CPU, memory, locks, I/O channels).
- **Consumable**: are created (produced) and destroyed (consumed) during use (messages, signals, interrupts).

Each resource type R_j has W_j instances (e.g., a system might have 3 printers, 4 tape drives, or 16 GB of memory allocated in 4 KB pages).

The distinction between reusable and consumable resources is important because deadlock theory primarily addresses reusable resources. Consumable resources introduce additional complexity: a process waiting for a message that has not yet been produced is not holding a resource in the classical sense, and the message may never exist. Deadlocks involving consumable resources are generally harder to detect and prevent.

Definition 8.2 (Resource Usage Protocol). Each process uses a resource in three steps:

1. **Request**: the process asks the operating system for the resource. If the resource is unavailable, the process waits.
2. **Use**: the process operates on the resource (prints, reads, writes, computes).
3. **Release**: the process returns the resource to the operating system.

Request and release are system calls (e.g., `open()`/`close()`, `malloc()`/`free()`, `lock()`/`unlock()`).

Example 8.0 (Resource Usage in Practice). A database transaction illustrates all three phases:

```
/* Request: acquire locks on rows */
BEGIN TRANSACTION;
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;  /* lock row 1 */

/* Use: modify the resource */
UPDATE accounts SET balance = balance - 100 WHERE id = 1;

/* Release: commit releases all locks */
COMMIT;
```

Between the `SELECT FOR UPDATE` and `COMMIT`, the transaction holds an exclusive lock on row 1. Any other transaction attempting to modify or exclusively lock the same row must wait. If two transactions lock different rows and then request each other's rows, deadlock results.

8.1.2 Formal State Description

At any instant, the state of a resource allocation system can be described by:

- A set of processes $P = \{P_1, P_2, \dots, P_n\}$.

- A set of resource types $R = \{R_1, R_2, \dots, R_m\}$ with instance counts $W = (W_1, W_2, \dots, W_m)$.
- An **allocation matrix** A where A_{ij} is the number of instances of resource R_j currently allocated to process P_i.
- A **request matrix** Q where Q_{ij} is the number of additional instances of R_j that process P_i needs.
- An **available vector** V where V_j is the number of free instances of R_j.

The conservation law holds: for each resource type j,

$$\sum_{i=1}^{n} A_{ij} + V_j = W_j$$

This equation states that every instance of every resource is either allocated to some process or available. Resources do not appear or disappear (we assume a static set of resources for the core theory).

8.1.3 State Transitions

The system transitions between states through three operations:

1. **Request** by P_i for k instances of R_j: if $k \leq V_j$, the request may be granted immediately. Otherwise, P_i blocks and Q_{ij} is updated.
2. **Acquisition** of previously requested resources: $A_{ij} \leftarrow A_{ij} + k$, $V_j \leftarrow V_j - k$, $Q_{ij} \leftarrow Q_{ij} - k$.
3. **Release** by P_i of k instances of R_j: $A_{ij} \leftarrow A_{ij} - k$, $V_j \leftarrow V_j + k$.

A deadlock is a state from which no sequence of acquisitions and releases can allow all processes to complete. The theory we develop in this chapter characterises precisely when such states arise and how to handle them.

Example 8.0b (State Space Size). For a system with $n = 10$ processes and $m = 5$ resource types, each with $W_j = 4$ instances, the number of possible allocation states is the number of ways to distribute instances among processes and the free pool. This is:

$$\prod_{j=1}^{m} \binom{n + W_j}{n} = \binom{14}{10}^5 = 1001^5 \approx 10^{15}$$

Even for this modest system, exhaustive analysis of all states is infeasible. This motivates efficient algorithms (Banker's Algorithm, wait-for graph analysis) that work on

the current state rather than exploring the entire state space.

8.2 Deadlock Characterisation

8.2.1 Coffman's Four Conditions

Theorem 8.1 (Coffman Conditions, 1971). A deadlock can arise if and only if all four of the following conditions hold simultaneously:

1. **Mutual exclusion**: at least one resource must be held in a non-sharable mode. Only one process at a time can use the resource.
2. **Hold and wait**: a process holds at least one resource and is waiting to acquire additional resources held by other processes.
3. **No preemption**: resources cannot be forcibly removed from a process; they must be released voluntarily by the holding process.
4. **Circular wait**: there exists a set $\{P_0, P_1, \dots, P_k\}$ of waiting processes such that P_0 is waiting for a resource held by P_1, P_1 is waiting for a resource held by P_2, ..., and P_k is waiting for a resource held by P_0.

These conditions are **necessary**: if any one is absent, deadlock cannot occur. For the case of single-instance resource types, they are also **sufficient**.

The circular wait condition is the most operational: it directly corresponds to a cycle in the resource allocation graph. The first three conditions are structural properties of the system and its protocols.

Let us prove that each condition is indeed necessary:

- **Without mutual exclusion**: if all resources are sharable, no process ever blocks waiting for a resource, so no waiting cycle can form.
- **Without hold and wait**: if a process must release all resources before requesting new ones, it never holds resources while waiting, so the chain P_0 holds R_a and waits for R_b cannot occur.
- **Without no preemption**: if resources can be preempted, a process in the circular wait can have its resources forcibly taken and given to another, breaking the cycle.
- **Without circular wait**: if no cycle exists in the wait-for relationship, the dependency graph is a DAG, and at least one process can proceed (the one with no outgoing wait edges).

Example 8.1 (Deadlock in File Locking). Consider two processes that need to update two files, F_A and F_B:

```
/* Process 1 */
lock(F_A);
lock(F_B);    /* blocks: F_B held by Process 2 */
/* update both files */
unlock(F_B);
unlock(F_A);

/* Process 2 */
lock(F_B);
lock(F_A);    /* blocks: F_A held by Process 1 */
/* update both files */
unlock(F_A);
unlock(F_B);
```

All four Coffman conditions hold: (1) file locks are exclusive, (2) each process holds one lock while waiting for the other, (3) locks cannot be preempted, (4) Process 1 $\rightarrow$ Process 2 $\rightarrow$ Process 1 forms a cycle.

Example 8.1b (Deadlock with Three Processes). The two-process case above generalises. Consider three threads in a web server:

```
/* Thread A: transfer between accounts */
lock(account_1);
lock(account_2);   /* blocks: held by Thread B */

/* Thread B: transfer between accounts */
lock(account_2);
lock(account_3);   /* blocks: held by Thread C */

/* Thread C: calculate interest across accounts */
lock(account_3);
lock(account_1);   /* blocks: held by Thread A */
```

The circular wait is $A \rightarrow B \rightarrow C \rightarrow A$. With two processes, deadlock requires only two resources; with k processes, it requires at least k resources. In general, the minimum number of resources for a deadlock involving k processes is k (one per process in the cycle).

8.2.2 Resource Allocation Graph

Definition 8.3 (Resource Allocation Graph). A resource allocation graph $G = (V, E)$ is a directed graph where:

- $V = P \cup R$ where P is the set of processes and R is the set of resource types.
- E consists of two kinds of edges:
  - **Request edge** $P_i \to R_j$: process P_i is waiting for an instance of resource R_j.
  - **Assignment edge** $R_j \to P_i$: an instance of resource R_j is allocated to process P_i.

Each resource type R_j is drawn as a box containing W_j dots (one per instance). Each process P_i is drawn as a circle.

The resource allocation graph provides both a visual tool for understanding deadlocks and a computational tool for detecting them.

Theorem 8.2 (Deadlock Detection via RAG). In a resource allocation graph:

- If the graph contains **no cycle**, no deadlock exists.
- If each resource type has exactly **one instance** and the graph contains a cycle, then deadlock exists.
- If resource types have **multiple instances**, a cycle is necessary but not sufficient for deadlock.

Proof of the single-instance case. Suppose each resource type has one instance and the RAG contains a cycle $P_0 \to R_{j_0} \to P_1 \to R_{j_1} \to \cdots \to P_k \to R_{j_k} \to P_0$.

Each edge $P_i \to R_{j_i}$ means P_i is requesting R_{j_i}. Each edge $R_{j_i} \to P_{i+1}$ means R_{j_i} is allocated to P_{i+1}. So P_i is waiting for a resource held by P_{i+1} (indices modulo $k + 1$). All four Coffman conditions are satisfied (mutual exclusion by the single-instance assumption, hold and wait because each P_i holds $R_{j_{i-1}}$ and waits for R_{j_i}, no preemption by assumption, and circular wait by the cycle). Therefore, deadlock exists. □

Example 8.2 (Cycle Without Deadlock). Consider:

- R_1 has 2 instances, R_2 has 1 instance.
- P_1 holds one instance of R_1, requests R_2.
- P_2 holds R_2, requests R_1.
- P_3 holds one instance of R_1.

The graph contains a cycle: $P_1 \to R_2 \to P_2 \to R_1 \to P_1$. But this is not a deadlock: P_3 can finish and release its instance of R_1, satisfying P_2's request, which then releases R_2, satisfying P_1. The cycle exists, but an alternative execution path resolves

it.

8.2.3 Graph Reduction

Definition 8.3b (Graph Reduction). A resource allocation graph can be **reduced** by the following operation: if a process P_i has all its requests satisfiable (for each resource R_j that P_i requests, enough free instances exist), then remove all edges to and from P_i and return its allocated resources to the available pool.

A state is deadlock-free if and only if the graph can be completely reduced (all process nodes removed). The set of processes remaining after no further reductions are possible is the deadlocked set.

Graph reduction is the graphical equivalent of the detection algorithm presented in Section 8.5.

Example 8.2b (Graph Reduction). Consider a system with three processes and two resource types:

- R_1: 3 instances. P_1 holds 1, P_2 holds 1, P_3 holds 1. Available: 0.
- R_2: 2 instances. P_1 holds 1, P_3 holds 1. Available: 0.
- Requests: P_1 requests 1 of R_2, P_2 requests 1 of R_1, P_3 has no outstanding requests.

Step 1: P_3 has no requests. Reduce P_3: release its resources. Available becomes $R_1 : 1, R_2 : 1$.

Step 2: P_2 requests 1 of R_1, and 1 is available. Reduce P_2: release its resources. Available becomes $R_1 : 2, R_2 : 1$.

Step 3: P_1 requests 1 of R_2, and 1 is available. Reduce P_1: complete.

All processes reduced: no deadlock.

8.3 Deadlock Prevention

Deadlock prevention works by ensuring that at least one of the four Coffman conditions can never hold. Each strategy has trade-offs in terms of resource utilisation, programming convenience, and system throughput.

8.3.1 Breaking Mutual Exclusion

If resources can be shared, there is no need for exclusive access, and deadlock cannot arise. However, many resources are inherently non-sharable: a printer cannot interleave pages from two documents, and a mutex is exclusive by definition.

Approach: use spooling and virtualisation. Instead of granting direct access to a printer, the OS accepts print jobs into a spool queue. Only the spooler daemon accesses the printer exclusively; user processes never hold the printer resource directly. Similarly, virtual memory eliminates deadlocks over physical memory by giving each process the illusion of its own address space.

Limitation: not all resources can be virtualised or spooled. Mutexes protecting data structures must be exclusive. Database row locks must be exclusive for writes. The mutual exclusion condition is inherent to the resource, not a design choice that can be eliminated.

Read-write locks partially address this: by allowing concurrent readers, they reduce the scope of mutual exclusion. However, write operations still require exclusive access, so deadlocks involving writers remain possible.

8.3.2 Breaking Hold and Wait

Approach 1: All-or-nothing. A process must request all resources it will ever need before beginning execution. The system either grants all of them or none.

```
/* Request all resources atomically */
if (request(R1, R2, R3) == SUCCESS) {
    /* use R1, R2, R3 */
    release(R1, R2, R3);
} else {
    /* wait and retry */
}
```

Approach 2: Release before requesting. A process must release all currently held resources before requesting new ones.

```
/* Phase 1: use R1 */
request(R1);
/* work with R1 */
release(R1);

/* Phase 2: use R1 and R2 together */
request(R1);   /* must re-request R1 */
```

```
request(R2);
/* work with R1 and R2 */
release(R1);
release(R2);
```

Drawbacks of all-or-nothing:

1. **Low resource utilisation**: resources are held long before they are needed. A process that needs a printer only in its final phase holds the printer throughout its entire execution.
2. **Potential starvation**: a process that needs many popular resources may never find them all free simultaneously. Each time it tries, at least one is held by another process.
3. **Requires advance knowledge**: the process must know all resources it will need before starting. Many programs discover their resource needs dynamically (e.g., a database query whose execution plan depends on data statistics).

Drawbacks of release-before-request:

1. **State loss**: releasing a resource may require discarding work in progress. A database transaction that releases a row lock loses its isolation guarantee.
2. **Increased overhead**: re-acquiring resources wastes time, especially if no other process used the resource in the interim.

Example 8.2c (Hold-and-Wait in Two-Phase Locking). Database systems use **two-phase locking** (2PL), which divides a transaction into a growing phase (only acquire locks) and a shrinking phase (only release locks). This ensures serialisability but permits hold-and-wait: during the growing phase, a transaction holds some locks while requesting others. 2PL therefore does not prevent deadlock; databases rely on deadlock detection and transaction rollback instead.

8.3.3 Breaking No Preemption

Approach: if a process holding some resources requests additional resources that cannot be immediately allocated, all resources the process currently holds are pre-empted (released implicitly), and the process is restarted when all needed resources become available.

This is practical only for resources whose state can be saved and restored:

- **CPU registers**: yes (context switch saves and restores register state)
- **Memory pages**: yes (swap to disk, reload later)

- **Locks**: no (the protected data structure may be in an inconsistent state)
- **Printer mid-page**: no (half-printed output is useless)
- **Network connections**: partially (TCP can recover from brief interruptions, but application state may be lost)

Example 8.3a (Preemption in Memory Management). Virtual memory is the most successful application of resource preemption. When physical memory is scarce, the OS preempts pages from processes by swapping them to disk. The process is not aware that its page was taken away; when it accesses the swapped page, a page fault occurs, and the OS transparently reloads the page. This preemption prevents deadlocks over physical memory frames, at the cost of performance degradation (disk I/O is 1000x slower than memory access).

Variant: priority-based preemption. In some systems, a higher-priority process can preempt resources from a lower-priority process. This breaks the no-preemption condition for the lower-priority process but introduces the **priority inversion** problem (Chapter 7): a low-priority process holding a resource needed by a high-priority process can block the high-priority process. Priority inheritance protocols address this by temporarily boosting the low-priority process's priority.

8.3.4 Breaking Circular Wait

Definition 8.4 (Resource Ordering). Assign a total ordering $f : R \to \mathbb{N}$ to all resource types. Require that each process requests resources in increasing order of f: if a process holds R_j, it may only request R_k where $f(R_k) > f(R_j)$.

Theorem 8.3 (Resource Ordering Prevents Deadlock). If all processes request resources in a fixed total order f, circular wait cannot occur.

Proof. Suppose, for contradiction, that a circular wait exists: P_0 waits for R_{j_0} held by P_1, P_1 waits for R_{j_1} held by P_2, ..., P_k waits for R_{j_k} held by P_0.

Since P_i holds $R_{j_{i-1}}$ and requests R_{j_i}, the ordering constraint requires $f(R_{j_{i-1}}) < f(R_{j_i})$ for each i (a process holding a lower-numbered resource requests a higher-numbered one). Following the chain:

$$f(R_{j_0}) < f(R_{j_1}) < \cdots < f(R_{j_k}) < f(R_{j_0})$$

This is a contradiction: $f(R_{j_0})$ cannot be strictly less than itself. Therefore, no circular wait can exist under the ordering constraint. □

Resource ordering is the most widely used deadlock prevention strategy in practice. The Linux kernel enforces a lock ordering discipline, and tools like **lockdep** detect

violations at runtime.

Example 8.3b (Lock Ordering in Practice). The Linux kernel's lockdep subsystem maintains a directed graph of lock acquisition orders observed at runtime. When a thread acquires lock B while holding lock A, lockdep records the edge $A \to B$. If a later execution acquires A while holding B, lockdep detects the cycle $A \to B \to A$ and reports a potential deadlock, even if the deadlock has not actually occurred yet. This is a dynamic variant of resource ordering verification.

The output includes the full lock dependency chain:

```
=====================================================
WARNING: possible circular locking dependency detected
-----------------------------------------------------
swapper/0/1 is trying to acquire lock:
 (&mm->mmap_lock){+.+.}-{3:3}, at: lock_mm_and_find_vma
but task is already holding lock:
 (&rq->__lock){-.-.}-{2:2}, at: __schedule
which lock already depends on the new lock.
```

Example 8.3c (Resource Ordering for Dining Philosophers). Recall the dining philosophers problem (Chapter 7): five philosophers, five chopsticks. Assign $f(\text{chopstick}_i) = i$ for $i = 0, 1, 2, 3, 4$. Each philosopher must pick up the lower-numbered chopstick first:

Philosopher	Left chopstick	Right chopstick	Acquisition order
0	0	1	0, then 1
1	1	2	1, then 2
2	2	3	2, then 3
3	3	4	3, then 4
4	4	0	**0, then 4** (reversed!)

Philosopher 4 must pick up chopstick 0 (lower number) before chopstick 4 (higher number), which is the opposite of their "natural" left-then-right order. This breaks the circular wait: philosopher 4 and philosopher 0 both compete for chopstick 0 first, and only one succeeds. The other waits, preventing the simultaneous "everyone holds left, waits for right" scenario.

8.3.5 Comparison of Prevention Strategies

Strategy	Condition broken	Practicality	Resource utilisation	Starvation risk
Spooling/v	Mutual exclusion	Limited to specific resources	High	Low
All-or-nothing	Hold and wait	Low (requires advance knowledge)	Low (resources held too long)	High
Preemptio	No preemption	Limited to saveable resources	Moderate	Low
Resource ordering	Circular wait	High (widely used)	High	Low

Example 8.3d (Prevention Strategy in a Web Application). Consider a web application that handles bank transfers. Each transfer requires locks on two accounts:

```
/* Deadlock-prone: lock order depends on transfer direction */
void transfer(account_t *from, account_t *to, int amount) {
    lock(from);
    lock(to);
    from->balance -= amount;
    to->balance += amount;
    unlock(to);
    unlock(from);
}
```

If Thread A calls `transfer(acct_1, acct_2, 100)` and Thread B calls `transfer(acct_2, acct_1, 50)` simultaneously, deadlock can occur.

Fix using resource ordering:

```
void transfer(account_t *from, account_t *to, int amount) {
    /* Always lock the account with the lower ID first */
    account_t *first = (from->id < to->id) ? from : to;
    account_t *second = (from->id < to->id) ? to : from;
    lock(first);
    lock(second);
```

```
    from->balance -= amount;
    to->balance += amount;
    unlock(second);
    unlock(first);
}
```

Now both threads acquire locks in the same order (by account ID), regardless of the transfer direction. This is a clean application of Theorem 8.3.

Programmer's Perspective

In Go, there is no built-in lock ordering enforcement, but the convention is the same: document the ordering and adhere to it. A common pattern in Go servers is to define a hierarchy: `globalMu > sessionMu > connMu`. Every goroutine that needs multiple locks acquires them in this order. The `go vet` tool does not check lock ordering, but the race detector (`go test -race`) will catch many resulting data races. For critical systems, external tools such as `go-deadlock` (a drop-in replacement for `sync.Mutex`) detect lock ordering violations at runtime with a mechanism similar to Linux's lockdep:

```go
package main

import (
    "fmt"
    "sync"
)

// Convention: always lock mu1 before mu2
var mu1 sync.Mutex
var mu2 sync.Mutex

func transferAtoB() {
    mu1.Lock()
    defer mu1.Unlock()
    mu2.Lock()
    defer mu2.Unlock()
    fmt.Println("A -> B transfer")
}

func transferBtoA() {
    mu1.Lock()         // same order as transferAtoB!
    defer mu1.Unlock()
    mu2.Lock()
    defer mu2.Unlock()
    fmt.Println("B -> A transfer")
}
```

Both functions acquire `mu1` before `mu2`, regardless of the logical direction of the transfer. This is the resource ordering principle applied to Go mutexes.

8.4 Deadlock Avoidance: The Banker's Algorithm

Deadlock prevention is conservative: it restricts how processes use resources, potentially reducing concurrency. Deadlock avoidance takes a different approach: it allows all four Coffman conditions to potentially hold, but makes each allocation decision carefully to ensure the system never enters an unsafe state.

8.4.1 Safe and Unsafe States

Definition 8.5 (Safe State). A state is **safe** if there exists a sequence $\langle P_{i_1}, P_{i_2}, \dots, P_{i_n} \rangle$ of all processes such that, for each P_{i_k}, the resources that P_{i_k} can still request can be satisfied by the currently available resources plus the resources held by all P_{i_j} with $j < k$ (i.e., processes that finish before P_{i_k} in the sequence).

Such a sequence is called a **safe sequence**.

Definition 8.6 (Unsafe State). A state is **unsafe** if no safe sequence exists. An unsafe state does not necessarily mean deadlock has occurred — it means that deadlock **may** occur depending on future requests.

The relationship is:

```
┌──────────────────────────────────────────────────────────────────┐
│                         All states                               │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │              Safe states                                 │  │
│  │    (guaranteed deadlock-free)                            │  │
│  └──────────────────────────────────────────────────────────┘   │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │              Unsafe states                               │  │
│  │  ┌──────────────────────────────────────────────────┐   │  │
│  │  │          Deadlocked states                       │  │  │
│  │  └──────────────────────────────────────────────────┘   │  │
│  └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
```

Every deadlocked state is unsafe, but not every unsafe state is deadlocked. An unsafe state is one from which a particular sequence of future requests *could* lead to deadlock; whether it actually does depends on the processes' behaviour.

Example 8.3d (Safe vs Unsafe). Consider a system with 12 instances of a single resource type and three processes:

Process	Allocation	Max	Need
P_0	5	10	5
P_1	2	4	2
P_2	2	9	7

Available $= 12 - 5 - 2 - 2 = 3$.

Safe sequence: $\langle P_1, P_0, P_2 \rangle$:

1. P_1 needs 2, available is 3. Grant. P_1 finishes, releases 4. Available $= 5$.
2. P_0 needs 5, available is 5. Grant. P_0 finishes, releases 10. Available $= 10$.
3. P_2 needs 7, available is 10. Grant. P_2 finishes. Done.

Now suppose we grant P_0 one more unit (Allocation becomes 6, Available becomes 2):

Process	Allocation	Max	Need
P_0	6	10	4
P_1	2	4	2
P_2	2	9	7

Available $= 2$.

P_1 can still finish (needs 2, available 2), releasing 4 units. Available becomes 4. But then neither P_0 (needs 4, available 4 — this works!) nor P_2 (needs 7, available 4 — this fails). So the safe sequence is $\langle P_1, P_0, P_2 \rangle$: after P_0 finishes, Available $= 10$, and P_2 can proceed. The state is still safe.

If instead we had granted P_2 one more unit (Allocation becomes 3, Available becomes 1):

Available $= 1$. Only P_1 needs ≤ 1... but P_1 needs 2. No process can proceed. **Unsafe state.** If P_0 and P_2 both request their remaining needs, deadlock is inevitable.

8.4.2 The Banker's Algorithm

Dijkstra's Banker's Algorithm (1965) implements deadlock avoidance. The name comes from an analogy: a banker decides whether to grant a loan (allocate resources)

based on whether, after the loan, they can still guarantee that all customers (processes) can eventually be satisfied.

Data structures:

- n = number of processes, m = number of resource types
- `Available`$[j]$ = number of available instances of resource type R_j
- `Max`$[i][j]$ = maximum demand of process P_i for resource R_j
- `Allocation`$[i][j]$ = number of instances of R_j currently allocated to P_i
- `Need`$[i][j] =$ `Max`$[i][j] -$ `Allocation`$[i][j]$ = remaining demand

Safety algorithm (determines whether a state is safe):

```
#include <stdbool.h>
#include <string.h>

#define MAX_P 64
#define MAX_R 16

int Available[MAX_R];
int Max[MAX_P][MAX_R];
int Allocation[MAX_P][MAX_R];
int Need[MAX_P][MAX_R];
int n, m;  /* number of processes and resource types */

bool is_safe(void) {
    int Work[MAX_R];
    bool Finish[MAX_P];
    memcpy(Work, Available, m * sizeof(int));
    memset(Finish, false, n * sizeof(bool));

    int count = 0;
    while (count < n) {
        bool found = false;
        for (int i = 0; i < n; i++) {
            if (Finish[i]) continue;

            /* Check if P_i's remaining needs can be satisfied */
            bool can_finish = true;
            for (int j = 0; j < m; j++) {
                if (Need[i][j] > Work[j]) {
                    can_finish = false;
```

```
                    break;
                }
            }

            if (can_finish) {
                /* P_i can finish: reclaim its resources */
                for (int j = 0; j < m; j++)
                    Work[j] += Allocation[i][j];
                Finish[i] = true;
                found = true;
                count++;
            }
        }
        if (!found) return false;  /* no process can proceed */
    }
    return true;  /* all processes can finish */
}
```

Resource request algorithm (called when process P_i requests resources):

```
bool request_resources(int i, int Request[]) {
    /* Step 1: Validate request against maximum claim */
    for (int j = 0; j < m; j++) {
        if (Request[j] > Need[i][j])
            return false;  /* exceeds declared maximum */
    }

    /* Step 2: Check resource availability */
    for (int j = 0; j < m; j++) {
        if (Request[j] > Available[j])
            return false;  /* not enough resources: must wait */
    }

    /* Step 3: Tentatively allocate */
    for (int j = 0; j < m; j++) {
        Available[j] -= Request[j];
        Allocation[i][j] += Request[j];
        Need[i][j] -= Request[j];
    }

    /* Step 4: Check safety of resulting state */
```

```
    if (is_safe()) {
        return true;  /* grant the request */
    } else {
        /* Rollback: restore old state */
        for (int j = 0; j < m; j++) {
            Available[j] += Request[j];
            Allocation[i][j] -= Request[j];
            Need[i][j] += Request[j];
        }
        return false;  /* deny: would lead to unsafe state */
    }
}
```

8.4.3 Worked Example 1

Example 8.4 (Banker's Algorithm Execution). Consider a system with 5 processes and 3 resource types with total instances $(10, 5, 7)$.

Current state:

Process	Allocation	Max	Need
P_0	$(0, 1, 0)$	$(7, 5, 3)$	$(7, 4, 3)$
P_1	$(2, 0, 0)$	$(3, 2, 2)$	$(1, 2, 2)$
P_2	$(3, 0, 2)$	$(9, 0, 2)$	$(6, 0, 0)$
P_3	$(2, 1, 1)$	$(2, 2, 2)$	$(0, 1, 1)$
P_4	$(0, 0, 2)$	$(4, 3, 3)$	$(4, 3, 1)$

Available $= (10 - 7, 5 - 2, 7 - 5) = (3, 3, 2)$.

Safety check:

1. `Work` $= (3, 3, 2)$. Find a process whose Need $\leq$ Work.
   - P_1: Need $= (1, 2, 2) \leq (3, 3, 2)$. Run P_1:
     `Work` $= (3 + 2, 3 + 0, 2 + 0) = (5, 3, 2)$.
2. `Work` $= (5, 3, 2)$.
   - P_3: Need $= (0, 1, 1) \leq (5, 3, 2)$. Run P_3:
     `Work` $= (5 + 2, 3 + 1, 2 + 1) = (7, 4, 3)$.
3. `Work` $= (7, 4, 3)$.

   - P_0: Need $= (7, 4, 3) \leq (7, 4, 3)$. Run P_0:
     `Work` $= (7 + 0, 4 + 1, 3 + 0) = (7, 5, 3)$.
4. `Work` $= (7, 5, 3)$.
   - P_2: Need $= (6, 0, 0) \leq (7, 5, 3)$. Run P_2:
     `Work` $= (7 + 3, 5 + 0, 3 + 2) = (10, 5, 5)$.
5. `Work` $= (10, 5, 5)$.
   - P_4: Need $= (4, 3, 1) \leq (10, 5, 5)$. Run P_4: `Work` $= (10, 5, 7)$.

Safe sequence: $\langle P_1, P_3, P_0, P_2, P_4 \rangle$. The state is safe.

Now P_1 requests $(1, 0, 2)$:

- Check: $(1, 0, 2) \leq$ Need$[1] = (1, 2, 2)$. OK.
- Check: $(1, 0, 2) \leq$ Available $= (3, 3, 2)$. OK.
- Tentative allocation: Available $= (2, 3, 0)$, Allocation$[1] = (3, 0, 2)$, Need$[1] = (0, 2, 0)$.
- Safety check with new state: find sequence $\langle P_1, P_3, P_0, P_2, P_4 \rangle$. Safe. Grant the request.

Now P_4 requests $(3, 3, 0)$:

- Check: $(3, 3, 0) \leq$ Need$[4] = (4, 3, 1)$. OK.
- Check: $(3, 3, 0) \leq$ Available $= (2, 3, 0)$. FAIL: $3 > 2$ for the first resource. P_4 must wait.

8.4.4 Worked Example 2: Unsafe State

Example 8.5 (Banker's Algorithm — Denied Request). Starting from the state after P_1's request was granted:

Available $= (2, 3, 0)$.

P_0 requests $(0, 2, 0)$:

- Check: $(0, 2, 0) \leq$ Need$[0] = (7, 4, 3)$. OK.
- Check: $(0, 2, 0) \leq$ Available $= (2, 3, 0)$. OK.
- Tentative: Available $= (2, 1, 0)$, Allocation$[0] = (0, 3, 0)$, Need$[0] = (7, 2, 3)$.

Safety check:

- `Work` $= (2, 1, 0)$.
- P_0: Need $(7, 2, 3)$. $7 > 2$: cannot finish.
- P_1: Need $(0, 2, 0)$. $2 > 1$: cannot finish.
- P_2: Need $(6, 0, 0)$. $6 > 2$: cannot finish.
- P_3: Need $(0, 1, 1)$. $1 > 0$ for resource C: cannot finish.
- P_4: Need $(4, 3, 1)$. $4 > 2$: cannot finish.

No process can be selected. The state is **unsafe**. The Banker's Algorithm denies P_0's request. P_0 must wait until more resources become available (e.g., after another process completes and releases resources).

8.4.5 Proof of Safety

Theorem 8.4 (Banker's Algorithm Correctness). If the Banker's Algorithm reports that a state is safe, then from that state, all processes can complete without deadlock, regardless of the order and timing of future requests (provided each process's total requests do not exceed its declared maximum).

Proof. The safety algorithm constructs a safe sequence $\langle P_{i_1}, \dots, P_{i_n} \rangle$ by induction.

Base case: P_{i_1}'s need can be satisfied by `Available`, so P_{i_1} can acquire all resources it needs, complete, and release all its resources. After P_{i_1} completes, `Available` increases by $\texttt{Allocation}[i_1]$.

Inductive step: assume $P_{i_1}, \dots, P_{i_{k-1}}$ have completed, releasing their resources. Then $\texttt{Work} = \texttt{Available} + \sum_{j=1}^{k-1} \texttt{Allocation}[i_j]$. The algorithm found P_{i_k} such that $\texttt{Need}[i_k] \leq \texttt{Work}$, so P_{i_k} can complete.

After all n processes complete, all resources are returned. Since the algorithm only grants a request if the resulting state is safe, the system can always find a completion ordering. Deadlock is a state from which no process can complete, which contradicts the existence of a safe sequence. □

Theorem 8.4b (Safe State Invariant). If the system starts in a safe state and the Banker's Algorithm is used for all allocation decisions, the system remains in a safe state at all times.

Proof. By induction on the sequence of allocation decisions. The initial state is safe (given). Each decision either grants the request (only if the resulting state is safe, as verified by the safety algorithm) or denies it (the state remains unchanged, and hence safe). Process completions can only improve safety: when a process finishes and releases resources, `Available` increases, which can only make it easier (not harder) to find a safe sequence. Therefore, the system is always in a safe state, and deadlock can never occur. □

8.4.6 Complexity and Limitations

The safety algorithm has time complexity $O(n^2 \cdot m)$: the outer while-loop runs at most n times (each iteration marks at least one process as finished), and each iteration scans up to n processes, comparing m resource types.

The resource request algorithm calls the safety algorithm once, preceded by $O(m)$ work for validation and tentative allocation. Therefore, each resource request costs $O(n^2 \cdot m)$.

Limitations of the Banker's Algorithm:

1. **Maximum demand must be declared in advance.** Many real systems cannot predict their maximum resource usage before execution. A web server cannot predict how many database connections a request handler will need.
2. **Fixed number of processes and resources.** The algorithm does not handle dynamic process creation (fork) or resource addition (hot-plugging memory or disks).
3. **Conservative.** It may deny requests that would not actually lead to deadlock, reducing concurrency. The algorithm assumes the worst case: every process may request its full remaining need simultaneously.
4. **Assumes processes eventually release resources.** If a process crashes while holding resources, the system may deadlock despite the Banker's guarantee.
5. **Centralised.** The algorithm requires a single decision point with global knowledge of all allocations and demands. This is impractical in distributed systems.

For these reasons, the Banker's Algorithm is rarely used in general-purpose operating systems. It finds application in specialised real-time and embedded systems where resource usage is predictable, the set of tasks is fixed, and deadlock is unacceptable.

8.4.7 The Single-Resource Banker's Algorithm

For systems with a single resource type, the Banker's Algorithm simplifies considerably. The safety check reduces to sorting processes by remaining need:

Theorem 8.4c (Single-Resource Safety). A system with one resource type, n processes, W total instances, and allocations a_i with maximum claims m_i is safe if and only if the processes can be ordered $P_{i_1}, P_{i_2}, \dots, P_{i_n}$ such that for each k:

$$m_{i_k} - a_{i_k} \leq V + \sum_{j=1}^{k-1} a_{i_j}$$

where $V = W - \sum_{i=1}^{n} a_i$ is the number of available instances. The optimal ordering is to sort by remaining need $m_i - a_i$ in ascending order.

Proof. The greedy strategy of finishing the process with the smallest remaining need first maximises the resources returned at each step, making it possible to satisfy subsequent processes. If the greedy ordering fails, no ordering can succeed (since the greedy choice maximises available resources at every step). □

This simplification reduces the complexity from $O(n^2 m)$ to $O(n \log n)$ (dominated by the sorting step) for a single resource type.

Example 8.5d (Single-Resource Banker's). System with 12 units, 4 processes:

Process	Allocation	Max	Need
P_0	3	9	6
P_1	2	4	2
P_2	2	7	5
P_3	1	3	2

Available $= 12 - 3 - 2 - 2 - 1 = 4$.

Sort by need: P_1 (2), P_3 (2), P_2 (5), P_0 (6).

1. P_1: need $2 \leq 4$. Finish. Available $= 4 + 2 = 6$.
2. P_3: need $2 \leq 6$. Finish. Available $= 6 + 1 = 7$.
3. P_2: need $5 \leq 7$. Finish. Available $= 7 + 2 = 9$.
4. P_0: need $6 \leq 9$. Finish.

Safe. Now suppose P_0 requests 2 more (allocation becomes 5, available becomes 2): Sort by need: P_1 (2), P_3 (2), P_0 (4), P_2 (5). P_1: need $2 \leq 2$. Available $= 4$. P_3: need $2 \leq 4$. Available $= 5$. P_0: need $4 \leq 5$. Available $= 10$. P_2: need $5 \leq 10$. Safe. Grant.

8.5 Deadlock Detection

If deadlock prevention and avoidance are too restrictive or impractical, the alternative is to allow deadlocks to occur and then detect and resolve them. This is the approach taken by most database systems and some operating systems.

8.5.1 Wait-For Graph

Definition 8.7 (Wait-For Graph). For a system where each resource type has exactly one instance, the wait-for graph $G_W = (V, E)$ is a directed graph where:

- $V = P$ (the set of processes).
- There is an edge $P_i \to P_j$ if and only if P_i is waiting for a resource held by P_j.

Deadlock exists if and only if G_W contains a cycle.

The wait-for graph is obtained from the resource allocation graph by collapsing resource nodes: the pair of edges $P_i \to R_k \to P_j$ (process P_i requests resource R_k, which is held by P_j) becomes the single edge $P_i \to P_j$.

Cycle detection in the wait-for graph can be performed in $O(|V| + |E|)$ time using depth-first search. For n processes, each waiting for at most one resource, the graph has at most n edges, so detection is $O(n)$.

The DFS-based cycle detection algorithm proceeds as follows:

```c
/* Wait-for graph cycle detection via DFS */
#define MAX_P 256

int adj[MAX_P];       /* adj[i] = j means P_i waits for P_j, -1 if none */
int visited[MAX_P];   /* 0 = unvisited, 1 = in progress, 2 = done */
int deadlocked[MAX_P]; /* 1 if process is in a deadlock cycle */

int dfs(int node) {
    visited[node] = 1;  /* mark as in progress */
    int next = adj[node];
    if (next >= 0) {
        if (visited[next] == 1) {
            /* Back edge: cycle detected */
            deadlocked[next] = 1;
            return next;  /* return start of cycle */
        }
        if (visited[next] == 0) {
            int cycle_start = dfs(next);
            if (cycle_start >= 0) {
                deadlocked[node] = 1;
                if (cycle_start == node)
                    return -1;  /* full cycle marked */
                return cycle_start;
            }
        }
    }
    visited[node] = 2;  /* mark as done */
    return -1;
```

```
}

void detect_deadlocks(int n) {
    for (int i = 0; i < n; i++) {
        visited[i] = 0;
        deadlocked[i] = 0;
    }
    for (int i = 0; i < n; i++) {
        if (visited[i] == 0)
            dfs(i);
    }
}
```

This algorithm visits each node and each edge exactly once, giving $O(n)$ time complexity for single-instance resource types (where each node has at most one outgoing edge).

Example 8.5b (Wait-For Graph Construction). Five processes with single-instance resources:

- P_1 holds R_1, requests R_2 (held by P_2)
- P_2 holds R_2, requests R_3 (held by P_3)
- P_3 holds R_3, requests R_1 (held by P_1)
- P_4 holds R_4, requests R_5 (held by P_5)
- P_5 holds R_5, no pending requests

Wait-for graph edges: $P_1 \to P_2$, $P_2 \to P_3$, $P_3 \to P_1$, $P_4 \to P_5$.

DFS from P_1 discovers the cycle $P_1 \to P_2 \to P_3 \to P_1$. Processes P_1, P_2, P_3 are deadlocked. P_4 and P_5 are not involved.

8.5.2 Detection Algorithm for Multiple-Instance Resources

When resource types have multiple instances, cycle detection in the wait-for graph is insufficient (as shown in Example 8.2). We use a generalisation of the safety algorithm:

```
bool detect_deadlock(void) {
    int Work[MAX_R];
    bool Finish[MAX_P];
    memcpy(Work, Available, m * sizeof(int));

    /* Mark processes with no outstanding requests as finished */
    for (int i = 0; i < n; i++) {
```

```
        Finish[i] = true;
        for (int j = 0; j < m; j++) {
            if (Request[i][j] > 0) {
                Finish[i] = false;
                break;
            }
        }
    }

    /* Iteratively find processes that can have requests satisfied */
    bool changed = true;
    while (changed) {
        changed = false;
        for (int i = 0; i < n; i++) {
            if (Finish[i]) continue;

            bool can_proceed = true;
            for (int j = 0; j < m; j++) {
                if (Request[i][j] > Work[j]) {
                    can_proceed = false;
                    break;
                }
            }

            if (can_proceed) {
                for (int j = 0; j < m; j++)
                    Work[j] += Allocation[i][j];
                Finish[i] = true;
                changed = true;
            }
        }
    }

    /* Any unfinished process is in deadlock */
    for (int i = 0; i < n; i++) {
        if (!Finish[i]) return true;
    }
    return false;
}
```

The key difference from the Banker's safety algorithm is that this algorithm uses the **current request** matrix (what each process is waiting for right now) rather than the **maximum need** matrix (what each process might ever request). This makes it more precise: it identifies actual deadlocks rather than potential ones.

Theorem 8.5 (Detection Algorithm Correctness). The detection algorithm correctly identifies all deadlocked processes. A process P_i is in deadlock if and only if `Finish`$[i]$ = `false` after the algorithm terminates.

Proof. The algorithm simulates an optimistic execution: it assumes that any process whose current requests can be satisfied will eventually complete and release all its resources. If a process cannot proceed even under this optimistic assumption, it is truly blocked.

More formally: the set $D = \{P_i \mid \texttt{Finish}[i] = \texttt{false}\}$ has the property that for each $P_i \in D$, P_i's request cannot be satisfied by `Work` (the resources available plus those released by all processes not in D). Since no process outside D holds resources needed by processes in D (if it did, those resources would have been added to `Work` when that process was marked finished), the processes in D are waiting only for resources held by other processes in D. This is precisely the deadlock condition. □

Example 8.5c (Detection Algorithm Execution). Consider:

Process	**Allocation** (A, B, C)	**Request** (A, B, C)
P_0	$(0, 1, 0)$	$(0, 0, 0)$
P_1	$(2, 0, 0)$	$(2, 0, 2)$
P_2	$(3, 0, 3)$	$(0, 0, 0)$
P_3	$(2, 1, 1)$	$(1, 0, 0)$
P_4	$(0, 0, 2)$	$(0, 0, 2)$

Available $= (0, 0, 0)$.

1. P_0 has no requests: `Finish`$[0]$ = `true`. Work $= (0 + 0, 0 + 1, 0 + 0) = (0, 1, 0)$.
2. P_2 has no requests: `Finish`$[2]$ = `true`. Work $= (0 + 3, 1 + 0, 0 + 3) = (3, 1, 3)$.
3. P_3: Request $(1, 0, 0) \leq (3, 1, 3)$. `Finish`$[3]$ = `true`. Work $= (3 + 2, 1 + 1, 3 + 1) = (5, 2, 4)$.
4. P_1: Request $(2, 0, 2) \leq (5, 2, 4)$. `Finish`$[1]$ = `true`. Work $= (5 + 2, 2 + 0, 4 + 0) = (7, 2, 4)$.

5. P_4: Request $(0, 0, 2) \leq (7, 2, 4)$. `Finish`$[4] =$ `true`.

All processes finished. No deadlock.

Now change P_2's request to $(0, 0, 1)$:

1. P_0: no requests. Finish. Work $= (0, 1, 0)$.
2. No other process can proceed: P_1 needs $(2, 0, 2)$, but Work has only $(0, 1, 0)$. P_2 needs $(0, 0, 1)$, but Work has $(0, 1, 0)$; $0 < 1$? No, $0 \geq 0$ for A and B, but $0 < 1$ for C. Cannot proceed. P_3 needs $(1, 0, 0)$; $1 > 0$. Cannot proceed. P_4 needs $(0, 0, 2)$; $2 > 0$. Cannot proceed.

$D = \{P_1, P_2, P_3, P_4\}$. These processes are deadlocked.

8.5.3 When to Run Detection

The frequency of deadlock detection involves a trade-off between detection latency and computational overhead:

- **On every resource request**: catches deadlocks immediately but has $O(n^2 \cdot m)$ overhead per request. Suitable only for small systems.
- **Periodically** (e.g., every 5 minutes or when CPU utilisation drops below a threshold): lower overhead but delayed detection. Deadlocked processes waste resources during the detection interval.
- **On specific triggers**: when a process has waited longer than a timeout, or when the system detects low throughput (high CPU idle time despite runnable processes).

Example 8.5d (Detection Frequency Trade-off). A database system with 100 concurrent transactions and 50 lock types would execute the detection algorithm with complexity $O(100^2 \times 50) = O(500{,}000)$ operations per run. Running this on every lock request (potentially thousands per second) adds significant overhead. Running it every 5 seconds is usually sufficient: the cost of 5 seconds of wasted resources is far less than the cost of continuous detection.

InnoDB (MySQL's storage engine) uses an aggressive approach: it runs deadlock detection on every lock wait. To bound the cost, it limits the search depth (the number of edges traversed in the wait-for graph) to 200. If the graph is too deep to analyse within this limit, InnoDB falls back to a timeout-based approach (`innodb_lock_wait_timeout`, default 50 seconds).

8.6 Deadlock Recovery

Once a deadlock is detected, the system must break it. There are two main approaches, each with significant trade-offs.

8.6.1 Process Termination

Abort all deadlocked processes. This is the simplest approach but the most expensive: all work done by the terminated processes is lost. It is guaranteed to break the deadlock.

Abort processes one at a time until the deadlock cycle is broken. After each termination, re-run the detection algorithm to check whether the deadlock is resolved. The order of termination matters; common criteria include:

1. **Priority**: terminate low-priority processes first. Preserves important work.
2. **Elapsed time**: terminate the process that has run for the shortest time. Minimises lost work.
3. **Resources held**: terminate the process holding the most resources. Maximises the resources freed.
4. **Resources needed**: terminate the process that needs the most additional resources. It was furthest from completion.
5. **Type**: terminate batch processes before interactive ones. Users are more sensitive to lost interactive sessions.
6. **Rollback cost**: terminate the process for which rollback is cheapest. Some processes can be easily restarted (idempotent batch jobs); others cannot (processes that have performed irrevocable actions like sending emails or dispensing cash).

8.6.2 Resource Preemption

Instead of terminating processes, preempt resources from some processes and give them to others. This requires:

1. **Selecting a victim**: which process will have its resources preempted? The same criteria as above apply, with an emphasis on minimising total cost.
2. **Rollback**: the victim process must be rolled back to a safe state from which it can restart. This requires **checkpointing**: periodically saving the process's state so that it can be restored.

3. **Starvation prevention**: the same process must not always be selected as the victim. A common solution is to include the number of times a process has been preempted in the cost function, increasing its priority over time.

Definition 8.7b (Checkpoint). A checkpoint is a saved snapshot of a process's state at a particular point in execution. The state includes register values, memory contents, open file descriptors, and any other information needed to resume execution from that point. Checkpoints enable rollback: if a process must be preempted, it is rolled back to its most recent checkpoint, and the resources it acquired after the checkpoint are released.

Example 8.6 (Deadlock Recovery in Databases). Database systems are the most common users of deadlock detection and recovery. When a deadlock is detected among transactions, the DBMS selects a **victim transaction** (typically the one that has done the least work, measured by the number of log records or the number of locks held) and **rolls it back** to its last savepoint or to the beginning of the transaction. The rollback releases all locks held by the victim, breaking the cycle. The victim transaction is then restarted automatically.

PostgreSQL detects deadlocks by running a wait-for graph cycle detection algorithm whenever a transaction has waited for a lock for more than `deadlock_timeout` (default: 1 second). When a cycle is found, it terminates one of the involved transactions:

```
ERROR:  deadlock detected
DETAIL: Process 12345 waits for ShareLock on transaction 67890;
        blocked by process 67891.
        Process 67891 waits for ShareLock on transaction 12345;
        blocked by process 12345.
HINT:   See server log for query details.
```

MySQL's InnoDB engine uses a different approach: it runs detection immediately on every lock wait and selects the victim based on transaction weight (number of rows modified). The victim's transaction is rolled back, and the application receives an error code that it should handle by retrying the transaction.

8.6.3 Checkpointing for Rollback

Effective resource preemption requires the ability to roll a process back to a previous consistent state. **Checkpointing** saves the process's state at regular intervals so that recovery involves replaying from the last checkpoint rather than from the beginning.

```c
/* Simplified checkpoint mechanism */
typedef struct {
    void *memory_snapshot;
```

```
    size_t snapshot_size;
    int file_positions[MAX_FD];
    int checkpoint_id;
} checkpoint_t;

void save_checkpoint(process_t *p, checkpoint_t *cp) {
    cp->snapshot_size = p->memory_size;
    cp->memory_snapshot = malloc(cp->snapshot_size);
    memcpy(cp->memory_snapshot, p->memory_base, cp->snapshot_size);
    for (int fd = 0; fd < p->num_open_files; fd++)
        cp->file_positions[fd] = lseek(fd, 0, SEEK_CUR);
    cp->checkpoint_id++;
}

void restore_checkpoint(process_t *p, checkpoint_t *cp) {
    memcpy(p->memory_base, cp->memory_snapshot, cp->snapshot_size);
    for (int fd = 0; fd < p->num_open_files; fd++)
        lseek(fd, cp->file_positions[fd], SEEK_SET);
    /* Resume execution from checkpoint */
}
```

The cost of checkpointing involves a trade-off:

- **Frequent checkpoints**: low rollback cost (less work to redo) but high overhead during normal execution (each checkpoint copies memory state).
- **Infrequent checkpoints**: low normal overhead but high rollback cost (more work is lost on preemption).

The optimal checkpoint interval T_c minimises total expected cost:

$$T_c^* = \sqrt{2 \cdot T_{\text{save}} \cdot T_{\text{mean}}}$$

where T_{save} is the time to save a checkpoint and T_{mean} is the mean time between failures requiring rollback.

Example 8.6b (Recovery Cost Analysis). Consider a deadlock involving three processes:

Process	CPU time consumed	Resources held	Rollback cost
P_1	45 minutes	3 locks, 2 files	High (data written to disk)

Process	CPU time consumed	Resources held	Rollback cost
P_2	2 minutes	1 lock	Low (in-memory only)
P_3	30 minutes	4 locks, 1 file	Medium (partial disk writes)

The optimal victim is P_2: it has consumed the least CPU time, holds the fewest resources, and has the lowest rollback cost. Terminating P_2 is likely to break the cycle and waste the least work.

However, if P_2 has been selected as a victim three times already, fairness considerations may dictate choosing P_1 or P_3 instead, even at higher cost.

8.7 Livelock and Starvation

8.7.1 Livelock

Definition 8.8 (Livelock). A livelock is a condition where processes continuously change state in response to each other but make no progress. Unlike deadlock (where processes are blocked and consume no CPU), livelocked processes are active and consuming resources, but their actions are futile.

Livelock is in some ways worse than deadlock: deadlocked processes at least do not waste CPU cycles. Livelocked processes consume CPU, memory bandwidth, and energy while accomplishing nothing. They are also harder to detect because the processes appear to be running normally.

Example 8.7 (Corridor Livelock). Two people approach each other in a narrow corridor. Both step left to avoid each other, then both step right, then left again, indefinitely. They are both active and responsive, but neither makes progress.

In computing, livelock occurs when processes repeatedly try and fail to acquire resources. Consider two processes using a "polite" lock acquisition strategy:

```
/* Process 1 */
while (1) {
    lock(A);
    if (trylock(B) == FAIL) {
        unlock(A);   /* be polite: release A and retry */
        continue;
    }
```

```
    /* critical section */
    unlock(B);
    unlock(A);
    break;
}

/* Process 2 */
while (1) {
    lock(B);
    if (trylock(A) == FAIL) {
        unlock(B);   /* be polite: release B and retry */
        continue;
    }
    /* critical section */
    unlock(A);
    unlock(B);
    break;
}
```

If both processes execute in lockstep, Process 1 always holds A when Process 2 tries it, and vice versa. Both repeatedly acquire one lock, fail to get the other, release, and retry. Neither thread blocks, but neither makes progress.

Example 8.7b (Livelock in Network Protocols). The Ethernet CSMA/CD protocol can livelock if two stations transmit simultaneously, detect the collision, wait the same backoff time, and retransmit simultaneously again. Without randomisation, this cycle repeats indefinitely. The protocol prevents livelock using **exponential random backoff**: after the k-th collision, a station waits a random time in $[0, 2^k - 1]$ slot times. The probability of sustained lockstep drops exponentially with each collision.

Solutions to livelock:

1. **Randomised backoff**: each process waits a random duration before retrying, making sustained lockstep extremely unlikely:

```
while (1) {
    lock(A);
    if (trylock(B) == FAIL) {
        unlock(A);
        usleep(rand() % 1000);  /* random backoff: 0-999 microseconds */
        continue;
    }
    /* critical section */
```

```
    unlock(B);
    unlock(A);
    break;
}
```

2. **Priority-based resolution**: assign each process a priority (e.g., based on process ID or timestamp). When two processes conflict, the lower-priority one backs off. Since priorities are distinct, only one process backs off, breaking the symmetry.
3. **Resource ordering**: if both processes acquire locks in the same order, livelock cannot occur (they never hold different subsets of locks simultaneously).

Theorem 8.6 (Randomised Backoff Terminates in Expected O(1) Rounds). If two processes use randomised backoff with delay chosen uniformly from $[0, D]$, the probability that they conflict in round k is at most T_{try}/D (where T_{try} is the time to perform a lock attempt). After k independent rounds, the probability of continued conflict is at most $(T_{\text{try}}/D)^k$, which converges to 0 exponentially.

Proof sketch. In each round, both processes choose a random delay independently. They conflict only if their retry times overlap within a window of T_{try}. For sufficiently large D, this probability is small. The expected number of rounds before one process proceeds without conflict is D/T_{try}, which is $O(1)$ for constant D. □

Example 8.7c (Livelock in Optimistic Concurrency Control). Database systems using optimistic concurrency control (OCC) can experience livelock. In OCC, transactions execute without acquiring locks, then validate at commit time. If validation fails (another transaction modified the same data), the transaction is rolled back and retried. If two transactions repeatedly modify the same rows, they can repeatedly invalidate each other:

```
Round 1: T1 modifies row A, T2 modifies row A.
         T1 commits first, T2's validation fails. T2 retries.
Round 2: T2 retries, modifies row A again. T1 starts new transaction
         on row A. T2 commits, T1's validation fails. T1 retries.
Round 3: Repeat...
```

The solution is again randomised backoff: a failed transaction waits a random exponentially increasing delay before retrying. PostgreSQL's serialisable snapshot isolation (SSI) includes this mechanism.

8.7.2 Starvation

Definition 8.9 (Starvation). Starvation occurs when a process is indefinitely denied access to a resource it needs, not because of deadlock, but because other processes are continuously favoured by the scheduling or allocation policy.

Starvation differs from deadlock in that the system as a whole is making progress — some processes are completing their work — but one or more specific processes are perpetually bypassed.

Example 8.8 (Starvation in Readers-Writers). In the first readers-writers solution (Chapter 7), a continuous stream of readers can prevent any writer from ever accessing the database. The writer is not deadlocked (it is waiting on a semaphore that could potentially be released), but it may wait forever if readers keep arriving. This is starvation.

The fix (discussed in Chapter 7) is to give writers preference: when a writer is waiting, new readers are blocked until the writer completes. This prevents writer starvation at the cost of potentially starving readers under heavy write contention. A fair solution alternates between readers and writers.

Example 8.8b (Starvation in Priority Scheduling). Consider a CPU scheduler that always runs the highest-priority process. If high-priority processes continuously arrive, a low-priority process may never receive CPU time. This is CPU starvation.

The solution is **ageing**: periodically increase the priority of waiting processes. After waiting long enough, even the lowest-priority process will become the highest priority and get scheduled. The rate of ageing determines the maximum wait time.

Starvation prevention mechanisms:

- **FIFO ordering**: serve requests in the order they arrive. Used in ticket locks (Chapter 7) and fair scheduling algorithms.
- **Ageing**: increase a process's priority the longer it waits. Guarantees that every process eventually reaches the highest priority.
- **Bounded bypass**: guarantee that at most B other processes can be served before any waiting process. The bounded waiting property of the critical section problem (Chapter 7) is a form of bounded bypass.
- **Fair locks**: Go's `sync.Mutex` enters starvation mode after 1 ms of waiting, switching from competitive (spinning) to FIFO (direct handoff) mode. This ensures bounded waiting in practice.

8.7.3 Formal Relationships

Definition 8.10 (Relationship Between Deadlock, Livelock, and Starvation).

- **Deadlock** $\implies$ no progress for any process in the deadlocked set. All deadlocked processes are blocked.
- **Livelock** $\implies$ no progress for any process in the livelocked set, despite all being active. The set collectively makes no progress.
- **Starvation** $\implies$ no progress for specific process(es), but the system overall makes progress.
- A starved process is not deadlocked (other processes are completing).
- A livelocked process is not deadlocked (it is not blocked).
- Deadlock is detectable (blocked processes are visible to the scheduler); livelock is harder to detect (active processes look normal); starvation depends on the fairness analysis of the scheduling policy.

8.8 Deadlock in Distributed Systems

8.8.1 The Distributed Deadlock Problem

In a distributed system, processes on different nodes may hold resources and request resources on other nodes. No single node has a complete view of the global wait-for graph. Detecting deadlocks requires coordination among nodes.

Definition 8.11 (Distributed Deadlock). A distributed deadlock occurs when a cycle exists in the global wait-for graph that spans multiple nodes. Each node's local wait-for graph shows only the edges corresponding to local resources; the global graph is the union of all local graphs plus inter-node wait edges.

Example 8.9 (Distributed Deadlock). Three database nodes:

```
Node 1: Transaction T1 holds lock L_A, requests L_B (on Node 2)
Node 2: Transaction T2 holds lock L_B, requests L_C (on Node 3)
Node 3: Transaction T3 holds lock L_C, requests L_A (on Node 1)
```

Each node sees only one edge of the cycle. Node 1 sees $T_1 \to T_2$ (an inter-node edge). Node 2 sees $T_2 \to T_3$. Node 3 sees $T_3 \to T_1$. No single node can detect the cycle.

8.8.2 Distributed Detection Algorithms

Centralised detection: one node (the coordinator) periodically collects all local wait-for graphs and constructs the global graph. Simple but creates a single point of failure and a communication bottleneck.

```
/* Pseudocode: centralised deadlock detection */
void coordinator_detect(void) {
    graph_t global_wfg;
    for (int node = 0; node < num_nodes; node++) {
        graph_t local = request_wait_for_graph(node);
        merge_graph(&global_wfg, &local);
    }
    if (has_cycle(&global_wfg)) {
        process_t victim = select_victim(&global_wfg);
        send_abort(victim);
    }
}
```

Distributed detection (probe-based, Chandy-Misra-Haas 1983): when a process P_i on node A is blocked waiting for a process P_j on node B, node A sends a **probe message** (P_i, P_j, P_i) to node B. If P_j is also blocked, the probe is forwarded along the wait-for chain, appending each blocked process. If the probe returns to its initiator (P_i appears as both the first and last element), a cycle (deadlock) is detected.

Example 8.9b (Chandy-Misra-Haas Probe Execution). Using the three-node scenario from Example 8.9:

```
Step 1: T1 (Node 1) is blocked. Sends probe (T1, T1, T2) to Node 2.
Step 2: Node 2 receives probe. T2 is blocked waiting for T3.
        Forwards probe (T1, T2, T3) to Node 3.
Step 3: Node 3 receives probe. T3 is blocked waiting for T1.
        Forwards probe (T1, T3, T1) to Node 1.
Step 4: Node 1 receives probe. The initiator (T1) matches the
        last element. Deadlock detected!
        Node 1 selects a victim (e.g., T1, the initiator) and aborts it.
```

The message complexity is $O(k)$ where k is the length of the cycle: one probe message per edge in the cycle. For cycles of length 2 (the most common case), only 2 messages are needed.

Edge-chasing algorithms generalise the probe approach. Each node maintains a partial wait-for graph. When a dependency crosses node boundaries, an edge-chasing message follows the dependency chain. The algorithm terminates when either a cycle

is detected or the chain ends at a non-blocked process.

8.8.3 The Phantom Deadlock Problem

Definition 8.12 (Phantom Deadlock). A phantom deadlock is a false positive in distributed deadlock detection: the algorithm reports a deadlock that does not actually exist, because the wait-for information collected from different nodes reflects different points in time. Between the time the probe was sent and the time it arrives, a process may have released its resources, breaking the cycle.

Example 8.10 (Phantom Deadlock). Consider three nodes:

```
Time 0: T1 (Node 1) holds L_A, waits for L_B (Node 2).
        T2 (Node 2) holds L_B, waits for L_C (Node 3).
        T3 (Node 3) holds L_C, waits for L_A (Node 1).
        -> True deadlock exists.

Time 1: Node 1 starts probe: sends (T1, T1, T2) to Node 2.

Time 2: T3 times out and releases L_C.
        T2 acquires L_C and proceeds. Deadlock is broken.

Time 3: Probe arrives at Node 2. T2 is no longer blocked.
        In theory, the probe should stop here.
        But if Node 2 processes the probe using stale state
        (before learning T3 released L_C), it forwards the probe.

Time 4: Probe reaches Node 3. T3 is still listed as waiting for L_A.
        Probe returns to Node 1: phantom deadlock detected!
        Node 1 aborts T1 unnecessarily.
```

The probe traversed edges that existed at different moments in time, detecting a cycle that was already broken. This is a phantom deadlock.

Phantom deadlocks arise because distributed snapshots are not instantaneous. They are a fundamental challenge in distributed deadlock detection and are one reason why many distributed systems prefer **timeout-based** deadlock handling: if a transaction waits longer than a threshold, it is aborted. This may occasionally abort non-deadlocked transactions (false positives), but it avoids the complexity and phantom-deadlock risk of distributed detection algorithms.

The trade-off between detection and timeout approaches can be quantified:

Approach	False positives	Detection latency	Message overhead
Probe-based detection	Possible (phantom deadlocks)	$O(k \cdot d)$ where k = cycle length, d = network delay	$O(k)$ messages per detection
Timeout-based	Common (any slow transaction appears deadlocked)	Fixed (= timeout value)	Zero (purely local)
Centralised detection	Rare (global snapshot)	$O(n \cdot d)$ where n = nodes	$O(n)$ per detection round

In practice, most production distributed databases (Google Spanner, CockroachDB, TiDB) use a combination: short timeouts for fast resolution of obvious deadlocks, plus periodic wait-for graph analysis for long-running transactions.

8.9 Practical Strategies in Real Systems

8.9.1 The Ostrich Algorithm

Many general-purpose operating systems (Linux, Windows, macOS) use the **ostrich algorithm**: they ignore deadlocks entirely. The reasoning is pragmatic:

1. Deadlocks are rare in practice (well-written applications use lock ordering).
2. Prevention and avoidance impose unacceptable overhead and restrict programming models.
3. When a deadlock does occur, the user notices (the system becomes unresponsive) and reboots or kills the offending processes manually.

This is a legitimate engineering trade-off: the cost of handling deadlocks exceeds the cost of occasional manual intervention. The name comes from the (apocryphal) belief that ostriches bury their heads in the sand to avoid danger.

The mathematical justification for the ostrich algorithm rests on a cost-benefit analysis. Let p be the probability of deadlock per unit time, C_d be the cost of a deadlock occurrence (including detection, recovery, and lost work), and C_p be the continuous cost of deadlock prevention or avoidance. The ostrich algorithm is optimal when:

$$p \cdot C_d < C_p$$

For a well-engineered system with lock ordering, p is extremely small (perhaps 10^{-6} per hour), C_d is moderate (a process restart), and C_p is substantial (the complexity and performance overhead of running the Banker's Algorithm on every resource request). The inequality is easily satisfied.

Example 8.10b (Cost of the Ostrich Algorithm). A web server handles 10,000 requests per second. With proper lock ordering, the probability of a deadlock is approximately 10^{-9} per request (one in a billion). Expected time between deadlocks: $10^9/10{,}000 = 100{,}000$ seconds ≈ 27 hours. The cost of a deadlock is a single process restart (10 seconds of downtime). The alternative — running deadlock detection on every lock acquisition — would add 1 microsecond per request, totalling 10,000 microseconds/second = 10 ms of CPU per second, or 0.86 seconds per day. Over 27 hours, this amounts to 24 seconds of CPU time to prevent 10 seconds of downtime — the ostrich algorithm is more efficient.

8.9.2 Combined Approaches

Real systems use different strategies for different resource types:

Resource	Strategy
Internal kernel locks	Prevention (lock ordering, enforced by lockdep)
Memory allocation	Prevention (allocation ordering) + detection (OOM killer)
File locks	Detection (timeout-based) + recovery (process termination)
Database locks	Detection (wait-for graph) + recovery (transaction rollback)
Printer/device access	Prevention (spooling eliminates mutual exclusion)
Network connections	Timeout-based (TCP keepalive, application-level timeouts)
Distributed transactions	Timeout-based + two-phase commit protocol

This heterogeneous approach reflects a fundamental principle: no single deadlock strategy is optimal for all resources. The right strategy depends on the resource's

characteristics (can it be preempted? shared? virtualised?), the cost of deadlock (is it catastrophic or merely inconvenient?), and the overhead of the strategy relative to normal operation.

Programmer's Perspective

In Go applications, the practical approach to deadlock management combines prevention and detection. Prevention comes from disciplined channel usage and lock ordering. Detection comes from three mechanisms:

1. **Runtime deadlock detection**: the Go runtime detects when all goroutines are asleep (blocked on channels, locks, or I/O) and no goroutine can make progress. It prints `fatal error: all goroutines are asleep - deadlock!` and dumps all goroutine stacks. This is a global deadlock detector — it catches the case where the entire program is stuck, but not partial deadlocks where some goroutines are still running.
2. **Goroutine dump analysis**: sending `SIGQUIT` (or `Ctrl-\`) to a Go process prints a full goroutine dump showing every goroutine's stack trace and blocking reason. This is invaluable for diagnosing partial deadlocks:

```
goroutine 1 [semacquire]:
sync.runtime_SemacquireMutex(0xc00001a0a8, 0x0, 0x1)
    /usr/local/go/src/runtime/sema.go:77 +0x25
sync.(*Mutex).lockSlow(0xc00001a0a0)
    /usr/local/go/src/sync/mutex.go:171 +0x152
sync.(*Mutex).Lock(...)
    /usr/local/go/src/sync/mutex.go:90
main.worker(0xc00001a0a0, 0xc00001a0c0)
    /home/user/deadlock.go:15 +0x45
```

3. **Context-based timeouts**: for production systems, the standard approach is to use `context.WithTimeout` to bound how long any operation can wait for a lock or resource:

```go
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

select {
case resource <- request:
    // acquired
case <-ctx.Done():
    log.Printf("timeout waiting for resource: possible deadlock")
    // handle timeout: retry, abort, or alert
}
```

This transforms potential deadlocks into timeouts, which are easier to detect, log, and recover from in a distributed system.

Programmer's Perspective

The Go race detector (`go test -race`) and deadlock detection address complementary problems. The race detector finds **data races** (unsynchronised concurrent access to shared memory), while deadlock detection finds **blocking cycles** (processes waiting for each other). A program can be race-free but deadlocked, or deadlock-free but racy. Production Go code should run both `-race` in testing and `GOTRACEBACK=all` in production (to get full goroutine dumps on crashes) to catch both classes of concurrency bugs.
For distributed Go services communicating over gRPC or HTTP, the deadlock model shifts from lock-based to resource-based: goroutine A is waiting for a response from service B, which is waiting for service C, which is waiting for service A. The detection strategy is context timeouts and distributed tracing (OpenTelemetry), not lock analysis. Every outgoing RPC call should carry a context with a deadline, and the system should be designed so that timeouts propagate: if A's context expires, it cancels its call to B, which cancels its call to C.

8.10 Summary

Deadlock is an inherent risk in any system where processes compete for resources. This chapter provided the theoretical framework for understanding and managing deadlocks:

Key results:

- Deadlock requires all four Coffman conditions: mutual exclusion, hold and wait, no preemption, and circular wait. Removing any one condition prevents deadlock.
- The resource allocation graph provides a visual and algorithmic tool for deadlock analysis. For single-instance resources, a cycle implies deadlock. For multiple-instance resources, graph reduction or the detection algorithm is needed.
- **Prevention** eliminates one Coffman condition statically. Resource ordering (breaking circular wait) is the most practical and widely used strategy, enforced by tools like Linux's lockdep.
- **Avoidance** (Banker's Algorithm) dynamically ensures the system stays in a safe state, at the cost of requiring advance knowledge of maximum resource

demands. It has $O(n^2m)$ time complexity per decision.

- **Detection** finds deadlocks after they occur using wait-for graphs ($O(n)$ for single-instance resources) or the generalised detection algorithm ($O(n^2m)$ for multiple-instance resources).
- **Recovery** involves process termination or resource preemption, guided by cost minimisation and fairness constraints.
- Livelock (active but futile processes) is solved by randomised backoff. Starvation (indefinite bypass) is solved by fairness mechanisms such as ageing and FIFO ordering.
- **Distributed deadlocks** require coordination across nodes and are subject to phantom deadlock false positives. Most distributed systems prefer timeout-based approaches. The Chandy-Misra-Haas algorithm detects distributed deadlocks with $O(k)$ messages per cycle of length k.
- Real systems use combined strategies tailored to each resource type, often including the ostrich algorithm for non-critical resources. The cost-benefit analysis $p \cdot C_d < C_p$ provides a principled framework for deciding when to invest in deadlock handling.
- The four strategies form a spectrum from most conservative (prevention) to most permissive (ostrich), with avoidance and detection-recovery as intermediate positions. No single strategy dominates all others; the optimal choice depends on the application domain, resource characteristics, and acceptable failure modes.
- **Checkpointing** enables resource preemption by saving process state at regular intervals. The optimal checkpoint interval balances the cost of saving state against the expected cost of lost work on rollback.

The next chapter moves from the correctness of individual synchronisation primitives to the design of complete concurrent data structures — structures that provide thread-safe operations through careful use of the primitives developed in Chapters 7 and 8.

Exercises

Exercise 8.1. A system has four processes P_1, P_2, P_3, P_4 and three resource types A (3 instances), B (2 instances), C (2 instances). The current state is:

Process	**Allocation** (A, B, C)	**Max** (A, B, C)	**Need** (A, B, C)
P_1	$(0, 1, 0)$	$(0, 1, 1)$	$(0, 0, 1)$
P_2	$(2, 0, 0)$	$(2, 1, 1)$	$(0, 1, 1)$
P_3	$(0, 0, 1)$	$(1, 0, 1)$	$(1, 0, 0)$
P_4	$(1, 0, 0)$	$(1, 1, 0)$	$(0, 1, 0)$

Available $= (0, 1, 1)$.

(a) Show that the system is in a safe state by finding a safe sequence.
(b) Process P_2 requests $(0, 0, 1)$. Should the request be granted? Use the Banker's Algorithm to justify your answer.
(c) If P_2's request is granted, can P_1 then request $(0, 0, 1)$? Justify.

Exercise 8.2. Prove that the Banker's Algorithm, when used to determine whether to grant each request, is sufficient to prevent deadlock. Specifically, show that if the system starts in a safe state and the Banker's Algorithm grants only requests that leave the system in a safe state, then no deadlock can ever occur.

Exercise 8.3. Consider a system with 5 processes and 3 resource types. The processes' maximum demands are:

Process	**Max** (R_1, R_2, R_3)
P_0	$(4, 1, 1)$
P_1	$(0, 2, 1)$
P_2	$(4, 2, 1)$
P_3	$(1, 1, 1)$
P_4	$(2, 1, 3)$

The total resources are $(6, 3, 4)$. (a) What is the minimum value of Available for each resource type that guarantees a safe initial state (before any allocations)? (b) Give an allocation that results in an unsafe state. (c) Give an allocation that results in deadlock.

Exercise 8.4. The time complexity of the Banker's safety algorithm is $O(n^2 \cdot m)$ where n is the number of processes and m is the number of resource types. Prove

this bound. Is a more efficient algorithm possible? (*Hint*: consider maintaining a sorted list of processes by remaining need and analyse the resulting complexity.)

Exercise 8.5. A distributed system has three nodes, each running a transaction manager. Node 1 holds lock L_A and requests L_B (held by Node 2). Node 2 holds L_B and requests L_C (held by Node 3). Node 3 holds L_C and requests L_A (held by Node 1). No single node can see the complete wait-for graph. (a) Describe a distributed deadlock detection algorithm based on probe messages (Chandy-Misra-Haas). (b) Analyse the message complexity of your algorithm as a function of the cycle length k. (c) Discuss the phantom deadlock problem: construct a scenario where the probe algorithm reports a deadlock that was already resolved by the time the probe completes.

Exercise 8.6. Write a program in C that deliberately creates a deadlock between two POSIX threads using `pthread_mutex_t`. Then modify the program to prevent the deadlock using resource ordering. Verify that the original program hangs and the modified program completes. Provide the complete source code for both versions.

Exercise 8.7. Prove that for a system with n processes and m resource types, each with a single instance, the problem of determining whether a state is deadlocked can be solved in $O(n + m)$ time by constructing the wait-for graph and running DFS for cycle detection. Contrast this with the $O(n^2 \cdot m)$ complexity of the general detection algorithm for multiple-instance resources. Under what conditions is the general algorithm necessary?

The study of deadlock connects fundamental theoretical concepts (graph theory, state-space analysis, resource allocation) with practical engineering concerns (lock ordering, timeout configuration, recovery strategies). The gap between theory and practice is narrower here than in most areas of operating systems: the Coffman conditions provide a precise characterisation, and the prevention, avoidance, and detection algorithms are directly implementable.

Chapter 9: Concurrent Data Structures

"A lock-free algorithm guarantees system-wide progress; a wait-free algorithm guarantees per-thread progress. The gap between these two guarantees is where most of the difficulty lies." — Maurice Herlihy

---

The synchronisation primitives of Chapter 7 and the deadlock theory of Chapter 8 give us the tools and understanding to build concurrent programs. But tools alone are not enough: the way we compose them into data structures determines the performance, scalability, and correctness of concurrent systems. A naively locked hash table that serialises every operation is correct but may be slower than a single-threaded version due to lock contention and cache-line bouncing.

This chapter develops the theory and practice of concurrent data structures. We begin with lock-based designs that improve on coarse-grained locking, then advance to lock-free and wait-free algorithms that eliminate locks entirely. We formalise correctness through linearisability, examine the memory ordering guarantees that underpin these algorithms, and study the memory reclamation problem that arises when pointers are accessed concurrently.

9.1 Lock-Based Data Structures

9.1.1 Coarse-Grained Locking

The simplest approach to making a data structure thread-safe is to protect it with a single lock:

```
#include <pthread.h>
#include <stdlib.h>

typedef struct node {
    int key;
    struct node *next;
```

```
} node_t;

typedef struct {
    node_t *head;
    pthread_mutex_t lock;
} list_t;

void list_init(list_t *l) {
    l->head = NULL;
    pthread_mutex_init(&l->lock, NULL);
}

int list_insert(list_t *l, int key) {
    pthread_mutex_lock(&l->lock);
    node_t *new_node = malloc(sizeof(node_t));
    if (!new_node) {
        pthread_mutex_unlock(&l->lock);
        return -1;
    }
    new_node->key = key;
    new_node->next = l->head;
    l->head = new_node;
    pthread_mutex_unlock(&l->lock);
    return 0;
}

int list_contains(list_t *l, int key) {
    pthread_mutex_lock(&l->lock);
    node_t *curr = l->head;
    while (curr) {
        if (curr->key == key) {
            pthread_mutex_unlock(&l->lock);
            return 1;
        }
        curr = curr->next;
    }
    pthread_mutex_unlock(&l->lock);
    return 0;
}
```

This is correct and simple, but all operations are serialised. If 32 threads perform lookups on a 10,000-element list, only one thread traverses the list at any time. The throughput is no better than single-threaded — and worse, due to lock acquisition overhead.

Definition 9.0b (Throughput and Scalability). The **throughput** of a concurrent data structure is the number of completed operations per unit time. **Scalability** measures how throughput changes as the number of threads increases. A perfectly scalable structure achieves linear speedup: p threads provide p times the throughput of one thread. In practice, contention, cache effects, and synchronisation overhead cause throughput to plateau or even decrease beyond a certain number of threads.

9.1.2 Fine-Grained Locking

Fine-grained locking assigns a separate lock to each node (or to each region of the data structure), allowing multiple threads to operate on different parts simultaneously.

Definition 9.1 (Fine-Grained Locking). A fine-grained locking strategy associates a lock with each independently accessible portion of a data structure. Threads acquire only the locks protecting the portions they access, allowing disjoint operations to proceed in parallel.

For a linked list, the natural granularity is per-node locking:

```c
typedef struct node {
    int key;
    struct node *next;
    pthread_mutex_t lock;
} node_t;

typedef struct {
    node_t *head;  /* sentinel node, always present */
} list_t;

void list_init(list_t *l) {
    l->head = malloc(sizeof(node_t));
    l->head->key = -1;  /* sentinel */
    l->head->next = NULL;
    pthread_mutex_init(&l->head->lock, NULL);
}
```

9.1.3 Hand-Over-Hand Locking (Lock Coupling)

Definition 9.2 (Hand-Over-Hand Locking). Hand-over-hand locking (also called lock coupling) is a traversal strategy for linked structures where a thread holds the lock on the current node while acquiring the lock on the next node, then releases the current node's lock. At any point, the thread holds at most two locks.

```
int list_contains_hoh(list_t *l, int key) {
    node_t *prev = l->head;
    pthread_mutex_lock(&prev->lock);
    node_t *curr = prev->next;

    while (curr != NULL) {
        pthread_mutex_lock(&curr->lock);
        if (curr->key == key) {
            pthread_mutex_unlock(&curr->lock);
            pthread_mutex_unlock(&prev->lock);
            return 1;
        }
        if (curr->key > key) {
            /* list is sorted: key not present */
            pthread_mutex_unlock(&curr->lock);
            pthread_mutex_unlock(&prev->lock);
            return 0;
        }
        pthread_mutex_unlock(&prev->lock);
        prev = curr;
        curr = curr->next;
    }

    pthread_mutex_unlock(&prev->lock);
    return 0;
}

int list_insert_hoh(list_t *l, int key) {
    node_t *prev = l->head;
    pthread_mutex_lock(&prev->lock);
    node_t *curr = prev->next;

    while (curr != NULL) {
        pthread_mutex_lock(&curr->lock);
```

```
        if (curr->key == key) {
            /* already present */
            pthread_mutex_unlock(&curr->lock);
            pthread_mutex_unlock(&prev->lock);
            return 0;
        }
        if (curr->key > key)
            break;  /* insert between prev and curr */
        pthread_mutex_unlock(&prev->lock);
        prev = curr;
        curr = curr->next;
    }

    node_t *new_node = malloc(sizeof(node_t));
    new_node->key = key;
    new_node->next = curr;
    pthread_mutex_init(&new_node->lock, NULL);
    prev->next = new_node;

    if (curr) pthread_mutex_unlock(&curr->lock);
    pthread_mutex_unlock(&prev->lock);
    return 1;
}
```

Hand-over-hand locking allows multiple threads to traverse the list simultaneously, provided they are at different positions. Thread A can be at node 5 while Thread B is at node 50.

Example 9.1 (Concurrency in Hand-Over-Hand Locking). Consider a sorted linked list with nodes $[1, 5, 10, 20, 50, 100]$. Thread A searches for 50 and Thread B inserts 15:

```
Time 1: A locks sentinel, B locks sentinel          (conflict!)
Time 2: A locks node(1), unlocks sentinel
         B locks node(1), unlocks sentinel           (A has moved on)
Time 3: A locks node(5), unlocks node(1)
         B locks node(5), unlocks node(1)
Time 4: A locks node(10), unlocks node(5)
         B locks node(10), unlocks node(5)
Time 5: A locks node(20), unlocks node(10)
         B inserts 15 between node(10) and node(20)   (B is done)
```

```
Time 6: A locks node(50), unlocks node(20)
         A finds 50. Done.
```

The two threads operate concurrently once they separate in the list. The key invariant is that a thread always holds the lock on its current position, preventing another thread from modifying that node.

Performance trade-off: fine-grained locking adds overhead (one lock per node) and complexity (lock ordering to prevent deadlock). For short lists, the overhead exceeds the benefit. For long lists with many concurrent operations on different regions, fine-grained locking provides significant speedups.

Theorem 9.0b (Amdahl's Law for Concurrent Data Structures). If a fraction f of operations on a data structure can proceed concurrently and the remaining fraction $1 - f$ must be serialised, the maximum speedup with p threads is:

$$S(p) = \frac{1}{(1 - f) + f/p}$$

For a coarse-grained locked list, $f = 0$ (all operations serialised), so $S(p) = 1$ regardless of p. For a hand-over-hand locked list with uniform access patterns, $f \approx 1 - 1/L$ where L is the list length (the serialised fraction is the probability that two threads access the same node simultaneously).

Example 9.1b (Scalability Comparison). Measured throughput for a sorted linked list with 10,000 elements, 80% lookups, 10% inserts, 10% deletes:

Threads	Coarse-grained	Hand-over-hand	Optimistic	Lock-free
1	1.0x	0.8x	0.7x	0.9x
2	0.9x	1.4x	1.5x	1.7x
4	0.8x	2.1x	2.8x	3.2x
8	0.7x	2.5x	4.1x	5.8x
16	0.6x	2.3x	5.2x	8.1x

(Throughput relative to single-threaded coarse-grained baseline.)

Key observations: coarse-grained locking degrades beyond 1 thread due to lock contention. Hand-over-hand locking improves but plateaus at 8 threads (lock coupling overhead limits scalability). Optimistic and lock-free approaches scale better because they minimise the time locks are held (or eliminate locks entirely).

9.1.4 Optimistic Locking

Definition 9.3 (Optimistic Locking). In optimistic locking, a thread traverses the data structure without holding any locks, then acquires locks on the relevant nodes and validates that the structure has not changed during the traversal. If validation fails, the thread retries.

This reduces lock hold times: locks are held only during the (short) modification phase, not during the (potentially long) traversal phase. The trade-off is the cost of validation and the possibility of retries under contention.

```
int list_contains_optimistic(list_t *l, int key) {
    while (1) {
        /* Phase 1: lock-free traversal */
        node_t *prev = l->head;
        node_t *curr = prev->next;
        while (curr != NULL && curr->key < key) {
            prev = curr;
            curr = curr->next;
        }

        /* Phase 2: lock and validate */
        pthread_mutex_lock(&prev->lock);
        if (curr) pthread_mutex_lock(&curr->lock);

        if (validate(l, prev, curr)) {
            int found = (curr != NULL && curr->key == key);
            if (curr) pthread_mutex_unlock(&curr->lock);
            pthread_mutex_unlock(&prev->lock);
            return found;
        }

        /* Validation failed: retry */
        if (curr) pthread_mutex_unlock(&curr->lock);
        pthread_mutex_unlock(&prev->lock);
    }
}
```

The validation function checks that `prev` is still reachable from the head and that `prev->next` still points to `curr`.

9.2 Lock-Free Programming

9.2.1 Progress Guarantees

Definition 9.4 (Progress Guarantees). Concurrent algorithms are classified by the progress guarantees they provide:

- **Blocking (lock-based)**: if a thread holding a lock is suspended (by the OS scheduler, a page fault, or a crash), other threads waiting for the lock cannot progress.
- **Obstruction-free**: a thread makes progress if it eventually executes in isolation (no other thread takes steps). This is the weakest non-blocking guarantee.
- **Lock-free**: at least one thread in the system makes progress in a finite number of steps, regardless of the execution speeds or failures of other threads. Individual threads may starve, but the system as a whole always advances.
- **Wait-free**: every thread makes progress in a bounded number of steps, regardless of other threads. No thread can starve.

The hierarchy is: wait-free $\subset$ lock-free $\subset$ obstruction-free $\subset$ blocking-free.

Theorem 9.1 (Lock-Free via CAS Loops). Any sequential data structure operation can be made lock-free using a compare-and-swap loop: read the current state, compute the new state, and attempt to install it with CAS. If CAS fails (another thread modified the state), retry.

Proof sketch. Each CAS failure implies another thread's CAS succeeded (made progress). Therefore, if thread T fails k times, at least k other operations completed. The system makes progress at a rate of at least one operation per step across all threads. □

9.2.2 CAS Loops

The fundamental pattern of lock-free programming is the **CAS loop** (also called the **retry loop** or **optimistic update**):

```
#include <stdatomic.h>

/* Lock-free counter increment */
void atomic_increment(atomic_int *counter) {
    int old_val, new_val;
    do {
```

```
        old_val = atomic_load(counter);
        new_val = old_val + 1;
    } while (!atomic_compare_exchange_weak(counter, &old_val, new_val));
}
```

This pattern generalises to arbitrary transformations:

```
/* Lock-free stack push */
typedef struct stack_node {
    int value;
    struct stack_node *next;
} stack_node_t;

typedef struct {
    _Atomic(stack_node_t *) top;
} lock_free_stack_t;

void stack_push(lock_free_stack_t *s, int value) {
    stack_node_t *new_node = malloc(sizeof(stack_node_t));
    new_node->value = value;
    stack_node_t *old_top;
    do {
        old_top = atomic_load(&s->top);
        new_node->next = old_top;
    } while (!atomic_compare_exchange_weak(&s->top, &old_top, new_node));
}

int stack_pop(lock_free_stack_t *s, int *value) {
    stack_node_t *old_top;
    stack_node_t *new_top;
    do {
        old_top = atomic_load(&s->top);
        if (old_top == NULL) return 0;  /* empty */
        new_top = old_top->next;
    } while (!atomic_compare_exchange_weak(&s->top, &old_top, new_top));
    *value = old_top->value;
    /* Cannot free old_top here! See Section 9.5 */
    return 1;
}
```

9.2.2b Correctness of the Lock-Free Stack

Theorem 9.1b (Treiber Stack Correctness). The lock-free stack (Treiber, 1986) satisfies:

1. **Linearisability**: each push operation is linearised at its successful CAS on `top`; each pop is linearised at its successful CAS on `top`.
2. **Lock-freedom**: in any execution where some thread is attempting a push or pop, at least one thread completes its operation in a finite number of steps.

Proof of lock-freedom. Consider a thread T that fails its CAS during push or pop. The CAS fails because another thread successfully modified `top` — that other thread made progress (either completed a push or a pop). Therefore, every CAS failure implies another thread's success. In any finite execution with k CAS failures across all threads, at least k operations completed. The system is never stuck. □

Note that the lock-free stack is **not** wait-free: a single thread's push can be delayed indefinitely if other threads keep modifying `top` between the thread's load and CAS. In practice, sustained starvation of a single thread is extremely unlikely due to the non-determinism of thread scheduling, but it is theoretically possible.

9.2.3 The ABA Problem

Definition 9.5 (ABA Problem). The ABA problem occurs when a CAS operation succeeds spuriously because the target location's value changed from A to B and back to A between the load and the CAS. The CAS sees value A and assumes nothing has changed, but the underlying state may have been modified.

Example 9.2 (ABA in a Lock-Free Stack). Consider a lock-free stack with elements $[A \to B \to C]$ (top is A):

```
Thread 1: old_top = A, new_top = B          (prepares to pop A)
          [suspended before CAS]

Thread 2: pops A (stack becomes [B -> C])
Thread 2: pops B (stack becomes [C])
Thread 2: pushes A back (stack becomes [A -> C])

Thread 1: [resumes] CAS(&top, A, B)
          Succeeds! (top is indeed A)
          Stack becomes [B -> ...]
          But B was already freed by Thread 2!
```

Thread 1's CAS succeeds because `top` is `A`, but the stack's structure has changed: `A-`

`>next` no longer points to B. The result is a corrupted stack with a dangling pointer.

Solutions to the ABA problem:

1. **Tagged pointers (version counters)**: pair each pointer with a monotonically increasing counter. CAS operates on the (pointer, counter) pair. Even if the pointer returns to A, the counter will have changed. On 64-bit systems, the upper 16 bits of a pointer (unused in current architectures with 48-bit virtual addresses) can store the counter.
2. **Hazard pointers**: protect nodes from being freed while any thread holds a reference (Section 9.5).
3. **Epoch-based reclamation**: defer freeing nodes until all threads have passed through a quiescent point (Section 9.5).

```c
/* Tagged pointer: pack pointer and counter into 64 bits */
typedef struct {
    stack_node_t *ptr;
    unsigned int tag;
} tagged_ptr_t;

/* On x86-64, use CMPXCHG16B for 128-bit CAS (double-width CAS) */
/* Or use the upper bits of the pointer as the tag */
typedef union {
    struct {
        stack_node_t *ptr;
        uint64_t tag;
    };
    __int128 combined;  /* for atomic 128-bit CAS */
} tagged_ptr_u;
```

Programmer's Perspective

Go's `sync/atomic` package provides `atomic.CompareAndSwapPointer` but does not support double-width CAS directly. However, Go's garbage collector eliminates the ABA problem for most use cases: a pointer to an object A remains valid as long as any goroutine holds a reference to it. The GC will not free (and therefore will not reallocate) the memory at address A while references exist. This is a significant advantage of garbage-collected languages for lock-free programming — an entire class of bugs (ABA, use-after-free, dangling pointers) is eliminated by the runtime. The trade-off is GC pause latency and reduced control over memory layout.

9.3 Wait-Free Data Structures

9.3.1 Herlihy's Universal Construction

Theorem 9.2 (Herlihy's Universality Result, 1991). Any sequential data structure can be transformed into a wait-free concurrent data structure using compare-and-swap (or any object with consensus number $\geq n$ for n threads). The construction guarantees that every operation completes in $O(n)$ steps, where n is the number of threads.

The construction works by maintaining the data structure as an immutable log of operations. Each thread:

1. Creates a node describing its operation.
2. Appends it to a shared linked list using CAS.
3. Replays the log from the last known state to compute the result.
4. If the CAS fails (another thread appended first), it helps the other thread complete its operation before retrying its own.

Definition 9.6 (Helping). In a wait-free algorithm, a thread that detects another thread's pending operation may **help** it complete. Helping ensures that no thread can be blocked indefinitely: even if a thread is suspended, other threads will complete its operation on its behalf.

```c
/* Simplified universal construction (pseudocode) */
typedef struct op_node {
    int (*operation)(void *state, void *arg);
    void *arg;
    _Atomic(int) result;
    _Atomic(int) done;
    _Atomic(struct op_node *) next;
} op_node_t;

typedef struct {
    _Atomic(op_node_t *) tail;
    void *state;  /* the sequential data structure */
} universal_t;

int apply(universal_t *u, op_node_t *my_op) {
    my_op->done = 0;
```

```
    my_op->next = NULL;

    /* Append to log */
    while (1) {
        op_node_t *last = atomic_load(&u->tail);
        op_node_t *expected = NULL;
        if (atomic_compare_exchange_strong(&last->next, &expected, my_op)) {
            atomic_compare_exchange_strong(&u->tail, &last, my_op);
            break;
        } else {
            /* Help the other thread advance the tail */
            atomic_compare_exchange_strong(&u->tail, &last, expected);
        }
    }

    /* Execute all pending operations in log order */
    /* (details omitted for brevity) */

    return my_op->result;
}
```

Example 9.3 (Wait-Free Counter). A wait-free counter for n threads can be implemented by giving each thread its own counter cell. The `increment` operation atomically increments the local cell (a single atomic write, always $O(1)$). The `read` operation sums all cells (an $O(n)$ operation, but wait-free since it reads each cell exactly once):

```
#include <stdatomic.h>

#define MAX_THREADS 64

typedef struct {
    _Alignas(64) atomic_long count;  /* one cache line per thread */
} counter_cell_t;

typedef struct {
    counter_cell_t cells[MAX_THREADS];
    int num_threads;
} wf_counter_t;

void wf_increment(wf_counter_t *c, int thread_id) {
```

```
    atomic_fetch_add(&c->cells[thread_id].count, 1);
    /* Wait-free: exactly one atomic operation, no retry */
}

long wf_read(wf_counter_t *c) {
    long total = 0;
    for (int i = 0; i < c->num_threads; i++)
        total += atomic_load(&c->cells[i].count);
    return total;
    /* Wait-free: exactly n atomic loads, no retry */
}
```

Note the `_Alignas(64)` to place each cell on its own cache line, avoiding false sharing (where atomic operations on different cells invalidate each other's cache lines because they happen to share a 64-byte cache line).

Definition 9.6b (False Sharing). False sharing occurs when two or more threads access different variables that reside on the same cache line. Even though the variables are logically independent, atomic operations on one variable invalidate the cache line for all threads accessing other variables on the same line, causing unnecessary cache coherence traffic. Padding variables to cache-line boundaries (typically 64 bytes) eliminates false sharing.

9.3.2 Practical Considerations

Herlihy's universal construction proves that wait-free implementations exist for any data structure, but the resulting algorithms are typically impractical: the overhead of logging, helping, and replaying operations dominates the actual work. In practice, most concurrent data structures use **lock-free** algorithms (which allow individual starvation but guarantee system-wide progress) because they are simpler and faster.

Wait-free algorithms are used in specialised domains:

- **Real-time systems**: where worst-case bounds on operation latency are required.
- **Hard real-time embedded systems**: where no operation may take more than a fixed number of steps.
- **Read-heavy workloads**: where wait-free reads (as in the counter example) are the common case.

9.3.3 The Price of Wait-Freedom

Theorem 9.2b (Wait-Free Overhead). Any wait-free implementation of a concurrent object from CAS has worst-case step complexity $\Omega(n)$ per operation, where n is the number of threads. This is because the helping mechanism requires each thread to examine the state of all other threads to ensure no thread is left behind.

This $\Omega(n)$ lower bound explains why wait-free algorithms are typically slower than lock-free alternatives for small numbers of threads. The overhead of helping is worthwhile only when absolute worst-case guarantees are required, or when the number of threads is small enough that the helping overhead is acceptable.

In practice, the progression of concurrent algorithm design is:

1. **Coarse-grained locking** (simplest, lowest concurrency)
2. **Fine-grained locking** (more complex, better concurrency)
3. **Lock-free** (CAS-based, system-wide progress guarantee)
4. **Wait-free** (per-thread progress guarantee, highest complexity and overhead)

Most production systems stop at step 3 (lock-free). Lock-free algorithms provide excellent practical performance and system-wide progress, and the theoretical possibility of individual starvation almost never occurs in practice because OS schedulers ensure fair CPU time distribution.

9.4 The Michael-Scott Lock-Free Queue

The Michael-Scott queue (1996) is the most widely used lock-free queue algorithm. It is used in the Java `ConcurrentLinkedQueue`, the .NET `ConcurrentQueue`, and forms the basis of many message-passing systems.

9.4.1 Structure

```c
#include <stdatomic.h>
#include <stdlib.h>

typedef struct queue_node {
    int value;
    _Atomic(struct queue_node *) next;
} queue_node_t;

typedef struct {
```

```
    _Atomic(queue_node_t *) head;
    _Atomic(queue_node_t *) tail;
} ms_queue_t;

void ms_queue_init(ms_queue_t *q) {
    queue_node_t *sentinel = malloc(sizeof(queue_node_t));
    sentinel->next = NULL;
    q->head = sentinel;
    q->tail = sentinel;
}
```

The queue uses a sentinel (dummy) node: the head always points to the sentinel, and the first real element is `head->next`. This eliminates special cases for empty queues.

9.4.2 Enqueue

```
void ms_enqueue(ms_queue_t *q, int value) {
    queue_node_t *new_node = malloc(sizeof(queue_node_t));
    new_node->value = value;
    atomic_store(&new_node->next, NULL);

    while (1) {
        queue_node_t *tail = atomic_load(&q->tail);
        queue_node_t *next = atomic_load(&tail->next);

        if (tail == atomic_load(&q->tail)) {  /* consistency check */
            if (next == NULL) {
                /* Tail is pointing to the last node */
                if (atomic_compare_exchange_weak(&tail->next,
                                                 &next, new_node)) {
                    /* Enqueue succeeded; try to advance tail */
                    atomic_compare_exchange_weak(&q->tail, &tail, new_node);
                    return;
                }
            } else {
                /* Tail is lagging; help advance it */
                atomic_compare_exchange_weak(&q->tail, &tail, next);
            }
        }
    }
}
```

Definition 9.7 (Helping in the Michael-Scott Queue). The enqueue operation exhibits cooperative helping: if a thread observes that `tail->next` is not null (meaning another thread has appended a node but has not yet updated `tail`), it helps advance `tail` before attempting its own enqueue. This ensures that `tail` never falls more than one node behind the actual end of the queue, and it guarantees lock-freedom.

9.4.3 Dequeue

```
int ms_dequeue(ms_queue_t *q, int *value) {
    while (1) {
        queue_node_t *head = atomic_load(&q->head);
        queue_node_t *tail = atomic_load(&q->tail);
        queue_node_t *next = atomic_load(&head->next);

        if (head == atomic_load(&q->head)) {  /* consistency check */
            if (head == tail) {
                if (next == NULL) {
                    return 0;  /* queue is empty */
                }
                /* Tail lagging; advance it */
                atomic_compare_exchange_weak(&q->tail, &tail, next);
            } else {
                /* Read value before CAS, in case another
                   dequeue frees the node */
                *value = next->value;
                if (atomic_compare_exchange_weak(&q->head,
                                                 &head, next)) {
                    /* Dequeue succeeded */
                    /* free(head); -- NOT SAFE! See Section 9.5 */
                    return 1;
                }
            }
        }
    }
}
```

Theorem 9.3 (Michael-Scott Queue is Lock-Free). The Michael-Scott queue's enqueue and dequeue operations are lock-free.

Proof. Consider any execution where some thread T is attempting an enqueue or dequeue. If T's CAS fails, it means another thread's CAS on the same location suc-

ceeded, which means that other thread made progress (either advanced `tail`, appended a node, or dequeued a node). In any finite interval where threads take steps, at least one CAS succeeds, so at least one operation completes. Therefore, the algorithm is lock-free. □

Example 9.4 (Michael-Scott Queue Interleaving). Consider two threads simultaneously enqueuing values 10 and 20 into a queue containing [sentinel]:

```
Initial: sentinel.next = NULL, tail = sentinel

Thread A: new_node(10), reads tail = sentinel, next = NULL
Thread B: new_node(20), reads tail = sentinel, next = NULL

Thread A: CAS(sentinel.next, NULL, node(10))  -- SUCCESS
          Queue: sentinel -> 10
          CAS(tail, sentinel, node(10))        -- SUCCESS
          tail = node(10)

Thread B: CAS(sentinel.next, NULL, node(20))  -- FAILS (next is now node(10))
          Reads next = node(10)
          next != NULL, so helps: CAS(tail, sentinel, node(10))
          (may fail: tail already advanced by Thread A)
          Retries: reads tail = node(10), next = NULL
          CAS(node(10).next, NULL, node(20))   -- SUCCESS
          Queue: sentinel -> 10 -> 20
          CAS(tail, node(10), node(20))        -- SUCCESS
```

Both operations complete. The queue is linearisable: the operations appear to take effect in the order $10, 20$.

9.5 Memory Reclamation

9.5.1 The Problem

In lock-free data structures, a fundamental challenge arises: when a node is removed from the structure, it cannot be immediately freed because other threads may still hold references to it. In the Michael-Scott queue, after a dequeue advances `head` past a node, another thread in the middle of a dequeue may still be reading that node's `next` pointer.

Definition 9.8 (Safe Memory Reclamation). The memory reclamation problem

for lock-free data structures asks: when is it safe to free (or reuse) a node that has been removed from a concurrent data structure? The node can be freed only when no thread holds a reference to it and no thread will access it in the future.

9.5.2 Hazard Pointers

Definition 9.9 (Hazard Pointers, Michael 2004). Each thread maintains a small set of **hazard pointers** — shared variables that advertise which nodes the thread is currently accessing. Before accessing a node, a thread publishes the node's address in its hazard pointer. Before freeing a node, a thread checks all hazard pointers across all threads; if any hazard pointer references the node, the free is deferred.

```
#include <stdatomic.h>

#define MAX_THREADS 64
#define HP_PER_THREAD 2
#define RETIRE_THRESHOLD 64

typedef struct {
    _Atomic(void *) hp[HP_PER_THREAD];
} hp_record_t;

hp_record_t hp_records[MAX_THREADS];

/* Thread-local retired list */
typedef struct {
    void *nodes[RETIRE_THRESHOLD * 2];
    int count;
} retired_list_t;

__thread retired_list_t retired = {.count = 0};

void hp_protect(int thread_id, int slot, void *ptr) {
    atomic_store(&hp_records[thread_id].hp[slot], ptr);
}

void hp_clear(int thread_id, int slot) {
    atomic_store(&hp_records[thread_id].hp[slot], NULL);
}

void hp_retire(int thread_id, void *ptr, int num_threads) {
```

```
    retired.nodes[retired.count++] = ptr;

    if (retired.count >= RETIRE_THRESHOLD) {
        /* Scan all hazard pointers */
        void *protected_ptrs[MAX_THREADS * HP_PER_THREAD];
        int pcount = 0;
        for (int t = 0; t < num_threads; t++) {
            for (int s = 0; s < HP_PER_THREAD; s++) {
                void *p = atomic_load(&hp_records[t].hp[s]);
                if (p) protected_ptrs[pcount++] = p;
            }
        }

        /* Try to free retired nodes not in the protected set */
        int new_count = 0;
        for (int i = 0; i < retired.count; i++) {
            int is_protected = 0;
            for (int j = 0; j < pcount; j++) {
                if (retired.nodes[i] == protected_ptrs[j]) {
                    is_protected = 1;
                    break;
                }
            }
            if (is_protected) {
                retired.nodes[new_count++] = retired.nodes[i];
            } else {
                free(retired.nodes[i]);
            }
        }
        retired.count = new_count;
    }
}
```

Using hazard pointers in the Michael-Scott dequeue:

```
int ms_dequeue_hp(ms_queue_t *q, int *value, int tid) {
    while (1) {
        queue_node_t *head = atomic_load(&q->head);
        hp_protect(tid, 0, head);  /* protect head */
        if (head != atomic_load(&q->head)) continue;  /* validate */
```

```
        queue_node_t *tail = atomic_load(&q->tail);
        queue_node_t *next = atomic_load(&head->next);
        hp_protect(tid, 1, next);  /* protect next */
        if (head != atomic_load(&q->head)) continue;  /* validate */

        if (head == tail) {
            if (next == NULL) {
                hp_clear(tid, 0);
                hp_clear(tid, 1);
                return 0;  /* empty */
            }
            atomic_compare_exchange_weak(&q->tail, &tail, next);
        } else {
            *value = next->value;
            if (atomic_compare_exchange_weak(&q->head, &head, next)) {
                hp_clear(tid, 0);
                hp_clear(tid, 1);
                hp_retire(tid, head, MAX_THREADS);
                return 1;
            }
        }
    }
}
```

9.5.3 Epoch-Based Reclamation

Definition 9.10 (Epoch-Based Reclamation, Fraser 2004). The system maintains a global epoch counter. Each thread records the epoch it observed when it last entered a critical region. Retired nodes are tagged with the epoch in which they were retired. A retired node from epoch e can be freed when all threads have observed an epoch greater than e — meaning no thread can hold a reference from epoch e or earlier.

```
#include <stdatomic.h>
#include <stdlib.h>

#define MAX_THREADS 64
#define NUM_EPOCHS 3

typedef struct retire_node {
    void *ptr;
```

```
    struct retire_node *next;
} retire_node_t;

typedef struct {
    atomic_int global_epoch;
    _Atomic(int) thread_epochs[MAX_THREADS];
    _Atomic(int) thread_active[MAX_THREADS];
    retire_node_t *retire_lists[MAX_THREADS][NUM_EPOCHS];
} epoch_t;

void epoch_enter(epoch_t *e, int tid) {
    int ge = atomic_load(&e->global_epoch);
    atomic_store(&e->thread_epochs[tid], ge);
    atomic_store(&e->thread_active[tid], 1);
    atomic_thread_fence(memory_order_seq_cst);
}

void epoch_exit(epoch_t *e, int tid) {
    atomic_store(&e->thread_active[tid], 0);
}

void epoch_retire(epoch_t *e, int tid, void *ptr) {
    int ge = atomic_load(&e->global_epoch);
    retire_node_t *node = malloc(sizeof(retire_node_t));
    node->ptr = ptr;
    node->next = e->retire_lists[tid][ge % NUM_EPOCHS];
    e->retire_lists[tid][ge % NUM_EPOCHS] = node;

    /* Try to advance the global epoch */
    int can_advance = 1;
    for (int t = 0; t < MAX_THREADS; t++) {
        if (atomic_load(&e->thread_active[t]) &&
            atomic_load(&e->thread_epochs[t]) != ge) {
            can_advance = 0;
            break;
        }
    }

    if (can_advance) {
        int new_epoch = ge + 1;
```

```
        if (atomic_compare_exchange_strong(&e->global_epoch,
                                           &ge, new_epoch)) {
            /* Free all nodes from two epochs ago */
            int old_epoch = (new_epoch - 2 + NUM_EPOCHS) % NUM_EPOCHS;
            for (int t = 0; t < MAX_THREADS; t++) {
                retire_node_t *list = e->retire_lists[t][old_epoch];
                while (list) {
                    retire_node_t *tmp = list;
                    list = list->next;
                    free(tmp->ptr);
                    free(tmp);
                }
                e->retire_lists[t][old_epoch] = NULL;
            }
        }
    }
}
```

Example 9.5 (Epoch Advancement). With 3 threads and global epoch at 5:

```
Time 1: Thread 0 enters (epoch 5), Thread 1 enters (epoch 5)
Time 2: Thread 0 retires node X (tagged epoch 5)
Time 3: Thread 1 exits, Thread 2 enters (epoch 5)
Time 4: Thread 0 exits, Thread 1 enters (epoch 5)
Time 5: All threads have seen epoch 5 -> advance to epoch 6
Time 6: Thread 2 exits, enters (epoch 6)
Time 7: All threads have seen epoch 6 -> advance to epoch 7
         Now safe to free nodes retired in epoch 5 (node X)
```

The two-epoch grace period ensures that no thread can hold a reference to a node from two or more epochs ago.

Comparison:

Property	Hazard Pointers	Epoch-Based
Memory overhead	$O(T \cdot K)$ pointers (T threads, K hazard pointers per thread)	$O(T \cdot R)$ retired nodes (R = max retire rate between epochs)
Reclamation latency	Per-scan (bounded by $T \cdot K$)	Per-epoch (may accumulate if a thread stalls)

Property	Hazard Pointers	Epoch-Based
Stalled thread	Does not delay reclamation of unprotected nodes	Delays all reclamation (epoch cannot advance)
Implementation complexity	Moderate	Lower

9.5.4 Choosing a Reclamation Strategy

The choice between hazard pointers and epoch-based reclamation depends on the application's requirements:

- **Use hazard pointers** when threads may be preempted for long periods (e.g., in a system with oversubscription, where more threads than cores exist). A stalled thread's hazard pointers protect only the specific nodes it references, so reclamation of other nodes proceeds normally.
- **Use epoch-based reclamation** when threads are cooperative and make frequent progress (e.g., in a database engine where worker threads are never preempted mid-operation). The simpler implementation and lower per-operation overhead outweigh the risk of delayed reclamation.
- **Use garbage collection** (as in Go, Java, C#) when the language provides it. GC eliminates the entire reclamation problem at the cost of pause latency and reduced memory control. For most applications, this is the right trade-off.

Example 9.5b (Reclamation in Practice). Major systems and their reclamation strategies:

System	Language	Reclamation	Reason
Linux kernel RCU	C	Epoch-based (read-copy-update)	Threads are kernel threads; quiescent detection is natural
Java ConcurrentHashMap	Java	Garbage collection	JVM provides GC
Go sync.Map	Go	Garbage collection	Go GC handles it

System	Language	Reclamation	Reason
Folly ConcurrentHashMap	C++	Hazard pointers	Long-lived threads may be preempted
crossbeam (Rust)	Rust	Epoch-based	Cooperative threads in async runtime

---

9.6 Memory Ordering

9.6.1 The Need for Memory Ordering

Modern processors and compilers reorder memory operations for performance. A store followed by a load to a different address may execute in the reverse order (the store sits in a write buffer while the load proceeds from cache). Without explicit ordering constraints, concurrent algorithms that depend on the order of memory accesses may break.

Definition 9.11 (Memory Model). A memory model specifies the set of allowable orderings of memory operations as observed by different threads. It defines which reorderings the hardware and compiler are permitted to perform, and which guarantees the programmer can rely on.

9.6.2 Sequential Consistency

Definition 9.12 (Sequential Consistency, Lamport 1979). A multiprocessor system is sequentially consistent if the result of any execution is the same as if the operations of all processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.

Sequential consistency (SC) is the most intuitive model: it behaves as if there is a single shared memory and threads take turns executing one operation at a time. However, SC is expensive to implement because it forbids most hardware and compiler optimisations.

Example 9.6 (Sequential Consistency Violation). Consider two threads:

```
/* Initially: x = 0, y = 0 */

/* Thread 1 */          /* Thread 2 */
x = 1;                  y = 1;
r1 = y;                 r2 = x;
```

Under sequential consistency, the possible outcomes are:

Interleaving	r1	r2
T1, T1, T2, T2	0	1
T1, T2, T1, T2	1	1
T1, T2, T2, T1	1	1
T2, T2, T1, T1	1	0
T2, T1, T2, T1	1	1
T2, T1, T1, T2	1	1

The outcome $r_1 = 0, r_2 = 0$ is **impossible** under SC: it would require both stores to occur after both loads, which contradicts program order.

However, on x86 (TSO), this outcome **is** possible: both stores may sit in write buffers while both loads read from cache (which still holds 0). This is the store-buffer reordering allowed by TSO.

9.6.3 Acquire-Release Semantics

Definition 9.13 (Acquire and Release). In the acquire-release memory model:

- An **acquire** operation (e.g., a load with acquire semantics) ensures that no subsequent memory operation (in program order) can be reordered before it. It acts as a one-way barrier: operations can move down past it, but not up.
- A **release** operation (e.g., a store with release semantics) ensures that no preceding memory operation (in program order) can be reordered after it. It acts as a one-way barrier: operations can move up past it, but not down.

When a release store in thread A is observed by an acquire load in thread B (they read the same value), a **synchronises-with** relationship is established: all memory operations before A's release are visible to all memory operations after B's acquire.

```c
#include <stdatomic.h>

int data = 0;
atomic_int flag = 0;

/* Thread 1 (producer) */
void producer(void) {
    data = 42;                                          /* ordinary store */
    atomic_store_explicit(&flag, 1, memory_order_release);  /* release */
}

/* Thread 2 (consumer) */
void consumer(void) {
    while (atomic_load_explicit(&flag, memory_order_acquire) == 0)
        ;  /* spin with acquire */
    /* data is guaranteed to be 42 here */
    printf("%d\n", data);
}
```

The release store of `flag = 1` by Thread 1 synchronises with the acquire load that reads `flag == 1` by Thread 2. This guarantees that Thread 2 sees `data = 42` — the store to `data` cannot be reordered past the release store.

9.6.3b Hardware Memory Models

Different processor architectures provide different memory ordering guarantees:

Definition 9.13b (Total Store Order, TSO). x86 processors implement Total Store Order (TSO): stores from a single core are seen by all other cores in program order. The only reordering permitted is a store followed by a load to a different address: the load may execute before the store becomes globally visible (due to the store buffer). All other orderings (store-store, load-load, load-store) are preserved.

Definition 9.13c (Weak Ordering). ARM and RISC-V processors implement weak ordering: both loads and stores may be reordered with respect to each other, subject to data dependencies. Explicit barrier instructions (`DMB`, `DSB` on ARM; `FENCE` on RISC-V) are required to enforce specific orderings. This provides maximum optimisation freedom for the hardware at the cost of requiring explicit programmer annotation.

The practical consequence is that lock-free algorithms that work correctly on x86 (TSO) may fail on ARM (weak ordering) because ARM permits more reorderings. The C11/C++11 memory model abstracts over these hardware differences: the program-

mer specifies the required ordering (acquire, release, seq_cst), and the compiler inserts the appropriate barriers for each target architecture.

Example 9.6b (Platform-Specific Barrier Costs).

Memory order	x86 (TSO)	ARM	RISC-V
Relaxed load	`MOV` (free)	`LDR` (free)	`LW` (free)
Acquire load	`MOV` (free, TSO provides it)	`LDAR` or `LDR + DMB`	`LW + FENCE`
Release store	`MOV` (free, TSO provides it)	`STLR` or `DMB + STR`	`FENCE + SW`
Seq_cst store	`MOV + MFENCE` or `LOCK XCHG`	`STLR + DMB ISH`	`FENCE + SW + FENCE`

On x86, acquire and release are free (TSO already provides them). Seq_cst is the only ordering that requires an explicit fence. On ARM, every ordering above relaxed requires explicit barrier instructions. This is why weakening memory orderings from seq_cst to acquire-release provides significant performance improvements on ARM but negligible improvement on x86.

9.6.4 Relaxed Ordering

Definition 9.14 (Relaxed Memory Order). A relaxed atomic operation guarantees only atomicity (no torn reads/writes) but provides no ordering constraints with respect to other memory operations. Relaxed operations are useful for statistics counters and other cases where the exact ordering of updates is irrelevant.

```c
/* Relaxed counter: counts are eventually consistent but always accurate */
atomic_int total_requests = 0;

void handle_request(void) {
    /* Relaxed: we only care about the final count, not ordering */
    atomic_fetch_add_explicit(&total_requests, 1, memory_order_relaxed);
}
```

9.6.5 Memory Order Hierarchy

The C11/C++11 standard defines six memory orderings, forming a hierarchy from weakest to strongest:

Ordering	Guarantees	Cost
`memory_order_relaxed`	Atomicity only	Cheapest (no fences)
`memory_order_consume`	Data-dependent ordering (deprecated in practice)	
`memory_order_acquire`	No reordering of subsequent ops before this load	Load fence
`memory_order_release`	No reordering of preceding ops after this store	Store fence
`memory_order_acq_rel`	Both acquire and release (for read-modify-write)	Both fences
`memory_order_seq_cst`	Total order across all seq_cst operations	Full fence

Theorem 9.4 (Sequential Consistency of Default Atomics). All C11/C++11 atomic operations default to `memory_order_seq_cst`. A program that uses only default atomic operations on all shared variables is sequentially consistent.

Proof. The C11 standard defines that `memory_order_seq_cst` operations participate in a single total order S consistent with the modification order of each atomic object and consistent with the "happens-before" relation. Since all operations are `seq_cst`, the total order S is a valid sequential execution. □

Programmer's Perspective

Go's `sync/atomic` package provides sequentially consistent operations only — there is no way to specify relaxed or acquire-release ordering. Every `atomic.LoadInt64`, `atomic.StoreInt64`, and `atomic.CompareAndSwapInt64` provides full sequential consistency. This is a deliberate design choice: the Go team prioritises correctness over the marginal performance gains of weaker orderings. The Go memory model (documented at `go.dev/ref/mem`) states that atomic operations behave as `sync.Mutex` acquire/release pairs, creating happens-before edges.

For performance-critical code that needs weaker ordering, Go programmers can use `unsafe.Pointer` with careful reasoning, but this is strongly discouraged. In practice, the overhead of seq_cst atomics on x86 (which has a naturally strong memory model) is negligible — the difference matters mainly on ARM and POWER architectures, where explicit barriers are needed for seq_cst but not for acquire-release.

```go
package main

import (
    "fmt"
    "sync/atomic"
)

func main() {
    var counter int64

    // All atomic operations are sequentially consistent in Go
    atomic.StoreInt64(&counter, 0)
    atomic.AddInt64(&counter, 1)
    val := atomic.LoadInt64(&counter)
    fmt.Println(val) // 1

    // CompareAndSwap: the foundation of lock-free programming
    swapped := atomic.CompareAndSwapInt64(&counter, 1, 42)
    fmt.Println(swapped, atomic.LoadInt64(&counter)) // true 42
}
```

9.7 The C11/C++11 Memory Model

9.7.1 Formalisation

Definition 9.15 (Happens-Before Relation). The happens-before relation $\xrightarrow{hb}$ is a partial order on memory operations defined as the transitive closure of:

1. **Sequenced-before** ($\xrightarrow{sb}$): if A appears before B in the same thread's program, then $A \xrightarrow{sb} B$.
2. **Synchronises-with** ($\xrightarrow{sw}$): if A is a release operation on atomic variable x and B is an acquire operation on x that reads the value written by A, then $A \xrightarrow{sw} B$.

If $A \xrightarrow{hb} B$, then A's effects are visible to B.

Definition 9.16 (Data Race). A data race occurs when two memory operations access the same non-atomic memory location, at least one is a write, and there is no happens-before relation between them. The C11/C++11 standards declare that programs with data races have **undefined behaviour**.

Theorem 9.5 (DRF-SC Guarantee). The C11/C++11 memory model provides the DRF-SC (Data-Race-Free implies Sequential Consistency) guarantee: if a program is free of data races (all conflicting accesses are ordered by happens-before), then its behaviour is sequentially consistent.

Proof sketch. In a data-race-free program, every pair of conflicting accesses is ordered by $\xrightarrow{hb}$. The happens-before relation is consistent with program order and with the synchronisation order. Any sequentially consistent execution respects these orderings. Since every pair of conflicting accesses is ordered, the interleaving of operations across threads cannot produce results that differ from some sequential interleaving. □

9.7.2 Modification Order and Coherence

Definition 9.16b (Modification Order). For each atomic variable x, the C11 memory model defines a **modification order** M_x: a total order on all stores to x. All threads agree on this order. This ensures **coherence**: if thread A sees store S_1 to x before store S_2, then no thread can see S_2 before S_1.

Coherence is weaker than sequential consistency: it applies per-variable, not globally. Two threads may see stores to different variables in different orders, as long as they agree on the order of stores to each individual variable.

Example 9.7b (Coherence vs Sequential Consistency). Consider:

```
atomic_int x = 0, y = 0;

/* Thread 1 */                /* Thread 2 */
x = 1;                        y = 1;

/* Thread 3 */                /* Thread 4 */
r1 = x; /* sees 1 */         r3 = y; /* sees 1 */
r2 = y; /* sees 0 */         r4 = x; /* sees 0 */
```

Under sequential consistency, if Thread 3 sees $x = 1$ before $y = 1$, then Thread 4 must also see $x = 1$ before $y = 1$ (there is a single global order). The outcome $r_1 = 1, r_2 = 0, r_3 = 1, r_4 = 0$ is impossible.

Under coherence (with relaxed atomics), each variable has its own order, but there is no global order across variables. Thread 3 may see x updated before y, while Thread 4 sees y updated before x. The outcome above is permitted.

This is why `memory_order_seq_cst` is the default: it provides the intuitive global ordering that programmers expect. Weakening to acquire-release sacrifices cross-variable ordering but preserves the happens-before relationship needed for correct synchronisation patterns.

9.7.3 Practical Implications

The DRF-SC guarantee is the foundation of correct concurrent programming in C and C++:

1. **Use atomic types for shared variables** (`atomic_int`, `_Atomic(int)`, `std::atomic<int>`).
2. **Use mutexes for compound operations** that involve multiple variables.
3. **Default to `memory_order_seq_cst`** (the default) and weaken only with careful reasoning and testing.
4. **Use tools**: ThreadSanitizer (TSan) detects data races dynamically.

Example 9.7 (Compiler Reordering). Without atomics, the compiler may reorder stores:

```
/* Thread 1 */
data = 42;    /* may be reordered after flag = 1 by the compiler */
flag = 1;

/* Thread 2 */
while (flag == 0) ;
printf("%d\n", data);  /* may print 0! */
```

With atomics:

```
#include <stdatomic.h>

int data = 0;
atomic_int flag = 0;

/* Thread 1 */
data = 42;
atomic_store(&flag, 1);  /* seq_cst: cannot be reordered before data = 42 */

/* Thread 2 */
while (atomic_load(&flag) == 0) ;
printf("%d\n", data);  /* always prints 42 */
```

The atomic store acts as a compiler and hardware barrier, ensuring the store to `data` is visible before the store to `flag`.

9.8 Linearisability

9.8.1 Definition

Definition 9.17 (Linearisability, Herlihy and Wing 1990). A concurrent execution is **linearisable** if each operation appears to take effect instantaneously at some point (the **linearisation point**) between its invocation and its response, and the resulting sequential history is consistent with the sequential specification of the data structure.

Formally: an execution history H is linearisable with respect to a sequential specification S if H can be extended (by completing pending operations or removing them) to a history H' such that:

1. H' is equivalent to a legal sequential history σ.
2. If operation o_1 completes before operation o_2 begins in H, then o_1 appears before o_2 in σ.

Example 9.8 (Linearisable vs Non-Linearisable). Consider a concurrent queue with enqueue (E) and dequeue (D) operations:

Linearisable execution:

```
Thread A: |--- E(1) ---|
```

```
Thread B:        |--- E(2) ---|
Thread C:                          |--- D() -> 1 ---|
Thread D:                                                |--- D() -> 2 ---|
```

Linearisation points: $E(1)$ before $E(2)$ before $D() \to 1$ before $D() \to 2$. Sequential history: $E(1), E(2), D() \to 1, D() \to 2$. Valid FIFO queue behaviour.

Non-linearisable execution:

```
Thread A: |--- E(1) ---|
Thread B:                    |--- E(2) ---|
Thread C:                                          |--- D() -> 2 ---|
```

$E(1)$ completes before $E(2)$ begins, so $E(1)$ must precede $E(2)$ in any linearisation. But $D() \to 2$ dequeues 2 before 1, violating FIFO order. This is not linearisable.

9.8.2 Linearisation Points

Definition 9.18 (Linearisation Point). The linearisation point of an operation is the atomic step at which the operation appears to take effect. For lock-based data structures, the linearisation point is typically within the critical section. For lock-free data structures, it is typically a successful CAS.

In the Michael-Scott queue:

- **Enqueue**: the linearisation point is the successful CAS that links the new node to the tail of the queue (`CAS(tail->next, NULL, new_node)`).
- **Dequeue**: the linearisation point is the successful CAS that advances the head pointer (`CAS(head, old_head, next)`).

9.8.3 Composability

Theorem 9.6 (Linearisability is Compositional). If objects O_1 and O_2 are each linearisable in isolation, then the system composed of both O_1 and O_2 is linearisable.

Proof sketch. The linearisation point of each operation on O_1 and O_2 is independent. A global linearisation order can be constructed by merging the individual linearisation orders, respecting the real-time ordering constraint (if an operation on O_1 completes before an operation on O_2 begins, the first precedes the second in the global order). Since each object's operations are individually consistent with their sequential specifications, the merged order is consistent with the combined specification. □

This composability property is unique to linearisability among common consistency conditions. It means that systems can be built from linearisable components with the guarantee that the whole system is linearisable.

9.8.4 Verifying Linearisability

Determining whether a concurrent execution history is linearisable is NP-complete in general (Gibbons and Korach, 1997). However, practical verification strategies exist:

1. **Linearisation point analysis**: for each operation in the algorithm, identify the single atomic step where the operation "takes effect". If every operation has a fixed linearisation point, linearisability follows directly.
2. **Testing with a linearisability checker**: tools like **Lowe's linearisability tester** or **Knossos** (used by Jepsen) record all invocations and responses in a concurrent execution and check whether there exists a valid linearisation.
3. **Proof by simulation**: construct a simulation relation between the concurrent implementation and the sequential specification. Show that every concurrent step maps to at most one step in the sequential specification.

Example 9.8b (Linearisability Verification of a Counter). Consider a concurrent counter with `increment` (returns void) and `read` (returns the count):

```
Thread A: |--- inc() ---|           |--- read() -> 2 ---|
Thread B:      |--- inc() ---|
Thread C:                      |--- inc() ---|
```

Possible linearisation: `inc_A` at some point in $[t_0, t_1]$, `inc_B` at some point in $[t_2, t_3]$, `inc_C` at some point in $[t_4, t_5]$, `read_A` at some point in $[t_6, t_7]$.

For `read -> 2` to be correct, exactly 2 of the 3 increments must have taken effect before the read's linearisation point. This means `inc_C`'s linearisation point must be after `read_A`'s. Is this consistent with the real-time constraint? `inc_C` starts before `read_A` starts, but overlaps with it, so their linearisation points can be in either order. If `inc_C` is linearised after `read_A`, this is valid. The execution is linearisable.

9.8.5 Alternatives to Linearisability

Linearisability provides strong guarantees but comes at a performance cost. Weaker consistency conditions are used when full linearisability is unnecessary:

- **Sequential consistency**: operations from each thread appear in program order, but there is no real-time ordering requirement between threads. A counter might show a stale value that was valid at some earlier point.
- **Quiescent consistency**: operations separated by a period of quiescence (no pending operations) appear in their real-time order. Between quiescent points, operations may be reordered.

- **Eventual consistency**: all replicas eventually converge to the same state, with no ordering guarantees for concurrent updates. Used in distributed key-value stores like Cassandra and DynamoDB.

The choice between consistency conditions depends on the application's requirements:

Application	Typical consistency	Reason
Bank account balance	Linearisable	Must reflect all completed transactions
Web page hit counter	Quiescent consistent	Approximate counts are acceptable
Social media feed	Eventually consistent	Stale reads are tolerable
Lock-free queue	Linearisable	FIFO ordering is the specification
Distributed cache	Eventually consistent	Performance over precision

9.8.6 Practical Correctness Testing

In practice, proving linearisability formally is difficult for complex data structures. Engineers rely on a combination of techniques:

1. **Stress testing**: run many threads performing random operations simultaneously, checking invariants after each operation (e.g., a queue should return elements in FIFO order).
2. **Linearisability checkers**: record operation start/end timestamps and results, then check whether a valid linearisation exists. The tool Lowe's LinTester and Knossos (used by the Jepsen test suite for distributed databases) implement this.
3. **Model checking**: tools like SPIN or TLA+ can exhaustively explore all possible interleavings for small instances, finding bugs that random testing would miss.
4. **ThreadSanitizer (TSan)**: built into Clang and GCC, TSan detects data races

at runtime by instrumenting every memory access. While it does not check linearisability directly, it finds the most common source of correctness bugs in concurrent code: missing synchronisation.

```
/* Compile with TSan enabled */
/* gcc -fsanitize=thread -g -O1 myprogram.c -o myprogram */

/* TSan output for a data race: */
/*
WARNING: ThreadSanitizer: data race (pid=12345)
  Write of size 4 at 0x7f...abc by thread T1:
    #0 push queue.c:42
  Previous read of size 4 at 0x7f...abc by thread T2:
    #0 pop queue.c:67
*/
```

Theorem 9.7 (Linearisability Verification is NP-Complete). Given a concurrent execution history H and a sequential specification S, determining whether H is linearisable with respect to S is NP-complete in general (Gibbons and Korach, 1997).

Proof sketch. Membership in NP is clear: a valid linearisation order is a polynomial-size certificate that can be verified in polynomial time (check that the sequential specification is satisfied and that the real-time ordering constraint holds). NP-hardness is proved by reduction from the set partitioning problem. The key difficulty is that overlapping operations can be linearised in any order, and the number of possible orderings is exponential in the number of overlapping operations. □

Despite NP-completeness in the worst case, practical linearisability checking works well because most realistic histories have limited overlap (few operations are truly concurrent), and heuristics prune the search space effectively. Tools like Knossos handle millions of operations in minutes for typical workloads.

Example 9.8c (Testing Strategy for a Lock-Free Queue). A comprehensive test suite for a Michael-Scott queue should include:

1. **Sequential correctness**: single-threaded enqueue/dequeue in various orders, verifying FIFO property.
2. **Concurrent stress test**: P producer threads each enqueue N unique values; C consumer threads dequeue all values. Verify that every enqueued value is dequeued exactly once, with no duplicates or losses.
3. **Memory safety**: run under AddressSanitizer (ASan) to detect use-after-free in the memory reclamation scheme.
4. **Linearisability check**: record timestamps of all operations, feed to a

linearisability checker, verify no violations.

5. **Performance regression**: measure throughput and latency at various thread counts; compare against the previous version.

9.9 Programmer's Perspective: Go's Lock-Free Channels

Programmer's Perspective

Go channels are the language's primary concurrency communication primitive. Internally, Go channels are implemented as a combination of lock-based and lock-free techniques, depending on the channel state.

A channel is represented by the `hchan` struct in the Go runtime:

```go
// Simplified from runtime/chan.go
type hchan struct {
    qcount   uint     // number of elements in buffer
    dataqsiz uint     // buffer capacity
    buf      unsafe.Pointer // circular buffer
    elemsize uint16
    closed   uint32
    sendx    uint     // send index into buf
    recvx    uint     // receive index into buf
    recvq    waitq    // list of blocked receivers
    sendq    waitq    // list of blocked senders
    lock     mutex    // protects all fields
}
```

The channel uses a mutex (not `sync.Mutex` but a lighter runtime mutex) to protect its state. However, several fast paths avoid locking entirely:

1. **Direct send**: if a receiver is already waiting (`recvq` is non-empty), the sender copies data directly into the receiver's stack frame using `memmove` and wakes the receiver. This avoids touching the buffer entirely.
2. **Empty channel fast path**: a non-blocking receive on an empty channel (`select` with `default`) checks `qcount == 0` with an atomic load and returns immediately without acquiring the lock.
3. **Closed channel detection**: receiving from a closed, empty channel is detected with atomic loads on the `closed` and `qcount` fields.

For truly lock-free communication between goroutines, the `sync/atomic.Value` type provides a lock-free box for storing and loading arbitrary values:

```go
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
)
```



```go
func main() {
    var config atomic.Value

    // Writer (infrequent)
```

9.10 Summary

This chapter developed the theory and practice of concurrent data structures, moving from simple lock-based designs through lock-free algorithms to formal correctness criteria.

Key results:

- **Lock-based designs** range from coarse-grained (one lock for the entire structure) to fine-grained (per-node locking) to hand-over-hand locking (lock coupling for traversals). Finer granularity increases concurrency but also complexity and overhead.
- **Lock-free programming** uses CAS loops to ensure system-wide progress without locks. The ABA problem is a fundamental challenge, solved by tagged pointers, hazard pointers, or epoch-based reclamation.
- **Wait-free data structures** guarantee per-thread progress. Herlihy's universal construction proves they exist for any sequential data type, but practical wait-free algorithms are complex and rarely necessary.
- **The Michael-Scott queue** is the canonical lock-free FIFO queue, using CAS on head and tail pointers with cooperative helping to maintain lock-freedom.
- **Memory reclamation** (hazard pointers and epoch-based reclamation) solves the problem of safely freeing nodes in lock-free structures.
- **Memory ordering** (sequential consistency, acquire-release, relaxed) governs how memory operations become visible across threads. The C11/C++11 memory model formalises these guarantees with the DRF-SC property.
- **Linearisability** is the gold standard for concurrent data structure correctness: each operation appears to take effect atomically at its linearisation point. It is the only common consistency condition that is compositional.

Design principles:

- Choose the simplest locking strategy that meets your performance requirements. Start with coarse-grained locking and optimise only when profiling demonstrates contention.
- Prefer well-known, proven algorithms (Michael-Scott queue, Treiber stack) over custom designs. Lock-free algorithms are subtle and easy to get wrong.

- Use the language's standard library when available. Go's `sync.Mutex`, Java's `ConcurrentHashMap`, and C++'s `std::atomic` are battle-tested and optimised for their respective platforms.
- Memory ordering is a leaky abstraction: default to the strongest ordering (seq_cst) and weaken only with profiling evidence and careful reasoning. The performance difference between seq_cst and acquire-release is negligible on x86 and moderate on ARM.
- Test concurrent code aggressively: stress tests, race detectors, linearisability checkers, and model checkers each catch different classes of bugs. No single technique is sufficient.

The concurrent data structures developed in this chapter build directly on the synchronisation primitives of Chapter 7 and the deadlock avoidance strategies of Chapter 8. Together, these three chapters form the foundation of concurrent systems programming.

Exercises

Exercise 9.1. Implement a lock-free stack in C using `atomic_compare_exchange_weak` for both `push` and `pop`. Your implementation must handle the ABA problem using tagged pointers (pack a version counter into the upper 16 bits of the pointer). Write a multi-threaded test that runs 4 threads, each pushing and popping 100,000 elements, and verify that no elements are lost or duplicated.

Exercise 9.2. Prove that the Michael-Scott queue is linearisable. Identify the linearisation point of each operation (enqueue and dequeue, including the empty-queue case). Show that the resulting sequential history satisfies the FIFO queue specification for any possible interleaving of concurrent operations.

Exercise 9.3. A **Treiber stack** uses a single CAS on the top pointer for both push and pop. (a) Prove it is lock-free. (b) Prove it is not wait-free by constructing an execution where one thread's pop is delayed indefinitely by other threads' pushes. (c) What is the maximum number of CAS retries a pop operation can experience if n concurrent push operations occur?

Exercise 9.4. Consider the following C11 program:

```
atomic_int x = 0, y = 0;

/* Thread 1 */
```

```
atomic_store_explicit(&x, 1, memory_order_relaxed);
atomic_store_explicit(&y, 1, memory_order_release);

/* Thread 2 */
int r1 = atomic_load_explicit(&y, memory_order_acquire);
int r2 = atomic_load_explicit(&x, memory_order_relaxed);
```

(a) Can the outcome $r_1 = 1, r_2 = 0$ occur? Justify your answer using the C11 memory model's happens-before relation. (b) What if both operations in Thread 1 use `memory_order_relaxed`? (c) What if all operations use `memory_order_seq_cst`?

Exercise 9.5. Implement an epoch-based memory reclamation scheme in C for a lock-free linked list. Your implementation must: (a) support `enter_epoch`, `exit_epoch`, and `retire_node` operations; (b) advance the global epoch when all threads have been observed in the current epoch; (c) free retired nodes that are two or more epochs old. Demonstrate correctness with a test using 4 threads performing concurrent insertions and deletions.

Exercise 9.6. A **wait-free bounded queue** for a single producer and a single consumer (SPSC) can be implemented using two atomic indices and a shared array, with no CAS operations required. (a) Design such a queue using only `atomic_load` and `atomic_store` with appropriate memory orderings. (b) Prove it is wait-free for both the producer and consumer. (c) Explain why this construction does not generalise to multiple producers or multiple consumers.

Exercise 9.7. Linearisability is compositional but sequential consistency is not. Construct a concrete counterexample: two objects O_1 and O_2, each sequentially consistent in isolation, but whose composition violates sequential consistency. (*Hint*: consider two registers and exploit the lack of real-time ordering in sequential consistency.)

The study of concurrent data structures sits at the intersection of algorithm design, hardware architecture, and programming language semantics. Getting these structures right requires understanding all three: the algorithmic logic (CAS loops, helping), the hardware reality (cache coherence, memory ordering, store buffers), and the language-level guarantees (the C11 memory model, the Go memory model). Errors in any dimension produce subtle bugs that manifest only under specific timing conditions, making concurrent programming one of the most demanding areas of systems engineering.

Chapter 10: Memory Management Fundamentals

Every instruction a processor executes, every variable a program reads, every stack frame a function creates — all of these require memory. The operating system's memory management subsystem is the gatekeeper that decides where each byte lives, which process can touch which address, and what happens when physical memory runs short. Get this wrong, and programs crash, corrupt one another's data, or grind to a halt. Get it right, and dozens of processes coexist peacefully, each believing it has the machine to itself.

This chapter builds the foundations of memory management from first principles. We begin with the distinction between logical and physical addresses, work through the hardware and software mechanisms that bind one to the other, and then analyse the allocation strategies that determine how memory is parcelled out to competing processes. Along the way, we encounter fragmentation — the silent tax that every allocator must pay — and swapping, the brute-force escape hatch for when physical memory is exhausted.

The ideas in this chapter predate virtual memory. They represent the techniques used in early batch and multiprogramming systems and remain relevant today because they underpin the more sophisticated schemes of Chapters 11–13.

10.1 Address Spaces

10.1.1 Logical and Physical Addresses

A central abstraction in memory management is the distinction between the addresses a program uses and the addresses the hardware presents to the memory bus.

Definition 10.1 (Logical Address). A *logical address* (also called a *virtual address*) is an address generated by the CPU during program execution. It is the address that appears in machine instructions, pointer variables, and program counters. The set of all logical addresses used by a process is its *logical address space*.

Definition 10.2 (Physical Address). A *physical address* is an address that appears on the memory bus and is understood by the memory controller. It identifies an actual location in the physical DRAM or SRAM. The set of all physical addresses available to the system is the *physical address space*.

In the simplest systems — a bare-metal microcontroller with no operating system — logical and physical addresses are identical. The address 0x2000 in the program refers to byte 0x2000 in the physical memory chip. This works when only one program runs at a time and when the program is always loaded at the same address.

The moment we want to run multiple programs, or load a program at an arbitrary address, we need a mapping from logical to physical addresses. This mapping is the core responsibility of the memory management unit (MMU), a hardware component that sits between the CPU and the memory bus.

Definition 10.3 (Memory Management Unit). The *MMU* is a hardware device that translates every logical address issued by the CPU into a physical address before it reaches the memory bus. The translation is governed by tables and registers maintained by the operating system.

The relationship between logical and physical addresses depends on when the binding between them is established. Before we investigate binding, we must understand the size of address spaces.

10.1.2 Address Space Size

For a processor with an n-bit address bus, the logical address space contains 2^n bytes. A 32-bit processor can address $2^{32} = 4{,}294{,}967{,}296$ bytes (4 GB). A 64-bit processor can theoretically address 2^{64} bytes (16 exabytes), but current implementations use only 48 or 57 bits of the virtual address, yielding 256 TB or 128 PB of virtual address space respectively.

The physical address space is determined by the width of the physical address bus, which may differ from the logical address width. A 32-bit processor might have a 36-bit physical address bus (Intel PAE), giving 64 GB of physical memory. Conversely, a 64-bit processor with 48-bit virtual addresses might have only 46 bits of physical address, giving 64 TB of physical addressability.

Example 10.1 (Address Space Arithmetic). A system has a 32-bit logical address and a 28-bit physical address.

- Logical address space: $2^{32} = 4$ GB
- Physical address space: $2^{28} = 256$ MB

Each process believes it has 4 GB of memory, but the total physical memory is only 256 MB. The operating system must manage this disparity through address translation and, when necessary, swapping or demand paging.

Example 10.2 (Modern x86-64 Address Spaces). An Intel Xeon processor implements 48-bit virtual addresses and 46-bit physical addresses.

- Virtual address space per process: $2^{48} = 256$ TB
- Physical address space: $2^{46} = 64$ TB
- Typical installed RAM: 256 GB $= 2^{38}$ bytes

The ratio of virtual to physical is $2^{48}/2^{38} = 1024$. Each process has a virtual address space 1024 times larger than the installed physical memory. The operating system uses demand paging to maintain the illusion that all 256 TB is available.

10.1.3 The Address Space as an Abstraction

The address space is not merely a bookkeeping device — it is one of the most powerful abstractions in computer science. It provides three essential properties:

1. **Isolation:** Each process has its own address space. Process A's address 0x4000 and Process B's address 0x4000 refer to different physical locations. Neither process can access the other's memory (unless explicitly shared).

2. **Transparency:** The program does not know (or need to know) which physical addresses it occupies. It can be compiled once and run at any physical location.

3. **Overcommitment:** The total virtual memory across all processes can far exceed physical memory. The OS manages the illusion through demand paging and swapping.

Definition 10.4 (Address Space Isolation). Two address spaces A_1 and A_2 are *isolated* if no logical address in A_1 maps to the same physical address as any logical address in A_2, unless both mappings are explicitly established by the operating system for the purpose of sharing.

10.2 Address Binding

The process of associating a logical address with a physical address is called *address binding*. It can occur at three stages in the lifecycle of a program. Understanding these stages is essential because each one offers different trade-offs between flexibility, performance, and complexity.

10.2.1 Compile-Time Binding

If the physical memory location where a program will reside is known at compile time, the compiler generates code with absolute physical addresses embedded directly in the instructions.

Definition 10.5 (Compile-Time Binding). *Compile-time binding* occurs when the compiler generates absolute addresses in the object code. The logical addresses in the program are identical to the physical addresses where the program will be loaded.

Example 10.3 (MS-DOS .COM Programs). Early MS-DOS .COM executables used compile-time binding. The program was always loaded at address 0x0100 in the segment, and the compiler generated all addresses relative to this fixed base. If the program needed to be loaded elsewhere, the source code had to be recompiled with a different base address.

Example 10.4 (Embedded System). Consider a traffic light controller running on an ARM Cortex-M0 microcontroller with 16 KB of flash at address 0x08000000. The linker script fixes the text segment at 0x08000000 and the data segment at 0x20000000 (SRAM). Every pointer in the compiled binary contains these absolute addresses. There is no MMU, no OS, and no possibility of loading the program elsewhere.

Compile-time binding is rigid: if the load address changes, the entire program must be recompiled. This is acceptable only when the system runs a single program at a fixed location — a common scenario in embedded systems but untenable for general-purpose operating systems.

10.2.2 Load-Time Binding

If the load address is not known at compile time, the compiler generates *relocatable code* containing relative addresses. When the loader places the program into memory, it adjusts all addresses by adding the actual base address.

Definition 10.6 (Load-Time Binding). *Load-time binding* occurs when the linker/loader resolves logical addresses to physical addresses at the moment the program is loaded into memory. The compiler generates relocatable addresses, and the loader adds the base address to produce physical addresses.

The relocatable code contains a *relocation table* — a list of all locations in the binary that contain addresses needing adjustment. The loader iterates through this table, adding the base address to each entry.

Example 10.5 (Relocation Table). Consider a program compiled with a base of 0. It contains an instruction `LOAD R1, [0x0040]` (load from address 0x40). The reloca-

tion table records that the address operand at binary offset 0x0012 refers to logical address 0x0040.

If the loader places the program at physical address 0x5000, it rewrites the instruction to `LOAD R1, [0x5040]` by adding 0x5000 to the address operand. Every entry in the relocation table is similarly adjusted.

Example 10.6 (Relocation Cost). A program with 10,000 relocatable addresses is loaded at address 0x80000. The loader must:

1. Read the relocation table (10,000 entries).
2. For each entry, read the current value at the specified offset, add 0x80000, and write back.
3. Total: 10,000 read-modify-write operations.

On a modern system, this takes microseconds. On early systems with slow disks, it was a noticeable delay.

Load-time binding is more flexible than compile-time binding because the program can be loaded at any address. However, once loaded, the program cannot move without repeating the relocation process. This limitation motivates execution-time binding.

10.2.3 Execution-Time Binding

The most flexible approach defers address translation to the moment each instruction accesses memory. The program uses logical addresses throughout its execution, and the hardware translates them to physical addresses on every memory reference.

Definition 10.7 (Execution-Time Binding). *Execution-time binding* (also called *dynamic binding* or *run-time binding*) occurs when address translation is performed by the MMU on every memory access during program execution. The program retains its logical addresses; the mapping to physical addresses can change without modifying the program.

Execution-time binding requires hardware support (the MMU) but provides critical capabilities:

- **Relocation**: A process can be moved in physical memory without changing its code. The OS simply updates the translation tables.
- **Protection**: The MMU can enforce access restrictions, preventing one process from reading or writing another's memory.
- **Sharing**: Multiple logical addresses (in different processes) can map to the same physical address, enabling shared libraries and inter-process communication.

- **Demand paging**: Pages can be loaded from disk as needed, because the MMU can detect accesses to pages not yet in memory.

Theorem 10.1 (Binding Flexibility Hierarchy). Let F_c, F_l, and F_e denote the sets of memory configurations achievable with compile-time, load-time, and execution-time binding, respectively. Then $F_c \subseteq F_l \subseteq F_e$. The inclusions are strict whenever the system supports more than one process.

Proof. Every configuration achievable by compile-time binding (fixed addresses) is trivially achievable by load-time binding (set the base to zero). Every configuration achievable by load-time binding is achievable by execution-time binding (set the MMU translation to the loader's base offset). The first inclusion is strict because load-time binding can place a program at an arbitrary address, which compile-time binding cannot (unless recompiled). The second inclusion is strict because execution-time binding permits relocation of a running process — moving its physical pages while preserving logical addresses — which load-time binding cannot achieve without halting and re-relocating the process. □

10.2.4 Dynamic Loading and Linking

Two related concepts extend the binding discussion:

Dynamic loading defers loading of a module until it is first called. The main program is loaded; subroutines remain on disk until invoked. This reduces memory consumption for programs with large libraries where only a fraction of routines are used in any given run.

Dynamic linking defers the linking of a shared library until load time or run time. Instead of including library code in the executable, the program contains *stubs* — small pieces of code that locate and call the actual library function. On the first call, the stub is replaced with the address of the real function. Subsequent calls go directly to the library.

Definition 10.8 (Dynamic Linking). *Dynamic linking* is a mechanism where references to shared library functions are resolved at load time or at first use (lazy binding). The executable contains *stubs* that, on first invocation, invoke the dynamic linker to resolve the symbol and patch the calling code.

On Linux, the dynamic linker is `ld-linux.so`. The Procedure Linkage Table (PLT) and Global Offset Table (GOT) implement the stub mechanism. Lazy binding means that the cost of resolving a symbol is paid only when the function is first called, not when the library is loaded.

Example 10.7 (Dynamic Linking on Linux). When a C program calls `printf`, the

following happens on the first call:

1. The call instruction jumps to the PLT entry for `printf`.
2. The PLT stub reads the GOT entry, which initially points back to the PLT stub's "resolver" code.
3. The resolver invokes `ld-linux.so`, which searches the loaded libraries for the symbol `printf`.
4. `ld-linux.so` finds `printf` in `libc.so` at address 0x7f4a38201a40 and writes this address into the GOT.
5. The resolver jumps to `printf`.
6. On subsequent calls, the PLT stub reads the GOT entry (now 0x7f4a38201a40) and jumps directly to `printf`, bypassing the resolver.

10.3 Base and Limit Registers

The simplest hardware mechanism for execution-time address translation uses two registers: a *base register* and a *limit register*.

10.3.1 Mechanism

Definition 10.9 (Base-Limit Address Translation). Given a base register B and a limit register L:

1. For every logical address a generated by the CPU, the MMU first checks $0 \leq a < L$.
2. If the check fails, the MMU raises a hardware trap (memory protection fault).
3. If the check passes, the MMU computes the physical address as $p = B + a$.

The base register defines where the process begins in physical memory. The limit register defines the size of the address space. Together, they confine the process to a contiguous region $[B, B + L)$ in physical memory.

```
Logical Address (from CPU)
        |
        v
   +---------+       +---------+
   | a < L ? |--NO-->|  TRAP   |
   +---------+       +---------+
        |
       YES
        |
```

```
        v
   +---------+
   | p = B+a |
   +---------+
        |
        v
Physical Address (to memory bus)
```

The entire translation adds a single comparison and a single addition to every memory access. On a pipelined processor, this can be overlapped with other stages, adding zero effective latency in the common case.

10.3.2 Context Switching

When the operating system switches from one process to another, it must save the current process's base and limit values and load the new process's values. This is part of the *context switch* overhead.

The base and limit registers are *privileged*: only the operating system (running in kernel mode) can modify them. A user-mode process cannot change its own base or limit, which prevents it from escaping its allocated region.

Definition 10.10 (Privileged Registers). The base and limit registers are *privileged* (or *protected*) — they can be modified only by instructions executed in kernel mode (ring 0 on x86). Any attempt to modify them from user mode raises a general protection fault.

10.3.3 Worked Examples

Example 10.8 (Two-Process Base-Limit). Suppose the operating system has loaded Process A at physical address 0x10000 with size 0x8000, and Process B at 0x20000 with size 0xC000.

Process	Base Register	Limit Register	Physical Range
A	0x10000	0x8000	0x10000–0x17FFF
B	0x20000	0xC000	0x20000–0x2BFFF

When Process A generates logical address 0x3000, the MMU computes: $0x3000 < 0x8000$ (pass), physical address $= 0x10000 + 0x3000 = 0x13000$.

When Process A generates logical address 0x9000, the MMU detects: $0x9000 \not< 0x8000$ (fail), and raises a segmentation fault.

Example 10.9 (Base-Limit with Multiple Processes). A system has 1 MB of physical memory (0x00000 – 0xFFFFF). The OS kernel occupies 0x00000 – 0x0FFFF (64 KB). Three user processes are loaded:

Process	Base	Limit	Physical Range
P1	0x10000	0x20000	0x10000 – 0x2FFFF
P2	0x30000	0x40000	0x30000 – 0x6FFFF
P3	0x70000	0x10000	0x70000 – 0x7FFFF

Free memory: 0x80000 – 0xFFFFF (512 KB).

If P2 terminates, the free memory becomes: 0x30000 – 0x6FFFF (256 KB) and 0x80000 – 0xFFFFF (512 KB). A new process requesting 300 KB cannot use the first hole (only 256 KB) and would be placed in the second hole.

10.3.4 Relocation with Base-Limit

Because addresses are translated dynamically, the operating system can *relocate* a process by:

1. Suspending the process.
2. Copying its memory contents from $[B_{\text{old}}, B_{\text{old}} + L)$ to $[B_{\text{new}}, B_{\text{new}} + L)$.
3. Updating the base register to B_{new}.
4. Resuming the process.

The process is unaware of the move because its logical addresses have not changed. This ability to relocate is essential for *compaction*, discussed in Section 10.6.

10.3.5 Limitations of Base-Limit

The base-limit scheme has several significant limitations:

1. **Contiguity requirement:** Each process must occupy a contiguous block of physical memory. This leads to external fragmentation.
2. **No sharing:** Two processes cannot share code or data, because each process's address space is a single contiguous block with no overlap.

3. **No fine-grained protection:** Every address in the range $[0, L)$ has the same read/write/execute permissions. You cannot make the code segment read-only while keeping the data segment read-write.
4. **Entire process must be in memory:** There is no mechanism for keeping only part of the process in physical memory.

These limitations motivate the more sophisticated mechanisms of segmentation and paging.

10.4 Contiguous Memory Allocation

In a contiguous allocation scheme, each process occupies a single contiguous block of physical memory. The operating system must decide how to divide physical memory among competing processes.

10.4.1 Fixed Partitions

The simplest approach divides memory into a fixed number of partitions at system startup.

Definition 10.11 (Fixed Partition Allocation). In *fixed partition allocation*, physical memory is divided into k partitions of predetermined sizes $s_1, s_2, \dots, s_k$ at boot time. Each incoming process is assigned to a partition large enough to hold it.

Two queuing strategies are possible:

One queue per partition. Each partition has its own ready queue. An incoming process of size n is placed in the queue of the smallest partition with $s_i \geq n$. This can lead to underutilisation if one partition's queue is empty while another is full.

Single shared queue. All processes share one queue. When a partition becomes free, the next process in the queue that fits is assigned to it.

Example 10.10 (Fixed Partitions). A system has 64 MB of memory divided into four fixed partitions of sizes 8 MB, 16 MB, 16 MB, and 24 MB. Three processes arrive with sizes 7 MB, 14 MB, and 20 MB.

Process	Size	Assigned Partition	Partition Size	Internal Fragmentation
P1	7 MB	Partition 1	8 MB	1 MB

Process	Size	Assigned Partition	Partition Size	Internal Fragmentation
P2	14 MB	Partition 2	16 MB	2 MB
P3	20 MB	Partition 4	24 MB	4 MB

Total memory used: 41 MB out of 64 MB. Internal fragmentation: 7 MB. Partition 3 (16 MB) is entirely unused.

Example 10.11 (Fixed Partition Scheduling Problem). Same system as above. A fourth process P4 of size 15 MB arrives. Under single-queue scheduling, P4 would fit in Partition 3 (16 MB). Under per-partition queuing, P4 joins the queue for the 16 MB partition (Partition 2 or 3) and must wait even though Partition 4 (24 MB) might be free.

This illustrates the *scheduling anomaly* of per-partition queuing: a process may wait in a queue while a larger partition sits idle. The single-queue approach avoids this but requires the OS to search for the best-fit partition for each process.

Fixed partitions are easy to implement but inflexible. The partition sizes are chosen at boot time and cannot adapt to workload changes. A system designed for a mix of small and large processes may perform poorly when the workload shifts.

Theorem 10.2 (Maximum Utilisation of Fixed Partitions). For a system with k partitions of sizes $s_1 \leq s_2 \leq \cdots \leq s_k$ and a stream of processes whose sizes are uniformly distributed on $[0, s_k]$, the expected memory utilisation is:

$$U = \frac{1}{k} \sum_{i=1}^{k} \left(1 - \frac{s_i}{2 \cdot s_i}\right) = \frac{1}{k} \sum_{i=1}^{k} \frac{1}{2} = \frac{1}{2}$$

That is, on average, half the allocated memory is wasted as internal fragmentation. This bound is tight when the partitions are well-sized for the workload.

10.4.2 Variable Partitions

A more flexible approach allows partitions to be created dynamically, sized exactly to the process's needs.

Definition 10.12 (Variable Partition Allocation). In *variable partition allocation*, each process is allocated a contiguous block of memory exactly equal to its size (or

slightly larger due to alignment). Partitions are created and destroyed as processes arrive and depart.

The operating system maintains a data structure (typically a linked list or a set of lists) tracking the free regions of memory, called *holes*.

Example 10.12 (Variable Partition Dynamics). Consider a system with 64 MB of memory. Initially, the entire memory is one free hole. Processes arrive and depart over time:

1. **P1 (10 MB) arrives:** Allocated at 0. Free: 10–63 (54 MB).
2. **P2 (20 MB) arrives:** Allocated at 10. Free: 30–63 (34 MB).
3. **P3 (15 MB) arrives:** Allocated at 30. Free: 45–63 (19 MB).
4. **P2 (20 MB) departs:** Free holes: 10–29 (20 MB) and 45–63 (19 MB).
5. **P4 (8 MB) arrives:** Allocated at 10 (first-fit). Free: 18–29 (12 MB) and 45–63 (19 MB).

After step 5, memory has three regions: P1 (0–9), P4 (10–17), free (18–29), P3 (30–44), free (45–63). The free memory is split into two non-contiguous holes.

10.4.3 Free List Data Structures

The data structure used to maintain the free list significantly affects allocation performance. The choice of data structure is itself an engineering trade-off between allocation speed, deallocation speed (including coalescing), and memory overhead.

Linked list (address-ordered). The simplest implementation: each hole is a node in a doubly-linked list, sorted by address. Allocation requires scanning the list; deallocation requires finding the correct position and potentially merging with adjacent free blocks.

- Allocation (first-fit): $O(n)$ scan until the first adequate hole.
- Deallocation: $O(n)$ to find the position by address, then $O(1)$ to merge with neighbours.

Example 10.12a (Free List Coalescing). The free list (sorted by address) contains: [10–19], [30–49], [70–99]. Process B (occupying 20–29) terminates.

1. Insert hole [20–29] into the list between [10–19] and [30–49].
2. Check left neighbour [10–19]: contiguous (end=19, new start=20). Merge: [10–29].
3. Check right neighbour [30–49]: contiguous (new end=29, right start=30). Merge: [10–49].
4. Result: [10–49], [70–99].

Two merges reduced three holes to one. Without coalescing, the free list would have four holes: [10–19], [20–29], [30–49], [70–99].

Segregated free lists. Maintain separate free lists for different size ranges: one list for holes < 4 KB, one for 4–16 KB, one for 16–64 KB, etc. Allocation first checks the appropriate list, falling back to larger lists if necessary. This reduces average search time because only a subset of holes is examined.

Definition 10.12a (Segregated Free List). A *segregated free list* organises holes into k size classes $[0, s_1), [s_1, s_2), \dots, [s_{k-1}, \infty)$. Each class has its own linked list. An allocation of size n first checks the list for the smallest class containing holes $\geq n$. If empty, the next larger class is checked, and so on.

Balanced tree (by size). An AVL tree or red-black tree keyed by hole size allows best-fit allocation in $O(\log n)$ time. This is used in some advanced allocators but adds complexity for insertion and deletion.

Bitmap. Divide memory into fixed-size units (e.g., 1 KB). Maintain a bitmap where bit i = 1 if unit i is allocated, 0 if free. Allocation: scan the bitmap for a run of k consecutive zeros. Deallocation: set the corresponding bits to 0. Space: M/U bits where M is total memory and U is the unit size.

Example 10.12b (Bitmap Allocation). Memory: 16 units. Bitmap: 1111000011100010.

Free regions: units 4–7 (4 units), units 11–11 (1 unit), units 13–14 (2 units), unit 15 is at end but bit=0. Wait, reading left to right: bits 0–3 = 1111 (allocated), bits 4–7 = 0000 (free), bits 8–10 = 111 (allocated), bit 11 = 0 (free), bit 12 = 0 (free), bit 13 = 0 (free), bit 14 = 1 (allocated), bit 15 = 0 (free).

Free regions: [4–7] (4 units), [11–13] (3 units), [15] (1 unit).

Request for 3 units (first-fit): allocate units 4–6. New bitmap: 1111111011100010.

10.4.4 Coalescing Policies

When a block is freed, adjacent free blocks should be merged to prevent the proliferation of small, unusable holes. Two coalescing policies exist:

Immediate coalescing: Every deallocation immediately checks for and merges adjacent free blocks. This keeps the free list clean but adds overhead to every deallocation.

Deferred coalescing: Freed blocks are not immediately merged. Coalescing is triggered periodically or when an allocation request cannot be satisfied. This reduces per-deallocation overhead but allows temporary fragmentation.

Theorem 10.2a (Coalescing Reduces Fragmentation). In a system with n allocations and n deallocations (in any order), the number of free holes at any time is at most n without coalescing. With immediate coalescing, the number of holes is bounded by $\lceil n/2 \rceil$ (by the fifty-percent rule). The total size of the largest free block is always at least as large with coalescing as without.

Proof sketch. Each deallocation creates at most one new hole. Without coalescing, n deallocations create at most n holes. With coalescing, each deallocation may merge with up to two neighbours, reducing the hole count. The merged hole is always at least as large as the sum of its constituent holes, so the largest hole size is non-decreasing under coalescing. □

10.5 Memory Allocation Strategies

When a process requests n bytes and multiple free holes can satisfy the request, the operating system must choose which hole to use. Three classical strategies exist.

10.5.1 First-Fit

Definition 10.13 (First-Fit). *First-fit* allocation scans the list of free holes from the beginning and allocates the first hole that is large enough.

First-fit is fast because it stops as soon as it finds a suitable hole. It tends to leave small fragments near the beginning of the list (where allocations happen frequently) and larger holes towards the end.

Complexity: $O(n)$ in the worst case, where n is the number of free holes. With optimisations (e.g., a roving pointer that starts searching where the last allocation ended, called *next-fit*), average-case performance improves.

10.5.2 Next-Fit

Definition 10.14 (Next-Fit). *Next-fit* is a variant of first-fit that begins the search at the position where the previous allocation ended, wrapping around to the beginning if necessary. This distributes allocations more evenly across memory.

Next-fit avoids the clustering of small fragments at the beginning of the free list that plagues first-fit. However, simulations show that next-fit often produces worse overall fragmentation than first-fit because it tends to break up the large holes at the end of memory that first-fit preserves.

10.5.3 Best-Fit

Definition 10.15 (Best-Fit). *Best-fit* allocation searches the entire list of free holes and selects the smallest hole that is large enough to satisfy the request.

Best-fit minimises the *leftover* from each allocation, producing the smallest possible residual hole. Paradoxically, this often leads to worse overall fragmentation because the tiny leftover holes are too small to be useful for future allocations.

Complexity: $O(n)$ to scan the entire free list. Can be reduced to $O(\log n)$ with a balanced tree sorted by hole size.

10.5.4 Worst-Fit

Definition 10.16 (Worst-Fit). *Worst-fit* allocation searches the entire list of free holes and selects the largest hole.

Worst-fit maximises the leftover, hoping that the residual hole will be large enough to satisfy future requests. In practice, it tends to produce mediocre results because it rapidly breaks large holes into medium-sized ones that may not serve any request well.

Complexity: $O(n)$ to scan the entire list, or $O(1)$ with a max-heap.

10.5.5 Comparative Analysis

Theorem 10.3 (First-Fit and Best-Fit Dominance). Simulation studies and analytical results consistently show that first-fit and best-fit outperform worst-fit in terms of both memory utilisation and allocation speed. Between the two, first-fit is generally preferred because it is faster (no need to scan the entire list) and produces comparable utilisation.

Justification. Knuth's analysis (1973) of the first-fit and best-fit algorithms using the *fifty-percent rule* shows that, under steady-state conditions with random request sizes, approximately one-third of the holes are unusable (external fragmentation). First-fit achieves this ratio while examining, on average, only a fraction of the free list. Best-fit achieves a slightly better ratio but at the cost of examining the entire list and creating more tiny, unusable fragments.

Example 10.13 (Strategy Comparison). Free holes: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB. Request: 212 KB.

Strategy	Chosen Hole	Leftover	Remark
First-fit	500 KB	288 KB	First hole $\geq$ 212 KB
Best-fit	300 KB	88 KB	Smallest adequate hole
Worst-fit	600 KB	388 KB	Largest hole

First-fit allocates from the 500 KB hole (the first adequate one). Best-fit chooses the 300 KB hole, leaving an 88 KB fragment that may be too small for most requests. Worst-fit uses the 600 KB hole, preserving a large 388 KB residual.

Example 10.14 (Extended Comparison). Consider a sequence of three requests applied to the same initial free list: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB. Requests: 212 KB, then 100 KB, then 450 KB.

First-fit trace:

1. Request 212 KB: allocate from 500 KB hole. Free: 100, 288, 200, 300, 600.
2. Request 100 KB: allocate from 100 KB hole (exact fit). Free: 288, 200, 300, 600.
3. Request 450 KB: allocate from 600 KB hole. Free: 288, 200, 300, 150.

Best-fit trace:

1. Request 212 KB: allocate from 300 KB hole. Free: 100, 500, 200, 88, 600.
2. Request 100 KB: allocate from 100 KB hole. Free: 500, 200, 88, 600.
3. Request 450 KB: allocate from 500 KB hole. Free: 50, 200, 88, 600.

Best-fit leaves more holes (four vs four) but with smaller sizes (50 KB and 88 KB are likely unusable). First-fit's smallest hole is 150 KB, which may still be useful.

10.5.6 The Fifty-Percent Rule

Theorem 10.4 (Knuth's Fifty-Percent Rule). In a system using first-fit allocation in steady state, if H is the number of holes and P is the number of allocated blocks, then:

$$H \approx \frac{P}{2}$$

Proof. Consider the steady-state condition where processes arrive and depart at equal rates. Each allocation either (a) splits a hole into an allocated block and a smaller hole (net change: $+1$ block, 0 holes) or (b) exactly fills a hole ($+1$ block, -1 hole). Each

deallocation either (a) creates a new hole adjacent to two allocated blocks ($+1$ hole) or (b) merges with one adjacent hole (0 net holes) or (c) merges with two adjacent holes (-1 hole).

Let p_s be the probability that an allocation splits a hole (as opposed to filling it exactly); in practice $p_s \approx 1$. Let p_0, p_1, p_2 be the probabilities that a deallocation creates a new hole, merges with one, or merges with two neighbours.

In steady state, the expected change in the number of holes per operation must be zero. An allocation changes holes by 0 (split) or -1 (exact fit). A deallocation changes holes by $+1$ (new), 0 (merge with one), or -1 (merge with two).

If we consider a random configuration of P allocated blocks and H holes in a linear memory, the probability that a block's left neighbour is a hole is $H/(P + H)$, and similarly for the right neighbour (approximately, ignoring boundary effects). The expected number of holes merged on deallocation is:

$$E[\text{merges}] = 2 \times \frac{H}{P + H}$$

In steady state, the hole creation rate equals the hole consumption rate. Through careful bookkeeping (see Knuth, *The Art of Computer Programming*, Vol. 1, Section 2.5), this yields $H = P/2$ under the assumption that processes are randomly distributed in memory. □

This result has a practical implication: if a system has P active processes, the memory will contain approximately $P/2$ free holes. Since each hole represents wasted (or at least fragmented) memory, the system is inherently inefficient — motivating the compaction techniques of Section 10.6.

Programmer's Perspective

The fifty-percent rule explains why long-running C programs that perform many `malloc`/`free` cycles eventually suffer from fragmentation. Modern allocators (glibc's ptmalloc, jemalloc, TCMalloc) combat this with *size classes*: instead of managing one free list, they maintain separate free lists for common sizes (8, 16, 32, 64, ... bytes). This turns a variable-partition problem into a fixed-partition problem within each size class, eliminating external fragmentation at the cost of modest internal fragmentation. Go's memory allocator uses 67 size classes, and we shall see its full design in Chapter 13.

10.5.7 Formal Analysis of Allocation Strategies

We can formalise the allocation problem as follows. Let $\mathcal{H} = \{h_1, h_2, \dots, h_k\}$ be the set of free holes with sizes $|h_1|, |h_2|, \dots, |h_k|$. A request of size n is *satisfiable* if $\exists i : |h_i| \geq n$.

Definition 10.17 (Allocation Function). An *allocation function* $\alpha : (\mathcal{H}, n) \to \mathcal{H} \cup \{\perp\}$ maps a set of holes and a request size to either a selected hole or $\perp$ (failure). For first-fit, α returns the first h_i (in address order) with $|h_i| \geq n$. For best-fit, α returns $\arg\min_{|h_i| \geq n} |h_i|$. For worst-fit, α returns $\arg\max_i |h_i|$ (if $|h_i| \geq n$).

Theorem 10.5 (No Optimal Online Strategy). No online allocation strategy (one that does not know future requests) can guarantee the minimum number of processes that fail to be allocated. The competitive ratio of first-fit against the optimal offline algorithm is $O(\log n)$ for n requests.

This is proved by reduction to the online bin-packing problem: allocating variable-sized processes into a fixed memory is equivalent to packing items into bins, and first-fit-decreasing achieves a competitive ratio of 11/9 in the offline case, but no online algorithm can do better than $\Omega(\log n)$ in the worst case.

10.6 Fragmentation

Fragmentation is the phenomenon where available memory is wasted because it is divided into pieces too small or too awkwardly placed to satisfy allocation requests.

10.6.1 Internal Fragmentation

Definition 10.18 (Internal Fragmentation). *Internal fragmentation* is the memory wasted inside an allocated block because the block is larger than the request. It occurs when memory is allocated in fixed-size units and the request does not evenly divide the unit size.

Example 10.15 (Internal Fragmentation). A system allocates memory in 4 KB pages. A process requests 4,100 bytes. The system allocates 2 pages (8,192 bytes). The internal fragmentation is $8{,}192 - 4{,}100 = 4{,}092$ bytes — nearly an entire page wasted.

Example 10.16 (Internal Fragmentation in Size Classes). A size-class allocator has classes at 8, 16, 32, 64, 128, 256, 512, and 1024 bytes. A request for 65 bytes is rounded up to the 128-byte class. Internal fragmentation: $128 - 65 = 63$ bytes (49%).

A request for 33 bytes is rounded up to 64 bytes. Internal fragmentation: $64-33 = 31$ bytes (48%).

A request for 64 bytes is an exact fit. Internal fragmentation: 0 bytes (0%).

The expected internal fragmentation per allocation, assuming requests are uniformly distributed in $[1, U]$ where U is the allocation unit size, is:

$$E[\text{internal fragmentation}] = \frac{U-1}{2}$$

This follows because the wasted space is uniformly distributed on $[0, U-1)$.

10.6.2 External Fragmentation

Definition 10.19 (External Fragmentation). *External fragmentation* is the condition where the total free memory is sufficient to satisfy a request, but no single contiguous block is large enough. The free memory is scattered across many small holes.

Example 10.17 (External Fragmentation). A system has three free holes: 50 KB, 30 KB, and 40 KB (total: 120 KB). A process requests 100 KB. Although 120 KB is free, no single hole is large enough, so the request fails.

Example 10.18 (Progressive External Fragmentation). A system with 256 KB of memory processes the following sequence:

1. Allocate A (80 KB) at 0. Free: 80–255 (176 KB).
2. Allocate B (60 KB) at 80. Free: 140–255 (116 KB).
3. Allocate C (40 KB) at 140. Free: 180–255 (76 KB).
4. Free A. Free: 0–79 (80 KB), 180–255 (76 KB).
5. Allocate D (50 KB) at 0 (first-fit). Free: 50–79 (30 KB), 180–255 (76 KB).
6. Free C. Free: 50–79 (30 KB), 140–179 (40 KB), 180–255 (76 KB).

Total free: 146 KB, in three holes. A request for 100 KB fails despite 146 KB being free.

External fragmentation is the bane of variable-partition allocation. Over time, as processes of varying sizes arrive and depart, memory becomes a patchwork of allocated blocks and small holes.

10.6.3 Quantifying External Fragmentation

We can measure external fragmentation as:

$$F_{\text{ext}} = 1 - \frac{\text{largest free block}}{\text{total free memory}}$$

When $F_{\text{ext}} = 0$, all free memory is in one contiguous block (no external fragmentation). When F_{ext} approaches 1, free memory is scattered into many tiny, unusable holes.

Example 10.19 (Fragmentation Metric). From Example 10.18, step 6: free holes of 30, 40, and 76 KB. Total free: 146 KB. Largest hole: 76 KB.

$$F_{\text{ext}} = 1 - \frac{76}{146} = 1 - 0.521 = 0.479$$

Nearly half the free memory is "lost" to fragmentation.

Theorem 10.6 (Fragmentation Bound). Statistical analysis by Knuth and others shows that with first-fit allocation and random request sizes drawn from a distribution with mean μ and finite variance, the fraction of memory lost to external fragmentation converges to approximately $1/3$ in the steady state. More precisely, if M is the total memory and U is the total memory in use, then:

$$\frac{M - U}{M} \approx \frac{1}{3}$$

in the long run, under mild assumptions on the request size distribution.

This "one-third rule" means that a system must have roughly 50% more memory than the total size of the processes it wishes to run simultaneously.

10.6.4 Compaction

Definition 10.20 (Compaction). *Compaction* is the process of relocating allocated blocks to consolidate all free memory into one large contiguous hole. It is possible only if the system uses execution-time address binding (so that processes can be moved without rewriting their code).

Compaction eliminates external fragmentation but has a significant cost: every byte of allocated memory may need to be copied, and all processes must be suspended during the operation (or a complex incremental scheme must be used).

```
Before compaction:              After compaction:

+--------+                      +--------+
|  P1    |                      |  P1    |
```

```
+--------+                      +--------+
|  free  |                      |  P3    |
+--------+                      +--------+
|  P3    |                      |  P5    |
+--------+                      +--------+
|  free  |                      |        |
+--------+                      |  free  |
|  P5    |                      |        |
+--------+                      |        |
|  free  |                      +--------+
+--------+
```

The cost of compaction is $O(M)$ where M is the total memory size, because in the worst case every byte must be copied. For a system with gigabytes of memory, this can take milliseconds to seconds — an eternity for interactive or real-time applications.

Theorem 10.7 (Optimal Compaction is NP-Hard). Given an initial configuration of allocated blocks and free holes, finding the compaction strategy that minimises the total number of bytes moved is NP-hard (reduction from the 3-partition problem).

Proof sketch. We reduce from the NP-hard problem of partitioning a multiset of integers into triples with equal sums. Given an instance of 3-partition, construct a memory configuration where each integer corresponds to a process size and the "bins" correspond to target positions. Minimising bytes moved in compaction corresponds to finding a valid 3-partition. □

In practice, operating systems use simple heuristics: move all allocated blocks to one end of memory (e.g., the low end), which is suboptimal but easy to implement. The cost is at most $\sum_i s_i$ bytes moved (the total size of all allocated blocks).

Programmer's Perspective

Modern systems almost never perform compaction of physical memory. Instead, they use paging (Chapter 11) to eliminate external fragmentation entirely: since processes are divided into fixed-size pages, and any page can be placed in any frame, contiguity in physical memory is unnecessary. However, compaction does appear in other contexts. Java's G1 and ZGC collectors compact live objects to reduce fragmentation. At the kernel level, Linux's `kcompactd` daemon performs *memory compaction* — not of process memory, but of free pages — to create contiguous blocks needed for huge pages and DMA buffers. You can trigger compaction manually:

```
echo 1 > /proc/sys/vm/compact_memory
```

And check the compaction status in `/proc/vmstat` (look for `compact_*` counters). The Go garbage collector uses a non-moving design, avoiding compaction entirely at the cost of some fragmentation, managed through the size-class system described in Chapter 13.

10.7 Swapping

When the total memory demands of all active processes exceed the available physical memory, the operating system can use *swapping* to temporarily move entire processes out of memory.

10.7.1 Basic Swapping

Definition 10.21 (Swapping). *Swapping* is the mechanism by which the operating system moves an entire process's memory image between main memory (RAM) and a secondary storage device (the *backing store* or *swap space*). A process that has been moved to the backing store is said to be *swapped out*; one that has been brought back is *swapped in*.

The basic swapping algorithm works as follows:

1. When a process must be loaded and no free memory is available, the scheduler selects a victim process currently in memory.
2. The victim's entire memory image is written to the backing store (*swap out*).
3. The new process is loaded into the freed memory (*swap in*).
4. The victim process is marked as "swapped out" in the process table.
5. When the victim is scheduled again, it must be swapped back in.

10.7.2 Backing Store

The backing store is a region of secondary storage (usually a disk partition or a file) reserved for swapped-out process images.

Definition 10.22 (Backing Store). The *backing store* is a contiguous region of secondary storage large enough to hold the memory images of all processes that might be swapped out simultaneously. It must support fast, direct access to any process's image.

The backing store must be at least as large as the total virtual memory of all swappable processes. On Linux, this is the *swap partition* (or *swap file*). The `mkswap` and `swapon` commands configure swap space.

Example 10.20 (Swap Configuration on Linux). Creating and enabling a 4 GB swap file:

```
# Create the swap file
dd if=/dev/zero of=/swapfile bs=1M count=4096

# Set permissions (only root should access swap)
chmod 600 /swapfile

# Format as swap
mkswap /swapfile

# Enable the swap
swapon /swapfile

# Verify
swapon --show
# NAME      TYPE SIZE  USED PRIO
# /swapfile file   4G   0B   -2
```

10.7.3 Cost of Swapping

The time to swap a process is dominated by the data transfer time. If a process occupies S bytes and the disk transfer rate is R bytes per second, the swap time is:

$$T_{\text{swap}} = \frac{S}{R}$$

For a complete swap (out and in), the total time is $2T_{\text{swap}}$.

Example 10.21 (Swap Time Comparison Across Storage Technologies).

A process occupies 100 MB. We compare three storage technologies:

Storage	Transfer Rate	Swap-Out Time	Swap-In Time	Total Cycle
HDD (7200 RPM)	150 MB/s	667 ms	667 ms	1,334 ms
SATA SSD	500 MB/s	200 ms	200 ms	400 ms
NVMe SSD	3,500 MB/s	29 ms	29 ms	57 ms

Even with the fastest NVMe SSD, swapping a 100 MB process takes 57 ms — enough to cause a noticeable stutter in an interactive application refreshing at 60 Hz (16.7 ms per frame).

Example 10.22 (Swap Time with Seek Latency). For a spinning HDD, the swap time includes seek time and rotational latency in addition to transfer time:

$$T_{\text{swap}} = T_{\text{seek}} + T_{\text{rotate}} + \frac{S}{R}$$

With $T_{\text{seek}} = 5$ ms, $T_{\text{rotate}} = 4$ ms (average for 7200 RPM), $R = 150$ MB/s, and $S = 100$ MB:

$$T_{\text{swap}} = 5 + 4 + \frac{100}{0.15} = 9 + 667 = 676 \text{ ms}$$

The seek and rotational latencies are negligible for large transfers but significant for small ones (swapping a single 4 KB page: $5 + 4 + 0.027 \approx 9$ ms, almost entirely latency).

10.7.4 Swapping Policy

Not all processes are equally good candidates for swapping. The OS considers:

- **Process state:** A process blocked on I/O should not be swapped out, because the I/O buffer may be in its address space (DMA writes to physical memory).
- **Priority:** Low-priority processes are preferred victims.
- **Time in memory:** A process that was recently swapped in should not be immediately swapped out again (hysteresis).
- **Size:** Smaller processes are cheaper to swap (less data to transfer).

10.7.5 Partial Swapping and Paging

The major drawback of basic swapping is its all-or-nothing nature: the entire process must be moved, even if only a small portion of its memory is actively used. This observation motivates *demand paging* (Chapter 11), which swaps individual pages rather than entire processes. Demand paging is the dominant memory management strategy in all modern operating systems.

Note

Modern operating systems (Linux, Windows, macOS) no longer perform process-level swapping in the traditional sense. Instead, they use demand paging with page-level swap. The kernel writes individual dirty pages to swap space and reads them back on demand. The term "swapping" persists in common usage (e.g., "the system is swapping") but refers to this page-level mechanism, not whole-process swaps.

10.8 The Memory Hierarchy Perspective

Memory management does not exist in isolation. It is part of a larger memory hierarchy designed to bridge the speed gap between the processor and storage.

Level	Technology	Typical Size	Access Time	Cost per GB
Registers	SRAM (on-die)	~1 KB	0.3 ns	–
L1 Cache	SRAM	32–64 KB	1 ns	~$1000
L2 Cache	SRAM	256 KB–1 MB	3–5 ns	~$100
L3 Cache	SRAM	8–64 MB	10–20 ns	~$30
Main Memory	DRAM	8–256 GB	50–100 ns	~$3
SSD	NAND Flash	0.5–8 TB	10–100 μs	~$0.10
HDD	Magnetic	1–20 TB	5–10 ms	~$0.02

The operating system's memory management subsystem operates primarily at the DRAM level, managing the allocation of physical frames to processes. Caching is handled by hardware (L1–L3) and by the OS page cache (for file data). Swapping extends the effective memory to include SSD/HDD at a dramatic performance cost.

Definition 10.23 (Effective Access Time with Swapping). If a fraction p of memory accesses cause a page to be swapped in (the *swap rate*), the effective access time is:

$$\text{EAT} = (1 - p) \times t_{\text{mem}} + p \times t_{\text{swap}}$$

where t_{mem} is the memory access time and t_{swap} is the time to service a swap (including disk I/O and context switching).

Example 10.23 (EAT with Swapping). $t_{\text{mem}} = 100$ ns, $t_{\text{swap}} = 10$ ms $= 10{,}000{,}000$ ns, $p = 0.001$ (one in a thousand accesses causes a swap).

$$\text{EAT} = 0.999 \times 100 + 0.001 \times 10{,}000{,}000 = 99.9 + 10{,}000 = 10{,}099.9 \text{ ns}$$

Even with a swap rate of just 0.1%, the effective access time increases by a factor of 100. This illustrates why swap avoidance is critical for performance.

Example 10.24 (EAT with SSD Swap). Same scenario but with NVMe SSD: $t_{\text{swap}} = 80\ \mu\text{s} = 80{,}000$ ns.

$$\text{EAT} = 0.999 \times 100 + 0.001 \times 80{,}000 = 99.9 + 80 = 179.9 \text{ ns}$$

Slowdown factor: $179.9/100 = 1.8\times$. Significantly better than HDD, but still a 80% performance penalty at a 0.1% swap rate.

10.9 Memory Layout of a Process

Understanding what a process's memory looks like is essential for both operating system designers and application programmers. A typical process on a modern Unix-like system has the following layout in its logical address space:

```
High address
+-------------------+
|     Stack         |  (grows downward)
|        |          |
|        v          |
+-------------------+
|                   |
|  (unmapped gap)   |
```

```
|                   |
+-------------------+
|        ^          |
|        |          |
|     Heap          |  (grows upward)
+-------------------+
|     BSS           |  (uninitialised globals, zero-filled)
+-------------------+
|     Data          |  (initialised globals)
+-------------------+
|     Text          |  (executable code, read-only)
+-------------------+
Low address
```

Definition 10.24 (Process Memory Segments). A process's logical address space is divided into *segments*:

- **Text (code) segment:** Contains the executable machine instructions. Typically read-only and shared among processes running the same program.
- **Data segment:** Contains initialised global and static variables. Read-write.
- **BSS segment:** Contains uninitialised global and static variables, zero-filled by the loader. Named after a historical assembler directive (*Block Started by Symbol*). Read-write.
- **Heap:** Dynamically allocated memory (via `malloc`, `new`, etc.), growing upward from the end of BSS.
- **Stack:** Function call frames, local variables, and return addresses, growing downward from a high address.

Example 10.25 (Segment Sizes). A compiled C program has the following segment sizes (from the `size` command):

```
$ size /usr/bin/ls
   text    data     bss     dec     hex filename
 138744    4784    4832  148360   243a8 /usr/bin/ls
```

The text segment is 135 KB, data is 4.7 KB, and BSS is 4.7 KB. The total static size is about 145 KB. The heap and stack grow dynamically during execution.

Programmer's Perspective: Linux /proc/PID/maps and Go's Memory Layout.

On Linux, you can inspect any process's memory layout through the `/proc` filesystem. The file `/proc/PID/maps` lists every mapped region in the process's address space:

```
/* Compile with: gcc -o maps_demo maps_demo.c */
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_init = 42;       /* .data segment */
int global_uninit;           /* .bss segment  */

int main(void) {
    int stack_var = 7;       /* stack */
    int *heap_var = malloc(sizeof(int));  /* heap */
    *heap_var = 99;

    printf("Text  (main):    %p\n", (void *)main);
    printf("Data  (g_init):  %p\n", (void *)&global_init);
    printf("BSS   (g_uninit):%p\n", (void *)&global_uninit);
    printf("Heap  (malloc):  %p\n", (void *)heap_var);
    printf("Stack (local):   %p\n", (void *)&stack_var);
    printf("\n--- /proc/%d/maps (excerpt) ---\n", getpid());

    char cmd[64];
    snprintf(cmd, sizeof(cmd), "cat /proc/%d/maps", getpid());
    system(cmd);

    free(heap_var);
    return 0;
}
```

Running this program produces output like:

```
Text  (main):    0x55a3b2c00149
Data  (g_init):  0x55a3b2e01010
BSS   (g_uninit):0x55a3b2e01014
Heap  (malloc):  0x55a3b40002a0
Stack (local):   0x7ffd4a3e1c5c
```

Notice the enormous gap between the heap (~0x55...) and the stack (~0x7ffd...). This is the unmapped region where memory-mapped files and shared libraries are loaded. The `maps` file shows each region with its address range, permissions, offset, device, inode, and pathname:

```
55a3b2c00000-55a3b2c01000 r-xp 00000000 08:01 12345  /home/user/maps_demo
55a3b2e01000-55a3b2e02000 rw-p 00001000 08:01 12345  /home/user/maps_demo
```

```
55a3b4000000-55a3b4021000 rw-p 00000000 00:00 0      [heap]
7f4a38000000-7f4a381c0000 r-xp 00000000 08:01 67890  /lib/x86_64-linux-gnu/libc-2.31.so
7ffd4a3c0000-7ffd4a3e2000 rw-p 00000000 00:00 0      [stack]
```

The permissions column uses the standard `rwxp/rwxs` format: `r` = read, `w` = write, `x` = execute, `p` = private (CoW), `s` = shared.

Go's memory layout differs significantly from C. The Go runtime reserves a large virtual address region at startup and manages it internally. The `runtime.ReadMemStats` function reports heap statistics, and `GODEBUG=gctrace=1` enables garbage collector tracing. Go goroutine stacks are allocated on the heap (not the OS stack), start small (2–8 KB), and grow dynamically by copying. This means that `&variable` in Go may point to an address that changes over time, which is why Go forbids storing Go pointers in C memory when using cgo. You can inspect a Go process's memory layout with:

```go
package main

import (
    "fmt"
    "os"
    "runtime"
)

func main() {
    var m runtime.MemStats
    runtime.ReadMemStats(&m)
    fmt.Printf("HeapAlloc:  %d MB\n", m.HeapAlloc/1024/1024)
    fmt.Printf("HeapSys:    %d MB\n", m.HeapSys/1024/1024)
    fmt.Printf("StackSys:   %d MB\n", m.StackSys/1024/1024)
    fmt.Printf("Sys (total):%d MB\n", m.Sys/1024/1024)

    pid := os.Getpid()
    data, _ := os.ReadFile(fmt.Sprintf("/proc/%d/maps", pid))
    lines := 0
    for i, b := range data {
        if b == '\n' {
            lines++
            if lines >= 15 {
                fmt.Print(string(data[:i+1]))
                break
```

```
            }
        }
    }
}
```

10.10 Protection and Sharing

Memory management must enforce two orthogonal goals: *protection* (preventing processes from interfering with each other) and *sharing* (allowing processes to access common data efficiently).

10.10.1 Protection Mechanisms

The base-limit mechanism provides basic protection: a process cannot access any address outside $[B, B + L)$. More sophisticated protection requires finer granularity:

- **Read/Write/Execute permissions:** Each memory region can be marked as readable, writable, executable, or any combination. The text segment is typically read+execute; the data and heap are read+write; the stack is read+write (and sometimes no-execute for security).
- **Ring-based protection:** x86 processors define four privilege rings (0–3). Ring 0 (kernel) can access all memory; Ring 3 (user) can only access memory permitted by the page tables.
- **Supervisor/user bit:** Each page table entry on x86-64 has a U/S bit. When U/S = 0, only supervisor mode (ring 0) can access the page.

Definition 10.25 (Protection Domain). A *protection domain* specifies, for each object (memory region, file, device), the set of operations a process can perform on it. The memory management hardware enforces protection domains for memory objects through permission bits in page table entries or segment descriptors.

10.10.2 Sharing

When multiple processes run the same program (e.g., multiple instances of a web server), they can share the text segment to save memory. With base-limit registers, sharing is impossible: each process has a single contiguous region. Paging (Chapter 11) solves this by allowing different page table entries in different processes to point to the same physical frame.

Example 10.26 (Shared Library Savings). The GNU C Library (glibc) is approximately 2 MB in size. On a typical Linux system with 200 processes, each using glibc:

- Without sharing: 200×2 MB $= 400$ MB
- With sharing: 2 MB (one copy in physical memory, mapped into all 200 address spaces)

Savings: 398 MB. This is why shared libraries exist.

Example 10.27 (Shared Libraries on a Real System). Using `pmap` on a Linux system:

```
$ pmap -x $(pidof bash) | head -10
Address           Kbytes     RSS   Dirty Mode  Mapping
000055a7e1200000     880     880       0 r-x-- bash
000055a7e14dc000      16      16      16 r---- bash
000055a7e14e0000      36      36      36 rw--- bash
00007f9c42600000    1820    1200       0 r-x-- libc-2.31.so
00007f9c427ca000    2048       0       0 ----- libc-2.31.so
00007f9c429ca000      16      16      16 r---- libc-2.31.so
```

The `r-x--` regions are shared (read-only text segments). The RSS column shows only the resident pages; the actual physical memory is shared among all processes using the same library.

10.10.3 Memory Protection Keys (MPK)

Modern x86-64 processors (Skylake and later) support *Memory Protection Keys*, a hardware feature that allows user-space code to change memory permissions without system calls.

Definition 10.26 (Memory Protection Keys). *Intel MPK* assigns each page a 4-bit protection key (one of 16 domains). A per-thread register (PKRU) specifies, for each key, whether access is disabled or write-disabled. Changing the PKRU register is a user-space instruction (WRPKRU), taking ~20 cycles — far faster than a system call to `mprotect()`.

MPK is used for security (isolating sensitive data within a single address space) and for performance (fast toggling of write protection for garbage collectors and software transactional memory).

10.11 Segmentation

Before paging became dominant, *segmentation* was the primary mechanism for providing logical address spaces with multiple regions of varying sizes.

Definition 10.27 (Segmentation). *Segmentation* divides a process's logical address space into a collection of *segments*, each with a name (or number), a base address, and a length. A logical address is a pair (segment number, offset). The segment table maps each segment number to a physical base address and a limit.

A logical address in a segmented system has the form (s, d) where s is the segment number and d is the offset within the segment. The hardware translates this as:

1. Look up segment s in the segment table to obtain base b_s and limit l_s.
2. Check $d < l_s$. If not, raise a trap.
3. Compute physical address $p = b_s + d$.

Example 10.28 (Segment Table). A process has four segments:

Segment	Name	Base	Limit
0	Code	0x4000	0x1000
1	Data	0x6000	0x0800
2	Heap	0x8000	0x2000
3	Stack	0xF000	0x1000

Logical address (2, 0x0100) translates to: $d = 0\text{x}0100 < 0\text{x}2000$ (pass), physical address $= 0\text{x}8000 + 0\text{x}0100 = 0\text{x}8100$.

Logical address (1, 0x0900) fails: $0\text{x}0900 \not< 0\text{x}0800$, segmentation fault.

Example 10.29 (Sharing via Segmentation). Two processes, P1 and P2, run the same editor program with different data.

P1's segment table:

Segment	Base	Limit
0 (code)	0x4000	0x3000
1 (data)	0xA000	0x2000

P2's segment table:

Segment	Base	Limit
0 (code)	0x4000	0x3000
1 (data)	0xD000	0x1800

Both processes share segment 0 (same base), which contains the editor code. Each has its own data segment. This saves 0x3000 bytes (12 KB) of physical memory.

Segmentation provides a natural model of program structure (code, data, stack are separate segments) and supports sharing (two processes can share a segment by having the same base in their segment tables). However, segmentation suffers from external fragmentation (segments are variable-sized contiguous blocks) and has largely been superseded by paging and paged segmentation.

Note

The Intel x86 architecture historically supported both segmentation and paging. In 64-bit long mode (x86-64), segmentation is largely disabled: the segment base is forced to zero for code and data segments, effectively giving each process a flat address space. Only the FS and GS segment registers retain non-zero bases, used by the OS for per-CPU data and by user space for thread-local storage (TLS).

10.12 Paged Segmentation

Some systems combine segmentation and paging to get the logical benefits of both.

Definition 10.28 (Paged Segmentation). In *paged segmentation*, each segment is divided into fixed-size pages. A logical address is a triple (segment number, page number, offset). The segment table entry for segment s points to a page table for that segment, and the page table maps pages to physical frames.

This hybrid approach eliminates external fragmentation (because pages are fixed-size) while preserving the logical structure of segmentation (separate code, data, stack segments with independent protection bits). The Intel 386 through Pentium 4 (in 32-bit mode) used this scheme.

The address translation for paged segmentation proceeds in four steps:

1. Use the segment number to index the segment table, obtaining the page table base address and segment limit.
2. Check that the (page number, offset) does not exceed the segment limit.
3. Use the page number to index the segment's page table, obtaining the physical frame number.
4. Concatenate the frame number with the offset to produce the physical address.

Example 10.30 (Paged Segmentation Translation). A system has 4 segments, 256 pages per segment, and 4 KB pages. A logical address is (segment=2, page=10, offset=0x0A0).

1. Segment table[2] gives: page table base at physical address 0x50000, limit = 200 pages.
2. Check: page 10 < 200 (pass).
3. Page table at 0x50000, entry 10 gives: frame number = 0x3A.
4. Physical address: $0\text{x}3\text{A} \times 0\text{x}1000 + 0\text{x}0\text{A}0 = 0\text{x}3\text{A}0\text{A}0$.

10.13 Historical Context

Memory management has evolved dramatically over the history of computing:

1950s (No OS): Programs were loaded at fixed addresses. The programmer managed memory manually.

1960s (Batch processing): Simple partitioning schemes (fixed partitions) allowed multiple jobs to reside in memory. IBM's OS/360 introduced relocatable code.

1964 (Multics): The first system to use segmentation with paging. Multics provided a flat, single-level virtual address space across all processes.

1970s (Unix): The original Unix used swapping (whole-process swap). The VAX-11/780 (1978) introduced demand paging to Unix (BSD).

1985 (Intel 80386): Hardware paging support in a mainstream processor, combining segmentation and paging.

1990s–2000s: Virtual memory with demand paging became universal. 64-bit address spaces (Alpha, SPARC64, x86-64) removed the 4 GB barrier.

2010s–present: Focus shifted to performance: huge pages, TLB optimisation, NUMA awareness, memory compression, and hardware support for memory protection (MPK, CET).

10.14 Summary

This chapter has established the fundamental concepts of memory management:

- **Logical vs physical addresses:** Programs use logical addresses; the MMU translates them to physical addresses.
- **Address binding** can occur at compile time, load time, or execution time, with increasing flexibility.
- **Base-limit registers** provide the simplest hardware-supported address translation and protection.
- **Contiguous allocation** (fixed and variable partitions) assigns each process a single block of physical memory.
- **Allocation strategies** (first-fit, best-fit, worst-fit, next-fit) determine which free hole satisfies a request, with first-fit generally preferred.
- **Fragmentation** (internal and external) is the inevitable cost of memory allocation; compaction is the brute-force remedy but is costly and NP-hard to optimise.
- **Swapping** moves entire processes between memory and disk, but its coarse granularity makes it inefficient.
- **The memory hierarchy** spans registers to disk, with access times ranging from 0.3 ns to 10 ms, a factor of 10^7.
- **Segmentation** provides a structured logical address space but suffers from external fragmentation.
- **Paged segmentation** combines the structure of segmentation with the fragmentation-free property of paging.

The critical limitation of all schemes in this chapter is the requirement for *contiguous* physical allocation (or contiguous segments). Chapter 11 introduces *virtual memory* with *paging*, which shatters this constraint by allowing a process's pages to be scattered anywhere in physical memory, dramatically improving both memory utilisation and the sophistication of memory management.

Exercise 10.1. A system has a 32-bit logical address space and a 24-bit physical address space, with a page size of 4 KB. Calculate: (a) the number of pages in the logical address space, (b) the number of frames in the physical address space, (c) the number of bits in the page offset, (d) the number of bits in the virtual page number, and (e) the number of bits in the physical frame number.

Exercise 10.2. A variable-partition system has the following free holes (in order): 150 KB, 350 KB, 200 KB, 400 KB, 100 KB. Determine which hole is selected for a 180 KB request under (a) first-fit, (b) best-fit, (c) worst-fit, and (d) next-fit (assuming

the last allocation ended at the beginning of the list). After the allocation, list the remaining free holes for each strategy.

Exercise 10.3. Prove that for a system with P active processes of equal size s and total memory $M = Ps + F$ (where F is total free memory), the expected number of holes under first-fit allocation in steady state is $P/2$, and the expected total external fragmentation is $Ps/2 \cdot k$ where k is the average ratio of hole size to process size. Under what assumptions does the fifty-percent rule hold exactly?

Exercise 10.4. A system uses base-limit address translation. Process A has base = 0x20000 and limit = 0x8000. Process B has base = 0x30000 and limit = 0x5000. (a) Can Process A access physical address 0x27000? Show the translation. (b) Can Process A access physical address 0x29000? Explain. (c) Do the address spaces of A and B overlap in physical memory? Could this cause a protection violation?

Exercise 10.5. Consider a system with 1 GB of physical memory using variable-partition allocation. Processes arrive and depart in this sequence: P1 (200 MB) arrives, P2 (300 MB) arrives, P3 (150 MB) arrives, P2 departs, P4 (280 MB) arrives (first-fit), P1 departs, P5 (200 MB) arrives (first-fit). Draw the memory layout after each event and calculate the external fragmentation ratio F_{ext} at each step.

Exercise 10.6. The effective access time with swapping is EAT $= (1 - p) \times t_{\text{mem}} + p \times t_{\text{swap}}$. (a) If $t_{\text{mem}} = 80$ ns and $t_{\text{swap}} = 8$ ms, what swap rate p causes a 100% slowdown (EAT = $2 \times t_{\text{mem}}$)? (b) If the acceptable performance degradation is at most 10%, what is the maximum tolerable swap rate? (c) Repeat parts (a) and (b) for an NVMe SSD with $t_{\text{swap}} = 80$ μs. How does the faster storage change the feasibility of swapping?

Exercise 10.7. A segmented memory system has the following segment table:

Segment	Base	Limit
0	0x0200	0x0400
1	0x0900	0x0100
2	0x1400	0x0800
3	0x2000	0x0300

Translate the following logical addresses to physical addresses, or indicate a segmentation fault: (a) (0, 0x0100), (b) (1, 0x00FF), (c) (2, 0x0900), (d) (3, 0x02FF), (e) (0, 0x0400). For each valid translation, show the computation.

Chapter 11: Virtual Memory

Chapter 10 established the fundamentals: logical addresses, physical addresses, contiguous allocation, and the fragmentation that plagues it. But contiguous allocation has a fatal flaw — it requires that every byte of a process reside in physical memory before the process can run, and that these bytes form a contiguous block. Both requirements are absurd for modern systems where processes have gigabytes of virtual address space but use only a fraction at any moment.

Virtual memory shatters both constraints. It allows a process to execute with only part of its memory in physical RAM, loading pages on demand as the program touches them. And because it maps fixed-size virtual pages to fixed-size physical frames, a process's memory can be scattered across physical RAM in any order. The result is a system where each process sees a large, private, contiguous address space while the operating system juggles a much smaller physical memory behind the scenes.

This chapter covers the mechanisms that make virtual memory work: demand paging, page tables (from single-level to the four-level tables of x86-64), the Translation Lookaside Buffer that makes it fast, and the subtleties of shared pages, copy-on-write, and page size trade-offs.

11.1 Demand Paging

11.1.1 The Concept

Definition 11.1 (Demand Paging). *Demand paging* is a memory management scheme in which pages of a process are loaded into physical memory only when they are accessed (demanded) by the CPU. Pages that are never accessed are never loaded. This is also called *lazy loading*.

When a process is created, none of its pages need to be in physical memory. The operating system sets up the page table entries to indicate "not present." As the process executes and references an address, the MMU checks the page table entry. If the page is present, the translation proceeds normally. If the page is not present, the MMU raises a *page fault*.

The distinction between demand paging and ordinary paging is important. Ordinary

paging loads all pages of a process before execution begins. Demand paging loads nothing in advance — every page is loaded only when first touched. This dramatically reduces startup time and memory consumption, especially for large programs that use only a fraction of their code and data.

11.1.2 Page Faults

Definition 11.2 (Page Fault). A *page fault* is a hardware exception raised by the MMU when a process attempts to access a page that is not currently mapped to a physical frame (i.e., the present bit in the page table entry is 0). The page fault transfers control to the operating system's page fault handler.

The page fault handling sequence is:

1. **Trap to kernel:** The CPU saves the faulting address (in the CR2 register on x86) and transfers control to the page fault handler. The current instruction is suspended.
2. **Validate the reference:** The OS checks whether the faulting address is a valid part of the process's address space. It consults the process's memory map (on Linux, the `vm_area_struct` list). If the address is not in any valid region (e.g., the process dereferenced a null pointer into an unmapped region), the OS delivers a segmentation fault signal (SIGSEGV on Unix).
3. **Classify the fault:** Valid faults fall into several categories:
   - *File-backed fault:* The page is mapped to a file (executable code, shared library, mmap'd file). The page must be read from the file.
   - *Anonymous fault:* The page is anonymous (heap, stack, mmap'd anonymous). If first access, the page is zero-filled. If previously swapped, the page must be read from swap.
   - *CoW fault:* The page is present but read-only due to copy-on-write. A new frame must be allocated and the page copied.
4. **Find a free frame:** The OS locates a free physical frame from the free list. If no free frame exists, the OS must evict an existing page (page replacement, Chapter 12).
5. **Read the page:** The OS issues a disk I/O request to read the page from the backing store (swap space or the executable file on disk) into the free frame. The process is blocked during this I/O.
6. **Update the page table:** Once the I/O completes, the OS sets the page table entry to map the virtual page to the physical frame and sets the present bit to 1.

7. **Restart the instruction:** The CPU re-executes the instruction that caused the fault. This time, the translation succeeds.

Example 11.1 (Page Fault Sequence). A process executes `MOV R1, [0x7FA0]`. The virtual page number for address 0x7FA0 (with 4 KB pages) is $\lfloor 0\text{x}7\text{FA}0/4096 \rfloor = 31$. The page table entry for page 31 has present = 0.

1. MMU raises page fault. CR2 $\leftarrow$ 0x7FA0.
2. OS validates: page 31 is a valid heap page.
3. OS classifies: anonymous first-access (zero-fill).
4. OS finds free frame 42.
5. OS zero-fills frame 42 (no disk I/O needed for a first-access anonymous page).
6. OS sets PTE[31] $\leftarrow$ (frame=42, present=1, dirty=0, accessed=1, R/W=1).
7. CPU re-executes `MOV R1, [0x7FA0]`. Translation: frame 42, offset 0xFA0. Physical address = $42 \times 4096 + 0\text{xFA}0 = 0\text{x}2\text{AFA}0$.

Example 11.2 (File-Backed Page Fault). A process executes code at virtual address 0x00400100. This is in the text segment, which is mapped to the executable file on disk.

1. Page fault on VPN = 0x400100 / 4096 = 0x400 (page 1024).
2. OS validates: this is the text segment, mapped to file `/usr/bin/myapp`, offset 0x100000.
3. OS finds free frame 200.
4. OS reads 4 KB from the file at offset 0x100000 into frame 200. This requires disk I/O.
5. OS updates PTE: frame=200, present=1, R/W=0 (text is read-only), NX=0 (executable).
6. CPU re-executes the instruction at 0x00400100.

11.1.3 Types of Page Faults

Definition 11.3 (Minor vs Major Page Fault).

- A *minor page fault* (soft fault) is one that can be resolved without disk I/O. The page is already in memory (e.g., in the page cache or shared by another process) but not yet mapped into the faulting process's page table.
- A *major page fault* (hard fault) requires disk I/O to read the page from storage (swap device or file system). Major faults are 1000–100,000 times slower than minor faults.

On Linux, `ps -o min_flt,maj_flt PID` reports the minor and major fault counts for a process. A well-tuned system should have very few major faults during steady-state

operation.

Example 11.3 (Fault Counts). A program processes a 500 MB data file:

```
$ /usr/bin/time -v ./process_data big_file.dat 2>&1 | grep -i fault
Minor (reclaiming a frame) page faults: 128432
Major (requiring I/O) page faults: 12
```

The 128,432 minor faults correspond to mapping pages already in the page cache (previously read by the kernel's readahead mechanism). The 12 major faults correspond to pages not yet in the cache — perhaps the first pages of the file before readahead kicks in.

11.1.4 Performance of Demand Paging

The key performance metric is the *page fault rate*, denoted p, defined as the probability that a given memory access causes a page fault.

Definition 11.4 (Effective Access Time for Demand Paging). If t_{mem} is the memory access time and t_{fault} is the time to service a page fault, the effective access time is:

$$\text{EAT} = (1 - p) \times t_{\text{mem}} + p \times t_{\text{fault}}$$

The page fault service time t_{fault} includes:

- Trap overhead: ~1 μs
- OS processing (validate, allocate frame, set up I/O): ~5 μs
- Page-in from disk: ~2–10 ms (HDD) or ~50–200 μs (SSD)
- Page table update and TLB invalidation: ~1 μs
- Process restart overhead: ~1 μs

Example 11.4 (EAT Calculation). $t_{\text{mem}} = 100$ ns. $t_{\text{fault}} = 8$ ms $= 8{,}000{,}000$ ns (HDD-backed).

$$\text{EAT} = (1 - p) \times 100 + p \times 8{,}000{,}000$$

For a 10% slowdown (EAT ≤ 110 ns):

$$110 \geq 100 - 100p + 8{,}000{,}000p$$

$$10 \geq 7{,}999{,}900p$$

$$p \leq 1.25 \times 10^{-6}$$

The page fault rate must be less than about one in 800,000 accesses to keep the slowdown under 10%. This is why effective page replacement algorithms (Chapter 12) are critical.

Theorem 11.1 (Page Fault Rate Bound for Acceptable Performance). For a system with memory access time t_{mem}, page fault service time t_{fault}, and acceptable slowdown factor $\alpha > 1$, the page fault rate must satisfy:

$$p \leq \frac{(\alpha - 1) \cdot t_{\text{mem}}}{t_{\text{fault}} - t_{\text{mem}}}$$

Proof. We require EAT $\leq \alpha \cdot t_{\text{mem}}$:

$$\begin{aligned}
(1 - p) \cdot t_{\text{mem}} + p \cdot t_{\text{fault}} &\leq \alpha \cdot t_{\text{mem}} \\
t_{\text{mem}} + p(t_{\text{fault}} - t_{\text{mem}}) &\leq \alpha \cdot t_{\text{mem}} \\
p(t_{\text{fault}} - t_{\text{mem}}) &\leq (\alpha - 1) \cdot t_{\text{mem}} \\
p &\leq \frac{(\alpha - 1) \cdot t_{\text{mem}}}{t_{\text{fault}} - t_{\text{mem}}} \quad \square
\end{aligned}$$

Example 11.5 (Acceptable Fault Rates Across Storage). Using the bound from Theorem 11.1 with $\alpha = 1.1$ (10% slowdown) and $t_{\text{mem}} = 100$ ns:

Storage	t_{fault}	**Max** p	**Max fault rate**
HDD	8 ms	1.25×10^{-6}	1 in 800,000
SATA SSD	200 μs	5.0×10^{-5}	1 in 20,000
NVMe SSD	50 μs	2.0×10^{-4}	1 in 5,000

With NVMe storage, the system can tolerate 160 times more page faults than with HDD for the same performance impact. This has profound implications for memory management policy: aggressive overcommitment is more viable with fast storage.

11.1.5 Pure Demand Paging vs Prepaging

Pure demand paging never loads a page until it is faulted. This means a process starts with zero pages in memory, and the first few instructions cause a cascade of page faults (one for the code page, one for the stack page, one for initialised data, etc.). This initial burst is called *cold start* or *cold cache* behaviour.

Prepaging (or *prefetching*) loads several pages at once, predicting which pages the process will need soon. The spatial locality principle suggests that if a process accesses page k, it is likely to access pages $k+1$, $k+2$, etc. Modern operating systems use a combination: demand paging with readahead (prefetching a window of pages beyond the faulted page).

Definition 11.5 (Readahead). *Readahead* is a prefetching strategy where, upon a page fault for page k, the OS loads not only page k but also pages $k+1, k+2, \ldots, k+W-1$, where W is the readahead window. If the process's access pattern is sequential, these prefetched pages avoid future faults.

Example 11.6 (Readahead Effectiveness). A process reads a 1 MB file sequentially. Page size: 4 KB. Total pages: 256.

Without readahead: 256 major page faults (one per page), each costing ~8 ms. Total: $256 \times 8 = 2048$ ms ≈ 2 seconds.

With readahead (window = 32 pages = 128 KB): the first fault triggers a read of 32 pages. Subsequent faults trigger additional readahead. Total major faults: $256/32 = 8$, but each reads 128 KB. Total: $8 \times 8 = 64$ ms (plus the amortised disk access is more efficient for larger reads). In practice, the sequential read rate of the disk (~150 MB/s for HDD, ~3 GB/s for NVMe) dominates, giving ~6.7 ms (HDD) or ~0.3 ms (NVMe).

Note

Linux's page cache implements aggressive readahead for file-backed pages. The `readahead()` system call allows applications to hint which file regions they will need. The `fadvise()` system call provides additional hints: `POSIX_FADV_SEQUENTIAL` increases the readahead window, `POSIX_FADV_RANDOM` disables it, and `POSIX_FADV_DONTNEED` tells the kernel to evict the pages (useful after processing a file, to free page cache for other uses). For anonymous pages (heap, stack), Linux relies on demand paging without readahead, since access patterns for anonymous memory are less predictable than sequential file reads.

11.2 Page Tables

The page table is the data structure that maps virtual page numbers to physical frame numbers. Its design is one of the most important engineering decisions in a virtual memory system.

11.2.1 Single-Level Page Table

Definition 11.6 (Page Table). A *page table* is an array indexed by the virtual page number (VPN). Each entry, called a *page table entry* (PTE), contains the physical frame number and a set of control bits (present, dirty, accessed, protection, etc.).

For a system with a v-bit virtual address and page size 2^p bytes, the parameters are:

- Number of virtual pages: $N_{\text{pages}} = 2^{v-p}$
- Page table entries: $N_{\text{PTE}} = 2^{v-p}$
- VPN bits: $v - p$
- Offset bits: p
- Physical frame number bits: PA width $- p$

Example 11.6a (Page Table Parameters). Common configurations:

System	VA bits (v)	PA bits	Page (2^p)	VPN bits	Offset	Pages	PTE size	PT size
16-bit micro	16	16	256 B	8	8	256	2 B	512 B
32-bit x86	32	32	4 KB	20	12	1M	4 B	4 MB
32-bit PAE	32	36	4 KB	20	12	1M	8 B	8 MB
48-bit x86-64	48	52	4 KB	36	12	64G	8 B	512 GB

The last row shows why single-level page tables are infeasible for 64-bit systems.

The address translation for a single-level page table:

1. Extract VPN from the virtual address: $\text{VPN} = \lfloor \text{VA}/2^p \rfloor = \text{VA} \gg p$
2. Index the page table: $\text{PTE} = \text{PageTable}[\text{VPN}]$
3. Check the present bit. If 0, raise page fault.
4. Extract the frame number: $\text{FN} = \text{PTE.frame}$
5. Compute physical address: $\text{PA} = \text{FN} \times 2^p + (\text{VA mod } 2^p)$

Or equivalently: $\text{PA} = (\text{FN} \ll p) \mid (\text{VA} \,\&\, (2^p - 1))$

Example 11.7 (Single-Level Translation). A system has 16-bit virtual addresses and 4 KB pages ($p = 12$).

- VPN bits: $16 - 12 = 4$ bits, giving 16 pages

- Offset bits: 12 bits

Virtual address 0x3A7F: - Binary: 0011 1010 0111 1111 - VPN: 0011 = 3 (top 4 bits) - Offset: 1010 0111 1111 = 0xA7F (bottom 12 bits) - Suppose PTE[3].frame = 7. Physical address: $(7 \ll 12) \mid \text{0xA7F} = \text{0x7000} + \text{0xA7F} = \text{0x7A7F}$.

Example 11.8 (Complete Page Table). A minimal 16-bit system with 4 KB pages (4-bit VPN). The page table for a process:

VPN	Frame	Present	R/W	Accessed	Dirty
0	5	1	0	1	0
1	9	1	1	1	1
2	–	0	–	–	–
3	2	1	1	0	0
4–14	–	0	–	–	–
15	12	1	1	1	0

Only pages 0, 1, 3, and 15 are present. Page 0 is read-only (code). Page 1 is read-write and dirty (modified data). Pages 2 and 4–14 are not present (not yet loaded or swapped out). Page 15 is the stack.

Translation of address 0x1B40: VPN = 1, offset = 0xB40. PTE[1]: present=1, frame=9. Physical address: 0x9B40.

Translation of address 0x2000: VPN = 2, offset = 0x000. PTE[2]: present=0. Page fault.

11.2.2 The Size Problem

A single-level page table for a 32-bit address space with 4 KB pages has $2^{20} = 1{,}048{,}576$ entries. If each entry is 4 bytes, the table is 4 MB. Every process needs its own page table, so 100 processes consume 400 MB of page table memory alone.

For a 64-bit address space (even using only 48 bits), the table would have 2^{36} entries — 256 GB per process. This is clearly infeasible.

Theorem 11.2 (Page Table Size). For a v-bit virtual address space with page size 2^p bytes and PTE size e bytes, the page table size is:

$$S = \frac{2^v}{2^p} \times e = 2^{v-p} \times e$$

For the table to fit in physical memory, we require $S \leq M$ where M is the total physical memory. This gives the constraint:

$$2^{v-p} \times e \leq M$$

For $v = 48$, $p = 12$, $e = 8$: $S = 2^{36} \times 8 = 512$ GB. This exceeds any reasonable physical memory size, mandating a hierarchical or inverted page table structure.

11.2.3 Multi-Level Page Tables

The solution is to *page the page table itself*. Instead of one enormous array, we use a tree of smaller tables.

Definition 11.7 (Multi-Level Page Table). A *k-level page table* is a tree of depth k where each internal node is a *page directory* containing pointers to the next level, and each leaf contains the physical frame number. The virtual page number is split into k fields, each used to index one level of the tree.

The key insight is sparsity: most processes use only a tiny fraction of their virtual address space. If a large region is unused, the corresponding subtree need not exist. The directory entry for that region is simply marked "not present," saving all the memory that would have been used for the missing page table pages.

Two-Level Page Table (32-bit x86):

A 32-bit virtual address with 4 KB pages is split as:

```
31          22 21          12 11           0
+------------+------------+--------------+
| Dir Index  | Table Index|   Offset     |
|  10 bits   |  10 bits   |   12 bits    |
+------------+------------+--------------+
```

- Level 1 (Page Directory): $2^{10} = 1024$ entries, each pointing to a page table.
- Level 2 (Page Table): $2^{10} = 1024$ entries, each mapping a page to a frame.

Each level fits in exactly one 4 KB page ($1024 \times 4 = 4096$ bytes), which is elegant and intentional.

Example 11.9 (Two-Level Savings). A 32-bit process uses only 12 MB of memory: 4 MB of code (starting at 0x08048000), 4 MB of heap, and 4 MB of stack (near the

top of the address space).

With a single-level page table: 2^{20} entries $\times$ 4 bytes = 4 MB.

With a two-level page table: - Page Directory: 1024 entries $\times$ 4 bytes = 4 KB (always present). - Code occupies VAs 0x08048000–0x0844FFFF. Directory entries: indices 32–33 (two entries). Page tables: 2, each 4 KB. - Heap (say 0x08450000–0x08850000): directory entries 33–34. Page tables: ~2. - Stack (say 0xBFC00000–0xC0000000): directory entries 767–768. Page tables: ~2. - Total page tables: ~6, each 4 KB. - Total: $4 + 6 \times 4 = 28$ KB.

Savings: 4 MB $\rightarrow$ 28 KB (a factor of ~146).

Example 11.10 (Two-Level Translation Step by Step). Virtual address: 0x0804A100 on a 32-bit x86 system with 4 KB pages and a two-level page table.

1. Split the address:
   - Directory index (bits 31–22): $0\text{x}0804\text{A}100 \gg 22 = 0\text{x}20 = 32$
   - Table index (bits 21–12): $(0\text{x}0804\text{A}100 \gg 12)\ \&\ 0\text{x}3\text{FF} = 0\text{x}4\text{A} = 74$
   - Offset (bits 11–0): $0\text{x}0804\text{A}100\ \&\ 0\text{xFFF} = 0\text{x}100$
2. Read Page Directory entry 32 from the page directory (base in CR3). Suppose it contains physical address 0x1F000 (the page table for this region).
3. Read Page Table entry 74 from the page table at 0x1F000. Suppose it contains frame number 0x3A0, present=1.
4. Physical address: $0\text{x}3\text{A}0 \times 0\text{x}1000 + 0\text{x}100 = 0\text{x}3\text{A}0100$.

11.2.4 Four-Level Page Table (x86-64)

Modern x86-64 processors use a 4-level page table to handle 48-bit virtual addresses:

Definition 11.8 (x86-64 Four-Level Page Table). The x86-64 architecture translates 48-bit virtual addresses using four levels of page tables:

Level	Name	Bits	Entries	Entry Size
4	PML4 (Page Map Level 4)	47–39	512	8 bytes
3	PDPT (Page Directory Pointer Table)	38–30	512	8 bytes
2	PD (Page Directory)	29–21	512	8 bytes
1	PT (Page Table)	20–12	512	8 bytes

Each level uses 9 bits of the virtual address to index 512 entries (each 8 bytes, fitting exactly in a 4 KB page). Bits 11–0 form the 12-bit page offset.

```
Virtual Address (48 bits used, 64 bits total):
63    48 47    39 38    30 29    21 20    12 11       0
+------+--------+--------+--------+--------+----------+
| Sign |  PML4  |  PDPT  |   PD   |   PT   |  Offset  |
|extend| 9 bits | 9 bits | 9 bits | 9 bits | 12 bits  |
+------+--------+--------+--------+--------+----------+
         |          |         |         |
         v          v         v         v
       PML4       PDPT       PD        PT
       Table      Table     Table     Table
      [512]      [512]     [512]     [512]
         |          |         |         |
         +----->----+---->----+--->-----+--> Frame Number
                                              + Offset
                                              = Physical Address
```

Bits 63–48 are *sign-extended* from bit 47. This means virtual addresses are either in the range 0x0000000000000000–0x00007FFFFFFFFFFF (user space) or 0xFFFF800000000000–0xFFFFFFFFFFFFFFFF (kernel space). Addresses in between are *non-canonical* and cause a general protection fault if used.

The translation requires four memory accesses (one per level) to walk the page table. This is why the TLB (Section 11.3) is critical for performance.

Example 11.11 (x86-64 Translation). Virtual address: 0x00007F4A38201A40.

We need to extract the indices for each level:

- PML4 index (bits 47–39): $(\text{0x7F4A38201A40} \gg 39)\ \&\ \text{0x1FF} = 254$
- PDPT index (bits 38–30): $(\text{0x7F4A38201A40} \gg 30)\ \&\ \text{0x1FF} = 328$
- PD index (bits 29–21): $(\text{0x7F4A38201A40} \gg 21)\ \&\ \text{0x1FF} = 257$
- PT index (bits 20–12): $(\text{0x7F4A38201A40} \gg 12)\ \&\ \text{0x1FF} = 1$
- Offset (bits 11–0): $\text{0x7F4A38201A40}\ \&\ \text{0xFFF} = \text{0xA40}$

The hardware performs: 1. Read PML4[254] from the PML4 table (base address in CR3 register). 2. Read PDPT[328] from the PDPT table pointed to by PML4[254]. 3. Read PD[257] from the PD table pointed to by PDPT[328]. 4. Read PT[1] from the PT table pointed to by PD[257]. 5. Extract frame number from PT[1], append offset 0xA40.

If any entry along the path has its present bit clear, the hardware raises a page fault.

11.2.5 Five-Level Page Tables

Intel's Ice Lake and later processors support *5-level paging* (LA57), adding a PML5 level that extends the virtual address space from 48 to 57 bits (128 PB). The PML5 uses 9 bits (bits 56–48), adding 512 more entries at the top level. This is motivated by applications (databases, in-memory analytics) that need more than 256 TB of virtual address space, and by persistent memory (Intel Optane) that blurs the line between storage and memory.

Definition 11.9 (Five-Level Paging). In five-level paging, the virtual address is extended to 57 bits:

Level	Name	Bits
5	PML5	56–48
4	PML4	47–39
3	PDPT	38–30
2	PD	29–21
1	PT	20–12

The PML5 table has 512 entries, each pointing to a PML4 table. The additional level costs one more memory access per page table walk.

11.2.6 Inverted Page Tables

An alternative to hierarchical page tables is the *inverted page table*, which has one entry per physical frame rather than one entry per virtual page.

Definition 11.10 (Inverted Page Table). An *inverted page table* is an array with one entry per physical frame. Each entry contains the virtual page number and process ID of the page currently occupying that frame. To translate a virtual address, the hardware (or software) searches the table for an entry matching the (process ID, VPN) pair.

The inverted page table has a fixed size proportional to the physical memory, not the virtual address space. For a system with 4 GB of physical memory and 4 KB pages, the inverted page table has 2^{20} entries, regardless of whether the virtual address space is 48 or 57 bits.

The drawback is the search: a naive linear scan is $O(n)$ per memory access. This is solved using a hash table:

$$\text{hash}(\text{PID}, \text{VPN}) \rightarrow \text{index into inverted page table}$$

Collisions are resolved by chaining. The IBM PowerPC and the IA-64 (Itanium) architectures used inverted page tables.

Example 11.12 (Inverted Page Table). Physical memory: 16 frames. Inverted page table:

Frame	PID	VPN	Present	Chain
0	1	5	1	–
1	2	3	1	–
2	1	0	1	–
3	–	–	0	–
4	1	12	1	–
...	...	...	...	...

To translate (PID=1, VPN=5): hash(1, 5) = 0. Frame 0 contains (1, 5). Match. Physical address = frame 0 + offset.

To translate (PID=1, VPN=7): hash(1, 7) = 3. Frame 3 is empty. No match found after following the chain. Page fault.

Theorem 11.3 (Inverted vs Hierarchical Trade-off). The inverted page table uses $O(M/P)$ space (where M is physical memory size and P is page size), independent of virtual address space size. The hierarchical page table uses $O(U/P)$ space (where U is the amount of virtual address space actually used by the process), which can be much smaller or much larger than the inverted table depending on the workload. The inverted table has $O(1)$ expected lookup time (with good hashing) but does not directly support shared pages.

Note

Inverted page tables make shared memory more complex. If two processes share a physical frame, the inverted page table entry can only store one (PID, VPN) pair. Sharing is typically handled by a separate mechanism, such as a supplementary hash chain or a segment table overlay.

11.3 Translation Lookaside Buffer (TLB)

11.3.1 The Performance Problem

A 4-level page table requires four memory accesses just to translate one virtual address, plus a fifth access to actually read or write the target data. This makes every memory operation five times slower — clearly unacceptable.

The solution is caching. The *Translation Lookaside Buffer* (TLB) is a small, fast cache that stores recently used page table entries.

Definition 11.11 (Translation Lookaside Buffer). The *TLB* is a fully associative (or set-associative) hardware cache within the MMU that stores recently used virtual-to-physical page translations. On each memory access, the TLB is consulted first. A *TLB hit* produces the physical frame number without any page table walk. A *TLB miss* triggers a page table walk, and the result is inserted into the TLB.

Typical TLB parameters:

Parameter	L1 dTLB	L1 iTLB	L2 TLB
Entries	64–128	64–128	512–2048
Associativity	4–8 way	4–8 way	8–12 way
Access time	0.5–1 ns	0.5–1 ns	3–7 ns

11.3.2 TLB Operation

```
Virtual Address
      |
      v
  +-------+
  |  TLB  |--HIT--> Frame Number --> Physical Address
  +-------+
      |
     MISS
      |
      v
  +------------+
  | Page Table |--PRESENT--> Frame Number --> TLB Update --> Physical Address
  |   Walk     |
  +------------+
```

```
      |
   NOT PRESENT
      |
      v
  +------------+
  | Page Fault |
  | Handler    |
  +------------+
```

The TLB is typically split into separate instruction TLB (iTLB) and data TLB (dTLB), mirroring the split L1 cache. Some processors have a unified L2 TLB that backs both. On modern Intel processors, the TLB hierarchy looks like:

```
L1 iTLB: 128 entries, 8-way, 4KB/2MB pages
L1 dTLB: 64 entries, 4-way, 4KB pages
         32 entries, 4-way, 2MB/1GB pages
L2 sTLB: 1536 entries, 12-way, 4KB/2MB pages (unified)
```

11.3.2a TLB Reach

Definition 11.11a (TLB Reach). The *TLB reach* is the total amount of virtual address space accessible through the TLB without a miss:

$$\text{TLB reach} = \text{TLB entries} \times \text{page size}$$

For a mixed TLB with entries for different page sizes:

$$\text{TLB reach} = \sum_i n_i \times P_i$$

where n_i is the number of TLB entries for page size P_i.

Example 11.12a (TLB Reach with Mixed Page Sizes). A TLB has 64 entries for 4 KB pages and 32 entries for 2 MB pages.

$$\text{TLB reach} = 64 \times 4 \text{ KB} + 32 \times 2 \text{ MB} = 256 \text{ KB} + 64 \text{ MB} = 64.25 \text{ MB}$$

The 32 huge page entries contribute 99.6% of the TLB reach despite being only 33% of the entries. This illustrates why huge pages are so effective at reducing TLB pressure.

11.3.3 TLB Hit Ratio and EAT

Definition 11.12 (Effective Access Time with TLB). Let h be the TLB hit ratio (probability of a TLB hit), t_{TLB} be the TLB lookup time, t_{mem} be the memory access time, and k be the number of page table levels. The effective access time is:

$$\text{EAT} = h \times (t_{\text{TLB}} + t_{\text{mem}}) + (1 - h) \times (t_{\text{TLB}} + k \times t_{\text{mem}} + t_{\text{mem}})$$

Simplifying (and noting that t_{TLB} is typically overlapped with the cache access in a pipelined processor):

$$\text{EAT} \approx h \times t_{\text{mem}} + (1 - h) \times (k + 1) \times t_{\text{mem}}$$

Example 11.13 (EAT with TLB). 4-level page table ($k = 4$), $t_{\text{mem}} = 100$ ns, TLB hit ratio $h = 0.99$.

$$\text{EAT} = 0.99 \times 100 + 0.01 \times 5 \times 100 = 99 + 5 = 104 \text{ ns}$$

With TLB: 4% slowdown. Without TLB: $5 \times 100 = 500$ ns (400% slowdown).

If the TLB hit ratio drops to 0.90:

$$\text{EAT} = 0.90 \times 100 + 0.10 \times 500 = 90 + 50 = 140 \text{ ns}$$

A 40% slowdown — the TLB hit ratio is the single most important factor in virtual memory performance.

Example 11.14 (TLB with Page Table Cache). In practice, the intermediate levels of the page table walk may hit in the data cache. If the L1 cache has a hit rate of 80% for page table entries and $t_{L1} = 4$ ns:

$$t_{\text{walk}} = k \times (0.8 \times t_{L1} + 0.2 \times t_{\text{mem}}) = 4 \times (0.8 \times 4 + 0.2 \times 100) = 4 \times (3.2 + 20) = 92.8 \text{ ns}$$

Compared to $k \times t_{\text{mem}} = 400$ ns. The data cache reduces the penalty of TLB misses by about 4x.

Theorem 11.4 (TLB Coverage). A TLB with n entries and page size P can cover $n \times P$ bytes of virtual address space without any misses. For a workload with working

set size W, TLB misses are approximately zero if $n \times P \geq W$, and the miss rate increases as $W/(n \times P)$ exceeds 1.

Proof sketch. Each TLB entry caches the translation for one page of size P. If all n entries hold translations for the working set, every access within the working set hits in the TLB. The working set fits if $W \leq n \times P$. When $W > n \times P$, the process cycles through more pages than TLB entries, causing capacity misses. For a uniform random access pattern over W/P pages with $n < W/P$, the miss rate is approximately $1 - nP/W$. □

Example 11.15 (TLB Coverage). A TLB has 1024 entries. Page size: 4 KB.

- TLB coverage: 1024×4 KB $= 4$ MB

A process with a working set of 2 MB fits entirely in the TLB — near-zero TLB misses. A process with a working set of 64 MB uses only $4/64 = 6.25\%$ of its working set through the TLB at any time, leading to frequent TLB misses.

With 2 MB huge pages: TLB coverage = 1024×2 MB $= 2$ GB. The 64 MB working set now fits easily.

11.3.4 TLB Miss Handling

Two approaches to handling TLB misses:

Hardware-managed TLB (x86, ARM): The CPU hardware contains a *page table walker* (PTW) that automatically traverses the page table on a TLB miss, loads the PTE, and inserts it into the TLB. The OS is not involved in TLB misses (only in page faults). The page table format is fixed by the hardware.

Software-managed TLB (MIPS, SPARC, PA-RISC): A TLB miss raises a special exception (TLB miss exception, not a page fault), and the OS's TLB miss handler software walks the page table and loads the TLB entry using special instructions (e.g., MIPS `tlbwr`). This gives the OS complete flexibility over the page table format but adds overhead (typically 10–50 cycles for the exception handling). The handler is performance-critical and is usually written in hand-optimised assembly.

Definition 11.13 (Page Table Walker). A *page table walker* (PTW) is a hardware state machine that traverses a multi-level page table to resolve a TLB miss. It reads page table entries from memory (potentially hitting in the data cache or dedicated page table caches), follows the chain of pointers from the top level to the leaf, and loads the resulting translation into the TLB.

11.3.5 Address Space Identifiers (ASID)

Definition 11.14 (Address Space Identifier). An *ASID* is a tag stored in each TLB entry that identifies which process's address space the entry belongs to. When the TLB is searched, both the virtual page number and the ASID must match. This allows entries from multiple processes to coexist in the TLB, eliminating the need to flush the TLB on every context switch.

Without ASIDs, a context switch requires flushing the entire TLB (invalidating all entries), because virtual page 5 in Process A maps to a different physical frame than virtual page 5 in Process B. With ASIDs, both mappings can coexist:

VPN	ASID	Frame	Valid
5	1	42	1
5	2	87	1
12	1	3	1

When Process 1 accesses VPN 5, the TLB matches (VPN=5, ASID=1) and returns frame 42. When Process 2 accesses VPN 5, the TLB matches (VPN=5, ASID=2) and returns frame 87.

x86-64 supports ASIDs through *Process Context Identifiers* (PCIDs), a 12-bit field that tags TLB entries. With 12 bits, up to 4096 distinct address spaces can coexist in the TLB. Linux enables PCIDs by default on supported hardware, significantly reducing context switch overhead.

Example 11.16 (PCID Impact on Context Switch). Without PCIDs: every context switch flushes the TLB. With 512 TLB entries and a 99% hit rate, the first ~512 memory accesses after a switch are TLB misses (cold cache). If context switches happen every 4 ms, and the miss penalty is 20 ns per miss, the overhead is $512 \times 20 = 10,240$ ns ≈ 10 μs per switch.

With PCIDs: TLB entries from the previous context remain valid. If the process was recently running (its entries are still in the TLB), most accesses hit immediately. The context switch overhead is reduced to the CR3 load latency (~50 ns) plus any conflicts for the same TLB entry.

Programmer's Perspective: Measuring TLB Performance.

On Linux, you can measure TLB behaviour using `perf stat`:

```
$ perf stat -e dTLB-load-misses,dTLB-loads,dTLB-store-misses,dTLB-stores \
            -e iTLB-load-misses,iTLB-loads \
            ./my_program
```

Sample output:

```
     2,847,321  dTLB-load-misses    #  0.03% of all dTLB loads
 8,421,567,890  dTLB-loads
       123,456  dTLB-store-misses
 2,105,391,972  dTLB-stores
        45,678  iTLB-load-misses    #  0.00% of all iTLB loads
 4,210,783,945  iTLB-loads
```

A data TLB miss rate of 0.03% is excellent. If the rate exceeds 1%, investigate:

1. Check if the working set exceeds TLB coverage (entries $\times$ page size).
2. Consider using huge pages to increase coverage (Section 11.7).
3. Restructure data for better spatial locality: arrays of structures vs structures of arrays, loop tiling, etc.
4. On NUMA systems, ensure memory is allocated on the local node (remote accesses add to the effective miss penalty).

In Go, the runtime's memory allocator is designed to minimise TLB pressure by allocating small objects from per-P caches backed by spans (contiguous runs of pages). Since objects of the same size class are packed into the same span, iterating over them exhibits good spatial locality. The `runtime.MemProfileRate` variable controls allocation profiling, which can help identify hot allocation paths.

11.4 Page Table Entry Format

Each page table entry contains the physical frame number and several control bits that govern access permissions and assist the OS in page management.

11.4.1 x86-64 PTE Format

Definition 11.15 (x86-64 Page Table Entry). An x86-64 PTE is 64 bits wide with the following fields:

Bits	Name	Description
0	P (Present)	1 if the page is in physical memory

Bits	Name	Description
1	R/W	0 = read-only, 1 = read/write
2	U/S	0 = supervisor only, 1 = user accessible
3	PWT	Page-level write-through
4	PCD	Page-level cache disable
5	A (Accessed)	Set by hardware when the page is read or written
6	D (Dirty)	Set by hardware when the page is written
7	PS (Page Size)	1 = large page (2 MB at PD level, 1 GB at PDPT level)
8–11	(Available)	Available for OS use
12–51	Frame Number	Physical frame number (40 bits $\rightarrow$ 1 TB addressable)
52–58	(Available)	Available for OS use
59–62	MPK	Memory protection key (4 bits, 16 domains)
63	NX (No Execute)	1 = page cannot be executed

The 40-bit frame number allows addressing up to $2^{40} \times 4$ KB $= 4$ PB of physical memory. The 11 available bits (4 + 7) are used by the OS for purposes such as swap location encoding, page state tracking, and reference counting.

11.4.2 The Present Bit

The present bit is the gateway to demand paging. When P = 0, all other bits are available for the OS to use as it sees fit. Linux uses the remaining bits to store the location of the page on disk (swap device number and offset), enabling the page fault handler to locate the page quickly.

11.4.3 The Dirty Bit

Definition 11.16 (Dirty Bit). The *dirty bit* (D) is set by the hardware whenever the CPU writes to a page. A page with D = 1 has been modified since it was loaded into memory and must be written back to disk before its frame can be reclaimed. A page

with D = 0 (clean) can be discarded without writing, since the copy on disk is still valid.

The dirty bit is critical for page replacement efficiency: evicting a clean page is free (no I/O), while evicting a dirty page requires a write to the backing store. This asymmetry influences both the replacement algorithm (prefer evicting clean pages) and the kernel's background writeback policy (periodically writing dirty pages to disk to reduce the number of dirty pages that accumulate).

11.4.4 The Accessed Bit

The accessed bit (A) is set by the hardware whenever the CPU reads or writes the page. The operating system periodically clears the accessed bits and uses them to approximate LRU behaviour for page replacement (the "clock algorithm," Chapter 12). Pages with A = 0 after the OS clears the bit have not been accessed in the recent past and are candidates for eviction.

11.4.5 The NX (No-Execute) Bit

Definition 11.17 (NX Bit). The *NX bit* (No eXecute), also called *XD* (eXecute Disable) on Intel or *EVP* on AMD, marks a page as non-executable. Any attempt to fetch an instruction from an NX-marked page raises a page fault. This is a critical security feature that prevents code injection attacks (e.g., buffer overflow exploits that place shellcode on the stack or heap).

Example 11.17 (NX Protection). A buffer overflow attack overwrites a return address on the stack to point to malicious code injected into the same stack buffer. Without NX: the CPU executes the injected code. With NX: the stack pages are marked NX, so the CPU raises a page fault when it tries to execute code from the stack. The attack fails.

This defence is not absolute: Return-Oriented Programming (ROP) attacks bypass NX by chaining together small sequences of existing code (gadgets) already present in executable pages. But NX eliminates the simplest class of code injection attacks.

11.5 Shared Pages

11.5.1 Shared Libraries

Multiple processes running the same program (or using the same library) can share the read-only code pages. Each process's page table maps the relevant virtual pages

to the same physical frames.

Definition 11.18 (Shared Page). A *shared page* is a physical frame that appears in the page tables of two or more processes. The processes may map the frame at different virtual addresses, but they all access the same physical memory.

Example 11.18 (Shared libc). Three processes (A, B, C) all use the C library (libc), which occupies 500 pages. Without sharing: $3 \times 500 = 1500$ frames. With sharing: 500 frames (one copy), mapped into all three address spaces. Savings: 1000 frames.

11.5.2 Copy-on-Write (CoW)

Definition 11.19 (Copy-on-Write). *Copy-on-write* (CoW) is an optimisation where two processes initially share the same physical pages (marked read-only). When either process writes to a shared page, the hardware triggers a protection fault. The OS then copies the page to a new frame, updates the page table of the writing process to point to the copy, and marks both the original and the copy as writable (if appropriate).

CoW is the foundation of efficient `fork()` in Unix. When a process calls `fork()`, the child inherits the parent's entire address space. Without CoW, this requires copying all of the parent's memory — potentially hundreds of megabytes. With CoW, `fork()` merely duplicates the page table (each entry pointing to the same physical frame, marked read-only). Pages are copied only when written, and pages that are never written (such as the text segment) are never copied.

Example 11.19 (Fork with CoW). A process with 1000 pages calls `fork()`. Immediately after fork:

- Parent and child page tables both point to the same 1000 physical frames.
- All 1000 frames are marked read-only in both page tables.
- Memory cost of fork: one page table (a few KB), not 1000 pages.

The child then writes to 3 pages. Each write triggers a CoW fault: 1. OS allocates a new frame. 2. Copies the original page to the new frame. 3. Updates the child's PTE to point to the new frame (read-write). 4. If the original frame now has ref_count = 1, marks it read-write in the parent's PTE.

Total pages allocated: $1000 + 3 = 1003$ (instead of $1000 + 1000 = 2000$).

Programmer's Perspective: mmap() in C and Go's Virtual Memory.

The `mmap()` system call is the primary interface for manipulating virtual memory in user space:

```c
#include <sys/mman.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main(void) {
    /* Anonymous mapping: allocate 4 pages of private memory */
    size_t len = 4 * 4096;
    void *anon = mmap(NULL, len,
                      PROT_READ | PROT_WRITE,
                      MAP_PRIVATE | MAP_ANONYMOUS,
                      -1, 0);
    if (anon == MAP_FAILED) { perror("mmap anon"); return 1; }

    memset(anon, 0x42, len);
    printf("Anonymous mapping at %p, first byte: 0x%02x\n",
           anon, *(unsigned char *)anon);

    /* File-backed mapping: map a file read-only */
    int fd = open("/etc/hostname", O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }
    void *file_map = mmap(NULL, 4096, PROT_READ,
                          MAP_PRIVATE, fd, 0);
    close(fd);
    if (file_map == MAP_FAILED) { perror("mmap file"); return 1; }

    printf("File mapping at %p: %.40s\n",
           file_map, (char *)file_map);

    munmap(anon, len);
    munmap(file_map, 4096);
    return 0;
}
```

In Go, the runtime uses `mmap` internally (via `runtime.sysAlloc`) to obtain memory from the OS. The Go heap is a large `mmap`'d region divided into 8 KB pages managed by the runtime's page allocator. You can inspect a Go process's virtual memory through `/proc/PID/maps`:

```go
package main
```

```go
import (
    "fmt"
    "os"
    "runtime"
)

func main() {
    var m runtime.MemStats
    runtime.ReadMemStats(&m)
    fmt.Printf("HeapAlloc:  %d MB\n", m.HeapAlloc/1024/1024)
    fmt.Printf("HeapSys:    %d MB\n", m.HeapSys/1024/1024)
    fmt.Printf("StackSys:   %d MB\n", m.StackSys/1024/1024)

    pid := os.Getpid()
    fmt.Printf("\n/proc/%d/maps (first 20 lines):\n", pid)
    data, _ := os.ReadFile(fmt.Sprintf("/proc/%d/maps", pid))
    lines := 0
    for i, b := range data {
        if b == '\n' {
            lines++
            if lines >= 20 {
                fmt.Print(string(data[:i+1]))
                break
            }
        }
    }
}
```

On Linux, you can also inspect the page-level mapping through `/proc/PID/pagemap`, a binary file where each 64-bit entry describes the physical frame (or swap location) of the corresponding virtual page. Tools like `pagemap` and `page-types` (from the kernel source tree) parse this file to show which pages are resident, swapped, or shared. The `smaps` file (`/proc/PID/smaps`) provides per-region statistics including resident size, shared vs private pages, and swap usage.

```c
/* Read /proc/PID/pagemap to check if a page is resident */
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <fcntl.h>
```

```c
#include <unistd.h>

#define PAGE_SIZE 4096

int main(void) {
    void *buf = aligned_alloc(PAGE_SIZE, PAGE_SIZE);
    if (!buf) { perror("aligned_alloc"); return 1; }

    *(volatile char *)buf = 'A';  /* Touch to make resident */

    char path[64];
    snprintf(path, sizeof(path), "/proc/%d/pagemap", getpid());
    int fd = open(path, O_RDONLY);
    if (fd < 0) { perror("open pagemap"); return 1; }

    uint64_t vaddr = (uint64_t)buf;
    uint64_t vpn = vaddr / PAGE_SIZE;
    off_t offset = vpn * sizeof(uint64_t);

    uint64_t entry;
    if (pread(fd, &entry, sizeof(entry), offset) != sizeof(entry)) {
        perror("pread"); return 1;
    }

    printf("Virtual address: %p\n", buf);
    printf("Present:  %lu\n", (entry >> 63) & 1);
    printf("Swapped:  %lu\n", (entry >> 62) & 1);
    if ((entry >> 63) & 1)
        printf("PFN:      %lu (phys: 0x%lx)\n",
               entry & ((1ULL << 55) - 1),
               (entry & ((1ULL << 55) - 1)) * PAGE_SIZE);

    close(fd);
    free(buf);
    return 0;
}
```

11.6 Page Size Trade-offs

The choice of page size has profound implications for system performance.

11.6.1 Arguments for Small Pages

- **Less internal fragmentation:** On average, each process wastes half a page in internal fragmentation. Smaller pages mean less waste per allocation.
- **Finer granularity:** More precise control over which memory is in RAM. A process's working set can be represented more accurately.
- **Faster page faults:** Less data to transfer per page fault.
- **Better sharing granularity:** Shared libraries can be shared at a finer granularity.

11.6.2 Arguments for Large Pages

- **Smaller page tables:** Fewer entries mean less memory consumed by page tables and less time spent on page table walks.
- **Better TLB coverage:** Each TLB entry covers more memory, reducing TLB misses.
- **More efficient disk I/O:** Disk transfers are more efficient in larger chunks (amortising the seek/rotational latency).
- **Fewer page faults:** Each fault loads more data, so fewer total faults for sequential access.

11.6.3 Quantitative Analysis

Theorem 11.5 (Optimal Page Size). Let s be the average process size, e be the page table entry size, and P be the page size. The total overhead per process from the page table and internal fragmentation is:

$$\text{Overhead}(P) = \frac{s \cdot e}{P} + \frac{P}{2}$$

The first term is the page table size (number of pages $\times$ entry size). The second term is the expected internal fragmentation (half a page on average). Minimising with respect to P:

$$\frac{d}{dP}\left(\frac{se}{P} + \frac{P}{2}\right) = -\frac{se}{P^2} + \frac{1}{2} = 0$$

$$P^* = \sqrt{2se}$$

Proof. The second derivative is $2se/P^3 > 0$, confirming a minimum. □

Example 11.20 (Optimal Page Size). Average process size $s = 1$ MB $= 2^{20}$ bytes. PTE size $e = 8$ bytes.

$$P^* = \sqrt{2 \times 2^{20} \times 8} = \sqrt{2^{24}} = 2^{12} = 4096 \text{ bytes} = 4 \text{ KB}$$

This analysis provides one justification for the ubiquitous 4 KB page size. Note that this is a rough estimate — real workloads have non-uniform process sizes and access patterns that complicate the analysis.

Example 11.21 (Optimal Page Size for Large Processes). Average process size $s = 1$ GB $= 2^{30}$ bytes. PTE size $e = 8$ bytes.

$$P^* = \sqrt{2 \times 2^{30} \times 8} = \sqrt{2^{34}} = 2^{17} = 131{,}072 \text{ bytes} = 128 \text{ KB}$$

For large processes, the optimal page size is much larger than 4 KB. This motivates huge pages (Section 11.7).

11.6.4 Common Page Sizes in Practice

Architecture	Standard Page	Large Page	Huge Page
x86-64	4 KB	2 MB	1 GB
ARM (AArch64, 4K granule)	4 KB	2 MB	1 GB
ARM (AArch64, 16K granule)	16 KB	32 MB	–
ARM (AArch64, 64K granule)	64 KB	512 MB	–
POWER	4 KB	64 KB	16 MB
SPARC	8 KB	64 KB	4 MB
RISC-V (Sv39)	4 KB	2 MB	1 GB

11.7 Huge Pages and TLB Pressure

Definition 11.20 (Huge Page). A *huge page* is a page larger than the architecture's standard page size (e.g., 2 MB or 1 GB on x86-64 instead of 4 KB). Huge pages reduce TLB pressure by covering more virtual address space per TLB entry.

The benefit is dramatic. A TLB with 1024 entries covers:

- With 4 KB pages: 1024×4 KB $= 4$ MB
- With 2 MB pages: 1024×2 MB $= 2$ GB
- With 1 GB pages: 1024×1 GB $= 1$ TB

Example 11.22 (TLB Miss Rate Reduction). A database server has a 16 GB working set. TLB: 1024 entries.

With 4 KB pages: working set requires 16 GB$/4$ KB $= 4{,}194{,}304$ pages. TLB can hold 1024. Coverage: 4 MB$/16$ GB $= 0.024\%$. Frequent TLB misses.

With 2 MB pages: working set requires 16 GB$/2$ MB $= 8{,}192$ pages. TLB can hold 1024. Coverage: 2 GB$/16$ GB $= 12.5\%$. Significant reduction in misses.

With 1 GB pages: working set requires 16 pages. TLB can hold all 16. Coverage: 100%. Near-zero TLB misses.

The cost of huge pages is increased internal fragmentation. A 2 MB page wastes, on average, 1 MB per allocation. This is acceptable only for large, long-lived allocations.

Huge pages also reduce the number of page table levels traversed on a TLB miss: a 2 MB page eliminates the PT level (translation stops at the PD level), and a 1 GB page eliminates both PT and PD levels.

Note

Linux supports huge pages in two forms: (1) *explicit huge pages* via `hugetlbfs`, configured at boot time with a fixed pool of huge pages, and (2) *Transparent Huge Pages* (THP), where the kernel automatically promotes contiguous 4 KB page regions to 2 MB pages when possible. THP is enabled by default on most Linux distributions but can cause latency spikes during promotion/demotion. Database vendors (PostgreSQL, Redis) often recommend disabling THP due to these latency concerns. We cover THP in detail in Chapter 13.

11.8 Virtual Memory in Kernel Space

On x86-64 Linux, the virtual address space is split:

- **User space:** 0x0000000000000000 – 0x00007FFFFFFFFFFF (128 TB)
- **Non-canonical gap:** 0x0000800000000000 – 0xFFFF7FFFFFFFFFFF
- **Kernel space:** 0xFFFF800000000000 – 0xFFFFFFFFFFFFFFFF (128 TB)

The kernel maps all of physical memory starting at 0xFFFF888000000000 (the "direct map" or "physmap"). This allows the kernel to access any physical address by simply adding a fixed offset:

$$\text{kernel virtual address} = \text{physical address} + 0\text{xFFFF888000000000}$$

Definition 11.21 (Direct Map). The *direct map* (or *physmap*) is a region of kernel virtual address space that maps all of physical memory contiguously, with a fixed offset. This allows the kernel to access any physical frame by computing a simple addition, without consulting page tables (beyond the initial translation, which is always cached in the TLB).

The kernel's page tables are shared across all processes: every process's PML4 table has the same entries for the upper half of the address space (indices 256–511). When any process makes a system call and enters kernel mode, the kernel can access its own data structures and any process's physical memory through the direct map.

11.9 Summary

Virtual memory transforms the rigid, contiguous, limited physical memory into a flexible, vast, per-process logical address space:

- **Demand paging** loads pages only when needed, keeping the page fault rate low to maintain acceptable performance. The EAT formula quantifies the relationship between page fault rate and performance.
- **Page tables** (single-level, multi-level, inverted) map virtual page numbers to physical frame numbers. Modern systems use 4-level or 5-level hierarchical tables to handle 48–57-bit address spaces efficiently. Multi-level tables exploit sparsity to reduce memory consumption dramatically.
- **The TLB** caches recent translations, turning the multi-step page table walk into a single fast lookup for the common case. TLB hit ratios above 99% are

typical for well-behaved workloads. ASIDs (PCIDs) eliminate TLB flushes on context switches.

- **Page table entries** carry present, dirty, accessed, protection, and NX bits that the OS uses for demand paging, replacement decisions, and security enforcement.
- **Shared pages** and **copy-on-write** enable efficient memory sharing between processes and fast `fork()` implementation.
- **Page size trade-offs** balance internal fragmentation against page table size and TLB pressure. Huge pages (2 MB, 1 GB) dramatically reduce TLB misses for large working sets.

Chapter 12 addresses the question that demand paging inevitably raises: when physical memory is full and a page must be loaded, which existing page should be evicted? The answer — page replacement algorithms — determines whether the system hums along or collapses into thrashing.

Exercise 11.1. A system has a 48-bit virtual address space, a 40-bit physical address space, and 4 KB pages. (a) How many bits are in the VPN and the page offset? (b) How many entries does a single-level page table require? (c) If each PTE is 8 bytes, what is the total size of the page table? (d) Why is this infeasible, and what is the solution?

Exercise 11.2. Consider a two-level page table for a 32-bit address space with 4 KB pages. The Page Directory has 1024 entries and each Page Table has 1024 entries. A process maps the following virtual address ranges: 0x00000000–0x00400000 (code, 4 MB), 0x10000000–0x10100000 (data, 1 MB), 0xBFE00000–0xC0000000 (stack, 2 MB). (a) How many Page Directory entries are valid? (b) How many Page Tables are allocated? (c) What is the total memory consumed by page tables? (d) Compare this to a single-level page table.

Exercise 11.3. A TLB has 256 entries and the page size is 4 KB. The TLB hit ratio is 0.98 and the memory access time is 80 ns. The system uses a 4-level page table. (a) Calculate the EAT. (b) What TLB hit ratio is needed for the EAT to be within 5% of the memory access time? (c) If the page size is increased to 2 MB (and the TLB still has 256 entries), what is the new TLB coverage? Discuss qualitatively how this affects the hit ratio for a process with a 32 MB working set.

Exercise 11.4. An x86-64 system uses a 4-level page table. A virtual address is 0x00007F0012345678. (a) Extract the PML4 index, PDPT index, PD index, PT index, and page offset. Show your work in binary or hexadecimal. (b) How many memory

accesses are needed to translate this address, assuming a TLB miss? (c) If the TLB hit ratio is 0.995, what is the average number of memory accesses per address translation (including the final data access)?

Exercise 11.5. Prove that for a k-level page table with branching factor b (entries per table), the maximum number of table pages is:

$$\sum_{i=0}^{k-1} b^i = \frac{b^k - 1}{b - 1}$$

For $k = 4$ and $b = 512$ (x86-64), compute this value. Explain why this worst-case is never reached in practice.

Exercise 11.6. A process calls `fork()` in a system with copy-on-write. The parent has 8000 pages mapped: 2000 pages of code (read-only), 3000 pages of heap (read-write), 1000 pages of stack (read-write), and 2000 pages of shared library (read-only, shared). After `fork()`, the child modifies 500 heap pages and 200 stack pages. (a) How many physical page copies are made by CoW faults? (b) What is the total number of distinct physical frames used by both parent and child after the child's modifications? (c) If CoW were not used, how many frames would be needed?

Exercise 11.7. Derive the optimal page size formula $P^* = \sqrt{2se}$ from the overhead function $f(P) = se/P + P/2$. Then calculate P^* for: (a) $s = 256$ KB, $e = 4$ bytes, (b) $s = 4$ MB, $e = 8$ bytes, (c) $s = 1$ GB, $e = 8$ bytes. Comment on why real systems do not use the optimal page size for case (c) and what mechanism addresses the TLB issue instead.

Chapter 12: Page Replacement and Thrashing

Demand paging creates an illusion of abundant memory, but the illusion has a hard limit: physical frames are finite. Sooner or later, a page fault occurs when every frame is occupied. The operating system must then choose a *victim* — a page currently in memory to evict, making room for the incoming page. The choice of victim is the *page replacement* problem, and it has a profound effect on system performance. A bad choice evicts a page that will be needed again immediately, causing another page fault, then another, in a cascading failure known as *thrashing*. A good choice evicts a page that will not be needed for a long time, minimising future faults.

This chapter develops the classical page replacement algorithms, analyses their properties with mathematical rigour, proves Belady's anomaly and the immunity of stack algorithms, and then addresses the systemic failure mode of thrashing and the mechanisms that prevent it.

12.1 The Page Replacement Problem

12.1.1 Formulation

Definition 12.1 (Page Replacement Problem). Given a sequence of page references $\omega = r_1, r_2, \dots, r_n$ (the *reference string*) and m physical frames, determine which page to evict upon each page fault so as to minimise the total number of page faults.

The reference string is an abstraction of the program's memory access pattern: each r_i is the page number accessed at time i. Consecutive references to the same page are often collapsed, since they do not cause additional faults.

Example 12.1 (Reference String). A program accesses the following addresses (with 4 KB pages): 0x1004, 0x1200, 0x5000, 0x5100, 0x1004, 0x3000, 0x5200, 0x3004.

Page numbers (dividing by 4096): 1, 1, 5, 5, 1, 3, 5, 3.

Collapsing consecutive duplicates: $\omega = 1, 5, 1, 3, 5, 3$.

12.1.2 Evaluation Metric

For a given replacement algorithm A, reference string ω, and m frames, let $f_A(\omega, m)$ denote the number of page faults. The goal is to minimise $f_A(\omega, m)$.

Theorem 12.1 (Monotonicity — General Case Fails). For an arbitrary page replacement algorithm, it is *not* necessarily true that $f_A(\omega, m+1) \leq f_A(\omega, m)$. That is, adding a frame can increase the number of page faults.

This counter-intuitive phenomenon is called *Belady's anomaly* and is discussed in Section 12.4.

12.2 Belady's Optimal Algorithm (OPT)

12.2.1 Definition

Definition 12.2 (OPT / Belady's Optimal Algorithm). Upon a page fault, OPT evicts the page that will not be used for the longest time in the future. If a page will never be used again, it is the ideal victim. Among pages that will be used again, the one whose next reference is furthest away is evicted.

Example 12.2 (OPT Execution). Reference string: $\omega = 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7,$ Frames: $m = 3$.

Step	Ref	Frame 0	Frame 1	Frame 2	Fault?	Victim	Reason
1	7	7	–	–	Yes	–	Cold start
2	0	7	0	–	Yes	–	Cold start
3	1	7	0	1	Yes	–	Cold start
4	2	2	0	1	Yes	7	7 next used at step 18 (furthest)
5	0	2	0	1	No	–	–
6	3	2	0	3	Yes	1	1 next used at step 14 (furthest)
7	0	2	0	3	No	–	–
8	4	2	4	3	Yes	0	0 next used at step 11 (furthest)
9	2	2	4	3	No	–	–
10	3	2	4	3	No	–	–

Step	Ref	Frame 0	Frame 1	Frame 2	Fault?	Victim	Reason
11	0	2	0	3	Yes	4	4 never used again
12	3	2	0	3	No	–	–
13	2	2	0	3	No	–	–
14	1	2	0	1	Yes	3	3 not used again
15	2	2	0	1	No	–	–
16	0	2	0	1	No	–	–
17	1	2	0	1	No	–	–
18	7	7	0	1	Yes	2	2 not used again
19	0	7	0	1	No	–	–
20	1	7	0	1	No	–	–

Total page faults: 9.

12.2.2 Proof of Optimality

Theorem 12.2 (Optimality of OPT). For any reference string ω and frame count m, OPT achieves the minimum possible number of page faults. That is, for any replacement algorithm A:

$$f_{\text{OPT}}(\omega, m) \leq f_A(\omega, m)$$

Proof. We prove this by an exchange argument. Let A be any algorithm. We show that OPT's decisions can replace A's decisions without increasing the fault count.

Consider the first time A and OPT make different replacement decisions. At time t, both have the same set of pages in memory (since they have made identical decisions up to this point). Both experience a page fault on page p. OPT evicts page q_{OPT} (the one with the furthest next use); A evicts page $q_A \neq q_{\text{OPT}}$.

After the replacement: - OPT has pages: $(S \setminus \{q_{\text{OPT}}\}) \cup \{p\}$ - A has pages: $(S \setminus \{q_A\}) \cup \{p\}$

Since q_{OPT}'s next use is at time $t' \geq t'_A$ (where t'_A is q_A's next use), every reference between t and t' that A handles without a fault can also be handled by OPT without

a fault (since OPT has all the pages A has, except it has q_A instead of q_{OPT}, and q_A is not referenced before q_{OPT}).

When q_A is eventually referenced (at time $t'_A \leq t'$), A does not fault (it still has q_A), but OPT might. However, at time t'_A, OPT can evict any page, and by choosing optimally at each subsequent fault, OPT accumulates at most as many faults as A from time t onward.

A formal inductive argument on the number of faults shows that $f_{\text{OPT}} \leq f_A$ for the entire string. □

12.2.3 Why OPT is Impractical

OPT requires knowledge of the entire future reference string, which is unavailable in an online system. Its value is as a benchmark: we can compare any practical algorithm's fault rate against OPT to measure how close to optimal the algorithm is. When evaluating a new replacement algorithm A, we compute the *competitiveness ratio*:

$$\rho_A(\omega, m) = \frac{f_A(\omega, m)}{f_{\text{OPT}}(\omega, m)}$$

For the example above, FIFO achieves $\rho_{\text{FIFO}} = 15/9 = 1.67$, meaning FIFO causes 67% more faults than optimal. LRU achieves $\rho_{\text{LRU}} = 12/9 = 1.33$, or 33% more faults.

12.2.4 OPT as an Offline Algorithm

Definition 12.2a (Online vs Offline Algorithm). An *online algorithm* makes decisions based only on past and present information (the reference string up to the current time). An *offline algorithm* has access to the entire input (the complete reference string) before making any decision. OPT is an offline algorithm; all practical replacement policies are online.

The study of online vs offline algorithms is a rich area of theoretical computer science. The *competitive ratio* of an online algorithm A is:

$$c_A = \sup_{\omega} \frac{f_A(\omega, m)}{f_{\text{OPT}}(\omega, m)}$$

Theorem 12.2a (Competitive Ratio of LRU and FIFO). For a system with m frames:

- LRU has competitive ratio m: $f_{\text{LRU}}(\omega, m) \leq m \cdot f_{\text{OPT}}(\omega, m)$.
- FIFO has competitive ratio m: $f_{\text{FIFO}}(\omega, m) \leq m \cdot f_{\text{OPT}}(\omega, m)$.
- No deterministic online algorithm can achieve a competitive ratio better than m.

Proof sketch (lower bound). Consider an adversary that always requests the page not currently in the online algorithm's cache. With m frames and $m + 1$ distinct pages, the online algorithm faults on every access. OPT, knowing the future, can evict the page that will be requested furthest in the future, faulting much less frequently. The ratio is m. □

This bound seems pessimistic, but it is tight in the worst case. In practice, locality of reference ensures that real workloads behave much better than the adversarial worst case.

12.3 FIFO (First-In, First-Out)

Definition 12.3 (FIFO Replacement). *FIFO* evicts the page that has been in memory the longest — the page that was loaded first. Pages are maintained in a queue; on a page fault, the page at the head of the queue is evicted, and the new page is added to the tail.

Example 12.3 (FIFO Execution). Reference string: $\omega = 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7,$
Frames: $m = 3$.

Step	Ref	Frame 0	Frame 1	Frame 2	Fault?	Evicted
1	7	7	–	–	Yes	–
2	0	7	0	–	Yes	–
3	1	7	0	1	Yes	–
4	2	2	0	1	Yes	7
5	0	2	0	1	No	–
6	3	2	3	1	Yes	0
7	0	2	3	0	Yes	1
8	4	4	3	0	Yes	2
9	2	4	2	0	Yes	3

Step	Ref	Frame 0	Frame 1	Frame 2	Fault?	Evicted
10	3	4	2	3	Yes	0
11	0	0	2	3	Yes	4
12	3	0	2	3	No	–
13	2	0	2	3	No	–
14	1	0	1	3	Yes	2
15	2	0	1	2	Yes	3
16	0	0	1	2	No	–
17	1	0	1	2	No	–
18	7	7	1	2	Yes	0
19	0	7	0	2	Yes	1
20	1	7	0	1	Yes	2

Total page faults: 15. Compare to OPT's 9.

FIFO is simple but performs poorly because it ignores access patterns. A page loaded long ago but still frequently accessed will be evicted, while a page loaded recently but never accessed again will be kept.

12.3.1 Analysis of FIFO

Theorem 12.2b (FIFO Performance Bound). For any reference string ω of length n with k distinct pages and m frames ($m < k$):

$$f_{\text{FIFO}}(\omega, m) \leq n$$

(trivially), and

$$f_{\text{FIFO}}(\omega, m) \geq k$$

(at least one compulsory fault per distinct page). The tight lower bound is k when the reference string is a permutation of the k pages and $m \geq k$.

FIFO's weakness is that it makes no distinction between "hot" and "cold" pages. Consider a process that loops over pages 1, 2, 3, 4, 5 with 4 frames. FIFO evicts the

oldest page, which is exactly the page about to be re-referenced in the next iteration. This produces a fault on every fifth access.

Example 12.3a (FIFO Worst Case). Reference string: $\omega = 1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5$ with $m = 4$ frames.

Since there are 5 distinct pages and only 4 frames, every time the process cycles through all 5 pages, it encounters a fault on the one that was evicted. FIFO evicts the oldest:

Step	Ref	Queue	Fault?
1	1	{1}	Yes
2	2	{1,2}	Yes
3	3	{1,2,3}	Yes
4	4	{1,2,3,4}	Yes
5	5	{2,3,4,5} evict 1	Yes
6	1	{3,4,5,1} evict 2	Yes
7	2	{4,5,1,2} evict 3	Yes
8	3	{5,1,2,3} evict 4	Yes
9	4	{1,2,3,4} evict 5	Yes
10	5	{2,3,4,5} evict 1	Yes
...	...	...	...

Every single access is a fault. Fault rate: 100%. This is the worst possible behaviour — identical to having no caching at all.

With LRU and the same reference string, the behaviour is identical (LRU also has 100% faults for a cyclic pattern larger than the cache). With OPT, faults = 5 (compulsory only, then every access hits because OPT evicts the page furthest from re-use, which is the page just used — effectively keeping the next 4 in the cycle).

12.4 Belady's Anomaly

12.4.1 The Counter-Example

Definition 12.4 (Belady's Anomaly). *Belady's anomaly* is the phenomenon where increasing the number of available frames increases the number of page faults for a given reference string under a given replacement algorithm.

Example 12.4 (FIFO Anomaly). Reference string: $\omega = 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5$.

With 3 frames:

Step	Ref	F0	F1	F2	Fault?
1	1	1	–	–	Yes
2	2	1	2	–	Yes
3	3	1	2	3	Yes
4	4	4	2	3	Yes
5	1	4	1	3	Yes
6	2	4	1	2	Yes
7	5	5	1	2	Yes
8	1	5	1	2	No
9	2	5	1	2	No
10	3	3	1	2	Yes
11	4	3	4	2	Yes
12	5	3	4	5	Yes – wait, let me redo this

Let me trace this carefully.

Step	Ref	Queue (oldest first)	Fault?
1	1	{1}	Yes
2	2	{1, 2}	Yes
3	3	{1, 2, 3}	Yes
4	4	{4, 2, 3} evict 1	Yes

Step	Ref	Queue (oldest first)	Fault?
5	1	{4, 1, 3} evict 2	Yes
6	2	{4, 1, 2} evict 3	Yes
7	5	{5, 1, 2} evict 4	Yes
8	1	{5, 1, 2}	No
9	2	{5, 1, 2}	No
10	3	{5, 3, 2} evict 1	Yes
11	4	{5, 3, 4} evict 2	Yes
12	5	{5, 3, 4}	No

Faults with 3 frames: **9**.

With 4 frames:

Step	Ref	Queue (oldest first)	Fault?
1	1	{1}	Yes
2	2	{1, 2}	Yes
3	3	{1, 2, 3}	Yes
4	4	{1, 2, 3, 4}	Yes
5	1	{1, 2, 3, 4}	No
6	2	{1, 2, 3, 4}	No
7	5	{2, 3, 4, 5} evict 1	Yes
8	1	{3, 4, 5, 1} evict 2	Yes
9	2	{4, 5, 1, 2} evict 3	Yes
10	3	{5, 1, 2, 3} evict 4	Yes
11	4	{1, 2, 3, 4} evict 5	Yes
12	5	{2, 3, 4, 5} evict 1	Yes

Faults with 4 frames: **10**.

More frames (4) produced more faults (10) than fewer frames (3, which had 9). This is Belady's anomaly.

12.4.2 Stack Algorithms and Immunity

Definition 12.5 (Stack Algorithm). A page replacement algorithm is a *stack algorithm* if, for every reference string ω and every time t, the set of pages in memory with m frames is a subset of the set of pages in memory with $m+1$ frames:

$$S_t(m) \subseteq S_t(m+1)$$

This property is called the *inclusion property*.

Theorem 12.3 (Stack Algorithms are Anomaly-Free). A stack algorithm cannot exhibit Belady's anomaly. That is, for any stack algorithm A and any reference string ω:

$$f_A(\omega, m+1) \leq f_A(\omega, m)$$

Proof. At any time t, a page fault occurs with m frames if and only if $r_t \notin S_t(m)$. Since $S_t(m) \subseteq S_t(m+1)$ (the inclusion property), we have:

$$r_t \notin S_t(m+1) \implies r_t \notin S_t(m)$$

The contrapositive states: a page fault with $m+1$ frames implies a page fault with m frames. Therefore, every fault with $m+1$ frames is also a fault with m frames, so $f_A(\omega, m+1) \leq f_A(\omega, m)$. □

Theorem 12.4 (OPT and LRU are Stack Algorithms). Both OPT and LRU satisfy the inclusion property and are therefore immune to Belady's anomaly.

Proof for LRU. LRU maintains pages ordered by their most recent access time. With m frames, LRU keeps the m most recently used pages. With $m+1$ frames, LRU keeps the $m+1$ most recently used pages. Since the m most recently used pages are a subset of the $m+1$ most recently used pages, $S_t(m) \subseteq S_t(m+1)$. □

Proof for OPT. OPT can be characterised by a *priority stack* where pages are ordered by their next use time. With m frames, OPT keeps the m pages with the soonest next use. With $m+1$ frames, OPT keeps the $m+1$ pages with the soonest next use. The m-frame set is a subset of the $(m+1)$-frame set. □

Theorem 12.5 (FIFO is Not a Stack Algorithm). FIFO does not satisfy the inclusion property. The counter-example in Example 12.4 serves as proof.

Proof. At step 7 of the 3-frame trace, $S_7(3) = \{5, 1, 2\}$. At step 7 of the 4-frame trace, $S_7(4) = \{2, 3, 4, 5\}$. We have $1 \in S_7(3)$ but $1 \notin S_7(4)$, so $S_7(3) \not\subseteq S_7(4)$. □

12.5 LRU (Least Recently Used)

12.5.1 Definition

Definition 12.6 (LRU Replacement). *LRU* evicts the page that has not been used for the longest time. It is based on the heuristic that pages used recently are likely to be used again soon (temporal locality), and pages not used recently are unlikely to be used soon.

LRU is OPT's "mirror image": where OPT looks forward in time to find the page with the most distant next use, LRU looks backward in time to find the page with the most distant last use. Under the assumption that past behaviour predicts future behaviour (the locality principle), LRU is a good approximation of OPT.

Example 12.5 (LRU Execution). Reference string: $\omega = 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7,$ Frames: $m = 3$.

Step	Ref	Frames (most recent first)	Fault?	Evicted
1	7	{7}	Yes	–
2	0	{0, 7}	Yes	–
3	1	{1, 0, 7}	Yes	–
4	2	{2, 1, 0}	Yes	7 (LRU)
5	0	{0, 2, 1}	No	–
6	3	{3, 0, 2}	Yes	1 (LRU)
7	0	{0, 3, 2}	No	–
8	4	{4, 0, 3}	Yes	2 (LRU)
9	2	{2, 4, 0}	Yes	3 (LRU)
10	3	{3, 2, 4}	Yes	0 (LRU)
11	0	{0, 3, 2}	Yes	4 (LRU)

Step	Ref	Frames (most recent first)	Fault?	Evicted
12	3	{3, 0, 2}	No	–
13	2	{2, 3, 0}	No	–
14	1	{1, 2, 3}	Yes	0 (LRU)
15	2	{2, 1, 3}	No	–
16	0	{0, 2, 1}	Yes	3 (LRU)
17	1	{1, 0, 2}	No	–
18	7	{7, 1, 0}	Yes	2 (LRU)
19	0	{0, 7, 1}	No	–
20	1	{1, 0, 7}	No	–

Total page faults: 12. Better than FIFO (15), worse than OPT (9).

12.5.2 Implementation Challenges

Exact LRU requires maintaining a total ordering of pages by their last access time. Two common hardware implementations:

Counter-based: Each PTE has a counter field. On every memory access, a system-wide clock increments, and the accessed page's counter is set to the current clock value. On a page fault, the OS scans all frames to find the one with the smallest counter (the least recently accessed). Cost: $O(m)$ per page fault for the scan. The counter must be wide enough to avoid wraparound during the process's lifetime; a 64-bit counter suffices.

Example 12.5a (Counter-Based LRU). A system with 4 frames. The system clock starts at 0 and increments on every memory access.

Access	Page	Clock	Frame 0 (cnt)	Frame 1 (cnt)	Frame 2 (cnt)	Frame 3 (cnt)
1	A	1	A (1)	–	–	–
2	B	2	A (1)	B (2)	–	–
3	C	3	A (1)	B (2)	C (3)	–
4	D	4	A (1)	B (2)	C (3)	D (4)

Access	Page	Clock	Frame 0 (cnt)	Frame 1 (cnt)	Frame 2 (cnt)	Frame 3 (cnt)
5	A	5	A (5)	B (2)	C (3)	D (4)
6	E	6	A (5)	E (6)	C (3)	D (4)

At access 6, page E causes a fault. The LRU page is B (counter=2, the smallest). B is evicted and replaced by E.

Stack-based: A doubly-linked list (stack) of page numbers. On every reference, the accessed page is moved to the top. On a page fault, the bottom of the stack is the LRU page. Cost: $O(1)$ per page fault, but moving the accessed page to the top on every memory access requires maintaining the linked list on *every* memory access — this means updating 4–6 pointers billions of times per second. Without specialised hardware, the overhead is prohibitive.

Why exact LRU is impractical for operating systems: Modern systems have millions of physical frames. Scanning all frames to find the minimum counter takes milliseconds. Maintaining a linked list on every memory access (which happens every few nanoseconds) is impossible in software. Even hardware assistance would add significant area and power cost. This motivates the LRU approximations in Section 12.6, which use only the single *accessed bit* maintained by standard MMU hardware.

12.5.3 LRU and the Stack Distance

Definition 12.6a (Stack Distance). For a reference to page p at time t, the *stack distance* is the number of distinct pages accessed since p was last referenced. If p has not been previously referenced, the stack distance is ∞.

Stack distance is a powerful tool for analysing LRU: a reference with stack distance d is a hit if $d \leq m$ (the page is within the m most recently used pages) and a miss if $d > m$. The stack distance distribution of a workload completely characterises its behaviour under LRU for all frame counts simultaneously.

Theorem 12.5a (LRU Miss Rate from Stack Distance). Let D be the random variable representing the stack distance of a reference. Then the LRU miss rate for m frames is:

$$p_{\text{miss}}(m) = P(D > m)$$

This means the miss rate is the tail probability of the stack distance distribution.

Example 12.5b (Stack Distance). Reference string: A, B, C, A, B, D, A, B, C.

Time	Ref	Stack (most recent first)	Stack Distance
1	A	A	∞
2	B	B, A	∞
3	C	C, B, A	∞
4	A	A, C, B	3
5	B	B, A, C	3
6	D	D, B, A, C	∞
7	A	A, D, B, C	2
8	B	B, A, D, C	3
9	C	C, B, A, D	4

With $m = 3$ frames: misses occur when stack distance > 3 or ∞. Misses at times 1, 2, 3, 6, 9 = 5 faults. Hits at times 4 (d=3 $\leq$ 3), 5 (d=3), 7 (d=2), 8 (d=3) = 4 hits.

With $m = 4$ frames: the additional miss at time 9 (d=4 $\leq$ 4) becomes a hit. Faults = 4.

The stack distance distribution also confirms the inclusion property: since a hit with m frames (stack distance $\leq m$) is certainly a hit with $m + 1$ frames (stack distance $\leq m + 1$), adding frames can only reduce faults.

12.6 LRU Approximations

12.6.1 The Clock Algorithm (Second-Chance)

Definition 12.7 (Clock Algorithm / Second-Chance). The *clock algorithm* uses the hardware-maintained *accessed bit* (A bit) in each PTE to approximate LRU. Frames are arranged in a circular buffer with a "clock hand" pointer.

On a page fault: 1. Examine the frame at the clock hand. 2. If its A bit is 0, evict this page (it has not been accessed since the last sweep). 3. If its A bit is 1, clear it to 0 (give the page a "second chance") and advance the clock hand. 4. Repeat until a victim with A = 0 is found.

```
        Clock hand
            |
            v
    +---+---+---+---+---+---+---+---+
    | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |  <-- A bits
    +---+---+---+---+---+---+---+---+
      0   1   2   3   4   5   6   7     <-- Frame numbers

Step 1: Frame 0, A=1 -> clear to 0, advance
Step 2: Frame 1, A=0 -> EVICT frame 1
```

The clock algorithm is O(m) in the worst case per page fault (if all A bits are 1, the hand must sweep the entire buffer). However, with frequent accesses resetting A bits, the amortised cost is much lower.

Theorem 12.6 (Clock as LRU Approximation). The clock algorithm partitions pages into two classes: recently accessed (A = 1) and not recently accessed (A = 0). It always evicts a page from the latter class. This provides an approximation of LRU where the "recently used" threshold is determined by the sweep rate of the clock hand. Under high memory pressure (frequent page faults), the hand sweeps quickly, providing finer-grained approximation. Under low memory pressure, the hand moves slowly, and the approximation is coarse.

12.6.2 Enhanced Second-Chance (NRU)

Definition 12.8 (Enhanced Second-Chance / NRU). The *enhanced second-chance* algorithm uses both the accessed bit (A) and the dirty bit (D) to classify pages into four categories, evicting pages from the lowest category first:

Class	(A, D)	Description
0	(0, 0)	Not accessed, not dirty (best victim)
1	(0, 1)	Not accessed, dirty
2	(1, 0)	Accessed, not dirty
3	(1, 1)	Accessed, dirty (worst victim)

The algorithm sweeps the circular buffer looking for a Class 0 page. If none is found, it looks for Class 1, then Class 2, then Class 3. Each sweep clears the A bits, so on the second pass, Class 2 and 3 pages become Class 0 and 1.

The rationale for preferring clean pages over dirty ones: evicting a clean page is free (no disk write needed), while evicting a dirty page requires writing it to the backing store first. The I/O cost difference is significant: evicting a clean page takes ~0 ns of I/O; evicting a dirty page requires a 4 KB write to disk, costing ~50 μs on SSD or ~8 ms on HDD.

Example 12.6a (Enhanced Clock Trace). 8 frames with initial state:

Frame	Page	A	D	Class
0	P	1	0	2
1	Q	0	1	1
2	R	1	1	3
3	S	0	0	0
4	T	1	0	2
5	U	0	0	0
6	V	1	1	3
7	W	0	1	1

Clock hand at frame 0. Page fault for page X.

Sweep for Class 0: start at frame 0 (Class 2, skip), frame 1 (Class 1, skip), frame 2 (Class 3, skip), frame 3 (Class 0, found).

Evict page S from frame 3. Load page X into frame 3. Set A=1, D=0. Advance clock hand to frame 4.

Note that frame 3 was chosen over frame 5 (also Class 0) because the clock hand encountered frame 3 first. The algorithm does not search for the "best" Class 0 page; it takes the first one encountered.

12.6.3 Linux's Two-List LRU

Linux does not use a simple clock algorithm. Instead, it maintains two LRU lists per memory zone:

Definition 12.8a (Linux Active/Inactive Lists). Linux's page reclaim uses two linked lists:

- **Active list:** Pages that have been recently accessed. Pages on this list are considered "hot" and are not candidates for eviction.
- **Inactive list:** Pages that have not been recently accessed. Pages on this list are candidates for eviction.

Pages are promoted from the inactive list to the active list when accessed (and their accessed bit is set). Pages are demoted from the active list to the inactive list when the active list becomes too large relative to the inactive list.

The lists are further divided by page type:

- **Anonymous pages** (heap, stack): tracked on `anon_active` and `anon_inactive` lists.
- **File-backed pages** (page cache): tracked on `file_active` and `file_inactive` lists.

This distinction allows the kernel to independently tune the eviction policy for anonymous and file-backed pages. The `vm.swappiness` parameter controls the relative weight: higher values evict more anonymous pages (writing them to swap), lower values evict more file-backed pages (which can be re-read from the file system).

12.6.4 Multi-Generation LRU (MGLRU)

Definition 12.8b (MGLRU). *Multi-Generation LRU* is a page reclaim algorithm introduced in Linux 6.1 that replaces the two-list (active/inactive) approach with multiple generations. Each page is assigned a generation number. Newer generations contain more recently accessed pages. The kernel scans pages by generation, evicting from the oldest generation first.

MGLRU provides finer-grained age tracking than the two-list approach. Instead of a binary "active/inactive" classification, pages pass through 4 generations, giving 4 levels of recency. This reduces both false positives (evicting a page that should be kept) and false negatives (keeping a page that should be evicted).

MGLRU is particularly effective for workloads with large working sets and gradual access pattern shifts, such as databases and virtual machines.

12.7 Counting-Based Algorithms

12.7.1 LFU (Least Frequently Used)

Definition 12.9 (LFU). *LFU* evicts the page with the lowest access count. Each page has a counter that is incremented on every access.

LFU has a problem: a page that was heavily used in the past but is no longer needed retains a high count and is never evicted. Solutions include periodically halving all counters (ageing) or using a decaying average.

12.7.2 MFU (Most Frequently Used)

Definition 12.10 (MFU). *MFU* evicts the page with the highest access count, on the theory that a page with many accesses has been in memory a long time and the page with the fewest accesses was just brought in and will be needed again soon.

MFU is counterintuitive and rarely performs well in practice. It is included for completeness.

Note

Neither LFU nor MFU is commonly used in operating system kernels. LRU approximations (clock algorithm, enhanced second-chance) dominate because they are simple, efficient, and provide adequate performance for the vast majority of workloads.

12.8 Frame Allocation

Before discussing page replacement policies, we must address a prerequisite question: how many frames does each process receive?

12.8.1 Minimum Number of Frames

Each process needs a minimum number of frames determined by the instruction set architecture. An instruction that can reference multiple memory operands (e.g., a memory-to-memory move with indirect addressing) may touch several pages during a single instruction execution. If the process has fewer frames than the instruction requires, the instruction cannot complete (it would page fault, evict a page needed by the same instruction, and fault again in an infinite loop).

Definition 12.11 (Minimum Frame Count). The minimum number of frames m_{min} for a process is determined by the maximum number of pages that can be referenced during the execution of a single instruction. For x86-64, this includes the instruction page(s), data operand page(s), and potentially a page table page, giving $m_{\text{min}} \approx 6$–8 depending on the instruction.

12.8.2 Equal Allocation

Definition 12.12 (Equal Allocation). With n processes and M total frames, each process receives $\lfloor M/n \rfloor$ frames. The remaining M mod n frames are kept in a free pool.

Equal allocation is simple but unfair: a 10 MB process gets the same number of frames as a 10 GB process.

12.8.3 Proportional Allocation

Definition 12.13 (Proportional Allocation). Each process receives frames in proportion to its size. If process i has virtual memory size s_i, it receives:

$$a_i = \left\lfloor \frac{s_i}{\sum_{j=1}^{n} s_j} \times M \right\rfloor$$

frames, where M is the total number of available frames.

Example 12.6 (Proportional Allocation). Total frames: $M = 256$. Two processes: P1 has $s_1 = 10$ MB, P2 has $s_2 = 126$ MB. Total: 136 MB.

$$a_1 = \left\lfloor \frac{10}{136} \times 256 \right\rfloor = \lfloor 18.8 \rfloor = 18$$
$$a_2 = \left\lfloor \frac{126}{136} \times 256 \right\rfloor = \lfloor 237.2 \rfloor = 237$$

Remaining frames: $256 - 18 - 237 = 1$ (kept in free pool).

Proportional allocation can also factor in process priority: higher-priority processes receive proportionally more frames.

12.8.4 Global vs Local Replacement

Definition 12.14 (Global Replacement). Under *global replacement*, when a process faults, the replacement algorithm can select a victim frame from any process in

the system (including the faulting process itself).

Definition 12.15 (Local Replacement). Under *local replacement*, when a process faults, the replacement algorithm selects a victim frame only from the faulting process's own set of frames.

Global replacement allows a high-priority or memory-hungry process to steal frames from other processes, potentially causing those processes to fault more. Local replacement isolates processes from each other but may lead to inefficiency: a process with idle frames cannot donate them to a process that needs them.

Example 12.6b (Global vs Local). A system has 100 frames and 3 processes with working set sizes: P1 = 20, P2 = 50, P3 = 40. Total demand: 110 frames (exceeds capacity).

Under equal allocation (33 frames each): - P1 has 33 frames for a 20-frame working set: no thrashing, 13 idle frames wasted. - P2 has 33 frames for a 50-frame working set: thrashing, 17 frames short. - P3 has 33 frames for a 40-frame working set: moderate thrashing, 7 frames short.

Under proportional allocation: - P1: $\lfloor 20/110 \times 100 \rfloor = 18$ frames (ok, working set = 20, slight deficit). - P2: $\lfloor 50/110 \times 100 \rfloor = 45$ frames (deficit of 5). - P3: $\lfloor 40/110 \times 100 \rfloor = 36$ frames (deficit of 4).

Better balanced, but all three processes have some deficit.

Under global replacement, P1 might "donate" its excess frames to P2 and P3 dynamically, but P1's fault rate could unpredictably increase if P2 or P3 aggressively steal its frames.

Example 12.6c (Priority-Based Proportional Allocation). Same system, but P1 has priority 3, P2 has priority 1, P3 has priority 2. We weight by priority:

$$a_i = \left\lfloor \frac{s_i \times \text{priority}_i}{\sum_j s_j \times \text{priority}_j} \times M \right\rfloor$$

Weighted sizes: P1 = $20 \times 3 = 60$, P2 = $50 \times 1 = 50$, P3 = $40 \times 2 = 80$. Total: 190.

- P1: $\lfloor 60/190 \times 100 \rfloor = 31$
- P2: $\lfloor 50/190 \times 100 \rfloor = 26$
- P3: $\lfloor 80/190 \times 100 \rfloor = 42$

High-priority P1 gets more than its working set (31 > 20), providing headroom. Low-priority P2 gets only 26 frames for a 50-frame working set and will thrash.

Theorem 12.7 (Global vs Local Trade-off). Global replacement generally achieves higher system throughput than local replacement because it allows frames to flow to the processes that need them most. However, global replacement makes individual process performance less predictable and can lead to cascading faults if one process consumes too many frames.

Programmer's Perspective

Linux uses a hybrid approach. The kernel maintains a global pool of free pages and per-cgroup memory limits. The `kswapd` daemon reclaims pages globally, using a modified clock algorithm (two-list LRU: active and inactive lists). When a cgroup (process group) exceeds its memory limit, only that cgroup's pages are scanned for eviction, providing local replacement semantics within the global system. The `memory.max` cgroup v2 setting controls this limit. In practice, most containerised workloads (Podman, Kubernetes) set memory limits to prevent any single container from starving others.

12.9 Working Set Model

12.9.1 Locality and the Working Set

Definition 12.16 (Working Set). The *working set* of a process at time t with window size Δ is the set of pages referenced in the most recent Δ memory accesses:

$$W(t, \Delta) = \{p : p \text{ was referenced in the interval } (t - \Delta, t]\}$$

The *working set size* is $|W(t, \Delta)|$.

The working set captures the process's current locality: the pages it is actively using. As the process moves through different phases of execution (e.g., initialisation, main loop, cleanup), its working set changes.

Example 12.7 (Working Set). Reference string: $\ldots 2, 6, 1, 5, 7, 7, 7, 7, 5, 1, \ldots$ with $\Delta = 4$.

At time $t = 10$ (after the reference to 1), the last 4 references are $7, 7, 5, 1$. So $W(10, 4) = \{7, 5, 1\}$ and $|W(10, 4)| = 3$.

At time $t = 5$ (after the reference to 7), the last 4 references are $1, 5, 7, 7$. So $W(5, 4) = \{1, 5, 7\}$ and $|W(5, 4)| = 3$.

At time $t = 3$ (after the reference to 1), the last 4 references are $\ldots, 2, 6, 1$. So $W(3, 4) = \{2, 6, 1\}$ and $|W(3, 4)| = 3$.

12.9.2 The Working Set Model

Theorem 12.8 (Working Set Principle, Denning 1968). If each process is allocated frames equal to its working set size $|W(t, \Delta)|$, then:

1. The page fault rate is low (most faults are compulsory faults for pages entering the working set for the first time).
2. The total number of frames in use across all processes is $D = \sum_i |W_i(t, \Delta)|$ (the *total demand*).
3. If $D > M$ (total demand exceeds available frames), at least one process must be suspended (swapped out) to prevent thrashing.

The working set model provides a principled approach to multiprogramming control: the OS should admit a new process only if there are enough free frames to hold its working set.

12.9.3 Choosing Δ

The window size Δ is critical:

- **Too small:** The working set underestimates the process's needs, leading to excessive page faults.
- **Too large:** The working set overestimates, wasting frames on pages no longer needed.

In practice, Δ is chosen empirically, often in the range of 10,000 to 100,000 memory references.

12.9.4 Implementing the Working Set Model

Exact implementation of the working set model is expensive: maintaining the set of pages accessed in the last Δ references requires recording every memory access. Practical approximations use a timer interrupt:

1. Every T milliseconds, the OS interrupts each process and examines its page table entries.
2. For each page with the accessed bit set, the page is in the working set. Clear the accessed bit.
3. For each page with the accessed bit clear, increment a "not accessed" counter. If the counter exceeds a threshold (corresponding to Δ/T intervals), the page is not in the working set and can be reclaimed.

Example 12.7a (Working Set Approximation). Timer interval $T = 10$ ms. Threshold: 5 intervals (50 ms $\approx \Delta$). A page P's history:

Interval	Accessed?	Counter	In Working Set?
0	Yes	0 (reset)	Yes
1	Yes	0	Yes
2	No	1	Yes
3	No	2	Yes
4	No	3	Yes
5	No	4	Yes
6	No	5	No (evict candidate)
7	Yes	0 (reset)	Yes

Page P drops out of the working set after 5 consecutive intervals without access (50 ms).

12.9.5 Locality and Phase Behaviour

Definition 12.16a (Locality). A process exhibits *temporal locality* if it tends to reference the same pages repeatedly in short time intervals. It exhibits *spatial locality* if it tends to reference pages near recently accessed pages. Most real programs exhibit both forms of locality.

Processes typically execute in *phases*: an initialisation phase (touching many pages), a main processing phase (working within a smaller set), and a cleanup phase. The working set changes at phase transitions, causing a burst of page faults. Between transitions, the working set is stable and the fault rate is low.

Example 12.7b (Phase Behaviour). A matrix multiplication program with three phases:

1. **Initialisation (t = 0–100):** Allocate and fill matrices A, B, C. Working set: ~1500 pages (all three matrices). Page faults: ~1500 (cold start).
2. **Computation (t = 100–10000):** Multiply $C = A \times B$ using a blocked algorithm with 64 KB blocks. Working set: ~50 pages (one block from each matrix plus temporaries). Page faults: rare (only at block boundaries).

3. **Output (t = 10000–10100):** Write result matrix C to disk. Working set: ~500 pages (matrix C, scanned sequentially). Page faults: moderate (sequential access, mitigated by readahead).

The working set size changes dramatically between phases, from 1500 to 50 to 500 pages.

12.10 Thrashing

12.10.1 Definition and Cause

Definition 12.17 (Thrashing). *Thrashing* is the condition where a process (or the entire system) spends more time servicing page faults than executing useful instructions. It occurs when the total working set demand of all active processes exceeds the available physical memory.

The feedback loop that causes thrashing:

1. The OS increases the degree of multiprogramming (admits more processes).
2. Total working set demand exceeds physical memory.
3. Page fault rates rise dramatically for all processes.
4. Processes spend most of their time waiting for page I/O.
5. CPU utilisation drops (because all processes are waiting for I/O).
6. The OS scheduler, observing low CPU utilisation, admits more processes to "improve" utilisation.
7. This further increases memory pressure, causing more faults. Goto step 3.

```
  CPU
Utilisation
   ^
   |          ___
   |        /     \
   |      /         \
   |    /             \
   |  /                 \  <-- Thrashing begins
   | /                    \
   |/                       \___________
   +------------------------------------------->
         Degree of Multiprogramming
```

12.10.2 Detecting Thrashing

The OS can detect thrashing by monitoring:

- **Page fault rate:** If the system-wide page fault rate exceeds a threshold, the system may be thrashing.
- **Page fault frequency (PFF):** If a specific process's fault rate exceeds an upper threshold, it needs more frames. If it falls below a lower threshold, frames can be reclaimed.
- **CPU utilisation vs multiprogramming:** If CPU utilisation drops as the degree of multiprogramming increases, thrashing is likely.

12.10.3 Preventing Thrashing

Definition 12.18 (Page Fault Frequency Control). *PFF control* monitors each process's page fault rate:

- If the fault rate exceeds an upper threshold τ_{upper}, the process is allocated additional frames.
- If the fault rate falls below a lower threshold τ_{lower}, frames are reclaimed from the process.
- If no frames are available to satisfy a process exceeding τ_{upper}, a process is suspended (swapped out).

Theorem 12.9 (Thrashing Prevention). Thrashing is prevented if and only if the sum of all processes' working set sizes does not exceed the available physical memory:

$$\sum_{i=1}^{n} |W_i(t, \Delta)| \leq M$$

If this condition is violated, the system must reduce the degree of multiprogramming by suspending one or more processes (moving their pages to swap and freeing their frames for the remaining processes).

Proof. The "if" direction: when the condition holds, each process can have its working set resident in memory, so page faults are limited to compulsory faults (accessing a page for the first time or after a working set transition). Compulsory faults are infrequent relative to the total access count. The "only if" direction: when the condition is violated, at least one process cannot fit its working set. By Denning's working set theorem, this process will fault on every access to a page outside its allocated frames. If the deficit is large, the fault rate approaches the access rate, which is thrashing by definition. □

Example 12.8 (Thrashing Scenario). A system has $M = 1000$ frames. Five processes have working set sizes: 200, 300, 250, 150, 200 frames. Total demand: 1100 frames > 1000.

The OS must suspend one process. Suspending the process with working set 300 reduces demand to 800, leaving 200 free frames as a buffer. Now all four remaining processes can operate within their working sets without thrashing.

Programmer's Perspective: Linux's Page Reclaim and the OOM Killer.

Linux's approach to page replacement and thrashing prevention involves several subsystems:

kswapd: The kernel swap daemon runs as a kernel thread on each NUMA node. It wakes up when the number of free pages drops below a watermark and reclaims pages by scanning the inactive LRU list. Pages are promoted from the inactive list to the active list when accessed; pages that remain on the inactive list without being accessed are reclaimed. The scanning rate increases under memory pressure.

vm.swappiness: This sysctl parameter (0–200, default 60) controls how aggressively the kernel swaps out anonymous pages (heap, stack) relative to file-backed pages (page cache). A value of 0 tells the kernel to avoid swapping anonymous pages as long as possible; 200 maximises anonymous page reclaim. Database servers often set `vm.swappiness = 1` to minimise swap usage.

```
/* Check current swappiness */
/* sysctl vm.swappiness */

/* Set temporarily */
/* sysctl -w vm.swappiness=10 */

/* Set permanently in /etc/sysctl.conf */
/* vm.swappiness = 10 */
```

OOM Killer: When the kernel cannot free enough memory through normal reclaim (and the system is effectively thrashing or out of memory), the Out-Of-Memory killer selects a process to terminate. The victim is chosen by a heuristic score (`/proc/PID/oom_score`) that considers process memory usage, CPU time, and other factors. The `oom_score_adj` value (-1000 to $+1000$) allows administrators to bias the selection. Critical processes (databases, application servers) should have a low `oom_score_adj` to avoid being killed.

```
/* Protect a critical process from OOM killer */
/* echo -1000 > /proc/PID/oom_score_adj */
```

In Go, the runtime interacts with these mechanisms through `madvise` calls. When the garbage collector frees pages, it calls `madvise(MADV_DONTNEED)` (or `MADV_FREE` on newer kernels) to tell the kernel that the pages can be reclaimed without swap. This makes Go processes "good citizens" under memory pressure: their freed-but-not-returned pages are quickly reclaimable by kswapd without the cost of swapping them to disk.

You can monitor page reclaim activity on Linux using several tools:

```
/* Monitor page faults for a specific process */
/* perf stat -e page-faults,major-faults,minor-faults ./my_program */

/* Watch kswapd activity in real time */
/* vmstat 1 */
/* Columns: si = swap in (KB/s), so = swap out (KB/s) */

/* Detailed memory pressure info */
/* cat /proc/pressure/memory */
/* Output: some avg10=0.50 avg60=1.20 avg300=0.80 total=12345678 */
```

The Pressure Stall Information (PSI) subsystem (Linux 4.20+) provides a quantitative measure of memory pressure. The `avg10` value indicates the percentage of time in the last 10 seconds that at least some tasks were stalled waiting for memory. Values above 10% indicate significant memory pressure; values above 40% indicate the system is likely thrashing.

In Go, you can detect memory pressure through `runtime.ReadMemStats`: a rapidly increasing `NumGC` with `PauseTotalNs` growing suggests the garbage collector is running frequently to free memory. The `GOMEMLIMIT` environment variable (Go 1.19+) sets a soft memory target, causing the GC to run more aggressively when the heap approaches the limit, which can help avoid triggering the OOM killer.

```go
package main

import (
    "fmt"
    "runtime"
    "runtime/debug"
    "time"
)

func main() {
    // Set a 500 MB memory limit
```

```go
    debug.SetMemoryLimit(500 * 1024 * 1024)

    var m runtime.MemStats

    // Monitor memory usage
    for i := 0; i < 10; i++ {
        runtime.ReadMemStats(&m)
        fmt.Printf("HeapAlloc: %d MB, NumGC: %d, "+
            "GCCPUFraction: %.4f\n",
            m.HeapAlloc/1024/1024,
            m.NumGC,
            m.GCCPUFraction)
        time.Sleep(time.Second)
    }
}
```

12.10a Quantifying Thrashing

12.10a.1 Page Fault Rate Model

Theorem 12.9a (Fault Rate vs Frame Allocation). For a process with a working set of size w allocated m frames ($m < w$), the page fault rate under LRU with random access to the working set is approximately:

$$p \approx \frac{w - m}{w}$$

When $m = w$, the fault rate drops to approximately 0 (only compulsory faults). When $m = 1$, the fault rate is $(w - 1)/w \approx 1$ (every access faults).

For processes exhibiting temporal locality, the actual fault rate is lower because frequently accessed pages are more likely to be in memory. The Least Recently Used stack distance distribution captures this effect precisely.

12.10a.2 The Lifetime Curve

Definition 12.18a (Lifetime Curve). The *lifetime curve* of a process plots the mean time between page faults (the "lifetime") as a function of the number of allocated frames m. As m increases, the lifetime increases. The curve typically has a "knee" at $m \approx |W|$ (the working set size), where the lifetime increases sharply.

```
Lifetime
(time between
 faults)
    ^
    |                           __________
    |                          /
    |                         /
    |                        /
    |                       /
    |                  ____/  <-- Knee at m = |W|
    |           ______/
    |    _____/
    |___/
    +----------------------------------------->
                  Frames allocated (m)
```

The knee of the lifetime curve is the optimal operating point: allocating more frames beyond the knee yields diminishing returns. The working set model aims to keep each process at or near the knee.

12.10b Practical Thrashing Prevention on Linux

Linux uses several mechanisms to prevent and mitigate thrashing:

12.10b.1 Watermarks

The kernel maintains three watermarks for each memory zone:

- **High watermark (W_{high}):** When free memory is above this level, no reclaim activity occurs.
- **Low watermark (W_{low}):** When free memory drops below this level, `kswapd` wakes up and begins background reclaim.
- **Minimum watermark (W_{min}):** When free memory drops below this level, direct reclaim occurs synchronously (the allocating process itself must free pages before its allocation can proceed).

Example 12.8a (Watermark Configuration). A system with 16 GB of RAM, default watermarks:

```
$ cat /proc/zoneinfo | grep -A 5 "Normal"
Node 0, zone   Normal
```

```
  pages free     1234567
        min      16384
        low      20480
        high     24576
```

$W_{\text{min}} = 16384 \times 4$ KB $= 64$ MB $W_{\text{low}} = 20480 \times 4$ KB $= 80$ MB $W_{\text{high}} = 24576 \times 4$ KB $= 96$ MB

When free memory drops below 80 MB, kswapd starts reclaiming. Below 64 MB, allocations are stalled until reclaim completes.

12.10b.2 Memory Cgroups

Definition 12.18b (Memory Cgroup). A *memory cgroup* (control group) limits the memory usage of a group of processes. When the cgroup exceeds its limit, only pages belonging to that cgroup are reclaimed. This provides per-group local replacement, preventing one group from causing thrashing in another.

Example 12.8b (Cgroup Memory Limit). Setting a 512 MB memory limit for a container:

```
# Cgroup v2
echo 536870912 > /sys/fs/cgroup/mycontainer/memory.max
echo 268435456 > /sys/fs/cgroup/mycontainer/memory.high

# memory.max: hard limit (OOM kill if exceeded)
# memory.high: soft limit (throttling and aggressive reclaim)
```

When the container uses more than 256 MB (memory.high), the kernel aggressively reclaims its pages. If it exceeds 512 MB (memory.max), the OOM killer terminates a process within the container.

12.10b.3 Swap Limits

Linux allows disabling swap entirely or limiting its use per-cgroup:

```
# Disable swap for a cgroup
echo 0 > /sys/fs/cgroup/mycontainer/memory.swap.max

# System-wide: set swappiness to 0 (avoid anonymous page swap)
sysctl vm.swappiness=0
```

For latency-sensitive applications (databases, game servers), disabling swap ensures that memory access times are predictable (never incurring a swap-in delay).

12.11 Comparative Summary of Algorithms

Algorithm	Faults (typical)	Anomaly?	Complexity	Practical?
OPT	Minimum	No	$O(n \cdot m)$	No (needs future)
LRU	Good	No	$O(m)$ per fault	Expensive (exact)
FIFO	Poor	Yes	$O(1)$	Yes (but poor)
Clock	Good	No	Amortised $O(1)$	Yes (standard)
Enhanced Clock	Good	No	Amortised $O(1)$	Yes (standard)
LFU	Variable	Yes	$O(\log m)$	Rarely used
MFU	Poor	Yes	$O(\log m)$	No

Note

The "standard" practical algorithms used in real operating systems are the clock algorithm and its enhanced variant. Linux uses a two-list LRU approximation (active/inactive lists) that is conceptually similar to the clock algorithm but with additional refinements for file-backed vs anonymous pages, multi-generation ageing (MGLRU in newer kernels), and NUMA-awareness.

12.12 Summary

This chapter addressed the central question of demand paging: when physical memory is full, which page should be evicted?

- **OPT** evicts the page with the most distant future reference, achieving the minimum fault count. It is provably optimal but requires future knowledge.
- **FIFO** evicts the oldest page. It is simple but performs poorly and is susceptible to Belady's anomaly.
- **LRU** evicts the least recently used page, approximating OPT by using past behaviour to predict future behaviour. It is a stack algorithm (immune to Belady's anomaly) but expensive to implement exactly.

- **Clock algorithm** (second-chance) uses the accessed bit to approximate LRU efficiently, making it the dominant practical algorithm.
- **Frame allocation** (equal, proportional, global, local) determines how many frames each process receives.
- **Working set model** and **PFF** control prevent thrashing by ensuring each process has enough frames for its current locality.
- **Thrashing** is a catastrophic feedback loop where the system spends all its time servicing page faults. Prevention requires monitoring working set sizes and reducing multiprogramming when demand exceeds supply.

Chapter 13 extends these foundations with advanced topics: copy-on-write mechanics, memory-mapped file I/O, kernel memory allocators (slab, SLUB, buddy system), NUMA-aware allocation, huge pages, memory compression, and address space randomisation.

Exercise 12.1. Given the reference string $\omega = 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6$ and $m = 4$ frames, compute the number of page faults for (a) FIFO, (b) LRU, and (c) OPT. Show the state of the frames after each reference.

Exercise 12.2. Construct a reference string and frame count that demonstrates Belady's anomaly for FIFO — specifically, show that FIFO with $m = 4$ frames produces more faults than with $m = 3$ frames. Verify that LRU does not exhibit the anomaly for your reference string.

Exercise 12.3. Prove that LRU is a stack algorithm by formally establishing the inclusion property $S_t(m) \subseteq S_t(m+1)$ for all t. Use induction on t, considering the cases where the reference at time t is a hit or a miss in both the m-frame and $(m+1)$-frame simulations.

Exercise 12.4. A system uses the clock algorithm with 8 frames. The initial state is: frames contain pages {A, B, C, D, E, F, G, H} with accessed bits {1, 0, 1, 1, 0, 0, 1, 0}, and the clock hand points to frame 0. Trace the algorithm for the page fault on page X: show which frames are examined, which accessed bits are cleared, and which page is evicted.

Exercise 12.5. A system has 5000 frames of physical memory shared among 10 processes. Process sizes (in frames): 200, 400, 300, 600, 100, 800, 500, 350, 250, 450. (a) Calculate the frame allocation for each process under proportional allocation. (b) If the working set sizes are 150, 350, 280, 500, 80, 700, 450, 300, 200, 400, is the system at risk of thrashing? Justify your answer.

Exercise 12.6. The effective access time for demand paging is EAT $= (1-p) \times t_{\text{mem}} +$

$p \times t_{\text{fault}}$. A system upgrades from HDD ($t_{\text{fault}} = 8$ ms) to NVMe SSD ($t_{\text{fault}} = 80$ μs). Memory access time is 100 ns. (a) For a page fault rate of $p = 0.001$, calculate the EAT and slowdown factor for both storage technologies. (b) What page fault rate on HDD gives the same EAT as $p = 0.01$ on NVMe SSD? (c) Discuss the implications for the viability of swap-heavy workloads on SSDs vs HDDs.

Exercise 12.7. A process has the following reference pattern over time, with working set window $\Delta = 5$:

Reference string: $3, 4, 3, 2, 1, 4, 4, 3, 2, 5, 1, 1, 2, 3, 3$

(a) Compute the working set $W(t, 5)$ and working set size $|W(t, 5)|$ for $t = 5, 8, 10, 13, 15$. (b) If the system allocates frames equal to the working set size (updated at each time step), what is the maximum frame allocation needed by this process? (c) If the system has only 3 frames available for this process, at which time steps will it be at risk of thrashing?

Chapter 13: Advanced Memory Topics

The preceding three chapters built the foundations of memory management: address translation, demand paging, page tables, TLBs, and page replacement. These mechanisms are sufficient to understand how a basic virtual memory system works. But modern operating systems go far beyond the basics. They exploit copy-on-write to make `fork()` nearly free, use memory-mapped files to unify the file system and the address space, deploy specialised kernel memory allocators that eliminate fragmentation for internal data structures, handle non-uniform memory architectures where some RAM is faster than other RAM, use huge pages to tame TLB pressure, compress memory to defer swapping, and randomise address layouts to frustrate attackers.

This chapter covers these advanced topics. Each one is a refinement or extension of the core mechanisms, motivated by a real engineering problem and implemented in production operating systems. Together, they represent the state of the art in memory management.

13.1 Copy-on-Write

13.1.1 The fork() Problem

The `fork()` system call creates a child process that is a copy of the parent. Naively, this requires duplicating the parent's entire address space — potentially gigabytes of memory. Most of the time, the child immediately calls `exec()` to replace its address space with a new program, rendering the copy wasteful.

Definition 13.1 (Copy-on-Write). *Copy-on-write* (CoW) is an optimisation that defers the copying of memory pages until a write occurs. After `fork()`, the parent and child share all physical frames. The page table entries in both processes are marked read-only. When either process writes to a shared page, the hardware triggers a protection fault. The OS page fault handler then:

1. Allocates a new physical frame.
2. Copies the contents of the original frame to the new frame.

3. Updates the writing process's page table to point to the new frame (marked read-write).
4. If the original frame now has only one reference, marks it read-write in the remaining process's page table.

13.1.2 Reference Counting

The OS maintains a reference count for each physical frame, tracking how many page table entries point to it. When a CoW fault occurs, the reference count determines the action:

- **ref_count > 1:** Copy the page, decrement the original's reference count, set the new page's count to 1.
- **ref_count = 1:** No copy needed; just mark the page read-write (the process is the sole owner).

Definition 13.2 (Frame Reference Count). The *reference count* of a physical frame f, denoted $\text{refcount}(f)$, is the number of page table entries across all processes that currently map to f. A frame can be freed (returned to the free list) only when $\text{refcount}(f) = 0$.

13.1.3 CoW Fault Handling

Example 13.1 (CoW Fault Sequence). A parent process P has 4 pages mapped. After `fork()`, child C shares all 4 frames:

```
Before fork:
  P: PTE[0]->F0(rw), PTE[1]->F1(rw), PTE[2]->F2(rw), PTE[3]->F3(rw)
  refcounts: F0=1, F1=1, F2=1, F3=1

After fork (CoW):
  P: PTE[0]->F0(ro), PTE[1]->F1(ro), PTE[2]->F2(ro), PTE[3]->F3(ro)
  C: PTE[0]->F0(ro), PTE[1]->F1(ro), PTE[2]->F2(ro), PTE[3]->F3(ro)
  refcounts: F0=2, F1=2, F2=2, F3=2

Child writes to page 1:
  1. Protection fault on C's PTE[1] (read-only)
  2. Allocate new frame F4
  3. Copy F1 -> F4
  4. C: PTE[1]->F4(rw)
  5. refcount(F1)-- => 1, so P: PTE[1]->F1(rw)
  refcounts: F0=2, F1=1, F2=2, F3=2, F4=1
```

```
After child writes to pages 1 and 2:
  P: PTE[0]->F0(ro), PTE[1]->F1(rw), PTE[2]->F2(ro), PTE[3]->F3(ro)
  C: PTE[0]->F0(ro), PTE[1]->F4(rw), PTE[2]->F5(rw), PTE[3]->F3(ro)
  Only pages actually written are copied.
```

13.1.4 Cost Analysis

Theorem 13.1 (CoW Cost Bound). Let N be the total number of pages in the parent process and W be the number of pages written by the child before calling `exec()`. The cost of `fork()` with CoW is:

$$C_{\text{CoW}} = C_{\text{table}} + W \times C_{\text{copy}}$$

where C_{table} is the cost of duplicating the page table and C_{copy} is the cost of copying one page. Without CoW:

$$C_{\text{naive}} = C_{\text{table}} + N \times C_{\text{copy}}$$

The savings ratio is:

$$\frac{C_{\text{naive}}}{C_{\text{CoW}}} \approx \frac{N}{W}$$

For a typical `fork()`/`exec()` pattern where $W \ll N$ (the child writes to only a few stack pages before calling `exec()`), this ratio can be 1000:1 or more.

13.1.5 CoW for Zero Pages

A further optimisation: when a process allocates memory (e.g., via `mmap` with `MAP_ANONYMOUS`), the OS does not immediately allocate physical frames. Instead, all pages in the region are mapped to a single read-only *zero page*. When the process writes to a page, a CoW fault allocates a new frame and fills it with zeros. This is called *demand zeroing* and means that `calloc()` or `mmap()` of a large region is nearly free until the memory is actually used.

Example 13.2 (Demand Zeroing). A process calls `mmap(NULL, 1 GB, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)`. The kernel maps 262,144 pages (at 4 KB each) to the zero page. Physical memory used: 0 bytes (plus one page table page for the mapping).

The process then writes to addresses in 100 of those pages. Each write triggers a CoW fault that allocates a fresh, zeroed frame. Physical memory used: 100 pages (400 KB), not 1 GB.

Note

This is why `htop` shows two memory columns for each process: VIRT (virtual memory, including all mapped-but-not-touched regions) and RES (resident set size, only pages backed by physical frames). A Java or Go process may show VIRT of several gigabytes but RES of only a few hundred megabytes.

13.2 Memory-Mapped Files

13.2.1 Concept

Definition 13.3 (Memory-Mapped File). A *memory-mapped file* is a file whose contents are mapped into a process's virtual address space. Reads and writes to the mapped region are translated into reads and writes to the file by the virtual memory system. The file's pages are loaded on demand (via page faults) and written back to disk by the OS's page writeback mechanism.

Memory-mapped files unify file I/O and memory access. Instead of using `read()`/`write()` system calls with user-space buffers, the program simply accesses memory addresses, and the kernel handles the rest.

13.2.2 mmap() Semantics

```c
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
int msync(void *addr, size_t length, int flags);
int munmap(void *addr, size_t length);
```

Definition 13.4 (mmap Parameters).

- `addr`: Suggested starting address (usually NULL, letting the kernel choose).
- `length`: Number of bytes to map.
- `prot`: Protection flags: `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`, `PROT_NONE`.
- `flags`: Mapping type:

  - `MAP_SHARED`: Writes are visible to other processes mapping the same file and are carried through to the underlying file.
  - `MAP_PRIVATE`: Writes create a private copy (CoW). Changes are not written to the file.
  - `MAP_ANONYMOUS`: Not backed by any file; used for allocating anonymous memory.
- `fd`: File descriptor of the file to map (-1 for anonymous).
- `offset`: Offset within the file where the mapping begins (must be page-aligned).

13.2.3 Private vs Shared Mappings

Definition 13.5 (Shared Mapping). A `MAP_SHARED` mapping creates a view of the file that is shared with all other processes that map the same file region. Writes by any process are visible to all others and are eventually written back to the file on disk.

Definition 13.6 (Private Mapping). A `MAP_PRIVATE` mapping creates a copy-on-write view of the file. The initial contents come from the file, but writes create private copies that are not visible to other processes and are not written to the file.

Shared mappings are used for inter-process communication and shared databases. Private mappings are used for loading executable code and read-only data (shared libraries are loaded via private mappings of their `.text` segments).

13.2.4 msync()

Definition 13.7 (msync). The `msync()` system call flushes modified pages of a shared mapping back to the underlying file. Without `msync()`, the kernel may buffer dirty pages indefinitely, and data could be lost on a crash.

- `MS_SYNC`: Synchronous flush; `msync()` blocks until all pages are written.
- `MS_ASYNC`: Asynchronous flush; schedules the writes but returns immediately.
- `MS_INVALIDATE`: Invalidates other mappings of the same file, forcing them to re-read from disk.

Example 13.3 (Memory-Mapped File I/O). Reading and modifying a file via `mmap`:

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
```

```c
int main(void) {
    int fd = open("data.bin", O_RDWR);
    if (fd < 0) { perror("open"); return 1; }

    struct stat sb;
    fstat(fd, &sb);

    /* Map the entire file as shared */
    char *map = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) { perror("mmap"); return 1; }
    close(fd);  /* fd can be closed; mapping persists */

    /* Read directly from memory */
    printf("First 10 bytes: %.10s\n", map);

    /* Write directly to memory -> writes to file */
    memcpy(map, "MODIFIED: ", 10);

    /* Ensure changes reach disk */
    msync(map, sb.st_size, MS_SYNC);

    munmap(map, sb.st_size);
    return 0;
}
```

13.2.5 Performance Advantages

Memory-mapped I/O avoids the double-copy inherent in `read()`/`write()`: data goes directly from the page cache to the process's address space (they share the same physical pages). There is no intermediate kernel buffer copy.

The `read()` system call path involves:

1. User calls `read(fd, buf, len)`. Context switch to kernel.
2. Kernel checks the page cache. If the pages are present, copies data from the page cache to the user buffer `buf`. If not, reads from disk into the page cache, then copies.
3. Return to user space.

The `mmap()` path:

1. User accesses `*ptr` (a memory load instruction). No system call.
2. If the page is in the TLB, the access completes in ~1 ns.
3. If TLB miss, the hardware walks the page table (~20–100 ns).
4. If the page is not in physical memory, a page fault loads it from disk.
5. No copy: the user reads directly from the page cache page.

Theorem 13.2 (mmap vs read/write). For sequential reads, `mmap` and `read()` have comparable performance because both benefit from the kernel's readahead. For random access patterns, `mmap` can be significantly faster because:

1. No system call overhead per access (after the initial `mmap` call). A `read()` system call costs ~200–500 ns; a TLB-cached `mmap` access costs ~1 ns.
2. No buffer copy. `read()` copies data from the page cache to user space; `mmap` shares the page cache pages directly.
3. The hardware TLB caches frequently accessed translations.

The advantage diminishes when the working set exceeds TLB coverage, as TLB misses add latency comparable to system call overhead.

Example 13.3a (Random Access Performance). A database index file of 10 GB is accessed randomly by key lookup. Each lookup reads 256 bytes from a random position.

Using `pread()`: each lookup requires a system call (~300 ns) plus a potential page fault. For 1 million lookups per second, the system call overhead alone is $10^6 \times 300 = 300$ ms/s (30% of a core).

Using `mmap()`: each lookup is a pointer dereference. If the page is in the TLB and page cache, the access takes ~5 ns (TLB lookup + L1 cache miss to page cache). For 1 million lookups: $10^6 \times 5 = 5$ ms/s (0.5% of a core). A 60x improvement in CPU efficiency.

13.2.6 Memory-Mapped IPC

`mmap()` with `MAP_SHARED` provides a high-performance inter-process communication mechanism. Two processes can map the same file (or anonymous shared mapping via `shm_open()`) and communicate by reading and writing the shared pages.

Definition 13.7a (Shared Memory IPC). Two or more processes can establish shared memory by:

1. Creating a shared memory object: `int fd = shm_open("/my_shm", O_CREAT | O_RDWR, 0600);`
2. Setting its size: `ftruncate(fd, size);`

3. Mapping it in both processes: `mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);`

Writes by one process are immediately visible to the other (they share the same physical pages). No kernel involvement is needed for the data transfer itself — only for the initial setup.

Shared memory is the fastest IPC mechanism because it avoids all copying and system calls during data transfer. The processes must coordinate access using synchronisation primitives (mutexes, semaphores, atomics) placed in the shared region.

Example 13.3b (Shared Memory Throughput). Two processes share a 1 GB mapping. Process A writes 1 MB chunks; Process B reads them.

- Pipe: ~4 GB/s (kernel copy: user → kernel → user).
- Unix domain socket: ~6 GB/s (similar to pipe but slightly more efficient).
- Shared memory: ~20 GB/s (limited by DRAM bandwidth, no copies).

Shared memory achieves 3–5x the throughput of pipe/socket IPC because it eliminates all copies. The actual throughput is bounded by memory bandwidth rather than kernel overhead.

Programmer's Perspective

Memory-mapped files are the foundation of many high-performance systems. BoltDB (used by etcd) maps its entire database file into memory, achieving read performance limited only by page fault and TLB miss rates. SQLite uses `mmap` as an optional I/O method. In Go, the `golang.org/x/exp/mmap` package provides a portable memory-mapped file reader. For write-heavy workloads, be aware that `mmap` with `MAP_SHARED` means the OS can write dirty pages to disk at any time, potentially interfering with application-level write ordering. Databases that require strict write ordering (WAL protocols) typically use `pwrite()` with `fdatasync()` rather than `mmap`.

POSIX shared memory in Go is accessible through the `golang.org/x/sys/unix` package:

```go
package main

import (
    "fmt"
    "golang.org/x/sys/unix"
    "unsafe"
)

func main() {
    // Create shared memory object
    fd, err := unix.ShmOpen("/go_shm",
        unix.O_CREAT|unix.O_RDWR, 0600)
    if err != nil {
        panic(err)
    }
    defer unix.Close(fd)

    size := 4096
    unix.Ftruncate(fd, int64(size))

    // Map into this process
    data, err := unix.Mmap(fd, 0, size,
        unix.PROT_READ|unix.PROT_WRITE,
        unix.MAP_SHARED)
    if err != nil {
        panic(err)
    }
    defer unix.Munmap(data)
```



```go
    // Write a message
    copy(data, []byte("Hello from Go!"))
    fmt.Printf("Wrote to shared memory at %p\n",
```

13.3 Kernel Memory Allocation

The kernel itself needs to allocate memory for its own data structures: process descriptors, file objects, inode caches, network buffers, page tables, and hundreds of other types. User-space allocation strategies (paging, demand allocation) are not always appropriate for kernel memory because:

1. Kernel memory is often non-pageable (cannot be swapped to disk).
2. Many kernel allocations are for small, fixed-size objects.
3. Kernel allocations must be fast (microseconds, not milliseconds).
4. Some allocations require physically contiguous memory (for DMA).

13.3.1 The Buddy System

Definition 13.8 (Buddy System). The *buddy system* is a memory allocation algorithm that divides memory into blocks whose sizes are powers of two. When a block of size 2^k is requested:

1. If a free block of size 2^k exists, allocate it.
2. Otherwise, find the smallest free block of size $2^j > 2^k$.
3. Split it repeatedly into two *buddies* of size 2^{j-1} until a block of size 2^k is obtained.
4. On deallocation, check if the freed block's buddy is also free. If so, merge them into a block of size 2^{k+1} (*coalescing*). Repeat upward.

Example 13.4 (Buddy System). Initial memory: one block of 256 KB. Request: 21 KB.

The smallest power of two $\geq$ 21 KB is 32 KB.

1. Split 256 KB $\rightarrow$ two 128 KB buddies.
2. Split one 128 KB $\rightarrow$ two 64 KB buddies.
3. Split one 64 KB $\rightarrow$ two 32 KB buddies.
4. Allocate one 32 KB block.

```
        256 KB
       /      \
   128 KB    128 KB
   /    \
 64 KB  64 KB
 /    \
```

```
32 KB 32 KB
[A]   [free]
```

Free blocks: 32 KB, 64 KB, 128 KB. Internal fragmentation: $32 - 21 = 11$ KB (34%).

Theorem 13.3 (Buddy System Fragmentation). The worst-case internal fragmentation of the buddy system is nearly 50%: a request for $2^{k-1}+1$ bytes receives a block of 2^k bytes, wasting $2^{k-1} - 1$ bytes. On average, for uniformly distributed request sizes in $[1, 2^n]$, the expected internal fragmentation is approximately 25%.

Proof. For a request of size s where $2^{k-1} < s \leq 2^k$, the allocated block is 2^k bytes, wasting $2^k - s$ bytes. The waste ratio is $(2^k - s)/2^k$. For s uniform on $(2^{k-1}, 2^k]$, the expected waste ratio is:

$$E\left[\frac{2^k - s}{2^k}\right] = \frac{1}{2^{k-1}} \int_{2^{k-1}}^{2^k} \frac{2^k - s}{2^k} ds = \frac{1}{2^k \cdot 2^{k-1}} \cdot \frac{(2^{k-1})^2}{2} = \frac{1}{4}$$

Hence the expected internal fragmentation is 25%. □

13.3.1a Buddy System: Deallocation and Coalescing

The buddy system's key advantage is fast coalescing: the buddy of a block at address A of size 2^k is at address $A \oplus 2^k$ (XOR with the block size). This makes buddy identification an $O(1)$ operation.

Example 13.4a (Buddy Coalescing). Continuing from Example 13.4, suppose we free the 21 KB allocation (block A at address 0, size 32 KB).

1. A's buddy is at address $0 \oplus 32\text{K} = 32\text{K}$. Check: is the 32 KB block at 32K free? Yes.
2. Merge: create a 64 KB block at address 0. Its buddy is at $0 \oplus 64\text{K} = 64\text{K}$. Check: is the 64 KB block at 64K free? Yes.
3. Merge: create a 128 KB block at address 0. Its buddy is at $0 \oplus 128\text{K} = 128\text{K}$. Check: is the 128 KB block at 128K free? Yes.
4. Merge: create a 256 KB block at address 0. No further merging possible (maximum size reached).

The memory is fully coalesced back into one 256 KB block. Total merges: 3. Each merge is $O(1)$ (XOR + lookup in the free list for that order).

Example 13.4b (Buddy System with Multiple Allocations). Starting with 256 KB. Sequence: allocate A=30KB, allocate B=20KB, allocate C=10KB, free B, allocate D=15KB, free A, free D, free C.

Step 1 – Allocate A=30KB (rounded to 32KB): Split 256$→$128 + 128, split128$→$64 + 64, split64$→$32+32. Allocate A at 0. Free: 32K@32, 64K@64, 128K@128.

Step 2 – Allocate B=20KB (rounded to 32KB): Use 32K@32. Free: 64K@64, 128K@128.

Step 3 – Allocate C=10KB (rounded to 16KB): Split 64K@64$→$32K@64 + 32K@96, split32K@64$→$16K@64+16K@80. Allocate C at 64. Free: 16K@80, 32K@96, 128K@128.

Step 4 – Free B (32KB at 32): Buddy at $32 \oplus 32 = 0$ (block A). A is allocated, so no merge. Free: 32K@32, 16K@80, 32K@96, 128K@128.

Step 5 – Allocate D=15KB (rounded to 16KB): Use 16K@80. Free: 32K@32, 32K@96, 128K@128.

Step 6 – Free A (32KB at 0): Buddy at $0 \oplus 32 = 32$. Block at 32 is free. Merge$→$64K@0. Buddy at $0 \oplus 64 = 64$. Block at 64 is C (allocated). No merge. Free: 64K@0, 32K@96, 128K@128.

Step 7 – Free D (16KB at 80): Buddy at $80 \oplus 16 = 64 + 16 = 64$. Wait: $80_{\text{dec}} = 0\text{x}50$, $16_{\text{dec}} = 0\text{x}10$. $0\text{x}50 \oplus 0\text{x}10 = 0\text{x}40 = 64$. Block at 64 is C (allocated). No merge. Free: 64K@0, 16K@80, 32K@96, 128K@128.

Step 8 – Free C (16KB at 64): Buddy at $64 \oplus 16 = 80$. Block at 80 is free (D was freed). Merge$→$32K@64. Buddy at $64 \oplus 32 = 96$. Block at 96 is free. Merge$→$64K@64. Buddy at $64 \oplus 64 = 0$. Block at 0 is free (64K). Merge$→$128K@0. Buddy at $0 \oplus 128 = 128$. Block at 128 is free. Merge$→$256K@0.

Full coalescing achieved after all frees. This illustrates the elegance of the buddy system: deferred coalescing happens automatically when both buddies are free.

Linux's use: The Linux kernel uses the buddy system as its primary page allocator. The `alloc_pages()` function returns 2^k contiguous pages (orders 0 through 10, giving 4 KB to 4 MB). The `/proc/buddyinfo` file shows the current distribution of free blocks:

```
$ cat /proc/buddyinfo
Node 0, zone      DMA      1    1    0    1    2    1    1    0    1    1    3
Node 0, zone    DMA32    130   84   36   18    5    2    1    0    0    0    0
Node 0, zone   Normal   3504 1827  912  480  241  120   60   30   15    7    3
```

Each column shows the number of free blocks of order 0 (4 KB), order 1 (8 KB), …, order 10 (4 MB). The rightmost column (order 10) shows how many 4 MB contiguous regions are available — essential for huge page allocation.

The buddy system's key advantage is fast coalescing: the buddy of a block at address A of size 2^k is at address $A \oplus 2^k$ (XOR with the block size). This makes buddy checking

an $O(1)$ operation.

Linux's use: The Linux kernel uses the buddy system as its primary page allocator. The `alloc_pages()` function returns 2^k contiguous pages (orders 0 through 10, giving 4 KB to 4 MB). The `/proc/buddyinfo` file shows the current distribution of free blocks:

```
$ cat /proc/buddyinfo
Node 0, zone   Normal    4   12   32   18    5    2    1    0    0    0    0
```

Each column shows the number of free blocks of order 0 (4 KB), order 1 (8 KB), ..., order 10 (4 MB).

13.3.2 The Slab Allocator

Definition 13.9 (Slab Allocator, Bonwick 1994). The *slab allocator* is a memory allocation system designed for kernel objects. It works as follows:

1. **Caches:** For each type of kernel object (e.g., `task_struct`, `inode`, `dentry`), a *cache* is created.
2. **Slabs:** Each cache consists of one or more *slabs*. A slab is a contiguous block of physical pages (obtained from the buddy system) divided into fixed-size *slots*, each capable of holding one object.
3. **States:** Each slab is in one of three states: *full* (all slots occupied), *partial* (some slots free), *empty* (all slots free).
4. **Allocation:** When an object is requested, the allocator finds a partial slab (or allocates a new slab from the buddy system) and returns a free slot.
5. **Deallocation:** When an object is freed, the allocator marks its slot as free. If the slab becomes empty, it may be returned to the buddy system.

Example 13.5 (Slab for task_struct). The `task_struct` in Linux is approximately 6 KB. The slab allocator creates a cache with:

- Slab size: 2 pages = 8192 bytes (from buddy system, order 1).
- Objects per slab: $\lfloor 8192/6144 \rfloor = 1$ (with some internal metadata).

For smaller objects like `dentry` (~192 bytes): - Slab size: 1 page = 4096 bytes. - Objects per slab: $\lfloor 4096/192 \rfloor \approx 21$.

The slab allocator provides three key benefits:

1. **No fragmentation for fixed-size objects:** Since each cache handles objects of exactly one size, there is no external fragmentation.
2. **Cache colouring:** Objects in different slabs are offset to reduce cache line conflicts.

3. **Constructor/destructor caching:** Freed objects retain their initialised state, so re-allocation can skip the constructor.

13.3.3 SLUB: The Simplified Slab

Definition 13.10 (SLUB Allocator). *SLUB* (the Unqueued Slab Allocator) is the default kernel memory allocator in Linux since 2.6.23. It simplifies the original slab allocator by eliminating the per-slab metadata and per-CPU queues. Instead, each CPU has a single "active" slab, and freed objects are placed on an in-slab free list (using the freed object's own memory as a linked list node).

SLUB provides better performance on modern multi-core systems because:

- Reduced metadata overhead (no separate slab descriptors for most caches).
- Better NUMA awareness (per-node partial slab lists).
- Simpler debugging (SLUB has built-in red-zone, poisoning, and tracking features enabled by `CONFIG_SLUB_DEBUG`).

The `slabinfo` command (or `/proc/slabinfo`) shows statistics for all active slab caches:

```
$ sudo cat /proc/slabinfo | head -5
# name          <active_objs> <num_objs> <objsize> <objperslab> <pagesperslab>
task_struct           452    460   6720      4      8
inode_cache          8291   8326    608     13      2
dentry              15204  15372    192     21      1
```

13.3.4 kmalloc and the Size-Class Hierarchy

For general-purpose kernel allocations (not tied to a specific object type), Linux provides `kmalloc()`, which allocates from a set of generic slab caches with sizes: 8, 16, 32, 64, 96, 128, 192, 256, 512, 1024, 2048, 4096, 8192 bytes. A request for n bytes is rounded up to the nearest size class.

Definition 13.11 (kmalloc Size Classes). The `kmalloc()` function allocates memory from a set of *size-class caches*. For a request of n bytes, the allocator selects the smallest size class $s_k \geq n$ and allocates one object from the corresponding cache. The internal fragmentation is at most $s_k - n$ bytes.

Example 13.6 (kmalloc Allocation). A kernel subsystem calls `kmalloc(100, GFP_KERNEL)`.

Available size classes: ..., 64, 96, 128, ...

The smallest class ≥ 100 is 128. Internal fragmentation: 28 bytes (22%).

The `ksize()` function returns the actual usable size of a `kmalloc` allocation:

```
void *p = kmalloc(100, GFP_KERNEL);
/* ksize(p) returns 128 */
```

Note

The GFP flags (`GFP_KERNEL`, `GFP_ATOMIC`, `GFP_DMA`, etc.) control how the allocator behaves. `GFP_KERNEL` allows sleeping (can trigger reclaim). `GFP_ATOMIC` is for interrupt context where sleeping is forbidden. `GFP_DMA` requests memory from the DMA-capable zone (lowest 16 MB on x86).

13.4 NUMA: Non-Uniform Memory Access

13.4.1 NUMA Architecture

Definition 13.12 (NUMA). In a *Non-Uniform Memory Access* architecture, physical memory is divided into *nodes*, each attached to a specific processor (or group of processors). Accessing memory on the local node is faster than accessing memory on a remote node.

```
+-------------------+       Interconnect       +-------------------+
|     Node 0        |<========================>|     Node 1        |
|                   |       (e.g. QPI, UPI)     |                   |
|  +------+  +----+ |                           | +------+  +----+ |
|  | CPU 0|  |RAM | |                           | | CPU 1|  |RAM | |
|  | CPU 1|  | 64 | |                           | | CPU 2|  | 64 | |
|  +------+  | GB | |                           | | CPU 3|  | GB | |
|            +----+ |                           | +------+  +----+ |
+-------------------+                           +-------------------+

Local access:  ~80 ns
Remote access: ~140 ns (1.75x slower)
```

13.4.2 NUMA Ratio

Definition 13.13 (NUMA Ratio). The *NUMA ratio* is the ratio of remote memory access latency to local memory access latency:

$$\text{NUMA ratio} = \frac{t_{\text{remote}}}{t_{\text{local}}}$$

Typical values range from 1.2 to 3.0, depending on the number of hops and interconnect technology.

13.4.3 NUMA-Aware Allocation

The operating system's memory allocator should preferentially allocate memory from the node local to the requesting CPU. Linux implements this through *NUMA policies*:

Definition 13.14 (NUMA Memory Policies).

- `MPOL_DEFAULT`: Allocate from the local node. Fall back to other nodes if local memory is exhausted.
- `MPOL_BIND`: Allocate only from a specified set of nodes. Fail (or trigger OOM) if those nodes are full.
- `MPOL_INTERLEAVE`: Spread allocations round-robin across a set of nodes. Useful for data structures accessed equally from all CPUs.
- `MPOL_PREFERRED`: Prefer a specific node but fall back to others if necessary.

Example 13.7 (NUMA Topology). A 2-node server has 128 GB RAM per node. Two database processes run on Node 0 and Node 1 respectively.

With NUMA-unaware allocation: each process might allocate half its memory from the remote node, suffering 1.75x latency on 50% of accesses. Average latency: $(0.5 \times 80 + 0.5 \times 140) = 110$ ns.

With NUMA-aware allocation (MPOL_DEFAULT): each process allocates from its local node. Average latency: 80 ns.

Performance improvement: $110/80 = 37.5\%$.

13.4.4 NUMA and Page Migration

When a process is migrated from one CPU to another on a different NUMA node (by the scheduler), its memory remains on the original node. This creates a *NUMA imbalance*: the process accesses remote memory for all its working set. Linux addresses this with *automatic NUMA balancing* (AutoNUMA):

Definition 13.14a (Automatic NUMA Balancing). *AutoNUMA* is a Linux kernel feature that detects NUMA-suboptimal page placements and migrates pages to the node where they are most frequently accessed. It works by:

1. Periodically unmapping pages from the process's page table (setting present=0).
2. When the process accesses an unmapped page, a NUMA hint fault occurs.

3. The kernel records which node caused the fault and compares it to the page's current node.
4. If the page is on a remote node, the kernel migrates it to the local node.

Example 13.7a (NUMA Migration). A process with a 1 GB working set is initially scheduled on Node 0, and all its pages are on Node 0. The scheduler migrates the process to Node 1.

Without AutoNUMA: all 262,144 pages remain on Node 0. Every memory access is remote: latency = 140 ns.

With AutoNUMA: the kernel gradually migrates hot pages to Node 1. After 10 seconds, 90% of the working set has been migrated. Average latency: $0.9 \times 80 + 0.1 \times 140 = 72 + 14 = 86$ ns.

Migration cost: each page migration requires copying 4 KB across the interconnect (~1 μs per page). For 90% of 262,144 pages: $0.9 \times 262{,}144 \times 1\ \mu\text{s} \approx 0.24$ seconds of migration overhead, spread over 10 seconds of runtime.

13.4.5 NUMA Topology Discovery

Applications can discover the NUMA topology programmatically:

```c
#include <numa.h>
#include <stdio.h>

int main(void) {
    if (numa_available() < 0) {
        printf("NUMA not available\n");
        return 1;
    }

    int num_nodes = numa_max_node() + 1;
    printf("NUMA nodes: %d\n", num_nodes);

    for (int i = 0; i < num_nodes; i++) {
        long size = numa_node_size(i, NULL);
        printf("Node %d: %ld MB\n", i, size / (1024 * 1024));

        /* Print distance to other nodes */
        for (int j = 0; j < num_nodes; j++) {
            printf("  Distance to node %d: %d\n",
                   j, numa_distance(i, j));
```

```
        }
    }
    return 0;
}
/* Compile with: gcc -o numa_info numa_info.c -lnuma */
```

Output on a 2-node system:

```
NUMA nodes: 2
Node 0: 65536 MB
  Distance to node 0: 10
  Distance to node 1: 21
Node 1: 65536 MB
  Distance to node 0: 21
  Distance to node 1: 10
```

The distance matrix (10 = local, 21 = remote) shows that remote access is 2.1x the cost of local access. Larger systems (4+ nodes) have asymmetric distances where some nodes are further than others.

The `numactl` command controls NUMA policy for user-space processes:

```
# Run a process on Node 0 with memory allocated from Node 0
numactl --cpunodebind=0 --membind=0 ./my_database

# Interleave memory across all nodes (useful for hash tables)
numactl --interleave=all ./my_application

# Show NUMA topology
numactl --hardware
```

The `numastat` command shows per-node allocation statistics:

```
$ numastat
                           node0           node1
numa_hit                12345678         9876543
numa_miss                   1234            5678
numa_foreign                5678            1234
local_node              12340000         9870000
other_node                  5678            6543
```

A high `numa_miss` count indicates the allocator is frequently falling back to remote nodes — a sign that the local node's memory is under pressure.

Theorem 13.4 (NUMA-Aware Scheduling). For a process with working set W al-

located on node N, the optimal scheduling decision is to run the process on a CPU attached to node N. Migrating the process to a CPU on node N' ($N' \neq N$) converts all memory accesses from local (t_{local}) to remote (t_{remote}), increasing the effective memory latency by a factor equal to the NUMA ratio. The performance degradation is:

$$\Delta = \frac{t_{\text{remote}} - t_{\text{local}}}{t_{\text{local}}} \times \frac{W_{\text{mem}}}{W_{\text{total}}}$$

where $W_{\text{mem}}/W_{\text{total}}$ is the fraction of execution time spent on memory accesses.

Programmer's Perspective

Go's runtime is partially NUMA-aware. The goroutine scheduler (GMP model) pins each P (logical processor) to an OS thread, and the OS thread is typically scheduled on the same CPU. However, Go's memory allocator (`mheap`) does not have explicit NUMA policies — it requests pages from the OS via `mmap`, and the kernel's default NUMA policy (allocate local) applies. For NUMA-intensive Go applications, you can use `numactl` to bind the process to a specific node, or use the `golang.org/x/sys/unix` package to call `set_mempolicy()` directly. Large Go services (databases, caches) running on multi-socket servers should profile with `numastat` and `perf stat -e node-load-misses` to quantify remote access overhead.

13.5 Huge Pages

13.5.1 TLB Pressure Revisited

As established in Chapter 11, the TLB caches virtual-to-physical translations. With standard 4 KB pages, a TLB with 1024 entries covers only 4 MB. For applications with working sets of tens or hundreds of gigabytes, TLB misses dominate performance.

Definition 13.15 (Huge Pages). *Huge pages* are memory pages larger than the standard page size. On x86-64:

- *Large pages:* 2 MB (using the PS bit in the Page Directory entry, eliminating one level of page table walk).
- *Gigantic pages:* 1 GB (using the PS bit in the PDPT entry, eliminating two levels of page table walk).

13.5.2 Explicit Huge Pages (hugetlbfs)

Linux supports explicit huge pages through the `hugetlbfs` pseudo-filesystem. The administrator reserves a pool of huge pages at boot time:

```
# Reserve 1024 huge pages (2 MB each) = 2 GB
echo 1024 > /proc/sys/vm/nr_hugepages

# Or at boot via kernel parameter:
# hugepages=1024
```

Applications allocate huge pages via `mmap` with `MAP_HUGETLB`:

```c
#include <sys/mman.h>
#include <stdio.h>

int main(void) {
    size_t size = 2 * 1024 * 1024;  /* 2 MB = one huge page */
    void *p = mmap(NULL, size,
                   PROT_READ | PROT_WRITE,
                   MAP_PRIVATE | MAP_ANONYMOUS | MAP_HUGETLB,
                   -1, 0);
    if (p == MAP_FAILED) {
        perror("mmap hugetlb");
        return 1;
    }
    printf("Huge page allocated at %p\n", p);
    *(int *)p = 42;
    munmap(p, size);
    return 0;
}
```

13.5.3 Transparent Huge Pages (THP)

Definition 13.16 (Transparent Huge Pages). *Transparent Huge Pages* (THP) is a Linux kernel feature that automatically promotes contiguous regions of 4 KB pages to 2 MB huge pages without application changes. The kernel's `khugepaged` daemon scans for opportunities to coalesce 512 contiguous 4 KB pages into a single 2 MB huge page.

THP operates transparently: the application allocates memory normally, and the kernel promotes eligible regions in the background. THP can be controlled via:

```
# Check THP status
cat /sys/kernel/mm/transparent_hugepage/enabled
# Output: [always] madvise never

# Disable THP (recommended for latency-sensitive databases)
echo never > /sys/kernel/mm/transparent_hugepage/enabled

# Enable only for regions explicitly marked with madvise
echo madvise > /sys/kernel/mm/transparent_hugepage/enabled
```

13.5.4 THP Drawbacks

THP can cause latency spikes due to:

1. **Compaction:** To create a 2 MB contiguous region, the kernel may need to migrate 4 KB pages, causing pauses.
2. **Splitting:** When part of a huge page is freed or swapped, the kernel must split the 2 MB page back into 512 small pages.
3. **Internal fragmentation:** A 2 MB page wastes on average 1 MB per allocation.

Example 13.8 (THP Latency Impact). A Redis instance runs with THP enabled. Under normal operation, p99 latency is 0.5 ms. Periodically, `khugepaged` triggers compaction, causing p99 spikes to 50 ms. After disabling THP:

```
echo never > /sys/kernel/mm/transparent_hugepage/enabled
```

The p99 spikes disappear. Redis's documentation explicitly recommends disabling THP.

13.6 Memory Compression

13.6.1 Motivation

When physical memory is scarce, the traditional solution is to swap pages to disk. But disk I/O (even SSD) is orders of magnitude slower than memory access. Memory compression offers a middle ground: instead of evicting a page to disk, compress it and store it in RAM. Decompression is much faster than disk I/O.

13.6.2 zswap

Definition 13.17 (zswap). *zswap* is a Linux kernel feature that intercepts pages being swapped out and compresses them into a dynamically allocated RAM-based pool. If the pool is full, the least recently used compressed pages are written to the real swap device.

The flow: 1. A page is selected for eviction. 2. Instead of writing to disk, zswap compresses the page (using LZ4, LZO, or zstd). 3. The compressed page is stored in a RAM pool. 4. If the page is faulted back in, zswap decompresses it from RAM (fast). 5. If the RAM pool fills up, the oldest compressed pages are written to the backing swap device.

Theorem 13.5 (Compression Ratio and Effective Memory). If the average compression ratio is r (compressed size / original size, $0 < r < 1$) and the zswap pool size is P bytes, the effective additional memory provided by zswap is:

$$M_{\text{effective}} = \frac{P}{r} - P = P\left(\frac{1}{r} - 1\right)$$

For $r = 0.5$ (2:1 compression) and $P = 4$ GB:

$$M_{\text{effective}} = 4 \times (2 - 1) = 4 \text{ GB}$$

The system effectively gains 4 GB of memory at the cost of CPU time for compression/decompression.

13.6.3 zram

Definition 13.18 (zram). *zram* creates a compressed block device in RAM. Unlike zswap (which acts as a cache in front of real swap), zram *is* the swap device. Pages swapped to zram are compressed and stored entirely in RAM. There is no backing disk swap.

zram is commonly used on memory-constrained devices (Android, ChromeOS, lightweight servers):

```
# Create a 4 GB zram device
modprobe zram
echo 4G > /sys/block/zram0/disksize
mkswap /dev/zram0
swapon /dev/zram0
```

```
# Check compression statistics
cat /sys/block/zram0/mm_stat
# orig_data_size  compr_data_size  mem_used  ...
```

Example 13.9 (zram Compression Savings). A system with 4 GB RAM and 4 GB zram. After compression, 4 GB of swapped pages occupy only 1.6 GB in the zram device (2.5:1 ratio).

Effective total memory: $4 + (4 - 1.6) = 6.4$ GB.

Decompression latency: ~1 μs (LZ4). Compared to SSD swap: ~50 μs. Compared to HDD swap: ~5 ms. zram is 50x faster than SSD swap and 5000x faster than HDD swap.

13.6.4 Compression Algorithms

The choice of compression algorithm for zswap/zram involves a trade-off between compression ratio and speed:

Algorithm	Compression Speed	Decompression Speed	Ratio	CPU Usage
LZO	~500 MB/s	~800 MB/s	2.0:1	Low
LZ4	~700 MB/s	~2000 MB/s	1.8:1	Very low
zstd	~200 MB/s	~600 MB/s	3.0:1	Moderate
deflate	~100 MB/s	~300 MB/s	3.5:1	High

For zram/zswap, decompression speed is more important than compression speed (decompression happens on the critical path of a page fault; compression happens during eviction, which can be batched). LZ4 is the default in most Linux distributions because it has the best decompression speed.

Example 13.9a (Compression Speed Impact). A page fault on a compressed page requires decompression of 4 KB:

- LZ4: $4 \text{ KB}/2000 \text{ MB/s} = 2\ \mu\text{s}$
- zstd: $4 \text{ KB}/600 \text{ MB/s} = 6.7\ \mu\text{s}$
- SSD read: $4 \text{ KB}/3000 \text{ MB/s} + 10\ \mu\text{s}$ latency $\approx 11\ \mu\text{s}$

LZ4 decompression is ~5.5x faster than an SSD read. zstd is still faster than SSD but provides better compression (more pages fit in the zswap pool).

13.6.5 When to Use Memory Compression

Memory compression is most effective when:

1. **Pages are compressible:** Text, structured data, and zero-heavy pages compress well (3:1 or better). Random binary data (encrypted pages, compressed files) compress poorly (1:1 or worse, meaning no benefit).
2. **The system is memory-constrained but not CPU-constrained:** Compression trades CPU for memory. On a server with idle CPU cores and tight memory, this is an excellent trade-off. On a CPU-bound system, compression adds unwanted overhead.
3. **Swap I/O is the bottleneck:** If the system is spending significant time on swap I/O (visible in `vmstat` as high `si`/`so` values), zswap can eliminate or reduce this I/O by keeping compressed pages in RAM.

Theorem 13.5a (Compression Break-Even). Let t_{compress} be the time to compress a page, $t_{\text{decompress}}$ be the time to decompress, t_{io} be the time for a disk I/O (read or write), and r be the compression ratio. Compression is beneficial if:

$$t_{\text{compress}} + t_{\text{decompress}} < t_{\text{io,write}} + t_{\text{io,read}}$$

For LZ4 ($t_{\text{compress}} + t_{\text{decompress}} \approx 8\ \mu$s) and HDD ($t_{\text{io}} \approx 8$ ms per page): $8\ \mu\text{s} \ll 16$ ms. Compression wins by a factor of 2000.

For LZ4 and NVMe SSD ($t_{\text{io}} \approx 20\ \mu$s per page): $8\ \mu\text{s} < 40\ \mu$s. Compression still wins by a factor of 5.

13.7 Address Space Layout Randomisation (ASLR)

13.7.1 The Security Problem

Buffer overflow attacks rely on knowing the addresses of key data structures: the stack (for return address overwrites), the heap (for object corruption), and shared libraries (for return-to-libc attacks). If these addresses are predictable, the attacker can craft exploits that reliably redirect control flow.

13.7.2 ASLR Mechanism

Definition 13.19 (ASLR). *Address Space Layout Randomisation* randomises the base addresses of key memory regions each time a process is loaded:

- Stack base: randomised within a range.
- Heap base (brk): randomised.
- mmap base (for shared libraries and anonymous mappings): randomised.
- Executable base (with PIE — Position-Independent Executable): randomised.
- VDSO (virtual dynamic shared object): randomised.

With ASLR, the same program loaded twice will have its stack, heap, and libraries at different addresses. An attacker who discovers the address layout of one process invocation cannot reuse that knowledge for another.

13.7.3 Entropy

Definition 13.20 (ASLR Entropy). The *entropy* of ASLR is the number of bits of randomness in the base address. For k bits of entropy, there are 2^k possible base addresses, and an attacker has a $1/2^k$ probability of guessing the correct address.

On x86-64 Linux, typical ASLR entropy:

Region	Entropy (bits)	Possible Addresses
Stack	30	$\sim 10^9$
mmap base	28	$\sim 2.7 \times 10^8$
Heap (brk)	13	~8,000
PIE executable	28	$\sim 2.7 \times 10^8$

Theorem 13.6 (Brute-Force Attack Complexity). With k bits of ASLR entropy, a brute-force attack (guessing the randomised address) succeeds with probability $1/2^k$ per attempt. The expected number of attempts is 2^k. If each failed attempt crashes the process (and the process is restarted with a new random layout), the expected time to succeed is:

$$E[T] = 2^k \times (t_{\text{crash}} + t_{\text{restart}})$$

For $k = 28$ and $t_{\text{crash}} + t_{\text{restart}} = 1$ second:

$$E[T] = 2^{28} \approx 2.7 \times 10^8 \text{ seconds} \approx 8.5 \text{ years}$$

13.7.4 Checking ASLR

```c
#include <stdio.h>
#include <stdlib.h>

int global_var = 42;

int main(void) {
    int stack_var = 7;
    int *heap_var = malloc(sizeof(int));

    printf("main():     %p\n", (void *)main);
    printf("global:     %p\n", (void *)&global_var);
    printf("stack:      %p\n", (void *)&stack_var);
    printf("heap:       %p\n", (void *)heap_var);
    printf("printf:     %p\n", (void *)printf);

    free(heap_var);
    return 0;
}
```

Running this program twice (compiled as PIE: `gcc -pie -o aslr_demo aslr_demo.c`):

```
$ ./aslr_demo
main():     0x5602a3c00149
global:     0x5602a3e01010
stack:      0x7ffd8a3e1c5c
heap:       0x5602a51002a0
printf:     0x7f4a38201a40

$ ./aslr_demo
main():     0x55a7e1200149
global:     0x55a7e1401010
stack:      0x7ffc12bf3c5c
heap:       0x55a7e22002a0
printf:     0x7f9c42801a40
```

Every address changes between runs. Without ASLR (or with `setarch -R`), the addresses would be identical.

13.7.5 ASLR Limitations

ASLR is not a complete defence:

- **Information leaks:** If the attacker can read a process's memory (via a format string vulnerability, side channel, or `/proc/PID/maps`), they can discover the randomised layout.
- **Low entropy:** On 32-bit systems, ASLR provides only ~8–16 bits of entropy, making brute-force feasible.
- **Non-PIE executables:** If the executable itself is not position-independent, its text segment is at a fixed address, providing known gadgets for ROP attacks.
- **Heap spraying:** By filling the heap with many copies of the payload, the attacker increases the probability that a random jump lands on valid exploit code.

Note

ASLR is one layer in a defence-in-depth strategy. It is typically combined with stack canaries, NX (no-execute) pages, Control-Flow Integrity (CFI), and shadow stacks.

13.8 Go's Memory Allocator

Programmer's Perspective: Go's Memory Allocator (mcache/mcentral/mheap).

Go's memory allocator is a descendant of Google's TCMalloc (Thread-Caching Malloc), adapted for Go's goroutine-based concurrency model and integrated with the garbage collector. Understanding its architecture illuminates many of the concepts from this chapter.

Three-level hierarchy:

```
// Conceptual structure (simplified from runtime/mheap.go)
//
// mcache (per-P, lock-free):
//   - One per logical processor (P in GMP model)
//   - Contains alloc[67] -- one mspan per size class
//   - Tiny allocator for objects <= 16 bytes with no pointers
//
// mcentral (per-size-class, locked):
//   - One per size class (67 classes)
```

```
//   - Manages partial and full spans
//   - mcache refills from mcentral when its span is exhausted
//
// mheap (global, locked):
//   - Manages the entire heap address space
//   - Allocates spans (contiguous runs of 8 KB pages)
//   - Backed by OS memory via mmap(MAP_ANONYMOUS)
```

Size classes: Go uses 67 size classes, from 8 bytes to 32,768 bytes. Each class is tuned to minimise internal fragmentation:

Class	Object Size	Objects per Span	Tail Waste
1	8 B	1024	0 B
2	16 B	512	0 B
5	48 B	170	32 B
10	128 B	64	0 B
30	1024 B	8	0 B
67	32768 B	1	0 B

Objects larger than 32 KB are allocated directly from the `mheap` as *large spans*, bypassing the size class mechanism entirely.

Allocation path for a 100-byte object:

1. Round up to size class 10 (128 bytes).
2. Check the current P's `mcache` for a span with free slots in class 10.
3. If found, return the next free slot (no lock needed — per-P).
4. If the `mcache` span is full, obtain a new partial span from `mcentral[10]` (requires lock).
5. If `mcentral` has no partial spans, allocate a new span from `mheap` (global lock).
6. If `mheap` has no free pages, call `mmap` to get more memory from the OS.

TCMalloc heritage: The per-P cache is the Go equivalent of TCMalloc's per-thread cache. The key difference: Go uses per-P caches (one per GOMAXPROCS) rather than per-thread caches. Since the number of Ps is typically small (equal to the number of CPU cores), the total cache memory is bounded. This is more efficient than TCMalloc's per-thread approach when there are thousands of threads.

Integration with GC: The allocator is tightly coupled with Go's concurrent, tri-colour

mark-and-sweep garbage collector. Each allocation checks whether a GC cycle should be triggered (based on `GOGC` / `GOMEMLIMIT`). The allocator marks newly allocated objects as white (unmarked), and the GC concurrently scans and marks reachable objects. Sweeping reclaims entire spans whose objects are all unmarked.

Key runtime statistics (inspectable via `runtime.ReadMemStats`):

```go
package main

import (
    "fmt"
    "runtime"
)

func main() {
    // Allocate some objects
    slices := make([][]byte, 1000)
    for i := range slices {
        slices[i] = make([]byte, 1024)
    }

    var m runtime.MemStats
    runtime.ReadMemStats(&m)

    fmt.Printf("Alloc:       %6d KB  (live heap objects)\n", m.Alloc/1024)
    fmt.Printf("TotalAlloc:  %6d KB  (cumulative)\n", m.TotalAlloc/1024)
    fmt.Printf("Sys:         %6d KB  (total OS memory)\n", m.Sys/1024)
    fmt.Printf("HeapObjects: %6d     (live objects)\n", m.HeapObjects)
    fmt.Printf("HeapInuse:   %6d KB  (spans with objects)\n", m.HeapInuse/1024)
    fmt.Printf("HeapIdle:    %6d KB  (spans without objects)\n", m.HeapIdle/1024)
    fmt.Printf("HeapSys:     %6d KB  (heap virtual memory)\n", m.HeapSys/1024)
    fmt.Printf("NumGC:       %6d     (completed GC cycles)\n", m.NumGC)

    _ = slices // keep alive
}
```

The gap between `HeapSys` (virtual memory reserved) and `HeapInuse` (physical memory backing live objects) directly reflects the concepts from this chapter: virtual memory is cheap (it is just page table entries pointing to the zero page), while physical memory is precious (it requires actual DRAM frames). Go reserves virtual address space aggressively but lets the OS manage physical frame allocation through demand paging

and `madvise`.

13.9 Summary

This chapter has covered seven advanced topics that extend the basic virtual memory framework:

- **Copy-on-write** defers page copying until a write occurs, making `fork()` nearly instantaneous and enabling demand zeroing for anonymous memory.
- **Memory-mapped files** unify file I/O and memory access, eliminating buffer copies and leveraging the page cache for high-performance data access.
- **Kernel memory allocators** — the buddy system provides contiguous page blocks; the slab/SLUB allocator eliminates fragmentation for fixed-size kernel objects; `kmalloc` provides general-purpose kernel allocation via size classes.
- **NUMA** introduces topology-aware allocation, where the OS preferentially allocates memory from the node closest to the requesting CPU to minimise access latency.
- **Huge pages** (explicit and transparent) reduce TLB pressure by covering more virtual address space per TLB entry, with trade-offs in internal fragmentation and latency variability.
- **Memory compression** (zswap, zram) provides a middle layer between RAM and disk swap, compressing evicted pages in RAM to defer costly disk I/O.
- **ASLR** randomises the layout of a process's address space to frustrate memory-corruption exploits, providing probabilistic security proportional to the entropy of the randomisation.

Together with the fundamentals of Chapters 10–12, these topics form a complete picture of how modern operating systems manage memory — from the hardware page tables and TLB, through the kernel's allocation and reclaim algorithms, to the user-space view presented to applications.

Exercise 13.1. A process with 2000 pages calls `fork()`. The child then: (a) calls `exec()` immediately (writing only 5 stack pages before `exec`), (b) modifies 200 pages before calling `exec()`, (c) never calls `exec()` and eventually modifies all 2000 pages. For each scenario, calculate the total number of page copies made with CoW and the peak physical memory usage (in pages) for the parent + child. Compare to the naive (full-copy) `fork()`.

Exercise 13.2. Write a C program that maps a 1 MB file using `mmap()` with

`MAP_SHARED`, modifies the first 100 bytes, calls `msync()` with `MS_SYNC`, and then reads the file using standard `read()` to verify the modification persists. Explain why `msync()` is necessary and what could happen without it.

Exercise 13.3. A buddy system manages 1 MB of memory (minimum block size: 1 KB, maximum: 1 MB). Trace the state of the free lists after the following sequence: (a) allocate A = 100 KB, (b) allocate B = 240 KB, (c) allocate C = 60 KB, (d) free B, (e) free A, (f) allocate D = 130 KB. For each step, show all free blocks and their sizes. Calculate the total internal fragmentation at each step.

Exercise 13.4. A NUMA system has 2 nodes, each with 64 GB of RAM. Local access latency: 80 ns. Remote access latency: 150 ns. A process has a 50 GB working set. Under three scenarios, calculate the average memory access latency: (a) all memory allocated on the local node (possible since 50 GB < 64 GB), (b) memory allocated 50/50 across both nodes with uniform access pattern, (c) memory interleaved at page granularity (alternating pages between nodes). Which policy is best, and under what conditions might interleaving outperform local allocation?

Exercise 13.5. A system has a TLB with 512 entries. A database application has a working set of 8 GB. Calculate the TLB miss rate (assuming uniform random access within the working set) for: (a) 4 KB pages, (b) 2 MB pages, (c) 1 GB pages. For each case, compute the effective access time given $t_{\text{mem}} = 100$ ns, TLB hit time = 1 ns, and a 4-level page walk on miss (each level costs one memory access). Assume the page walk is reduced by one level for 2 MB pages and by two levels for 1 GB pages.

Exercise 13.6. Explain why database systems (PostgreSQL, MySQL) often recommend disabling Transparent Huge Pages. Your answer should address: (a) the mechanism by which THP causes latency spikes, (b) why the default THP promotion/splitting behaviour conflicts with database memory access patterns, and (c) what alternatives databases use to gain the TLB benefits of huge pages without the latency drawbacks.

Exercise 13.7. A 64-bit Linux system has ASLR enabled with the following entropy: stack = 30 bits, mmap base = 28 bits, heap = 13 bits, PIE text = 28 bits. (a) An attacker discovers a heap address via an information leak. How many bits of the overall layout remain unknown? (b) If the attacker can make 1000 guesses per second (each failed guess crashes and restarts the process), how long would a brute-force attack on the stack base take on average? (c) Why does 32-bit ASLR provide significantly less security than 64-bit ASLR? Calculate the brute-force time for a 32-bit system with 8 bits of stack entropy at the same guess rate.

Chapter 14: File Systems

"A file system is not just a way to store data — it is a contract between the operating system and the user, promising that bits entrusted to storage today will be faithfully returned tomorrow." — Marshall Kirk McKusick, architect of the Berkeley Fast File System

---

14.1 The File Abstraction

Every modern operating system provides a **file** as the fundamental unit of persistent storage. At the hardware level, storage devices deal in sectors, blocks, and flash pages. The file abstraction hides this complexity, presenting users and applications with a uniform, named container for data.

File. A file is a named collection of related information recorded on secondary storage. From the operating system's perspective, a file is the smallest allotment of logical secondary storage — data cannot be written to secondary storage unless they are within a file.

14.1.1 File Attributes

Every file carries metadata that describes its properties beyond the raw data content. The specific attributes vary across file systems, but a common set includes:

Attribute	Description
Name	Human-readable symbolic name
Identifier	Unique numeric tag (inode number in Unix)
Type	Regular file, directory, symbolic link, device, socket, pipe
Location	Pointer to the data blocks on the storage device
Size	Current size in bytes (and possibly in blocks)

Attribute	Description
Protection	Access control information (owner, group, permissions, ACLs)
Timestamps	Creation time, last modification time, last access time
Link count	Number of hard links referencing this file

On Unix-like systems, the **inode** stores all attributes except the file name. The name-to-inode mapping is maintained by the directory that contains the file. This separation is fundamental: it allows multiple names (hard links) to reference the same underlying file.

Example 14.1 (Inode Metadata on Linux). On an ext4 file system, each inode is a 256-byte structure containing the file's mode, owner UID, group GID, size, timestamps (atime, ctime, mtime, crtime), link count, block count, and pointers to data blocks. The `stat` system call retrieves this information:

```c
#include <sys/stat.h>
#include <stdio.h>
#include <time.h>

int main(void) {
    struct stat sb;
    if (stat("/etc/passwd", &sb) == -1) {
        perror("stat");
        return 1;
    }
    printf("Inode:       %lu\n", (unsigned long)sb.st_ino);
    printf("Size:        %lld bytes\n", (long long)sb.st_size);
    printf("Blocks:      %lld\n", (long long)sb.st_blocks);
    printf("Links:       %lu\n", (unsigned long)sb.st_nlink);
    printf("Permissions: %o\n", sb.st_mode & 07777);
    printf("Modified:    %s", ctime(&sb.st_mtime));
    return 0;
}
```

14.1.2 File Operations

The operating system provides a set of system calls for manipulating files. These operations form the **file API** — the contract between applications and the file system.

The fundamental file operations are:

1. **Create:** Allocate space in the file system for a new file, create a directory entry, and initialise metadata.
2. **Open:** Search the directory structure, verify permissions, load metadata into an in-memory structure (the open file table entry), and return a file descriptor.
3. **Read:** Transfer data from the file into a user-space buffer. The current file-position pointer determines where reading begins.
4. **Write:** Transfer data from a user-space buffer to the file at the current file-position pointer. The file may grow if writing past the current end.
5. **Seek (Reposition):** Adjust the file-position pointer without performing any I/O. Only meaningful for direct-access files.
6. **Close:** Release the open file table entry, flush any buffered writes, and update metadata (such as access time).
7. **Delete (Unlink):** Remove a directory entry. If the link count drops to zero and no process holds the file open, the file's data blocks and inode are freed.
8. **Truncate:** Release the data blocks of a file while keeping its attributes intact, resetting the file size to zero.

Open File Table. The operating system maintains two levels of open file tables. The **system-wide open file table** contains one entry per open file, holding the inode information, current file size, and access counts. The **per-process open file table** contains one entry per file descriptor in the process, holding the current file-position pointer and a reference to the system-wide entry.

When a process calls `open()`, the kernel searches for an existing entry in the system-wide table. If found, it creates a new per-process entry pointing to it and increments the reference count. This is why two processes that open the same file independently have separate file-position pointers but share the underlying inode data.

14.1.3 File Access Methods

The manner in which data within a file is accessed defines its **access method**. The three fundamental access methods are sequential, direct, and indexed.

Sequential Access. This is the simplest and most common access method. Data is read or written in order, one record after another. The file-position pointer advances automatically after each operation. Rewinding is possible, but random jumps are not part of the basic model.

$$\text{read_next}() \to \text{record}_i, \quad \text{position} \leftarrow \text{position} + 1$$

Sequential access is natural for text files, log files, and streaming data. Magnetic tape, the original storage medium, supported only sequential access.

Direct (Random) Access. The file is viewed as a numbered sequence of fixed-length blocks. Any block can be read or written independently by specifying its block number. The file-position pointer can be set to any position before performing I/O.

$$\text{read}(n) \to \text{block}_n, \quad \text{position} \leftarrow n$$

Direct access is essential for databases, where individual records must be retrieved without scanning the entire file.

Indexed Access. An index structure maps keys to block positions within the file. To find a record, the system first searches the index, then uses direct access to fetch the target block. The index itself may be organised as a B-tree, hash table, or multi-level index.

Example 14.2 (Access Method Comparison). Consider a file of 1,000,000 records. To find a specific record:

- **Sequential access:** On average, scan 500,000 records. Time complexity: $O(n)$.
- **Direct access:** If the record number is known, access in $O(1)$ time (one disk seek plus one block read).
- **Indexed access (B-tree):** With a branching factor of 100, the index has depth $\lceil \log_{100}(1{,}000{,}000) \rceil = 3$. Three index lookups plus one data block read: $O(\log_B n)$ where B is the branching factor.

14.2 Directory Structure

A **directory** is a special file that contains a mapping from human-readable file names to file metadata (or, more precisely, to inodes). The organisation of directories pro-

foundly affects how users navigate the file system and how the system resolves path names.

14.2.1 Single-Level Directory

The simplest directory structure is a single flat list of all files in the system. Every file must have a unique name.

```
Single-Level Directory
+------+------+------+------+------+------+------+
| cat  | mail | prog | data | test | lib  | memo |
+------+------+------+------+------+------+------+
```

This approach was used in early systems (CP/M). It is impractical for modern use because:

- Name collisions become inevitable as the number of files grows.
- No logical grouping is possible — a user's documents, system binaries, and temporary files all share one namespace.
- Multi-user systems are impossible without some form of isolation.

14.2.2 Two-Level Directory

A two-level directory adds one level of hierarchy: each user has their own private directory, all of which sit beneath a **master file directory** (MFD).

```
Master File Directory
+----------+----------+----------+
| user_a   | user_b   | user_c   |
+----+-----+----+-----+----+-----+
     |          |          |
 +---+---+  +--+---+  +---+---+
 |cat|prog|  |data|cat|  |test|lib|
 +---+----+  +---+----+  +---+---+
```

This solves the multi-user isolation problem: user A and user B can both have a file named `cat` without collision. However, it provides no grouping within a user's files, and sharing files between users requires special mechanisms (such as path names that include the user name).

14.2.3 Tree-Structured Directory

Modern file systems use a **tree** (or hierarchical) directory structure. Each directory can contain both files and subdirectories, creating an arbitrarily deep hierarchy.

```
/
+-- bin/
|   +-- ls
|   +-- cat
+-- home/
|   +-- alice/
|   |   +-- documents/
|   |   |   +-- report.tex
|   |   +-- code/
|   |       +-- main.go
|   +-- bob/
|       +-- notes.txt
+-- etc/
    +-- passwd
    +-- fstab
```

Every process maintains a **current working directory**, enabling the use of relative path names. An **absolute path** begins at the root `/` and specifies the complete path through the tree.

Path Name Resolution. Given a path `/home/alice/code/main.go`, the kernel resolves it as follows: 1. Start at the root directory inode (inode 2 on ext4). 2. Search the root directory for the entry `home`, obtaining its inode number. 3. Read the `home` directory and search for `alice`. 4. Read the `alice` directory and search for `code`. 5. Read the `code` directory and search for `main.go`. 6. Return the inode of `main.go`.

Each step requires reading a directory from disk (unless cached), making deep paths more expensive to resolve.

14.2.4 Acyclic-Graph Directory

A **tree** forbids sharing: a file can exist in exactly one directory. An **acyclic-graph** directory structure allows a file or subdirectory to appear in multiple directories through **hard links** or **symbolic (soft) links**.

Hard Link. A hard link is an additional directory entry that maps a (possibly different) name to the same inode. Hard links cannot span file system boundaries (because inodes are local to a file system), and they cannot point to directories (to prevent

cycles in the graph).

Symbolic Link. A symbolic link (symlink) is a special file whose content is the path name of another file. When the kernel encounters a symlink during path resolution, it substitutes the symlink's content and continues resolution from that path. Symlinks can cross file system boundaries and can point to directories.

```c
#include <unistd.h>
#include <stdio.h>

int main(void) {
    /* Create a hard link: "backup.txt" points to the same inode as "data.txt" */
    if (link("data.txt", "backup.txt") == -1) {
        perror("link");
        return 1;
    }

    /* Create a symbolic link: "shortcut" contains the string "/home/alice/data.txt" */
    if (symlink("/home/alice/data.txt", "shortcut") == -1) {
        perror("symlink");
        return 1;
    }

    printf("Hard link and symbolic link created.\n");
    return 0;
}
```

The key difference is visible upon deletion. If the original file `data.txt` is deleted:

- The hard link `backup.txt` still works — it references the same inode, and the inode is not freed until all hard links are removed (link count reaches zero).
- The symbolic link `shortcut` becomes a **dangling symlink** — the path it contains no longer resolves to a valid file.

14.2.5 General Graph Directory

If we allow arbitrary links, including cycles, the directory structure becomes a general graph. Cycles create serious problems:

- **Infinite traversal:** A recursive directory listing (`find /`, `ls -R`) may loop forever.
- **Deletion complexity:** When can storage be reclaimed if there are circular

references?

To handle cycles, file systems may use:

- **Garbage collection:** Traverse all reachable files from the root; anything unreachable is freed. This is expensive and rarely used.
- **Cycle prevention:** Prohibit hard links to directories (as Linux does). Symlinks can technically create apparent cycles, but since symlinks are resolved at access time, they do not create true reference cycles in the inode graph.
- **Reference counting with cycle detection:** Maintain link counts but supplement with periodic cycle checks.

14.3 File System Mounting and VFS

14.3.1 Mounting

A storage device contains a **file system** — a self-contained structure with its own root directory, free-space metadata, and allocation tables. Before the files on a device can be accessed, the file system must be **mounted** onto the existing directory tree at a designated **mount point**.

Mount Point. A mount point is an existing directory in the file system hierarchy where a new file system is attached. After mounting, accessing the mount point directory transparently redirects to the root of the mounted file system.

```
Before mounting /dev/sdb1 on /mnt/usb:
/
+-- mnt/
|   +-- usb/        (empty directory)
+-- home/

After mounting:
/
+-- mnt/
|   +-- usb/        (now shows contents of /dev/sdb1)
|       +-- photos/
|       +-- music/
+-- home/
```

On Linux, the `mount` system call attaches a file system:

```c
#include <sys/mount.h>

/* Mount an ext4 file system from /dev/sdb1 onto /mnt/usb */
int ret = mount("/dev/sdb1", "/mnt/usb", "ext4", MS_RDONLY, NULL);
```

The kernel maintains a **mount table** that maps mount points to the corresponding superblock structures. During path name resolution, whenever the kernel crosses a mount point, it switches to the mounted file system's root directory.

14.3.2 The Virtual File System (VFS) Layer

A modern operating system must support multiple file system types simultaneously: ext4 for the root partition, FAT32 for a USB drive, NFS for a network share, procfs for process information. The **Virtual File System** (VFS) layer provides a uniform interface that abstracts the differences between file system implementations.

Virtual File System (VFS). The VFS is a kernel subsystem that provides a common interface for file system operations. It defines a set of abstract operations (open, read, write, lookup, mkdir, etc.) that every concrete file system must implement. User-space applications interact only with the VFS; the VFS dispatches calls to the appropriate file system driver.

The VFS defines four principal object types:

1. **Superblock object:** Represents a mounted file system. Contains metadata about the file system (block size, maximum file size, file system type) and a pointer to the root inode.
2. **Inode object:** Represents a file (in the broad sense: regular file, directory, symlink, device). Contains all metadata except the file name.
3. **Dentry (directory entry) object:** Represents a single component of a path. The dentry cache (dcache) accelerates path name resolution by caching recently resolved name-to-inode mappings.
4. **File object:** Represents an open file. Contains the file-position pointer and a pointer to the file operations structure.

Each object type has an associated **operations structure** — a table of function pointers that the concrete file system fills in:

```c
/* Simplified VFS inode operations (Linux kernel) */
struct inode_operations {
    struct dentry * (*lookup)(struct inode *, struct dentry *, unsigned int);
    int (*create)(struct inode *, struct dentry *, umode_t, bool);
```

```
    int (*link)(struct dentry *, struct inode *, struct dentry *);
    int (*unlink)(struct inode *, struct dentry *);
    int (*symlink)(struct inode *, struct dentry *, const char *);
    int (*mkdir)(struct inode *, struct dentry *, umode_t);
    int (*rmdir)(struct inode *, struct dentry *);
    int (*rename)(struct inode *, struct dentry *,
                  struct inode *, struct dentry *, unsigned int);
    /* ... many more operations ... */
};

/* Simplified VFS file operations */
struct file_operations {
    ssize_t (*read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char __user *, size_t, loff_t *);
    loff_t  (*llseek)(struct file *, loff_t, int);
    int     (*open)(struct inode *, struct file *);
    int     (*release)(struct inode *, struct file *);
    int     (*mmap)(struct file *, struct vm_area_struct *);
    long    (*ioctl)(struct file *, unsigned int, unsigned long);
    /* ... */
};
```

When ext4 registers with the VFS, it provides its own implementations of these function pointers:

```
const struct inode_operations ext4_dir_inode_operations = {
    .create     = ext4_create,
    .lookup     = ext4_lookup,
    .link       = ext4_link,
    .unlink     = ext4_unlink,
    .symlink    = ext4_symlink,
    .mkdir      = ext4_mkdir,
    .rmdir      = ext4_rmdir,
    .rename     = ext4_rename2,
    /* ... */
};
```

This is the **strategy pattern** applied at the operating system level: the VFS defines the interface, and each file system provides its own strategy.

Programmer's Perspective: VFS and Go's `io/fs` Package. The Linux VFS architecture has a direct parallel in Go's `io/fs` package (introduced in Go 1.16). The `fs.FS`

interface defines the minimal contract for a file system:

```go
package fs

type FS interface {
    Open(name string) (File, error)
}

type File interface {
    Stat() (FileInfo, error)
    Read([]byte) (int, error)
    Close() error
}
```

Just as the Linux VFS allows any file system to plug in by implementing the `inode_operations` and `file_operations` structures, Go allows any type to act as a file system by implementing `fs.FS`. The `os.DirFS()` function wraps a real directory, `embed.FS` serves embedded files, and `testing/fstest.MapFS` provides an in-memory file system for tests — all through the same interface.

The `fs.WalkDir()` function traverses any `fs.FS` implementation, just as the kernel's `path_walk()` traverses any VFS-compliant file system. When writing Go applications, designing around `fs.FS` instead of direct `os` calls makes code testable (swap in `MapFS` for tests) and portable (swap `os.DirFS` for `embed.FS` when deploying as a single binary).

```go
package main

import (
    "fmt"
    "io/fs"
    "os"
)

func countFiles(fsys fs.FS) (int, error) {
    count := 0
    err := fs.WalkDir(fsys, ".", func(path string, d fs.DirEntry, err error) error {
        if err != nil {
            return err
        }
        if !d.IsDir() {
            count++
```

```
        }
        return nil
    })
    return count, err
}

func main() {
    fsys := os.DirFS("/etc")
    n, err := countFiles(fsys)
    if err != nil {
        fmt.Fprintf(os.Stderr, "error: %v\n", err)
        os.Exit(1)
    }
    fmt.Printf("Files under /etc: %d\n", n)
}
```

14.4 Allocation Methods

When a file is created and data is written to it, the file system must decide which disk blocks to allocate. The allocation strategy profoundly affects both performance (sequential read speed, random access time, fragmentation) and the complexity of the metadata structures.

14.4.1 Contiguous Allocation

In contiguous allocation, each file occupies a set of **contiguous blocks** on the disk. The directory entry records the starting block and the length (number of blocks).

Contiguous Allocation. A file of n blocks is stored in blocks $b, b+1, b+2, \ldots, b+n-1$. The directory entry stores the pair (b, n).

Advantages:

- Sequential access is maximally efficient: the disk head (on a rotational drive) makes no seeks.
- Direct access is trivial: block i of the file is at disk block $b + i$.
- Minimal metadata: only two numbers per file.

Disadvantages:

- **External fragmentation:** Over time, free space becomes fragmented into small holes that cannot accommodate new files. This is the same problem that plagues contiguous memory allocation.
- **File growth is difficult:** If the blocks after a file are occupied, the file cannot grow in place and must be relocated — an expensive operation.
- **Pre-allocation requires knowing the final size:** The user must declare the file size at creation time, or the system must over-allocate.

The fragmentation problem is quantified by:

$$\text{Fragmentation ratio} = 1 - \frac{\text{largest free extent}}{\text{total free space}}$$

A ratio near 1 indicates severe fragmentation: plenty of total free space exists, but no single contiguous region is large enough.

Example 14.3 (Contiguous Allocation Fragmentation). A disk has 100 blocks. Files A (10 blocks), B (20 blocks), C (15 blocks) are allocated contiguously. Then B is deleted, freeing blocks 10–29. Now the free space is: blocks 10–29 (20 blocks) and blocks 45–99 (55 blocks). Total free space: 75 blocks. Largest free extent: 55 blocks.

$$\text{Fragmentation ratio} = 1 - \frac{55}{75} = 1 - 0.733 = 0.267$$

If we then need to allocate a file of 60 blocks, neither free extent is large enough, even though 75 blocks are free in total.

Contiguous allocation is still used in limited contexts: CD-ROM file systems (ISO 9660) use it because the media is write-once and the file sizes are known at mastering time. Some real-time systems use contiguous extents for predictable access times.

14.4.2 Linked Allocation

In linked allocation, each file is a **linked list of disk blocks**. Each block contains a pointer to the next block in the chain. The directory entry stores the pointer to the first block.

Linked Allocation. Each disk block contains data and a pointer to the next block. The directory entry stores the first and last block pointers. The last block's pointer is `NULL` (or a sentinel value).

Advantages:

- No external fragmentation: any free block can be used, regardless of its position.
- Files can grow dynamically by appending new blocks to the chain.
- No need to pre-declare file sizes.

Disadvantages:

- **Sequential access only:** To read block i, the system must follow i pointers from the start of the chain. Direct access requires $O(n)$ time.
- **Pointer overhead:** Each block loses space to the pointer. With 512-byte blocks and 4-byte pointers, 0.78% of space is wasted. With 4 KB blocks, the overhead drops to 0.1%.
- **Fragility:** If a single pointer is corrupted (due to a bad sector or software bug), the remainder of the file is lost.
- **Poor locality:** Blocks may be scattered across the disk, causing excessive seek operations.

The File Allocation Table (FAT). MS-DOS and its successors (Windows 95/98) improved upon basic linked allocation by moving all the pointers into a separate table: the **File Allocation Table**. Instead of embedding the next-block pointer in each data block, the FAT stores one entry per block. Entry i contains the number of the block that follows block i in the chain.

```
FAT (File Allocation Table)
Block:  0    1    2    3    4    5    6    7    8    9
Entry: [3]  [7]  [-]  [4]  [6]  [-]  [EOF] [EOF] [-]  [-]

File A starts at block 0: 0 -> 3 -> 4 -> 6 -> EOF
File B starts at block 1: 1 -> 7 -> EOF
```

The FAT is small enough to cache entirely in memory, which restores $O(1)$ direct access (search the FAT chain in memory without disk I/O). However, the FAT itself becomes a critical structure: if it is corrupted, the entire file system is lost. FAT file systems therefore maintain two copies of the FAT.

14.4.3 Indexed Allocation (The Inode)

Indexed allocation solves linked allocation's direct access problem by gathering all block pointers into a single **index block** (or **inode**) for each file.

Indexed Allocation. Each file has an index block that contains an array of disk block

pointers. The i-th entry in the index block points to the i-th data block of the file. The directory entry points to the index block.

For small files, a single index block suffices. For large files, the index block would need to be very large. Unix file systems solve this with a **multi-level index** scheme:

- **Direct pointers:** The inode contains 12 direct pointers, each pointing to a data block. For a 4 KB block size, this covers files up to $12 \times 4\,\text{KB} = 48\,\text{KB}$.
- **Single indirect pointer:** Points to a block that itself contains pointers to data blocks. With 4-byte pointers and 4 KB blocks, one indirect block holds $4096/4 = 1024$ pointers, covering an additional $1024 \times 4\,\text{KB} = 4\,\text{MB}$.
- **Double indirect pointer:** Points to a block of single-indirect pointers. Coverage: $1024^2 \times 4\,\text{KB} = 4\,\text{GB}$.
- **Triple indirect pointer:** Points to a block of double-indirect pointers. Coverage: $1024^3 \times 4\,\text{KB} = 4\,\text{TB}$.

Theorem 14.1 (Maximum File Size with Multi-Level Indexing). Given a block size of B bytes and pointer size of P bytes, the maximum file size with the Unix inode scheme (12 direct, 1 single-indirect, 1 double-indirect, 1 triple-indirect) is:

$$S_{\text{max}} = \left(12 + \frac{B}{P} + \left(\frac{B}{P}\right)^2 + \left(\frac{B}{P}\right)^3\right) \times B$$

For $B = 4096$ and $P = 4$: $S_{\text{max}} = (12 + 1024 + 1024^2 + 1024^3) \times 4096 \approx 4\,\text{TB}$.

For $B = 4096$ and $P = 8$ (64-bit pointers): $S_{\text{max}} = (12 + 512 + 512^2 + 512^3) \times 4096 \approx 512\,\text{GB}$.

Example 14.4 (Block Access Cost). To read byte 50,000 of a file on a 4 KB block system with 4-byte pointers:

1. Byte 50,000 is in logical block $\lfloor 50000/4096 \rfloor = 12$.
2. Block 12 is beyond the 12 direct pointers (blocks 0–11), so it is the first block addressed by the single-indirect pointer.
3. The kernel reads the inode (1 disk access), then the indirect block (1 disk access), then the data block (1 disk access) — 3 accesses total.

For a byte at position $5{,}000{,}000$ (logical block 1220), which falls in the double-indirect range: the kernel reads the inode, the double-indirect block, the appropriate single-indirect block, and the data block — 4 accesses total.

Extent-Based Allocation. Modern file systems like ext4, XFS, and Btrfs use **extents** rather than individual block pointers. An extent is described by a triple: (logical block, physical block, length). A 100 MB contiguous file requires a single extent descriptor instead of 25,600 individual block pointers (with 4 KB blocks).

```
Ext4 Extent:
+-----------------+--------------------+--------+
| Logical block: 0 | Physical block: 500 | Len: 25600 |
+-----------------+--------------------+--------+
   => Blocks 0-25599 of the file map to disk blocks 500-26099
```

Ext4 uses an extent tree (a B-tree variant) stored in the inode for files with many extents, and stores up to four extents directly in the inode for small files.

14.5 Free-Space Management

The file system must track which blocks are free and which are allocated. The choice of free-space management method affects allocation speed, fragmentation, and the overhead of metadata storage.

14.5.1 Bitmap (Bit Vector)

A **bitmap** uses one bit per block: 1 for free, 0 for allocated (or vice versa). The bitmap for a disk with n blocks requires $\lceil n/8 \rceil$ bytes.

Free-Space Bitmap. A bit vector $B[0 \dots n-1]$ where $B[i] = 1$ if block i is free and $B[i] = 0$ if block i is allocated.

Example 14.5 (Bitmap Size). A 1 TB disk with 4 KB blocks has $1{,}099{,}511{,}627{,}776/4096 = 268{,}435{,}456$ blocks. The bitmap requires $268{,}435{,}456/8 = 33{,}554{,}432$ bytes $= 32$ MB. This is 0.003% of the disk capacity — a very modest overhead.

Finding free blocks is efficient with a bitmap. To find the first free block, scan the bitmap for the first word that is not all zeros, then use bit manipulation (e.g., `__builtin_ctz` or `ffs`) to find the first set bit:

```c
#include <stdint.h>
#include <string.h>

#define BLOCKS_PER_GROUP 32768
#define WORDS (BLOCKS_PER_GROUP / 64)
```

```c
/* Find the first free block in a bitmap group */
int find_free_block(uint64_t bitmap[WORDS]) {
    for (int i = 0; i < WORDS; i++) {
        if (bitmap[i] != 0) {
            int bit = __builtin_ctzll(bitmap[i]); /* count trailing zeros */
            return i * 64 + bit;
        }
    }
    return -1; /* no free block */
}

/* Allocate a block (clear the bit) */
void alloc_block(uint64_t bitmap[WORDS], int block) {
    bitmap[block / 64] &= ~(1ULL << (block % 64));
}

/* Free a block (set the bit) */
void free_block(uint64_t bitmap[WORDS], int block) {
    bitmap[block / 64] |= (1ULL << (block % 64));
}
```

Finding a **contiguous** run of k free blocks is more expensive but still practical with word-level operations.

Advantages: Efficient for finding contiguous free space (important for reducing fragmentation). The entire bitmap can fit in memory for moderate-sized disks.

Disadvantages: For very large disks, even the bitmap can be large. Scanning the bitmap is $O(n/w)$ where w is the machine word size.

14.5.2 Linked List

A **free list** links all free blocks together. The head of the list is stored in a known location (e.g., the superblock). Each free block contains a pointer to the next free block.

Advantages: Simple to implement. Allocation is $O(1)$ (take the head of the list).

Disadvantages: To allocate k contiguous blocks, the system must traverse the list looking for adjacent blocks — this is $O(n)$ in the worst case. The free list cannot be cached efficiently because it is scattered across the disk.

14.5.3 Grouping

An optimisation of the free list: the first free block stores the addresses of n free blocks. The last of these n addresses points to a block that stores the next n free addresses, and so on.

This reduces the number of disk reads needed to find free blocks: one read of a group block yields $n - 1$ usable free blocks and a pointer to the next group.

14.5.4 Counting

Since free blocks often occur in contiguous runs (especially after a fresh format or defragmentation), the **counting** method stores each free extent as a pair: (starting block, count). A list of such pairs is maintained, often sorted by starting block.

Example 14.6 (Counting vs. Bitmap). A freshly formatted 1 TB disk with 4 KB blocks has one free extent: (0, 268435456). The counting method needs just one entry (8 bytes), compared to the bitmap's 32 MB. After heavy use with many small allocations and deletions, the counting list may grow large, but it remains much smaller than the bitmap as long as the number of free extents is small relative to the total block count.

Ext4 uses a hybrid approach: it maintains both a **block bitmap** and an **extent-based free space cache** that tracks free extents for fast allocation.

14.6 Journaling File Systems

A file system update often involves multiple writes to different locations: modifying data blocks, updating the inode, adjusting the bitmap, and updating the directory entry. If the system crashes (power failure, kernel panic) in the middle of this multi-step update, the file system can be left in an **inconsistent state**.

File System Consistency. A file system is consistent if all of its metadata structures (superblock, bitmaps, inodes, directory entries) agree with one another. Inconsistency arises when a crash interrupts a multi-step update, leaving some structures modified and others not.

14.6.1 The Consistency Problem

Consider creating a new file, which requires:

1. Allocate an inode (mark it as used in the inode bitmap).

2. Initialise the inode with the file's metadata.
3. Allocate data blocks (mark them as used in the block bitmap).
4. Write data to the allocated blocks.
5. Add a directory entry mapping the file name to the inode.

If the system crashes after step 1 but before step 5, the inode is allocated but no directory entry points to it — an **orphaned inode** that wastes space. If the crash occurs after step 5 but before step 1 completes, the directory points to an uninitialised inode — potentially catastrophic.

Before journaling, the solution was `fsck` (file system check): a program that scans the entire file system at boot time, checking every inode, every block pointer, every bitmap entry, and every directory entry for consistency. For a large file system, `fsck` can take **hours**.

14.6.2 Write-Ahead Logging

Journaling (also called write-ahead logging) solves the consistency problem by recording intended changes in a sequential **log** (or **journal**) before applying them to the file system. The protocol is:

1. **Journal write:** Write a description of all the changes (the **transaction**) to the journal.
2. **Journal commit:** Write a commit record marking the transaction as complete.
3. **Checkpoint:** Apply the changes to their actual locations in the file system.
4. **Journal free:** Mark the journal space as reusable.

If a crash occurs:

- Before the commit record is written: the transaction is incomplete and is simply discarded. The file system remains in its pre-transaction state.
- After the commit but before the checkpoint completes: the recovery process **replays** the journal, re-applying the committed transaction to bring the file system to a consistent state.

The critical insight is that the journal is written **sequentially**, which is fast on both rotational and solid-state drives, and the commit record is a single atomic write (or is made atomic via a checksum).

14.6.3 Journaling Modes

File systems offer different journaling modes that trade recovery completeness against performance:

Journal (Full Data Journaling). Both metadata and data blocks are written to the journal before being checkpointed to their final locations. This provides the strongest consistency guarantee — after a crash, both metadata and file contents are recoverable. However, every block of data is written **twice** (once to the journal, once to the final location), halving the effective write bandwidth.

Ordered (Metadata-Only with Ordering). Only metadata is journaled. However, the file system guarantees that data blocks are written to their final locations **before** the metadata transaction is committed. After a crash, metadata is consistent, and data blocks pointed to by committed metadata contain the intended data (not garbage from a previous file). This is the default mode for ext4.

Writeback (Metadata-Only without Ordering). Only metadata is journaled, and no ordering is enforced between data and metadata writes. This is the fastest mode but can result in files containing stale data after a crash (the inode points to blocks that have not yet been written with the new data).

Example 14.7 (Journaling Overhead Comparison). Consider appending 1 MB to a file on ext4 with a 4 KB block size:

- **Writeback mode:** 256 data block writes + metadata journal writes (approximately 1–2 blocks for the transaction). Total: approximately 258 block writes.
- **Ordered mode:** Same as writeback, but the data blocks must complete before the journal commit. The total number of writes is the same, but the ordering constraint may reduce parallelism.
- **Journal mode:** 256 data block writes to the journal + journal metadata + commit record, then 256 data block writes to the final location + metadata writes. Total: approximately 514 block writes — nearly double.

Theorem 14.2 (Journal Recovery Correctness). If each journal transaction has a valid commit record (verified by checksum), then replaying that transaction from the journal produces the same file system state as if the original operation had completed without interruption. Transactions without valid commit records are safely discarded.

Proof sketch. The journal contains a complete description of all blocks to be written. The commit record's checksum verifies the integrity of the transaction. Replaying the transaction writes exactly the intended blocks to their intended locations, which is

idempotent (replaying the same transaction multiple times produces the same result). Therefore, recovery produces the intended final state regardless of how many times the crash interrupted the checkpoint phase. □

14.6.4 ext4 Journaling Implementation

The ext4 file system uses the **JBD2** (Journaling Block Device 2) layer, which implements a circular log:

```
Journal Structure (circular):
+--------+--------+--------+--------+--------+--------+
| TXN 1  | TXN 2  | TXN 3  | TXN 4  |  FREE  |  FREE |
+--------+--------+--------+--------+--------+--------+
^                           ^                  ^
|                           |                  |
Journal start              Journal head        Journal end
(oldest committed)         (next write)
```

The journal is a fixed-size region (typically 128 MB by default). When it fills up, the system must checkpoint old transactions (write them to their final locations) before reclaiming journal space. If the journal fills up completely, the system stalls until checkpointing frees space.

Mount options control the journaling mode:

```
# Full journaling (safest, slowest)
mount -o data=journal /dev/sda1 /mnt

# Ordered mode (default, good balance)
mount -o data=ordered /dev/sda1 /mnt

# Writeback mode (fastest, least safe)
mount -o data=writeback /dev/sda1 /mnt
```

14.6.5 XFS Journaling

XFS, originally developed by Silicon Graphics (SGI), uses a similar write-ahead log but with several innovations:

- **Log-space reservation:** Before beginning a transaction, XFS reserves sufficient log space, preventing log-full stalls.
- **Delayed allocation:** XFS delays the assignment of physical blocks until data is flushed from the page cache, allowing better allocation decisions (larger

extents, less fragmentation).

- **Allocation groups:** XFS divides the file system into allocation groups, each with its own inode and free-space structures. This enables parallel allocation across multiple threads.

14.7 Log-Structured File Systems

Journaling file systems use the log only for crash recovery — the actual data lives in fixed locations on disk. **Log-structured file systems** take the logging concept to its logical extreme: the log IS the file system. All writes — both data and metadata — are appended sequentially to the log.

14.7.1 The LFS Concept

The Log-structured File System (LFS) was proposed by Rosenblum and Ousterhout in 1992. Their key observation was that as memory sizes grew, read performance was increasingly dominated by the buffer cache (most reads are cache hits), while **write performance remained limited by disk seek time**. By converting all writes to sequential appends, LFS eliminates write-time seeks entirely.

Log-Structured File System. A file system in which all data and metadata are written sequentially to a log. The log is divided into fixed-size **segments** (typically 512 KB to several MB). Within each segment, data blocks, inodes, and inode maps are packed sequentially. An **inode map** maintains the current disk address of each inode, since inodes move with each update.

The write path is simple: buffer writes in memory until a full segment is accumulated, then write the entire segment sequentially. This maximises disk bandwidth and minimises seek overhead.

The read path uses the **inode map** (a small, mostly cached structure) to locate the current version of any inode, then follows the inode's block pointers as usual.

14.7.2 Garbage Collection

The fundamental challenge of LFS is **garbage collection**. When a file is modified, the new version is appended to the log, but the old version remains in its previous segment, now occupying dead space. Over time, segments become a mixture of live and dead blocks.

The **segment cleaner** reclaims space by:

1. Selecting a segment to clean (typically one with a high proportion of dead blocks).
2. Identifying the live blocks in that segment (using a **segment summary** block).
3. Copying the live blocks to the head of the log.
4. Marking the cleaned segment as free.

The cleaning policy critically affects performance. The **cost-benefit** policy considers both the proportion of dead blocks and the age of the segment:

$$\text{cost-benefit} = \frac{\text{free space generated} \times \text{age of youngest block}}{\text{cost of cleaning}} = \frac{(1-u) \times \text{age}}{1+u}$$

where u is the utilisation (fraction of live blocks) of the segment. This policy prefers to clean segments that are mostly dead (high $1-u$) and whose live data is old (unlikely to be overwritten soon).

14.7.3 F2FS: Flash-Friendly File System

F2FS (Flash-Friendly File System), developed by Samsung and merged into the Linux kernel in 2012, is a modern log-structured file system designed specifically for flash storage (SSDs, eMMC, SD cards).

F2FS addresses several flash-specific concerns:

- **Write amplification:** Flash devices can only erase in large units (erase blocks, typically 256 KB–1 MB) but write in smaller units (pages, typically 4–16 KB). Rewriting a single page may require erasing an entire block. LFS's sequential write pattern aligns well with flash: new data is always written to clean space, and the garbage collector consolidates live data.
- **Hot/cold data separation:** F2FS classifies data into six temperature levels (hot/warm/cold for both data and metadata) and writes each category to separate zones. This improves garbage collection efficiency because segments containing only cold data will have high liveness for a long time (fewer live blocks need to be copied).
- **Multi-head logging:** F2FS maintains six active log segments (one per temperature), enabling concurrent writes to different zones without mixing data temperatures.
- **NAT (Node Address Table):** F2FS's version of the inode map. The NAT translates node IDs to physical addresses and is updated lazily to reduce write amplification.

14.8 Copy-on-Write File Systems

Copy-on-write (CoW) file systems take a radically different approach to consistency: instead of overwriting data in place, every modification creates a **new copy** of the modified blocks, and the metadata tree is updated from the leaves to the root to point to the new copies.

14.8.1 The CoW Principle

Copy-on-Write (CoW) File System. A file system in which no block is ever overwritten in place. When data is modified, the new data is written to a new location, the parent metadata block is updated to point to the new data (itself written to a new location), and this propagation continues up to the root of the metadata tree. The root pointer is then atomically updated to point to the new tree.

The key insight is that the atomic update of a single root pointer transitions the entire file system from one consistent state to another. There is no window of inconsistency, and **no journal is needed**.

```
Before modification:          After CoW modification:
      [Root]                       [Root'] (new)
      /    \                       /    \
   [A]    [B]                    [A]    [B'] (new)
   / \    / \                    / \    / \
 [1] [2] [3] [4]              [1] [2] [3'] [4]
                                          (new)
                               Old [Root], [B], [3] are now
                               unreferenced and can be freed.
```

14.8.2 Btrfs

Btrfs (B-tree File System, pronounced “butter FS”), initially developed by Oracle and merged into the Linux kernel in 2009, is a CoW file system designed for enterprise and desktop Linux.

B-tree structure. Btrfs organises all data and metadata in a set of B-trees. The key trees are:

- **FS tree:** Contains inodes, directory entries, and extent data references.
- **Extent allocation tree:** Tracks which extents are allocated and their reference counts.

- **Checksum tree:** Contains CRC32C checksums for every data extent.
- **Chunk tree:** Maps logical addresses to physical addresses (supporting multi-device configurations).

Snapshots. Because CoW never modifies existing blocks, creating a snapshot is nearly instantaneous: simply create a new root pointer that references the same tree. The snapshot and the original share all blocks initially. As modifications occur to either the snapshot or the original, CoW causes the modified paths to diverge while unchanged blocks remain shared.

```
Snapshot creation (O(1)):
[Original Root] -----> [Tree of shared blocks]
[Snapshot Root] -----/
```

Checksums. Btrfs computes CRC32C checksums for every data block and metadata block. Checksums are stored in the checksum tree, separate from the data they protect. On every read, Btrfs verifies the checksum and reports (or automatically repairs, on redundant configurations) any corruption.

14.8.3 ZFS

ZFS (Zettabyte File System), originally developed by Sun Microsystems and now maintained by the OpenZFS project, is the most feature-rich CoW file system. ZFS integrates the file system and volume manager into a single layer.

Storage pool. ZFS manages storage as a **pool** (zpool) of physical devices. File systems (datasets) are created within the pool and share its storage dynamically — no fixed partitions.

Merkle tree checksumming. ZFS stores the checksum of each block in its **parent** block, forming a Merkle tree rooted at the **uberblock** (the root of the pool's metadata). This provides end-to-end data integrity verification: every read can be verified from the data block up through the metadata tree to the root.

Merkle Tree. A hash tree in which every leaf node contains the hash of a data block, and every non-leaf node contains the hash of its children's hashes. Verification of any single leaf requires $O(\log n)$ hash checks along the path from the leaf to the root.

Example 14.8 (ZFS Merkle Tree Verification). A ZFS file system with 4 levels of indirect blocks stores a data block D with checksum $H(D)$ in its parent indirect block I_1. Block I_1 has checksum $H(I_1)$ stored in I_2, and so on up to the uberblock:

$$\text{uberblock} \xrightarrow{H(I_3)} I_3 \xrightarrow{H(I_2)} I_2 \xrightarrow{H(I_1)} I_1 \xrightarrow{H(D)} D$$

If data block D is silently corrupted (a bit flip in storage), then $H(D)$ will not match the checksum stored in I_1, and ZFS will detect the corruption immediately upon read. If the pool has redundancy (mirror or RAID-Z), ZFS will automatically fetch the correct copy from another device and repair the corrupted copy.

ZFS key features:

- **Snapshots and clones:** Zero-cost snapshots (as with all CoW file systems). Clones are writable snapshots.
- **Built-in RAID:** RAID-Z1 (single parity), RAID-Z2 (double parity), RAID-Z3 (triple parity), mirrors.
- **Compression:** Transparent LZ4 or ZSTD compression at the block level.
- **Deduplication:** Optional block-level deduplication using a dedup table (DDT) with SHA-256 checksums.
- **Send/receive:** Efficient incremental replication by sending the delta between two snapshots.

Programmer's Perspective: Working with File Systems from Go. Go provides multiple layers of file system interaction. The `os` package wraps POSIX system calls, while the `io/fs` package provides an abstraction layer.

For direct file operations, Go's `os` package maps closely to the system calls discussed in this chapter:

```go
package main

import (
    "fmt"
    "os"
    "syscall"
)

func main() {
    // Create a file (equivalent to open() with O_CREAT)
    f, err := os.Create("example.txt")
    if err != nil {
        fmt.Fprintf(os.Stderr, "create: %v\n", err)
        os.Exit(1)
    }

    // Write data (the write() system call)
```

```go
    _, err = f.WriteString("Hello, file systems!\n")
    if err != nil {
        fmt.Fprintf(os.Stderr, "write: %v\n", err)
        os.Exit(1)
    }

    // Sync to disk (the fsync() system call --- forces write-back from
    // the page cache to the storage device)
    err = f.Sync()
    if err != nil {
        fmt.Fprintf(os.Stderr, "sync: %v\n", err)
        os.Exit(1)
    }

    // Stat the file (the stat() system call --- retrieves inode metadata)
    info, err := f.Stat()
    if err != nil {
        fmt.Fprintf(os.Stderr, "stat: %v\n", err)
        os.Exit(1)
    }
    fmt.Printf("Size: %d bytes\n", info.Size())
    fmt.Printf("Mode: %s\n", info.Mode())

    // Access underlying system info (Linux-specific)
    if sys, ok := info.Sys().(*syscall.Stat_t); ok {
        fmt.Printf("Inode: %d\n", sys.Ino)
        fmt.Printf("Links: %d\n", sys.Nlink)
        fmt.Printf("Block size: %d\n", sys.Blksize)
        fmt.Printf("Blocks (512-byte): %d\n", sys.Blocks)
    }

    f.Close()

    // Create a hard link
    err = os.Link("example.txt", "example_link.txt")
    if err != nil {
        fmt.Fprintf(os.Stderr, "link: %v\n", err)
    }

    // Create a symbolic link
```

```
    err = os.Symlink("example.txt", "example_symlink.txt")
    if err != nil {
        fmt.Fprintf(os.Stderr, "symlink: %v\n", err)
    }
}
```

Note the `f.Sync()` call. Without it, data may linger in the kernel's page cache and be lost on a crash. This is the application-level equivalent of the journaling discussion: your application must decide what consistency guarantees it needs. Databases call `fsync()` after every transaction commit. Log-append workloads may accept the risk of losing the most recent entries in exchange for higher throughput.

14.9 Crash Recovery Analysis

Understanding crash recovery requires analysing exactly what can go wrong at each step of a multi-block update. This section formalises the crash recovery guarantees provided by different file system architectures.

14.9.1 The Crash Consistency Model

Crash Consistency. A file system provides crash consistency if, after a crash and recovery, the file system is in some **valid state** — either the state before the operation or the state after the operation, but never an intermediate state where metadata is inconsistent with data.

Consider the operation of appending a block to a file. This requires three writes:

1. **D:** Write the new data block.
2. **I:** Update the inode (increment size, add block pointer).
3. **B:** Update the block bitmap (mark the new block as allocated).

Without any consistency mechanism, six crash scenarios are possible (each subset of {D, I, B} that reaches disk):

Writes completed	Outcome
None	Safe: old state preserved

Writes completed	Outcome
D only	Data written but unreferenced (orphaned block). Block bitmap says free, no pointer. Minor space leak.
I only	Inode points to an unallocated block that may contain old data. **Dangerous:** reading the file returns garbage.
B only	Block marked allocated but no inode references it. Space leak.
D + I	Inode points to new data, but bitmap says the block is free. Another file could be allocated the same block. **Data corruption risk.**
D + B	Block allocated and written, but inode does not reference it. Space leak, data inaccessible.
I + B	Inode references the block and bitmap marks it allocated, but the data was not written. **File contains garbage.**
D + I + B	Complete: correct new state.

Theorem 14.3 (Journaling Reduces Crash States). With metadata journaling (ordered mode), the only possible post-crash states are:

1. The operation was not committed (journal has no commit record): the file

system is in the pre-operation state. No data corruption.

2. The operation was committed: recovery replays the journal, bringing metadata to the post-operation state. Because ordered mode ensures D was written before the commit, the data block contains valid data.

No intermediate or inconsistent state is possible. The number of reachable crash states is reduced from $2^n - 1$ (where n is the number of writes) to exactly 2 (before or after).

14.9.2 fsck: The Pre-Journaling Approach

Before journaling, the `fsck` (file system check) utility scanned the entire file system at boot time to detect and repair inconsistencies. The checks include:

1. **Superblock validation:** Is the magic number correct? Are the block counts consistent?
2. **Inode scan:** For each allocated inode, verify that all block pointers point to valid blocks within the file system boundaries.
3. **Block allocation cross-check:** Verify that the block bitmap agrees with the block pointers in all inodes. Blocks referenced by inodes should be marked allocated; blocks not referenced by any inode should be marked free.
4. **Directory structure validation:** Every directory entry should point to a valid inode. The `.` entry should point to the directory itself; `..` should point to the parent.
5. **Link count verification:** The link count in each inode should match the number of directory entries pointing to it.
6. **Orphaned inode recovery:** Inodes with zero link count but non-zero reference count (open by a process at crash time) are placed in the `/lost+found` directory.

Example 14.12 (fsck Time Estimate). A 10 TB ext3 file system with 4 KB blocks has $10 \times 10^{12}/4096 \approx 2.44 \times 10^9$ blocks and approximately 10^8 inodes (at the default bytes-per-inode ratio). At a scan rate of 100 MB/s (sequential read of all metadata), `fsck` needs to read:

- All inode tables: $10^8 \times 128\,\text{bytes} = 12.8\,\text{GB}$
- All block and inode bitmaps: approximately 600 MB
- All directory data for directory structure validation

Total metadata: approximately 15–20 GB. At 100 MB/s: approximately 150–200 seconds. However, `fsck` also performs random I/O for cross-referencing, making the

actual time 10–60 minutes for a 10 TB file system.

With journaling, recovery reads only the journal (128 MB by default) and replays uncommitted transactions: approximately 1–5 seconds regardless of file system size.

14.10 Performance and Caching

14.9.1 The Buffer Cache (Page Cache)

The kernel maintains a **page cache** — a region of main memory that caches recently accessed disk blocks. All file I/O passes through the page cache:

- **Read:** If the requested block is in the cache (a **cache hit**), the data is copied to the user buffer without any disk I/O. On a **cache miss**, the block is read from disk, stored in the cache, and then copied to the user buffer.
- **Write:** Data is written to the page cache and the corresponding page is marked **dirty**. The actual disk write is deferred. The kernel's **writeback** mechanism flushes dirty pages to disk periodically (typically every 5 seconds on Linux) or when memory pressure rises.

The page cache is extremely effective because of temporal and spatial locality in file access patterns. On a typical Linux server, cache hit rates of 90–99% are common for read-heavy workloads.

Page cache eviction. When memory pressure rises and free pages are scarce, the kernel must evict pages from the cache. Linux uses a **two-list LRU** (Least Recently Used) algorithm:

- **Active list:** Pages that have been accessed at least twice recently. These are considered "hot" and are protected from eviction.
- **Inactive list:** Pages that have been accessed only once or have aged out from the active list. Pages are evicted from the tail of the inactive list.

Pages enter the inactive list when first loaded. If accessed again while on the inactive list, they are promoted to the active list. This two-list scheme avoids the problem of a single large sequential scan (e.g., `cp` of a multi-gigabyte file) evicting all cached pages: the scanned pages enter the inactive list and are quickly evicted without displacing frequently-accessed pages on the active list.

Example 14.13 (Page Cache Effectiveness). A database server has 128 GB of RAM, of which 100 GB is available for the page cache (after kernel and process memory).

The database's hot working set is 80 GB (the frequently-accessed indices and recent data). The total database is 2 TB.

Cache hit rate for the hot working set: approximately 100% (fully cached). Cache hit rate for cold queries: approximately 0% (must read from disk). If 90% of queries access the hot working set:

$$\text{Average hit rate} = 0.90 \times 1.0 + 0.10 \times 0.0 = 0.90 = 90\%$$

The effective I/O latency is:

$$L_{\text{eff}} = 0.90 \times 100\,\text{ns} + 0.10 \times 100\,\mu\text{s} = 90\,\text{ns} + 10{,}000\,\text{ns} = 10.09\,\mu\text{s}$$

This is approximately 10x faster than if every access went to the SSD ($100\,\mu$s) and 500x faster than HDD ($5\,$ms).

14.9.2 Memory-Mapped File I/O

The `mmap()` system call maps a file (or a portion of it) directly into the process's virtual address space. After mapping, the file's contents can be accessed using ordinary pointer dereferences — no `read()` or `write()` system calls are needed.

```c
#include <sys/mman.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    int fd = open("data.bin", O_RDWR);
    if (fd < 0) { perror("open"); return 1; }

    /* Get file size */
    off_t size = lseek(fd, 0, SEEK_END);

    /* Map the entire file into memory */
    char *map = mmap(NULL, size, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) { perror("mmap"); return 1; }

    /* Access file contents directly via pointer arithmetic */
```

```
    printf("First 10 bytes: %.10s\n", map);

    /* Modify the file by writing to the mapped region */
    memcpy(map, "MODIFIED!", 9);

    /* Sync changes to disk (optional --- kernel will flush eventually) */
    msync(map, size, MS_SYNC);

    munmap(map, size);
    close(fd);
    return 0;
}
```

Memory-Mapped I/O (`mmap`). A mechanism that maps a file's contents into a process's virtual address space. The file's pages are loaded on demand via **page faults**: accessing an unmapped page triggers a fault, the kernel reads the page from disk into the page cache, and updates the page table to map the virtual address to the cached page. Subsequent accesses to the same page are direct memory references with no kernel involvement.

Advantages of mmap:

- **Zero-copy reads:** Data goes directly from the page cache to the process's address space (via page table mapping). No `copy_to_user()` overhead.
- **Lazy loading:** Only pages that are actually accessed are read from disk. A process that `mmap()`s a 1 GB file but accesses only 10 pages reads only 40 KB from disk.
- **Shared memory:** Multiple processes that `mmap()` the same file share the same physical pages, enabling efficient inter-process communication.

Disadvantages of mmap:

- **Page fault overhead:** Each first access to a page triggers a page fault (approximately 2–10 μs), which is more expensive than a single large `read()` that brings in many pages at once.
- **No error handling:** A disk error during a page fault results in a `SIGBUS` signal, which is difficult to handle gracefully compared to checking the return value of `read()`.
- **TLB pressure:** Mapping large files consumes TLB entries. With 4 KB pages, a 1 GB mapping requires 262,144 page table entries.

14.9.3 The Dentry Cache

Path name resolution is one of the most frequent operations in the kernel. The **dentry cache** (dcache) caches the results of name-to-inode lookups. After resolving `/home/alice/code/main.go`, the dcache stores entries for each component: `/`, `home`, `alice`, `code`, `main.go`. Subsequent accesses to any file under `/home/alice/` will find partial dcache hits, reducing the number of directory reads.

14.9.3 Read-Ahead

When the kernel detects sequential access patterns, it performs **read-ahead**: proactively reading blocks beyond what the application has requested, in anticipation that they will be needed soon. Linux's read-ahead algorithm adapts dynamically:

- Initial read-ahead window: 128 KB (32 pages with 4 KB pages).
- The window grows up to a configurable maximum (typically 256 KB) if sequential access continues.
- Read-ahead is disabled for random access patterns.

14.9.4 Write Ordering and Barriers

On modern storage stacks, writes may be reordered by the kernel's I/O scheduler, the device driver, or the drive's own write cache. This reordering improves throughput but can violate the ordering assumptions of journaling file systems.

A **write barrier** (or **flush**) forces all previously submitted writes to reach persistent storage before any subsequent writes. The ext4 journal issues a barrier after writing the commit record, ensuring that the commit record is not persisted before the transaction data.

Since Linux 2.6.37, barriers are implemented via the **REQ_PREFLUSH** and **REQ_FUA** (Force Unit Access) flags:

- **REQ_PREFLUSH:** Flush the device's write cache before writing this block.
- **REQ_FUA:** Write this specific block directly to persistent media, bypassing the device's write cache.

14.10 File System Comparison

The following table summarises the key characteristics of the file systems discussed in this chapter:

Feature	ext4	XFS	Btrfs	ZFS	F2FS
Architecture	Journaling	Journaling	CoW B-tree	CoW Merkle	Log-structured
Max file size	16 TB	8 EB	16 EB	16 EB	3.94 TB
Max volume size	1 EB	8 EB	16 EB	256 ZB	16 TB
Checksums	Metadata only	Metadata only	CRC32C (data + metadata)	SHA-256/fletche	CRC32 (metadata)
Snapshots	No	No	Yes (CoW)	Yes (CoW)	No
Compression	No	No	LZO, ZLIB, ZSTD	LZ4, ZSTD	LZO, LZ4, ZSTD
Best use case	General-purpose	Large files, databases	Desktop, NAS	Enterprise NAS	Flash storage

Programmer's Perspective: Choosing a File System for Your Application. The choice of file system affects application behaviour in ways that developers often overlook:

- **Database workloads (PostgreSQL, MySQL):** ext4 with `data=ordered` or XFS. Both provide good `fsync()` performance. Avoid Btrfs for databases that use `O_DIRECT` (direct I/O) — Btrfs's CoW semantics interact poorly with database-managed page caches.
- **Container storage:** Btrfs or ZFS. Both support snapshots, which container runtimes (Podman, Docker) use to implement layered images efficiently.
- **Embedded/mobile (Android, IoT):** F2FS. Its log-structured design minimises write amplification on flash storage, extending device lifespan.
- **Data integrity requirements:** ZFS. End-to-end checksumming with automatic repair on redundant pools. For Linux systems where ZFS licensing is a concern, Btrfs provides similar (though less mature) checksumming.

When writing applications that must survive crashes, remember these rules: 1. Always `fsync()` files that must persist (transaction logs, database WAL files). 2. `fsync()` the parent directory after creating a new file — the directory entry update needs to

be flushed too. 3. Use `rename()` for atomic file replacement: write to a temporary file, `fsync()` it, then `rename()` over the target. This leverages the file system's guarantee that `rename()` is atomic.

```go
package main

import (
    "os"
    "path/filepath"
)

// AtomicWriteFile writes data to filename atomically using rename.
func AtomicWriteFile(filename string, data []byte, perm os.FileMode) error {
    dir := filepath.Dir(filename)
    tmp, err := os.CreateTemp(dir, ".tmp-*")
    if err != nil {
        return err
    }
    tmpName := tmp.Name()

    // Clean up on failure
    success := false
    defer func() {
        if !success {
            tmp.Close()
            os.Remove(tmpName)
        }
    }()

    // Write data
    if _, err := tmp.Write(data); err != nil {
        return err
    }

    // Sync file content to disk
    if err := tmp.Sync(); err != nil {
        return err
    }

    // Set permissions
```

```go
    if err := tmp.Chmod(perm); err != nil {
        return err
    }

    if err := tmp.Close(); err != nil {
        return err
    }

    // Atomic rename
    if err := os.Rename(tmpName, filename); err != nil {
        return err
    }

    // Sync parent directory to persist the directory entry
    d, err := os.Open(dir)
    if err != nil {
        return err
    }
    defer d.Close()
    if err := d.Sync(); err != nil {
        return err
    }

    success = true
    return nil
}
```

14.11 File Locking

Concurrent access to shared files is a common source of data corruption. Two processes writing to the same file simultaneously can interleave their writes, producing a garbled result. File locking provides a mechanism for processes to coordinate access.

14.11.1 Advisory vs Mandatory Locking

Advisory Lock. A lock that is enforced only by cooperation among processes. A process that does not attempt to acquire the lock before accessing the file can bypass the lock entirely. Advisory locking is the default on Unix systems and is used by well-

behaved applications (databases, editors) that follow a locking protocol.

Mandatory Lock. A lock enforced by the kernel. Any process that attempts to read or write a locked region is blocked (or receives an error) regardless of whether it requested the lock. Mandatory locking was historically available on System V Unix and Linux (via the `sgid` bit with group-execute cleared), but it is deprecated in modern Linux kernels (removed in Linux 5.15) due to complexity and performance issues.

14.11.2 POSIX File Locking

The `fcntl()` system call provides POSIX record locking, which can lock **byte ranges** within a file:

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int lock_region(int fd, off_t start, off_t len, int type) {
    struct flock fl = {
        .l_type   = type,      /* F_RDLCK, F_WRLCK, F_UNLCK */
        .l_whence = SEEK_SET,
        .l_start  = start,     /* Starting offset */
        .l_len    = len,       /* Length; 0 = to end of file */
    };
    return fcntl(fd, F_SETLKW, &fl);  /* F_SETLKW: blocking */
}

int main(void) {
    int fd = open("shared.dat", O_RDWR);
    if (fd < 0) { perror("open"); return 1; }

    /* Acquire an exclusive lock on bytes 0-99 */
    printf("Acquiring lock...\n");
    if (lock_region(fd, 0, 100, F_WRLCK) == -1) {
        perror("lock");
        return 1;
    }
    printf("Lock acquired. Writing data...\n");

    /* Critical section: write to the locked region */
    lseek(fd, 0, SEEK_SET);
```

```
    write(fd, "Hello, locked world!\n", 21);

    /* Release the lock */
    lock_region(fd, 0, 100, F_UNLCK);
    printf("Lock released.\n");

    close(fd);
    return 0;
}
```

Lock types follow reader-writer semantics:

- **Shared lock (`F_RDLCK`):** Multiple processes can hold shared locks on the same region simultaneously. Prevents exclusive locks from being acquired.
- **Exclusive lock (`F_WRLCK`):** Only one process can hold an exclusive lock on a region. Prevents all other locks (shared and exclusive) from being acquired.
- **Unlock (`F_UNLCK`):** Releases the lock on the specified region.

Example 14.9 (Deadlock with File Locks). Process A locks bytes 0–99, then requests bytes 100–199. Process B locks bytes 100–199, then requests bytes 0–99. Both processes block indefinitely — a classic deadlock. POSIX `fcntl` locking detects this cycle and returns `EDEADLK` to one of the processes, breaking the deadlock.

14.11.3 Open File Description Locks (OFD Locks)

Traditional POSIX locks are associated with the **process**, not the file descriptor. This causes surprising behaviour: closing **any** file descriptor to a file releases **all** locks held by the process on that file, even locks acquired through a different file descriptor. Additionally, locks are not inherited across `fork()`.

Linux provides **Open File Description (OFD) locks** (since Linux 3.15) that are associated with the open file description rather than the process. OFD locks use the same `struct flock` but with commands `F_OFD_SETLK`, `F_OFD_SETLKW`, and `F_OFD_GETLK`:

```
struct flock fl = {
    .l_type   = F_WRLCK,
    .l_whence = SEEK_SET,
    .l_start  = 0,
    .l_len    = 0,    /* Entire file */
    .l_pid    = 0,    /* Must be 0 for OFD locks */
};
/* OFD lock: associated with this file descriptor, not the process */
```

```
fcntl(fd, F_OFD_SETLKW, &fl);
```

OFD locks are the correct choice for multi-threaded applications and for any code that manages multiple file descriptors to the same file.

14.12 The ext4 On-Disk Format

Understanding a concrete file system implementation solidifies the abstract concepts discussed earlier. ext4, the default file system on most Linux distributions, is a direct descendant of ext2 (1993) and ext3 (2001).

14.12.1 Block Groups

ext4 divides the disk into **block groups**, each containing a fixed number of blocks (32,768 blocks = 128 MB for 4 KB blocks). Each block group contains:

```
Block Group Layout:
+--------+--------+---------+--------+--------+---...---+
| Super  | Group  | Block   | Inode  | Inode  | Data    |
| Block  | Desc.  | Bitmap  | Bitmap | Table  | Blocks  |
| (copy) | Table  | (1 blk) | (1 blk)| (N blk)|         |
+--------+--------+---------+--------+--------+---...---+
```

- **Superblock:** Contains global file system parameters (total blocks, total inodes, block size, magic number). Copies are stored in select block groups for redundancy.
- **Group Descriptor Table:** Contains the locations of the bitmaps and inode table for each block group.
- **Block Bitmap:** One bit per block in the group. 1 = allocated, 0 = free.
- **Inode Bitmap:** One bit per inode slot. 1 = in use, 0 = free.
- **Inode Table:** An array of inode structures. Each inode is 256 bytes in ext4.
- **Data Blocks:** The actual file and directory data.

14.12.2 The ext4 Inode Structure

The ext4 inode is 256 bytes and contains:

```
ext4 Inode (256 bytes):
Offset  Size  Field
```

```
------  ----  -----
0x00    2     i_mode (file type + permissions)
0x02    2     i_uid (owner, low 16 bits)
0x04    4     i_size_lo (file size, low 32 bits)
0x08    4     i_atime (last access time)
0x0C    4     i_ctime (inode change time)
0x10    4     i_mtime (last modification time)
0x14    4     i_dtime (deletion time)
0x18    2     i_gid (group, low 16 bits)
0x1A    2     i_links_count (hard link count)
0x1C    4     i_blocks_lo (block count, low 32 bits)
0x20    4     i_flags (EXT4_EXTENTS_FL, etc.)
0x28    60    i_block (12 direct + 3 indirect, or extent tree)
0x64    4     i_generation (NFS generation number)
0x68    4     i_file_acl_lo (extended attributes block)
0x6C    4     i_size_high (file size, high 32 bits)
...
0x80    4     i_extra_isize (size of extra inode fields)
0x84    4     i_ctime_extra (nanosecond precision)
0x88    4     i_mtime_extra
0x8C    4     i_atime_extra
0x90    4     i_crtime (creation time)
0x94    4     i_crtime_extra
```

The 60-byte `i_block` field is overloaded: for files using the traditional indirect block scheme, it stores 12 direct pointers + 3 indirect pointers. For files using extents (the default in ext4), it stores an **extent header** followed by up to 4 extent entries.

14.12.3 Directory Entries

ext4 directories use a **hash tree** (HTree) structure for fast lookups. The HTree is a B-tree variant that hashes file names and uses the hash to direct the search to the correct directory block.

For small directories (fewer than one block), ext4 uses a simple linear list of directory entries:

```
ext4 Directory Entry:
+----------+-----------+----------+---------+-----------+
| inode    | rec_len   | name_len | type    | name      |
| (4 bytes)| (2 bytes) | (1 byte) | (1 byte)| (variable)|
+----------+-----------+----------+---------+-----------+
```

The `rec_len` field spans from this entry to the next, allowing entries to be of variable length (padded to 4-byte boundaries). Deletion is handled by adjusting the previous entry's `rec_len` to skip the deleted entry.

Example 14.10 (ext4 Directory Lookup Performance). A directory contains 100,000 files. Without HTree, a linear scan requires checking up to 100,000 entries (average 50,000). With HTree (using a half-MD4 hash), the search is $O(\log n)$: the hash directs the lookup to the correct block in approximately 2–3 block reads, regardless of the directory size. This is why `ls` in a directory with 100,000 files is fast on ext4 but was painfully slow on ext2 (which lacked HTree).

14.13 Special File Systems

Linux employs several **virtual file systems** that do not store data on disk but instead provide interfaces to kernel data structures or runtime information.

14.13.1 procfs (`/proc`)

The **proc file system** exposes process information and kernel parameters as a hierarchy of virtual files:

- `/proc/[pid]/status` — Process state, memory usage, UID/GID
- `/proc/[pid]/maps` — Memory mappings of the process
- `/proc/[pid]/fd/` — Symbolic links to open file descriptors
- `/proc/meminfo` — System-wide memory statistics
- `/proc/cpuinfo` — CPU information
- `/proc/sys/` — Tunable kernel parameters (writable)

These files are generated on the fly when read; no disk I/O occurs. The `proc` file system is the primary mechanism for monitoring system state from user space.

14.13.2 sysfs (`/sys`)

The **sysfs** file system (mounted at `/sys`) exports the kernel's device model to user space. Every device, driver, bus, and class in the kernel has a corresponding directory in sysfs:

```
/sys/
+-- block/           (block devices)
|   +-- sda/
|       +-- queue/   (I/O scheduler parameters)
```

```
|       +-- stat     (I/O statistics)
+-- bus/             (bus types: pci, usb, i2c, ...)
+-- class/           (device classes: net, input, tty, ...)
+-- devices/         (the device tree)
+-- fs/              (file system parameters)
+-- kernel/          (kernel subsystems)
+-- module/          (loaded kernel modules)
```

14.13.3 tmpfs

tmpfs is a RAM-based file system that uses the kernel's page cache and swap space. Files in tmpfs exist only in memory (and potentially swap); they are lost on reboot.

```
# Mount a 1 GB tmpfs
mount -t tmpfs -o size=1G tmpfs /mnt/ramdisk
```

tmpfs is used for `/tmp` (temporary files), `/run` (runtime data), and `/dev/shm` (POSIX shared memory). Because it operates entirely in RAM, tmpfs provides extremely low latency — there is no disk I/O for reads or writes (unless the system swaps).

tmpfs. A memory-backed file system that stores data in the kernel's page cache. Unlike a ramdisk (`/dev/ram0`), tmpfs dynamically grows and shrinks as files are created and deleted, using only as much memory as its contents require. It also supports swapping: if memory pressure is high, tmpfs pages can be evicted to swap space and brought back on demand.

14.13.4 devtmpfs (`/dev`)

devtmpfs is a special tmpfs used for the device node directory `/dev`. The kernel automatically creates device nodes in devtmpfs when devices are detected, and `udev` (the user-space device manager) applies rules to set ownership, permissions, and create symlinks.

14.14 File System Performance Tuning

14.14.1 Block Size Selection

The block size affects both performance and space efficiency:

Block Size	Internal Fragmentation	Metadata Overhead	Sequential Throughput
1 KB	Low (avg. 512 B wasted per file)	High (many block pointers)	Low (many small I/Os)
4 KB	Moderate (avg. 2 KB wasted per file)	Moderate	Good
64 KB	High (avg. 32 KB wasted per file)	Low	Excellent

Internal Fragmentation. The wasted space within the last block of a file. For a file of size S bytes and block size B bytes, the wasted space is $B - (S \bmod B)$ if $S \bmod B \neq 0$, or 0 if S is a multiple of B. On average, each file wastes $B/2$ bytes.

Example 14.11 (Block Size Trade-Off). A file system stores 1 million files with an average size of 8 KB.

With 4 KB blocks: - Average internal fragmentation per file: 2 KB - Total wasted space: $10^6 \times 2\,\text{KB} = 2\,\text{GB}$ - Blocks per average file: 2 - Block pointers needed: 2 per file (direct pointers)

With 64 KB blocks: - Average internal fragmentation per file: 32 KB - Total wasted space: $10^6 \times 32\,\text{KB} = 32\,\text{GB}$ - Blocks per average file: 1 - But 56 KB wasted in that single block!

The 4 KB block size wastes 2 GB; the 64 KB block size wastes 32 GB — a 16x difference. For many small files, smaller blocks are strongly preferred.

14.14.2 Inode Density

The number of inodes on an ext4 file system is fixed at creation time. The `bytes-per-inode` ratio (default: 16 KB per inode) determines how many inodes are created. If the file system will store many small files, a lower ratio (e.g., 4 KB per inode) is needed to avoid running out of inodes while free space remains.

```
# Create ext4 with more inodes (for many small files)
mkfs.ext4 -i 4096 /dev/sda1

# Check inode usage
df -i /
# Filesystem     Inodes   IUsed   IFree  IUse%  Mounted on
# /dev/sda1     65536000  142000 65394000   1%   /
```

14.14.3 Mount Options for Performance

Key ext4 mount options that affect performance:

- `noatime` — Do not update the access time on reads. This eliminates a write (to update the inode) for every read operation. Recommended for almost all workloads.
- `lazytime` — Buffer inode timestamp updates in memory and write them only when the inode is updated for other reasons. Provides the benefits of `noatime` while still recording access times for applications that need them.
- `barrier=0` — Disable write barriers. Improves write throughput but risks data loss on power failure if the drive has a volatile write cache. Only safe if the drive has a battery-backed cache or if data loss is acceptable.
- `commit=N` — Set the journal commit interval to N seconds (default 5). Higher values reduce journal traffic but increase the window of data at risk.

Exercises

Exercise 14.1. An ext4 file system uses 4 KB blocks and 4-byte block pointers. Each inode has 12 direct pointers, 1 single-indirect pointer, 1 double-indirect pointer, and 1 triple-indirect pointer. (a) Calculate the maximum file size. (b) How many disk block reads are needed to access the very last byte of a maximum-size file? (c) If the pointer size is increased to 8 bytes (to support larger disks), how does the maximum file size change?

Exercise 14.2. A file system uses a bitmap for free-space management on a 2 TB disk with 4 KB blocks. (a) How many bits does the bitmap contain? (b) How much space does the bitmap occupy? (c) If the system can scan the bitmap at memory speed (10 GB/s), how long does it take to find a free block in the worst case? (d) Describe how you would optimise the search using a **summary bitmap** (one bit per group of 64 blocks, indicating whether the group has any free blocks).

Exercise 14.3. Compare the three ext4 journaling modes (journal, ordered, write-back) for the following scenario: a process appends 100 MB to an existing file, then calls `fsync()`. For each mode, describe (a) the total number of bytes written to the storage device, (b) the ordering constraints on those writes, and (c) the state of the file after a crash that occurs immediately after the `fsync()` returns. Assume a 4 KB block size and a 128 MB journal.

Exercise 14.4. A log-structured file system has segments of size 1 MB. The file system is 70% full. The segment cleaner uses the cost-benefit policy: score(s) = $\frac{(1-u(s))\times \text{age}(s)}{1+u(s)}$ where $u(s)$ is the fraction of live blocks in segment s and age(s) is the time since the segment was last written. Two candidate segments have: (a) $u = 0.3$, age $= 100$ seconds; (b) $u = 0.8$, age $= 10{,}000$ seconds. Compute the score for each and determine which segment should be cleaned first. Explain intuitively why the policy might prefer a mostly-live segment if it is very old.

Exercise 14.5. ZFS uses a Merkle tree for data integrity. A ZFS pool has 5 levels of indirect blocks and uses SHA-256 (32-byte digests). (a) If a single bit flips in a data block, at which point during a read operation is the corruption detected? (b) How many hash comparisons are needed to verify a single data block read? (c) ZFS stores the checksum of each block in its parent. Explain why this is more robust than storing each block's checksum alongside the block itself (as a traditional CRC would).

Exercise 14.6. Consider a file system that uses extent-based allocation. A file is written in three phases: first, 64 KB is written sequentially; then the application seeks to offset 1 MB and writes 32 KB; finally, it returns to offset 64 KB and writes 16 KB. Assuming the file system can allocate contiguous space for each write, draw the extent tree for this file. How many extents are needed? Compare the metadata overhead to a traditional block-pointer inode (4 KB blocks, 4-byte pointers).

Exercise 14.7. Btrfs supports snapshots via CoW. Explain with a concrete example how a snapshot is created, how subsequent writes to the original file system cause the snapshot and the original to diverge, and how blocks are freed when a snapshot is deleted. Specifically: (a) draw the block tree before the snapshot, (b) draw it after the snapshot, (c) draw it after modifying a file in the original, and (d) describe the reference counting mechanism that determines when a shared block can be freed.

Chapter 15: I/O Systems

"The I/O system is the part of the operating system that mediates between the chaos of the physical world and the orderly abstractions that software expects. It is, by necessity, the messiest part of any kernel." — Robert Love, *Linux Kernel Development*

15.1 I/O Hardware Foundations

The I/O subsystem of an operating system must manage an extraordinary diversity of devices: keyboards that produce ten bytes per second, NVMe drives that sustain seven gigabytes per second, and network interfaces that must process millions of packets per second with microsecond latency. Despite this diversity, I/O hardware shares a common architectural pattern: a **controller** mediates between the device's physical mechanism and the system bus, presenting a set of **registers** that software can read and write.

15.1.1 Ports, Buses, and Controllers

A modern computer system connects devices through a hierarchy of **buses**:

```
CPU <---> Memory Controller <---> DRAM
 |
 +---> PCIe Root Complex
        |
        +---> PCIe x16: GPU
        +---> PCIe x4:  NVMe SSD
        +---> PCIe x1:  Network card
        |
        +---> DMI/Chipset
               |
               +---> SATA Controller ---> SATA SSD/HDD
               +---> USB Controller  ---> USB devices
               +---> Audio Controller
               +---> LPC/eSPI ---> Keyboard, TPM
```

Device Controller. A device controller is a hardware component that acts as the

interface between the system bus and the device. It contains a set of registers (status, control, data-in, data-out) that the CPU reads and writes to communicate with the device. Complex controllers (SATA, NVMe, USB) contain their own microprocessors and firmware.

Every device controller exposes at least three types of registers:

- **Status register:** Reports the current state of the device (busy, ready, error). The CPU reads this register to determine whether the device is ready for the next operation.
- **Control (command) register:** The CPU writes to this register to issue commands to the device (start transfer, reset, configure parameters).
- **Data registers:** Used to transfer data between the CPU and the device. Simple devices have a single data register; complex devices may have FIFO buffers.

15.1.2 Register Access: Port-Mapped vs Memory-Mapped I/O

The CPU can access device registers through two mechanisms:

Port-Mapped I/O (PMIO). The device registers occupy a separate address space from memory, accessed with dedicated I/O instructions. On x86, the `IN` and `OUT` instructions transfer data to/from 16-bit port addresses (0x0000–0xFFFF). The x86 I/O address space is limited to 64 KB.

```c
#include <stdint.h>

/* Port-mapped I/O: read a byte from a port */
static inline uint8_t inb(uint16_t port) {
    uint8_t val;
    __asm__ volatile ("inb %1, %0" : "=a"(val) : "Nd"(port));
    return val;
}

/* Port-mapped I/O: write a byte to a port */
static inline void outb(uint16_t port, uint8_t val) {
    __asm__ volatile ("outb %0, %1" : : "a"(val), "Nd"(port));
}

/* Example: read the CMOS real-time clock seconds register */
uint8_t read_rtc_seconds(void) {
    outb(0x70, 0x00);     /* Select register 0 (seconds) */
```

```
    return inb(0x71);      /* Read the value */
}
```

Memory-Mapped I/O (MMIO). The device registers are mapped into the CPU's physical address space. Normal load/store instructions access the registers. The memory controller routes accesses to the appropriate device based on the physical address.

```
#include <stdint.h>

/* Memory-mapped I/O: typical for ARM, PCIe devices, framebuffers */
#define UART_BASE  0x101F1000  /* PL011 UART on ARM Versatile */
#define UART_DR    (*(volatile uint32_t *)(UART_BASE + 0x00))
#define UART_FR    (*(volatile uint32_t *)(UART_BASE + 0x18))
#define UART_FR_TXFF  (1 << 5)  /* Transmit FIFO full */

void uart_putchar(char c) {
    while (UART_FR & UART_FR_TXFF) {
        /* Spin until transmit FIFO has space */
    }
    UART_DR = c;
}
```

Volatile Access. When accessing memory-mapped device registers, the `volatile` qualifier (in C/C++) tells the compiler that the value at this address may change at any time (due to hardware activity) and that accesses must not be optimised away, reordered, or cached in a CPU register. Without `volatile`, the compiler may eliminate "redundant" reads or defer writes, causing the driver to malfunction.

Example 15.1 (Why Volatile Matters). Consider polling a status register:

```
/* WITHOUT volatile --- compiler may optimise this to an infinite loop */
uint32_t *status = (uint32_t *)0x40001000;
while (*status & BUSY_FLAG) {
    /* wait */
}
/* The compiler sees no writes to *status in the loop body,
   concludes *status never changes, and hoists the read
   out of the loop. The loop either never executes or
   never terminates. */

/* WITH volatile --- each iteration re-reads from the hardware */
volatile uint32_t *status = (volatile uint32_t *)0x40001000;
```

```c
while (*status & BUSY_FLAG) {
    /* wait --- each iteration reads the actual hardware register */
}
```

Modern systems predominantly use MMIO. PCIe devices are always memory-mapped; legacy x86 devices (keyboard controller, serial ports, PIC) use port-mapped I/O.

15.2 I/O Communication Mechanisms

The CPU must coordinate with devices to transfer data and signal completion. Three mechanisms exist, each representing a different trade-off between CPU utilisation and latency.

15.2.1 Polling (Programmed I/O)

In polling, the CPU repeatedly reads the device's status register in a tight loop, waiting for the device to signal readiness. This is also called **busy waiting** or **spin waiting**.

```c
/* Polling-based serial port output */
#define SERIAL_PORT   0x3F8
#define LINE_STATUS   (SERIAL_PORT + 5)
#define THR_EMPTY     0x20

void serial_putchar_polling(char c) {
    /* Poll the line status register until the transmit
       holding register is empty */
    while ((inb(LINE_STATUS) & THR_EMPTY) == 0) {
        /* busy wait */
    }
    outb(SERIAL_PORT, c);
}
```

Analysis: Polling wastes CPU cycles during the wait. If the device responds in t_{device} time and the CPU polls every t_{poll} cycle, the CPU wastes up to $t_{\text{device}}/t_{\text{poll}}$ poll iterations. For a slow device (disk: $t_{\text{device}} \sim 5\,\text{ms}$) and a fast CPU ($t_{\text{poll}} \sim 1\,\text{ns}$), that is 5 million wasted iterations.

When polling is appropriate:

- The device responds extremely quickly (latency < 1 μs), making the overhead of an interrupt greater than the polling cost.
- The system is dedicated to a single task (embedded controller waiting for a sensor).
- High-throughput networking where the cost of interrupt processing exceeds the cost of polling (DPDK, kernel bypass).

15.2.2 Interrupt-Driven I/O

In interrupt-driven I/O, the device signals the CPU via a hardware **interrupt** when it requires attention. The CPU is free to execute other code between I/O operations.

Hardware Interrupt. A signal from a device controller to the CPU indicating that the device requires attention (data is ready, transfer is complete, error occurred). The CPU suspends its current execution, saves state, and transfers control to an **interrupt handler** (also called an **interrupt service routine**, ISR) associated with the device.

The interrupt mechanism involves:

1. **Interrupt request (IRQ):** The device asserts an interrupt line.
2. **Interrupt controller:** The Programmable Interrupt Controller (PIC) or Advanced Programmable Interrupt Controller (APIC) prioritises and routes the interrupt to the appropriate CPU.
3. **CPU response:** The CPU finishes the current instruction, pushes the return address and flags onto the stack, disables further interrupts (optionally), and jumps to the ISR via the **interrupt descriptor table** (IDT on x86).
4. **ISR execution:** The ISR handles the event (reads data from the device, acknowledges the interrupt, wakes up a waiting process).
5. **Return from interrupt:** The ISR restores state and returns control to the interrupted code.

```c
/* Simplified interrupt-driven serial driver (Linux kernel style) */
#include <linux/interrupt.h>

static irqreturn_t serial_isr(int irq, void *dev_id) {
    struct serial_port *port = dev_id;
    uint8_t status = inb(port->base + LINE_STATUS);

    if (status & DATA_READY) {
        uint8_t data = inb(port->base);
```

```
        /* Place data in a ring buffer for the reading process */
        ring_buffer_put(&port->rx_buf, data);
        /* Wake up any process waiting for data */
        wake_up_interruptible(&port->rx_wait);
        return IRQ_HANDLED;
    }
    return IRQ_NONE;  /* Not our interrupt */
}

/* During driver initialisation: */
int ret = request_irq(IRQ_SERIAL, serial_isr, IRQF_SHARED,
                      "serial", port);
```

Message-Signalled Interrupts (MSI/MSI-X). Traditional interrupt lines are physical wires shared among multiple devices (requiring the ISR to poll each device to determine which raised the interrupt). Modern PCIe devices use **Message-Signalled Interrupts** (MSI), where the device signals an interrupt by writing a specific value to a specific memory address.

MSI-X (Message Signalled Interrupts — Extended). An interrupt delivery mechanism where each device can have up to 2048 independent interrupt vectors. Each vector has its own message address and message data. The device writes the message data to the message address (a write to a special region of the LAPIC's address space), which the CPU treats as an interrupt. MSI-X eliminates shared interrupt lines, enables per-queue interrupts (critical for NVMe, which has thousands of queues), and allows interrupt steering to specific CPU cores.

With MSI-X, an NVMe SSD can direct completion interrupts for each queue pair to the CPU core that submitted the request, eliminating cross-core interrupt forwarding and improving cache locality.

Example 15.13 (Interrupt Routing with MSI-X). An NVMe SSD has 8 I/O queue pairs. With MSI-X, each queue pair is assigned its own interrupt vector, routed to the CPU core that manages that queue:

Queue Pair	MSI-X Vector	Target CPU
0	Vector 33	CPU 0
1	Vector 34	CPU 1
2	Vector 35	CPU 2
3	Vector 36	CPU 3

Queue Pair	MSI-X Vector	Target CPU
4	Vector 37	CPU 4
5	Vector 38	CPU 5
6	Vector 39	CPU 6
7	Vector 40	CPU 7

When a completion arrives on queue pair 3, the NVMe controller writes to vector 36's message address, which the APIC routes to CPU 3. CPU 3 processes the completion without any inter-processor interrupt (IPI) or cross-core cache transfer.

Theorem 15.1 (Interrupt vs. Polling Break-Even). Let C_{poll} be the CPU cost of one poll iteration, C_{intr} be the total CPU cost of servicing an interrupt (context save, ISR execution, context restore), and T_{device} be the average device response time. Polling is more efficient than interrupts when:

$$\frac{T_{\text{device}}}{C_{\text{poll}}} < \frac{C_{\text{intr}}}{C_{\text{poll}}}$$

That is, when the number of poll iterations (wasted cycles) is less than the interrupt overhead measured in poll-equivalent cycles. With modern CPUs, $C_{\text{intr}} \approx 1{,}000$–$5{,}000$ cycles and $C_{\text{poll}} \approx 10$–$50$ cycles, so polling wins when the device responds in fewer than approximately 100–500 poll iterations (~ 1–$5\,\mu$s).

15.2.3 Direct Memory Access (DMA)

For bulk data transfers (disk I/O, network packets, GPU data), having the CPU copy data byte-by-byte between device registers and memory is grossly inefficient. **Direct Memory Access** (DMA) offloads the data transfer to a dedicated DMA controller, freeing the CPU for other work.

Direct Memory Access (DMA). A mechanism by which a device controller transfers data directly between the device and main memory without CPU intervention. The CPU sets up the transfer (source address, destination address, byte count, direction) in the DMA controller's registers, then the DMA controller independently moves data over the system bus. When the transfer completes, the DMA controller raises an interrupt.

The DMA transfer sequence is:

1. The CPU programs the DMA controller: memory address, byte count, direction (device-to-memory or memory-to-device), and device identifier.
2. The CPU issues a command to the device controller to begin the transfer.
3. The DMA controller arbitrates for the system bus and transfers data directly between the device and memory, one word or burst at a time.
4. Upon completion, the DMA controller raises an interrupt to notify the CPU.

```
DMA Transfer:
                    1. CPU programs DMA
CPU ---[setup]---> DMA Controller ---[bus master]---> Memory
                        |                                ^
                        |         2. Data transfer     |
                    Device Controller -----------------+
                        |
                    3. Completion interrupt
                        +---> CPU (ISR)
```

Bus mastering. Modern DMA-capable devices (PCIe NVMe controllers, network cards) are **bus masters**: they can initiate transfers on the bus without needing a separate DMA controller chip. The device itself contains the DMA engine.

Example 15.2 (DMA Throughput Advantage). Reading a 4 KB block from disk via programmed I/O (PIO) versus DMA:

PIO: The CPU executes 1024 iterations of a 32-bit `IN` instruction (4 bytes per iteration). Each iteration takes approximately 100 ns (I/O bus latency). Total CPU time: $1024 \times 100\,\text{ns} = 102.4\,\mu\text{s}$. The CPU is 100% busy during the transfer.

DMA: The CPU programs the DMA controller (approximately 500 ns), then is free. The DMA controller transfers 4 KB at bus speed (e.g., 3.2 GB/s for PCIe 3.0 x1): $4096/3.2 \times 10^9 \approx 1.3\,\mu\text{s}$. The CPU handles the completion interrupt (approximately 2 μs). Total CPU time: approximately 2.5 μs. The CPU is free for approximately 1.3 μs during the transfer.

For large transfers (1 MB), the advantage is dramatic: PIO ties up the CPU for 25.6 ms; DMA takes 312 μs of bus time with approximately 2.5 μs of CPU time.

Scatter-Gather DMA. Modern DMA controllers support **scatter-gather lists**: a single DMA operation can transfer data between the device and multiple non-contiguous memory regions. This is essential for performance because:

- The operating system uses virtual memory, and a logically contiguous buffer may span multiple physical pages.

- Network packets have headers and payloads in different memory regions.
- File system buffers for a single read request may come from different pages in the page cache.

The scatter-gather list is an array of (physical address, length) pairs:

```
struct scatterlist {
    unsigned long  page_link;    /* page pointer + flags */
    unsigned int   offset;       /* offset within the page */
    unsigned int   length;       /* transfer length */
    dma_addr_t     dma_address;  /* DMA (bus) address */
    unsigned int   dma_length;   /* DMA transfer length */
};
```

15.2.4 The IOMMU: Protecting DMA

DMA is powerful but dangerous: a misbehaving or compromised device could DMA to arbitrary physical memory, reading secrets from other processes or overwriting kernel data structures. The **IOMMU** (I/O Memory Management Unit) provides hardware-enforced isolation for DMA transfers.

IOMMU (I/O Memory Management Unit). A hardware unit that translates device-visible addresses (I/O virtual addresses, or IOVA) to physical addresses, and enforces access control on DMA transactions. The IOMMU performs the same function for devices that the MMU performs for CPUs: address translation and protection.

On Intel platforms, the IOMMU is called **VT-d** (Virtualization Technology for Directed I/O). On AMD platforms, it is called **AMD-Vi**.

Without an IOMMU, the kernel must give devices physical addresses for DMA buffers. Any device can read or write any physical address. This is problematic for:

- **Security:** A compromised device (malicious USB device, firmware-hacked NIC) can read kernel memory, exfiltrate data, or inject code.
- **Virtualisation:** To assign a device to a virtual machine (PCI passthrough), the hypervisor must ensure the device can only access the VM's memory, not the host's or other VMs' memory.
- **Error containment:** A buggy device driver that programs incorrect DMA addresses can corrupt arbitrary memory. With an IOMMU, the transaction is blocked and an error is reported.

The IOMMU maintains **I/O page tables** (similar to CPU page tables) that map IOVAs to physical addresses. The kernel configures these page tables when setting up DMA

mappings:

```
/* Linux DMA mapping API (simplified) */
#include <linux/dma-mapping.h>

/* Map a kernel buffer for DMA access by a device */
dma_addr_t dma_addr = dma_map_single(dev, buffer, size, DMA_TO_DEVICE);
if (dma_mapping_error(dev, dma_addr)) {
    /* Mapping failed */
    return -ENOMEM;
}

/* Give dma_addr to the device --- this is an IOVA, not a physical address */
write_device_register(dev, DMA_ADDR_REG, dma_addr);

/* After DMA completes, unmap */
dma_unmap_single(dev, dma_addr, size, DMA_TO_DEVICE);
```

Example 15.12 (IOMMU Address Translation). A process allocates a buffer at virtual address `0x7fff_0000_0000`, which maps to physical address `0x0000_0234_5000`. The kernel programs the IOMMU to map IOVA `0x0000_0100_0000` to physical `0x0000_0234_5000`. The device receives IOVA `0x0000_0100_0000` and initiates DMA.

The IOMMU translates: IOVA `0x0100_0000` → Physical `0x0234_5000`

If the device attempts to DMA to IOVA `0x0000_0200_0000` (which has no mapping), the IOMMU blocks the transaction and raises an interrupt (DMAR fault on Intel, reported in `dmesg`).

15.3 Kernel I/O Subsystem

The kernel's I/O subsystem provides services that are common across all device types, shielding device drivers from low-level details and applications from device diversity.

15.3.1 I/O Scheduling

When multiple processes issue I/O requests simultaneously, the kernel must decide the order in which to service them. The goals of I/O scheduling are:

- **Fairness:** No process should be starved of I/O service.
- **Throughput:** Maximise the number of I/O operations completed per second.

- **Latency:** Minimise the time between issuing a request and receiving a response.

These goals often conflict. For rotational disks, reordering requests to minimise seek distance improves throughput but may increase latency for some requests. The kernel's **I/O scheduler** (also called the **elevator**) mediates this trade-off.

15.3.2 Buffering

Buffering is the use of a memory area to temporarily store data being transferred between two entities (application and device, or two devices) that operate at different speeds or with different transfer granularities.

I/O Buffering. The kernel uses buffers to decouple the production and consumption of data. Three levels of buffering are common:

1. **Single buffering:** One kernel buffer. The device fills the buffer; when full, the kernel copies data to the user buffer. Transfer and processing cannot overlap.
2. **Double buffering:** Two kernel buffers. While the device fills one buffer, the kernel processes (copies to user space) the other. Transfer and processing overlap.
3. **Circular (ring) buffering:** n buffers arranged in a ring. The producer (device) fills the next empty buffer; the consumer (kernel/application) drains the next full buffer. This is the standard pattern for high-throughput I/O.

Example 15.3 (Double Buffering Throughput). A device transfers data at rate R_d and the CPU processes (copies) data at rate R_c. With single buffering, the total time for n transfers is:

$$T_{\text{single}} = n \times (T_d + T_c) = n \times \left(\frac{B}{R_d} + \frac{B}{R_c} \right)$$

where B is the buffer size. With double buffering, the transfer and copy overlap:

$$T_{\text{double}} = T_d + (n-1) \times \max(T_d, T_c) + T_c$$

For $R_d = R_c$, single buffering gives throughput $R_d/2$; double buffering gives throughput R_d — a twofold improvement.

15.3.3 Caching

The **page cache** (discussed in Chapter 14) is the kernel's primary caching mechanism for block devices. Additionally, the kernel caches:

- **Dentry cache (dcache):** Name-to-inode translations.
- **Inode cache:** Recently accessed inode structures.
- **Buffer heads:** Metadata about cached disk blocks.

Caching and buffering are distinct concepts. A **cache** stores a copy of data that also exists elsewhere, trading memory for reduced I/O. A **buffer** holds the only copy of data that is in transit. The page cache serves both roles: it caches frequently read data and buffers dirty pages awaiting writeback.

15.3.4 Spooling

Spooling (Simultaneous Peripheral Operations On-Line) queues I/O operations for devices that can serve only one request at a time, such as printers. Instead of blocking a process while the printer is busy, the system copies the output to a spool directory on disk. A **spool daemon** dequeues jobs and sends them to the printer one at a time.

The print spooler is a classic example: `lp` on Unix copies the file to `/var/spool/cups/`, and the CUPS daemon manages the queue. This converts a non-sharable device into a shared resource.

15.4 Block Devices and Character Devices

The kernel classifies devices into two fundamental categories based on their data transfer characteristics.

Block Device. A device that stores data in fixed-size blocks (typically 512 bytes or 4 KB) and supports random access. Each block has a unique address. Examples: hard disks, SSDs, USB flash drives, CD-ROMs.

Character Device. A device that produces or consumes a stream of bytes sequentially, without block structure or random access. Examples: serial ports, keyboards, mice, audio devices, printers.

In Linux, the distinction is visible in the device file system:

```
$ ls -l /dev/sda /dev/ttyS0
brw-rw---- 1 root disk  8, 0 Apr 16 10:00 /dev/sda      (block device)
crw-rw---- 1 root dialout 4, 64 Apr 16 10:00 /dev/ttyS0  (character device)
```

The leading `b` or `c` indicates the device type. The two numbers (8,0 and 4,64) are the **major** and **minor** device numbers: the major number identifies the driver, and the minor number identifies the specific device managed by that driver.

Block devices support an additional abstraction layer: the **request queue**. I/O requests to a block device are placed in a queue where the I/O scheduler can reorder and merge them before dispatching to the driver. Character devices have no such queue; requests are passed directly to the driver.

Programmer's Perspective: Go's `io.Reader` and `io.Writer` as OS Abstractions. Go's `io.Reader` and `io.Writer` interfaces are the language-level equivalent of the kernel's block/character device abstraction. Every I/O source in Go implements one or both:

```go
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}
```

This mirrors how the kernel presents devices to user space: everything is a file descriptor that supports `read()` and `write()`. The power of this abstraction is composition. Just as the kernel can stack a buffer cache on top of a block device, Go can stack `bufio.Reader` on top of any `io.Reader`:

```go
package main

import (
    "bufio"
    "compress/gzip"
    "fmt"
    "io"
    "os"
)

func processStream(r io.Reader) error {
    scanner := bufio.NewScanner(r)
    lines := 0
    for scanner.Scan() {
        lines++
    }
```

```go
    if err := scanner.Err(); err != nil {
        return err
    }
    fmt.Printf("Processed %d lines\n", lines)
    return nil
}

func main() {
    // Open a gzip-compressed file
    f, err := os.Open("access.log.gz")
    if err != nil {
        fmt.Fprintf(os.Stderr, "open: %v\n", err)
        os.Exit(1)
    }
    defer f.Close()

    // Stack: file -> gzip decompressor -> line scanner
    // Each layer implements io.Reader
    gz, err := gzip.NewReader(f)
    if err != nil {
        fmt.Fprintf(os.Stderr, "gzip: %v\n", err)
        os.Exit(1)
    }
    defer gz.Close()

    // processStream works with ANY io.Reader:
    // a file, a network connection, a gzip stream, or a test stub
    if err := processStream(gz); err != nil {
        fmt.Fprintf(os.Stderr, "process: %v\n", err)
        os.Exit(1)
    }
}
```

This composability is directly analogous to the kernel's I/O stack. In the Linux kernel, a read from an encrypted file system on a RAID array passes through: VFS → ext4 → dm-crypt → md (RAID) → block layer → NVMe driver. Each layer implements the same `bio` (block I/O) interface. In Go, `f` (file) → `gz` (decompressor) → `scanner` (line buffer) — each implements `io.Reader`.

The key design lesson: when you define I/O interfaces in your own code, accept `io.Reader` and `io.Writer` rather than concrete types. This makes your code testable

(pass a `bytes.Buffer` in tests), composable (wrap with compression, encryption, buffering), and resilient to changes in the underlying I/O source.

15.5 Disk Scheduling Algorithms

For rotational hard disk drives (HDDs), the time to service an I/O request has three components:

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}$$

where:

- T_{seek} is the time to move the disk arm to the target track (0.5–10 ms, depending on distance).
- T_{rotation} is the time to wait for the target sector to rotate under the head. On average, half a revolution: for a 7200 RPM drive, $T_{\text{rotation}} = \frac{1}{2} \times \frac{60}{7200} = 4.17\,\text{ms}$.
- T_{transfer} is the time to read or write the data once the head is positioned (typically < 0.1 ms for a single sector).

Since seek time dominates, disk scheduling algorithms focus on **minimising total seek distance** across a set of pending requests.

15.5.1 FCFS (First-Come, First-Served)

Requests are serviced in the order they arrive. This is fair but can result in wildly inefficient head movement.

Example 15.4 (FCFS Scheduling). The disk has 200 tracks (0–199). The head starts at track 53. The request queue is: 98, 183, 37, 122, 14, 124, 65, 67.

Head movement: $53 \to 98 \to 183 \to 37 \to 122 \to 14 \to 124 \to 65 \to 67$

Total seek distance: $|53 - 98| + |98 - 183| + |183 - 37| + |37 - 122| + |122 - 14| + |14 - 124| + |124 - 65| + |65 - 67| = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640$ tracks.

15.5.2 SSTF (Shortest Seek Time First)

Select the request closest to the current head position. This is a greedy algorithm that minimises seek time locally.

Example 15.5 (SSTF Scheduling). Same setup as Example 15.4. Head at 53.

1. Closest to 53: 65 (distance 12). Move to 65.
2. Closest to 65: 67 (distance 2). Move to 67.
3. Closest to 67: 37 (distance 30). Move to 37.
4. Closest to 37: 14 (distance 23). Move to 14.
5. Closest to 14: 98 (distance 84). Move to 98.
6. Closest to 98: 122 (distance 24). Move to 122.
7. Closest to 122: 124 (distance 2). Move to 124.
8. Closest to 124: 183 (distance 59). Move to 183.

Total: $12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236$ tracks.

SSTF significantly reduces total seek distance compared to FCFS. However, it can cause **starvation**: requests at the extremes of the disk may wait indefinitely if new requests keep arriving near the current head position.

Theorem 15.2 (SSTF Optimality). SSTF does not minimise total seek distance in general. The problem of minimising total seek distance for a set of pending requests is equivalent to the Travelling Salesman Problem on a line (1D TSP), which can be solved optimally in $O(n^2)$ time by dynamic programming. However, in an online setting where new requests arrive continuously, no algorithm can guarantee optimality.

15.5.3 SCAN (Elevator Algorithm)

The SCAN algorithm moves the head in one direction, servicing all requests in its path, until it reaches the end of the disk. It then reverses direction and services requests on the return sweep.

Example 15.6 (SCAN Scheduling). Head at 53, moving towards 0.

Downward sweep: $53 \to 37 \to 14 \to 0$ (reverse at end) Upward sweep: $0 \to 65 \to 67 \to 98 \to 122 \to 124 \to 183$

Total: $(53-37)+(37-14)+(14-0)+(0-0)+(65-0)+(67-65)+(98-67)+(122-98)+(124-122)+(183-124) = 16+23+14+65+2+31+24+2+59 = 236$ tracks.

Note: SCAN always goes to the end of the disk (track 0) before reversing, even if no requests remain in that direction.

SCAN provides a bounded waiting time: every request is serviced within at most two full sweeps of the disk. It avoids SSTF's starvation problem.

15.5.4 C-SCAN (Circular SCAN)

C-SCAN services requests only in one direction. When the head reaches the end of the disk, it **jumps back** to the beginning without servicing any requests on the return, then resumes scanning in the original direction.

This provides more **uniform waiting times**: the head does not service some requests on both the forward and backward sweeps (which SCAN does, giving a bias towards the middle tracks).

Example 15.7 (C-SCAN Scheduling). Head at 53, moving upward.

Forward sweep: $53 \to 65 \to 67 \to 98 \to 122 \to 124 \to 183 \to 199$ (end of disk) Jump to 0 (no service during jump) Forward sweep: $0 \to 14 \to 37$

Total seek distance (counting the jump): $12+2+31+24+2+59+16+199+14+23 = 382$ tracks.

Without counting the return jump (which is fast on real hardware): $12+2+31+24+2+59+16+14+23 = 183$ tracks.

15.5.5 LOOK and C-LOOK

LOOK is a practical variant of SCAN: instead of going to the physical end of the disk, the head reverses when it reaches the **last request** in the current direction. **C-LOOK** is the corresponding variant of C-SCAN.

Example 15.8 (C-LOOK Scheduling). Head at 53, moving upward.

Forward sweep: $53 \to 65 \to 67 \to 98 \to 122 \to 124 \to 183$ (last request in this direction; jump to lowest pending request) Jump to 14 (no service during jump) Forward sweep: $14 \to 37$

Total (excluding jump): $12 + 2 + 31 + 24 + 2 + 59 + 23 = 153$ tracks.

15.5.6 Comparison Summary

Algorithm	Total Seek (Example)	Starvation	Variance
FCFS	640	No	High
SSTF	236	Yes	Medium
SCAN	236	No	Medium
C-SCAN	382 (183 effective)	No	Low

Algorithm	Total Seek (Example)	Starvation	Variance
C-LOOK	153	No	Low

In practice, the Linux kernel's default I/O scheduler for rotational disks is **mq-deadline**, which combines elements of SCAN with deadline-based request aging to prevent starvation.

15.5.7 Seek Time Modelling

The seek time function for a typical HDD is not linear. It has two components: the time to accelerate and decelerate the arm, and the time to traverse the distance. A common model is:

$$T_{\text{seek}}(d) = a + b\sqrt{d}$$

where d is the seek distance in tracks, a is the arm start/stop overhead (approximately 0.5–1 ms), and b is a scaling constant that depends on the drive's actuator.

Example 15.9 (Seek Time Estimation). A drive has $a = 0.8\,\text{ms}$ and $b = 0.2\,\text{ms}/\sqrt{\text{track}}$. Track seek times:

Seek Distance (tracks)	Seek Time
1	$0.8 + 0.2 \times 1.0 = 1.0\,\text{ms}$
10	$0.8 + 0.2 \times 3.16 = 1.43\,\text{ms}$
100	$0.8 + 0.2 \times 10.0 = 2.8\,\text{ms}$
1,000	$0.8 + 0.2 \times 31.6 = 7.12\,\text{ms}$
10,000	$0.8 + 0.2 \times 100.0 = 20.8\,\text{ms}$

The square-root model captures the physics: most of the seek time is spent accelerating and decelerating the arm, with the middle portion of a long seek at maximum velocity. This is why reducing seek distance by reordering requests (SSTF, SCAN) yields significant latency reductions.

Theorem 15.3 (Expected Seek Distance Under Uniform Random Requests). If the disk has N tracks and requests arrive at uniformly random positions, the expected seek distance under FCFS scheduling is:

$$E[d] = \frac{N}{3}$$

Proof. The expected value of $|X-Y|$ where X and Y are independent uniform random variables on $[0, N]$ is:

$$E[|X-Y|] = \int_0^N \int_0^N \frac{|x-y|}{N^2}\, dx\, dy = \frac{N}{3}$$

The integral evaluates by splitting into regions $x > y$ and $x < y$, each contributing $N/6$. □

This result means that under random I/O, the average FCFS seek traverses one-third of the disk. For a 10,000-track disk, the expected seek distance is approximately 3,333 tracks — explaining why random I/O is so much slower than sequential I/O on HDDs.

15.6 NVMe and Modern Storage

Solid-state drives (SSDs), particularly those using the **NVMe** (Non-Volatile Memory Express) protocol over PCIe, have fundamentally changed the I/O landscape. Traditional disk scheduling algorithms were designed to minimise mechanical seek time — a concern that does not exist for SSDs.

15.6.1 NVMe Architecture

NVMe communicates directly over PCIe, bypassing the SATA/AHCI controller that was designed for rotational disks. Key architectural features:

NVMe Submission and Completion Queues. NVMe uses a paired queue model. Each I/O path consists of a **Submission Queue (SQ)** where the host places commands, and a **Completion Queue (CQ)** where the device places completion entries. The host writes a **doorbell register** to notify the device of new submissions. The device writes to the CQ and optionally raises an interrupt.

```
NVMe Queue Architecture:
                                                  NVMe SSD
Host CPU                                     +----------------+
+------------------+                         |                |
| Application      |                         | Flash          |
```

```
+--------+---------+                          | Controller     |
         |                                    |                |
+--------v---------+    PCIe Bus             |  +----------+  |
| NVMe Driver      |<=====================>|  | SQ Parser |  |
| +---+ +---+ +---+|                        |  +----+-----+  |
| |SQ1| |SQ2| |SQn||  Submission Queues     |       |        |
| +---+ +---+ +---+|  (in host memory)      |  +----v-----+  |
| +---+ +---+ +---+|                        |  | Command   |  |
| |CQ1| |CQ2| |CQn||  Completion Queues     |  | Execution |  |
| +---+ +---+ +---+|  (in host memory)      |  +----+-----+  |
+------------------+                         |       |        |
                                             |  +----v-----+  |
                                             |  | CQ Writer |  |
                                             |  +----------+  |
                                             +----------------+
```

NVMe supports up to 65,535 I/O queue pairs, one per CPU core or more. This eliminates the single-queue bottleneck of SATA/AHCI (which supported only 1 command queue of depth 32). A modern NVMe SSD can handle 64,000+ outstanding commands across all queues.

15.6.2 Command Queues and Parallelism

Each NVMe submission queue entry is 64 bytes and describes a single command (read, write, flush, etc.). The queue is a circular buffer in host memory:

```c
/* Simplified NVMe submission queue entry */
struct nvme_sqe {
    uint8_t  opcode;         /* Read, Write, Flush, etc. */
    uint8_t  flags;
    uint16_t command_id;     /* Unique ID for matching completions */
    uint32_t nsid;           /* Namespace ID (like a partition) */
    uint64_t reserved;
    uint64_t metadata;       /* Metadata pointer */
    uint64_t prp1;           /* Physical Region Page 1 (data pointer) */
    uint64_t prp2;           /* PRP 2 or PRP List pointer */
    uint32_t cdw10;          /* Command-specific: start LBA (low) */
    uint32_t cdw11;          /* Command-specific: start LBA (high) */
    uint32_t cdw12;          /* Command-specific: block count */
    uint32_t cdw13;
    uint32_t cdw14;
```

```
    uint32_t cdw15;
};
```

The completion queue entry (16 bytes) reports the result:

```
struct nvme_cqe {
    uint32_t result;         /* Command-specific result */
    uint32_t reserved;
    uint16_t sq_head;        /* SQ head pointer (consumed commands) */
    uint16_t sq_id;          /* SQ identifier */
    uint16_t command_id;     /* Matches the submission's command_id */
    uint16_t status;         /* Status field (success/error) */
};
```

15.6.3 Interrupt Coalescing

With SSDs capable of completing millions of I/O operations per second (IOPS), raising an interrupt for every completion would overwhelm the CPU. **Interrupt coalescing** batches multiple completions into a single interrupt:

- **Time-based:** The device raises an interrupt after a configurable time threshold (e.g., 100 μs) since the last interrupt.
- **Count-based:** The device raises an interrupt after a configurable number of completions (e.g., 16 completions) since the last interrupt.
- **Hybrid:** The first condition met triggers the interrupt.

The trade-off is latency (delay before the CPU learns of completions) versus CPU efficiency (fewer interrupts). For latency-sensitive workloads, coalescing is disabled.

15.6.4 No Mechanical Seek

Because SSDs have no moving parts, the traditional disk scheduling algorithms (FCFS, SSTF, SCAN) provide **no benefit**. Access time is independent of the address:

$$T_{\text{SSD}} \approx T_{\text{controller}} + T_{\text{flash}} \approx 10\text{–}100\,\mu\text{s}$$

The Linux kernel uses the **none** (noop) I/O scheduler for NVMe devices: requests are passed directly to the device in submission order, as the device's internal controller can optimise parallelism across flash channels better than the host.

15.7 Asynchronous I/O

Traditional Unix I/O is **synchronous**: `read()` and `write()` block the calling thread until the operation completes. For high-performance servers handling thousands of concurrent connections or I/O streams, this model is inadequate — one thread per I/O source does not scale.

15.7.1 The Scalability Problem

Consider a web server handling 10,000 concurrent connections. With synchronous I/O, it needs 10,000 threads, each blocked in `read()` most of the time. The overhead of 10,000 thread stacks (each 1–8 MB), context switches, and scheduler pressure makes this approach impractical.

Multiplexing (`select()`, `poll()`, `epoll()`) partially solves this by allowing a single thread to monitor many file descriptors. But the actual I/O operations are still synchronous: after `epoll_wait()` returns, the thread must call `read()` for each ready descriptor, and each `read()` involves a system call (user-kernel transition).

15.7.2 POSIX AIO

POSIX Asynchronous I/O (`aio_read()`, `aio_write()`) was an early attempt at asynchronous I/O. The application submits an I/O request and continues execution; the kernel signals completion later.

```c
#include <aio.h>
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    int fd = open("data.bin", O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }

    char buf[4096];
    struct aiocb cb;
    memset(&cb, 0, sizeof(cb));
    cb.aio_fildes = fd;
    cb.aio_buf = buf;
    cb.aio_nbytes = sizeof(buf);
```

```
    cb.aio_offset = 0;

    /* Submit asynchronous read */
    if (aio_read(&cb) == -1) {
        perror("aio_read");
        close(fd);
        return 1;
    }

    /* Do other work while I/O is in progress... */
    printf("I/O submitted, doing other work...\n");

    /* Wait for completion */
    while (aio_error(&cb) == EINPROGRESS) {
        /* Could do useful work here instead of busy-waiting */
        usleep(1000);
    }

    ssize_t n = aio_return(&cb);
    printf("Read %zd bytes\n", n);

    close(fd);
    return 0;
}
```

POSIX AIO has significant limitations on Linux: the glibc implementation uses a thread pool internally (each `aio_read` spawns or reuses a thread that calls synchronous `read()`), negating much of the performance benefit. The kernel's native AIO (`io_submit()`) works only with `O_DIRECT` files, limiting its applicability.

15.7.3 Event-Driven I/O: select, poll, epoll

Before io_uring, the dominant approach to handling many concurrent I/O sources was **event-driven** (or **multiplexed**) I/O. A single thread monitors many file descriptors and is notified when any becomes ready for reading or writing.

`select()` (1983). The original Unix mechanism. The caller provides three bitmasks (read, write, exception) of file descriptors to monitor. The kernel scans all file descriptors in each bitmask and returns which are ready. Limitations: maximum 1024 file descriptors (FD_SETSIZE), $O(n)$ kernel scan on every call, and the bitmasks must be rebuilt after each call.

`poll()` (1986). Replaces bitmasks with an array of `struct pollfd`, removing the 1024-descriptor limit. But the $O(n)$ kernel scan remains: the kernel must check every descriptor in the array, even if only one is ready.

`epoll()` (Linux 2.5.44, 2002). A scalable event notification mechanism that avoids the $O(n)$ scan:

epoll. A Linux-specific I/O event notification facility. An `epoll` instance maintains a set of monitored file descriptors using a red-black tree. When a file descriptor becomes ready, the kernel adds it to a **ready list** via a callback. The `epoll_wait()` call returns only the ready descriptors, in $O(k)$ time where k is the number of ready descriptors (not the total number monitored).

```c
#include <sys/epoll.h>
#include <stdio.h>
#include <unistd.h>

#define MAX_EVENTS 64

int main(void) {
    /* Create an epoll instance */
    int epfd = epoll_create1(0);

    /* Add stdin (fd 0) to the interest list */
    struct epoll_event ev = {
        .events = EPOLLIN,   /* Interested in read readiness */
        .data.fd = 0         /* User data: the fd itself */
    };
    epoll_ctl(epfd, EPOLL_CTL_ADD, 0, &ev);

    /* Wait for events (blocking) */
    struct epoll_event events[MAX_EVENTS];
    int n = epoll_wait(epfd, events, MAX_EVENTS, -1);

    for (int i = 0; i < n; i++) {
        if (events[i].events & EPOLLIN) {
            char buf[256];
            ssize_t len = read(events[i].data.fd, buf, sizeof(buf));
            if (len > 0) {
                printf("Read %zd bytes from fd %d\n", len, events[i].data.fd);
            }
```

```
        }
    }

    close(epfd);
    return 0;
}
```

Example 15.14 (Scalability of epoll vs select). A server monitors 10,000 concurrent connections. On each call:

- **`select()`:** The kernel scans 10,000 bits in each of three bitmasks. Even if only 5 connections are ready, the scan takes $O(10{,}000)$ time. Additionally, the user must rebuild the bitmasks before each call, copying 3.75 KB to/from the kernel each time.
- **`epoll_wait()`:** The kernel returns only the 5 ready descriptors. Cost: $O(5)$, independent of the total number of monitored descriptors. The interest list is persistent (no rebuild needed).

For high-concurrency servers (10,000+ connections), epoll is 100x–1000x more efficient than select/poll. This is why every modern Linux network server (nginx, Node.js, Go's netpoll) uses epoll internally.

Go's runtime uses `epoll` (on Linux) internally for all network I/O. When a goroutine calls `conn.Read()` on a network connection, the Go runtime registers the socket with the runtime's `epoll` instance, parks the goroutine, and wakes it when data arrives. From the programmer's perspective, the code looks synchronous, but the underlying I/O is fully event-driven. This is Go's "goroutines on multiplexed I/O" model — the best of both worlds.

15.7.3 Linux io_uring

io_uring, introduced in Linux 5.1 (2019) by Jens Axboe, is a modern asynchronous I/O framework that addresses all the limitations of previous approaches. It is the most significant addition to the Linux I/O stack in over a decade.

io_uring. An asynchronous I/O interface that uses two shared ring buffers between user space and the kernel: a **Submission Queue (SQ)** where the application places I/O requests, and a **Completion Queue (CQ)** where the kernel places results. The ring buffers are in shared memory, allowing both submission and completion to be performed without system calls in the fast path.

```
io_uring Architecture:
+-------------------------------------------------+
```

```
|                  User Space                      |
|                                                  |
|   Application                                    |
|   +----------+              +----------+         |
|   |  Submit  |              |  Reap    |         |
|   |  SQEs    |              |  CQEs    |         |
|   +----+-----+              +----+-----+         |
|        |                         ^                |
|        v                         |                |
|   +----+-------------------------+----+           |
|   |    Shared Memory Ring Buffers     |           |
|   | +----------+    +----------+      |           |
|   | |    SQ    |    |    CQ    |      |           |
|   | | (submit) |    | (complete)|     |           |
|   | +----+-----+    +-----+----+     |          |
|   +------|-----------------|----------+           |
+---------|-----------------|---------   ----------+
|   +-----v-----------------v-----+                |
|   |      Kernel I/O Engine      |                |
|   | (processes SQEs, posts CQEs)|                |
|   +-----------------------------+                |
|                Kernel Space                       |
+-----------------------------------------------+
```

The key innovation is that submission and completion are **lock-free** operations on shared memory. The application writes a Submission Queue Entry (SQE) to the SQ ring buffer and advances the tail pointer. The kernel reads from the head pointer. No system call is needed unless the kernel is sleeping and needs to be woken up (via `io_uring_enter()`).

```c
#include <liburing.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>

#define QUEUE_DEPTH 64
#define BLOCK_SIZE  4096

int main(void) {
    struct io_uring ring;
    int ret = io_uring_queue_init(QUEUE_DEPTH, &ring, 0);
```

```c
if (ret < 0) {
    fprintf(stderr, "io_uring_queue_init: %s\n", strerror(-ret));
    return 1;
}

int fd = open("data.bin", O_RDONLY | O_DIRECT);
if (fd < 0) { perror("open"); return 1; }

/* Allocate aligned buffer for O_DIRECT */
void *buf;
posix_memalign(&buf, BLOCK_SIZE, BLOCK_SIZE);

/* Prepare a read SQE */
struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
io_uring_prep_read(sqe, fd, buf, BLOCK_SIZE, 0);
sqe->user_data = 42;  /* Tag for matching completions */

/* Submit --- one system call for all pending SQEs */
io_uring_submit(&ring);

/* Wait for completion */
struct io_uring_cqe *cqe;
ret = io_uring_wait_cqe(&ring, &cqe);
if (ret < 0) {
    fprintf(stderr, "io_uring_wait_cqe: %s\n", strerror(-ret));
    return 1;
}

if (cqe->res < 0) {
    fprintf(stderr, "I/O error: %s\n", strerror(-cqe->res));
} else {
    printf("Read %d bytes (tag=%llu)\n", cqe->res,
           (unsigned long long)cqe->user_data);
}

/* Mark CQE as consumed */
io_uring_cqe_seen(&ring, cqe);

free(buf);
close(fd);
```

```
    io_uring_queue_exit(&ring);
    return 0;
}
```

io_uring key features:

- **Batching:** Multiple SQEs can be submitted with a single `io_uring_submit()` call, amortising system call overhead.
- **SQ Polling (SQPOLL):** A kernel thread polls the SQ for new submissions, eliminating system calls entirely. The application just writes to shared memory. This mode achieves the highest possible IOPS.
- **Linked operations:** SQEs can be chained so that the next operation starts only after the previous one completes (e.g., read then write).
- **Fixed buffers and files:** Pre-registering buffers and file descriptors eliminates per-operation kernel lookups.
- **Beyond file I/O:** io_uring supports network operations (`accept`, `connect`, `send`, `recv`), timers, and even `open`/`close`/`stat` — it is evolving into a general-purpose asynchronous syscall interface.

Example 15.9 (io_uring Performance). In benchmarks, io_uring achieves:

- **Random 4 KB reads:** Up to 1.7 million IOPS per core with SQPOLL (versus 400K IOPS with synchronous `read()` and `io_submit` with 700K IOPS).
- **System call overhead:** With SQPOLL, zero system calls in the submission path. Without SQPOLL, one `io_uring_enter()` per batch of submissions.
- **Latency:** P99 latency of 10–15 μs for NVMe reads, versus 20–40 μs for synchronous reads.

15.8 Device Drivers

A **device driver** is a kernel module that implements the interface between the kernel's abstract I/O model and the specific hardware of a particular device. Drivers are the most common source of kernel code: in the Linux kernel, `drivers/` accounts for over 60% of the total source code.

15.8.1 Driver Architecture

A Linux device driver typically consists of:

1. **Initialisation and cleanup functions:** `module_init()` and `module_exit()` register and deregister the driver.
2. **Operations structure:** A struct of function pointers (`file_operations`, `block_device_operations`, `net_device_ops`) that the kernel calls when applications perform I/O.
3. **Interrupt handler:** Services hardware interrupts from the device.
4. **Internal data structures:** Device-specific state, ring buffers, command queues.

```c
/* Minimal Linux character device driver skeleton */
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "mydev"
#define BUF_SIZE    4096

static dev_t dev_num;
static struct cdev my_cdev;
static struct class *my_class;
static char device_buffer[BUF_SIZE];
static int buffer_len;

static int mydev_open(struct inode *inode, struct file *filp) {
    pr_info("mydev: opened\n");
    return 0;
}

static ssize_t mydev_read(struct file *filp, char __user *buf,
                          size_t count, loff_t *f_pos) {
    int to_copy = min((int)count, buffer_len - (int)*f_pos);
    if (to_copy <= 0) return 0;
    if (copy_to_user(buf, device_buffer + *f_pos, to_copy))
        return -EFAULT;
    *f_pos += to_copy;
    return to_copy;
}
```

```
static ssize_t mydev_write(struct file *filp, const char __user *buf,
                           size_t count, loff_t *f_pos) {
    int to_copy = min((int)count, BUF_SIZE);
    if (copy_from_user(device_buffer, buf, to_copy))
        return -EFAULT;
    buffer_len = to_copy;
    return to_copy;
}

static int mydev_release(struct inode *inode, struct file *filp) {
    pr_info("mydev: closed\n");
    return 0;
}

static const struct file_operations mydev_fops = {
    .owner   = THIS_MODULE,
    .open    = mydev_open,
    .read    = mydev_read,
    .write   = mydev_write,
    .release = mydev_release,
};

static int __init mydev_init(void) {
    alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME);
    cdev_init(&my_cdev, &mydev_fops);
    cdev_add(&my_cdev, dev_num, 1);
    my_class = class_create(DEVICE_NAME);
    device_create(my_class, NULL, dev_num, NULL, DEVICE_NAME);
    pr_info("mydev: registered as %d:%d\n", MAJOR(dev_num), MINOR(dev_num));
    return 0;
}

static void __exit mydev_exit(void) {
    device_destroy(my_class, dev_num);
    class_destroy(my_class);
    cdev_del(&my_cdev);
    unregister_chrdev_region(dev_num, 1);
    pr_info("mydev: unregistered\n");
}
```

```
module_init(mydev_init);
module_exit(mydev_exit);
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Minimal character device driver");
```

15.8.2 Top-Half and Bottom-Half Processing

Interrupt handlers must execute quickly because they run with interrupts disabled (or at elevated priority), blocking other interrupts and delaying process scheduling. However, many interrupt-triggered tasks require significant processing: assembling network packets, updating file system metadata, signalling waiting processes.

The solution is to split interrupt handling into two halves:

Top-Half (Hard IRQ Handler). The function that runs immediately when the interrupt fires. It must be fast: acknowledge the interrupt, read essential device registers, and schedule deferred work. It runs in **interrupt context** (cannot sleep, cannot call blocking functions, cannot access user memory).

Bottom-Half (Deferred Work). Processing that can be deferred to a safer context. Bottom halves run with interrupts enabled and can perform more complex operations. Linux provides several mechanisms for bottom-half processing.

Linux bottom-half mechanisms:

1. **Softirqs:** Fixed-number, statically defined deferred functions. Used by the networking stack (`NET_TX_SOFTIRQ`, `NET_RX_SOFTIRQ`) and block layer. Softirqs can run concurrently on multiple CPUs.
2. **Tasklets:** Built on softirqs but dynamically allocatable. A given tasklet is serialised (cannot run on two CPUs simultaneously), simplifying locking. Suitable for simple per-device deferred work.
3. **Workqueues:** Deferred work that runs in process context (a kernel thread). Workqueue handlers **can sleep**, making them suitable for complex operations that may need to allocate memory, acquire mutexes, or perform I/O. The default workqueue (`system_wq`) is shared; drivers can create dedicated workqueues for performance isolation.
4. **Threaded IRQs:** A modern alternative where the interrupt handler is split into a fast primary handler (top-half) and a threaded handler that runs as a kernel thread. The `IRQF_ONESHOT` flag keeps the interrupt line masked until the threaded handler completes.

```
/* Example: threaded IRQ handler */
```

```c
static irqreturn_t my_isr_primary(int irq, void *dev_id) {
    struct my_device *dev = dev_id;
    uint32_t status = readl(dev->regs + STATUS_REG);
    if (!(status & MY_IRQ_PENDING))
        return IRQ_NONE;
    /* Acknowledge the interrupt in hardware */
    writel(status, dev->regs + STATUS_REG);
    /* Save status for the threaded handler */
    dev->irq_status = status;
    return IRQ_WAKE_THREAD;  /* Wake the threaded handler */
}

static irqreturn_t my_isr_threaded(int irq, void *dev_id) {
    struct my_device *dev = dev_id;
    /* This runs in process context --- can sleep, allocate memory, etc. */
    if (dev->irq_status & DATA_READY) {
        process_incoming_data(dev);      /* May sleep */
        wake_up(&dev->wait_queue);       /* Wake user-space readers */
    }
    return IRQ_HANDLED;
}

/* Registration */
ret = request_threaded_irq(irq, my_isr_primary, my_isr_threaded,
                           IRQF_ONESHOT, "mydev", dev);
```

Programmer's Perspective: The Linux Block Layer. When you call `read()` on a file, your request passes through a surprisingly deep stack before reaching the hardware:

```
User space:     read(fd, buf, 4096)
                    |
Kernel VFS:     vfs_read() -> file->f_op->read_iter()
                    |
File system:    ext4_file_read_iter() -> generic_file_read_iter()
                    |
Page cache:     filemap_read() -> page not cached? ->
                    |
Block layer:    submit_bio() -> blk_mq_submit_bio()
                    |
I/O scheduler:  mq-deadline / bfq / none
```

```
                    |
Device driver:  nvme_queue_rq() -> writes SQE to NVMe submission queue
                    |
Hardware:       NVMe SSD processes command, DMAs data to host memory
                    |
Completion:     NVMe CQE -> nvme_irq() -> blk_mq_complete_request()
                -> end_bio() -> unlocks page -> wakes up reader
```

The block layer uses the `bio` (block I/O) structure to represent I/O requests:

```
struct bio {
    struct block_device *bi_bdev;   /* Target device */
    unsigned int         bi_opf;    /* Operation (READ/WRITE) + flags */
    sector_t             bi_iter.bi_sector; /* Starting sector */
    struct bio_vec      *bi_io_vec; /* Scatter-gather list of pages */
    /* ... */
};
```

The `bio` structure is the universal currency of the block layer. File systems create `bios`; the I/O scheduler reorders and merges them; device drivers consume them. This clean interface means that any file system works with any block device driver — the block layer provides the abstraction boundary.

For Go programmers, the equivalent insight is that `io.Reader` is Go's `bio` — the universal unit of data flow. Just as the Linux block layer can stack encryption (dm-crypt), RAID (md), and thin provisioning (dm-thin) between a file system and a device driver by transforming `bio` objects, Go programs stack `io.Reader` implementations for compression, buffering, and decryption.

15.9 Performance Considerations

15.9.1 System Call Overhead

Every I/O operation that goes through the kernel requires at least one system call. On modern x86-64 Linux, a system call takes approximately 100–300 ns (the `syscall` instruction itself plus kernel entry/exit overhead). For a simple `read()` that hits the page cache, the system call overhead may dominate the actual work.

Strategies to reduce system call overhead:

- **Batching:** `readv()`/`writev()` (vectored I/O) perform multiple reads/writes in a single system call.

- **Memory mapping:** `mmap()` maps a file into the process's address space. Subsequent accesses are regular memory loads/stores — no system calls. Page faults bring data from disk on demand.
- **io_uring:** As discussed, eliminates system calls in the fast path.

15.9.2 Zero-Copy I/O

Traditional I/O involves multiple data copies:

```
Traditional file-to-network transfer:
1. read(file_fd, buf, n)     -> DMA: disk -> kernel buffer
                              -> CPU: kernel buffer -> user buffer
2. write(sock_fd, buf, n)    -> CPU: user buffer -> kernel buffer (socket)
                              -> DMA: kernel buffer -> NIC

Total: 4 copies, 2 system calls, 4 user-kernel transitions
```

Zero-copy techniques eliminate unnecessary copies:

- **`sendfile()`:** Transfers data directly from one file descriptor to another within the kernel:

```
#include <sys/sendfile.h>

/* Send a file over a socket without copying to user space */
off_t offset = 0;
ssize_t sent = sendfile(sock_fd, file_fd, &offset, file_size);
/* Only 2 copies: disk -> kernel buffer (DMA), kernel buffer -> NIC (DMA) */
```

- **`splice()`:** More general than `sendfile()`; moves data between a file descriptor and a pipe without copying.
- **io_uring with fixed buffers:** Pre-registered buffers avoid per-operation mapping overhead.

15.9.3 I/O Latency Hierarchy

Understanding the latency hierarchy is essential for performance engineering:

Operation	Latency	Relative
L1 cache hit	1 ns	1x
L2 cache hit	4 ns	4x

Operation	Latency	Relative
L3 cache hit	10 ns	10x
DRAM access	60–100 ns	100x
NVMe SSD read (4 KB)	10–20 μs	15,000x
SATA SSD read (4 KB)	50–100 μs	75,000x
HDD random read (4 KB)	5–10 ms	7,500,000x
Network round-trip (same DC)	0.5 ms	500,000x
Network round-trip (cross-continent)	100 ms	100,000,000x

Programmer's Perspective: Measuring I/O Performance in Go. Go provides excellent tools for benchmarking I/O performance. Here is a benchmark comparing buffered versus unbuffered reads:

```go
package iobench

import (
    "bufio"
    "io"
    "os"
    "testing"
)

func BenchmarkUnbufferedRead(b *testing.B) {
    f, err := os.Open("/dev/zero")
    if err != nil {
        b.Fatal(err)
    }
    defer f.Close()

    buf := make([]byte, 1)
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        f.Read(buf) // One system call per byte
    }
}
```

```go
func BenchmarkBufferedRead(b *testing.B) {
    f, err := os.Open("/dev/zero")
    if err != nil {
        b.Fatal(err)
    }
    defer f.Close()

    r := bufio.NewReaderSize(f, 4096) // Buffer 4 KB at a time
    buf := make([]byte, 1)
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        r.Read(buf) // System call only every 4096 bytes
    }
}

func BenchmarkReadAll(b *testing.B) {
    for i := 0; i < b.N; i++ {
        f, err := os.Open("/dev/zero")
        if err != nil {
            b.Fatal(err)
        }
        // Read exactly 1 MB
        _, err = io.CopyN(io.Discard, f, 1024*1024)
        f.Close()
        if err != nil {
            b.Fatal(err)
        }
    }
}
```

Typical results on a modern system:

```
BenchmarkUnbufferedRead    5000000    230 ns/op   (system call per byte)
BenchmarkBufferedRead     50000000     25 ns/op   (system call per 4096 bytes)
BenchmarkReadAll              2000   650000 ns/op  (1 MB via io.Copy)
```

The buffered read is approximately 10x faster per byte because it amortises system call overhead. This is why Go's `bufio` package exists: it is the user-space equivalent of the kernel's buffer cache. Always wrap raw `os.File` readers in `bufio.Reader` for byte-at-a-time or line-at-a-time processing.

15.10 The Linux I/O Scheduler in Detail

15.10.1 Multi-Queue Block Layer (blk-mq)

Since Linux 3.13, the block layer uses the **multi-queue** (blk-mq) architecture, which replaces the single request queue of the legacy block layer with per-CPU software queues and per-hardware-queue dispatch queues:

```
blk-mq Architecture:

CPU 0        CPU 1        CPU 2        CPU 3
  |            |            |            |
  v            v            v            v
[SW Queue 0] [SW Queue 1] [SW Queue 2] [SW Queue 3]
  |            |            |            |
  +-----+------+-----+------+-----+------+
        |            |            |
        v            v            v
   [HW Queue 0] [HW Queue 1] [HW Queue 2]
        |            |            |
        +------+-----+------+-----+
               |
        Device Driver
```

Software queues are per-CPU and lockless (each CPU enqueues requests to its own queue without contention). Hardware queues map to the device's actual command queues (e.g., NVMe submission queues). The I/O scheduler sits between the software and hardware queues.

15.10.2 Available I/O Schedulers

Linux provides three blk-mq I/O schedulers:

`none` (noop). Requests are passed directly from software queues to hardware queues in FIFO order. No reordering, no merging. This is the default for NVMe devices, where the device's internal parallelism handles optimisation better than the host.

`mq-deadline`. Maintains two sorted queues (one for reads, one for writes) and two deadline queues. Requests are normally dispatched in sector order (SCAN-like), but any request that has waited longer than a deadline (default: 500 ms for reads, 5 s for writes) is promoted. Reads are given priority over writes because reads are typically synchronous (a process is waiting) while writes are typically asynchronous (buffered by the page cache).

`bfq` (Budget Fair Queueing). A proportional-share scheduler that assigns each process a "budget" of sectors. Processes that perform sequential I/O get larger budgets (because sequential access is efficient). BFQ provides excellent interactive performance (low latency for desktop applications) but has higher CPU overhead than mq-deadline.

Example 15.10 (Changing the I/O Scheduler). The scheduler can be changed at runtime via sysfs:

```
# View available and current scheduler for /dev/sda
cat /sys/block/sda/queue/scheduler
# [mq-deadline] bfq none

# Switch to BFQ
echo bfq > /sys/block/sda/queue/scheduler

# View scheduler parameters
ls /sys/block/sda/queue/iosched/
# fifo_expire_async  fifo_expire_sync  read_expire  write_expire  ...

# Tune mq-deadline: reduce read deadline to 100ms
echo 100 > /sys/block/sda/queue/iosched/read_expire
```

15.10.3 I/O Priorities and cgroups

Linux supports I/O priorities through the `ionice` utility and the `ioprio_set()` system call. Three scheduling classes are available:

- **Real-time (class 1):** Highest priority. The process gets first access to the disk. Eight priority levels (0–7). Only available to root.
- **Best-effort (class 2):** Default class. Eight priority levels. Processes at the same level share the disk fairly.
- **Idle (class 3):** Lowest priority. The process gets disk time only when no other process needs it. Useful for background tasks (backup, indexing).

For containerised workloads, the **blkio cgroup** controller provides per-group I/O bandwidth limits and weight-based scheduling:

```
# Limit a cgroup to 50 MB/s read and 20 MB/s write on /dev/sda
echo "8:0 rbps=52428800 wbps=20971520" > \
    /sys/fs/cgroup/my_container/io.max
```

```
# Set relative weight (100-10000, default 100)
echo "8:0 weight=500" > /sys/fs/cgroup/my_container/io.bfq.weight
```

15.11 Power Management and I/O

Modern I/O devices support multiple power states to reduce energy consumption:

15.11.1 Device Power States

Device Power States. ACPI (Advanced Configuration and Power Interface) defines device power states D0 through D3:

- **D0:** Fully operational. Maximum power consumption, zero wake-up latency.
- **D1/D2:** Intermediate low-power states. Partial functionality preserved. Wake-up latency: microseconds to milliseconds.
- **D3hot:** Device powered but non-functional. Context may be preserved. Wake-up latency: milliseconds.
- **D3cold:** Device fully powered off. All context lost. Wake-up latency: seconds.

For storage devices:

- **HDD standby:** The platters stop spinning. Saves 5–10 W but incurs a 5–15 second spin-up delay.
- **SSD power states:** NVMe defines power states PS0 (active) through PS4 (deepest sleep). PS0 consumes 5–25 W; PS4 consumes 2–5 mW. Transitions take microseconds to milliseconds. The **Autonomous Power State Transition (APST)** feature allows the drive to enter low-power states automatically after a configurable idle timeout.

15.11.2 I/O Impact on System Power

On a modern laptop, the storage subsystem accounts for 5–15% of total power consumption during active use and can be the dominant consumer during idle periods (if the system fails to enter a low-power state). Key strategies:

- **Aggressive PM timeouts:** Spin down HDDs after 60–120 seconds of inactivity.
- **Write coalescing:** Buffer writes and flush in batches rather than writing individually (each write resets the idle timer).
- **Read-ahead tuning:** Aggressive read-ahead reduces the number of future I/O operations (and future wake-ups).

- **Avoid unnecessary I/O:** Mount with `noatime` to prevent read operations from triggering writes (access time updates).

Example 15.11 (NVMe Power State Calculation). An NVMe SSD supports 5 power states:

State	Power	Entry Latency	Exit Latency
PS0	25 W	–	–
PS1	8 W	5 μs	10 μs
PS2	5 W	50 μs	100 μs
PS3	30 mW	2 ms	5 ms
PS4	4 mW	10 ms	25 ms

With APST configured: idle $\rightarrow$ PS1 after 100 ms, PS1 $\rightarrow$ PS3 after 2 s, PS3 $\rightarrow$ PS4 after 30 s.

During a typical desktop workload (active 20% of the time, idle 80%), the average power is:

$$P_{\text{avg}} = 0.20 \times 25\,\text{W} + 0.10 \times 8\,\text{W} + 0.10 \times 5\,\text{W} + 0.30 \times 0.03\,\text{W} + 0.30 \times 0.004\,\text{W}$$

$$= 5.0 + 0.8 + 0.5 + 0.009 + 0.0012 \approx 6.31\,\text{W}$$

Without power management (always PS0): $P_{\text{avg}} = 25\,\text{W}$. APST reduces SSD power by 75%.

15.12 Error Handling in I/O

15.12.1 Error Classification

I/O errors fall into several categories that require different handling strategies:

- **Transient errors:** Temporary conditions that may succeed on retry (bus contention, CRC mismatch due to noise, timeout). The kernel automatically retries most transient errors (typically 3–5 times).

- **Media errors:** Permanent damage to the storage medium (bad sector on HDD, worn-out NAND page on SSD). The drive's firmware may transparently remap the sector to a spare area. If remapping fails, the error is reported to the kernel.
- **Protocol errors:** Violations of the communication protocol (malformed response, unexpected state). Often indicate a firmware bug or hardware failure. May require a device reset.
- **Path errors:** Failures in the communication path (cable, controller, bus). Multi-path configurations (common in enterprise storage) can route I/O through an alternate path.

15.12.2 Error Propagation

When a block device reports an error, the error propagates up the I/O stack:

```
Error propagation:
NVMe controller -> NVMe driver -> block layer -> file system -> VFS -> user space

At each layer:
- NVMe driver:    Decode the NVMe status code, log the error,
                   retry if transient, report to block layer
- Block layer:    Mark the bio as failed, invoke the bio completion callback
- File system:    ext4 may attempt to read from a mirror or journal backup;
                   marks the file system as having errors (remount read-only if critica
- VFS:            Returns -EIO to the user-space read()/write() call
- User space:     Application receives errno = EIO
```

I/O Error Handling Policy. The file system's response to I/O errors is configurable. ext4 provides three error modes via mount options:

- `errors=continue` — Log the error but continue operating. Risky: further operations may corrupt the file system.
- `errors=remount-ro` — Remount the file system read-only. Prevents further damage but disrupts service. (Default for ext4.)
- `errors=panic` — Halt the system immediately. Appropriate for critical servers where any inconsistency is unacceptable and a controlled restart is preferred.

15.12.3 Timeout Handling

Every I/O operation must have a timeout. Without timeouts, a hung device can block a thread indefinitely, eventually exhausting system resources. The Linux block layer en-

forces a default timeout (typically 30 seconds for SCSI/SATA, configurable via sysfs):

```
# View and set the timeout for /dev/sda (in seconds)
cat /sys/block/sda/device/timeout
# 30

# Increase timeout for slow devices (e.g., tape drives)
echo 120 > /sys/block/sda/device/timeout
```

When a timeout expires, the block layer invokes the driver's error handler, which may attempt a device reset, abort the command, or report the error to the upper layers. The NVMe specification defines a **Controller Fatal Status** bit; if set, the driver performs a full controller reset and replays all outstanding commands.

The I/O subsystem is the most hardware-dependent part of the operating system, yet its layered architecture — from device registers and interrupts at the bottom, through DMA and the block layer in the middle, to the VFS and io_uring at the top — provides clean abstractions that isolate complexity at each level. Understanding these layers is essential for diagnosing performance bottlenecks, writing device drivers, and designing high-throughput applications.

Exercises

Exercise 15.1. A system has a device with a status register at port 0x300 and a data register at port 0x301. The status register's bit 0 is set when data is available. Write a polling-based C function that reads n bytes from the device into a buffer. Then write an interrupt-driven version using a ring buffer and a wait queue. Compare the CPU utilisation of both approaches when the device produces data at a rate of 1000 bytes per second and the CPU runs at 3 GHz.

Exercise 15.2. A disk has 5000 cylinders (0–4999). The disk arm is currently at cylinder 2150, moving towards cylinder 0. The pending request queue is: 2069, 1212, 2296, 2800, 544, 1618, 356, 1523, 4965, 3681. Calculate the total seek distance for each of the following algorithms: (a) FCFS, (b) SSTF, (c) SCAN, (d) C-SCAN, (e) LOOK, (f) C-LOOK. Which algorithm gives the smallest total seek distance?

Exercise 15.3. Explain why DMA requires the use of **physical addresses** rather than virtual addresses. Describe the role of the **IOMMU** (I/O Memory Management Unit) in modern systems. What security vulnerability would exist if devices could DMA to arbitrary physical addresses without an IOMMU?

Exercise 15.4. An NVMe SSD has 8 I/O queue pairs, each with a queue depth of 1024. The SSD can sustain 500,000 random read IOPS at a queue depth of 32. (a) What is the maximum theoretical IOPS with all queues fully loaded? (b) Why does increasing queue depth beyond a certain point not increase IOPS? (c) Calculate the average I/O latency at 500,000 IOPS with queue depth 32 using Little's Law: $L = \lambda W$ where L is the average number of requests in the system, λ is the arrival rate, and W is the average waiting time.

Exercise 15.5. io_uring uses shared ring buffers between user space and the kernel. (a) Explain how the SQ tail pointer and SQ head pointer coordinate between user space (producer) and kernel (consumer) without locks. (b) What happens if user space writes SQEs faster than the kernel can process them? (c) Why does io_uring's SQPOLL mode require elevated privileges (or `CAP_SYS_NICE`)?

Exercise 15.6. In the split interrupt model (top-half/bottom-half), a network card driver receives a packet. The top-half reads the packet from the device's DMA buffer and queues it. The bottom-half (softirq) runs the TCP/IP stack processing. (a) Why cannot the TCP/IP processing run in the top-half? (b) What happens if softirqs are deferred for too long? (c) Linux's NAPI (New API) mechanism switches between interrupt-driven and polling modes based on packet rate. Explain the rationale: when should the driver use interrupts, and when should it poll?

Exercise 15.7. Design a simple block I/O scheduler in pseudocode that implements the LOOK algorithm with **deadline-based aging**. The scheduler should maintain a sorted queue of requests and service them in LOOK order, but any request that has waited longer than a deadline D milliseconds should be promoted to the front of the queue. Analyse the worst-case latency with and without the deadline mechanism.

Chapter 16: Storage Reliability

"There are two kinds of hard drives: those that have failed, and those that will fail."
— Storage engineering proverb

16.1 Disk Structure and Geometry

Before analysing reliability, we must understand the physical structure of storage devices. Although modern drives abstract their internal geometry behind a Logical Block Addressing (LBA) interface, knowledge of the underlying structure is essential for understanding failure modes, RAID design, and performance characteristics.

16.1.1 Hard Disk Drive (HDD) Anatomy

A hard disk drive consists of one or more rotating **platters** coated with a magnetic material. Data is read and written by **heads** that float on a thin air cushion above the platter surface (the flying height is approximately 3–5 nanometres — less than the diameter of a smoke particle).

Disk Geometry Terms.

- **Platter:** A circular disk coated with magnetic material. Each platter has two **surfaces** (top and bottom).
- **Track:** A concentric ring of data on a single surface. A typical modern drive has hundreds of thousands of tracks per surface.
- **Sector:** The smallest unit of data that can be read or written. Traditional sector size is 512 bytes; modern drives use 4 KB **Advanced Format** sectors.
- **Cylinder:** The set of tracks at the same radial position across all platters. All tracks in a cylinder can be accessed without moving the disk arm.
- **Head:** The read/write transducer. One head per surface. All heads are mounted on a single **actuator arm** and move together.

```
Hard Disk Drive Structure (side view):

    Actuator arm
```

```
    +===========+
    |  Head 0 --+------->  Surface 0 (Platter 1, top)
    |  Head 1 --+------->  Surface 1 (Platter 1, bottom)
    |  Head 2 --+------->  Surface 2 (Platter 2, top)
    |  Head 3 --+------->  Surface 3 (Platter 2, bottom)
    +===========+
         |
    Pivot point        Spindle motor
                       (7200/10000/15000 RPM)


Top view (single platter):

    +------- Track 0 (outermost) --------+
    |   +---- Track 1 ----+              |
    |   |  +- Track 2 -+  |              |
    |   |  |            |  |              |
    |   |  |  Spindle   |  |              |
    |   |  |     *      |  |              |
    |   |  |            |  |              |
    |   |  +------------+  |              |
    |   +------------------+              |
    +-------------------------------------+

    Each track is divided into sectors:
    Sector 0, Sector 1, ..., Sector N
```

16.1.2 Logical Block Addressing (LBA)

Modern drives present a simple linear array of logical blocks (LBA 0, LBA 1, ..., LBA $n-1$) to the operating system. The drive's firmware maps each LBA to the appropriate physical location (platter, track, sector), handling complexities such as:

- **Zone Bit Recording (ZBR):** Outer tracks are physically longer and contain more sectors than inner tracks. The drive's firmware accounts for the varying number of sectors per track.
- **Spare sectors:** The drive reserves spare sectors to transparently remap sectors that develop defects (via the **grown defect list**).
- **Track skew and cylinder skew:** Sectors are staggered across adjacent tracks and cylinders to account for head switching and seek time, allowing sequential

reads to proceed without missing a revolution.

Logical Block Addressing (LBA). A addressing scheme in which each sector on the drive is assigned a sequential integer address from 0 to $n - 1$, where n is the total number of sectors. The drive's firmware translates LBAs to physical locations. This abstraction decouples the operating system from the drive's physical geometry.

16.1.3 Solid-State Drive (SSD) Structure

An SSD replaces the mechanical components of an HDD with NAND flash memory. The internal structure is hierarchical:

```
SSD Internal Structure:

Controller (ARM/RISC processor)
    |
    +--- Channel 0          Channel 1       ...  Channel N
         |                   |
         +--- Die 0         +--- Die 0
         |    +--- Plane 0  |    +--- Plane 0
         |    |    +- Block  |    |    +- Block
         |    |    |  +Page  |    |    |  +Page (4-16 KB)
         |    |    |  +Page  |    |    |  +Page
         |    |    |  +Page  |    |    |  ...
         |    |    +- Block  |    |    +- Block
         |    +--- Plane 1  |    +--- Plane 1
         +--- Die 1         +--- Die 1
```

NAND Flash Hierarchy.

- **Page:** The smallest unit of read and write (4–16 KB). Data can only be written to a **clean** (erased) page.
- **Block:** A group of pages (64–512 pages, i.e., 256 KB – 8 MB). The smallest unit of **erasure**. An entire block must be erased before any of its pages can be rewritten.
- **Die:** Contains multiple planes. Each plane has its own page buffer, enabling parallel operations.
- **Channel:** A communication bus between the controller and one or more dies. Independent channels enable concurrent access.

The asymmetry between write granularity (page) and erase granularity (block) is the fundamental source of complexity in SSD design, giving rise to write amplification,

garbage collection, and wear levelling — topics we address later in this chapter.

16.1.4 NAND Cell Types and Reliability

NAND flash cells store data by trapping electrons in a floating gate (or charge trap). The number of bits per cell determines the cell type and directly affects both capacity and reliability:

NAND Cell Types.

- **SLC (Single-Level Cell):** 1 bit per cell. Two voltage levels (0/1). Highest endurance (50,000–100,000 P/E cycles), fastest read/write, but lowest density and highest cost per bit.
- **MLC (Multi-Level Cell):** 2 bits per cell. Four voltage levels (00/01/10/11). Endurance: 3,000–10,000 P/E cycles. Good balance of cost and performance.
- **TLC (Triple-Level Cell):** 3 bits per cell. Eight voltage levels. Endurance: 500–3,000 P/E cycles. Most common in consumer SSDs.
- **QLC (Quad-Level Cell):** 4 bits per cell. Sixteen voltage levels. Endurance: 100–1,000 P/E cycles. Lowest cost per bit, suitable for read-heavy workloads.

The reliability challenge increases with each additional bit per cell because the voltage margins between adjacent levels shrink:

Example 16.19 (Voltage Margin Comparison). Suppose the NAND cell operates with a voltage range of 0–4 V:

- **SLC:** 2 levels. Voltage margin between levels: $4/2 = 2\,\text{V}$. Large margin, easy to distinguish.
- **MLC:** 4 levels. Margin: $4/4 = 1\,\text{V}$.
- **TLC:** 8 levels. Margin: $4/8 = 0.5\,\text{V}$.
- **QLC:** 16 levels. Margin: $4/16 = 0.25\,\text{V}$.

As the margin shrinks, the cell becomes more susceptible to noise, charge leakage, and read disturb errors. This is why QLC NAND requires more sophisticated ECC (LDPC codes correcting hundreds of errors per page) compared to SLC (simple BCH codes correcting a few errors).

The read latency also increases with cell density because the controller must use more precise (and slower) voltage sensing:

Cell Type	Read Latency	Write Latency	Erase Latency
SLC	25 μs	200 μs	1.5 ms
MLC	50 μs	600 μs	3 ms
TLC	75 μs	1.5 ms	5 ms
QLC	100–150 μs	5 ms	10 ms

Modern SSDs use **SLC caching**: a portion of the NAND is operated in SLC mode (writing only 1 bit per cell) to absorb burst writes at high speed. When the SLC cache fills, data is folded into the TLC/QLC region in the background. This is why SSD write speed often drops sharply after writing continuously for several tens of gigabytes — the SLC cache has been exhausted.

16.2 RAID: Redundant Arrays of Independent Disks

Individual drives fail. Enterprise environments require storage that survives drive failures without data loss or significant downtime. **RAID** provides this by combining multiple drives into an array that appears as a single logical volume, with built-in redundancy to tolerate failures.

RAID (Redundant Array of Independent Disks). A technique for combining multiple physical disk drives into a logical unit for the purpose of improved performance, reliability, or both. Originally defined by Patterson, Gibson, and Katz at UC Berkeley in 1988, RAID distributes data and redundancy information across the member drives according to a specific **RAID level**.

16.2.1 RAID 0: Striping

RAID 0 distributes data across N drives in fixed-size **stripes** without any redundancy.

```
RAID 0 (4 drives, stripe size = 64 KB):

         Drive 0    Drive 1    Drive 2    Drive 3
Block 0  [Stripe 0] [Stripe 1] [Stripe 2] [Stripe 3]
Block 1  [Stripe 4] [Stripe 5] [Stripe 6] [Stripe 7]
Block 2  [Stripe 8] [Stripe 9] [Stripe 10][Stripe 11]
...
```

Capacity: $N \times C$ where C is the capacity of the smallest drive.

Performance: Read and write throughput scales linearly with N, since I/O operations can be parallelised across all drives.

Reliability: Worse than a single drive. If **any** drive fails, all data is lost.

Theorem 16.1 (RAID 0 Reliability). For N independent drives, each with failure probability p over a given time period, the probability that a RAID 0 array survives the period is:

$$P(\text{RAID 0 survives}) = (1-p)^N$$

For $N = 4$ drives with annual failure rate $p = 0.02$ (2%):

$$P(\text{survives 1 year}) = (0.98)^4 = 0.922$$

The probability of data loss in one year is $1 - 0.922 = 7.8\%$, nearly four times the single-drive failure rate.

RAID 0 is appropriate only for data that can be easily regenerated (temporary files, caches, scratch space) and where performance is paramount.

16.2.2 RAID 1: Mirroring

RAID 1 maintains identical copies of all data on two (or more) drives. Every write is performed on all drives simultaneously.

```
RAID 1 (2 drives):

         Drive 0    Drive 1
Block 0  [Data A]   [Data A]    <- identical copies
Block 1  [Data B]   [Data B]
Block 2  [Data C]   [Data C]
...
```

Capacity: C (the capacity of one drive) regardless of how many mirrors exist. Usable capacity is $C \times N/2$ for N-drive mirrors, but typically $N = 2$.

Performance: Read throughput can be doubled (different reads go to different drives). Write throughput is limited by the slowest drive (both must complete).

Reliability: The array survives the failure of any one drive (or $N - 1$ drives in an N-way mirror).

Theorem 16.2 (RAID 1 Reliability). For a two-drive mirror with independent failure probability p:

$$P(\text{data loss}) = p^2$$

For $p = 0.02$: $P(\text{data loss}) = 0.0004 = 0.04\%$ per year.

More precisely, considering that the second drive can fail **during the rebuild period** after the first failure:

$$P(\text{data loss}) = p \times \frac{T_{\text{rebuild}}}{\text{MTBF}_2} = p \times \frac{T_{\text{rebuild}} \times p_2}{T_{\text{period}}}$$

where T_{rebuild} is the time to rebuild onto a replacement drive and MTBF_2 is the mean time between failures of the second drive.

16.2.3 RAID 5: Distributed Parity

RAID 5 stripes data and parity across $N \geq 3$ drives. For each stripe, one drive holds the **parity** block (the XOR of the corresponding data blocks on the other drives). The parity role rotates across drives to distribute the parity write load.

```
RAID 5 (4 drives):

         Drive 0    Drive 1    Drive 2    Drive 3
Row 0    [D0]       [D1]       [D2]       [P0]       P0 = D0 XOR D1 XOR D2
Row 1    [D3]       [D4]       [P1]       [D5]       P1 = D3 XOR D4 XOR D5
Row 2    [D6]       [P2]       [D7]       [D8]       P2 = D6 XOR D7 XOR D8
Row 3    [P3]       [D9]       [D10]      [D11]      P3 = D9 XOR D10 XOR D11
```

Capacity: $(N-1) \times C$. One drive's worth of capacity is consumed by parity.

Parity calculation. The parity block is the bitwise XOR of all data blocks in the stripe:

$$P = D_0 \oplus D_1 \oplus D_2 \oplus \cdots \oplus D_{N-2}$$

If drive k fails, its data can be reconstructed by XORing the remaining data and parity:

$$D_k = D_0 \oplus \cdots \oplus D_{k-1} \oplus D_{k+1} \oplus \cdots \oplus D_{N-2} \oplus P$$

This works because XOR is its own inverse: $x \oplus x = 0$ and $x \oplus 0 = x$.

Example 16.1 (RAID 5 Parity Reconstruction). Three data blocks (4 bits each):

$D_0 = 1010$, $D_1 = 1100$, $D_2 = 0111$

Parity: $P = 1010 \oplus 1100 \oplus 0111 = 0001$

If D_1 fails, reconstruct: $D_1 = D_0 \oplus D_2 \oplus P = 1010 \oplus 0111 \oplus 0001 = 1100$. This matches the original value of D_1.

Example 16.2 (RAID 5 Write Penalty). Updating a single data block D_k on RAID 5 requires:

1. Read the old value of D_k (1 read).
2. Read the old parity P (1 read).
3. Compute new parity: $P' = P \oplus D_k^{\text{old}} \oplus D_k^{\text{new}}$ (CPU operation).
4. Write the new D_k (1 write).
5. Write the new P' (1 write).

Total: 2 reads + 2 writes = **4 I/O operations** for a single logical write. This is the **RAID 5 write penalty**.

An alternative **reconstruct write** reads all other data blocks in the stripe and computes the parity from scratch: $(N-2)$ reads + 2 writes. For small N, the read-modify-write approach (4 I/O) is cheaper.

16.2.4 RAID 6: Double Parity

RAID 6 extends RAID 5 with a second, independent parity block, allowing the array to survive any **two** simultaneous drive failures.

```
RAID 6 (5 drives):

         Drive 0    Drive 1    Drive 2    Drive 3    Drive 4
Row 0    [D0]       [D1]       [D2]       [P]        [Q]
Row 1    [D3]       [D4]       [P]        [Q]        [D5]
...

P = XOR parity (same as RAID 5)
Q = Reed-Solomon parity (using Galois Field GF(2^8) arithmetic)
```

The second parity Q uses a different mathematical function from P. Typically, Q is computed using multiplication in Galois Field $GF(2^8)$:

$$Q = g^0 \cdot D_0 \oplus g^1 \cdot D_1 \oplus g^2 \cdot D_2 \oplus \cdots$$

where g is a generator of the field and $\oplus$ denotes XOR. This ensures that P and Q provide two independent equations, sufficient to solve for two unknowns (two failed drives).

Capacity: $(N-2) \times C$.

Reliability: Survives any two drive failures. This is increasingly important as drive capacities grow, because rebuild times lengthen (a 16 TB drive takes 10–20 hours to rebuild), and the probability of a second failure during rebuild is non-negligible.

16.2.5 RAID 10: Mirrored Stripes

RAID 10 combines RAID 1 (mirroring) and RAID 0 (striping). Data is first mirrored (RAID 1), and the mirrors are then striped (RAID 0). This requires $N \geq 4$ drives (always an even number).

```
RAID 10 (4 drives):

         Drive 0    Drive 1    Drive 2    Drive 3
         Mirror 0   Mirror 0   Mirror 1   Mirror 1
Block 0  [Stripe 0] [Stripe 0] [Stripe 1] [Stripe 1]
Block 1  [Stripe 2] [Stripe 2] [Stripe 3] [Stripe 3]
...
```

Capacity: $N/2 \times C$ (50% usable).

Performance: Excellent for both reads (up to $N\times$ throughput) and writes (up to $N/2\times$ throughput, since each write goes to two drives).

Reliability: Survives any single drive failure. Can survive multiple failures as long as no mirror pair loses both drives simultaneously.

16.2.6 RAID Level Comparison

Level	Min Drives	Capacity	Read Speed	Write Speed	Fault Tolerance
0	2	NC	$N\times$	$N\times$	None
1	2	C	$2\times$	$1\times$	1 drive
5	3	$(N-1)C$	$(N-1)\times$	Reduced (write penalty)	1 drive

Level	Min Drives	Capacity	Read Speed	Write Speed	Fault Tolerance
6	4	$(N-2)C$	$(N-2)\times$	Reduced (higher penalty)	2 drives
10	4	$NC/2$	$N\times$	$N/2\times$	1 per mirror

Programmer's Perspective: mdadm RAID Management on Linux. Linux provides software RAID through the `md` (multiple devices) subsystem, managed with the `mdadm` utility. Here is how to create and manage RAID arrays from the command line:

```
# Create a RAID 5 array with 4 drives
mdadm --create /dev/md0 --level=5 --raid-devices=4 \
    /dev/sdb /dev/sdc /dev/sdd /dev/sde

# Check array status
cat /proc/mdstat
# md0 : active raid5 sde[3] sdd[2] sdc[1] sdb[0]
#       3145728 blocks super 1.2 level 5, 512k chunk, algorithm 2 [4/4] [UUUU]

# View detailed array information
mdadm --detail /dev/md0

# If /dev/sdc fails, mark it and remove it
mdadm /dev/md0 --fail /dev/sdc
mdadm /dev/md0 --remove /dev/sdc

# Add a replacement drive and rebuild
mdadm /dev/md0 --add /dev/sdf
# Rebuild progress visible in /proc/mdstat

# Monitor the array for failures (sends email alerts)
mdadm --monitor --daemonise --mail=admin@example.com /dev/md0

# Save configuration for boot-time assembly
mdadm --detail --scan >> /etc/mdadm/mdadm.conf
```

For ZFS, which integrates the file system and volume manager:

```
# Create a RAID-Z1 pool (equivalent to RAID 5)
zpool create tank raidz1 /dev/sdb /dev/sdc /dev/sdd /dev/sde

# Create a RAID-Z2 pool (equivalent to RAID 6)
zpool create tank raidz2 /dev/sdb /dev/sdc /dev/sdd /dev/sde /dev/sdf

# Check pool status and health
zpool status tank
#   pool: tank
#  state: ONLINE
# config:
#     NAME        STATE     READ WRITE CKSUM
#     tank        ONLINE       0     0     0
#       raidz1-0  ONLINE       0     0     0
#         sdb     ONLINE       0     0     0
#         sdc     ONLINE       0     0     0
#         sdd     ONLINE       0     0     0
#         sde     ONLINE       0     0     0

# Replace a failed drive
zpool replace tank /dev/sdc /dev/sdf

# Run a scrub (verify all checksums, repair if possible)
zpool scrub tank

# Check scrub progress
zpool status tank
# scan: scrub in progress, 45.2% done
```

ZFS is preferable to mdadm + ext4/XFS for critical data because it provides end-to-end checksumming. With mdadm RAID, a silent corruption in one data block is indistinguishable from valid data — the parity check passes because the parity was computed from the corrupted data. ZFS detects this through its independent checksums and can repair it from the redundant copy.

16.3 Reliability Analysis

16.3.1 The Bathtub Curve

Drive failure rates are not constant over a drive's lifetime. They follow the well-known **bathtub curve**:

Bathtub Curve. The failure rate function $h(t)$ of a population of drives typically exhibits three phases:

1. **Infant mortality (early life):** Elevated failure rate due to manufacturing defects, weak components, and firmware bugs. Duration: 0–6 months. Failures are screened out by manufacturer burn-in testing, but some escape to the field.
2. **Useful life (middle):** Approximately constant failure rate. The exponential distribution model ($h(t) = \lambda$) is valid during this phase. Duration: 6 months to 3–5 years.
3. **Wear-out (end of life):** Increasing failure rate due to mechanical wear (HDD bearing degradation, spindle motor failure) or NAND cell exhaustion (SSD). The Weibull distribution with shape parameter $\beta > 1$ models this phase.

The Weibull reliability function captures all three phases:

$$R(t) = e^{-(t/\eta)^\beta}$$

where η is the scale parameter (characteristic life) and β is the shape parameter:

- $\beta < 1$: Decreasing failure rate (infant mortality)
- $\beta = 1$: Constant failure rate (exponential distribution, useful life)
- $\beta > 1$: Increasing failure rate (wear-out)

Example 16.16 (Real-World Drive Failure Data). Backblaze, a cloud storage provider, publishes quarterly drive failure statistics from its fleet of over 250,000 drives. Key findings from their data (2013–2025):

- The fleet-wide AFR averages 1.5–2.0% across all drive models.
- Some models have AFR below 0.5%; others exceed 5%.
- Failure rates are highest in the first year (infant mortality) and after year 4 (wear-out).
- Drive failures are correlated: when one drive in a batch fails, others from the same batch are more likely to fail soon.
- Temperature has a measurable but modest effect on failure rate; vibration and workload have a stronger effect.

These observations challenge the assumption of independent, identically distributed failures that underlies simple MTTF calculations. In practice, correlated failures make RAID less reliable than the theoretical formulas predict.

16.3.2 Mean Time To Failure (MTTF)

Mean Time To Failure (MTTF). The expected time until a non-repairable component fails, assuming it starts in a working state. For a component with constant failure rate λ (failures per unit time):

$$\text{MTTF} = \frac{1}{\lambda}$$

The **Annualised Failure Rate (AFR)** is the probability of failure within one year:

$$\text{AFR} \approx \frac{1}{\text{MTTF (in years)}}$$

for small failure probabilities (where the approximation $1 - e^{-\lambda t} \approx \lambda t$ holds).

Manufacturers typically quote MTTF in hours. For example, an enterprise HDD with MTTF = 1,200,000 hours corresponds to:

$$\text{AFR} \approx \frac{8760}{1{,}200{,}000} = 0.73\%$$

Example 16.3 (MTTF for a RAID Array). A RAID 5 array has 4 drives, each with MTTF = 500,000 hours. The array has $\text{MTTF}_{\text{group}}$ for the first failure:

$$\text{MTTF}_{\text{first failure}} = \frac{\text{MTTF}_{\text{drive}}}{N} = \frac{500{,}000}{4} = 125{,}000 \text{ hours} \approx 14.3 \text{ years}$$

This is the expected time until the array enters a **degraded** state (one drive failed, parity providing data from the missing drive).

16.3.3 Mean Time To Repair (MTTR)

Mean Time To Repair (MTTR). The expected time from a component failure to full restoration of redundancy. For a RAID array, MTTR includes:

1. **Detection time:** How long until the failure is noticed (seconds for active monitoring, hours or days for unmonitored systems).

2. **Response time:** Time to obtain and install a replacement drive.
3. **Rebuild time:** Time to reconstruct the failed drive's data on the replacement.

Rebuild time is a function of drive capacity and rebuild speed:

$$T_{\text{rebuild}} = \frac{\text{Drive capacity}}{\text{Rebuild speed}}$$

For a 16 TB drive at a rebuild speed of 200 MB/s (limited by the remaining drives' throughput):

$$T_{\text{rebuild}} = \frac{16 \times 10^{12}}{200 \times 10^{6}} = 80{,}000 \text{ seconds} \approx 22 \text{ hours}$$

16.3.4 RAID Reliability Calculation

Theorem 16.3 (RAID 5 MTTF). For a RAID 5 array with N drives, each with failure rate $\lambda = 1/\text{MTTF}_{\text{drive}}$, the mean time to data loss is:

$$\text{MTTF}_{\text{RAID 5}} = \frac{\text{MTTF}_{\text{drive}}^2}{N \times (N-1) \times \text{MTTR}}$$

Proof. The first failure occurs at rate $N\lambda$ (any of N drives). After the first failure, the array is in a degraded state for an expected duration of MTTR. During this period, if any of the remaining $N-1$ drives fails, data is lost. The probability of a second failure during MTTR is approximately $(N-1) \times \lambda \times \text{MTTR}$. Therefore:

$$\text{MTTF}_{\text{RAID 5}} = \frac{1}{N\lambda} \times \frac{1}{(N-1)\lambda \times \text{MTTR}} = \frac{1}{N(N-1)\lambda^2 \times \text{MTTR}} = \frac{\text{MTTF}^2}{N(N-1) \times \text{MTTR}}$$

□

Example 16.4 (RAID 5 vs. RAID 6 Reliability). Consider an array of 6 drives, each with MTTF = 1,000,000 hours, and MTTR = 24 hours.

RAID 5:

$$\text{MTTF}_{\text{RAID 5}} = \frac{(10^6)^2}{6 \times 5 \times 24} = \frac{10^{12}}{720} \approx 1.39 \times 10^9 \text{ hours} \approx 158{,}000 \text{ years}$$

RAID 6:

$$\text{MTTF}_{\text{RAID 6}} = \frac{\text{MTTF}^3}{N \times (N-1) \times (N-2) \times \text{MTTR}^2}$$
$$= \frac{(10^6)^3}{6 \times 5 \times 4 \times 24^2} = \frac{10^{18}}{69{,}120} \approx 1.45 \times 10^{13} \text{ hours}$$

RAID 6 is approximately 10,000 times more reliable than RAID 5 in this configuration. As drive capacities increase (and MTTR lengthens), RAID 6 becomes increasingly important.

Example 16.5 (The Danger of Large Drives in RAID 5). With 8 TB drives (MTTR = 16 hours), 8 drives, MTTF = 800,000 hours:

$$\text{MTTF}_{\text{RAID 5}} = \frac{(800{,}000)^2}{8 \times 7 \times 16} = \frac{6.4 \times 10^{11}}{896} \approx 714{,}285{,}714 \text{ hours} \approx 81{,}500 \text{ years}$$

This looks safe, but the calculation assumes independent failures. In practice, **correlated failures** (drives from the same batch, same age, same thermal environment) significantly reduce reliability. Studies by Google, Backblaze, and CMU have found that AFR for deployed drives is 2–8%, much higher than manufacturer specifications.

With AFR = 5% (MTTF = 175,200 hours) and MTTR = 24 hours:

$$\text{MTTF}_{\text{RAID 5}} = \frac{(175{,}200)^2}{8 \times 7 \times 24} = \frac{3.069 \times 10^{10}}{1344} \approx 22{,}838{,}000 \text{ hours} \approx 2{,}607 \text{ years}$$

For 1000 such arrays (a mid-sized data centre), the expected time to a first data loss event is 2.6 years — well within planning horizons.

Example 16.6 (Markov Model for RAID 1). A RAID 1 mirror with two drives can be modelled as a three-state continuous-time Markov chain:

- **State 0 (Normal):** Both drives operational. Transition rate to State 1: 2λ (either drive can fail).
- **State 1 (Degraded):** One drive has failed; the mirror is rebuilding. Transition rate to State 2 (data loss): λ (the surviving drive fails before rebuild completes). Transition rate back to State 0: $\mu = 1/\text{MTTR}$ (rebuild completes).
- **State 2 (Data Loss):** Both drives have failed. Absorbing state.

```
              2*lambda            lambda
[Normal] ----------> [Degraded] ----------> [Data Loss]
   ^                      |
   |        mu            |
```

```
+--------------------+
(rebuild completes)
```

The MTTF is the expected time to reach State 2 from State 0:

$$\text{MTTF}_{\text{RAID 1}} = \frac{1}{2\lambda} + \frac{1}{2\lambda} \times \frac{\mu}{\lambda + \mu} \times \text{MTTF}_{\text{RAID 1}}$$

Solving: $\text{MTTF}_{\text{RAID 1}} = \frac{\mu+\lambda}{2\lambda^2} \approx \frac{\mu}{2\lambda^2} = \frac{\text{MTTF}^2}{2\times\text{MTTR}}$ for $\lambda \ll \mu$.

For drives with MTTF = 500,000 hours and MTTR = 8 hours:

$$\text{MTTF}_{\text{RAID 1}} = \frac{(500{,}000)^2}{2 \times 8} = \frac{2.5 \times 10^{11}}{16} \approx 1.56 \times 10^{10}\,\text{hours} \approx 1{,}785{,}000\,\text{years}$$

16.3.5 Unrecoverable Read Errors (URE) During Rebuild

Unrecoverable Read Error (URE). A read operation that the drive's internal ECC cannot correct. The drive reports a read error to the operating system. The URE rate is specified by the manufacturer:

- Consumer HDDs: 1 in 10^{14} bits read ($\approx$ 1 URE per 12.5 TB read)
- Enterprise HDDs: 1 in 10^{15} bits read ($\approx$ 1 URE per 125 TB read)
- Enterprise SSDs: 1 in 10^{17} bits read ($\approx$ 1 URE per 12.5 PB read)

During a RAID rebuild, the system must read every sector from the surviving drives to reconstruct the failed drive's data. If a URE occurs on any surviving drive during this process, the affected stripe cannot be reconstructed — data is lost even though only one drive has failed.

Theorem 16.4 (URE Probability During Rebuild). For a RAID 5 array with N drives, each of capacity C bits, and URE rate p_{URE} (probability of URE per bit read), the probability of encountering at least one URE during rebuild is:

$$P(\text{URE during rebuild}) = 1 - (1 - p_{\text{URE}})^{(N-1)\times C} \approx (N-1) \times C \times p_{\text{URE}}$$

for small p_{URE}.

Example 16.17 (URE Risk for Large Drives). A RAID 5 array has 8 drives of 16 TB each, with consumer URE rate $1/10^{14}$ bits.

During rebuild, the system reads $(N-1) \times C = 7 \times 16 \times 10^{12} \times 8 = 8.96 \times 10^{14}$ bits.

$$P(\text{URE during rebuild}) \approx \frac{8.96 \times 10^{14}}{10^{14}} = 8.96$$

Since the probability exceeds 1, a URE during rebuild is **virtually certain**. This means that for a RAID 5 array of 8 consumer 16 TB drives, a single drive failure will almost certainly result in data loss due to a URE during rebuild.

With enterprise drives (URE rate $1/10^{15}$):

$$P(\text{URE}) \approx \frac{8.96 \times 10^{14}}{10^{15}} = 0.896 \approx 90\%$$

Even with enterprise drives, the risk is unacceptably high. This is the primary reason that RAID 5 is considered obsolete for large-capacity drives. RAID 6 tolerates one URE during rebuild (because it has double parity), making it the minimum acceptable configuration for drives larger than 2–4 TB.

16.4 Error Detection and Correction

Storage media are not perfectly reliable. Bits can be corrupted by physical defects, electromagnetic interference, cosmic rays, or firmware bugs. Multiple layers of error detection and correction protect against these errors.

16.4.1 Cyclic Redundancy Check (CRC)

Cyclic Redundancy Check (CRC). A hash function based on polynomial division over $GF(2)$ (the field with two elements). The data is treated as a polynomial $D(x)$, divided by a generator polynomial $G(x)$ of degree r. The remainder $R(x) = D(x) \cdot x^r \bmod G(x)$ is appended to the data as the CRC. The receiver divides the received data (with CRC) by $G(x)$; a non-zero remainder indicates an error.

The CRC is not a general-purpose hash function; it is specifically designed to detect common error patterns:

- **All single-bit errors** (as long as $G(x)$ has more than one term).
- **All double-bit errors** (if $G(x)$ has a factor that does not divide $x^k + 1$ for any $k < n$).
- **All odd numbers of bit errors** (if $G(x)$ has the factor $x + 1$).
- **All burst errors of length** $\leq r$ (where r is the degree of $G(x)$).

Example 16.6 (CRC-32 Calculation). CRC-32 uses the generator polynomial:

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^{8} + x^{7} + x^{5} + x^{4} + x^{2} + x + 1$$

For a 4 KB data block, CRC-32 appends a 4-byte checksum. The probability that a random error pattern goes undetected is approximately $2^{-32} \approx 2.3 \times 10^{-10}$.

In practice, CRC-32 is used in Ethernet frames, ZIP files, PNG images, and the ext4 file system (for metadata checksums).

16.4.2 Error-Correcting Codes (ECC)

While CRC detects errors, it cannot correct them. **Error-correcting codes** (ECC) add enough redundancy to both detect and correct a limited number of bit errors.

Error-Correcting Code (ECC). A code that maps k data bits to n coded bits ($n > k$), with $n - k$ redundancy bits. The **Hamming distance** d of the code determines its error-correcting capability:

- **Detect** up to $d - 1$ bit errors.
- **Correct** up to $\lfloor (d-1)/2 \rfloor$ bit errors.

ECC in DRAM. Server-grade DRAM uses SEC-DED (Single Error Correcting, Double Error Detecting) Hamming codes. For a 64-bit data word, 8 parity bits are added (72 bits total), providing:

- Correction of any single-bit error.
- Detection of any double-bit error.

The encoding uses a parity-check matrix H such that $H \cdot \mathbf{c}^T = \mathbf{0}$ for valid codewords $\mathbf{c}$. When a single bit i is flipped, $H \cdot \mathbf{c}'^T = \mathbf{h}_i$ (the i-th column of H), which identifies the bit position.

ECC in SSDs. NAND flash has inherently higher bit error rates than magnetic media, especially as cells are programmed and erased repeatedly. Modern SSDs use sophisticated ECC:

- **BCH codes:** Correcting 40–70 bit errors per 1 KB page (for MLC NAND).
- **LDPC codes:** Correcting hundreds of bit errors per page (for TLC and QLC NAND). LDPC provides higher correction capability at the cost of greater decoder complexity and latency.

16.4.3 Checksums in ZFS and Btrfs

File systems that implement checksumming provide a layer of error detection above the drive's built-in ECC. This catches errors that ECC misses or cannot correct, as well as errors introduced by the controller, cable, or driver (the so-called **data path** errors).

ZFS checksums. By default, ZFS uses **fletcher4**, a fast checksum algorithm that computes four 64-bit accumulators. For maximum integrity, **SHA-256** can be selected. Checksums are stored in the parent block (forming a Merkle tree), ensuring that a corrupted checksum is itself detected by its parent's checksum.

Btrfs checksums. Btrfs uses **CRC32C** (the hardware-accelerated variant using the Castagnoli polynomial). Btrfs stores checksums in a dedicated checksum tree, separate from the data.

Theorem 16.5 (End-to-End Checksum Coverage). A storage system has n components in the data path (controller, cable, driver, file system, page cache). Each component has an independent probability p_i of introducing an undetected error. With per-component error checking, the probability of an undetected end-to-end error is:

$$P_{\text{undetected}} = 1 - \prod_{i=1}^{n}(1 - p_i) \approx \sum_{i=1}^{n} p_i$$

for small p_i. An **end-to-end checksum** computed at the source and verified at the destination reduces $P_{\text{undetected}}$ to the false-negative probability of the checksum itself (e.g., 2^{-256} for SHA-256), regardless of the number of intermediate components.

16.5 The Flash Translation Layer (FTL)

The Flash Translation Layer is the firmware component that makes a NAND flash chip look like a block device. The FTL translates logical block addresses (LBAs) from the host into physical page addresses within the NAND, and it manages the complexities of flash: out-of-place writes, block erasure, garbage collection, and wear levelling.

16.5.1 FTL Address Mapping

Flash Translation Layer (FTL). A firmware layer in the SSD controller that maintains a **mapping table** from host LBAs to physical NAND page addresses. When the host writes to LBA x, the FTL allocates a clean physical page p, writes the data to

p, and updates the mapping: $\text{map}[x] \leftarrow p$. The old physical page (if any) is marked invalid.

The mapping table can be organized at different granularities:

- **Page-level mapping:** One entry per 4 KB page. For a 1 TB SSD with 4 KB pages, the table has 256×10^6 entries $\times$ 4 bytes $= 1$ GB. This must be stored in the SSD's DRAM buffer.
- **Block-level mapping:** One entry per erase block (e.g., 256 pages). Much smaller table ($256 \times 10^6/256 = 10^6$ entries), but any write within a block requires copying the entire block (high write amplification).
- **Hybrid mapping:** A combination of page-level mapping for frequently-written data (in a log region) and block-level mapping for the rest. This balances mapping table size with write amplification.

Example 16.18 (FTL Mapping Table Size). A 4 TB enterprise NVMe SSD with 4 KB pages:

- Pages: $4 \times 10^{12}/4096 = 976{,}562{,}500 \approx 10^9$
- Page-level mapping table: $10^9 \times 4\,\text{bytes} = 4\,\text{GB}$
- The SSD needs 4 GB of DRAM just for the mapping table.

This is why enterprise SSDs have substantial DRAM (4–8 GB for multi-TB drives) and why "DRAMless" consumer SSDs (which use host memory buffer, HMB, or reduced mapping tables) have lower random write performance.

16.5.2 Over-Provisioning

Over-Provisioning (OP). The difference between the SSD's raw NAND capacity and the user-accessible capacity. Over-provisioned space is invisible to the host and is used by the FTL for garbage collection, wear levelling, and bad block replacement.

$$\text{OP (\%)} = \frac{\text{NAND capacity} - \text{User capacity}}{\text{User capacity}} \times 100$$

Consumer SSDs typically have 7–13% over-provisioning (a "1 TB" SSD has approximately 1.07–1.13 TB of NAND). Enterprise SSDs may have 28–100% over-provisioning for sustained write performance and longer lifespan.

Higher over-provisioning means more free blocks are available for garbage collection at any time, reducing write amplification. This is why enterprise SSDs sustain much higher random write IOPS than consumer SSDs of the same capacity.

16.6 Write Amplification and Wear Levelling

16.6.1 The Write Amplification Factor

Write Amplification Factor (WAF). The ratio of the total amount of data physically written to the NAND flash to the amount of data logically written by the host:

$$\text{WAF} = \frac{\text{Data written to NAND}}{\text{Data written by host}}$$

A WAF of 1.0 means no amplification (ideal). In practice, WAF ranges from 1.1 (best case, sequential writes) to 10 or more (worst case, random small writes on a nearly full drive).

Write amplification arises because:

1. **Page-level writes, block-level erases:** To overwrite a page, the SSD must:
   a. Read all valid pages from the block.
   b. Erase the entire block.
   c. Write back the valid pages plus the new data to a clean block.
2. **Garbage collection:** The SSD's FTL (Flash Translation Layer) must periodically reclaim blocks containing a mixture of valid and invalid pages, copying valid pages to new blocks before erasing.
3. **Wear levelling:** The FTL must distribute writes evenly across all blocks to prevent some blocks from wearing out prematurely, which may involve moving data from infrequently-written blocks.

Example 16.7 (Write Amplification Calculation). An SSD has blocks of 256 pages (each 4 KB, so 1 MB per block). A block is 75% full of valid data (192 valid pages, 64 invalid pages).

Garbage collection: read 192 valid pages (768 KB read) and write them to a new block (768 KB written). The block is then erased. If the host wrote 256 KB of new data that triggered this GC:

$$\text{WAF} = \frac{768\,\text{KB (GC writes)} + 256\,\text{KB (host data)}}{256\,\text{KB (host data)}} = \frac{1024}{256} = 4.0$$

If over-provisioning (OP) is increased from 7% to 28%, the average block utilisation at GC time drops to 50%, and WAF decreases to:

$$\text{WAF} = \frac{512 + 256}{256} = 3.0$$

16.6.2 Wear Levelling

NAND flash cells have a limited number of program/erase (P/E) cycles before they become unreliable:

NAND Type	P/E Cycles	Bits per Cell
SLC (Single)	50,000–100,000	1
MLC (Multi)	3,000–10,000	2
TLC (Triple)	500–3,000	3
QLC (Quad)	100–1,000	4

Wear Levelling. An algorithm implemented by the SSD's Flash Translation Layer (FTL) that distributes write and erase operations evenly across all NAND blocks to maximise the drive's lifespan.

- **Dynamic wear levelling:** Only considers blocks that are currently being written. Blocks containing static (cold) data are never moved and may receive disproportionately few erases.
- **Static wear levelling:** Periodically moves cold data to heavily-worn blocks and reassigns less-worn blocks to receive new writes. This ensures that all blocks approach the P/E limit at roughly the same rate, but adds write amplification from data migration.

Example 16.8 (SSD Lifespan Calculation). A 1 TB TLC SSD has:

- Total NAND capacity: 1.15 TB (15% over-provisioning)
- P/E cycle limit: 1500 cycles
- WAF: 2.0 (typical for mixed workloads)
- Host write rate: 50 GB/day

Total bytes writeable (TBW):

$$\text{TBW} = \frac{\text{NAND capacity} \times \text{P/E cycles}}{\text{WAF}} = \frac{1.15 \times 10^{12} \times 1500}{2.0} = 862.5 \text{ TB}$$

Drive lifespan:

$$\text{Lifespan} = \frac{\text{TBW}}{\text{Daily writes}} = \frac{862.5 \times 10^3\,\text{GB}}{50\,\text{GB/day}} = 17{,}250\,\text{days} \approx 47\,\text{years}$$

Even with conservative assumptions, the SSD will outlast its other components. However, at 500 GB/day (a busy database server), the lifespan drops to 4.7 years, which is within planning horizons.

16.7 TRIM and Garbage Collection

16.7.1 The TRIM Command

When a file system deletes a file, it marks the blocks as free in its metadata (bitmap or extent tree) but does not inform the SSD. The SSD still considers those blocks as containing valid data, which degrades garbage collection efficiency.

TRIM (ATA DISCARD / NVMe Deallocate). A command from the host to the SSD indicating that specific logical blocks are no longer in use and their contents may be discarded. The SSD's FTL marks the corresponding pages as invalid, improving garbage collection efficiency by increasing the proportion of invalid pages in each block.

```
/* Linux: issue TRIM via ioctl */
#include <linux/fs.h>
#include <sys/ioctl.h>

uint64_t range[2];
range[0] = start_byte;   /* Start of the range to discard */
range[1] = length_bytes;  /* Length of the range */

ioctl(fd, BLKDISCARD, range);
```

On Linux, TRIM is typically issued in two ways:

1. **Continuous TRIM:** The file system issues TRIM commands as files are deleted (`mount -o discard`). Simple but adds latency to every delete operation.
2. **Periodic TRIM:** A scheduled job runs `fstrim` to discard all unused blocks at once (e.g., weekly via `systemd` timer). This batches TRIM operations, reducing overhead.

16.7.2 Garbage Collection in SSDs

The SSD's internal garbage collector runs autonomously in the firmware:

```
Garbage Collection Process:

Before GC:                          After GC:
+---+---+---+---+---+---+---+---+  +---+---+---+---+---+---+---+---+
| V | I | V | I | V | I | I | V |  |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+  +---+---+---+---+---+---+---+---+
Block A (4 valid, 4 invalid)         Block A (erased, free for writes)

                                    +---+---+---+---+---+---+---+---+
                                    | V | V | V | V |   |   |   |   |
                                    +---+---+---+---+---+---+---+---+
                                    Block B (4 valid pages copied here)

V = Valid page, I = Invalid page
```

The garbage collector must choose which blocks to clean. Common strategies:

- **Greedy:** Select the block with the most invalid pages. Minimises the number of valid pages to copy (lowest short-term write amplification).
- **Cost-benefit:** Similar to LFS (Chapter 14). Considers both the proportion of invalid pages and the age of valid pages. Old valid pages are less likely to be overwritten soon, so copying them is "cheaper" in terms of future work.
- **Wear-aware:** Factors in the erase count of each block. Blocks nearing their P/E limit are avoided for erasure.

16.8 Backup Strategies

RAID protects against drive failure but not against accidental deletion, software bugs, ransomware, or catastrophic events (fire, flood). **Backups** provide the last line of defence.

16.8.1 Backup Types

Backup Types.

- **Full backup:** A complete copy of all data. Provides the simplest restoration (single copy) but requires the most time and storage.
- **Incremental backup:** Copies only the data that has changed since the **last backup** (full or incremental). Minimal time and storage for each backup, but restoration requires the last full backup plus all subsequent incrementals, applied in order.
- **Differential backup:** Copies all data that has changed since the **last full backup**. More storage than incremental but simpler restoration: only the last full backup plus the latest differential are needed.

Example 16.9 (Backup Storage Comparison). A system has 1 TB of data. Each day, 20 GB changes.

Over a 7-day cycle (full on Sunday, daily backups Mon–Sat):

Full only: $7 \times 1\,\text{TB} = 7\,\text{TB}$

Full + Incremental: - Sunday: 1 TB (full) - Monday: 20 GB (changes since Sunday) - Tuesday: 20 GB (changes since Monday) - ... - Saturday: 20 GB (changes since Friday) - Total: $1\,\text{TB} + 6 \times 20\,\text{GB} = 1.12\,\text{TB}$

Full + Differential: - Sunday: 1 TB (full) - Monday: 20 GB (changes since Sunday) - Tuesday: 40 GB (changes since Sunday) - ... - Saturday: 120 GB (changes since Sunday) - Total: $1\,\text{TB} + (20 + 40 + 60 + 80 + 100 + 120)\,\text{GB} = 1.42\,\text{TB}$

Restoration of Saturday's state: - **Full only:** Restore Saturday's full backup (1 TB). - **Incremental:** Restore Sunday's full (1 TB) + Monday's incremental + ... + Saturday's incremental (6 restore operations). - **Differential:** Restore Sunday's full (1 TB) + Saturday's differential (120 GB) (2 restore operations).

16.8.2 The 3-2-1 Rule

The 3-2-1 Backup Rule. A widely-adopted guideline for data protection:

- **3** copies of data (1 primary + 2 backups).
- **2** different storage media (e.g., local disk + tape, or local disk + cloud).
- **1** copy offsite (protects against localised disasters: fire, flood, theft).

Modern extensions include **3-2-1-1-0:**

- **1** copy offline or air-gapped (protects against ransomware that encrypts all accessible storage).
- **0** errors (verified backups — regularly test restoration).

16.8.3 Snapshot-Based Backups

Copy-on-write file systems (ZFS, Btrfs) enable **snapshot-based backups** that combine the speed of incremental backups with the simplicity of full backups:

```
# ZFS snapshot-based backup workflow

# Create a snapshot (instantaneous, zero I/O)
zfs snapshot tank/data@backup-2026-04-16

# Send the snapshot to a backup pool (full)
zfs send tank/data@backup-2026-04-16 | zfs receive backup/data

# Next day: create a new snapshot
zfs snapshot tank/data@backup-2026-04-17

# Send only the delta (incremental)
zfs send -i tank/data@backup-2026-04-16 tank/data@backup-2026-04-17 \
    | zfs receive backup/data

# Restore from a snapshot (rollback)
zfs rollback tank/data@backup-2026-04-16
```

The `zfs send -i` (incremental send) transmits only the blocks that changed between two snapshots. This is more efficient than traditional file-level incremental backups because it operates at the block level and captures all changes, including metadata modifications that file-level tools might miss.

16.9 Data Integrity and Silent Corruption

16.9.1 The Problem of Silent Corruption

Not all storage errors are immediately visible. A **silent corruption** (also called a **silent data error** or **bit rot**) occurs when stored data is altered without the storage system detecting or reporting the error.

Silent Data Corruption. A data integrity violation where the stored data differs from what was written, but the storage device reports a successful read. The device's own error detection (ECC) either fails to detect the error or was bypassed by the error (e.g., a firmware bug writes data to the wrong location).

Sources of silent corruption include:

- **Misdirected writes:** The drive's firmware writes data to the wrong LBA. The data at the intended LBA is now stale, and the data at the unintended LBA is overwritten.
- **Lost writes:** The drive acknowledges a write that never reaches persistent storage (e.g., a power failure before the drive's volatile write cache is flushed).
- **Bit decay:** Gradual degradation of magnetic or charge-based storage over time. Affects both HDDs (magnetic domain decay) and SSDs (charge leakage in NAND cells).
- **Firmware bugs:** The drive's firmware has thousands of lines of code. Bugs in error handling, wear levelling, or garbage collection can corrupt data.
- **Cosmic rays:** High-energy particles can flip bits in both DRAM and NAND flash. At sea level, the rate is approximately 10^{-15} bit flips per bit per hour for DRAM.

Studies by CERN (2007) and NetApp (2008) found that silent corruption occurs at rates of approximately 1 in 10^{14} to 10^{16} bits read — rare on a per-bit basis but significant at scale. For a data centre reading 1 PB per day, the expected number of silent corruptions is:

$$\frac{10^{15} \times 8}{10^{15}} = 8 \text{ corrupted bytes per day}$$

16.9.2 Scrubbing

Scrubbing. A proactive data integrity verification process that reads all stored data, verifies checksums, and repairs any detected corruption from redundant copies. Scrubbing detects latent errors before they accumulate and cause data loss.

RAID scrubbing reads every block in the array and verifies parity consistency. If a mismatch is found:

- In a RAID with checksums (ZFS, Btrfs): the checksum identifies which copy is correct, and the corrupted copy is repaired.
- In a traditional RAID (mdadm): the parity indicates a mismatch, but the system cannot determine which block is wrong. It logs the error but cannot automatically repair it.

```
# ZFS scrub: verify every block's checksum, repair from redundancy
zpool scrub tank
```

```
# Check scrub results
zpool status tank
#   scan: scrub repaired 0B in 2h30m with 0 errors on Wed Apr 16 03:00:00 2026
#   errors: No known data errors

# mdadm scrub: verify RAID parity consistency
echo check > /sys/block/md0/md/sync_action

# Monitor progress
cat /proc/mdstat
# md0 : active raid5 ...
#       [=====>...............]  check = 28.3% ...
```

Example 16.10 (Scrub Schedule Calculation). A ZFS pool has 20 TB of data. Scrub read speed is 500 MB/s (limited by the slowest drive). Scrub duration:

$$T_{\text{scrub}} = \frac{20 \times 10^{12}}{500 \times 10^{6}} = 40{,}000\,\text{s} \approx 11.1\,\text{hours}$$

Running a weekly scrub adds a read load of $11.1/168 = 6.6\%$ of total hours. This is a modest cost for comprehensive integrity verification. ZFS recommends monthly scrubs for most workloads and weekly scrubs for critical data.

16.9.3 End-to-End Checksums

End-to-End Data Integrity. A design principle where checksums are computed by the data producer and verified by the data consumer, with no intermediate component trusted to maintain integrity. Any corruption introduced anywhere in the data path — by the drive, controller, cable, driver, file system, or page cache — is detected at read time.

The key insight is that checksums computed **by the drive** (its internal ECC) cannot protect against errors **above the drive**: firmware bugs, controller errors, cable corruption, or kernel bugs. Only checksums computed by the file system and verified by the file system provide true end-to-end protection.

This is why ZFS's and Btrfs's approach of storing checksums in the file system (rather than relying on the drive's ECC) is fundamentally more robust:

```
Traditional storage stack:
Application -> File system -> Block layer -> Driver -> Controller -> Media
                                                                  ^
```

```
                                                        |
                                                    ECC here only
                                                (protects media errors,
                                                 not path errors)

ZFS/Btrfs:
Application -> File system -> Block layer -> Driver -> Controller -> Media
                ^                                                       ^
                |                                                       |
            Checksum verified here                                  ECC here
            (protects ENTIRE path)                              (protects media)
```

Programmer's Perspective: Monitoring Storage Health with `smartctl`. The Self-Monitoring, Analysis and Reporting Technology (SMART) system is built into every modern HDD and SSD. It tracks internal health metrics and can predict failures before they occur. The `smartctl` utility (from the `smartmontools` package) reads these metrics.

```
# View overall SMART health status
smartctl -H /dev/sda
# === START OF READ SMART DATA SECTION ===
# SMART overall-health self-assessment test result: PASSED

# View all SMART attributes
smartctl -A /dev/sda
# ID# ATTRIBUTE_NAME          FLAG  VALUE WORST THRESH TYPE     RAW_VALUE
#   1 Raw_Read_Error_Rate     0x002f  200   200   051   Pre-fail    0
#   5 Reallocated_Sector_Ct   0x0033  200   200   140   Pre-fail    0
#   9 Power_On_Hours          0x0032  095   095   000   Old_age     22847
# 197 Current_Pending_Sector  0x0032  200   200   000   Old_age     0
# 198 Offline_Uncorrectable   0x0030  200   200   000   Old_age     0

# Critical attributes to monitor:
# 5   Reallocated_Sector_Ct  - sectors remapped due to defects (rising = failing)
# 197 Current_Pending_Sector - sectors awaiting reallocation (instability)
# 198 Offline_Uncorrectable  - sectors that failed ECC during offline scan

# For NVMe SSDs:
smartctl -a /dev/nvme0n1
# Critical NVMe attributes:
# Percentage Used:         3%         (of rated lifespan)
```

```
# Data Units Written:      12,345,678 (units of 512 KB = 6.3 TB)
# Media and Data Integrity Errors:  0
# Error Information Log Entries:    0

# Run a long self-test
smartctl -t long /dev/sda
# (takes several hours, scans entire surface)

# Check test results
smartctl -l selftest /dev/sda
```

For production systems, configure `smartd` (the SMART daemon) to monitor all drives and alert on critical attribute changes:

```
# /etc/smartd.conf
# Monitor all drives, alert on any attribute change, email on failure
/dev/sda -a -o on -S on -s (S/../.././02|L/../../7/03) \
    -m admin@example.com -M exec /usr/share/smartmontools/smartd_warning.sh
/dev/sdb -a -o on -S on -s (S/../.././02|L/../../7/03) \
    -m admin@example.com -M exec /usr/share/smartmontools/smartd_warning.sh
```

A robust monitoring setup combines SMART monitoring, ZFS scrub schedules, and `mdadm` monitoring:

```go
package main

import (
    "fmt"
    "os"
    "os/exec"
    "strings"
)

// CheckSMARTHealth runs smartctl and checks for critical warnings.
func CheckSMARTHealth(device string) error {
    cmd := exec.Command("smartctl", "-H", device)
    out, err := cmd.Output()
    if err != nil {
        // smartctl returns non-zero for failing drives
        if exitErr, ok := err.(*exec.ExitError); ok {
            if exitErr.ExitCode()&0x08 != 0 {
                return fmt.Errorf("SMART prefailure: %s", device)
```

```
            }
        }
        return fmt.Errorf("smartctl error for %s: %w", device, err)
    }
    if !strings.Contains(string(out), "PASSED") {
        return fmt.Errorf("SMART check did not pass for %s", device)
    }
    return nil
}

// CheckReallocatedSectors reads the reallocated sector count.
func CheckReallocatedSectors(device string) (int, error) {
    cmd := exec.Command("smartctl", "-A", device)
    out, err := cmd.Output()
    if err != nil {
        return -1, err
    }
    for _, line := range strings.Split(string(out), "\n") {
        if strings.Contains(line, "Reallocated_Sector_Ct") {
            fields := strings.Fields(line)
            if len(fields) >= 10 {
                var count int
                fmt.Sscanf(fields[9], "%d", &count)
                return count, nil
            }
        }
    }
    return 0, nil
}

func main() {
    devices := []string{"/dev/sda", "/dev/sdb", "/dev/sdc", "/dev/sdd"}
    for _, dev := range devices {
        if err := CheckSMARTHealth(dev); err != nil {
            fmt.Fprintf(os.Stderr, "WARNING: %v\n", err)
            continue
        }
        count, err := CheckReallocatedSectors(dev)
        if err != nil {
            fmt.Fprintf(os.Stderr, "Error reading SMART for %s: %v\n", dev, err)
```

```
            continue
        }
        if count > 0 {
            fmt.Fprintf(os.Stderr, "ALERT: %s has %d reallocated sectors\n", dev, count
        } else {
            fmt.Printf("%s: healthy (0 reallocated sectors)\n", dev)
        }
    }
}
```

The key insight for programmers: do not trust storage. Every production system should verify its data. Use checksums in your application layer (not just the file system), test backups by actually restoring them, and monitor drive health proactively. The cost of verification is negligible compared to the cost of data loss.

16.10 Putting It All Together: A Reliability Stack

A well-designed storage system employs defence in depth — multiple independent mechanisms that protect against different failure modes:

Layer	Mechanism	Protects Against
Application	Application-level checksums	Logic bugs, data corruption above file system
File system	ZFS/Btrfs checksums	Silent corruption, misdirected writes
File system	Journaling/CoW	Crash inconsistency
Volume manager	RAID 5/6/Z1/Z2	Drive failure
Drive firmware	ECC (BCH/LDPC)	Bit errors in media
Drive firmware	Wear levelling	Premature cell death (SSD)
Drive firmware	SMART monitoring	Predictive failure detection
Operational	Scrubbing	Latent error accumulation
Operational	Backups (3-2-1)	All of the above + human error + disasters

Theorem 16.6 (Defence in Depth Reliability). For a system with n independent protection layers, each with effectiveness e_i (probability of catching a given failure), the probability of an undetected failure is:

$$P_{\text{undetected}} = \prod_{i=1}^{n}(1 - e_i)$$

For example, with $n = 4$ layers each having 99% effectiveness:

$$P_{\text{undetected}} = (0.01)^4 = 10^{-8}$$

The product of independent probabilities rapidly approaches zero, which is why layered defences are so effective even when no single layer is perfect.

Example 16.11 (Complete Storage Design). An engineering team needs to store 50 TB of critical data with the following requirements:

- Survive any 2 simultaneous drive failures.
- Detect and repair silent corruption.
- Recover from accidental deletion within 30 days.
- Survive a site-wide disaster.

Design:

1. **Primary storage:** ZFS RAID-Z2 (6 $\times$ 16 TB drives). Usable capacity: $4 \times 16 = 64$ TB. Survives any 2 drive failures. End-to-end checksums detect silent corruption.
2. **Scrubbing:** Monthly ZFS scrub (approximately 28 hours at 500 MB/s). Detects and repairs latent errors before they accumulate.
3. **Snapshots:** Hourly snapshots retained for 48 hours, daily snapshots retained for 30 days. Enables recovery from accidental deletion without restoring from backup.
4. **Local backup:** ZFS send/receive to a second server (RAID-Z1, since the primary already has RAID-Z2). Incremental sends nightly.
5. **Offsite backup:** Weekly full ZFS send to cloud storage (encrypted). Monthly restore test.
6. **Monitoring:** `smartctl` checks every 12 hours, ZFS scrub errors trigger immediate alerts, RAID degradation triggers automatic ticket creation.

This design provides: $2\times$ drive failure tolerance (RAID-Z2), silent corruption detec-

tion (checksums + scrub), 30-day recovery window (snapshots), local disaster recovery (second server), and offsite disaster recovery (cloud backup).

16.11 Erasure Coding Beyond RAID

16.11.1 Limitations of Traditional RAID

Traditional RAID was designed for small arrays of disks (4–16 drives) in a single server. Modern distributed storage systems (cloud object stores, Hadoop HDFS, Ceph) manage thousands of drives across hundreds of servers. At this scale, traditional RAID has several limitations:

- **Rebuild bottleneck:** In a RAID array, only the surviving drives contribute to rebuilding a failed drive. For N drives in a RAID 5 group, the rebuild speed is limited by the throughput of $N - 1$ drives. In a distributed system, hundreds of drives can participate in the rebuild.
- **Correlated failures:** Drives in the same server share power supply, cooling, and network connectivity. A server failure takes out all drives simultaneously. RAID within a server does not protect against server failure.
- **Fixed redundancy:** RAID levels provide fixed protection (1 or 2 drive failures). Erasure codes allow tunable redundancy.

16.11.2 Reed-Solomon Erasure Codes

Erasure Code. An error-correcting code applied at the storage block level, where the locations of missing (erased) blocks are known (because the system knows which drives have failed). An (n, k) erasure code encodes k data blocks into n coded blocks (where $n > k$) such that any k of the n blocks suffice to reconstruct the original data. The code tolerates up to $n - k$ simultaneous block losses.

RAID 5 is an $(n, n-1)$ erasure code (XOR parity: one redundancy block). RAID 6 is an $(n, n-2)$ code (two redundancy blocks using Reed-Solomon in $GF(2^8)$). General Reed-Solomon codes support arbitrary values of n and k.

Example 16.12 (Erasure Code Storage Efficiency). Consider storing 10 GB of data with tolerance for 2 failures:

RAID 6 (8+2): 10 data blocks + 2 parity blocks = 12 blocks. Storage overhead: $12/10 = 1.2\times$ (20% overhead).

3-way replication: 3 copies of each block. Storage overhead: $3.0\times$ (200% overhead).

Reed-Solomon (10, 14): 10 data blocks + 4 parity blocks = 14 blocks. Tolerates any 4 failures. Storage overhead: $14/10 = 1.4\times$ (40% overhead for much greater protection).

Erasure coding provides far better storage efficiency than replication while maintaining configurable fault tolerance. This is why cloud storage systems (Google Colossus, Facebook f4, Azure LRC) use erasure codes rather than replication for cold data.

Theorem 16.7 (MDS Property). A **Maximum Distance Separable** (MDS) code with parameters (n, k) achieves the maximum possible minimum distance $d = n - k + 1$. Reed-Solomon codes are MDS: any k of n blocks suffice to reconstruct the data, and this is optimal — no code with the same redundancy can tolerate more erasures.

Proof sketch. The Singleton bound states that for any (n, k) code, $d \leq n - k + 1$. Reed-Solomon codes achieve this bound because their generator matrix is a Vandermonde matrix, which has the property that every $k \times k$ submatrix is invertible (non-zero determinant). Therefore, any k received blocks form an invertible system that uniquely determines the k data blocks. □

16.11.3 Locally Repairable Codes (LRC)

Reed-Solomon codes have a repair cost problem: reconstructing a single failed block requires reading k other blocks. For a (14, 10) code, repairing one block requires reading 10 blocks from 10 different drives — significant network and I/O overhead.

Locally Repairable Codes (LRCs), developed by Microsoft Research (2012), add **local parity** blocks that protect small groups of data blocks. A single failure within a group can be repaired by reading only the other blocks in that group (typically 3–5 blocks) rather than all k data blocks.

Locally Repairable Code (LRC). An erasure code where each data block has a **repair group** of size $r \ll k$. A single block failure is repaired by reading only the $r - 1$ other blocks in its repair group plus the local parity block, rather than all k data blocks. The code also has global parity blocks for multi-failure tolerance.

Example 16.13 (Azure LRC). Microsoft Azure Storage uses an LRC(12, 2, 2) code: 12 data blocks are divided into 2 groups of 6. Each group has 1 local parity block (computed as XOR of its 6 data blocks). Additionally, 2 global parity blocks (computed using Reed-Solomon over all 12 data blocks) provide multi-failure tolerance.

Total blocks: 12 (data) + 2 (local parity) + 2 (global parity) = 16. Storage overhead: $16/12 = 1.33\times$.

- **Single failure:** Repaired from the local group (read 6 blocks, not 12).
- **Two failures in different groups:** Each repaired from its local group independently.
- **Two failures in the same group:** Use the global parity (read 12 blocks).
- **Up to 4 failures:** Recoverable using the full code.

Erasure codes are a strictly more general framework than traditional RAID. RAID 5 and RAID 6 are specific instances of erasure codes with $n - k = 1$ and $n - k = 2$ respectively. Modern distributed storage systems use erasure codes with higher redundancy and locality properties, reflecting the different failure modes and repair costs of large-scale networked storage versus local disk arrays.

16.12 Storage Tiering and Caching

16.12.1 The Storage Hierarchy

Modern storage systems often employ multiple tiers with different performance and cost characteristics:

Tier	Technology	Latency	Cost per GB	Use Case
0	NVMe SSD	10–20 μs	High	Hot data, databases
1	SATA SSD	50–100 μs	Medium	Warm data, active files
2	HDD (10K RPM)	3–5 ms	Low	Cold data, archives
3	Tape (LTO)	10–60 s	Very low	Archival, compliance
Cloud	Object storage	50–200 ms	Variable	Offsite, disaster recovery

16.12.2 SSD Caching (bcache, dm-cache, ZFS L2ARC)

A common pattern is to use a small SSD as a read/write cache in front of a large HDD array:

SSD Cache. A block-layer caching mechanism that intercepts I/O requests to a slow device (HDD) and serves frequently accessed blocks from a fast device (SSD). The cache operates transparently: the file system sees a single block device.

- **Read cache (write-around):** Only reads are cached. Writes go directly to the HDD. Simple and safe — SSD failure does not lose data.
- **Write-back cache:** Both reads and writes are cached. Writes are buffered on the SSD and flushed to the HDD asynchronously. Higher performance but SSD failure can lose uncommitted writes.
- **Write-through cache:** Writes are cached on the SSD and simultaneously written to the HDD. Safe (no data loss on SSD failure) but no write acceleration.

Linux provides several SSD caching mechanisms:

```
# bcache: kernel block layer cache
# Back device: HDD, Cache device: SSD
make-bcache -B /dev/sda -C /dev/nvme0n1p1
# Set cache mode
echo writeback > /sys/block/bcache0/bcache/cache_mode

# dm-cache: device-mapper based cache
# Similar concept but uses device-mapper infrastructure

# ZFS L2ARC (Level 2 Adaptive Replacement Cache)
# Add an SSD as a read cache to a ZFS pool
zpool add tank cache /dev/nvme0n1p1
```

Example 16.14 (Cache Hit Rate and Effective Latency). A system uses an NVMe SSD cache (latency $= 15\,\mu$s) in front of an HDD array (latency $= 5$ ms). The cache hit rate is $h = 0.95$ (95%).

Effective average latency:

$$L_{\text{eff}} = h \times L_{\text{SSD}} + (1-h) \times L_{\text{HDD}} = 0.95 \times 15\,\mu\text{s} + 0.05 \times 5000\,\mu\text{s}$$

$$= 14.25\,\mu\text{s} + 250\,\mu\text{s} = 264.25\,\mu\text{s}$$

This is 19x faster than the raw HDD latency. Even a modest cache hit rate of 80% gives:

$$L_{\text{eff}} = 0.80 \times 15 + 0.20 \times 5000 = 12 + 1000 = 1012\,\mu\text{s} \approx 1\,\text{ms}$$

Still a 5x improvement over raw HDD access.

16.13 Storage Protocols and Interfaces

16.13.1 Protocol Evolution

The evolution of storage interfaces reflects the increasing performance demands of storage media:

Protocol	Interface	Max Bandwidth	Queue Depth	Year
ATA/IDE	Parallel cable	133 MB/s	1	1986
SATA I	Serial cable	150 MB/s	32	2003
SATA III	Serial cable	600 MB/s	32	2009
SAS-3	Serial cable	1200 MB/s	254	2013
NVMe (PCIe 3.0 x4)	PCIe	3500 MB/s	65535	2011
NVMe (PCIe 4.0 x4)	PCIe	7000 MB/s	65535	2019
NVMe (PCIe 5.0 x4)	PCIe	14000 MB/s	65535	2022

The key insight is that **protocol overhead**, not media speed, is often the bottleneck. SATA's single 32-command queue was designed for HDDs that could handle only one command at a time. When SSDs arrived with sub-millisecond latency and massive internal parallelism, SATA became the bottleneck. NVMe was designed specifically for flash storage, with 65,535 queues and zero legacy overhead.

16.13.2 Network-Attached Storage Protocols

For storage accessed over a network, additional protocols are involved:

Storage Protocols.

- **NFS (Network File System):** A file-level protocol. The client mounts a remote directory and accesses files using normal file operations. The NFS server translates these into local file system operations. NFS is stateless (NFSv3) or stateful (NFSv4).
- **iSCSI (Internet SCSI):** A block-level protocol that encapsulates SCSI commands in TCP/IP packets. The remote storage appears as a local block device. The operating system sees `/dev/sda` but the actual storage is on a remote server.

- **NVMe-oF (NVMe over Fabrics):** NVMe commands transported over a network fabric (RDMA, TCP, or Fibre Channel). Provides near-local NVMe performance over the network by eliminating the SCSI translation layer.

Example 16.15 (Protocol Overhead Comparison). Accessing a 4 KB block on a remote NVMe SSD:

iSCSI over TCP: SCSI command → iSCSI encapsulation → TCP segmentation → IP routing → Ethernet framing. Round-trip latency: 100–500 μs (dominated by TCP processing and network latency).

NVMe-oF over RDMA: NVMe command → RDMA send (kernel bypass, zero-copy). Round-trip latency: 10–30 μs. Approaches local NVMe latency because RDMA bypasses the kernel's network stack entirely.

NFS: File read request → RPC call → TCP/UDP → network. Server: VFS lookup → file system read → RPC response. Round-trip latency: 200–1000 μs (includes file system overhead on both client and server).

16.14 Practical Drive Selection

16.14.1 HDD vs SSD Decision Matrix

The choice between HDD and SSD depends on workload characteristics:

Factor	Choose HDD	Choose SSD
Cost per TB	Under tight budget ($15–20/TB)	Performance justifies cost ($50–100/TB)
Access pattern	Sequential (streaming, backup, archival)	Random (databases, VMs, containers)
Capacity needed	Very large (50+ TB per drive)	Moderate (1–8 TB typical)
Latency requirement	Tolerant (> 5 ms acceptable)	Sensitive (< 1 ms required)
Write endurance	Infinite (within mechanical lifespan)	Limited (TBW rating)
Power per TB	Higher (6–12 W per drive)	Lower (2–5 W per drive)
Vibration sensitivity	Sensitive (affects adjacent drives)	Immune

16.14.2 Enterprise vs Consumer Drives

Enterprise drives differ from consumer drives in several critical ways:

- **Vibration tolerance:** Enterprise HDDs include rotational vibration sensors that adjust the servo to compensate for vibrations from adjacent drives in a dense server chassis.
- **Power loss protection (PLP):** Enterprise SSDs include supercapacitors that provide enough power to flush the volatile write cache to NAND on sudden power loss. Consumer SSDs may lose cached writes.
- **Consistent latency:** Enterprise SSDs prioritise consistent latency (low P99) over peak throughput. Consumer SSDs may have occasional high-latency outliers during garbage collection.
- **Endurance rating:** Enterprise SSDs are rated for 1–10 DWPD (Drive Writes Per Day) over a 5-year warranty. Consumer SSDs are typically rated for 0.3–0.6 DWPD.

DWPD (Drive Writes Per Day). A measure of SSD write endurance. A drive rated at 1 DWPD can sustain writing its entire capacity once per day for the warranty period.

For a 1 TB drive at 1 DWPD over 5 years:

$$\text{TBW} = 1\,\text{TB} \times 365\,\text{days} \times 5\,\text{years} = 1825\,\text{TB}$$

Example 16.21 (Drive Selection for a Database Server). A PostgreSQL database server has the following requirements: 4 TB usable storage, write rate of 100 GB/day, P99 read latency below 1 ms, must survive 1 drive failure.

Option A: 4 x 2 TB consumer SATA SSDs in RAID 10. - Usable capacity: $4 \times 2/2 = 4$ TB. Meets requirement. - Endurance: Consumer TLC at 0.5 DWPD. TBW = $2000 \times 0.5 \times 365 \times 5 = 1{,}825$ TB. At 100 GB/day with WAF 2.0, daily NAND writes = 200 GB. Lifespan = $1825 \times 10^3/200 \approx 9{,}125$ days ≈ 25 years. Adequate. - Latency: SATA SSDs: P99 ≈ 0.5–2 ms. Marginal — may exceed 1 ms during GC. - No power loss protection. Risk of data loss on power failure.

Option B: 4 x 2 TB enterprise NVMe SSDs in RAID 10. - Usable capacity: 4 TB. Meets requirement. - Endurance: Enterprise TLC at 3 DWPD. TBW = 10,950 TB. Lifespan = 54,750 days $\approx$ 150 years. More than adequate. - Latency: NVMe P99 ≈ 0.1–0.3 ms. Well within requirement. - Power loss protection included. Safe for database use. - Cost: approximately 3x higher than Option A.

For a production database, Option B is the correct choice. The enterprise NVMe

drives provide the consistent latency, power loss protection, and endurance that a database demands. The consumer drives would work for development and testing but risk data loss and latency spikes in production.

16.14.3 Drive Qualification and Testing

Before deploying drives in production, responsible operations teams perform **drive qualification** testing:

1. **Stress testing:** Sustained random write workloads at maximum queue depth for 24–72 hours. Monitors for latency outliers, error counts, and thermal throttling.
2. **Power cycle testing:** Repeated power cuts during active writes, followed by data integrity verification. This tests the drive's power loss protection mechanism.
3. **Firmware verification:** Confirm the firmware version and verify that it does not have known bugs (drive manufacturers publish firmware advisories).
4. **Compatibility testing:** Verify correct operation with the specific RAID controller, HBA (Host Bus Adapter), or software RAID stack in use. Driver and firmware interactions can cause subtle issues.

16.15 Long-Term Data Preservation

16.15.1 Media Longevity

Every storage medium has a finite lifespan, even when not in active use:

Medium	Expected Data Retention	Failure Mode
HDD (powered off)	3–5 years	Lubricant degradation, stiction
SSD (powered off)	1–2 years (consumer), 3 months (enterprise at 40 C)	Charge leakage from NAND cells
Magnetic tape (LTO)	15–30 years	Binder degradation, demagnetisation
Optical disc (M-DISC)	100+ years (claimed)	Physical degradation of recording layer

Medium	Expected Data Retention	Failure Mode
Cloud storage	Indefinite (while paying)	Provider bankruptcy, policy change

Example 16.20 (SSD Data Retention). NAND flash cells lose their charge over time. The retention period depends on the number of P/E cycles the cell has endured (more cycles = thinner oxide = faster leakage) and temperature:

- A fresh TLC cell (0 P/E cycles) retains data for approximately 10 years at 25 C.
- A worn TLC cell (1000 P/E cycles) retains data for approximately 1 year at 25 C.
- At 40 C (a warm shelf), retention halves approximately.

This is why enterprise SSD specifications distinguish between **active** retention (the drive is powered on periodically, allowing the controller to refresh weak cells) and **unpowered** retention. For archival storage, SSDs are a poor choice compared to tape or optical media.

16.15.2 Format Longevity

Data preservation requires not only physical media longevity but also the ability to **read** the data decades later. This requires:

- **File format longevity:** Proprietary formats may become unreadable. Open standards (PDF/A, plain text, TIFF, SQLite) are safer bets for long-term archives.
- **File system longevity:** A backup on a Btrfs volume is useless if future systems cannot mount Btrfs. For archival backups, simple formats (tar archives on ext4, or ISO images) are preferable to sophisticated file systems.
- **Hardware interface longevity:** A backup on a SCSI tape is useless without a SCSI tape drive. Regular migration to current media (every 5–10 years) is essential.

The US Library of Congress recommends the **3-2-1 rule** combined with **format migration**: maintain data in open formats on current media, with periodic refreshes to new media and formats as technology evolves.

Storage reliability is ultimately a systems engineering problem: no single technology or technique provides absolute protection. Defence in depth — combining RAID for drive failure, checksums for silent corruption, journaling for crash consistency, back-

ups for human error and disasters, and periodic verification for latent errors — is the only path to truly reliable storage.

Exercises

Exercise 16.1. A data centre has 10,000 hard drives, each with an annual failure rate (AFR) of 3%. (a) What is the expected number of drive failures per year? (b) What is the expected number of drive failures per day? (c) If drives are organised into RAID 5 arrays of 8 drives each, how many arrays are there, and what is the expected number of array failures (data loss events) per year? Assume MTTR = 24 hours and that failures are independent. (d) How does the answer change for RAID 6?

Exercise 16.2. Derive the formula for RAID 6 MTTF:

$$\text{MTTF}_{\text{RAID 6}} = \frac{\text{MTTF}_{\text{drive}}^3}{N \times (N-1) \times (N-2) \times \text{MTTR}^2}$$

Start from the three-state Markov model: Normal → Degraded (1 failure) → Critical (2 failures) → Data Loss (3 failures). Calculate the transition rates between states and the expected time to reach the Data Loss state.

Exercise 16.3. An SSD has the following specifications: 2 TB capacity, 1500 P/E cycle endurance (TLC NAND), 7% over-provisioning. The drive is used in a database server that writes 200 GB of data per day. (a) Calculate the drive's total bytes written (TBW) rating assuming WAF = 1.5. (b) Estimate the drive's lifespan in years. (c) What WAF would reduce the lifespan below 3 years? (d) Describe two strategies the database administrator can use to reduce write amplification.

Exercise 16.4. Explain the difference between CRC-based error detection and ECC-based error correction. A storage system uses CRC-32 on 4 KB data blocks. (a) What is the probability that a random 4-byte corruption goes undetected? (b) What types of error patterns does CRC-32 guarantee to detect? (c) If the system additionally uses a SEC-DED Hamming code on each 64-bit word, which errors can be corrected automatically, and which require higher-level intervention (e.g., restoring from a RAID parity or backup)?

Exercise 16.5. A ZFS pool has 4 vdevs, each a RAID-Z1 group of 5 drives. Each drive has an MTTF of 1,000,000 hours and an unrecoverable read error rate of 1 in 10^{14} bits. (a) What is the probability of encountering an unrecoverable read error during the rebuild of a single 8 TB drive? (b) This is called a **URE during rebuild** scenario. Explain why this makes RAID 5 increasingly dangerous for large drives. (c)

How does ZFS's checksum-based approach mitigate this risk compared to traditional mdadm RAID?

Exercise 16.6. Design a backup schedule for a 10 TB file server using the 3-2-1 rule. Specify: (a) the backup types (full, incremental, differential) and their frequency; (b) the retention policy; (c) the estimated total storage required for backups over 1 year; (d) the procedure for restoring data from 2 weeks ago. Assume 50 GB of data changes daily.

Exercise 16.7. A storage system uses the following integrity mechanisms: (1) CRC-32 on each 4 KB block, stored alongside the block; (2) SHA-256 checksum on each 1 MB extent, stored in a separate metadata tree; (3) RAID-Z2 parity across 6 drives. A cosmic ray flips a single bit in a data block. Trace the detection and repair process through all three layers. What would happen if layer (2) were absent? What if layer (3) were absent?

Chapter 17: Security and Protection

"The only truly secure system is one that is powered off, cast in a block of concrete, and sealed in a lead-lined room with armed guards — and even then I have my doubts."
— Eugene Spafford

An operating system manages resources on behalf of multiple principals — users, processes, services — each with different levels of trust. **Protection** is the mechanism by which the OS controls access to those resources. **Security** is the broader goal of ensuring that the protection mechanisms cannot be circumvented by adversaries, whether they attack through software vulnerabilities, side channels in the hardware, or social engineering. This chapter examines both dimensions: the formal models that govern access control, the concrete mechanisms that Linux and other systems implement, and the hardware-level attacks that have reshaped how we think about trust boundaries.

We begin with the abstract framework — protection domains, the access matrix, and its decompositions — then move through the three major access control paradigms (DAC, MAC, RBAC), the Linux Security Modules framework, sandboxing, hardware-level attacks on speculative execution, trusted computing, and finally the oldest and most persistent class of software vulnerability: the buffer overflow.

17.1 Protection Domains and the Access Matrix

Every process executes within a **protection domain** that defines the set of resources it may access and the operations it may perform on each resource.

Protection Domain. A protection domain is a set of pairs (o_i, R_i) where o_i is an object (file, memory segment, device, socket) and $R_i \subseteq \{\text{read}, \text{write}, \text{execute}, \text{append}, \text{delete}, \ldots\}$ is the set of access rights the domain holds for that object.

A process may belong to one domain at a time, but domain switching is permitted — for example, a user-mode process switches to the kernel domain via a system call, or a setuid binary switches to the file owner's domain upon execution. Domain switching

is itself a controlled operation: it requires a specific access right (the `switch` right) in the current domain.

The concept of a protection domain is powerful because it separates the **policy** (what a process should be allowed to do) from the **mechanism** (how the OS enforces it). The access matrix, which we examine next, provides the formal framework for expressing policies.

17.1.1 The Access Matrix Model

The **access matrix**, formalised by Lampson (1971) and extended by Graham and Denning (1972), provides the foundational abstraction for all access control systems.

Access Matrix. An access matrix A is a two-dimensional array where rows represent **domains** (subjects) and columns represent **objects**. The entry $A[d_i, o_j]$ contains the set of access rights that domain d_i holds over object o_j.

Example 17.1 (Access Matrix). Consider a system with three domains (D_1, D_2, D_3) and four objects (File1, File2, Printer, Port80):

	File1	File2	Printer	Port80
D_1	read, write	read		
D_2	read	read, write	print	
D_3		read		bind, listen

Domain D_1 can read and write File1 but can only read File2. Domain D_3 can bind and listen on Port80 (a network port treated as an object). No domain has universal access.

The access matrix can include domains themselves as objects, enabling **domain switching**: if $A[D_1, D_2]$ contains the right `switch`, then a process in D_1 may transition to D_2. This captures setuid execution, kernel entry, and role assumption. Furthermore, the access matrix can include **control rights** that govern modification of the matrix itself:

- **Owner** right: the ability to grant or revoke access to an object.
- **Copy** right: the ability to copy an access right to another domain (with or without the copy right itself, yielding **transfer** vs **limited copy**).

Harrison-Ruzzo-Ullman (HRU) Model. The HRU model extends the access matrix with a set of commands that modify the matrix. Each command is a sequence of

primitive operations:

- `enter r into A[s, o]`: grant right r to subject s for object o.
- `delete r from A[s, o]`: revoke right r from s for o.
- `create subject s` / `create object o`: add a new row/column.
- `destroy subject s` / `destroy object o`: remove a row/column.

Commands execute only if specific preconditions (the presence of certain rights in certain cells) are satisfied.

Theorem 17.1 (HRU Safety Problem). The **safety problem** — given an access matrix and a set of commands, determine whether a specific access right can ever be granted to a specific subject — is undecidable in the general case.

Proof sketch. Lampson showed that the HRU model can simulate a Turing machine. The subjects and objects form the tape, the access rights encode the tape symbols, and the commands simulate the transition function. Since the halting problem for Turing machines is undecidable, determining whether a specific right can ever appear in the matrix is also undecidable. Harrison, Ruzzo, and Ullman (1976) provided the formal proof, showing a reduction from the halting problem to the safety problem for systems with unrestricted command sets.

For restricted models (e.g., commands with a single operation, or systems without subject/object creation), the safety problem may be decidable, but the general result means there is no algorithm that can verify the safety of an arbitrary access control policy. □

The undecidability of the safety problem has profound implications: we cannot, in general, prove that a given access control configuration will never grant an unintended right. This motivates the development of restricted, tractable models such as Bell-LaPadula, Biba, and RBAC.

The matrix is conceptually clean but impractical to store directly — a system with 10,000 users and 1,000,000 files would require a 10^{10}-entry matrix, almost entirely empty. In practice, the matrix is stored in decomposed form: either by column (access control lists) or by row (capability lists).

17.1.2 Access Control Lists (ACLs)

An **access control list** stores the access matrix by column: each object carries a list of (domain, rights) pairs.

Access Control List. For an object o_j, its ACL is the list $\{(d_i, A[d_i, o_j]) \mid A[d_i, o_j] \neq \emptyset\}$.

When a process in domain d attempts to access object o, the system searches o's ACL for an entry matching d and checks whether the requested operation is in the associated rights set. If no entry exists, access is denied.

ACLs are the dominant access control mechanism in Unix, Windows, and most file systems. Their advantages include:

- **Easy per-object auditing**: to see who can access a file, read its ACL.
- **Straightforward revocation**: to revoke a user's access, remove their entry from the ACL.
- **Locality**: the ACL is stored with the object (e.g., in the inode), so access checks require no additional lookups.

Their disadvantage is that determining all objects accessible to a given domain requires scanning every object's ACL — an $O(|\text{objects}|)$ operation. This makes it difficult to audit a user's total permissions.

17.1.3 Capability Lists

A **capability list** stores the access matrix by row: each domain holds a list of (object, rights) pairs, called **capabilities**.

Capability. A capability is an unforgeable token that grants its holder specific access rights to a specific object. Formally, a capability for domain d_i is a pair $(o_j, A[d_i, o_j])$.

Capabilities invert the lookup: a process can enumerate all objects it may access by examining its capability list. Revocation, however, is harder — to revoke access to an object, the system must locate and invalidate all capabilities pointing to that object across all domains.

The **principle of least privilege** is naturally enforced in a capability system: a process holds only the capabilities it needs and cannot acquire new ones without explicit delegation. This contrasts with ACL systems, where a process in a privileged domain (e.g., root) can access any object.

Example 17.2 (ACL vs Capability). Consider revoking user Bob's access to File1.

With an ACL: the system opens File1's ACL, removes Bob's entry, and the change takes effect immediately for subsequent access attempts.

With capabilities: the system must find Bob's capability list (or all capability lists) and remove the capability for File1. If Bob has copied the capability to another process, that copy must also be found and invalidated. Capability systems address this through **indirection tables** (a capability points to an entry in a table, and revocation clears

the table entry) or **generations** (each capability carries a generation number that must match the object's current generation).

Historically, capability-based systems (KeyKOS, EROS, seL4) have offered stronger security properties. File descriptors in Unix are a limited form of capability: a process with an open file descriptor can read/write the file regardless of whether the underlying permissions have changed. However, file descriptors lack the full properties of capabilities (they cannot be freely delegated, and they do not encode fine-grained rights).

17.1.4 Comparing ACLs and Capabilities

Property	ACLs	Capabilities
Storage	Per-object (column of matrix)	Per-subject (row of matrix)
Access check	Search object's ACL for subject	Subject presents capability
Revocation	Easy (edit the ACL)	Hard (find all copies)
Audit (who can access X?)	Easy (read the ACL)	Hard (scan all subjects)
Audit (what can X access?)	Hard (scan all objects)	Easy (read the capability list)
Least privilege	Not natural (ambient authority)	Natural (explicit delegation)
Delegation	Via ACL modification (requires owner rights)	Via capability passing
Confused deputy problem	Vulnerable	Resistant

The **confused deputy problem** (Hardy, 1988) occurs when a privileged program (the "deputy") is tricked into misusing its authority on behalf of an attacker. With ACLs, the deputy's authority is ambient — it can access anything its domain permits, even if the request comes from an untrusted source. With capabilities, the deputy acts only on the capabilities it receives, so the attacker can only trigger operations they already have capabilities for.

17.2 Discretionary Access Control (DAC)

In **discretionary access control**, the owner of a resource decides who may access it. The owner is trusted to set permissions correctly. This is the model used by traditional Unix and Windows systems.

Discretionary Access Control (DAC). An access control policy in which the owner of an object has discretion to grant or revoke access rights to other subjects. The system enforces the policy but does not override owner decisions.

The weakness of DAC is trust delegation: if Alice grants Bob access to a file, Bob can copy the file's contents to a location accessible by Eve. The system cannot prevent this because Bob has legitimate access. DAC protects against accidents and casual snooping, not against determined information exfiltration.

17.2.1 Unix Permission Bits

The traditional Unix permission model encodes access rights in a 12-bit field associated with each inode:

```
┌── setuid (4000)
│┌── setgid (2000)
│└┌─ sticky (1000)
│└│
│└│  ┌── owner (rwx)
│└│  │┌── group (rwx)
│└│  │└┌─ other (rwx)
│└│  │└│
sst  rwx rwx rwx
```

Each `rwx` triple encodes read (4), write (2), and execute (1) permissions. For a regular file, `r` allows reading the file's contents, `w` allows modifying them, and `x` allows executing the file as a program. For a directory, `r` allows listing entries, `w` allows creating or deleting entries, and `x` allows traversing (using the directory in a path lookup).

Example 17.3 (Permission Bits). The permission bits `0755` (octal) decode as:

- Owner: `rwx` (7 = 4 + 2 + 1) — full access
- Group: `r-x` (5 = 4 + 0 + 1) — read and execute
- Other: `r-x` (5 = 4 + 0 + 1) — read and execute

This is the typical permission for an executable binary: the owner can modify it, but everyone can run it.

The permission bits `0644` decode as:

- Owner: `rw-` (6 = 4 + 2 + 0) — read and write
- Group: `r--` (4 = 4 + 0 + 0) — read only
- Other: `r--` (4 = 4 + 0 + 0) — read only

This is the typical permission for a configuration file.

The **umask** controls the default permissions for newly created files. A umask of `022` means that the group write and other write bits are cleared from the default permissions. If a program creates a file with mode `0666` (the typical default for a regular file), the effective permissions are $0666 \,\&\, \sim 0022 = 0644$.

The **setuid** bit (4000) is a domain-switching mechanism: when a setuid executable is run, the process executes with the effective UID of the file's owner rather than the invoking user. The classic example is `/usr/bin/passwd`, which is owned by root and setuid, allowing any user to change their own password by writing to `/etc/shadow` (which requires root access).

The setuid bit is one of the most dangerous features in Unix security. A vulnerability in a setuid-root program gives an attacker full root access. The **setgid** bit (2000) is analogous for group identity. When set on a directory, new files created in that directory inherit the directory's group rather than the creator's primary group.

The **sticky bit** (1000), when set on a directory (such as `/tmp`), prevents users from deleting files they do not own, even if they have write permission on the directory.

17.2.2 Linux Capabilities

Traditional Unix has a binary privilege model: either a process runs as root (UID 0) with full access to everything, or it runs as a regular user with limited access. This violates the principle of least privilege — a program that needs only to bind to a privileged port (port < 1024) runs with full root access.

Linux capabilities decompose root's privileges into approximately 40 distinct capabilities:

Capability	Privilege
`CAP_NET_BIND_SERVICE`	Bind to ports below 1024
`CAP_NET_RAW`	Use raw sockets (ping, packet capture)
`CAP_SYS_ADMIN`	Broad administrative operations (mount, namespace creation)
`CAP_SYS_PTRACE`	Trace and inspect other processes

Capability	Privilege
CAP_DAC_OVERRIDE	Bypass file read/write/execute permission checks
CAP_CHOWN	Make arbitrary changes to file ownership
CAP_KILL	Send signals to any process
CAP_SETUID / CAP_SETGID	Set UID/GID of processes

A program can be granted specific capabilities without running as root:

```c
#include <sys/capability.h>
#include <stdio.h>
#include <unistd.h>

int main(void) {
    /* Get current process capabilities */
    cap_t caps = cap_get_proc();
    if (caps == NULL) {
        perror("cap_get_proc");
        return 1;
    }

    /* Print current capabilities */
    char *text = cap_to_text(caps, NULL);
    printf("Current capabilities: %s\n", text);
    cap_free(text);

    /* Drop all capabilities except CAP_NET_BIND_SERVICE */
    cap_clear(caps);
    cap_value_t cap_list[] = { CAP_NET_BIND_SERVICE };
    cap_set_flag(caps, CAP_EFFECTIVE, 1, cap_list, CAP_SET);
    cap_set_flag(caps, CAP_PERMITTED, 1, cap_list, CAP_SET);

    if (cap_set_proc(caps) == -1) {
        perror("cap_set_proc");
        cap_free(caps);
        return 1;
    }
```

```
    printf("Dropped to CAP_NET_BIND_SERVICE only\n");
    /* Now we can bind to port 80 but cannot do anything else root can do */

    cap_free(caps);
    return 0;
}
```

Containers routinely use capability dropping: by default, Docker and Podman drop all capabilities except a small whitelist, and administrators can further restrict or expand the set.

17.2.3 POSIX ACLs

Traditional Unix permission bits are limited to three categories (owner, group, other). **POSIX ACLs** (IEEE 1003.1e, withdrawn but widely implemented) extend this with fine-grained per-user and per-group entries.

A POSIX ACL consists of entries of the form:

```
type:qualifier:permissions
```

where `type` is one of `user`, `group`, `mask`, or `other`, `qualifier` is a username or group name (empty for the owner or other entries), and `permissions` is a subset of `{r, w, x}`.

```
#include <sys/acl.h>
#include <acl/libacl.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    /* Set an ACL granting user "alice" read-write access to a file */
    acl_t acl = acl_from_text("u::rw-,u:alice:rw-,g::r--,m::rw-,o::r--");
    if (acl == NULL) {
        perror("acl_from_text");
        return 1;
    }

    if (acl_set_file("/tmp/shared.txt", ACL_TYPE_ACCESS, acl) == -1) {
        perror("acl_set_file");
        acl_free(acl);
        return 1;
```

```
    }

    printf("ACL set successfully\n");
    acl_free(acl);
    return 0;
}
```

The `mask` entry acts as an upper bound on the permissions granted to named users and groups: the effective permission for a named user entry is the intersection of the entry's permissions and the mask. This ensures that `chmod` (which modifies the traditional permission bits) remains compatible with POSIX ACLs — `chmod` adjusts the mask, capping all named entries.

Example 17.4 (POSIX ACL Interaction with chmod). A file has the following ACL:

```
user::rw-          (owner: read, write)
user:alice:rw-     (alice: read, write)
group::r--         (owning group: read)
mask::rw-          (mask: read, write)
other::r--         (other: read)
```

Alice's effective permissions are user:alice:rw- $\cap$ mask::rw- = rw-. Now the administrator runs `chmod 640` on the file, which sets the group bits to `r--`. On a system with POSIX ACLs, this changes the mask to `r--`:

```
user::rw-
user:alice:rw-     (effective: r--, because mask is now r--)
group::r--
mask::r--          (changed by chmod)
other::---
```

Alice can now only read the file, even though her ACL entry still says `rw-`. The mask ensures backward compatibility: old tools that use `chmod` still control access as expected, even on files with extended ACLs.

17.3 Mandatory Access Control (MAC)

In **mandatory access control**, the system enforces a policy that individual users cannot override. Even the owner of a file cannot grant access that violates the system-wide policy.

Mandatory Access Control (MAC). An access control policy in which the system assigns security labels to subjects and objects, and a system-wide policy determines

access based on these labels. Neither subjects nor object owners can alter the policy or the labels (except through a trusted administrator or a trusted process).

MAC is essential in environments where information leakage must be prevented by policy — military classified systems, healthcare records, and financial data processing.

17.3.1 The Bell-LaPadula Model (Confidentiality)

The **Bell-LaPadula** (BLP) model, developed for the US Department of Defense in 1973, formalises a **multi-level security** policy focused on protecting **confidentiality**.

Security Levels and Lattice. A set of security levels $L = \{L_1, L_2, \dots, L_n\}$ forms a **lattice** under a dominance relation $\leq$. In the military setting, the standard linear order is:

$$\text{Unclassified} < \text{Confidential} < \text{Secret} < \text{Top Secret}$$

In practice, security labels also include **compartments** (categories): a label is a pair (ℓ, C) where $\ell \in L$ and $C \subseteq \mathcal{C}$ is a set of compartments. The dominance relation is:

$$(\ell_1, C_1) \leq (\ell_2, C_2) \iff \ell_1 \leq \ell_2 \text{ and } C_1 \subseteq C_2$$

Each subject s has a **clearance level** $C(s)$, and each object o has a **classification level** $C(o)$.

The BLP model enforces two properties:

Theorem 17.2 (Bell-LaPadula Properties).

1. **Simple Security Property (No Read Up):** A subject s can read object o only if $C(s) \geq C(o)$. A Secret-cleared analyst cannot read Top Secret documents.
2. **Star Property (No Write Down):** A subject s can write to object o only if $C(o) \geq C(s)$. A Top Secret-cleared analyst cannot write to a Secret-level file, because doing so could leak Top Secret information into a lower-classified container.

Together, these properties ensure that information can only flow upward in the security lattice: from lower classifications to higher ones, never downward.

Proof of information flow containment. Let $I(t)$ denote the set of information possessed by all entities at level $\leq \ell$ at time t. Suppose both properties hold. Any

information that enters a subject at clearance level ℓ can only be written to objects at level $\geq \ell$ (star property), and those objects can only be read by subjects at level $\geq \ell$ (simple security). Therefore $I(t) \subseteq I(t')$ for all $t \leq t'$: no information flows from level ℓ to any level $\ell' < \ell$. $\square$

The BLP model is sometimes summarised as "read down, write up" — though this can be misleading, since subjects typically need to read and write at their own level as well. The formal statement uses $\geq$ and $\leq$ with the subject's current security level.

Example 17.5 (BLP Information Flow). A military intelligence system has four security levels. Analyst Alice has Secret clearance. She:

- Can read Unclassified and Confidential documents (read down: Secret $\geq$ Unclassified, Secret $\geq$ Confidential).
- Can read Secret documents (at her own level).
- Cannot read Top Secret documents (Secret $\ngeq$ Top Secret).
- Can write to Secret and Top Secret containers (write up: Secret $\leq$ Secret, Secret $\leq$ Top Secret).
- Cannot write to Confidential or Unclassified containers (Confidential $\ngeq$ Secret).

This prevents Alice from (accidentally or maliciously) copying Secret information into a Confidential file that lower-cleared personnel could read.

A practical limitation of BLP is the **write-up problem**: if a Top Secret user cannot write to lower levels, they cannot send unclassified emails or create publicly readable documents. Real systems address this with **trusted subjects** that are permitted to violate the star property under controlled conditions (declassification).

17.3.2 The Biba Model (Integrity)

Where BLP protects confidentiality, the **Biba model** (1977) protects **integrity** — the trustworthiness of data.

Biba Integrity Model. Each subject s has an integrity level $I(s)$ and each object o has an integrity level $I(o)$, drawn from a lattice. The Biba strict integrity policy enforces:

1. **Simple Integrity Property (No Read Down):** Subject s can read object o only if $I(o) \geq I(s)$. A high-integrity process should not read low-integrity (potentially corrupted) data.

2. **Integrity Star Property (No Write Up):** Subject s can write to object o only if $I(s) \geq I(o)$. A low-integrity process should not corrupt high-integrity data.

Theorem 17.3 (Biba is the Dual of BLP). The Biba strict integrity model is the

exact dual of the Bell-LaPadula model. If we reverse the lattice order (replacing $\leq$ with $\geq$ and vice versa), the BLP simple security property becomes the Biba simple integrity property, and the BLP star property becomes the Biba integrity star property.

Proof. BLP simple security: s reads o only if $C(s) \geq C(o)$ (subject dominates object). Reversing the lattice: s reads o only if $I(o) \geq I(s)$ (object dominates subject). This is the Biba simple integrity property.

BLP star: s writes o only if $C(o) \geq C(s)$ (object dominates subject). Reversing: s writes o only if $I(s) \geq I(o)$ (subject dominates object). This is the Biba integrity star property.

Information flow: BLP allows flow upward (low confidentiality to high). Biba allows flow downward (high integrity to low). In the reversed lattice, "upward" becomes "downward." □

Example 17.6 (Biba in Practice). Consider a web server with two integrity levels: High (trusted system configuration) and Low (user-uploaded content).

- The web server process runs at High integrity.
- User uploads are stored at Low integrity.
- Under Biba, the web server can write to High and Low files ($I(\text{High}) \geq I(\text{Low})$).
- The web server cannot read Low-integrity user uploads directly ($I(\text{Low}) \ngeq I(\text{High})$). To process user content, the server must use a sanitisation process that validates the data and raises its integrity level.

This prevents an attacker from uploading a malicious file that the high-integrity server process trusts blindly.

17.3.3 Combining BLP and Biba

In practice, systems need both confidentiality and integrity protection. Unfortunately, BLP and Biba can conflict: BLP requires "read down, write up" while Biba requires "read up, write down." A subject that must read lower-classified data for confidentiality purposes cannot read lower-integrity data for integrity purposes.

Resolving this requires careful system design:

- **Chinese Wall model** (Brewer-Nash): prevents conflicts of interest by dynamically restricting access based on which datasets a user has already accessed.
- **Clark-Wilson model**: focuses on well-formed transactions and separation of duty, using certified data items (CDIs) and transformation procedures (TPs) rather than security levels.

Property	Bell-LaPadula	Biba
Goal	Confidentiality	Integrity
Read rule	No read up	No read down
Write rule	No write down	No write up
Information flow	Upward only	Downward only
Threat model	Information leakage	Data corruption

17.4 Role-Based Access Control (RBAC)

Both DAC and MAC have practical limitations. DAC gives users too much discretion; MAC requires rigid labelling that is difficult to manage in large organisations. **Role-Based Access Control** (RBAC) provides a middle ground by introducing an abstraction layer between users and permissions.

Role-Based Access Control (RBAC). An access control model in which permissions are not assigned directly to users but to **roles**. Users are assigned to roles, and they acquire the permissions associated with those roles. Formally:

- U = set of users, R = set of roles, P = set of permissions
- $UA \subseteq U \times R$ = user-role assignment
- $PA \subseteq P \times R$ = permission-role assignment
- A user u has permission p if there exists a role r such that $(u, r) \in UA$ and $(p, r) \in PA$.

RBAC simplifies administration: when an employee changes departments, the administrator changes their role assignment rather than individually revoking and granting hundreds of permissions. RBAC also supports several advanced features:

Role hierarchies: a "senior engineer" role may inherit all permissions from the "engineer" role plus additional ones. Formally, if $r_1 \geq r_2$ in the role hierarchy, then all permissions of r_2 are also permissions of r_1.

Constraints: rules that restrict role assignments. The most important is **separation of duty** — the requirement that no single user holds two conflicting roles (e.g., "cheque preparer" and "cheque approver"). Separation of duty can be **static** (the constraint is checked at role assignment time) or **dynamic** (the constraint is checked at runtime — a user may hold both roles but cannot activate them simultaneously).

Example 17.7 (RBAC in a Hospital). Consider a hospital information system:

Role	Permissions
Doctor	read patient records, write diagnoses, prescribe medication
Nurse	read patient records, update vitals, administer medication
Receptionist	read patient demographics, schedule appointments
Admin	all of the above, plus manage users and roles

When Dr. Smith is hired, the administrator assigns them the Doctor role. Dr. Smith immediately has the correct permissions without any per-file ACL changes. When Dr. Smith moves to administration, the administrator removes the Doctor role and assigns the Admin role.

Static separation of duty might require that no user holds both the "Prescriber" and "Pharmacist" roles, preventing a single person from prescribing and dispensing a controlled substance.

Programmer's Perspective: RBAC in Linux and Go. Linux implements a coarse form of RBAC through **groups**: each user belongs to one or more groups, and file permissions include a group access class. More sophisticated RBAC is available through PAM modules and SELinux roles.

In Go web applications, RBAC is typically implemented with middleware that checks the authenticated user's roles against the required permissions for each endpoint:

```go
package main

import (
    "net/http"
    "strings"
)

type Role string

const (
    RoleAdmin  Role = "admin"
    RoleEditor Role = "editor"
    RoleViewer Role = "viewer"
)

// permissions maps roles to permitted HTTP method prefixes.
```

```go
var permissions = map[Role][]string{
    RoleAdmin:  {"GET", "POST", "PUT", "DELETE"},
    RoleEditor: {"GET", "POST", "PUT"},
    RoleViewer: {"GET"},
}

func RequireRole(roles ...Role) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            userRole := Role(r.Header.Get("X-User-Role")) // simplified
            for _, required := range roles {
                if userRole == required {
                    for _, method := range permissions[required] {
                        if strings.EqualFold(r.Method, method) {
                            next.ServeHTTP(w, r)
                            return
                        }
                    }
                }
            }
            http.Error(w, "Forbidden", http.StatusForbidden)
        })
    }
}

func main() {
    mux := http.NewServeMux()
    mux.Handle("/api/articles",
        RequireRole(RoleAdmin, RoleEditor, RoleViewer)(
            http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
                w.Write([]byte("Article data"))
            }),
        ),
    )
    mux.Handle("/api/admin/users",
        RequireRole(RoleAdmin)(
            http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
                w.Write([]byte("User management"))
            }),
        ),
```

```
    )
    http.ListenAndServe(":8080", mux)
}
```

This is a DAC-style implementation of RBAC at the application level. For system-level RBAC enforcement, SELinux provides mandatory role-based policies that the application cannot override.

17.5 Linux Security Modules: SELinux and AppArmor

The Linux Security Modules (LSM) framework provides a set of hooks in the kernel that allow **pluggable security modules** to make access control decisions. When a process attempts an operation (open a file, create a socket, send a signal), the kernel invokes the LSM hooks, and the active security module can permit or deny the operation.

The LSM architecture is deliberately minimal: it provides only hooks, not policy. Policy is defined by the security module itself. The LSM hooks are located at **authoritative** points in the kernel — after standard DAC checks but before the operation is performed.

```
User-space process: open("/etc/shadow", O_RDONLY)
    │
    ▼
Kernel VFS layer:
    1. Path lookup (resolve "/etc/shadow" to inode)
    2. DAC check (traditional permission bits)
       │
       ▼ (if DAC permits)
    3. LSM hook: security_inode_permission(inode, MAY_READ)
       │
       ├── SELinux: check type enforcement rules
       │   allow httpd_t shadow_t:file { read }?  → DENY
       │
       └── AppArmor: check profile rules
           /etc/shadow r?  → DENY
    │
    ▼ (if LSM permits)
    4. Perform the operation
```

17.5.1 SELinux

Security-Enhanced Linux (SELinux), developed by the NSA and contributed to the Linux kernel in 2003, implements a comprehensive MAC policy engine.

SELinux assigns a **security context** (also called a **label**) to every process and every object. A security context has the form:

```
user:role:type:level
```

For example, `system_u:system_r:httpd_t:s0` labels the Apache web server process. The policy rules determine what operations are permitted based on these labels. The central mechanism is **Type Enforcement** (TE): rules of the form:

```
allow httpd_t httpd_config_t:file { read getattr open };
allow httpd_t httpd_log_t:file { append create getattr open };
deny httpd_t shadow_t:file { read write };
```

The first rule permits any process with type `httpd_t` to read, getattr, and open any file with type `httpd_config_t`. Everything not explicitly allowed is denied (default-deny). A typical SELinux policy for a RHEL system contains tens of thousands of rules.

SELinux also enforces **Multi-Level Security (MLS)**, implementing the Bell-LaPadula model with security levels and compartments. The `level` field in the security context carries the MLS label.

SELinux operates in three modes:

- **Enforcing**: policy violations are denied and logged.
- **Permissive**: policy violations are logged but not denied (useful for policy development).
- **Disabled**: SELinux is not active.

Example 17.8 (SELinux in Action). A web server compromise:

```
# Attacker exploits a vulnerability in Apache (type: httpd_t)
# Attacker tries to read /etc/shadow (type: shadow_t)

# Without SELinux: Apache runs as root or www-data.
#   If www-data, DAC blocks the read (shadow is 640, root:shadow).
#   But if the attacker escalates to root, DAC is bypassed.

# With SELinux: even if the attacker escalates to root within
# the httpd_t domain, SELinux denies the read because there is
# no "allow httpd_t shadow_t:file { read }" rule.
```

```
$ ausearch -m avc -ts recent
type=AVC msg=audit(1650000000.000:100): avc: denied { read }
  for pid=1234 comm="httpd" name="shadow"
  scontext=system_u:system_r:httpd_t:s0
  tcontext=system_u:object_r:shadow_t:s0
  tclass=file permissive=0
```

The SELinux denial is logged in the audit log, providing forensic evidence of the attack attempt.

17.5.2 AppArmor

AppArmor takes a different approach: instead of labelling every object, it confines individual programs using **profiles** that specify what files, capabilities, and network operations each program may access.

```
# /etc/apparmor.d/usr.sbin.nginx
/usr/sbin/nginx {
  /etc/nginx/**          r,
  /var/log/nginx/**      w,
  /var/www/html/**       r,
  /run/nginx.pid         rw,
  /proc/sys/net/core/somaxconn r,
  network inet tcp,
  network inet6 tcp,
  capability net_bind_service,
  capability setuid,
  capability setgid,
  deny /etc/shadow      r,
  deny /root/**         rwx,
}
```

This profile confines Nginx to reading its configuration, writing to its logs, serving files from `/var/www/html/`, and binding to privileged ports. Any attempt to access other paths is denied.

AppArmor profiles can be in **enforce** mode (violations are denied) or **complain** mode (violations are logged but permitted). The `aa-genprof` tool can generate profiles automatically by monitoring a program's behaviour and recording the resources it accesses.

Feature	SELinux	AppArmor
Approach	Label-based (all objects)	Path-based (per program)
Granularity	Very fine (type enforcement)	Moderate (file paths, capabilities)
Complexity	High (requires labelling the entire filesystem)	Lower (profiles per application)
Default distro	RHEL, Fedora, CentOS	Ubuntu, SUSE
Policy language	Type Enforcement rules	Path glob rules
Rename resistance	Labels follow inodes (rename-safe)	Paths may break on rename

17.6 Sandboxing

Sandboxing goes beyond access control lists and labels: it restricts a process's ability to **make system calls at all**, dramatically reducing the attack surface. If a process cannot call `open()`, it cannot open files, regardless of what DAC or MAC says.

17.6.1 seccomp-bpf

The **seccomp** (secure computing) facility in Linux, extended with BPF filters (seccomp-bpf), allows a process to install a filter that specifies which system calls it may invoke and with which arguments.

The original seccomp mode (Linux 2.6.12) was brutally simple: a process called `prctl(PR_SET_SECCOMP, SECCOMP_MODE_STRICT)`, and from that point it could only use `read()`, `write()`, `exit()`, and `sigreturn()`. Any other system call terminated the process.

seccomp-bpf (Linux 3.5) replaced this with a programmable filter using the BPF (Berkeley Packet Filter) instruction set:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/prctl.h>
#include <linux/seccomp.h>
#include <linux/filter.h>
```

```c
#include <linux/audit.h>
#include <sys/syscall.h>
#include <stddef.h>

/* BPF filter that allows only read, write, exit, and sigreturn */
static struct sock_filter filter[] = {
    /* Load the syscall number */
    BPF_STMT(BPF_LD | BPF_W | BPF_ABS,
             offsetof(struct seccomp_data, nr)),
    /* Allow read (0) */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_read, 0, 1),
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),
    /* Allow write (1) */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_write, 0, 1),
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),
    /* Allow exit_group (231) */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_exit_group, 0, 1),
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),
    /* Allow sigreturn (15) */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_rt_sigreturn, 0, 1),
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),
    /* Kill the process for anything else */
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_KILL),
};

int main(void) {
    struct sock_fprog prog = {
        .len = sizeof(filter) / sizeof(filter[0]),
        .filter = filter,
    };

    printf("Installing seccomp filter...\n");

    /* Required before installing a filter without CAP_SYS_ADMIN */
    if (prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) == -1) {
        perror("prctl(NO_NEW_PRIVS)");
        return 1;
    }

    if (prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, &prog) == -1) {
```

```
        perror("prctl(SECCOMP)");
        return 1;
    }

    /* This works: write is allowed */
    const char msg[] = "Sandboxed! Only read/write/exit allowed.\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);

    /* This would kill the process: open is not allowed */
    /* FILE *f = fopen("/etc/passwd", "r"); */

    return 0;
}
```

After installing the filter, the process can only invoke read, write, exit_group, and sigreturn. Any other system call — open, mmap, fork, execve — results in immediate process termination (or, with `SECCOMP_RET_ERRNO`, an error return). This is extraordinarily restrictive, but it is exactly how Chrome's renderer processes and OpenSSH's post-authentication child are sandboxed.

The seccomp filter is inherited across `fork()` and `execve()`, and once installed, it cannot be removed or relaxed. Multiple filters can be stacked (each filter is applied in sequence), providing defence in depth.

17.6.2 pledge and unveil (OpenBSD)

OpenBSD provides two complementary system calls for sandboxing that are simpler than seccomp-bpf but equally effective:

- **`pledge(promises, execpromises)`**: restricts the process to a set of named promise groups. Each promise group covers a set of system calls:
  - `"stdio"`: read, write, close, dup, mmap, clock_gettime, and other basic operations.
  - `"rpath"`: read-only filesystem access (open for reading, stat, readdir).
  - `"wpath"`: write filesystem access.
  - `"cpath"`: create/delete filesystem entries.
  - `"inet"`: Internet socket operations.
  - `"dns"`: DNS resolution.
  - `"proc"`: fork, exec, wait.

  Once pledged, the process cannot expand its promises.

- **`unveil(path, permissions)`**: restricts the process's filesystem view. After

calling `unveil("/var/www", "r")` and `unveil(NULL, NULL)` (to lock the set), the process can only see `/var/www`, and only for reading.

Example 17.9 (pledge + unveil for a Web Server). An OpenBSD web server might execute:

```
/* During initialisation (before handling requests): */
unveil("/var/www/html", "r");     /* serve static files */
unveil("/var/log/httpd", "w");    /* write logs */
unveil("/etc/ssl/certs", "r");    /* TLS certificates */
unveil(NULL, NULL);                /* lock: no more unveil calls */

pledge("stdio rpath wpath inet", NULL);
/* Now the server can:
   - do basic I/O (stdio)
   - read files under /var/www/html and /etc/ssl/certs (rpath)
   - write to /var/log/httpd (wpath)
   - use TCP/UDP sockets (inet)
   It CANNOT:
   - fork/exec (no "proc")
   - modify files outside unveiled paths
   - access the network beyond inet
   - load kernel modules, mount filesystems, etc.
*/
```

Even if an attacker exploits a vulnerability in the server, they cannot write files outside `/var/log/httpd`, cannot execute commands, and cannot access anything outside the unveiled paths. The kernel enforces these restrictions — they cannot be bypassed from user space.

17.6.3 Containers as Isolation Boundaries

Linux containers (discussed in detail in Chapter 18) provide process-level isolation using namespaces and cgroups. From a security perspective, a container restricts:

- **Visibility**: PID namespaces hide host processes; mount namespaces hide the host filesystem.
- **Network access**: Network namespaces isolate network interfaces and routing tables.
- **System calls**: Default seccomp profiles block dangerous syscalls (e.g., `reboot`, `mount`, `kexec_load`).
- **Capabilities**: Containers drop most Linux capabilities by default, retaining

only those needed for normal operation.

- **Resource limits**: Cgroups prevent a container from exhausting CPU, memory, or I/O bandwidth.

However, containers share the host kernel, so a kernel vulnerability can breach containment. This is fundamentally different from virtualisation, where each guest has its own kernel. A container escape (privilege escalation from inside a container to the host) is a kernel exploit.

Programmer's Perspective: Go's syscall.AllThreadsSyscall for Sandbox Enforcement. Go's runtime multiplexes goroutines across OS threads. When a security-sensitive operation must apply to all threads in a process (such as installing a seccomp filter or calling `prctl`), Go provides `syscall.AllThreadsSyscall`:

```go
package main

import (
    "fmt"
    "syscall"
)

func main() {
    // PR_SET_NO_NEW_PRIVS must be set on ALL threads before seccomp.
    // Go's runtime may have multiple OS threads (one per GOMAXPROCS).
    r1, r2, err := syscall.AllThreadsSyscall(
        syscall.SYS_PRCTL,
        uintptr(0x26), // PR_SET_NO_NEW_PRIVS = 38
        1, 0,
    )
    if err != 0 {
        fmt.Printf("AllThreadsSyscall failed: %v\n", err)
        return
    }
    fmt.Printf("PR_SET_NO_NEW_PRIVS set on all threads: r1=%d r2=%d\n", r1, r2)

    // After this, a seccomp filter can be installed.
    // Without AllThreadsSyscall, a goroutine might be scheduled
    // on a thread that was NOT sandboxed, creating a security gap.
}
```

This function ensures the system call executes on **every** OS thread managed by the Go runtime — critical because seccomp filters and prctl settings are per-thread in

Linux. Without `AllThreadsSyscall`, a goroutine might be scheduled on a thread that was not sandboxed, creating a security gap.

Linux **namespaces** are the building blocks of containers. Each namespace type isolates a different resource: PID namespaces give each container its own PID 1, mount namespaces give each container its own filesystem tree, and user namespaces allow unprivileged users to appear as root inside the container. We will examine the namespace API in detail in Chapter 18.

17.7 Hardware Security: Spectre and Meltdown

In January 2018, the disclosure of the **Spectre** and **Meltdown** vulnerabilities shattered a fundamental assumption of operating system security: that the hardware correctly enforces privilege boundaries during speculative execution.

17.7.1 Speculative Execution Background

Modern processors execute instructions **speculatively**: when encountering a conditional branch, the CPU predicts the outcome and begins executing the predicted path before the branch condition is resolved. If the prediction is wrong, the CPU rolls back the architectural state (registers, memory writes) and executes the correct path.

Speculative execution is essential for performance: a modern out-of-order CPU can have 100–200 instructions in flight simultaneously. Without speculation, every branch would stall the pipeline for 15–30 cycles (the branch resolution latency).

The key insight of Spectre and Meltdown is that although the architectural state is rolled back on a misprediction, **microarchitectural state** — specifically, the contents of the CPU cache — is not. An attacker can use cache timing to infer data that was transiently accessed during speculative execution.

17.7.2 Meltdown (CVE-2017-5754)

Meltdown. An attack that exploits out-of-order execution on Intel (and some ARM) processors to read kernel memory from user space. The attack has three phases:

1. **Transient access**: the attacker executes a load instruction that reads a byte from a kernel address. This load will eventually raise a page fault (because user mode cannot access kernel memory), but the CPU executes it speculatively before the fault is raised, placing the kernel byte in a register.
2. **Encoding**: the transiently loaded byte is used as an index into an attacker-controlled **probe array** (256 pages, one per possible byte value). This

brings one specific page of the probe array into the L1 cache.

3. **Extraction**: the CPU detects the fault and rolls back the register state, but the probe array page remains in cache. The attacker times accesses to each of the 256 pages; the page that loads fast (cache hit) reveals the value of the kernel byte.

In pseudocode, the attack is remarkably simple:

```
/* Meltdown attack kernel */
uint8_t probe[256 * 4096];  /* 256 pages, spaced by page size */

/* Flush the probe array from cache */
for (int i = 0; i < 256; i++)
    _mm_clflush(&probe[i * 4096]);

/* Transient execution: read kernel byte, encode in cache */
/* (suppressing the page fault via exception handling or TSX) */
uint8_t kernel_byte = *(uint8_t *)KERNEL_ADDRESS;  /* faults! */
uint8_t dummy = probe[kernel_byte * 4096];          /* brings one page into cache */

/* After fault recovery: time each probe page */
for (int i = 0; i < 256; i++) {
    uint64_t t0 = __rdtsc();
    uint8_t dummy2 = probe[i * 4096];
    uint64_t dt = __rdtsc() - t0;
    if (dt < THRESHOLD) {
        printf("Kernel byte = 0x%02x\n", i);
        break;
    }
}
```

Meltdown allowed a user-space process to read the entire physical memory of the machine at speeds of up to 500 KB/s, including kernel data structures, other processes' memory, and cryptographic keys.

17.7.3 Spectre (CVE-2017-5753, CVE-2017-5715)

Spectre exploits **branch prediction** rather than out-of-order execution of faulting loads.

Variant 1 (Bounds Check Bypass): The attacker mistrains the branch predictor so that an array bounds check is speculatively bypassed:

```
if (x < array1_size) {           /* branch prediction: taken (trained) */
    y = array2[array1[x] * 256]; /* speculatively executed even when x >= array1_size *
}
```

By choosing x to be an out-of-bounds value that indexes into sensitive memory (e.g., kernel memory mapped in the same address space), the attacker can leak arbitrary data through the cache side channel, just as in Meltdown.

Variant 2 (Branch Target Injection): The attacker poisons the **Branch Target Buffer (BTB)** to redirect indirect branches to a gadget that leaks data. This is more powerful than Variant 1 because it can attack any indirect branch in the kernel or other processes.

Unlike Meltdown, Spectre affects virtually all modern processors (Intel, AMD, ARM, RISC-V) and cannot be fully mitigated in software. It requires a combination of:

- **Compiler barriers** (`lfence` after bounds checks) for Variant 1.
- **Retpolines** (a compiler technique that replaces indirect branches with a return-based trampoline that prevents BTB poisoning) for Variant 2.
- **Microcode updates** (IBRS, IBPB, STIBP) that flush or restrict branch prediction state.
- **Hardware redesign** in newer processor generations.

17.7.4 KPTI: Kernel Page Table Isolation

The primary software mitigation for Meltdown is **Kernel Page Table Isolation** (KPTI), also known by its earlier name KAISER.

Kernel Page Table Isolation (KPTI). A mitigation that maintains two separate page tables for each process:

1. **User page table**: maps user-space memory and a minimal kernel trampoline (just enough to handle the transition to kernel mode). The rest of the kernel is **unmapped**.
2. **Kernel page table**: maps both user-space and kernel memory (the full address space).

When the CPU is in user mode, it uses the user page table. Upon a system call or interrupt, the trampoline switches to the kernel page table. Upon return to user mode, it switches back.

Because the kernel is unmapped in user mode, a Meltdown-style speculative load cannot access kernel memory — there is simply no page table entry for it, and the speculative load reads zeroes or garbage from unmapped memory.

The cost of KPTI is performance: every system call and interrupt requires a page table switch (a write to the CR3 register on x86), which flushes the TLB. On workloads with high system call rates, KPTI can impose a 5–30% performance overhead, depending on the processor's support for **PCIDs** (Process Context Identifiers) that allow TLB entries from different page tables to coexist.

Example 17.10 (KPTI and Spectre Status). On a Linux system, the mitigation status can be checked:

```
$ cat /sys/devices/system/cpu/vulnerabilities/meltdown
Mitigation: PTI

$ cat /sys/devices/system/cpu/vulnerabilities/spectre_v1
Mitigation: usercopy/swapgs barriers and __user pointer sanitization

$ cat /sys/devices/system/cpu/vulnerabilities/spectre_v2
Mitigation: Retpolines, IBPB: conditional, IBRS_FW, STIBP: conditional, RSB filling

$ cat /sys/devices/system/cpu/vulnerabilities/spec_store_bypass
Mitigation: Speculative Store Bypass disabled via prctl
```

A system call-intensive benchmark (such as a database performing many small reads) might show a 10–20% throughput reduction with KPTI enabled. Compute-bound workloads (matrix multiplication, rendering) are largely unaffected because they make few system calls.

17.7.5 Beyond Spectre and Meltdown

Since 2018, researchers have discovered numerous additional speculative execution vulnerabilities:

Vulnerability	Year	Mechanism	Affected
Meltdown	2018	Speculative permission bypass on loads	Intel, some ARM
Spectre V1	2018	Bounds check bypass via branch prediction	All modern CPUs
Spectre V2	2018	Branch target injection	All modern CPUs

Vulnerability	Year	Mechanism	Affected
Foreshadow (L1TF)	2018	L1 cache terminal fault	Intel
MDS (Zombieload)	2019	Microarchitectural data sampling	Intel
Spectre-BHB	2022	Branch history buffer poisoning	Intel, ARM

The lesson is clear: **performance optimisations in hardware create security vulnerabilities**. The CPU's speculative execution engine is a massive trusted computing base that was never designed with adversarial models in mind. The tension between performance and security in hardware is a defining challenge for the next decade.

17.8 Trusted Computing

Trusted computing extends the security boundary below the operating system, into the hardware platform itself. The goal is to establish a **chain of trust** from the moment the machine powers on, so that software at every level can verify that the software below it has not been tampered with.

17.8.1 Trusted Platform Module (TPM)

Trusted Platform Module (TPM). A dedicated cryptographic co-processor soldered to the motherboard (or implemented in firmware as fTPM). A TPM provides:

1. **Platform Configuration Registers (PCRs)**: hash registers that accumulate measurements of the boot process. A PCR can only be extended (new value = hash of old value concatenated with new measurement), never directly written. The extend operation is:

$$\mathrm{PCR}_{\mathrm{new}} = \mathrm{SHA\text{-}256}(\mathrm{PCR}_{\mathrm{old}} \| \mathrm{measurement})$$

2. **Key generation and storage**: the TPM can generate and store RSA/ECC keys that never leave the chip.
3. **Sealing**: encrypting data such that it can only be decrypted when the PCRs contain specific values (i.e., the platform is in a known-good state).

4. **Remote attestation**: proving to a remote party that the platform is running specific software, by signing the PCR values with the TPM's endorsement key.

17.8.2 Secure Boot and Measured Boot

Secure Boot (part of the UEFI specification) enforces that each stage of the boot process — firmware, bootloader, kernel — is signed by a trusted authority. If any component's signature is invalid, the boot halts.

Measured Boot does not block unsigned code; instead, it records a cryptographic measurement (hash) of each boot stage into the TPM's PCRs. Software or a remote verifier can later inspect the PCR values to determine exactly what code was loaded.

```
Power On
  │
  ├── UEFI firmware ──────────── measures itself into PCR[0]
  │         │
  │         ├── Bootloader ────── measures bootloader into PCR[4]
  │         │       │
  │         │       ├── Kernel ── measures kernel into PCR[8]
  │         │       │     │
  │         │       │     ├── initrd ── measures initrd into PCR[9]
  │         │       │     │
  │         │       │     └── OS booted, PCRs contain full chain
  │         │       │
  │         │   Secure Boot: verify signature at each step
  │         │   Measured Boot: hash and extend PCR at each step
```

The combination of secure boot (prevent tampering) and measured boot (detect tampering) provides defence in depth: secure boot stops known-bad code from loading, and measured boot catches code that is not on the blocklist but is nonetheless unexpected.

Example 17.11 (TPM-Sealed Disk Encryption). Linux LUKS full-disk encryption can be configured to seal the encryption key to specific PCR values:

1. During initial setup, the disk encryption key is sealed to the TPM with PCR policy: "decrypt only if PCR[0,4,7,8] match these values."
2. On normal boot, the firmware, bootloader, and kernel are measured. If the measurements match, the TPM releases the key, and the disk decrypts automatically.
3. If an attacker modifies the bootloader (e.g., installs a bootkitobject), the PCR values change, the TPM refuses to release the key, and the disk remains

encrypted.

This is the mechanism behind Windows BitLocker and Linux systemd-cryptenroll with TPM2 support.

17.9 Buffer Overflow Attacks and Mitigations

Buffer overflows are among the oldest and most prevalent classes of security vulnerabilities in systems software. They exploit the fact that C (and to a lesser extent, C++) does not enforce array bounds at runtime.

17.9.1 Stack Smashing

A **stack buffer overflow** occurs when a function writes beyond the bounds of a stack-allocated buffer, overwriting the saved return address on the stack.

Stack Smashing. An attack in which the attacker supplies input that overflows a stack buffer, overwriting the saved return address with the address of attacker-controlled code (or a code gadget). When the function returns, execution jumps to the attacker's chosen address.

Consider the classic vulnerable function:

```c
#include <string.h>

void vulnerable(const char *input) {
    char buffer[64];
    /* No bounds check: if input exceeds 64 bytes, it overflows */
    strcpy(buffer, input);
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        vulnerable(argv[1]);
    }
    return 0;
}
```

The stack frame for `vulnerable` looks like:

```
High addresses
┌──────────────────────────┐
│  Return address (8 bytes) │  <── overwritten by attacker
```

```
├──────────────────────────┤
│  Saved RBP (8 bytes)     │  <── overwritten
├──────────────────────────┤
│  buffer[63]              │
│  ...                     │  <── overflow writes past here
│  buffer[0]               │
└──────────────────────────┘

Low addresses
```

If the attacker provides 80 bytes of input, the last 16 bytes overwrite the saved RBP and return address. By carefully crafting the input, the attacker can redirect execution to arbitrary code.

The first publicly documented stack smashing exploit was the Morris Worm (1988), which exploited a buffer overflow in the `fingerd` daemon. Aleph One's "Smashing the Stack for Fun and Profit" (Phrack, 1996) popularised the technique.

17.9.2 Heap Overflow and Use-After-Free

Stack overflows are not the only type: **heap overflows** corrupt dynamically allocated memory, potentially overwriting function pointers, vtable pointers, or allocator metadata. **Use-after-free** occurs when a program dereferences a pointer to memory that has been freed and potentially reallocated for a different purpose.

Both are common in C/C++ and are the root cause of a significant fraction of real-world exploits.

17.9.3 Return-Oriented Programming (ROP)

Modern mitigations (particularly the NX bit) prevent the attacker from executing code on the stack. **Return-Oriented Programming** (ROP) circumvents this by chaining together small snippets of existing code (called **gadgets**), each ending in a `ret` instruction.

Return-Oriented Programming (ROP). An exploitation technique that constructs arbitrary computations by chaining together short instruction sequences (gadgets) already present in the program's executable code or loaded libraries. Each gadget ends with a `ret` instruction, which pops the next gadget's address from the attacker-controlled stack and jumps to it.

A ROP chain to call `execve("/bin/sh", NULL, NULL)` on x86-64 Linux might look like:

```
Stack layout (attacker-controlled):

┌──────────────────────────────┐
```

```
│ 0x401100: pop rdi; ret      │ ── loads "/bin/sh" address into rdi
├─────────────────────────────┤
│ 0x402000: "/bin/sh" address │
├─────────────────────────────┤
│ 0x401200: pop rsi; ret      │ ── loads 0 into rsi (argv = NULL)
├─────────────────────────────┤
│ 0x0000000000000000          │
├─────────────────────────────┤
│ 0x401300: xor rdx,rdx; ret  │ ── sets rdx = 0 (envp = NULL)
├─────────────────────────────┤
│ 0x401350: mov eax,59; ret   │ ── sets syscall number to 59 (execve)
├─────────────────────────────┤
│ 0x401400: syscall            │ ── executes execve(rdi, rsi, rdx)
└─────────────────────────────┘
```

Each "gadget" is a short sequence like `pop rdi; ret` found somewhere in the existing binary or in libc. The attacker never injects new code — they reuse the code that is already present. This is **Turing-complete**: any computation can be expressed as a ROP chain, given a sufficiently rich set of gadgets.

17.9.4 Defence: ASLR, Stack Canaries, NX Bit

Modern operating systems deploy multiple layers of defence against buffer overflow attacks:

Address Space Layout Randomisation (ASLR):

ASLR. A mitigation that randomises the base addresses of the stack, heap, shared libraries, and (with PIE — Position-Independent Executable) the executable itself each time a process is loaded. An attacker who does not know the address of their target gadget or shellcode cannot construct a reliable exploit.

On a 64-bit Linux system with full ASLR, the stack base has approximately 30 bits of entropy, making brute-force guessing infeasible ($2^{30} \approx 10^9$ attempts, each of which crashes the target process). However, 32-bit systems have much less entropy (~16 bits for the stack), and information leaks can completely defeat ASLR (see exercises).

Stack Canaries:

A **stack canary** (also called a **stack protector** or **stack cookie**) is a random value placed between the local variables and the saved return address. Before the function returns, the compiler-inserted check verifies that the canary has not been modified. If it has, the process is terminated.

```c
/* Compiler inserts this logic (GCC -fstack-protector-strong) */
void vulnerable(const char *input) {
    unsigned long canary = __stack_chk_guard;  /* random value */
    char buffer[64];
    strcpy(buffer, input);

    if (canary != __stack_chk_guard) {
        __stack_chk_fail();  /* terminates the process */
    }
}
```

The canary typically includes a null byte, a newline, and a carriage return at specific positions, making it difficult to overwrite with string functions like `strcpy` that terminate on null bytes.

GCC provides three levels of stack protection:

- `-fstack-protector`: protects functions with local buffers > 8 bytes.
- `-fstack-protector-strong`: protects functions with any local array or address-taken variables.
- `-fstack-protector-all`: protects all functions (highest overhead).

NX Bit (No-eXecute):

NX Bit. A hardware feature (called NX on AMD, XD on Intel, XN on ARM) that marks memory pages as non-executable. With NX enabled, the stack and heap are marked non-executable, so even if an attacker injects shellcode, the CPU refuses to execute it, raising a page fault instead.

The combination of ASLR + stack canaries + NX makes traditional buffer overflow exploitation extremely difficult. However, none is individually sufficient:

- ASLR can be defeated by information leaks.
- Stack canaries can be bypassed by overwriting specific variables without touching the canary (e.g., overwriting a function pointer stored before the canary on the stack).
- NX is bypassed by ROP (which reuses existing code, not injected code).

The principle of **defence in depth** requires all three, plus additional hardening.

Example 17.12 (Checking Mitigations on Linux). The `checksec` tool inspects a binary's security properties:

```
$ checksec --file=/usr/bin/ssh
RELRO           STACK CANARY      NX            PIE
Full RELRO      Canary found      NX enabled    PIE enabled
```

```
$ checksec --file=./vulnerable_demo
RELRO           STACK CANARY      NX            PIE
Partial RELRO   No canary found   NX enabled    No PIE
```

The first binary (OpenSSH) has all mitigations enabled: full RELRO (read-only GOT, preventing GOT overwrite attacks), stack canary, NX, and PIE (position-independent executable, enabling full ASLR). The second (our demo) lacks a canary and position-independent execution — it would be much easier to exploit.

Programmer's Perspective: Memory Safety and Go. Go is largely immune to buffer overflow attacks because:

1. **Bounds checking**: every array and slice access is bounds-checked at runtime. An out-of-bounds access causes a panic (controlled crash), not memory corruption.
2. **No pointer arithmetic**: Go does not permit arbitrary pointer arithmetic. You cannot increment a pointer past the end of an allocation.
3. **Garbage collection**: use-after-free is impossible because the GC will not free memory that is still reachable.
4. **No uninitialised memory**: all variables are zero-initialised. Reading uninitialised memory (a common source of information leaks in C) is impossible.

However, Go's `unsafe` package bypasses all safety guarantees:

```go
package main

import (
    "fmt"
    "unsafe"
)

func main() {
    x := [4]int{10, 20, 30, 40}
    // Unsafe pointer arithmetic: read past the array
    p := unsafe.Pointer(&x[0])
    for i := 0; i < 8; i++ {
        val := *(*int)(unsafe.Add(p, uintptr(i)*unsafe.Sizeof(x[0])))
        fmt.Printf("offset %d: %d\n", i, val)
    }
    // Offsets 4-7 read garbage from the stack --- undefined behaviour.
```

```
    // This code is as dangerous as C.
}
```

Code that uses `unsafe` is as vulnerable as C. The Go compiler and tooling flag `unsafe` usage, and it should be audited rigorously. For systems programming that requires pointer manipulation (e.g., device drivers, memory allocators), consider Rust, which provides `unsafe` blocks with stricter ownership rules and borrow checking even for unsafe code's safe interfaces.

17.10 Putting It All Together: Defence in Depth

No single security mechanism is sufficient. Modern operating systems layer multiple defences, each independent, so that the failure of one does not compromise the system:

Layer	Mechanism	Protects Against
Hardware	NX bit, ASLR (randomised layout)	Code injection, address prediction
Hardware	KPTI, retpolines	Spectre, Meltdown
Hardware	TPM, secure boot	Firmware tampering, rootkits
Kernel	SELinux / AppArmor (MAC)	Privilege escalation, lateral movement
Kernel	seccomp-bpf	Syscall-based attacks, kernel exploits
Kernel	Namespaces, cgroups	Container escape, resource abuse
Kernel	Linux capabilities	Over-privileged processes
Compiler	Stack canaries, fortified functions	Stack smashing, format string attacks
Compiler	RELRO, PIE	GOT overwrite, code reuse attacks
Application	RBAC, capability-based design	Over-privileged processes

Layer	Mechanism	Protects Against
Application	Input validation, bounds checking	Injection attacks

Programmer's Perspective: Linux Namespaces and Container Security. The security of a container depends on the correct composition of multiple kernel mechanisms. A minimal container setup in C uses five namespace types:

```c
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mount.h>

#define STACK_SIZE (1024 * 1024)

static int child_fn(void *arg) {
    (void)arg;

    /* New PID namespace: we are PID 1 */
    printf("Child PID (in namespace): %d\n", getpid());

    /* Set a new hostname */
    sethostname("container", 9);

    /* Mount a new proc for the new PID namespace */
    mount("proc", "/proc", "proc", 0, NULL);

    /* Execute a shell in the isolated namespace */
    char *argv[] = {"/bin/sh", NULL};
    execv("/bin/sh", argv);
    perror("execv");
    return 1;
}

int main(void) {
```

```
    char *stack = malloc(STACK_SIZE);
    if (!stack) {
        perror("malloc");
        return 1;
    }

    /* Create child in new PID, UTS, mount, and network namespaces */
    int flags = CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | CLONE_NEWNET;
    pid_t pid = clone(child_fn, stack + STACK_SIZE, flags | SIGCHLD, NULL);
    if (pid == -1) {
        perror("clone");
        return 1;
    }

    printf("Parent: child PID = %d\n", pid);
    waitpid(pid, NULL, 0);

    free(stack);
    return 0;
}
```

This creates a process isolated in four namespaces: it has its own PID space (where it is PID 1), its own hostname, its own mount table, and its own network stack. Combined with a seccomp filter, dropped capabilities, cgroup resource limits, and a restricted root filesystem, this approximates what container runtimes like `runc` and Podman do.

The key insight is that no single mechanism provides adequate isolation. A container without seccomp can exploit kernel vulnerabilities through dangerous syscalls. A container without capability dropping retains dangerous privileges. A container without a user namespace runs as real root on the host. **Defence in depth is not optional — it is the architecture.**

---

1. **Access Matrix Decomposition.** Given an access matrix with n subjects and m objects, where on average each subject has access to k objects ($k \ll m$), compare the storage costs of (a) the full matrix, (b) ACLs, and (c) capability lists. Under what conditions is each representation most space-efficient? For a system with $n = 10{,}000$ users and $m = 1{,}000{,}000$ files where $k = 100$, calculate the storage for each representation, assuming each right set requires 4 bytes.

2. **BLP Lattice with Compartments.** Consider a Bell-LaPadula system with

security levels $\{U, C, S, TS\}$ (Unclassified $<$ Confidential $<$ Secret $<$ Top Secret) and two compartments $\{A, B\}$. A security label is a pair (ℓ, S) where ℓ is a level and $S \subseteq \{A, B\}$. Define the dominance relation on labels and draw the resulting lattice (a Hasse diagram). How many distinct labels exist? A subject with label $(S, \{A\})$ wishes to write to an object with label $(C, \{A, B\})$. Is this permitted under BLP? Explain carefully.

3. **Biba Dual.** Prove formally that the Biba strict integrity model is the exact dual of Bell-LaPadula: take the BLP properties, reverse the lattice order, and show that the resulting properties are precisely the Biba simple integrity and star properties. Then explain why a system cannot simultaneously enforce both BLP and Biba in their strict forms without using trusted subjects.

4. **seccomp Filter Design.** Design a seccomp-bpf filter (specify the complete allowed syscall set) for a program that: (a) reads a file specified as a command-line argument, (b) performs a computation on the file contents in memory, and (c) writes the result to stdout. The filter should be as restrictive as possible while allowing the program to function. Justify each allowed syscall, including any that are needed for process startup and termination.

5. **Meltdown Attack Mechanics.** Explain why the original Meltdown attack does not affect AMD processors. The answer involves a microarchitectural difference in how Intel and AMD handle permission checks during speculative loads. What is this difference, and why does it prevent the attack? Are there any Meltdown-type attacks that do affect AMD?

6. **ROP Gadget Chain.** Given a binary that contains the following gadgets at known addresses: `pop rdi; ret` at `0x401100`, `pop rsi; ret` at `0x401200`, `xor rdx, rdx; ret` at `0x401300`, `mov eax, 59; ret` at `0x401350`, and `syscall` at `0x401400`. The string `"/bin/sh"` is stored at address `0x402000`. Construct a ROP chain (as a sequence of 8-byte stack values) that invokes `execve("/bin/sh", NULL, NULL)` on x86-64 Linux (syscall number 59). Draw the complete stack layout and trace the execution of each gadget.

7. **Defence Bypass Analysis.** Describe a realistic attack scenario in which an attacker bypasses ASLR on a 64-bit Linux system. Your answer should explain: (a) what type of information leak is used (e.g., format string vulnerability, partial overwrite, timing side channel), (b) how the leaked information enables further exploitation (e.g., calculating gadget addresses), and (c) which additional mitigation (from those discussed in this chapter) could prevent the attack from succeeding even after ASLR is bypassed, and why.

Chapter 18: Virtualisation

"Any problem in computer science can be solved by another level of indirection, except the problem of too many levels of indirection." — David Wheeler (attributed)

Virtualisation is the art of presenting a convincing illusion: making one physical machine appear as many, or making many physical machines appear as one. The operating system itself is a virtualiser — it virtualises the CPU (processes), memory (virtual address spaces), and storage (files). This chapter studies the techniques for virtualising the **entire machine**, allowing multiple operating systems to run simultaneously on the same hardware, and the lighter-weight alternatives that virtualise only the operating system's view of its environment.

We begin with the theoretical foundation — the Popek-Goldberg theorem that defines when hardware is virtualisable — then progress through the two families of hypervisors, the mechanisms for virtualising CPU, memory, and I/O, and finally the container revolution that replaced full machine virtualisation for most workloads.

18.1 Why Virtualise?

The motivations for virtualisation have evolved since IBM's CP-40 in 1967:

- **Server consolidation**: run multiple workloads on a single physical server, improving utilisation from the typical 5–15% to 60–80%. A company that ran 20 physical servers at 10% utilisation each can consolidate to 3 servers at 70% utilisation.
- **Isolation**: bugs or security compromises in one guest do not affect others, because each guest has its own kernel. This is stronger than process isolation, which shares a kernel (and therefore its vulnerabilities).
- **Development and testing**: run multiple OS versions and configurations without dedicated hardware. A developer can test against Ubuntu 22.04, RHEL 9, and Windows Server 2022 simultaneously on a single workstation.
- **Migration**: move running workloads between physical hosts for load balancing and maintenance (**live migration**). The VM's memory is copied to the

destination host while it continues running, with a brief pause at the end (typically < 50 ms) to transfer the final dirty pages.

- **Snapshotting and cloning**: capture the complete state of a VM (memory, disk, CPU registers) and restore it later, or clone it to create identical copies. This enables rapid disaster recovery and reproducible testing environments.
- **Cloud computing**: the entire public cloud model rests on virtualisation — a cloud provider sells virtual machines to tenants who share the same physical infrastructure. Without virtualisation, multi-tenancy would require each tenant to have dedicated hardware.

18.2 Full Virtualisation vs Paravirtualisation

The two fundamental approaches to virtualisation differ in whether the guest OS knows it is virtualised.

Full Virtualisation. A virtualisation technique in which the guest operating system runs unmodified, believing it is executing on bare hardware. The hypervisor intercepts privileged operations and emulates them transparently. The guest's binary is identical to what would run on physical hardware.

Paravirtualisation. A virtualisation technique in which the guest operating system is modified to be aware that it is running in a virtual machine. Instead of executing privileged instructions that must be trapped and emulated, the guest makes explicit **hypercalls** to the hypervisor — analogous to system calls, but from the guest kernel to the hypervisor.

Aspect	Full Virtualisation	Paravirtualisation
Guest modification	None required	Guest kernel must be modified
Performance	Overhead from trapping and emulating	Near-native (hypercalls are efficient)
Hardware support	Requires VT-x/AMD-V (or binary translation)	Can work without hardware support
Guest OS support	Any OS	Only modified kernels
Complexity	Hypervisor handles transparency	Shared between hypervisor and guest
Example	KVM, VMware ESXi	Xen PV, virtio drivers

Aspect	Full Virtualisation	Paravirtualisation

In practice, modern systems use a **hybrid**: the CPU and memory are fully virtualised using hardware support (VT-x/AMD-V, EPT/NPT), while I/O uses paravirtualised drivers (virtio) for performance. This gives the best of both worlds: unmodified guest kernels with near-native I/O performance.

18.3 The Popek-Goldberg Theorem

In 1974, Popek and Goldberg formalised the requirements for a virtualisation-capable architecture. Their result is the theoretical foundation of all hardware-assisted virtualisation.

Instruction Classification. Instructions on a processor can be classified as:

1. **Privileged instructions**: those that trap (cause a processor exception) when executed in user mode. Examples: `HLT`, `LIDT`, `MOV to CR3`.
2. **Sensitive instructions**: those whose behaviour depends on the processor mode (user vs supervisor) or that can alter the machine's configuration. Sensitive instructions are further divided into:
   - **Control-sensitive**: can change the configuration of resources (e.g., load page table base, set interrupt mask).
   - **Behaviour-sensitive**: have different effects depending on the current processor mode (e.g., `POPF` on x86 silently ignores the interrupt flag in user mode but modifies it in kernel mode).

Theorem 18.1 (Popek-Goldberg Virtualisation Theorem, 1974). A computer architecture is **efficiently virtualisable** if the set of sensitive instructions is a subset of the set of privileged instructions.

$$\text{Sensitive Instructions} \subseteq \text{Privileged Instructions}$$

If every sensitive instruction is also privileged, then the hypervisor can run guest code directly in user mode and rely on hardware traps to intercept all sensitive operations. No instruction can silently change system state without the hypervisor's knowledge.

Proof. Consider a hypervisor H that runs a guest OS G in user mode on the bare hardware. H maintains a virtual machine state (virtual registers, virtual memory map, virtual interrupt state) for G.

(1) Any non-sensitive instruction executed by G has the same effect in user mode as it would in kernel mode. H need not intervene — the instruction executes at native speed.

(2) Any sensitive instruction executed by G must be intercepted by H, because if it were not, the instruction could either reveal the true machine state (violating equivalence) or alter it (violating resource control). By assumption, every sensitive instruction is privileged, so it traps when executed in user mode. H catches the trap, examines the instruction, emulates its effect on the virtual machine state, and resumes G.

(3) Non-sensitive instructions execute at native speed (no overhead), and sensitive instructions are trapped and emulated. The guest's behaviour is **equivalent** to execution on bare hardware, and the hypervisor maintains **control** over all resources. □

The x86 architecture famously violated this theorem: several sensitive instructions were not privileged. This is why x86 virtualisation required either binary translation or hardware extensions.

Example 18.1 (Sensitive but Non-Privileged Instructions on x86). The x86 ISA contains 17 instructions that are sensitive but not privileged. The most notable:

Instruction	Sensitivity	Why Problematic
`SGDT`	Reveals GDTR (host state)	Does not trap in user mode
`SIDT`	Reveals IDTR (host state)	Does not trap in user mode
`SLDT`	Reveals LDTR	Does not trap in user mode
`SMSW`	Reveals CR0 bits	Does not trap in user mode
`PUSHF/POPF`	Reveals/sets EFLAGS.IF	Silently ignores IF in user mode
`LAR/LSL`	Reveals segment permissions	Does not trap in user mode

`SGDT` (Store Global Descriptor Table Register) reads the GDTR register and stores its contents in memory. It is sensitive because the value of GDTR differs between

the host and the guest (each has its own GDT). However, on classic x86, `SGDT` is not privileged — it executes without trapping in user mode, revealing the host's GDTR value to the guest.

`POPF` is behaviour-sensitive: in kernel mode, it modifies the interrupt flag (EFLAGS.IF); in user mode, it silently ignores the IF bit. A guest kernel that uses `POPF` to enable interrupts would fail silently.

Solutions: - **VMware (1999)**: binary translation. The hypervisor scanned the guest's instruction stream and replaced sensitive-but-non-privileged instructions with calls to emulation routines. - **Xen (2003)**: paravirtualisation. The guest kernel was modified to use hypercalls instead of sensitive instructions. - **Intel VT-x (2005) / AMD-V (2006)**: hardware extensions that make all sensitive instructions cause VM exits, regardless of privilege level.

18.4 Type-1 Hypervisors

A **Type-1** (bare-metal) hypervisor runs directly on the hardware, with no host operating system beneath it. It is the most common architecture for server virtualisation.

Type-1 Hypervisor (Bare-Metal). A hypervisor that runs directly on the physical hardware, managing all hardware resources and presenting virtual machines to guest operating systems. The hypervisor is the first software to execute after the bootloader. It has direct access to hardware and does not rely on a host OS for device drivers or scheduling.

18.4.1 Xen

Xen (developed at the University of Cambridge, first released 2003) pioneered paravirtualisation on x86. Its architecture is distinctive:

- **Domain 0 (Dom0)**: a privileged guest (typically a Linux kernel) that has direct access to hardware and runs the management toolstack. Dom0 is trusted — it can create, destroy, and configure other domains.
- **Domain U (DomU)**: unprivileged guests that access hardware only through Xen's paravirtualised interfaces or hardware pass-through.
- **Xen hypervisor**: a thin layer (approximately 150,000 lines of code) that handles scheduling, memory management, and trap forwarding.

```
┌──────────────────────────────────────────────────────────────┐
│                      Hardware                                 │
├──────────────────────────────────────────────────────────────┤
│                   Xen Hypervisor                              │
```

```
│  Scheduling | Memory Management | Trap Forwarding            │
├──────────────┼────────────────┼──────────────────────────────┤
│   Domain 0   │   Domain U1    │   Domain U2                  │
│   (Linux)    │   (Linux PV)   │   (Windows HVM)              │
│              │                │                              │
│ Device drivers│  PV drivers    │  Emulated + PV drivers       │
│ Management   │  (front-end)   │  (front-end)                 │
│ toolstack    │                │                              │
│              │                │                              │
│ PV backends  │                │                              │
│ (netback,    │                │                              │
│  blkback)    │                │                              │
└──────────────┴────────────────┴──────────────────────────────┘
```

Paravirtualised Xen guests replace privileged operations with **hypercalls**:

```c
/* Xen paravirtualised guest: requesting a page table update.
   Instead of directly writing to a page table entry (privileged),
   the guest issues a hypercall to Xen. */
#include <xen/xen.h>

static inline int xen_update_pte(pte_t *ptep, pte_t pte) {
    struct mmu_update update;
    update.ptr = virt_to_machine(ptep).maddr;
    update.val = pte_val(pte);
    return HYPERVISOR_mmu_update(&update, 1, NULL, DOMID_SELF);
}

/* Similarly, setting the interrupt flag becomes: */
static inline void xen_safe_halt(void) {
    /* Instead of STI; HLT (privileged), call Xen: */
    HYPERVISOR_sched_op(SCHEDOP_block, NULL);
}
```

The hypercall interface is essentially a system call to the hypervisor, using the `HYPERCALL` instruction (or `INT 0x82` on older Xen) instead of `SYSCALL`/`INT 0x80`.

With the introduction of VT-x/AMD-V, Xen added **HVM** (Hardware Virtual Machine) mode, allowing unmodified guests. Modern Xen uses a hybrid: HVM for CPU virtualisation, PV drivers (virtio-style) for I/O.

18.4.2 KVM

Kernel-based Virtual Machine (KVM), merged into the Linux kernel in 2007, takes a radically different approach: it turns the Linux kernel itself into a Type-1 hypervisor.

KVM leverages the existing Linux kernel for scheduling, memory management, and device drivers, adding only the virtualisation-specific code as a kernel module. The module exposes the `/dev/kvm` device, through which user-space processes (typically QEMU) manage virtual machines.

KVM requires hardware virtualisation support (VT-x or AMD-V). The architecture has three execution modes:

1. **Host kernel mode**: the Linux kernel with KVM loaded, managing physical hardware.
2. **Host user mode**: QEMU (or Firecracker, Cloud Hypervisor, etc.), which handles device emulation and VM management.
3. **Guest mode**: the virtual machine's code, running directly on the CPU in a hardware-isolated context (VMCS on Intel, VMCB on AMD).

The KVM approach has several advantages:

- **Code reuse**: KVM inherits Linux's mature scheduler, memory manager, device drivers, and networking stack. This represents millions of lines of tested code that a from-scratch hypervisor would have to reimplement.
- **Ecosystem**: any Linux tool (cgroups, namespaces, perf, ftrace) works with KVM VMs.
- **Maintenance**: KVM is maintained as part of the Linux kernel, benefiting from the kernel's development process and security updates.

Example 18.2 (KVM Execution Flow). When a guest executes a sensitive instruction:

1. The CPU exits guest mode (**VM exit**) and returns to the host kernel (KVM).
2. KVM examines the exit reason (stored in the VMCS exit reason field). Common exit reasons include: I/O port access, MSR read/write, EPT violation (page fault), CPUID, HLT, and external interrupt.
3. If KVM can handle the exit in-kernel (e.g., a simple MSR read, an EPT violation that just requires mapping a page), it emulates the instruction, updates the VMCS guest state, and resumes the guest (**VM entry** via `VMRESUME`).
4. If the exit requires complex device emulation (e.g., a disk write to an emulated virtio-blk device), KVM returns to QEMU in user space via the `KVM_RUN` ioctl. QEMU emulates the device, then calls `KVM_RUN` again to resume the guest.

The performance-critical path is entirely in-kernel: a VM exit that KVM handles di-

rectly costs approximately 1–3 microseconds (the cost of saving/restoring guest state and executing the emulation logic). Exits that require QEMU add the overhead of a user-kernel round trip (approximately 5–10 microseconds).

Typical VM exit rates vary from hundreds per second (compute-intensive workloads) to hundreds of thousands per second (I/O-intensive workloads). Reducing VM exit frequency is the primary goal of virtualisation optimisations (virtio, EPT, posted interrupts).

18.4.3 VMware ESXi

VMware ESXi is a proprietary Type-1 hypervisor optimised for enterprise server virtualisation. It boots its own microkernel (vmkernel) that provides scheduling, memory management, and a storage stack. Unlike KVM, ESXi does not use a general-purpose OS kernel — the hypervisor is purpose-built, with a minimal attack surface.

VMware pioneered **binary translation** for x86 virtualisation before hardware support existed. The binary translator scanned blocks of guest code, identified sensitive instructions, and replaced them with safe equivalents stored in a translation cache. Translated blocks were cached, so the translation cost was amortised over repeated execution. Today, ESXi uses VT-x/AMD-V with optimisations for large-scale deployments (distributed resource scheduling, fault tolerance, live migration across shared storage).

18.4.4 Trap-and-Emulate with Hardware Assistance

Intel VT-x introduces a new execution mode with two root/non-root levels:

VT-x (Intel Virtualisation Technology). A set of CPU extensions that add **VMX root** and **VMX non-root** execution modes. The hypervisor runs in VMX root mode; guests run in VMX non-root mode. Sensitive instructions executed in VMX non-root mode cause **VM exits** (traps to the hypervisor), regardless of whether they are privileged in the classical sense. This satisfies the Popek-Goldberg condition by hardware design.

The key data structure is the **Virtual Machine Control Structure (VMCS)**, a hardware-managed region that stores:

- **Guest state area**: general-purpose registers, segment registers, control registers (CR0, CR3, CR4), IDTR, GDTR, EFLAGS, RIP, RSP, and interrupt state.
- **Host state area**: hypervisor's register values to restore on VM exit.
- **VM execution control fields**: which events cause VM exits (bitmap of I/O ports, MSR accesses, specific exceptions, interrupt types). These can be

configured per-VM for performance — for example, the hypervisor might not trap CPUID if it does not need to hide CPU features.

- **VM exit information fields**: the reason for the most recent VM exit, the qualifying data (e.g., which I/O port was accessed, what address caused an EPT violation), and the guest's instruction pointer at the time of exit.
- **VM entry control fields**: what to inject on entry (e.g., a pending interrupt).

```
VM Entry (VMLAUNCH / VMRESUME)

┌──────────────────────────────────────┐
│  VMX Root Mode (Hypervisor)          │
│                                      │
│  1. Configure VMCS                   │
│  2. VMLAUNCH ────────────────────┐   │
│                                  │   │
│  4. Handle VM exit               │   │
│  5. VMRESUME ────────────────────┤   │
│                                  │   │
└──────────────────────────────────┘───┘
                                   │
┌──────────────────────────────────┐───┐
│  VMX Non-Root Mode (Guest)       │   │
│                                  ▼   │
│  3. Guest executes                   │
│     - Non-sensitive: native speed    │
│     - Sensitive: VM exit to host     │
│     - VM exit costs ~1-3 us          │
│                                      │
└──────────────────────────────────────┘
```

AMD-V provides an equivalent mechanism with the **VMCB** (Virtual Machine Control Block) and the `VMRUN`/`#VMEXIT` instructions. The VMCB is a 4 KB structure with similar guest state, host state, and control fields. AMD-V also introduced **Nested Page Tables (NPT)** before Intel's EPT, giving AMD an early advantage in memory virtualisation performance.

Programmer's Perspective: The KVM API in C. KVM is controlled through ioctl calls on `/dev/kvm`. The following C program creates a minimal virtual machine that executes a few instructions in 16-bit real mode and then halts:

```c
#include <fcntl.h>
#include <linux/kvm.h>
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <unistd.h>

int main(void) {
    /* Step 1: Open the KVM device */
    int kvm_fd = open("/dev/kvm", O_RDWR);
    if (kvm_fd < 0) { perror("open /dev/kvm"); return 1; }

    /* Step 2: Check API version (must be 12) */
    int api_ver = ioctl(kvm_fd, KVM_GET_API_VERSION, 0);
    printf("KVM API version: %d\n", api_ver);

    /* Step 3: Create a VM (returns a VM file descriptor) */
    int vm_fd = ioctl(kvm_fd, KVM_CREATE_VM, 0);
    if (vm_fd < 0) { perror("KVM_CREATE_VM"); return 1; }

    /* Step 4: Allocate guest physical memory (4 KB page) */
    size_t mem_size = 0x1000;
    void *mem = mmap(NULL, mem_size, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

    /* Guest code (16-bit real mode):
       mov al, 0x42       ; load 0x42 into al
       out 0x01, al        ; write al to I/O port 1 (VM exit)
       mov al, 0x43        ; load 0x43
       out 0x01, al        ; write again (VM exit)
       hlt                  ; halt (VM exit) */
    unsigned char code[] = {
        0xB0, 0x42,       /* mov al, 0x42 */
        0xE6, 0x01,       /* out 0x01, al */
        0xB0, 0x43,       /* mov al, 0x43 */
        0xE6, 0x01,       /* out 0x01, al */
        0xF4,             /* hlt          */
    };
    memcpy(mem, code, sizeof(code));

    /* Step 5: Map guest memory at physical address 0 */
```

```
struct kvm_userspace_memory_region region = {
    .slot = 0,
    .guest_phys_addr = 0,
    .memory_size = mem_size,
    .userspace_addr = (unsigned long)mem,
};
ioctl(vm_fd, KVM_SET_USER_MEMORY_REGION, &region);

/* Step 6: Create a vCPU */
int vcpu_fd = ioctl(vm_fd, KVM_CREATE_VCPU, 0);

/* Step 7: Map the vCPU's kvm_run structure (shared with kernel) */
size_t run_size = ioctl(kvm_fd, KVM_GET_VCPU_MMAP_SIZE, 0);
struct kvm_run *run = mmap(NULL, run_size, PROT_READ | PROT_WRITE,
                           MAP_SHARED, vcpu_fd, 0);

/* Step 8: Set initial segment registers (16-bit real mode) */
struct kvm_sregs sregs;
ioctl(vcpu_fd, KVM_GET_SREGS, &sregs);
sregs.cs.base = 0;
sregs.cs.selector = 0;
ioctl(vcpu_fd, KVM_SET_SREGS, &sregs);

/* Step 9: Set initial general-purpose registers */
struct kvm_regs regs = { .rip = 0, .rflags = 0x2 };
ioctl(vcpu_fd, KVM_SET_REGS, &regs);

/* Step 10: Run the vCPU in a loop */
for (;;) {
    ioctl(vcpu_fd, KVM_RUN, 0);
    switch (run->exit_reason) {
    case KVM_EXIT_IO:
        if (run->io.port == 0x01 &&
            run->io.direction == KVM_EXIT_IO_OUT) {
            char val = *(char *)((char *)run + run->io.data_offset);
            printf("Guest wrote 0x%02X to port 0x01\n", val);
        }
        break;
    case KVM_EXIT_HLT:
        printf("Guest halted.\n");
```

```
            goto done;
        default:
            printf("Unexpected exit reason: %d\n", run->exit_reason);
            goto done;
        }
    }
done:
    close(vcpu_fd);
    close(vm_fd);
    close(kvm_fd);
    munmap(mem, mem_size);
    return 0;
}
```

This program demonstrates the complete KVM lifecycle: create a VM, allocate guest memory, create a vCPU, set registers, and run. The guest code writes two bytes to I/O port 1 (each causing a VM exit that the host handles) and then halts (another VM exit). This is the foundation on which QEMU, Firecracker, Cloud Hypervisor, and crosvm are built.

To compile and run: `gcc -o kvm_demo kvm_demo.c && sudo ./kvm_demo` (requires `/dev/kvm` access, typically via the `kvm` group).

18.5 Type-2 Hypervisors

A **Type-2** hypervisor runs as an application on a host operating system.

Type-2 Hypervisor (Hosted). A hypervisor that runs on top of a conventional host operating system, using the host's process and memory management facilities. The hypervisor is a regular user-space application, and VMs are host processes.

18.5.1 VirtualBox

Oracle VirtualBox is a popular Type-2 hypervisor for desktop use. It runs as a user-space application on Linux, Windows, and macOS, using hardware virtualisation (VT-x/AMD-V) when available and falling back to software techniques otherwise. VirtualBox provides a GUI for VM management, shared folders between host and guest, and seamless mode (guest windows appear alongside host windows).

18.5.2 QEMU

QEMU (Quick Emulator) is a versatile machine emulator and virtualiser that operates in multiple modes:

- **Full system emulation**: QEMU can emulate an entire machine (CPU, memory, devices), allowing a guest of one architecture (e.g., ARM) to run on a host of another (e.g., x86). This uses **Tiny Code Generator (TCG)**, a JIT compiler that translates guest instructions to host instructions at runtime.
- **Hardware-accelerated virtualisation**: when paired with KVM (on Linux), Xen, or macOS Hypervisor.framework, QEMU delegates CPU execution to hardware and provides device emulation. In this mode, QEMU is the user-space component of a Type-1 hypervisor.
- **User-mode emulation**: QEMU can emulate individual Linux binaries of a foreign architecture (e.g., running an ARM binary on x86) by translating system calls and CPU instructions without emulating hardware devices.

The distinction between Type-1 and Type-2 blurs: QEMU/KVM is technically Type-1 (KVM is in the kernel), but the user interacts with QEMU, which runs in user space. The architecture is best described as **split-model**: the CPU and memory are virtualised by the kernel (KVM), while device emulation and VM management are handled by user space (QEMU).

18.6 Binary Translation vs Hardware-Assisted Virtualisation

Before VT-x and AMD-V, full virtualisation on x86 required **binary translation**: the hypervisor scanned the guest's instruction stream, identified sensitive instructions, and replaced them with safe equivalents.

Binary Translation. A virtualisation technique in which the hypervisor translates blocks of guest code at runtime, replacing sensitive or privileged instructions with calls to emulation routines. The translated blocks are cached for reuse in a **translation cache**.

The translation process works on **basic blocks** (sequences of instructions ending at a branch):

1. **Fetch**: read a basic block from the guest's instruction stream.
2. **Scan**: identify sensitive or privileged instructions within the block.
3. **Translate**: replace sensitive instructions with safe equivalents (e.g., replace `SGDT` with a load from a virtual GDTR variable; replace `POPF` with a routine that

correctly emulates IF-sensitive behaviour).

4. **Cache**: store the translated block in the translation cache, indexed by guest address.
5. **Execute**: jump to the translated block. Subsequent executions of the same guest code use the cached translation.

Example 18.3 (Binary Translation of POPF). The guest kernel executes:

```
Guest code:          Translated code:
  pushf                pushf
  popf                 call __emulate_popf
                       ;; __emulate_popf reads the pushed flags,
                       ;; updates the virtual IF in the VMM state,
                       ;; and returns with flags modified appropriately
```

The translation replaces `popf` with a call to an emulation function that correctly handles the interrupt flag. The guest sees the expected behaviour; the host's real interrupt flag is never modified.

Aspect	Binary Translation	Hardware-Assisted
Mechanism	JIT rewriting of guest code	CPU modes (VMX root/non-root)
First-execution cost	Translation overhead	None (direct execution)
Steady-state cost	Near-native (cached translations)	VM exit/entry overhead per trap
Cross-architecture	Yes (QEMU TCG: ARM on x86)	No (guest must match host ISA)
Guest modification	None required	None required
Historical use	VMware (1999–2008), QEMU TCG	VT-x (2006+), AMD-V (2006+)

Hardware-assisted virtualisation is now dominant for same-architecture guests. Binary translation remains essential for cross-architecture emulation (e.g., Apple's Rosetta 2 translating x86 binaries on ARM, QEMU TCG for development and testing of embedded ARM software on x86 desktops).

18.7 Memory Virtualisation

The guest OS manages its own page tables, mapping guest virtual addresses (GVA) to guest physical addresses (GPA). But GPAs are not real physical addresses — they must be translated to host physical addresses (HPA). Memory virtualisation solves this two-level translation problem.

Two-Level Address Translation. In a virtualised system, a memory access requires translating:

1. **GVA to GPA**: the guest's page table maps guest virtual addresses to guest physical addresses (managed by the guest OS).
2. **GPA to HPA**: the hypervisor's mapping translates guest physical addresses to host physical addresses (managed by the hypervisor).

The composition GVA $\rightarrow$ GPA $\rightarrow$ HPA produces the final hardware address.

18.7.1 Shadow Page Tables

Before hardware support, hypervisors maintained **shadow page tables** that directly mapped GVA to HPA, short-circuiting the two-level translation.

Shadow Page Tables. A software technique in which the hypervisor maintains a set of page tables (the "shadow") that map guest virtual addresses directly to host physical addresses. The CPU's MMU uses the shadow tables, not the guest's tables. The guest's page tables exist only as a data structure in guest memory that the hypervisor monitors.

The hypervisor intercepts all guest page table modifications (by write-protecting guest page table pages and trapping on writes). When the guest modifies its page tables, the hypervisor updates the shadow tables to reflect the guest's intended GVA-to-GPA mapping composed with the hypervisor's GPA-to-HPA mapping.

Example 18.4 (Shadow Page Table Walk). Consider a guest accessing virtual address `0x7F00_1000`:

Without virtualisation: The MMU walks the guest's page table hierarchy (PML4 $\rightarrow$ PDPT $\rightarrow$ PD $\rightarrow$ PT), finds that GVA `0x7F00_1000` maps to physical address `0x4000`, and accesses the memory at `0x4000`.

With shadow page tables: The hypervisor has constructed a shadow page table that maps GVA `0x7F00_1000` directly to HPA `0xA000`. This mapping reflects the composition: the guest's GPA `0x4000` is backed by HPA `0xA000` in the hypervisor's memory allocation. The MMU walks the shadow table and accesses HPA `0xA000`. The guest never sees

the shadow table — it sees its own page tables, which appear to map `0x7F00_1000` to `0x4000`.

The overhead arises when the guest modifies its page tables: every page table write causes a page fault (because the hypervisor write-protected the guest's page table pages), triggering a VM exit. The hypervisor then computes the new GVA-to-HPA mapping and updates the shadow table. On a context switch-heavy workload, the guest might modify hundreds of page table entries per millisecond, each causing a VM exit.

Shadow page tables are correct but expensive: every guest page table modification causes a VM exit costing 1–3 microseconds. For workloads with frequent page table changes (e.g., process creation, context switches, memory mapping), the overhead can be 10–30% of total CPU time.

18.7.2 Extended Page Tables (EPT / NPT)

Extended Page Tables (EPT on Intel, Nested Page Tables / NPT on AMD) add a second level of hardware-managed address translation, eliminating the need for shadow page tables.

Extended Page Tables (EPT). A hardware feature that adds a second page table hierarchy, managed by the hypervisor, that translates guest physical addresses (GPA) to host physical addresses (HPA). The CPU's MMU performs a **nested page walk**:

1. Walk the guest's page table to translate GVA to GPA.
2. At each step of the guest walk, walk the EPT to translate the physical address of the guest page table entry from GPA to HPA.
3. After completing the guest walk, walk the EPT one final time to translate the target GPA to HPA.

The guest can modify its own page tables freely without causing VM exits.

The nested walk increases the worst-case memory accesses per TLB miss. With 4-level paging on both the guest and the EPT:

- Each level of the guest page walk requires an EPT walk to find the physical location of the next guest page table page.
- Each EPT walk itself traverses 4 levels.
- Total worst case: 4 guest levels $\times 4$ EPT levels $+ 4$ EPT levels for final GPA $= 20$ memory accesses.

Example 18.5 (EPT Nested Walk Cost). Translating GVA `0x7F00_1000` with 4-level paging:

```
Guest Page Walk:                    EPT Walks:
  PML4[i] at GPA g1                  g1 -> HPA: 4 memory accesses
  PDPT[j] at GPA g2                  g2 -> HPA: 4 memory accesses
  PD[k]   at GPA g3                  g3 -> HPA: 4 memory accesses
  PT[l]   at GPA g4                  g4 -> HPA: 4 memory accesses
  Data    at GPA g5                  g5 -> HPA: 4 memory accesses

Total worst-case memory accesses: 5 * 4 = 20
(or 4 * 4 + 4 = 20, depending on counting convention)
```

Despite this cost, EPT outperforms shadow page tables for most workloads because it completely eliminates VM exits on guest page table modifications. The TLB caches both guest and EPT translations (using tagged TLB entries with a VPID — Virtual Processor ID), so the 20-access worst case is rare in practice. TLB hit rates are typically > 99% for well-behaved workloads.

Theorem 18.2 (EPT vs Shadow Page Table Trade-off). Let E_{shadow} be the exit rate due to guest page table modifications (exits/second) with shadow page tables, C_{exit} be the cost per VM exit, M_{EPT} be the rate of TLB misses, and C_{nested} be the additional cost per TLB miss due to the nested walk. Then:

- Shadow page table overhead: $E_{\text{shadow}} \cdot C_{\text{exit}}$
- EPT overhead: $M_{\text{EPT}} \cdot C_{\text{nested}}$

EPT is preferred when $M_{\text{EPT}} \cdot C_{\text{nested}} < E_{\text{shadow}} \cdot C_{\text{exit}}$, which holds for virtually all server workloads because TLB misses are much less frequent than page table modifications, and C_{nested} (a few hundred nanoseconds) is much less than C_{exit} (1–3 microseconds).

18.8 I/O Virtualisation

I/O is the most challenging aspect of virtualisation: devices are diverse, stateful, and performance-sensitive. The three approaches — emulation, paravirtualisation, and pass-through — form a spectrum of complexity, compatibility, and performance.

18.8.1 Emulated Devices

The simplest approach: the hypervisor presents a virtual device that mimics a real hardware device (e.g., an emulated Intel e1000 NIC or an emulated IDE disk controller). The guest uses standard drivers for these devices, and the hypervisor translates the guest's I/O operations into operations on the real hardware.

This approach is compatible with unmodified guests (they already have drivers for e1000) but slow: every I/O register access causes a VM exit, and the emulation logic

in the hypervisor must faithfully replicate the device's behaviour.

18.8.2 Virtio

Virtio is a standard for paravirtualised I/O devices, designed for efficiency in virtualised environments.

Virtio. A standardised interface for paravirtualised devices, consisting of:

1. **Virtqueues**: shared-memory ring buffers between the guest and the host. Each virtqueue has three regions:
   - **Descriptor table**: an array of descriptors, each pointing to a guest memory buffer.
   - **Available ring**: guest writes descriptor indices here to submit I/O requests.
   - **Used ring**: host writes descriptor indices here to report completions.
2. **Feature negotiation**: the guest and host negotiate which optional features they both support at device initialisation.
3. **Device types**: virtio-net (networking), virtio-blk (block storage), virtio-scsi (SCSI), virtio-fs (filesystem sharing), virtio-gpu (graphics), virtio-vsock (host-guest communication), and others.

The virtqueue design minimises VM exits by **batching**: the guest can queue multiple I/O requests (filling the available ring) before notifying the host (a single VM exit via a PCI register write or `VMCALL`), and the host can complete multiple requests before notifying the guest (a single interrupt injection).

```
Guest (virtio driver)                      Host (virtio backend)
┌──────────────────────┐                  ┌──────────────────────┐
│                      │                  │                      │
│  1. Allocate buffers │                  │                      │
│     in guest memory  │                  │                      │
│                      │   Shared Memory  │                      │
│  2. Write descriptor │   (descriptor    │                      │
│     indices to       │    table, avail  │  4. Read indices from│
│     available ring ──┼──  ring, used  ──┼──>  available ring   │
│                      │    ring)         │                      │
│  3. Notify host   ───┼── VM exit ───────┼──> 5. Process I/O    │
│     (kick)           │                  │     (read/write disk,│
│                      │                  │      send/recv pkt)  │
│                      │                  │                      │
```

```
│  7. Process          │                  │  6. Write indices to │
│     completions  <───┼──────────────────┼─── used ring +       │
│     from used ring   │                  │     inject IRQ       │
│                      │                  │                      │
└──────────────────────┘                  └──────────────────────┘
```

Example 18.6 (Virtio Batching Performance). Consider a guest sending 1000 network packets:

Emulated e1000: Each packet requires multiple I/O register writes (transmit descriptor, doorbell), each causing a VM exit. Total: approximately 2000–3000 VM exits for 1000 packets.

Virtio-net: The guest fills the available ring with 1000 descriptor indices, then kicks once. Total: 1 VM exit for 1000 packets (the host processes all pending descriptors in a single batch). With notification suppression (`VRING_AVAIL_F_NO_INTERRUPT`), even the completion interrupt can be suppressed until the guest polls.

18.8.3 SR-IOV and VFIO

For near-native I/O performance, hardware can be passed directly to a guest.

SR-IOV (Single Root I/O Virtualisation) is a PCI Express standard that allows a single physical device (a **Physical Function**, PF) to present multiple **Virtual Functions** (VFs), each of which can be assigned to a different VM. Each VF has its own PCI configuration space, BAR (Base Address Register) space, and DMA engine, but shares the physical hardware (e.g., the same NIC ports).

SR-IOV. A hardware standard that allows a PCIe device to create multiple lightweight virtual instances (Virtual Functions) that can be directly assigned to VMs. Each VF has its own I/O path, data plane, and DMA engine. The hypervisor is not involved in the data path — packets flow directly between the guest and the VF hardware.

VFIO (Virtual Function I/O) is the Linux framework for assigning PCI devices (or SR-IOV VFs) directly to VMs. VFIO uses the **IOMMU** (VT-d on Intel, AMD-Vi on AMD) to isolate DMA: the assigned device can only DMA to the guest's memory, not to the host or other guests. Without IOMMU protection, a malicious guest could program the device to DMA into arbitrary host memory.

```
Without IOMMU:                      With IOMMU (VT-d/AMD-Vi):
Device DMA -> Physical Memory       Device DMA -> IOMMU -> Physical Memory
(any address!)                      (only guest's pages!)
```

```
│          Physical NIC (SR-IOV)    │
│                                   │
│   PF (host driver manages)        │
│   ├── VF0 ──> VM1 (via VFIO)     │
│   ├── VF1 ──> VM2 (via VFIO)     │
│   └── VF2 ──> VM3 (via VFIO)     │
│                                   │
└───────────────────────────────────┘
          │ DMA protected by IOMMU
```

Method	Latency	Throughput	Guest Modification	VM Migration
Emulated device	High (~100 μs)	Low	None	Easy (virtual hardware)
Virtio	Medium (~10 μs)	High	Virtio driver needed	Easy (standard interface)
SR-IOV / VFIO	Very low (~1 μs)	Near-native	VF driver needed	Difficult (hardware state)

18.9 Containers

Containers provide **OS-level virtualisation**: instead of virtualising the hardware, they virtualise the operating system's view of itself. All containers share the host kernel but have isolated views of processes, filesystems, networks, and resource limits.

18.9.1 Linux Namespaces

Linux Namespace. A kernel feature that partitions system resources so that processes in one namespace cannot see or affect processes in another. Each namespace type isolates a different class of system resource.

Linux provides eight namespace types:

Namespace	Flag	Isolates	Introduced
Mount	`CLONE_NEWNS`	Filesystem mount points	Linux 2.4.19

Namespace	Flag	Isolates	Introduced
UTS	`CLONE_NEWUTS`	Hostname and domain name	Linux 2.6.19
IPC	`CLONE_NEWIPC`	System V IPC, POSIX message queues	Linux 2.6.19
PID	`CLONE_NEWPID`	Process IDs	Linux 2.6.24
Network	`CLONE_NEWNET`	Network interfaces, routing, iptables	Linux 2.6.29
User	`CLONE_NEWUSER`	UIDs/GIDs	Linux 3.8
Cgroup	`CLONE_NEWCGROUP`	Cgroup root directory	Linux 4.6
Time	`CLONE_NEWTIME`	CLOCK_MONOTONI CLOCK_BOOTTIME	Linux 5.6

Namespaces are created via `clone()` with the appropriate flags, or via `unshare()` for an existing process, or via `setns()` to join an existing namespace. Each namespace type has its own semantics:

PID namespace: The first process in a new PID namespace becomes PID 1 (the init process of that namespace). It cannot see processes in the parent namespace or other PID namespaces. The parent namespace, however, can see the container's processes (with different PIDs).

Mount namespace: Each container has its own mount table. Mounting or unmounting a filesystem inside a container does not affect the host or other containers. This is the foundation of container filesystem isolation.

User namespace: UIDs and GIDs inside the container are mapped to different UIDs/GIDs on the host. A process that appears as root (UID 0) inside the container might be mapped to UID 100000 on the host, meaning it has no special privileges on the host system.

Example 18.7 (Namespace Isolation Demonstration). On the host, running `ps aux` shows all processes. Inside a PID namespace:

```
# On host:
```

```
$ ps aux | wc -l
347

# Inside container (new PID namespace):
$ ps aux
USER   PID %CPU %MEM    VSZ   RSS TTY  STAT START TIME COMMAND
root     1  0.0  0.0   4628  3512 pts/0 Ss  12:00 0:00 /bin/sh
root     5  0.0  0.0   7060  1604 pts/0 R+  12:00 0:00 ps aux

# The container sees only its own processes.
# Host sees container's PID 1 as PID 28547 (or similar).
```

The UTS namespace demonstrates hostname isolation:

```
# On host:
$ hostname
production-server

# Inside container:
$ hostname mycontainer
$ hostname
mycontainer

# Back on host:
$ hostname
production-server    # unchanged
```

18.9.2 Cgroups v2

Control groups (cgroups) limit, account for, and isolate the resource usage of process groups. Cgroups v2 (unified hierarchy, the default since Linux 5.0) organises all resource controllers under a single tree.

Cgroups v2. The unified cgroup hierarchy in Linux, which provides per-group resource limits and accounting. Each cgroup (a directory in `/sys/fs/cgroup/`) can have limits set for:

- `cpu.max`: maximum CPU bandwidth (format: `$QUOTA $PERIOD`; e.g., `50000 100000` means 50% of one CPU).
- `cpu.weight`: proportional CPU share (1–10000, default 100).
- `memory.max`: hard memory limit in bytes.
- `memory.high`: soft memory limit (the kernel applies back-pressure via

throttling).

- `io.max`: maximum I/O bandwidth per device.
- `pids.max`: maximum number of processes (prevents fork bombs).

```
# Create a cgroup with 50% CPU and 256 MB memory limit
mkdir /sys/fs/cgroup/mycontainer
echo "50000 100000" > /sys/fs/cgroup/mycontainer/cpu.max
echo "268435456" > /sys/fs/cgroup/mycontainer/memory.max
echo "100" > /sys/fs/cgroup/mycontainer/pids.max

# Move a process into the cgroup
echo $PID > /sys/fs/cgroup/mycontainer/cgroup.procs

# The kernel now enforces:
# - Process can use at most 50% of one CPU core
# - Total memory usage (RSS + cache) cannot exceed 256 MB
# - Cannot create more than 100 processes
```

If the processes in the cgroup exceed the memory limit, the kernel's OOM killer targets processes **within the cgroup**, not the host system. If they exceed the CPU limit, the scheduler throttles them (pauses execution until the next period).

18.9.3 OCI Runtime Specification and runc

The **Open Container Initiative (OCI)** defines two specifications:

1. **Runtime Specification**: how to run a container given a root filesystem (a directory tree) and a configuration file (`config.json`). The configuration specifies namespaces, cgroups, mounts, capabilities, seccomp filters, user mappings, and other isolation parameters.

2. **Image Specification**: how container images are structured — as layers of filesystem changes (tarballs), with manifests and digests for content-addressable storage.

runc is the reference implementation of the OCI runtime specification. It is a standalone binary that creates and runs containers:

```
$ mkdir -p mycontainer/rootfs
$ tar -C mycontainer/rootfs -xf alpine-rootfs.tar
$ cd mycontainer
$ runc spec                    # Generate default config.json
$ runc create mybox            # Create (namespaces, cgroups, mounts)
$ runc start mybox             # Start the container's init process
```

```
$ runc list                    # List running containers
$ runc exec mybox /bin/echo hi # Execute a command inside
$ runc delete mybox            # Clean up
```

Container runtimes like containerd and CRI-O use runc (or a compatible runtime like **crun** in C, or **youki** in Rust) as the low-level component that actually creates the container. The higher-level runtime manages image pulling, storage, networking, and lifecycle.

18.9.4 Podman vs Docker

Both Podman and Docker create and manage OCI containers, but their architectures differ fundamentally:

Aspect	Docker	Podman
Architecture	Client-server: `docker` CLI talks to `dockerd` daemon via socket	Daemonless: `podman` CLI forks containers directly
Root requirement	`dockerd` runs as root (rootless mode added later)	Rootless by default (uses user namespaces)
Process model	All containers are children of `dockerd` PID	Each container is a child of the invoking process
Daemon dependency	If `dockerd` crashes, all containers lose their parent	No single point of failure
Systemd integration	Requires separate systemd unit for dockerd	`podman generate systemd` creates per-container units
Socket exposure	`/var/run/docker.sock` (root-owned, major security risk)	No system-wide socket by default
Compose	docker-compose (v1: Python, v2: Go plugin)	podman-compose (compatible wrapper)
OCI compliance	containerd + runc	conmon (monitor) + runc (or crun)

Aspect	Docker	Podman
Pod support	No native pod concept (added via Compose)	Native pod support (shared namespaces)

Example 18.8 (Rootless Container with Podman). Running a container as an unprivileged user:

```
$ id
uid=1000(alice) gid=1000(alice)

$ podman run --rm -it alpine sh
/ # id
uid=0(root) gid=0(root)
/ # cat /proc/self/uid_map
         0       1000          1
         1     100000      65536
/ # exit

$ podman run --rm alpine cat /etc/hostname
f7a3b2c1d4e5
```

Inside the container, Alice appears as root (UID 0) thanks to user namespace mapping: the container's UID 0 is mapped to Alice's host UID 1000. UIDs 1–65536 in the container are mapped to subordinate UIDs 100000–165535 on the host (configured in `/etc/subuid`).

No daemon, no root privileges, no socket exposure. If Alice's account is compromised, the attacker is confined to Alice's user namespace — they cannot escalate to real root.

The daemonless architecture of Podman eliminates a critical attack surface: Docker's daemon runs as root and listens on a Unix socket (`/var/run/docker.sock`). Any process with access to that socket effectively has root access to the host (it can mount the host filesystem, run privileged containers, and more). Podman's direct-fork model means there is no single process whose compromise grants system-wide control.

Programmer's Perspective: Building a Container Runtime in Go. Go is the dominant language for container infrastructure. Docker, containerd, runc, CRI-O, and Podman are all written in Go. The key system calls are `clone` (with namespace flags), `pivot_root`, `mount`, and `prctl`.

```go
package main
```

```go
import (
    "fmt"
    "os"
    "os/exec"
    "syscall"
)

func main() {
    switch os.Args[1] {
    case "run":
        run()
    case "child":
        child()
    default:
        fmt.Println("Usage: container run <cmd> [args...]")
    }
}

func run() {
    // Re-exec ourselves as "child" in new namespaces.
    // This two-step approach is necessary because some namespaces
    // (e.g., PID) only take effect for child processes, not the
    // calling process.
    cmd := exec.Command("/proc/self/exe",
        append([]string{"child"}, os.Args[2:]...)...)
    cmd.Stdin = os.Stdin
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr
    cmd.SysProcAttr = &syscall.SysProcAttr{
        Cloneflags: syscall.CLONE_NEWUTS |
            syscall.CLONE_NEWPID |
            syscall.CLONE_NEWNS,
        // For rootless: add CLONE_NEWUSER and UidMappings/GidMappings
    }
    if err := cmd.Run(); err != nil {
        fmt.Fprintf(os.Stderr, "run error: %v\n", err)
        os.Exit(1)
    }
}
```

```go
func child() {
    // We are now in new namespaces. In the PID namespace,
    // we are PID 1 (the init process).
    fmt.Printf("Container PID: %d\n", os.Getpid())

    // Set a container hostname (UTS namespace)
    syscall.Sethostname([]byte("container"))

    // Change root filesystem (requires a prepared rootfs)
    syscall.Chroot("/var/lib/containers/rootfs")
    syscall.Chdir("/")

    // Mount /proc for the new PID namespace
    // (without this, ps/top will show host processes)
    syscall.Mount("proc", "/proc", "proc", 0, "")

    // Execute the requested command
    cmd := exec.Command(os.Args[2], os.Args[3:]...)
    cmd.Stdin = os.Stdin
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr
    if err := cmd.Run(); err != nil {
        fmt.Fprintf(os.Stderr, "child error: %v\n", err)
    }

    // Clean up
    syscall.Unmount("/proc", 0)
}
```

This simplified runtime creates new UTS, PID, and mount namespaces, sets a hostname, pivots to a root filesystem, mounts `/proc`, and executes a command. A production runtime (runc, crun) adds: user namespaces with UID/GID mapping, cgroup configuration, seccomp filter installation, capability dropping, AppArmor/SELinux profile loading, proper `pivot_root` (not just `chroot`), console/PTY setup, and signal forwarding.

The Go standard library's `os/exec` and `syscall` packages provide clean abstractions for these operations. The `SysProcAttr` struct maps directly to the `clone()` flags, making Go a natural choice for container runtime development.

18.10 The Virtualisation Spectrum

The techniques discussed in this chapter form a spectrum from heavyweight isolation (full VMs) to lightweight sharing (processes):

```
More isolation                                          Less isolation
More overhead                                           Less overhead
◄──────────────────────────────────────────────────────────────────►

Full VM          Lightweight VM      Container         Process
(KVM+QEMU)       (Firecracker)      (Podman/runc)     (fork+exec)
|                 |                  |                  |
| Own kernel      | Own kernel       | Shared kernel    | Shared kernel
| Own devices     | Minimal devices  | Namespace        | No namespace
| Full hardware   | Reduced attack   |  isolation       |  isolation
|  emulation      |  surface         | Cgroup limits    | ulimit only
|                 |                  |                  |
| ~2-30 s boot    | ~125 ms boot     | ~50-500 ms boot  | ~1 ms fork
| ~200 MB base    | ~5 MB base       | ~1-10 MB base    | ~0 overhead
```

Firecracker (developed by Amazon for Lambda and Fargate) is a lightweight VMM built on KVM that boots a **microVM** in approximately 125 ms with ~5 MB of memory overhead. It provides the security of hardware virtualisation (each function or container gets its own kernel) with near-container startup times. Firecracker achieves this by eliminating most device emulation — it provides only a virtio-net, virtio-blk, and a serial console. No USB, no GPU, no sound, no PCI bus enumeration.

gVisor (Google) takes yet another approach: it implements a user-space kernel (Sentry) that intercepts guest system calls and re-implements them safely. gVisor provides stronger isolation than containers (the host kernel is not directly exposed) with lower overhead than full VMs (no hardware virtualisation).

Programmer's Perspective: Choosing the Right Isolation Level. The choice between VMs and containers is fundamentally a security/performance trade-off:

- **Multi-tenant cloud** (AWS, GCP): use VMs (KVM/Firecracker) for strong isolation between tenants. A kernel vulnerability in a container could let Tenant A access Tenant B's data. AWS Lambda uses Firecracker microVMs — each function invocation gets its own VM.
- **Single-tenant deployment** (your own servers running your own code): containers (Podman) provide sufficient isolation with better density and faster startup.

- **Serverless functions** (AWS Lambda, Cloudflare Workers): Firecracker microVMs or V8 isolates (WebAssembly sandboxes) combine fast startup with hardware-level or language-level isolation.
- **Development and CI/CD**: containers are overwhelmingly preferred for their fast iteration cycle, reproducibility, and ecosystem (registries, Compose files, Kubernetes).

In Go, container orchestration is managed through the `containerd/containerd` client library. For KVM-level virtualisation from Go, the `firecracker-microvm/firecracker-go-sdk` provides a Go API to the Firecracker VMM.

---

1. **Popek-Goldberg Classification.** The ARM architecture (pre-ARMv7) included the `MRS` instruction, which reads the CPSR (Current Program Status Register) without trapping when executed in user mode. Explain why this violates the Popek-Goldberg theorem (classify the instruction as sensitive, and show that it is not privileged). How does ARMv8 (AArch64) address this? What about RISC-V — does it satisfy the Popek-Goldberg requirements natively?
2. **Shadow Page Table Cost Analysis.** Consider a guest with a working set of 1,000 pages. The guest modifies 50 page table entries per millisecond (due to page faults, context switches, etc.). Each page table modification causes a VM exit costing 2 microseconds. (a) Calculate the overhead of shadow page tables as a percentage of elapsed time per second. (b) Now suppose EPT is used instead, eliminating these VM exits but increasing each TLB miss cost from 4 memory accesses to 20 memory accesses. If TLB misses occur at a rate of 10,000 per second and each memory access takes 100 ns, calculate the EPT overhead per second. (c) Which approach is more efficient for this workload, and by how much?
3. **Virtio Ring Buffer Design.** The virtio specification uses a split virtqueue with three regions: descriptor table, available ring, and used ring. (a) Explain why a single shared ring buffer would be insufficient. (b) What concurrency issues does the split design address? (c) How does the available ring's `idx` field enable lock-free operation between guest and host? (d) Explain the role of the `VRING_USED_F_NO_NOTIFY` flag in reducing VM exit frequency.
4. **Namespace Composition.** A process creates a new PID namespace and a new network namespace but does **not** create a new mount namespace. The process then runs `ps aux` and `ip addr show` inside the new namespaces. (a) What does each command show? (b) Why might failing to create a new mount namespace cause `ps` to show unexpected results? (c) What happens if the process tries to

mount a new `/proc` without a mount namespace?

5. **Container Escape Analysis.** Describe a concrete scenario in which a container escapes its isolation and gains host-level access. Your answer should specify: (a) which namespace or cgroup boundary is breached, (b) what kernel vulnerability or misconfiguration enables the escape, (c) what mitigation (from Chapter 17 or this chapter) would have prevented it, and (d) why this escape would not be possible from a KVM virtual machine.

6. **Hypervisor Scheduling.** A Type-1 hypervisor hosts four VMs, each with two vCPUs, on a host with four physical CPUs. Design a scheduling algorithm that ensures: (a) no vCPU starvation, (b) co-scheduling of vCPUs belonging to the same VM when the VM performs inter-vCPU synchronisation (e.g., spinlocks), and (c) fair CPU time distribution across VMs. Discuss the trade-offs between gang scheduling, relaxed co-scheduling, and independent vCPU scheduling. What pathology occurs when a VM's vCPUs are not co-scheduled and the guest uses spinlocks?

7. **Rootless Containers.** Explain the complete mechanism by which Podman runs containers without root privileges. Your answer should cover: (a) user namespace UID/GID mapping and how `/etc/subuid` and `/etc/subgid` are used, (b) how `newuidmap`/`newgidmap` setuid helpers bridge the gap between user namespaces and subordinate ID allocation, (c) what operations are still impossible in a rootless container (and why), (d) how rootless networking works (slirp4netns or pasta), and (e) the security implications compared to Docker's root-daemon model.

Chapter 19: Distributed Operating System Concepts

"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable." — Leslie Lamport

---

A distributed operating system extends the abstractions of a traditional OS — processes, files, memory, communication — across multiple machines connected by a network. The machines may fail independently, the network may lose or reorder messages, and there is no shared clock. These constraints make distributed systems fundamentally harder to reason about than single-machine systems, and the theoretical results that govern their behaviour — the FLP impossibility theorem, the CAP theorem, the bounds on clock synchronisation — are among the deepest in computer science.

This chapter examines the core problems of distributed computing from an OS perspective: how to order events without a global clock, how to achieve mutual exclusion across machines, how to reach agreement (consensus) despite failures, how to build distributed file systems, how to reason about the consistency of shared state, and how to detect failures in a world where silence is ambiguous.

19.1 Motivation and Design Goals

19.1.1 Why Distribute?

Single machines have limits: finite CPU cores, finite memory, finite storage bandwidth, and a single point of failure. When any of these limits is reached, the only option is to spread the workload across multiple machines. But distribution introduces new problems that do not exist on a single machine:

- **Partial failure**: some machines fail while others continue. The system must continue operating (or gracefully degrade) despite failures.
- **No global state**: no single machine knows the complete state of the system. Decisions must be made with incomplete information.

- **Unbounded delay**: messages can be delayed arbitrarily by the network. A slow machine is indistinguishable from a crashed one.
- **Clock skew**: each machine has its own clock, and clocks drift at different rates.
- **Concurrency**: operations on different machines happen truly in parallel, not just interleaved.

19.1.2 Transparency

Transparency. A distributed system provides transparency when it hides the distribution from users and applications. The ISO Reference Model for Open Distributed Processing (RM-ODP, ISO/IEC 10746) identifies eight forms:

1. **Access transparency**: local and remote resources are accessed using identical operations.
2. **Location transparency**: resources are accessed without knowledge of their physical location.
3. **Migration transparency**: resources can move between machines without affecting access.
4. **Replication transparency**: multiple copies of a resource exist without the user's knowledge.
5. **Concurrency transparency**: multiple users can share resources without interference.
6. **Failure transparency**: failures of components are hidden from the user.
7. **Performance transparency**: the system can be reconfigured for performance without user impact.
8. **Scaling transparency**: the system can scale without structural changes.

In practice, perfect transparency is impossible — Deutsch's eight fallacies of distributed computing remind us that the network is not reliable, latency is not zero, bandwidth is not infinite, and topology does change. The art of distributed system design is choosing which transparencies to sacrifice and how to expose the remaining non-transparency to applications gracefully.

19.1.3 System Models

System Model. A distributed system model specifies assumptions about three dimensions:

1. **Process behaviour**:
    - *Crash-stop*: a process either runs correctly or halts permanently. Once crashed, it never recovers.

- *Crash-recovery*: a process may crash and later restart, potentially with persistent state (e.g., from a write-ahead log).
- *Byzantine*: a process may behave arbitrarily, including sending contradictory messages to different recipients, lying, or colluding with other Byzantine processes.

2. **Communication**:
   - *Reliable links*: every message sent is eventually delivered, with no duplication. (Typically built on top of TCP.)
   - *Fair-loss links*: messages may be lost, but a message sent infinitely often is eventually delivered.
   - *Arbitrary links*: messages may be lost, duplicated, corrupted, or delivered out of order.
3. **Timing**:
   - *Synchronous*: known upper bounds on message delay and processing time. An algorithm can use timeouts to detect failures reliably.
   - *Partially synchronous*: bounds exist but are unknown, or hold only after some (unknown) point in time (Global Stabilisation Time).
   - *Asynchronous*: no timing bounds at all. An algorithm cannot distinguish a slow process from a crashed one.

Most practical systems assume crash-recovery processes, reliable communication (built on TCP), and partial synchrony. The theoretical results we study in this chapter often consider the asynchronous model, which gives the strongest impossibility results — if something is impossible in the asynchronous model, no algorithm can guarantee it without timing assumptions.

19.2 Distributed Clocks

In a single-machine OS, all events can be ordered by a single hardware clock. In a distributed system, each machine has its own clock, and these clocks drift at different rates (typically 10–100 ppm, or 1–10 ms/day). The fundamental question is: how do we assign meaningful timestamps to events in the absence of a shared clock?

19.2.1 Happened-Before and Lamport Timestamps

Leslie Lamport's seminal 1978 paper "Time, Clocks, and the Ordering of Events in a Distributed System" introduced the **happened-before** relation and logical clocks. The key insight is that we do not need a global clock — we only need to capture **causal ordering**: if event a could have influenced event b, then a must be ordered before b.

Happened-Before Relation. For events in a distributed system, the happened-

before relation $\rightarrow$ is the smallest transitive relation satisfying:

1. If a and b are events in the same process and a occurs before b in that process's local order, then $a \rightarrow b$.
2. If a is the sending of a message by one process and b is the receipt of that message by another process, then $a \rightarrow b$.
3. If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$ (transitivity).

If neither $a \rightarrow b$ nor $b \rightarrow a$, the events are **concurrent**, written $a \| b$. Concurrency does not mean the events happened at the same real-world time — it means neither could have causally influenced the other.

Lamport Timestamp. Each process P_i maintains a monotonically increasing counter C_i. The timestamp rules are:

1. Before each local event, increment: $C_i \leftarrow C_i + 1$.
2. When sending a message m, attach the current timestamp: send(m, C_i).
3. When receiving a message m with timestamp t: $C_i \leftarrow \max(C_i, t) + 1$.

The Lamport timestamp $L(e)$ of event e is the value of the counter when e occurs.

Theorem 19.1 (Lamport Clock Consistency). If $a \rightarrow b$, then $L(a) < L(b)$.

Proof. By structural induction on the happened-before relation.

Base case (same process): If a and b are in the same process and a occurs before b, then C_i was incremented at least once between a and b (once for each intervening event). Therefore $L(a) < L(b)$.

Base case (message): If a is the send of message m with timestamp $L(a) = C_i$, and b is the receipt of m at process P_j, then $C_j \leftarrow \max(C_j, L(a)) + 1 \geq L(a) + 1$. Therefore $L(b) \geq L(a) + 1 > L(a)$.

Inductive step: If $a \rightarrow b$ and $b \rightarrow c$, then by induction $L(a) < L(b)$ and $L(b) < L(c)$, so $L(a) < L(c)$ by transitivity of $<$. □

Crucially, the **converse does not hold**: $L(a) < L(b)$ does not imply $a \rightarrow b$. Two concurrent events may happen to have different Lamport timestamps. This means Lamport timestamps can provide a **total order** (by breaking ties with process IDs: $(L(a), i) < (L(b), j)$ iff $L(a) < L(b)$ or ($L(a) = L(b)$ and $i < j$)), but they cannot detect concurrency.

Example 19.1 (Lamport Timestamps). Three processes P_1, P_2, P_3:

```
Time -->

P1:  a(1) ──send m1──>           c(5)
```

```
                        \
P2:       d(1)  e(2)    b(3)  ──send m2──>  f(6)
                                                \
P3:            g(1)  h(2)                        i(7)
```

- Event a at P_1: $C_1 = 1$, sends m_1 to P_2.
- Event d at P_2: $C_2 = 1$ (local event).
- Event e at P_2: $C_2 = 2$ (local event).
- Event b at P_2: receives m_1 with timestamp 1. $C_2 = \max(2, 1) + 1 = 3$.
- Events $g(1), h(2)$ at P_3 proceed locally.
- Event c at P_1: $C_1 = 5$ (local events between a and c).
- Event f at P_2: $C_2 = 6$, sends m_2 to P_3.
- Event i at P_3: receives m_2 with timestamp 6. $C_3 = \max(2, 6) + 1 = 7$.

Observations: $a \to b$ (via message m_1), and indeed $L(a) = 1 < 3 = L(b)$. Events a and d are concurrent ($a \| d$): neither happened before the other, even though $L(a) = L(d) = 1$.

19.2.2 Vector Clocks

Vector clocks extend Lamport timestamps to capture **causality precisely**: they can distinguish "happened before" from "concurrent."

Vector Clock. In a system of n processes, each process P_i maintains a vector $V_i[1..n]$ of n counters, initialised to all zeros. The rules are:

1. Before each local event at P_i: increment own entry: $V_i[i] \leftarrow V_i[i] + 1$.
2. When sending a message, attach the entire vector V_i (after incrementing).
3. When receiving a message with vector V_m: for each $j \in \{1, \dots, n\}$, set $V_i[j] \leftarrow \max(V_i[j], V_m[j])$; then increment own entry: $V_i[i] \leftarrow V_i[i] + 1$.

The partial order on vectors is defined componentwise:

$$V(a) \leq V(b) \iff \forall j : V(a)[j] \leq V(b)[j]$$

$$V(a) < V(b) \iff V(a) \leq V(b) \text{ and } V(a) \neq V(b)$$

Two vectors are **incomparable** (concurrent) if neither $V(a) \leq V(b)$ nor $V(b) \leq V(a)$.

Theorem 19.2 (Vector Clock Characterisation). For events a and b in a distributed system with vector clocks:

$$a \to b \iff V(a) < V(b)$$

Events a and b are concurrent ($a \| b$) if and only if neither $V(a) < V(b)$ nor $V(b) < V(a)$.

Proof. The forward direction ($a \to b \Rightarrow V(a) < V(b)$) follows from the same structural induction as Theorem 19.1, extended to vectors. Each component can only increase, and at least one component (the sender's) strictly increases across a message.

The reverse direction ($V(a) < V(b) \Rightarrow a \to b$) uses the fact that $V_i[i]$ is a faithful count of events at P_i: if $V(a)[i] \leq V(b)[i]$ and $V(a) \neq V(b)$, then there exists a causal chain from a to b through the message sends that propagated process i's counter to the process where b occurred. The formal proof appears in Fidge (1988) and Mattern (1989). □

Example 19.2 (Vector Clocks Detecting Concurrency). Two processes P_1 and P_2:

```
P1:  a[1,0] ──send──>        c[1,2]
                      \
P2:       b[0,1]      d[1,2]
                       (received msg from P1)
```

- After event a: $V_1 = [1, 0]$. P_1 sends message to P_2.
- After event b: $V_2 = [0, 1]$ (local event before receiving P_1's message).

Compare $V(a) = [1, 0]$ and $V(b) = [0, 1]$: $V(a)[1] = 1 > 0 = V(b)[1]$, but $V(a)[2] = 0 < 1 = V(b)[2]$. Neither $V(a) \leq V(b)$ nor $V(b) \leq V(a)$, so $a \| b$ — correctly identified as concurrent.

After event d (receiving P_1's message): $V_2 = [\max(0, 1), \max(1, 0) + 1] = [1, 2]$. Compare $V(a) = [1, 0]$ and $V(d) = [1, 2]$: $V(a) \leq V(d)$ and $V(a) \neq V(d)$, so $V(a) < V(d)$, confirming $a \to d$ (as expected, since d received a's message).

The disadvantage of vector clocks is their $O(n)$ space per timestamp, where n is the number of processes. In systems with millions of processes, this is prohibitive. Practical alternatives include:

- **Bounded vector clocks**: truncate to the k most recently updated entries.
- **Dotted version vectors**: compact representation for replicated data stores.
- **Hybrid logical clocks**: combine physical and logical timestamps (see below).

19.2.3 Physical Clock Synchronisation: NTP and PTP

Logical clocks order events causally but do not provide wall-clock time. For timestamps that correspond to real-world time (log correlation, certificate validity, financial transactions, lease expiry), physical clock synchronisation is necessary.

NTP (Network Time Protocol) synchronises clocks over unreliable networks. NTP uses a hierarchical system of **stratum** levels: stratum-0 devices are atomic clocks or GPS receivers; stratum-1 servers connect directly to stratum-0; stratum-2 synchronise from stratum-1; and so on.

NTP achieves accuracy of 1–10 ms over the internet and sub-millisecond on a LAN. The algorithm estimates the clock offset θ as:

$$\theta = \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$$

where t_1 is the client send time (client clock), t_2 the server receive time (server clock), t_3 the server send time (server clock), and t_4 the client receive time (client clock). This formula assumes **symmetric network delay**: the one-way delay from client to server equals the one-way delay from server to client. When this assumption is violated (e.g., asymmetric routing), NTP's accuracy degrades.

Theorem 19.3 (Lundelius-Lynch Clock Synchronisation Bound, 1984). In a system of n processes where message delay is bounded by $[d - u, d]$ (uncertainty u), no deterministic algorithm can synchronise clocks to better than:

$$\epsilon \geq \frac{u}{2}\left(1 - \frac{1}{n}\right)$$

Proof sketch. The proof constructs two executions of the synchronisation protocol that are indistinguishable to $n - 1$ of the n processes but result in different clock values. The adversary shifts message delays within the uncertainty window to create these indistinguishable executions. Since no process can tell which execution it is in, it must set its clock to the same value in both executions, leading to a synchronisation error of at least $u(1 - 1/n)/2$. □

PTP (Precision Time Protocol, IEEE 1588) achieves sub-microsecond synchronisation using **hardware timestamping**: network interface cards timestamp packets at the physical layer, eliminating the jitter introduced by operating system scheduling, interrupt handling, and network stack processing. PTP is used in financial trading (where nanosecond precision is valuable), telecommunications (5G synchronisation), and industrial control (motion control, power grid synchronisation).

Programmer's Perspective: Hybrid Logical Clocks in Go. Go's `time.Now()` returns the local wall clock, which is subject to NTP adjustments (including backwards jumps when NTP corrects a fast clock). For measuring elapsed time, always use `time.Since()` with a monotonic clock reading (Go automatically tracks monotonic time for `Time` values created by `time.Now()`).

For distributed systems, **hybrid logical clocks** (HLCs) combine physical and logical timestamps, giving the best of both worlds: timestamps that are close to real time and that capture causal ordering.

```go
package hlc

import (
    "sync"
    "time"
)

// Timestamp is a hybrid logical clock value.
type Timestamp struct {
    WallTime int64  // physical time (nanoseconds since epoch)
    Logical  uint32 // logical counter for events at same wall time
}

// Less reports whether ts < other.
func (ts Timestamp) Less(other Timestamp) bool {
    if ts.WallTime != other.WallTime {
        return ts.WallTime < other.WallTime
    }
    return ts.Logical < other.Logical
}

// Clock implements a hybrid logical clock.
type Clock struct {
    mu  sync.Mutex
    now func() time.Time // injectable time source for testing
    ts  Timestamp
}

// NewClock creates a new HLC using the given time source.
func NewClock(now func() time.Time) *Clock {
    return &Clock{now: now}
```

```go
}

// Now returns a new timestamp for a local event.
func (c *Clock) Now() Timestamp {
    c.mu.Lock()
    defer c.mu.Unlock()

    pt := c.now().UnixNano()
    if pt > c.ts.WallTime {
        // Physical clock advanced: reset logical counter
        c.ts.WallTime = pt
        c.ts.Logical = 0
    } else {
        // Physical clock did not advance (same nanosecond
        // or NTP jumped backwards): increment logical counter
        c.ts.Logical++
    }
    return c.ts
}

// Update merges a received timestamp with the local clock.
// Called when a message with timestamp 'received' arrives.
func (c *Clock) Update(received Timestamp) Timestamp {
    c.mu.Lock()
    defer c.mu.Unlock()

    pt := c.now().UnixNano()

    if pt > c.ts.WallTime && pt > received.WallTime {
        // Physical clock is ahead of both: use it
        c.ts.WallTime = pt
        c.ts.Logical = 0
    } else if c.ts.WallTime == received.WallTime {
        // Local and received have same wall time: advance logical
        if c.ts.Logical > received.Logical {
            c.ts.Logical++
        } else {
            c.ts.Logical = received.Logical + 1
        }
    } else if c.ts.WallTime > received.WallTime {
```

```
        // Local is ahead: just advance local logical
        c.ts.Logical++
    } else {
        // Received is ahead: adopt received wall time
        c.ts.WallTime = received.WallTime
        c.ts.Logical = received.Logical + 1
    }
    return c.ts
}
```

HLCs are used by CockroachDB, YugabyteDB, and other distributed databases. The wall-time component provides rough real-time ordering (useful for TTL, lease expiry, and human-readable timestamps). The logical component ensures causal consistency: if event a caused event b, then $\text{HLC}(a) < \text{HLC}(b)$.

19.3 Distributed Mutual Exclusion

In a single OS, mutual exclusion is achieved through locks, semaphores, or atomic instructions backed by cache coherence protocols. In a distributed system, there is no shared memory and no atomic test-and-set. Distributed mutual exclusion algorithms must use only message passing.

Distributed Mutual Exclusion. An algorithm that ensures at most one process in a distributed system can be in the critical section at any time, using only message passing. The algorithm must satisfy:

1. **Safety (Mutual Exclusion)**: at most one process is in the critical section at any time.
2. **Liveness (Progress)**: every request to enter the critical section is eventually granted.
3. **Ordering (Fairness)**: requests are granted in happened-before order. If process P_i requests entry before process P_j (in the happened-before sense), then P_i enters before P_j.

19.3.1 Centralised Algorithm

The simplest approach: a designated coordinator grants permission to enter the critical section.

1. To enter the CS, process P_i sends a REQUEST to the coordinator.
2. The coordinator maintains a queue of pending requests. If no process is in the CS, it sends a GRANT to the requester. Otherwise, it queues the request.

3. When a process exits the CS, it sends a RELEASE to the coordinator, which grants the CS to the next queued process.

This requires 3 messages per CS entry (REQUEST, GRANT, RELEASE), but the coordinator is a single point of failure and a potential bottleneck.

19.3.2 Ricart-Agrawala Algorithm

The Ricart-Agrawala algorithm (1981) is a fully distributed, permission-based algorithm that requires $2(n-1)$ messages per critical section entry.

Ricart-Agrawala Algorithm. When process P_i wants to enter the critical section:

1. P_i broadcasts a REQUEST message with its Lamport timestamp (T_i, i) to all $n-1$ other processes.
2. When P_j receives REQUEST(T_i, i):
   - If P_j is not requesting and not in the CS: immediately send REPLY to P_i.
   - If P_j is in the CS: defer (queue) the REPLY.
   - If P_j is also requesting the CS: compare timestamps. If $(T_i, i) < (T_j, j)$ (i.e., P_i's request has priority): send REPLY to P_i. Otherwise: defer.
3. P_i enters the CS after receiving REPLY from all $n-1$ processes.
4. When P_i exits the CS: send REPLY to all deferred requests.

Theorem 19.4 (Ricart-Agrawala Mutual Exclusion). The Ricart-Agrawala algorithm guarantees mutual exclusion.

Proof. Suppose, for contradiction, that P_i and P_j are both in the critical section simultaneously. Without loss of generality, assume P_i's request has the smaller Lamport timestamp: $(T_i, i) < (T_j, j)$. For P_j to be in the CS, it must have received REPLY from all $n-1$ processes, including P_i.

When P_i received P_j's REQUEST with timestamp (T_j, j), P_i was either: (a) requesting with $(T_i, i) < (T_j, j)$, in which case P_i defers P_j's request (no REPLY sent), or (b) already in the CS, in which case P_i also defers. In neither case does P_i send REPLY to P_j.

Therefore P_j cannot have received REPLY from P_i and cannot be in the CS — contradiction. □

Example 19.3 (Ricart-Agrawala Execution). Three processes P_1, P_2, P_3, where P_1 and P_2 simultaneously request the critical section with timestamps $(5, 1)$ and $(3, 2)$ respectively:

```
P1: REQUEST(5,1) ——> P2, P3    | Receives REQUEST(3,2): (3,2)<(5,1), send REPLY to P2
P2: REQUEST(3,2) ——> P1, P3    | Receives REQUEST(5,1): (3,2)<(5,1), defer P1's request
```

```
P3: Not requesting              | Receives both: send REPLY to both

Timeline:
  P2 has REPLYs from P1 and P3: enters CS
  P2 exits CS: sends deferred REPLY to P1
  P1 has REPLYs from P2 and P3: enters CS
```

The algorithm correctly serialises access: P_2 (with the earlier timestamp) enters first.

19.3.3 Token Ring

An alternative approach: a logical token circulates among processes. Only the token holder may enter the critical section.

Token Ring Mutual Exclusion. Processes are arranged in a logical ring $P_1 \to P_2 \to \cdots \to P_n \to P_1$. A special TOKEN message circulates continuously. When a process receives the token: if it wants the CS, it enters and holds the token; otherwise, it forwards the token to the next process.

Token ring requires between 0 messages (if the process already holds the token) and n messages (the token must traverse the entire ring) per CS entry. The average is $n/2$ under uniform load. The algorithm is simple and fair, but it has two issues: a lost token (due to process crash or message loss) requires a token regeneration protocol, and the token circulates even when no process needs the CS (wasting bandwidth).

19.4 Consensus

Consensus is the fundamental problem of distributed computing: getting a group of processes to agree on a single value, even when some processes may fail. Consensus underlies state machine replication, atomic commit, leader election, and configuration management.

Consensus. A set of n processes, each proposing a value, must reach agreement satisfying:

1. **Agreement**: all non-faulty processes decide the same value.
2. **Validity**: the decided value was proposed by some process. (This prevents trivial solutions like “always decide 0.”)
3. **Termination**: every non-faulty process eventually decides.

19.4.1 The FLP Impossibility Result

Theorem 19.5 (Fischer, Lynch, Paterson, 1985). In an asynchronous distributed system with reliable message delivery, there is no deterministic algorithm that solves consensus if even one process may crash.

Proof sketch. The proof proceeds in three steps:

Step 1: Bivalent initial configurations exist. An initial configuration is *bivalent* if both 0 and 1 are reachable as decision values from that configuration (depending on the schedule of events). By a valence argument on the boundary between 0-valent and 1-valent configurations (varying one process's initial value at a time), at least one bivalent initial configuration must exist.

Step 2: From any bivalent configuration, the adversary can maintain bivalence. Consider a bivalent configuration C and a pending event e (a message delivery). The set of configurations reachable from C by applying e either contains a bivalent configuration (the adversary applies e and remains bivalent) or all configurations reachable are univalent but with different valences. In the latter case, the adversary can show a contradiction by constructing two executions that are indistinguishable to all but one process.

Step 3: The adversary constructs an infinite non-deciding execution. Starting from the bivalent initial configuration, the adversary repeatedly applies Step 2 to keep the system bivalent. Each process's pending messages are eventually delivered (ensuring fairness), but the adversary delays the "critical" message just enough to prevent the system from reaching a univalent (decided) state.

The key insight is that in an asynchronous system, there is no way to distinguish a slow process from a crashed one. Any timeout-based decision risks violating agreement (if the "slow" process is actually alive and decides differently). □

The FLP result does not mean consensus is impossible in practice — it means there is no algorithm that is guaranteed to terminate in **all** asynchronous executions. Practical consensus algorithms work around FLP by:

- **Partial synchrony**: assuming that the system eventually becomes synchronous (unknown GST), which allows liveness after GST while maintaining safety always.
- **Randomisation**: Las Vegas algorithms that terminate with probability 1 (Ben-Or, 1983).
- **Failure detectors**: oracles that provide hints about which processes have crashed ($\diamond\mathcal{S}$, the eventually perfect failure detector, is sufficient for consensus).

19.4.2 Paxos

Paxos, proposed by Leslie Lamport in 1989 (published in 1998 as "The Part-Time Parliament"), is the canonical consensus algorithm for partially synchronous systems.

Paxos Roles. A Paxos system has three roles (a single node may play multiple roles):

1. **Proposer**: proposes values for the consensus decision.
2. **Acceptor**: votes on proposed values. A quorum of acceptors (a majority: $\lfloor n/2 \rfloor + 1$ out of n) must agree for a value to be chosen.
3. **Learner**: learns the decided value (for applying to the state machine).

The single-decree (single-value) Paxos algorithm proceeds in two phases:

Phase 1 — Prepare:

1. A proposer selects a globally unique, monotonically increasing proposal number n (typically using a pair $(round, proposer_id)$) and sends PREPARE(n) to a quorum of acceptors.

2. An acceptor receiving PREPARE(n): if n is greater than any PREPARE it has previously responded to, it replies with PROMISE(n, v_a, n_a), where (v_a, n_a) is the highest-numbered proposal it has previously accepted (or $(\perp, 0)$ if none). The acceptor **promises** not to accept any proposal with number less than n.

Phase 2 — Accept:

1. If the proposer receives PROMISE from a quorum, it selects a value v:

   - If any acceptor reported a previously accepted value, the proposer **must** use the value with the highest proposal number (this is what prevents conflicting decisions).
   - If no acceptor has accepted anything ($v_a = \perp$ for all), the proposer may choose any value.

   The proposer sends ACCEPT(n, v) to the quorum.

2. An acceptor receiving ACCEPT(n, v): if $n \geq$ the highest-numbered PREPARE it has promised, it accepts the proposal and sends ACCEPTED(n, v) to the learners. Otherwise, it ignores the request.

Theorem 19.6 (Paxos Safety). If a value v is chosen (accepted by a quorum), then no other value $v' \neq v$ can ever be chosen.

Proof sketch. Suppose v is chosen with proposal number n. We prove by strong induction that every proposal with number $n' > n$ also proposes v.

Consider any proposal with number $n' > n$. In Phase 1, the proposer of n' receives

PROMISE from a quorum Q'. Because any two quorums overlap ($|Q| + |Q'| > n_{\text{acceptors}}$, since both are majorities), at least one acceptor a is in both the quorum that accepted (n, v) and the quorum Q' that responded to the PREPARE(n').

Acceptor a has accepted (n, v) (or something with an even higher number, which by induction also proposes v). Acceptor a reports this in its PROMISE response. The proposer of n', following the protocol, must use the value from the highest-numbered accepted proposal, which is v (or a value proposed at $\geq n$, which by induction is also v).

Therefore, proposal n' proposes v, and the only value that can be chosen is v. □

Example 19.4 (Paxos Execution). Three acceptors A_1, A_2, A_3. Proposer P_1 proposes value "x", proposer P_2 proposes value "y".

```
P1: PREPARE(1)  ──> A1, A2, A3
  A1: PROMISE(1, null, 0)
  A2: PROMISE(1, null, 0)
  (A3 is slow)

P1: received quorum (A1, A2). No accepted values. Chooses "x".
P1: ACCEPT(1, "x") ──> A1, A2
  A1: accepts (1, "x")
  A2: accepts (1, "x")
  Value "x" is CHOSEN (majority accepted).

P2: PREPARE(2) ──> A1, A2, A3  (higher proposal number)
  A1: PROMISE(2, "x", 1)      (reports previously accepted value)
  A3: PROMISE(2, null, 0)

P2: received quorum (A1, A3). A1 reports ("x", 1).
P2: MUST propose "x" (the value from the highest accepted proposal).
P2: ACCEPT(2, "x") ──> A1, A3

Result: "x" is the consensus value, even though P2 originally wanted "y".
```

19.4.3 Raft: A Detailed Walkthrough

Raft (Ongaro and Ousterhout, 2014) was designed as an understandable alternative to Paxos. Where Paxos decomposes consensus into roles (proposer, acceptor, learner), Raft decomposes it into sub-problems: leader election, log replication, and safety.

Raft. A consensus algorithm for replicated state machines. Key concepts:

1. **Term**: a logical time period, monotonically increasing. Each term has at most one leader. Terms serve the role of Paxos proposal numbers.
2. **Log**: an ordered sequence of entries, each containing a client command and the term when it was received.
3. **Commit index**: the index of the highest log entry known to be replicated on a majority. Committed entries are safe to apply to the state machine.
4. **State machine**: the application logic (e.g., a key-value store) that applies committed log entries in order.

Leader Election:

1. All nodes start as **followers**. Each follower has a randomised **election timeout** (e.g., 150–300 ms). If a follower does not hear from a leader (via AppendEntries RPC or heartbeat) within its timeout, it becomes a **candidate**.
2. The candidate increments its current term, votes for itself, and sends RequestVote RPCs to all other nodes.
3. A node votes for at most one candidate per term (first-come-first-served), and only if the candidate's log is at least as up-to-date as the voter's (compared by last entry's term, then log length). This is the **election restriction** that ensures safety.
4. A candidate that receives votes from a majority becomes the **leader** for that term. It immediately sends heartbeats (empty AppendEntries) to all followers to establish authority and prevent new elections.
5. If no candidate wins (split vote — two candidates each get some votes but not a majority), all candidates time out and start a new election with a higher term. The randomised timeout makes split votes unlikely.

Log Replication:

Example 19.5 (Raft Log Replication). A 5-node Raft cluster with S_1 as leader in term 3:

```
Client request: SET x=5
   │
   ▼
S1 (Leader, Term 3):
  Log: [T1:SET a=1] [T2:SET b=2] [T3:SET x=5]
                                     │ (index 3, new entry)
                           AppendEntries RPC
                      (term=3, prevLogIndex=2, prevLogTerm=2,
                       entries=[{T3,SET x=5}], leaderCommit=2)
```

```
                                  |
          ┌──────────────┬──────────┼──────────┬──────────────┐
          ▼              ▼          ▼          ▼              ▼
   S2: match OK    S3: match OK   S4: slow   S5: match OK
   append entry    append entry              append entry
   reply success   reply success             reply success

S1 receives 3 successes (S2, S3, S5) + itself = 4/5 = majority
  => Entry at index 3 is COMMITTED
  => S1 applies SET x=5 to state machine, replies to client: OK
  => Next AppendEntries will carry leaderCommit=3
  => S4 eventually catches up and applies the entry
```

Even though S_4 is slow, the entry is committed because 4 out of 5 nodes have it. S_4 will receive the entry in a later AppendEntries and catch up. The state machine is consistent: all nodes that apply the entry will apply the same command in the same order.

Safety — Leader Completeness:

Raft's key safety invariant: if a log entry is committed in a given term, that entry is present in the logs of all leaders for all higher terms.

Theorem 19.7 (Raft Leader Completeness). If a log entry e at index i is committed in term t, then every leader elected in any term $t' > t$ has e at index i in its log.

Proof sketch. Entry e is committed, so it is stored on a majority M of nodes. For a candidate to win election in term t', it must receive votes from a majority V of nodes. Since $|M| + |V| > n$, $M \cap V \neq \emptyset$: at least one node in V has entry e.

The election restriction ensures that a voter only votes for a candidate whose log is at least as up-to-date. A node with entry e (committed at term t, index i) will reject any candidate whose last log entry has a term $< t$, or the same term but a shorter log. Therefore, only candidates that already have entry e can win the election. □

Example 19.6 (Log Divergence and Repair). After a leader crash, logs may diverge. Consider a 5-node cluster where leader S_1 crashes after replicating an entry (index 4, term 3) to only S_2:

```
S1: [T1][T2][T3][T3]  <-- crashed, had index 4 in term 3
S2: [T1][T2][T3][T3]  <-- has the unreplicated entry
S3: [T1][T2][T3]      <-- does not have index 4
S4: [T1][T2][T3]
S5: [T1][T2][T3]
```

S_3 wins election in term 4 (receives votes from S_4 and S_5, plus itself = 3/5 majority). S_3's log ends at (term 3, index 3), which is at least as up-to-date as S_4 and S_5's logs.

S_3 receives a new client request and appends it at index 4, term 4:

```
S3 (new leader): [T1][T2][T3][T4]  <-- index 4 now has term 4 entry
```

When S_3 sends AppendEntries to S_2, S_2's log has a conflicting entry at index 4 (term 3 vs term 4). The Raft protocol requires S_2 to **delete** the conflicting entry and replace it with the leader's entry. This is safe because the term 3 entry at index 4 was never committed (it was only on S_1 and S_2, not a majority).

19.5 Distributed File Systems

A distributed file system provides the abstraction of a single, shared filesystem across multiple machines.

19.5.1 NFS (Network File System)

NFS, originally developed by Sun Microsystems (1984), is the most widely deployed distributed file system. NFS v3 uses a **stateless** server design:

Stateless NFS Server. The NFS server does not maintain per-client state about open files. Each request (READ, WRITE, LOOKUP, GETATTR) is self-contained, carrying a **file handle** (an opaque reference to the inode), the byte offset, and the data. If the server crashes and restarts, it can continue serving requests without a recovery protocol — clients simply retry.

The stateless design simplifies crash recovery but limits performance and semantics:

- **No read-ahead or write-behind per client**: the server does not know which files a client has open.
- **No close-to-open consistency guarantee**: clients cache file data aggressively and may serve stale reads.
- **No locking protocol in the core protocol**: a separate NLM (Network Lock Manager) provides advisory locking, adding complexity.

NFS v4 (2003) adds statefulness: clients open files and acquire **leases** (time-limited locks). This enables better caching semantics (delegations), atomic open-with-create, and mandatory locking. NFSv4.1 (2010) adds session semantics and pNFS (parallel NFS) for distributed data access.

19.5.2 GFS and HDFS

Google File System (GFS, 2003) and its open-source descendant **Hadoop Distributed File System** (HDFS) are designed for a different workload: very large files (gigabytes to terabytes), sequential reads, and append-heavy writes, running on commodity hardware that fails frequently.

GFS/HDFS Architecture. The system has three components:

1. **Master/NameNode**: a single node that manages metadata — the directory tree, file-to-chunk mapping, and chunk locations. The master does not store file data.
2. **Chunkservers/DataNodes**: store file data in fixed-size chunks (64 MB in GFS, 128 MB default in HDFS). Each chunk is replicated on 3 nodes by default for fault tolerance.
3. **Client library**: contacts the master for metadata (which chunkserver has chunk k of file f?), then reads/writes directly from/to chunkservers (data never flows through the master).

The single-master design simplifies consistency and enables fast metadata operations (the entire directory tree fits in RAM), but it is a bottleneck and single point of failure. HDFS NameNode HA uses a **standby NameNode** with a shared edit journal (via JournalNodes implementing a consensus protocol) for automatic failover.

Example 19.7 (HDFS Read Path).

```
Client                    NameNode                DataNode1    DataNode2    DataNode3
  |                         |                         |            |            |
  | open("/data/log.gz")    |                         |            |            |
  |──────────────────────────>|                         |            |            |
  | block locations:        |                         |            |            |
  | blk_0: DN1, DN2, DN3   |                        |            |            |
  | blk_1: DN2, DN3, DN1   |                        |            |            |
  |<──────────────────────────|                         |            |            |
  |                         |                         |            |            |
  | read blk_0 from DN1    |                        |            |            |
  | (closest replica)       |                         |            |            |
  |──────────────────────────────────────────────────────>|            |            |
  | data                    |                         |            |            |
  |<──────────────────────────────────────────────────────|            |            |
  |                         |                         |            |            |
  | read blk_1 from DN2    |                        |            |            |
```

```
| (closest replica)       |                        |            |            |
|──────────────────────────────────────────────────────────────────────>|            |
| data                    |                        |            |            |
|<──────────────────────────────────────────────────────────────────────|            |
```

The client reads each block from the closest replica (rack-aware placement). The NameNode is contacted only once for block locations, not on every read. Data flows directly between client and DataNodes.

19.5.3 Ceph

Ceph eliminates the single-metadata-server bottleneck with **CRUSH** (Controlled Replication Under Scalable Hashing), an algorithmic data placement strategy.

CRUSH Algorithm. Instead of looking up data locations in a metadata table, CRUSH computes the location of any object deterministically from the object's name, a cluster map (describing the topology of storage nodes), and placement rules. Both clients and storage nodes can independently compute where data should be stored, without consulting a central authority.

The CRUSH algorithm takes as input: the object's ID (a hash), the cluster map (a hierarchy of racks, hosts, and disks), and a placement rule (e.g., "place 3 replicas, each on a different rack"). It outputs a list of storage targets. Because the computation is deterministic, any node that has the cluster map can compute the same result.

Ceph provides three interfaces: **RBD** (block storage for VMs), **CephFS** (POSIX file system with distributed metadata servers), and **RADOS Gateway** (S3/Swift-compatible object storage). The underlying **RADOS** layer handles replication, recovery, and rebalancing automatically.

19.6 Distributed Shared Memory

Distributed Shared Memory (DSM) provides the abstraction of a single shared address space across multiple machines, allowing processes to read and write shared variables using normal memory operations. The DSM system transparently handles the communication necessary to keep the shared state consistent.

19.6.1 Consistency Models

Consistency Model. A contract between the memory system and the programmer that defines which orderings of reads and writes are legal. Stronger models restrict the set of possible orderings (easier to reason about) but require more communication (harder to implement efficiently).

The hierarchy from strongest to weakest:

Strict Consistency: Every read returns the value of the most recent write, globally. This requires instantaneous propagation of writes — physically impossible when machines are separated by network latency.

Linearisability (Herlihy and Wing, 1990):

Linearisability. An execution is linearisable if each operation appears to take effect instantaneously at some point between its invocation and its response, and the resulting sequence of operations is consistent with the object's sequential specification.

Linearisability is the strongest achievable consistency model in a distributed system. It is the "C" in the CAP theorem (see Section 19.7).

Sequential Consistency (Lamport, 1979):

Sequential Consistency. A system is sequentially consistent if the result of any execution is the same as if the operations of all processes were executed in some sequential order, and the operations of each individual process appear in this sequence in the order issued by that process.

Example 19.8 (Sequential vs Linearisable). Two processes:

```
P1: W(x, 1)  W(x, 3)
P2: W(x, 2)  R(x) -> ?
```

Under **sequential consistency**, any total order preserving each process's program order is valid. $R(x) = 3$ (order: $W_2(x, 2), W_1(x, 1), W_1(x, 3), R_2(x)$) is valid even though $W_1(x, 3)$ finished (in real time) before $R_2(x)$ started.

Under **linearisability**, the total order must also be consistent with real-time ordering: if $W_1(x, 3)$ completed before $R_2(x)$ started, then $R_2(x)$ must return 3 or a later value. Linearisability is strictly stronger.

Causal Consistency: Writes that are causally related (one happened before the other) are seen in the same order by all processes. Concurrent writes may be seen in different orders by different processes.

Eventual Consistency: If no new writes are made, all replicas will eventually converge to the same value. This is the weakest useful model and is used by many internet-scale systems (DNS, Cassandra, DynamoDB in its default mode, web caches).

19.6.2 Trade-offs

Model	Programmability	Latency	Availability
Linearisable	Easiest	High (requires quorum)	Low during partitions
Sequential	Easy	Medium	Low during partitions
Causal	Moderate	Low-medium	Moderate
Eventual	Hardest (application resolves conflicts)	Low	High

19.7 The CAP Theorem

The **CAP theorem** (Brewer's conjecture, 2000; proved by Gilbert and Lynch, 2002) formalises the fundamental trade-off in distributed data stores.

Theorem 19.8 (CAP Theorem). A distributed data store can provide at most two of the following three guarantees simultaneously:

1. **Consistency (C)**: every read receives the most recent write (linearisability).
2. **Availability (A)**: every request to a non-failing node receives a response (possibly stale).
3. **Partition tolerance (P)**: the system continues to operate despite arbitrary message loss between nodes.

Proof. Consider a system with two nodes N_1 and N_2 that replicate a variable x, initially $x = 0$. Suppose a network partition separates N_1 and N_2 (they cannot communicate).

A client writes $x = 1$ to N_1. Another client reads x from N_2.

Case 1 — Consistency + Availability: N_2 must return $x = 1$ (consistency) and must respond (availability). But N_2 has not received the write (partition). Therefore N_2 cannot return 1 without communicating with N_1, which is impossible. Contradiction: C + A + P cannot all hold.

Case 2 — Consistency + Partition tolerance (CP): N_2 refuses to respond until the partition heals (sacrificing availability). After the partition heals, N_2 receives the write and can respond correctly.

Case 3 — Availability + Partition tolerance (AP): N_2 responds immediately with

the stale value $x = 0$ (sacrificing consistency). After the partition heals, a reconciliation mechanism updates N_2.

Since network partitions are inevitable in any real distributed system, the practical choice is between CP and AP. □

Example 19.9 (CP vs AP Systems in Practice).

CP system (etcd, ZooKeeper): During a network partition, the **minority** partition becomes unavailable — it cannot elect a leader (requires a majority) and cannot process writes or linearisable reads. The majority partition continues operating normally. After the partition heals, the minority nodes rejoin and catch up.

This is appropriate for: coordination services, distributed locks, configuration stores, and leader election — systems where stale data is worse than unavailability.

AP system (Cassandra with CL=ONE, DynamoDB default): During a partition, **both** sides continue serving reads and writes. After the partition heals, a conflict resolution mechanism reconciles the divergent values. Common strategies:

- **Last-writer-wins (LWW):** the write with the highest timestamp wins. Simple but can lose concurrent writes.
- **CRDTs (Conflict-free Replicated Data Types):** data structures designed to merge automatically without conflicts (e.g., G-counter, OR-set).
- **Application-level resolution:** the application is notified of conflicts and decides (e.g., Amazon's shopping cart merges items from both versions).

This is appropriate for: social media feeds, shopping carts, DNS, caching, and any system where temporary staleness is acceptable.

Programmer's Perspective: etcd and Raft in Go. etcd is a distributed key-value store built on Raft, written in Go. It is the coordination backbone of Kubernetes (storing all cluster state). etcd is a CP system: during a partition, the minority partition cannot process writes.

```go
package main

import (
    "context"
    "fmt"
    "log"
    "time"

    clientv3 "go.etcd.io/etcd/client/v3"
    "go.etcd.io/etcd/client/v3/concurrency"
```

```go
)

func main() {
    // Connect to etcd cluster
    cli, err := clientv3.New(clientv3.Config{
        Endpoints:   []string{"localhost:2379", "localhost:2380", "localhost:2381"},
        DialTimeout: 5 * time.Second,
    })
    if err != nil {
        log.Fatal(err)
    }
    defer cli.Close()

    ctx := context.Background()

    // Linearisable write (goes through Raft consensus)
    _, err = cli.Put(ctx, "service/leader", "node-3")
    if err != nil {
        log.Fatal(err)
    }

    // Linearisable read (reads go through the leader)
    resp, err := cli.Get(ctx, "service/leader")
    if err != nil {
        log.Fatal(err)
    }
    for _, kv := range resp.Kvs {
        fmt.Printf("%s = %s (mod revision %d)\n",
            kv.Key, kv.Value, kv.ModRevision)
    }

    // Distributed lock using etcd (built on Raft consensus)
    session, err := concurrency.NewSession(cli)
    if err != nil {
        log.Fatal(err)
    }
    defer session.Close()

    mutex := concurrency.NewMutex(session, "/locks/myresource")
```

```go
    if err := mutex.Lock(ctx); err != nil {
        log.Fatal(err)
    }
    fmt.Println("Acquired distributed lock")

    // Critical section: safe across all nodes in the cluster
    // ... do work ...

    if err := mutex.Unlock(ctx); err != nil {
        log.Fatal(err)
    }
    fmt.Println("Released distributed lock")
}
```

Every `Put` in etcd goes through Raft consensus: the leader replicates the write to a majority before acknowledging. The distributed lock is implemented using etcd's lease mechanism: the lock holder creates a key with a lease, and if the holder crashes, the lease expires and the lock is released automatically.

19.8 Failure Detection

Detecting whether a remote process has failed is a prerequisite for many distributed protocols (consensus, group membership, leader election). In an asynchronous system, perfect failure detection is impossible (FLP). Practical failure detectors use timeouts and heuristics, accepting that they will occasionally make mistakes (suspecting alive processes or failing to suspect crashed ones).

19.8.1 Heartbeats

The simplest approach: each process periodically sends "I am alive" messages to a monitor (or to all other processes). If the monitor does not receive a heartbeat within a timeout period T, it suspects the process has failed.

The challenge is choosing T:

- T **too short**: the detector suspects alive-but-slow processes (**false positive**). This can trigger unnecessary leader elections, failovers, and data rebalancing.
- T **too long**: a truly crashed process is not detected for a long time, leaving the system in a degraded state.

19.8.2 The Phi Accrual Failure Detector

Phi Accrual Failure Detector (Hayashibara et al., 2004). Instead of producing a binary "alive/dead" output, the phi accrual detector computes a continuous **suspicion level** ϕ based on the statistical distribution of heartbeat inter-arrival times.

Let Δt be the time since the last heartbeat, and let F be the CDF of the estimated heartbeat inter-arrival distribution (typically normal, fitted to the most recent W observations). The suspicion level is:

$$\phi(\Delta t) = -\log_{10}(1 - F(\Delta t))$$

Interpretation: - $\phi = 1$: $P(\text{alive node has such a long gap}) = 10^{-1} = 10\%$ - $\phi = 3$: $P = 10^{-3} = 0.1\%$ - $\phi = 8$: $P = 10^{-8} \approx 0$ — almost certainly failed

The application chooses a threshold based on its tolerance for false positives.

The advantage of the phi accrual detector is **adaptivity**: it learns the network's characteristics (jitter, latency distribution) from observed heartbeat times and adjusts automatically. A noisy network with high variance produces a wider distribution, requiring a larger Δt before ϕ exceeds the threshold. Apache Cassandra uses this detector with a default threshold of $\phi = 8$.

19.8.3 SWIM (Scalable Weakly-consistent Infection-style Membership)

SWIM Protocol (Das et al., 2002). A membership protocol that detects failures and disseminates membership information in $O(\log n)$ time with $O(1)$ bandwidth per member per protocol period. Each period:

1. A process P_i selects a random member P_j and sends a PING.
2. If P_j replies with an ACK within a timeout, P_j is alive.
3. If P_j does not reply, P_i selects k other members (the **indirect probe group**) and asks them to PING P_j on its behalf. If any of the k members receives an ACK from P_j, they relay it to P_i.
4. If none of the k indirect probes succeed, P_i marks P_j as **suspected** and disseminates this suspicion via **piggyback gossip**: membership updates are attached to the regular PING/ACK messages, spreading through the cluster like an epidemic.
5. After a configurable grace period, if P_j does not refute the suspicion, it is declared **confirmed dead** and removed from the membership.

SWIM's genius is combining failure detection with information dissemination: membership changes (joins, failures, suspicions) are piggybacked on the PING/ACK mes-

sages that are already being exchanged, achieving $O(n)$ total bandwidth across the cluster with $O(\log n)$ convergence time (each message carries updates that spread exponentially, like a rumour).

HashiCorp's **Serf** and **Consul** use SWIM (with extensions: Lifeguard, which reduces false positive rates by allowing suspected nodes to refute before being evicted).

Programmer's Perspective: gRPC for Distributed Communication. gRPC (Google Remote Procedure Call) is the dominant RPC framework for distributed systems. Built on Protocol Buffers (efficient binary serialisation) and HTTP/2 (multiplexed streams, flow control, header compression), gRPC provides features critical for distributed systems:

- **Deadlines**: a client sets a deadline, and it propagates through the entire RPC chain. If a downstream service cannot complete before the deadline, it cancels and returns early.
- **Cancellation**: cancelling a parent context cancels all in-flight RPCs.
- **Streaming**: unary (request-response), server streaming, client streaming, and bidirectional streaming.
- **Health checking**: a standard `grpc.health.v1.Health` service that load balancers use for failure detection.
- **Interceptors**: middleware for logging, authentication, retry, and circuit breaking.

```go
package main

import (
    "context"
    "log"
    "net"
    "sync"

    "google.golang.org/grpc"
    "google.golang.org/grpc/health"
    healthpb "google.golang.org/grpc/health/grpc_health_v1"
    pb "example.com/kvstore" // generated from .proto
)

type server struct {
    pb.UnimplementedKVStoreServer
    mu    sync.RWMutex
    store map[string]string
```

```go
}

func (s *server) Get(ctx context.Context, req *pb.GetRequest) (*pb.GetResponse, error)
    s.mu.RLock()
    defer s.mu.RUnlock()
    val, ok := s.store[req.Key]
    return &pb.GetResponse{Value: val, Found: ok}, nil
}

func (s *server) Put(ctx context.Context, req *pb.PutRequest) (*pb.PutResponse, error)
    s.mu.Lock()
    defer s.mu.Unlock()
    s.store[req.Key] = req.Value
    return &pb.PutResponse{}, nil
}

func main() {
    lis, err := net.Listen("tcp", ":50051")
    if err != nil {
        log.Fatalf("listen: %v", err)
    }

    s := grpc.NewServer()

    // Register application service
    kvServer := &server{store: make(map[string]string)}
    pb.RegisterKVStoreServer(s, kvServer)

    // Register health service (used by load balancers and SWIM-like detectors)
    healthServer := health.NewServer()
    healthpb.RegisterHealthServer(s, healthServer)
    healthServer.SetServingStatus("kvstore", healthpb.HealthCheckResponse_SERVING)

    log.Println("KVStore gRPC server on :50051")
    if err := s.Serve(lis); err != nil {
        log.Fatalf("serve: %v", err)
    }
}
```

In a Raft-based system like etcd, gRPC is used for both client communication and

inter-node communication: the leader sends AppendEntries as gRPC streams, and candidates send RequestVote as unary RPCs. The deadline mechanism ensures that a slow follower does not block the leader indefinitely.

19.9 Theoretical Boundaries

The results in this chapter establish fundamental limits on what is achievable in distributed systems:

Theorem 19.9 (Summary of Impossibility and Lower Bound Results).

1. **FLP (1985)**: no deterministic consensus algorithm terminates in all asynchronous executions with even one crash failure. Implication: practical consensus requires partial synchrony or randomisation.
2. **CAP (2002)**: no distributed system can simultaneously guarantee linearisability, availability, and partition tolerance. Implication: during a partition, choose CP or AP.
3. **Lundelius-Lynch (1984)**: in a system of n processes where message delay uncertainty is u, clocks cannot be synchronised to better than $u(1 - 1/n)/2$. Implication: clock synchronisation has a fundamental precision limit.
4. **Two Generals (1975)**: two processes communicating over an unreliable channel cannot reach agreement in a finite number of messages. Implication: TCP's three-way handshake is a practical workaround (not a theoretical solution).
5. **Byzantine Generals (Lamport, Shostak, Pease, 1982)**: consensus tolerating f Byzantine failures requires $n \geq 3f + 1$ processes. Implication: tolerating one lying node requires at least 4 nodes.

These are not engineering limitations — they are mathematical theorems. No amount of clever design, faster hardware, or better networks can circumvent them. The art of distributed systems is choosing which guarantees to relax and how to build useful systems within these constraints.

---

1. **Vector Clock Comparison.** Three processes P_1, P_2, P_3 produce the following events with vector clocks: a at P_1 with $V(a) = [2, 0, 0]$, b at P_2 with $V(b) = [1, 3, 1]$, c at P_3 with $V(c) = [1, 2, 4]$, d at P_1 with $V(d) = [3, 3, 1]$. For each of the six pairs of events, determine whether one happened before the other or they are concurrent. Show your reasoning by comparing vectors

componentwise. Draw a space-time diagram consistent with these vector clocks.

2. **Ricart-Agrawala Message Count.** (a) In a system of n processes, prove that the Ricart-Agrawala algorithm requires exactly $2(n-1)$ messages per critical section entry in the absence of failures. (b) Describe the Roucairol-Carvalho optimisation that reduces the average message count. Explain how it works, prove that it maintains correctness, and characterise the workloads where it provides the greatest improvement.

3. **Paxos Liveness.** Paxos guarantees safety (no two different values are chosen) but not liveness (the algorithm may not terminate). (a) Describe a "duelling proposers" scenario in which two proposers prevent each other from ever completing Phase 2. (b) How does **Multi-Paxos** (with a stable leader) address this? (c) Prove that Multi-Paxos requires only one round trip per consensus decision (after leader election), compared to two round trips for basic Paxos.

4. **Raft Log Divergence.** Consider a 5-node Raft cluster where the leader (S_1, term 3) crashes after replicating an entry at index 4 to only S_2. (a) Draw the logs of all five nodes immediately after the crash. (b) Show how a new leader is elected (which nodes can win, and why). (c) Draw the logs after the new leader receives and commits one new client request. (d) What happens to the unreplicated entry on S_2, and at what point is it safe for the new leader to overwrite it?

5. **CAP Classification.** For each of the following systems, classify it as CP or AP and justify with a specific partition scenario: (a) a single-node PostgreSQL database, (b) a 3-node ZooKeeper ensemble, (c) a Cassandra cluster with replication factor 3 and consistency level ONE, (d) a Cassandra cluster with replication factor 3 and consistency level QUORUM. For system (c), describe a concrete scenario in which a client reads stale data.

6. **Eventual Consistency Anomaly.** Two users, Alice and Bob, share a collaborative document stored with eventual consistency and three replicas. Alice deletes paragraph 3; Bob concurrently edits paragraph 3 (adding a sentence). Describe what each user sees immediately after their operation, and what all users see after the system converges, under: (a) last-writer-wins conflict resolution with wall-clock timestamps, (b) operational transformation, and (c) a CRDT-based approach (suggest which CRDT type would be appropriate). Which approach best preserves user intent?

7. **SWIM Protocol Analysis.** (a) Prove that if a node P_j permanently fails, every non-faulty node detects P_j's failure within $O(\log n)$ protocol periods with high

probability, given that gossip dissemination achieves $O(\log n)$ rounds for full propagation. (b) Derive an expression for the false positive rate as a function of the timeout parameter and the network's RTT distribution. (c) Describe the trade-off between detection speed and false positive rate, and explain how the Lifeguard protocol extension reduces false positives.

Chapter 20: The Future of Operating Systems

"The best way to predict the future is to invent it." — Alan Kay

---

The operating system as we know it — a monolithic kernel managing hardware on behalf of user-space processes — was designed for a world of single-core CPUs, small memories, and spinning disks. That world no longer exists. Today's hardware features hundreds of cores, terabytes of persistent memory, network interfaces that outperform disks, and transistors small enough that speculative execution leaks secrets through cache timing. The OS must evolve.

This chapter surveys the frontiers of operating system design: unikernels that compile away the kernel-application boundary, eBPF programs that extend the kernel safely, io_uring that bypasses the system call interface for I/O, capability-based hardware that enforces memory safety in silicon, persistent memory that blurs the line between storage and RAM, verified microkernels that carry mathematical proofs of correctness, Rust in the kernel for memory safety without garbage collection, and WebAssembly as a universal runtime.

These are not speculations — every technology discussed here has working implementations deployed in production. The question is not whether the OS will change, but which of these ideas will define the next generation.

20.1 Unikernels Revisited

A **unikernel** collapses the traditional OS/application boundary: the application is compiled directly against the kernel services it needs, producing a single-address-space image that runs on bare metal or a hypervisor.

Unikernel. A specialised, single-address-space machine image constructed by compiling an application together with only the OS library components it requires. A unikernel has:

- **No separate kernel and user space**: the application and OS code run in the same address space, at the same privilege level.

- **No process isolation**: there is only one "process" — the application itself.
- **No shell, no SSH, no package manager**: there is no way to log in to a running unikernel.
- **No general-purpose OS services**: only the services the application uses are compiled in.

20.1.1 Compile-Time Specialisation

The key insight is that a deployed service knows at compile time which OS services it needs. A web server needs TCP, HTTP parsing, and file I/O — it does not need a process scheduler, a virtual memory system for multiple address spaces, a USB driver stack, or a sound subsystem. A unikernel includes only what is needed, achieving three benefits:

1. **Reduced attack surface**: the kernel attack surface shrinks from millions of lines of code (Linux: ~25M LOC) to tens of thousands (typically 10K–100K LOC). System calls that the application does not use simply do not exist in the image.
2. **Whole-program optimisation**: because the application and OS are compiled together, the compiler can inline OS functions, eliminate dead code paths, and optimise across the application-OS boundary. A `read()` call does not require a context switch — it is a direct function call.
3. **Fast boot**: there is no kernel initialisation for hardware the application does not use, no service manager, no device enumeration. Boot times of 1–50 ms are typical.

Example 20.1 (Unikernel vs Traditional OS). Consider deploying a DNS server:

Property	Linux VM	Unikernel
Boot time	2–30 seconds	10–50 milliseconds
Image size	500 MB – 2 GB	1–10 MB
Memory footprint	200+ MB	10–50 MB
Attack surface	Entire kernel (25M+ LOC)	DNS handler + network stack (~50K LOC)
System calls available	~400	0 (no user/kernel boundary)

Property	Linux VM	Unikernel
Process isolation	Full (but unused — single service)	None (not needed — single service)
Runtime debugging	SSH in, attach debugger, read logs	External only (serial console, network)

The unikernel boots in milliseconds because there is nothing to initialise except the application's dependencies. The image is tiny because it contains no shell, no systemd, no coreutils, no Python — just the DNS server and a minimal network stack.

20.1.2 MirageOS

MirageOS is a unikernel framework written in OCaml. Applications are structured as OCaml functors parameterised over abstract module signatures for OS services (network, storage, time, random). At compile time, these signatures are instantiated with concrete implementations for the target platform:

- **Xen backend**: MirageOS runs as a Xen PV guest, using Xen's grant tables for network and block I/O. The entire "OS" is an OCaml library linked with the application.
- **Unix backend**: MirageOS runs as a regular Unix process, using sockets and files — useful for development and testing.
- **Solo5 backend**: MirageOS runs on Solo5, a minimal monitor that provides a hardware abstraction layer for KVM, bhyve, or bare metal.

The OCaml type system provides additional safety: many classes of bugs (null pointer dereferences, type confusions) are caught at compile time, and the garbage collector prevents memory leaks and use-after-free.

20.1.3 Unikraft

Unikraft (part of the Linux Foundation) takes a modular approach: the OS is decomposed into **micro-libraries** that can be selected and configured at build time. Each micro-library provides a specific service: scheduling (cooperative or preemptive), memory allocation (buddy, slab, or simple bump allocator), network stack (lwIP or a custom stack), file system (initrd, 9pfs, virtiofs), and so on.

Unlike MirageOS, Unikraft targets POSIX-compatible applications — existing C/C++ programs can be compiled as unikernels with minimal or no modification. The build

system uses KConfig (the same configuration system as the Linux kernel) to select which micro-libraries are included.

Unikraft achieves boot times under 1 ms for some configurations (a "hello world" unikernel boots in ~300 μs), with images as small as a few hundred kilobytes.

Programmer's Perspective: When to Use a Unikernel. Unikernels are compelling for:

- **Microservices in the cloud**: each service becomes a dedicated VM with a minimal footprint and boot time. AWS Firecracker was partly inspired by this model.
- **IoT and embedded**: a sensor node running a unikernel has a tiny attack surface and boots instantly after power loss.
- **Network functions**: DNS resolvers, load balancers, firewalls — single-purpose services that benefit from specialisation.
- **Serverless functions**: a unikernel can boot, execute a function, and shut down in milliseconds, matching the serverless execution model.

Unikernels are a poor fit for:

- **General-purpose computing**: no multi-process support, no shell, no runtime debugging tools.
- **Applications with dynamic dependencies**: if the application loads plugins or spawns subprocesses, the unikernel model breaks down.
- **Development**: debugging a unikernel is harder than debugging a process in a full OS. Most frameworks provide a Unix backend for development.

In practice, containers have captured most of the use cases that unikernels targeted, because containers are much easier to build, debug, and deploy. Unikernels remain relevant where the additional security (smaller TCB) and performance (faster boot, lower memory) justify the operational complexity.

20.2 eBPF: Safe Kernel Extension

eBPF (extended Berkeley Packet Filter) is arguably the most important innovation in the Linux kernel since containers. It allows user-space programs to inject bytecode into the kernel that runs in response to kernel events — safely, efficiently, and without modifying the kernel source or loading kernel modules.

eBPF (extended Berkeley Packet Filter). A register-based virtual machine inside the Linux kernel that runs sandboxed programs attached to kernel events. eBPF programs are:

1. **Verified**: a static analyser (the eBPF verifier) checks every program before loading, proving it terminates (bounded loops only), does not access invalid memory, does not dereference null pointers, and does not perform unsafe operations.
2. **JIT-compiled**: after verification, the bytecode is compiled to native machine code for the host architecture (x86-64, ARM64, RISC-V, etc.).
3. **Efficient**: eBPF programs run at near-native speed, with no context switch overhead (they execute in kernel context, at the same privilege level as the kernel).
4. **Composable**: multiple eBPF programs can be attached to different hooks, and they can communicate through shared maps.

20.2.1 The eBPF Architecture

```
User Space                        Kernel Space
┌──────────────┐                  ┌──────────────────────────┐
│              │                  │                          │
│  eBPF        │  bpf() syscall │  eBPF Verifier          │
│  compiler    │ ────────────────>│  (static analysis,       │
│  (clang/LLVM │                  │   type tracking,         │
│   -target bpf)                  │   bounds checking)       │
│              │                  │        │                 │
│  User app    │                  │        ▼                 │
│  (libbpf,    │                  │  JIT Compiler            │
│   reads maps)│ <────────────────│  (bytecode -> native     │
│              │   shared maps    │   x86-64/ARM64)          │
│              │   (hash, array,  │        │                 │
│              │    ringbuf,      │        ▼                 │
│              │    LRU, ...)     │  Attached to hook:       │
│              │                  │  - kprobe (any function) │
│              │                  │  - tracepoint (stable)   │
│              │                  │  - XDP (network ingress) │
│              │                  │  - tc  (traffic control) │
│              │                  │  - cgroup (per-group)    │
│              │                  │  - LSM (security hooks)  │
│              │                  │  - sched_ext (scheduler) │
│              │                  │  - fentry/fexit          │
│              │                  │  - uprobe (userspace)    │
└──────────────┘                  └──────────────────────────┘
```

20.2.2 The eBPF Verifier

The verifier is what makes eBPF safe. Without it, injecting code into the kernel would be equivalent to loading an arbitrary kernel module — a massive security risk. The verifier performs:

1. **Control flow analysis**: the program's control flow graph must be a DAG with respect to back edges (no unbounded loops). Since Linux 5.3, bounded loops are permitted: the verifier proves the loop terminates by tracking the loop variable's bounds.
2. **Register type tracking**: the verifier maintains a type for every register at every instruction (scalar, pointer to map value, pointer to stack, pointer to packet data, pointer to context structure). It ensures that pointers are not used as scalars and vice versa.
3. **Bounds tracking**: for every scalar register, the verifier tracks the possible value range (min/max for signed and unsigned). Array accesses are checked against the tracked bounds.
4. **Pointer safety**: eBPF programs can only access kernel data through verified **helper functions** (e.g., `bpf_map_lookup_elem`, `bpf_probe_read_kernel`, `bpf_get_current_pid_tgid`). Direct pointer dereferences of arbitrary kernel addresses are forbidden.
5. **Stack safety**: the eBPF stack is limited to 512 bytes. The verifier ensures no stack overflow.
6. **Instruction limit**: a program can have at most 1 million verified instructions (to bound verification time).

If the verifier rejects a program, it provides a detailed log explaining which instruction failed and why. This is essentially a lightweight proof that the program is safe.

20.2.3 Applications of eBPF

Networking (XDP): eXpress Data Path allows eBPF programs to process packets at the earliest point in the network stack — before the kernel allocates an `sk_buff`. Packets can be dropped, redirected, or modified at line rate (100 Gbps+). Meta's Katran L4 load balancer and Cloudflare's DDoS mitigation run as XDP programs.

Observability: eBPF programs attach to kernel functions (kprobes), tracepoints, or user-space functions (uprobes), collecting performance data without modifying the kernel or restarting services. Tools like `bpftrace`, BCC, and Cilium's Hubble use eBPF for zero-overhead tracing.

Security (LSM): eBPF programs attached to LSM hooks make access control decisions based on runtime context. Cilium Tetragon provides security observability — monitoring system calls, file access, and network connections at the kernel level without the overhead of a user-space agent.

Scheduling (sched_ext): Since Linux 6.12, eBPF can implement custom CPU scheduling policies. The scheduler is loaded as an eBPF program that the kernel calls on scheduling events (enqueue, dequeue, pick_next). This allows experimentation with scheduling algorithms (e.g., EEVDF variants, game-optimised schedulers) without recompiling the kernel.

Example 20.2 (eBPF in Production).

Organisation	Use Case	eBPF Hook	Scale
Meta (Facebook)	L4 load balancing (Katran)	XDP	Billions of packets/day
Cloudflare	DDoS mitigation	XDP	100M+ RPS
Netflix	System-wide performance tracing	kprobes, tracepoints	Thousands of servers
Google/Isovalent	Kubernetes networking (Cilium)	tc, XDP, cgroup	Production clusters
Isovalent	Security observability (Tetragon)	LSM, kprobes	Runtime enforcement
Android	Network traffic management	cgroup/skb	Billions of devices

Programmer's Perspective: Writing an eBPF Program in C. An eBPF program that counts system calls per process, using the CO-RE (Compile Once, Run Everywhere) framework:

```
/* syscall_count.bpf.c -- eBPF kernel program */
#include <vmlinux.h>          /* BTF-generated kernel types */
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
```

```
/* Map: PID -> syscall count */
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 10240);
    __type(key, __u32);    /* PID */
    __type(value, __u64);  /* count */
} syscall_counts SEC(".maps");

/* Attach to the raw_syscalls:sys_enter tracepoint.
   This fires on every system call, for every process. */
SEC("tracepoint/raw_syscalls/sys_enter")
int count_syscalls(struct trace_event_raw_sys_enter *ctx) {
    __u32 pid = bpf_get_current_pid_tgid() >> 32;

    __u64 *count = bpf_map_lookup_elem(&syscall_counts, &pid);
    if (count) {
        /* Atomic increment: safe even with concurrent access
           from multiple CPUs */
        __sync_fetch_and_add(count, 1);
    } else {
        __u64 init_val = 1;
        bpf_map_update_elem(&syscall_counts, &pid, &init_val, BPF_ANY);
    }
    return 0;
}

char LICENSE[] SEC("license") = "GPL";
```

The verifier proves that:

- The `bpf_map_lookup_elem` return value is checked for NULL before dereference (line: `if (count)`).
- The map key and value sizes match the map definition.
- The program terminates (no loops).
- The stack usage is within 512 bytes.
- The program only accesses the tracepoint context fields, not arbitrary kernel memory.

Compile: `clang -target bpf -D__TARGET_ARCH_x86 -O2 -g -c syscall_count.bpf.c -o syscall_count.bpf.o`

A user-space loader (using libbpf) opens the `.bpf.o` file, loads and verifies the pro-

gram, attaches it to the tracepoint, and periodically reads the hash map to display per-process syscall rates. The entire pipeline requires no kernel recompilation, no module loading, and no reboot.

20.3 io_uring: Kernel-Bypass I/O

Traditional Unix I/O uses system calls: `read()`, `write()`, `sendmsg()`, `recvmsg()`. Each system call crosses the user-kernel boundary, costing approximately 100–1000 ns (plus TLB flush overhead with KPTI). For high-frequency I/O (millions of operations per second), this overhead dominates.

io_uring (introduced in Linux 5.1, 2019, by Jens Axboe) eliminates the system call overhead by using shared memory ring buffers between user space and the kernel.

io_uring. A Linux kernel interface for asynchronous I/O based on two lock-free shared ring buffers:

1. **Submission Queue (SQ)**: user space writes I/O requests (Submission Queue Entries, SQEs) here. Each SQE specifies an opcode, file descriptor, buffer, offset, and user-data tag.
2. **Completion Queue (CQ)**: the kernel writes I/O results (Completion Queue Entries, CQEs) here. Each CQE contains the result code and the user-data tag from the corresponding SQE.

Both queues are memory-mapped (via `mmap` on the io_uring file descriptor) and accessed lock-free using memory barriers. In polling mode, no system calls are needed on the hot path.

20.3.1 Submission Modes

io_uring supports three modes, trading CPU usage for latency:

Default mode: User space writes SQEs, then calls `io_uring_enter()` to submit them. The kernel processes the SQEs asynchronously and writes CQEs. The submission path requires one system call per batch.

SQ polling (`IORING_SETUP_SQPOLL`): The kernel spawns a dedicated polling thread that continuously checks the SQ for new entries. User space writes SQEs directly to shared memory — no system call needed. The polling thread picks them up automatically. If the SQ is empty for a configurable period, the polling thread sleeps (and wakes on the next submission via `io_uring_enter`).

IO polling (`IORING_SETUP_IOPOLL`): The kernel polls the hardware for completions

instead of waiting for interrupts. Combined with SQ polling, this achieves fully polled I/O: zero system calls and zero interrupts on the hot path. This mode requires a block device that supports polled I/O (most modern NVMe drives do).

```
User Space                              Kernel Space

┌──────────────────────┐              ┌──────────────────────────┐
│                      │              │                          │
│  Application         │              │  io_uring engine         │
│  │                   │              │  │                       │
│  ├─> Write SQE to SQ ─┼─ (mmap) ────┼─> Read SQE from SQ    │
│  │   (store + barrier, │  shared     │  │  (SQ poll thread     │
│  │    no syscall)    │  memory     │  │   or io_uring_enter)│
│  │                   │             │  │                       │
│  │                   │             │  ├─> Submit to block    │
│  │                   │             │  │   layer / network    │
│  │                   │             │  │                       │
│  └─< Read CQE from CQ <┼─ (mmap) ────┼─< Write CQE to CQ    │
│      (load + barrier, │             │     (after I/O        │
│       no syscall)    │             │      completes)        │
│                      │             │                          │
└──────────────────────┘              └──────────────────────────┘
```

20.3.2 Performance Impact

Example 20.3 (io_uring Performance on NVMe). Benchmarks on modern NVMe SSDs (4 KB random reads, single core):

Interface	IOPS	Notes
Synchronous `read()`	~200K	One syscall per I/O
`libaio` (Linux AIO)	~400K	Async, but still uses syscalls for submit/reap
io_uring (default)	~800K	Batched submissions, async completions
io_uring (SQ + IO poll)	~1.2M	Zero syscalls, zero interrupts
SPDK (full kernel bypass)	~1.5M	User-space NVMe driver, no kernel involvement

io_uring achieves 80% of the performance of full kernel bypass (SPDK) while retaining the kernel's block layer, file systems, and scheduler. For most applications, this is the optimal trade-off: near-maximum performance without giving up the kernel's storage management.

20.3.3 Advanced Features

io_uring has grown far beyond basic read/write:

- **Linked SQEs**: chain operations so that one starts only after the previous completes (e.g., open then read then close).
- **Fixed files and buffers**: register file descriptors and buffers once, avoiding per-I/O `fget/fput` and page table walks.
- **Zero-copy send** (`IORING_OP_SEND_ZC`): pass user-space buffers directly to the NIC's DMA engine.
- **Multishot accept**: a single SQE accepts multiple incoming connections, each generating a CQE.
- **Registered rings**: nested io_uring instances for hierarchical I/O management.
- **IORING_OP_WAITID**, `IORING_OP_FUTEX`: extending beyond I/O to general-purpose async operations.

20.4 Capability-Based Hardware: CHERI

Software-based memory safety (bounds checking, garbage collection, safe languages) imposes runtime overhead. **CHERI** (Capability Hardware Enhanced RISC Instructions) moves memory safety into the hardware, enforcing it at the speed of a pointer dereference.

CHERI Capability. A CHERI capability is a hardware-enforced pointer that carries metadata:

1. **Address**: the current pointer value (the address being pointed to).
2. **Base and bounds**: the memory region the capability is authorised to access: $[\text{base}, \text{base} + \text{length})$.
3. **Permissions**: a bitmask of allowed operations (read, write, execute, load capability, store capability, seal, unseal).
4. **Object type**: for sealed capabilities (opaque references that cannot be dereferenced, only passed and unsealed by authorised code).
5. **Tag bit**: a 1-bit hardware flag that marks a register or memory word as containing a valid capability. The tag bit is maintained by the hardware and **cleared** if the capability is modified through non-capability instructions (e.g., integer arithmetic on the address). This makes capabilities **unforgeable**.

CHERI capabilities are 128 bits wide on 64-bit architectures (compared to 64-bit raw pointers), using a compressed bounds encoding (CHERI Concentrate) that represents most practical bounds exactly.

Example 20.4 (CHERI Preventing a Buffer Overflow). The vulnerable C function from Chapter 17:

```
void vulnerable(const char *input) {
    char buffer[64];
    strcpy(buffer, input);  /* overflow! */
}
```

On a CHERI processor:

1. `buffer` is a capability with address = stack pointer, base = stack pointer, length = 64, permissions = read/write.
2. `strcpy` increments the destination pointer as it copies bytes. When the pointer reaches `base + 64`, the next store instruction triggers a **capability bounds fault** — a hardware exception, analogous to a page fault.
3. The fault occurs **at the exact instruction that overflows**, before any data is corrupted. No canary check, no ASLR, no NX bit needed.

CHERI also prevents **use-after-free**: when `free()` deallocates a buffer, the runtime can revoke (clear the tag bit of) all capabilities pointing to that buffer, using a hardware-assisted revocation mechanism. Any subsequent dereference through a revoked capability traps.

The **ARM Morello** board (2022) is the first production-quality CHERI implementation, based on ARMv8.2-A with CHERI extensions. Research at the University of Cambridge has demonstrated that existing C/C++ code can be compiled for CHERI (using the CHERI Clang/LLVM toolchain) with modest source modifications, and the performance overhead is typically 5–15% for the "pure capability" ABI (all pointers are capabilities). A "hybrid" ABI allows mixing capability and raw pointers, reducing overhead at the cost of partial protection.

20.5 Persistent Memory

Persistent memory (PMEM) is byte-addressable storage that retains data across power cycles, sitting on the memory bus alongside DRAM. Intel Optane DC Persistent Memory was the first commercial product (2019–2022); CXL-attached persistent memory is the successor technology.

Persistent Memory. A storage class that combines byte-addressability and low la-

tency with persistence:

Property	DRAM	PMEM (Optane)	NVMe SSD
Interface	DDR bus, load/store	DDR bus, load/store	PCIe, block I/O
Read latency	~80 ns	~300 ns	~10 μs
Write latency	~80 ns	~100 ns	~20 μs
Persistence	No	Yes	Yes
Granularity	64 bytes (cache line)	64 bytes (cache line)	4 KB (sector/page)
Capacity	~TB	~TB (per socket)	~TB
Endurance	Unlimited	~10^{15} writes	~10^{17} writes (TLC)

20.5.1 DAX File Systems

DAX (Direct Access) file systems allow applications to `mmap` a file on PMEM and access it directly via load/store instructions, **bypassing the page cache entirely**. The page cache exists to buffer slow block I/O in DRAM; with PMEM, the storage is already on the memory bus, so caching in DRAM is counterproductive (it wastes DRAM and adds latency).

Linux supports DAX on ext4 and XFS (mount with `-o dax=always`).

```c
#include <fcntl.h>
#include <sys/mman.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>
#include <immintrin.h>  /* for _mm_clwb, _mm_sfence */

int main(void) {
    /* Open a file on a DAX-mounted filesystem */
    int fd = open("/mnt/pmem0/data.bin", O_RDWR | O_CREAT, 0644);
    ftruncate(fd, 4096);

    /* mmap with MAP_SHARED_VALIDATE | MAP_SYNC for DAX guarantees:
       - MAP_SHARED: changes are visible to other processes
```

```
       - MAP_SYNC: msync is not needed; stores + clwb + sfence suffice */
    char *pmem = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                      MAP_SHARED_VALIDATE | MAP_SYNC, fd, 0);
    if (pmem == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    /* Write directly to persistent memory */
    strcpy(pmem, "Hello, persistent memory!");

    /* CRITICAL: CPU caches sit between the core and PMEM.
       A store may be in L1/L2/L3 cache, not yet on PMEM.
       We must explicitly flush cache lines and fence. */
    _mm_clwb(pmem);    /* write-back cache line (non-invalidating) */
    _mm_sfence();       /* ensure all prior stores and flushes are ordered */

    /* After sfence, the data is guaranteed to be on PMEM.
       It will survive a power failure. */

    printf("Persisted: %s\n", pmem);

    munmap(pmem, 4096);
    close(fd);
    return 0;
}
```

20.5.2 The Crash Consistency Challenge

PMEM introduces a new programming challenge: **crash consistency without a journal or log**. With traditional storage, the file system's journal ensures that a crash leaves the file system in a consistent state. With PMEM and DAX, the application writes directly to persistent storage, and the CPU's cache hierarchy introduces ambiguity about what has actually been persisted.

The store ordering through the memory hierarchy is:

$$\text{CPU register} \xrightarrow{\text{store}} \text{Store buffer} \xrightarrow{\text{drain}} \text{L1 cache} \xrightarrow{\text{evict}} \text{L2/L3} \xrightarrow{\text{evict}} \text{PMEM}$$

A power failure at any point in this chain causes data loss for stores that have not

reached PMEM. The programmer must explicitly manage persistence:

- `CLWB` (Cache Line Write Back): writes back a cache line to PMEM without invalidating the cache (the line remains cached for subsequent reads). Available on Intel since Skylake.
- `CLFLUSH` / `CLFLUSHOPT`: write back and invalidate the cache line. `CLFLUSHOPT` is more efficient (can be pipelined).
- `SFENCE` (Store Fence): ensures all prior stores and cache flushes are ordered before subsequent stores.

Example 20.5 (PMEM Crash Consistency Bug). Consider updating a linked list node on PMEM:

```
/* WRONG: if power fails between the two stores, the node is
   inconsistent --- data is new but next points to the old target */
node->data = new_value;
node->next = new_next;

/* The stores might reach PMEM in any order, or only partially.
   After a crash, we might see:
   - data=new_value, next=old_next  (partially updated)
   - data=old_value, next=new_next  (store reordering)
   - data=old_value, next=old_next  (neither persisted)
   - data=new_value, next=new_next  (both persisted, correct)
*/

/* CORRECT: use CLWB + SFENCE to enforce ordering */
node->data = new_value;
_mm_clwb(&node->data);
_mm_sfence();         /* data is now on PMEM */

node->next = new_next;
_mm_clwb(&node->next);
_mm_sfence();         /* next is now on PMEM, after data */
```

Even with correct fencing, the application must be designed for crash recovery. Common approaches:

- **Write-ahead logging**: write the new values to a log on PMEM, fence, then apply them to the data structure, fence, then mark the log entry as committed. On recovery, replay uncommitted log entries.
- **Copy-on-write**: allocate a new node, populate it, fence, then atomically update the parent's pointer (a single 8-byte store that is naturally atomic on x86-64).

- **Transactional libraries**: PMDK (Persistent Memory Development Kit, now PMEMOBJ) provides a transactional API that handles fencing and logging automatically.

20.6 Microkernels Redux: seL4 and Fuchsia

The microkernel vs monolithic kernel debate, which appeared settled in favour of monolithic kernels in the 1990s (largely due to performance arguments), has been reopened by two developments: hardware that makes IPC fast (modern x86 and ARM cores achieve <1 μs IPC), and formal verification that makes microkernels provably correct.

20.6.1 seL4: The Verified Kernel

seL4. A third-generation microkernel (approximately 10,000 lines of C and 600 lines of assembly) with machine-checked proofs of correctness covering:

1. **Functional correctness**: the C implementation is proven (in Isabelle/HOL) to behave exactly as specified by the abstract mathematical model. Every execution of the C code corresponds to a valid execution of the specification.
2. **Integrity** (information flow): a compromised user-space process cannot affect the kernel or other processes that it does not have capabilities to access.
3. **Confidentiality** (information flow): information flows only through explicitly authorised channels.
4. **Binary correctness**: the compiled ARM binary is proven to behave as the C source (via translation validation against the GCC/LLVM output).
5. **Worst-case execution time (WCET)**: proven bounds on interrupt latency, enabling hard real-time guarantees.

Theorem 20.1 (seL4 Functional Correctness). Let S be the abstract specification of seL4 (a mathematical model in Isabelle/HOL of all kernel operations), and let I be the C implementation. The seL4 verification proves:

$$\forall \text{ inputs } x, \text{ states } \sigma : \text{exec}(I, x, \sigma) \text{ refines } \text{exec}(S, x, \sigma)$$

Every behaviour of the implementation is a valid behaviour of the specification. This implies: no buffer overflows, no null pointer dereferences, no use-after-free, no integer overflows, no uninitialised memory reads, no race conditions, and no deviations from the specification — in the kernel code.

Scope of the proof: the verification covers the kernel's C code and its compiled binary

(on ARM). It does **not** cover: the hardware (assumed correct), the bootloader, user-space code, or the compiler for non-verified targets. Approximately 200,000 lines of Isabelle/HOL proof were required for 10,000 lines of C — a 20:1 proof-to-code ratio.

seL4 is a **capability-based** microkernel: every kernel resource (memory frames, page tables, endpoints for IPC, interrupt handlers) is accessed through capabilities. A process can only perform operations on resources for which it holds capabilities, and capabilities can be selectively delegated. This naturally enforces the principle of least privilege.

20.6.2 Fuchsia and Zircon

Google's **Fuchsia** operating system uses **Zircon**, a microkernel written in C++. Zircon is also capability-based: every resource is accessed through a **handle** (the user-space representation of a capability), and handles carry specific rights (read, write, duplicate, transfer).

Unlike seL4, Zircon is not formally verified. Its strength is engineering quality and modern design: support for 64-bit only, ASLR by default, user-space device drivers, a component-based application framework (built on FIDL — Fuchsia Interface Definition Language for IPC), and a capability-based security model throughout the stack.

Fuchsia is deployed in production on the Google Nest Hub (a consumer smart display), demonstrating that microkernels can achieve acceptable performance for interactive consumer products.

20.7 Rust in the Kernel

The Linux kernel is written in C, a language with no memory safety guarantees at the language level. Buffer overflows, use-after-free, and data races account for approximately 65–70% of kernel security vulnerabilities (per analyses by Microsoft, Google, and the Chromium project). **Rust** offers memory safety at compile time, without the runtime overhead of garbage collection.

20.7.1 Rust's Safety Model

Rust's Ownership System. Rust enforces memory safety through three compile-time rules, checked by the **borrow checker**:

1. **Ownership**: every value has exactly one owner variable. When the owner goes out of scope, the value is dropped (its destructor runs and its memory is freed).

2. **Borrowing**: references to a value can be shared (`&T`, multiple immutable borrows allowed) or exclusive (`&mut T`, at most one mutable borrow allowed). Shared and exclusive borrows cannot coexist for the same value at the same time.
3. **Lifetimes**: every reference has a lifetime (a lexical scope during which it is valid). The compiler ensures that no reference outlives the value it points to.

These rules prevent, at compile time, four of the most common classes of memory bugs:

Bug Class	Rust Prevention Mechanism
Use-after-free	Ownership + lifetimes (reference cannot outlive value)
Double free	Unique ownership (only one owner can drop)
Data races	`Send`/`Sync` traits + borrow checker (no concurrent mutable access)
Null pointer dereference	No null pointers; `Option<T>` requires explicit handling
Buffer overflow	Bounds-checked indexing on `Vec<T>`, `[T]`, etc.

20.7.2 Linux Rust Modules

Since Linux 6.1 (December 2022), the kernel supports modules written in Rust. The Rust-for-Linux project provides safe abstractions over kernel C APIs, allowing driver authors to write in safe Rust.

Example 20.6 (Rust Kernel Module).

```
// SPDX-License-Identifier: GPL-2.0

//! A minimal Rust kernel module demonstrating safe abstractions.

use kernel::prelude::*;
use kernel::sync::Mutex;
use kernel::new_mutex;

module! {
```

```
    type: CounterModule,
    name: "counter_rust",
    author: "Example",
    description: "A counter demonstrating Rust safety in the kernel",
    license: "GPL",
}

struct CounterModule {
    counter: Mutex<u64>,
}

impl kernel::Module for CounterModule {
    fn init(_module: &'static ThisModule) -> Result<Self> {
        pr_info!("Rust counter module loaded\n");

        let module = CounterModule {
            counter: new_mutex!(0u64, "counter"),
        };

        // Demonstrate safe mutation: the Mutex enforces that
        // only one thread can access the counter at a time.
        {
            let mut guard = module.counter.lock();
            *guard += 1;
            pr_info!("Counter incremented to {}\n", *guard);
            // guard is dropped here, releasing the lock
        }

        Ok(module)
    }
}

impl Drop for CounterModule {
    fn drop(&mut self) {
        let guard = self.counter.lock();
        pr_info!("Module unloaded, final counter: {}\n", *guard);
    }
}
```

Key differences from a C kernel module:

- Every memory allocation returns a `Result`, forcing the programmer to handle allocation failure. In C, `kmalloc` returns NULL, and forgetting to check is a bug. In Rust, ignoring a `Result` is a compile error.
- The `Mutex` wrapper ensures that the protected data can only be accessed through the lock guard. In C, nothing prevents accessing the data without holding the lock.
- The `Drop` trait ensures cleanup on module unload — no forgotten `kfree` calls.
- Buffer overflows in safe Rust are impossible. Array/slice indexing is bounds-checked.

20.7.3 The Safe/Unsafe Boundary

Kernel code necessarily interacts with hardware registers, DMA buffers, and lock-free data structures that cannot be expressed in safe Rust. The Rust-for-Linux approach encapsulates `unsafe` operations in carefully audited safe abstractions:

```
┌──────────────────────────────────────────┐
│  Driver code (100% safe Rust)            │
│  - No unsafe blocks                      │
│  - Uses safe abstractions: Mutex<T>,     │
│    Pin<T>, DmaBuffer<T>, IoMem<T>        │
├──────────────────────────────────────────┤
│  Safe abstractions (Rust wrappers)       │
│  - Contains unsafe blocks               │
│  - Provides safe API enforced by types   │
│  - Invariants documented and maintained  │
│  - Small, auditable surface             │
├──────────────────────────────────────────┤
│  Kernel C API (FFI bindings via bindgen) │
│  - All functions are extern "C", unsafe  │
│  - Generated from kernel C headers       │
└──────────────────────────────────────────┘
```

The goal is to maximise the ratio of safe to unsafe code. As more abstractions are built, driver authors write entirely in safe Rust. The Android team has reported that introducing Rust for new Binder driver code has resulted in zero memory safety vulnerabilities in the Rust components, compared to a steady stream in the C components.

Programmer's Perspective: Rust Kernel Modules and the Safety Guarantee. The value proposition of Rust in the kernel is not about performance (C and Rust produce comparable machine code). It is about **eliminating classes of bugs at**

compile time.

Consider a kernel driver that manages a DMA buffer. In C, the programmer must manually ensure:

1. The buffer is allocated with the correct alignment for the device.
2. The buffer is freed exactly once (no double free).
3. No reference to the buffer is used after free (no use-after-free).
4. The buffer is not accessed concurrently without proper locking (no data race).
5. The physical address (not virtual) is passed to the device for DMA.
6. The buffer is not larger than the allocated size (no overflow).

Any mistake in any of these is a potential kernel panic or security vulnerability. In Rust:

```
// Hypothetical safe DMA buffer abstraction
struct DmaBuffer<T> {
    vaddr: *mut T,        // virtual address (unsafe internally)
    paddr: PhysAddr,      // physical address
    size: usize,          // allocated size
    _marker: PhantomData<T>,
}

impl<T> DmaBuffer<T> {
    // Allocation: returns Result, not raw pointer
    fn alloc(dev: &Device, count: usize) -> Result<Self> { ... }

    // Access: returns a slice with bounds checking
    fn as_slice(&self) -> &[T] { ... }
    fn as_mut_slice(&mut self) -> &mut [T] { ... }

    // Physical address: method, not cast
    fn phys_addr(&self) -> PhysAddr { self.paddr }
}

// Drop: automatically freed when DmaBuffer goes out of scope
impl<T> Drop for DmaBuffer<T> {
    fn drop(&mut self) { /* dma_free_coherent(...) */ }
}

// Send: can be transferred to another thread
// Sync: cannot be shared between threads without external sync
```

```
unsafe impl<T: Send> Send for DmaBuffer<T> {}
```

The Rust compiler enforces all six invariants through the type system. Incorrect code is rejected at compile time, not discovered through fuzzing or in production.

20.8 WebAssembly as a Portable OS Interface

WebAssembly (Wasm) was originally designed for browsers, but its properties — sandboxed execution, near-native performance, language independence, and deterministic semantics — make it compelling as a universal runtime for server-side and edge computing.

WebAssembly (Wasm). A portable, size-efficient binary instruction format with the following properties:

1. **Memory safety**: Wasm modules execute in a linear memory sandbox (a flat byte array). They cannot access host memory, the stack of the host process, or other Wasm modules' memory without explicit imports.
2. **Control flow integrity**: indirect calls go through a type-checked function table. There are no arbitrary jumps.
3. **Near-native performance**: Wasm is designed for efficient AOT and JIT compilation. Performance is typically within 10–30% of native code.
4. **Language independence**: C, C++, Rust, Go, AssemblyScript, Zig, and many other languages compile to Wasm.
5. **Deterministic semantics**: Wasm's specification leaves no undefined behaviour (unlike C). The same module produces the same results on any compliant runtime.

20.8.1 WASI: The WebAssembly System Interface

WASI (WebAssembly System Interface) defines a set of APIs that Wasm modules can use to interact with the outside world: file I/O, networking, clocks, random numbers, and environment variables. WASI is **capability-based**: a Wasm module can only access files and network connections that the host explicitly grants through capability handles.

```
┌──────────────────────────────────────┐
│  Wasm Module                         │
│  (compiled from C, Rust, Go, etc.)   │
│  │                                   │
│  ├─ fd_read(fd, iovs, nread)         │
│  ├─ fd_write(fd, iovs, nwritten)     │
```

```
│  ├─ path_open(dir_fd, path, ...)   │  WASI API calls
│  ├─ sock_accept(fd, ...)           │
│  └─ clock_time_get(clock_id, ...) │
└──────────────────────────────────┘
             │
             │ only granted capabilities
             ▼
┌──────────────────────────────────┐
│  Wasm Runtime (Wasmtime, Wasmer)   │
│  │                                │
│  ├─ Validates and compiles module  │
│  ├─ Enforces sandbox boundaries    │
│  ├─ Maps WASI calls to host OS     │
│  └─ Provides only granted dirs,    │
│     files, network access          │
└──────────────────────────────────┘
             │
             │ real system calls
             ▼
┌──────────────────────────────────┐
│  Host Operating System             │
└──────────────────────────────────┘
```

20.8.2 Wasm as a Container Alternative

Solomon Hykes, co-founder of Docker, stated in 2019: "If WASM+WASI existed in 2008, we wouldn't have needed to create Docker." The reasoning:

- **Portability**: a Wasm binary runs on any platform with a compliant runtime — no container image layers, no architecture-specific builds, no base images.
- **Sandboxing**: WASI's capability model provides fine-grained isolation without kernel namespaces or cgroups.
- **Startup time**: Wasm modules start in microseconds (pre-compiled AOT), not hundreds of milliseconds.
- **Size**: a Wasm module is typically kilobytes to low megabytes, versus hundreds of megabytes for a container image.

However, Wasm currently lacks the full POSIX interface that most server applications expect. WASI preview 2 (the Component Model) adds composability: Wasm modules can be linked together like shared libraries, with typed interfaces. But full threading, shared memory, and unrestricted networking are still evolving.

Wasmtime (Bytecode Alliance: Mozilla, Fastly, Intel, Red Hat) and **Wasmer** are the leading Wasm runtimes. **Spin** (Fermyon) and **wasmCloud** provide developer-friendly

frameworks for building Wasm-based microservices. Cloudflare Workers uses V8's Wasm engine to run user code at edge locations with sub-millisecond cold start.

20.9 The Convergence of OS and Language Runtime

A recurring theme in this chapter is the erosion of the boundary between the operating system and the language runtime.

20.9.1 Go and the GMP Scheduler

Go's runtime is a user-space operating system: it schedules goroutines (the GMP model), manages memory (mspan/mcentral/mheap), provides synchronisation (channels, mutexes), performs garbage collection, and handles I/O multiplexing (netpoller wrapping epoll/kqueue). The Linux kernel sees a Go program as an ordinary multi-threaded process; the real scheduling and memory management happens in user space, invisible to the kernel.

20.9.2 The Erlang BEAM

The Erlang BEAM virtual machine goes further: it manages millions of lightweight processes with per-process garbage collection (each process has a tiny heap, and GC is incremental per-process), preemptive scheduling (based on reduction counts: each process gets ~4000 function calls before being preempted), and built-in distribution (processes on different machines communicate transparently using the same syntax as local message passing).

20.9.3 Green Threads Everywhere

The trend towards user-space scheduling is accelerating across all major languages:

Runtime	Unit	Stack	Scheduling	Preemption
Go	goroutine	2–8 KB (growable)	GMP M:N	Async signal (Go 1.14+)
Erlang/BEAM	process	~300 words	Preemptive	Reduction count
Java (Loom)	virtual thread	~1 KB	M:N	Cooperative (yield at I/O)

Runtime	Unit	Stack	Scheduling	Preemption
Rust (tokio)	task	stackless (state machine)	M:N	Cooperative (async/await)
C# (.NET)	Task	stackless (state machine)	Thread pool	Cooperative (async/await)
Kotlin	coroutine	stackless	Dispatchers	Cooperative (suspend)

Example 20.7 (User-Space vs Kernel Scheduling Costs).

Operation	Kernel Thread (pthread)	Goroutine (Go)	Ratio
Creation	~100 μs, 1–8 MB stack	~1 μs, 2 KB stack	100x
Context switch	~1–10 μs (kernel mode)	~100–200 ns (user mode)	10–100x
Max concurrent	~10,000 (stack space limited)	~1,000,000+	100x
Memory per unit	1–8 MB (fixed stack)	2–8 KB (growable)	125–4000x

The orders-of-magnitude differences explain why high-concurrency servers (web servers handling 100K+ connections, databases with thousands of concurrent queries, network proxies) benefit enormously from user-space scheduling. The kernel's thread abstraction was designed for tens to hundreds of concurrent activities, not millions.

Programmer's Perspective: The Emerging Systems Stack. The technologies in this chapter are converging into a new systems architecture:

```
Traditional Stack (2000--2020)      Emerging Stack (2020+)

┌──────────────────┐               ┌──────────────────────────┐
│  Application     │               │  Application             │
│  (C/C++/Java)    │               │  (Rust/Go/Wasm)          │
```

```
├──────────────────┤              ├──────────────────────────┤
│  libc + syscalls │              │  io_uring (async I/O)    │
│  (read/write/    │              │  eBPF hooks (tracing,    │
│   mmap/ioctl)    │              │   networking, security)  │
├──────────────────┤              ├──────────────────────────┤
│  Linux kernel    │              │  Linux kernel            │
│  (C, ~25M LOC)   │              │  + Rust modules (safe)   │
│                  │              │  + eBPF extensions       │
│                  │              │  + sched_ext (custom     │
│                  │              │    schedulers)            |
├──────────────────┤              ├──────────────────────────┤
│  x86/ARM         │              │  CHERI (capability HW)   │
│  (raw pointers)  │              │  CXL (PMEM, disaggregated|
│                  │              │   memory)                │
└──────────────────┘              └──────────────────────────┘
```

The key shift is from **runtime checking** to **compile-time and hardware-time checking**:

- Memory safety: from ASLR + canaries (runtime) to Rust (compile-time) + CHERI (hardware).
- Kernel extension: from kernel modules (unrestricted) to eBPF (verified before loading).
- I/O: from synchronous syscalls (kernel boundary crossing) to io_uring (shared-memory rings).
- Isolation: from processes and VMs to Wasm modules (language-level sandboxing).

For a systems programmer today, the most impactful technologies to master are:

1. **Rust**: kernel modules, drivers, systems libraries, and embedded.
2. **eBPF**: observability, networking, and security at the kernel level.
3. **io_uring**: high-performance I/O for databases, proxies, and storage engines.
4. **Go**: distributed systems, container runtimes, and cloud infrastructure.
5. **Wasm/WASI**: portable, sandboxed edge computing and plugin systems.

20.10 Summary: The OS in the Next Decade

The operating system is not dying — it is **decomposing** into specialised layers, each verifiable, composable, and hardware-aware:

Layer	Traditional (2020)	Emerging (2025+)
Kernel	Monolithic C (Linux 25M+ LOC)	Verified microkernel (seL4) or monolithic + Rust modules + eBPF
I/O	Synchronous syscalls	io_uring (shared rings), kernel bypass (DPDK, SPDK)
Memory safety	ASLR + canaries + NX (reactive)	CHERI (proactive, hardware), Rust (proactive, language)
Isolation	Processes + VMs + containers	+ Wasm sandboxes + unikernels + Firecracker microVMs
Scheduling	Kernel scheduler (fixed policy)	+ eBPF sched_ext + user-space (goroutines, virtual threads)
Storage	Block I/O + page cache	PMEM + DAX (byte-addressable persistence)
Extension	Kernel modules (unsafe, unrestricted)	eBPF (verified, sandboxed, JIT-compiled)
Verification	Testing + fuzzing	Formal proofs (seL4) + static analysis (eBPF verifier)

The common thread is **pushing guarantees closer to the foundation**: memory safety into silicon (CHERI), correctness into proofs (seL4), safe extensibility into static analysis (eBPF verifier), and performance-critical paths into shared memory (io_uring). The operating system of the next decade will be less a monolithic artefact and more a **verified, composable, hardware-software co-design**.

---

1. **Unikernel Trade-offs.** A company deploys 50 microservices, each running in its own container on a shared Kubernetes cluster. The security team proposes migrating each microservice to a unikernel running in its own VM. (a) What

security benefits does the unikernel approach provide over containers? Quantify the TCB reduction. (b) What operational challenges does it introduce (deployment pipelines, debugging, monitoring, log collection, live updates)? (c) Under what threat model and scale conditions would the migration be justified? (d) Could Firecracker microVMs provide a middle ground? Explain.

2. **eBPF Verifier Limits.** The eBPF verifier imposes a maximum of 1 million verified instructions per program. (a) Explain why this limit exists (consider the verifier's time complexity). (b) Describe a legitimate eBPF program that might exceed this limit (e.g., a complex packet parser with many protocol branches). (c) Explain how **tail calls** (one eBPF program calling another, up to 33 levels) can restructure such a program to work within the per-program limit while performing the same total computation.

3. **io_uring vs Traditional I/O.** A network proxy forwards packets between two TCP connections. For each packet, it reads from one socket and writes to the other. The proxy handles 100,000 concurrent connections, each producing 10 I/O operations per second. (a) Calculate the total system call overhead per second using traditional `read()`/`write()` (assume 500 ns per syscall). (b) Calculate the overhead using io_uring with SQ polling (assume 0 ns per operation, plus 1 μs per batch of 100 operations for ring synchronisation). (c) Calculate the percentage improvement. (d) Recalculate parts (a) and (b) with KPTI enabled, assuming syscall cost increases to 1500 ns. What happens to the relative improvement?

4. **CHERI Pointer Width.** A CHERI capability is 128 bits (compared to 64-bit pointers). (a) For a program with 10 million heap-allocated objects averaging 100 bytes each, with an average of 3 pointers per object, calculate the total memory used by pointers on a conventional 64-bit system and on CHERI. (b) What is the percentage overhead? (c) Explain why the compressed bounds encoding (CHERI Concentrate) does not simply use 128 raw bits for base and length. What does it actually encode, and what alignment restrictions result?

5. **seL4 Verification Scope.** The seL4 verification proves functional correctness of the C implementation against a formal specification. (a) List three specific categories of bugs that the verification does NOT cover. (b) For each, explain why it falls outside the proof's scope. (c) For each, propose a complementary technique or technology (from this chapter or Chapter 17) that could address it.

6. **Rust Soundness in the Kernel.** A Rust kernel module needs to read a 32-bit status register from a memory-mapped device at fixed physical address `0xFEED_0000`. (a) Write the Rust code (using `unsafe` and `core::ptr::read_volatile`) that reads the register. (b) Explain why this

operation must be `unsafe` (which of Rust's safety invariants cannot be statically verified). (c) Design a safe wrapper type `StatusRegister` that encapsulates this `unsafe` operation and prevents misuse. What invariants should the constructor verify? What should the public API look like?

7. **Convergence Analysis.** Compare the isolation properties of five approaches: (a) a Linux process, (b) a Linux container (namespaces + cgroups + seccomp), (c) a KVM virtual machine with virtio, (d) a Wasm module in Wasmtime with WASI, (e) a unikernel on Xen. For each, determine: the trusted computing base (TCB) size in approximate LOC, the classes of bugs in the TCB that could break isolation, the performance overhead relative to bare metal, and the recovery mode if isolation is breached. Which combination of these technologies provides the strongest security for a multi-tenant serverless platform? Justify your answer.